ПОЛНАЯ ВЕРСИЯ УЧЕБНИКА

Учебник Go

Последовательный русскоязычный курс по Go: от первой программы до backend-проекта.

Автор: Данил Чугаев

Версия: v0.1.0

Актуально на: 20.10.2026

Содержание

Ссылки ведут к соответствующим материалам внутри PDF.

Учебник

1. Предисловие 11

Глава 1: Введение в Go

1. Что такое Go и зачем он нужен? 15
2. Установка и настройка 16
3. Первая программа и Go-модуль 18
4. Самопроверка 22
5. Задания: первый модуль 24
6. Рекомендации по коду: старт с Go 30

Глава 2: Переменные и базовые типы

1. Переменные и константы 31
2. Базовые типы и преобразования 35
3. Самопроверка 38
4. Задания: значения для задачи 40
5. Рекомендации по коду: переменные и базовые типы 46

Глава 3: Строки и обработка текста

1. Строки и обработка текста 48
2. Самопроверка 51
3. Задания: нормализация и поиск названия 53
4. Рекомендации по коду: строки и текст 59

Глава 4: Операторы и выражения

1. Операторы и выражения 61
2. Самопроверка 66
3. Задания: проверки задачи 69
4. Рекомендации по коду: операторы и выражения 77

Глава 5: Условия — if и switch

1. Условия: if и switch 79
2. Самопроверка 83

3. Задания: валидация задачи 86
4. Рекомендации по коду: условия 94

Глава 6: Циклы — for и range

1. Циклы: for и range 96
2. Самопроверка 101
3. Задания: нумерация и название задачи 103
4. Рекомендации по коду: циклы 111

Глава 7: Функции

1. Функции 113
2. Самопроверка 119
3. Задания: функции Task Manager API 121
4. Рекомендации по коду: функции 130

Глава 8: Массивы и слайсы

1. Массивы и слайсы 131
2. Самопроверка 137
3. Задания: список названий задач 140
4. Рекомендации по коду: массивы и слайсы 148

Глава 9: Map

1. Map: ключ и значение 150
2. Самопроверка 155
3. Задания: задачи по ID 158
4. Рекомендации по коду: map 166

Глава 10: Структуры

1. Структуры: модель задачи 168
2. Самопроверка 173
3. Задания: модель Task 176
4. Рекомендации по коду: структуры 185

Глава 11: Методы

1. Методы 186
2. Самопроверка 191
3. Задания: поведение Task 193
4. Рекомендации по коду: методы 200

Глава 12: Указатели

1. Указатели 201
2. Самопроверка 207
3. Задания: изменение Task 210
4. Рекомендации по коду: указатели 217

Глава 13: Интерфейсы

1. Интерфейсы 218
2. Самопроверка 223
3. Задания: TaskFormatter 226
4. Рекомендации по коду: интерфейсы 233

Глава 14: Обработка ошибок

1. Обработка ошибок 234
2. Самопроверка 240
3. Задания: ошибки при переименовании задачи 242
4. Рекомендации по коду: ошибки 252

Глава 15: defer, panic и recover

1. defer, panic и recover 253
2. Самопроверка 259
3. Задания: безопасная работа с файлами 261
4. Рекомендации по коду: defer, panic и recover 269

Глава 16: Пакеты и модули

1. Пакеты и модули 270
2. Самопроверка 277
3. Задания: пакет internal/task 280
4. Рекомендации по коду: пакеты и модули 289

Глава 17: Тестирование

1. Тестирование 290
2. Самопроверка 297
3. Задания: тесты Task и Store 299
4. Рекомендации по коду: тестирование 308

Глава 18: Файлы и ввод-вывод

1. Файлы и ввод-вывод 309
2. Самопроверка 317
3. Задания: текстовый экспорт задач 320
4. Рекомендации по коду: файлы и ввод-вывод 331

Глава 19: JSON и сериализация

1. JSON и сериализация 333
2. Самопроверка 341
3. Задания: сохранение задач в JSON 344
4. Рекомендации по коду: JSON 355

Глава 20: Время и дедлайны

1. Время и дедлайны 356
2. Самопроверка 361
3. Задания: дедлайн задачи 364
4. Рекомендации по коду: время и дедлайны 375

Глава 21: Goroutine

1. Goroutine 376
2. Самопроверка 383
3. Задания: параллельный экспорт 386
4. Рекомендации по коду: goroutine 396

Глава 22: Каналы

1. Каналы 398
2. Самопроверка 405
3. Задания: результаты экспортов через канал 408
4. Рекомендации по коду: каналы 417

Глава 23: Select и таймауты

1. Select и таймауты 418
2. Самопроверка 424
3. Задания: таймаут экспортов 427
4. Рекомендации по коду: select и таймауты 437

Глава 24: Mutex и синхронизация

1. Mutex и синхронизация 439
2. Самопроверка 449

3. Задания: безопасный TaskStore 452
4. Рекомендации по коду: mutex и синхронизация 464

Глава 25: Context

1. Context: отмена и дедлайны 465
2. Самопроверка 474
3. Задания: отменяемый экспорт 477
4. Рекомендации по коду: context 489

Глава 26: Логирование и отладка

1. Логирование и отладка 490
2. Самопроверка 497
3. Задания: наблюдаемый TaskStore 500
4. Рекомендации по коду: логирование и отладка 509

Глава 27: HTTP-сервер и JSON API

1. HTTP-сервер и JSON API 511
2. Самопроверка 521
3. Задания: первый Task Manager API 524
4. Рекомендации по коду: HTTP API на net/http 535

Глава 28: HTTP API на Gin

1. HTTP API на Gin 537
2. Самопроверка 547
3. Задания: миграция Task Manager API на Gin 549
4. Рекомендации по коду: Gin 561

Глава 29: HTTP-клиент

1. HTTP-клиент: внешние API 562
2. Самопроверка 570
3. Задания: клиент подсказок приоритета 573
4. Рекомендации по коду: HTTP-клиент 584

Глава 30: HTTP-клиент на Resty

1. HTTP-клиент на Resty 586
2. Самопроверка 594
3. Задания: перенос клиента подсказок на Resty 596
4. Рекомендации по коду: Resty 607

Глава 31: Конфигурация и жизненный цикл

1. Конфигурация и жизненный цикл приложения 609
2. Самопроверка 617
3. Задания: управляемый запуск Task Manager API 620
4. Рекомендации по коду: конфигурация и жизненный цикл 631

Глава 32: Docker и локальное окружение

1. Docker и локальное окружение 633
2. Самопроверка 641
3. Задания: Docker-окружение Task Manager API 643
4. Рекомендации по коду: Docker и локальное окружение 650

Глава 33: SQL и PostgreSQL

1. SQL и PostgreSQL 652
2. Самопроверка 663
3. Задания: PostgreSQL-репозиторий Task Manager 666
4. Рекомендации по коду: SQL и PostgreSQL 675

Глава 34: CI/CD и деплой

1. CI/CD и деплой 677
2. Самопроверка 685
3. Задания: доставка Task Manager API 688
4. Рекомендации по коду: CI/CD и деплой 697

Глава 35: Итоговая интеграция

1. Итоговая интеграция 699
2. Самопроверка 707
3. Задания: выпуск Task Manager API 710
4. Рекомендации по коду: итоговый проект 718

Продвинутая глава A: Ошибки и ресурсы

1. Продвинутый Go A: ошибки и управление ресурсами 720
2. defer и жизненный цикл ресурсов 726
3. panic и recover в библиотечном коде 732
4. Ошибки HTTP API: контракт, логи и panic 737
5. Самопроверка: ошибки и управление ресурсами 742
6. Задания: ошибки и управление ресурсами 745

7. Рекомендации по коду: ошибки и ресурсы 757

Продвинутая глава B: Типы и generics

1. Продвинутый Go B: типы и generics 759
2. Параметры типов и generic-функции 764
3. Constraints, type sets и оператор ~ 770
4. Generic-типы и выбор абстракции 775
5. Самопроверка: типы и generics 780
6. Задания: типы и generics 783
7. Рекомендации по коду: типы и generics 795

Продвинутая глава C: Конкурентные паттерны

1. Продвинутый Go C: конкурентные паттерны 797
2. Fan-out, fan-in и pipeline 801
3. errgroup, отмена и graceful shutdown 806
4. Утечки goroutine и диагностика 812
5. Самопроверка: конкурентные паттерны 817
6. Задания: конкурентные паттерны 820
7. Рекомендации по коду: конкурентные паттерны 832

Продвинутая глава D: Производительность

1. Продвинутый Go D: производительность и диагностика 835
2. Benchmarks и benchstat 838
3. CPU, память и pprof 843
4. Аллокации, escape analysis и точечные оптимизации 847
5. Самопроверка: производительность и диагностика 851
6. Задания: производительность и диагностика 854
7. Рекомендации по коду: производительность и диагностика 865

Продвинутая глава E: Reflection, unsafe и cgo

1. Продвинутый Go E: reflection, unsafe и cgo 868
2. Reflection: reflect.Type и reflect.Value 870
3. unsafe: память и границы гарантий 876
4. cgo: вызов C-кода из Go 879
5. Самопроверка: reflection, unsafe и cgo 883
6. Задания: reflection, unsafe и cgo 886
7. Рекомендации по коду: reflection, unsafe и cgo 899

Продвинутая глава F: Production-архитектура

1. Продвинутый Go F: архитектура production-сервиса 901
2. Конфигурация, секреты и health-checks 906
3. Observability: логи, метрики и tracing 910
4. Timeouts, retries, idempotency и rate limiting 914
5. Безопасность API и code review 919
6. Самопроверка: архитектура production-сервиса 923
7. Задания: архитектура production-сервиса 926
8. Рекомендации по коду: архитектура production-сервиса 936

Справочник

1. Базовые понятия 939
2. Продвинутые понятия 951

Дополнительные ресурсы

1. Дополнительные ресурсы 959

Контрибьюторы

Спасибо всем, кто создавал и улучшал материалы учебника.

- Danil Chugaev

Предисловие

Я начал этот учебник, чтобы упорядочить собственное изучение Go и собрать материал в том виде, в котором мне самому было бы удобно проходить язык: от коротких объяснений и запуска первых программ до создания настоящего backend-приложения.

Учебник построен не как энциклопедия языка, а как последовательность небольших шагов: сначала вы осваиваете синтаксис и модели данных, затем тесты и конкурентный код, после — HTTP API, PostgreSQL, Docker и доставку приложения.

Это не перевод официальной документации и не попытка заменить её. Я хочу сделать понятный русскоязычный маршрут для человека, который начинает с нуля и регулярно пишет код; первоисточники остаются обязательными для точных деталей и реального production-кода.

Сквозной проект — **Task Manager API**. Почти каждая тема получает прикладной контекст: задача, её валидация, хранение, экспорт, HTTP-обработчик, база данных или инфраструктура. В финале получается не набор несвязанных упражнений, а API, которое можно запустить локально, проверить, упаковать в контейнер и доставить в окружение.

Для кого этот учебник

Учебник рассчитан прежде всего на новичка в Go и backend-разработке. Предыдущий опыт программирования не обязателен, но будут полезны базовые навыки работы с файлами, терминалом и редактором кода.

Он также подойдёт разработчикам из других языков, которые хотят пройти Go последовательно, а не только сравнить его синтаксис со знакомым стеком. Если вы уже пишете на Go, используйте содержание как карту тем, а задания и рекомендации по коду — как точки для самопроверки.

Для глав об инфраструктуре понадобятся Docker Desktop или Docker Engine с Docker Compose. Для финального production-сценария полезен GitHub-аккаунт и, при желании пройти деплой целиком, VPS. Эти инструменты появляются постепенно: в начале достаточно Go и терминала.

К какому результату мы идём

Цель — не выучить набор конструкций наизусть, а научиться применять Go для backend-разработки. К концу основного маршрута вы сможете:

- создавать Go-модули, собирать и форматировать программы;
- работать с типами, строками, функциями, слайсами, map, структурами, методами, интерфейсами и ошибками;
- писать unit-тесты, работать с файлами и JSON, запускать `go vet` и race detector;
- понимать goroutine, каналы, `select`, mutex и `context`;
- проектировать слои handler → service → repository и не смешивать HTTP, бизнес-правила и инфраструктуру;
- создавать JSON API сначала на `net/http`, затем на Gin;

- вызывать внешние API через стандартный `http.Client` и Resty;
- настраивать env и `.env`, structured logging, request ID, graceful shutdown и health-check;
- хранить данные в PostgreSQL через `database/sql` и pgx, управлять схемой через Goose и проверять её в DBeaver;
- запускать API и PostgreSQL в Docker Compose, собирать multi-stage image;
- настраивать GitHub Actions, integration-тесты, публикацию image в GHCR, базовый deploy на VPS и rollback по SHA-версии.

Основной маршрут состоит из 35 глав. После него начинается отдельный **продвинутый блок**: углублённые ошибки и ресурсы, generics, конкурентные паттерны, performance и profiling. Он не обязателен для первого прохода: сначала завершите основной Task Manager API.

Карта пути

Основной курс разбит на несколько частей:

1. **Основа языка** — первая программа, типы, строки, условия, циклы и функции.
2. **Данные и модели** — слайсы, map, структуры, методы, указатели, интерфейсы, ошибки и пакеты.
3. **Надёжность и ввод-вывод** — тесты, файлы, JSON, время и дедлайны.
4. **Конкурентность** — goroutine, каналы, `select`, mutex и `context`.
5. **Backend и production-подготовка** — логирование, `net/http`, Gin, HTTP-клиенты, configuration, Docker, PostgreSQL, CI/CD и итоговая интеграция.

Подробный порядок, ссылки и статус тем находятся в содержании. Не перескакивайте сразу к Gin, Docker или PostgreSQL, если ещё не уверены в функциях, ошибках, `context` и тестах: поздние главы опираются на эти навыки.

Как устроен урок

Каждая глава основного маршрута содержит:

- **цель урока** — измеримый результат после прохождения;
- **теорию** — только необходимое объяснение с небольшими исполняемыми примерами;
- **частые ошибки** — типичные неверные решения и их причины;
- **самопроверку** — короткий квиз по ключевым понятиям;
- **задания** — задачу, критерии готовности, подсказки и скрытые решения;
- **рекомендации по коду** — краткий checklist решений, которые стоит сохранить в проекте.

Кодовые фрагменты иллюстрируют конкретную главу, а полный проект вы собираете постепенно. Иногда пример намеренно проще финальной архитектуры: например, сначала API работает в памяти, а PostgreSQL появляется только после Docker и SQL-глав. Не переносите учебный фрагмент в production без понимания его ограничений.

Как проходить учебник с пользой

1. Не ограничивайтесь чтением: набирайте примеры сами, запускайте `go run .` и меняйте входные данные.
2. Перед открытием решения попробуйте выполнить задание и свериться с критериями готовности.
3. После каждой главы запускайте рекомендованные проверки: минимум `gofmt`, `go test` и, где уместно, `go test -race`.
4. Используйте кнопку копирования у примера и Go Playground для небольших экспериментов. Для Task Manager API работайте локально в редакторе и Git-репозитории.
5. Не пропускайте тесты, ошибки и `context`: именно они связывают синтаксис Go с реальной backend-разработкой.
6. В инфраструктурных главах сначала проверяйте команды локально, затем переносите их в Docker и CI. Не добавляйте secrets, `.env` и production DSN в Git.
7. Если пример, термин или ошибка непонятны, вернитесь к предыдущей теме или к справочнику, а не пытайтесь запомнить объяснение без практики.

Об официальной документации

Этот учебник объясняет путь и даёт практику, но окончательным источником поведения языка остаются официальные материалы Go:

- документация и руководства;
- спецификация языка;
- стандартная библиотека;
- Effective Go.

Для сторонних инструментов сверяйтесь и с их официальной документацией: Gin, Resty, PostgreSQL, pgx, Goose, Docker, GitHub Actions и DBeaver развиваются независимо от Go. Особенно важно проверять документацию при обновлении версии, настройке безопасности, производительности и production-инфраструктуры.

Использование ИИ и проверка материалов

При подготовке учебника я использую ИИ как рабочий инструмент: он помогает составлять черновики, предлагать структуру уроков, формулировать объяснения, находить пробелы и готовить варианты упражнений. Это ускоряет работу, но не делает результат автоматически достоверным.

Поэтому для материала действует следующая политика:

- ИИ не является источником истины и не заменяет официальную документацию Go.
- Я редактирую сгенерированные фрагменты, проверяю утверждения и привожу терминологию к единому стилю.
- Новые или переработанные исполняемые примеры по возможности запускаются локально; документация и PWA-сборка также проверяются перед публикацией.
- Для спорных, зависящих от версии или низкоуровневых тем приоритет имеют официальная документация, спецификация и воспроизводимый код.

- В уже опубликованных главах могут оставаться ошибки, упрощения или устаревшие формулировки. Если вы заметили такую проблему, пожалуйста, сообщите о ней.

Примеры в учебнике предназначены для обучения. Перед использованием кода в рабочем сервисе его нужно самостоятельно проверить с учётом версии Go, нагрузки, безопасности, обработки ошибок, secrets, TLS, backup-стратегии и требований конкретной инфраструктуры.

Обратная связь

Если нашли неточность, нерабочий пример, опечатку или видите способ сделать объяснение понятнее, создайте issue в репозитории проекта. Хорошее сообщение содержит:

- ссылку на страницу или её заголовок;
- версию Go и операционную систему, если проблема связана с запуском кода;
- фактический результат или текст ошибки;
- ожидаемый результат, если он очевиден.

Спасибо, что проходите этот путь вместе со мной. Начните с содержания, затем перейдите к первой главе и первому запуску Go-программы.

Что такое Go и зачем он нужен?

Что вы узнаете

В этом уроке — что представляет собой Go, где его обычно применяют и как будет устроен курс.

Что такое Go

Go — компилируемый статически типизированный язык программирования с открытым исходным кодом. Его начали разрабатывать в Google Роб Пайк, Кен Томпсон и Роберт Гриземер; первый публичный релиз вышел в 2009 году.

Программа на Go обычно компилируется в исполняемый файл, а компилятор проверяет множество ошибок ещё до запуска: например, несовместимость типов или неиспользуемый импорт.

Почему Go подходит для backend-разработки

- **Небольшой и последовательный язык.** У Go сравнительно мало синтаксических конструкций, поэтому основной набор инструментов можно освоить достаточно быстро.
- **Стандартная библиотека.** В ней есть инструменты для HTTP, JSON, работы с файлами, тестов, контекста и конкурентного кода.
- **Быстрая сборка и простой запуск.** Для многих программ достаточно собрать один исполняемый файл.
- **Конкурентность.** Goroutine, каналы и пакет `context` помогают управлять параллельными задачами. Эти темы появятся позже, когда будет освоена база.
- **Экосистема для серверов и инструментов.** Go часто используют для HTTP API, CLI-утилит, инфраструктурных сервисов и облачных приложений.

Что важно не ожидать от Go

- В Go нет классов и классического наследования. Вместо них используют структуры, методы, интерфейсы и композицию.
- Ошибки чаще возвращают как обычное значение `error`, а не выбрасывают исключения. Это потребует привычки.
- Go не избавляет от ошибок времени выполнения: обращение к `nil`-указателю, выход за границы слайса или гонка данных всё ещё могут привести к проблеме. Язык и инструменты помогают замечать их раньше.
- Собранный файл нельзя автоматически запускать на любой машине: должны совпадать операционная система и архитектура, а программы с `cgo` могут иметь системные зависимости.

Что мы будем делать в этом учебнике

Цель курса — не только разобрать синтаксис, но и создать **Task Manager API**. По мере прохождения уроков мы будем развивать проект:

1. начнём с моделей задач и простой логики;
2. сохраним данные в JSON-файл;
3. создадим HTTP API;
4. добавим тесты, PostgreSQL, логирование и запуск в Docker.

Каждая глава будет состоять из теории, коротких примеров, самопроверки и практики. Не пытайтесь запомнить всё сразу: запускайте примеры, меняйте их и наблюдайте результат.

Дальше

Установим Go, настроим редактор и проверим, что окружение готово к работе.

Установка и настройка

Цель урока

Установить актуальную версию Go, проверить её в терминале и подготовить редактор для работы с кодом.

Установите Go

Откройте официальную страницу Download and install Go и выберите свою операционную систему. Следуйте инструкции именно оттуда: она поддерживается авторами Go и не привязана к устаревшему номеру версии.

Коротко по платформам:

- **Windows:** скачайте и запустите официальный MSI-инсталлятор.
- **macOS:** скачайте официальный пакет для Intel или Apple Silicon и пройдите шаги установщика.
- **Linux:** скачайте архив с официальной страницы, распакуйте его по инструкции и добавьте Go в `PATH`.

> **Не настраивайте лишнего**
>
> Не нужно вручную задавать `GOROOT` или создавать `GOPATH`, чтобы начать писать современные Go-проекты. Установщик на Windows и macOS обычно настраивает путь к команде `go`; проекты можно хранить в любой удобной директории.

Если команда `go` не найдена после установки, закройте и снова откройте терминал. На Linux проверьте шаг с добавлением `/usr/local/go/bin` в `PATH` в официальной инструкции.

Проверьте установку

Откройте терминал — PowerShell или Windows Terminal в Windows, Terminal в macOS либо привычный эмулятор терминала в Linux — и выполните:

```
go version
```

Команда должна вывести установленную версию Go и вашу платформу, например:

```
go version go1.x.x darwin/arm64
```

Число версии и платформа будут отличаться — это нормально.

Для инструментов, которые позднее будут установлены через `go install`, полезно добавить каталог `$(go env GOPATH)/bin` в `PATH`. На данном этапе это не обязательно.

Выберите редактор

Подойдёт любой текстовый редактор, но для начала рекомендуются:

- Visual Studio Code с официальным расширением Go;
- GoLand — коммерческая IDE;
- редактор, которым вы уже уверенно пользуетесь, если он поддерживает Go и `gopls`.

После установки расширения Go VS Code может предложить установить дополнительные инструменты. Соглашайтесь на рекомендуемые: среди них будет `gopls`, который показывает ошибки, подсказывает код и запускает форматирование.

Чек-лист

Перед следующим уроком убедитесь, что:

- команда `go version` выполняется без ошибки;
- вы можете открыть терминал в папке проекта;
- выбран редактор и установлена поддержка Go.

Если что-то не получилось

- `go: command not found` или «команда не найдена» — перезапустите терминал и проверьте `PATH` по официальной инструкции.
- Команда показывает неожиданно старую версию — возможно, в системе осталась старая установка. Воспользуйтесь разделом Managing Go installations.
- Не удаётся установить расширение редактора — это не мешает продолжить: Go можно писать в обычном текстовом редакторе и запускать из терминала.

Первая программа и Go-модуль

Цель урока

Создать первый Go-проект, понять назначение `go.mod`, запустить программу через `go run .` и разобрать её основные части.

Создайте папку проекта

В терминале перейдите в место, где хотите хранить учебные проекты. Например, создайте каталог `go-learning` в домашней директории:

```
mkdir -p ~/go-learning
cd ~/go-learning
mkdir hello-go
cd hello-go
```

В Windows можно создать и открыть папку через проводник, а затем выбрать в ней **Open in Terminal**. Главное — выполнять дальнейшие команды внутри папки `hello-go`.

Создайте модуль

Выполните:

```
go mod init example.com/hello-go
```

Go создаст файл `go.mod`. В нём хранится имя вашего модуля, версия Go и позднее появятся зависимости проекта.

> **Что такое модуль**
>
> Модуль — это единица проекта и управления зависимостями в Go. Для учебного проекта имя `example.com/hello-go` подходит: такой адрес не обязан существовать в интернете. Когда код попадёт в GitHub, имя обычно станет похожим на `github.com/ваше-имя/hello-go`.

Проверьте содержимое папки:

```
hello-go/
└── go.mod
```

Напишите программу

Создайте в этой папке файл `main.go` и вставьте код:

```
package main

import "fmt"

func main() {
  fmt.Println("Привет, Go!")
}
```

Запустите проект

В той же папке выполните:

```
go run .
```

Точка означает «текущая папка». Команда найдёт пакет `main` в текущем модуле, временно скомпилирует его и запустит. Вы увидите:

```
Привет, Go!
```

> **Почему `go run .`, а не `go run main.go`**
>
> Обе команды возможны для простого одного файла. Но `go run .` запускает пакет целиком и остаётся корректным, когда в нём появится несколько `.go`-файлов. Поэтому далее в учебнике будем использовать этот вариант.

Разберём программу

`package main`

Каждый Go-файл начинается с объявления пакета. Файлы одной папки с одинаковым именем пакета компилируются вместе.

`main` — специальное имя пакета для исполняемой программы. Чтобы Go мог запустить пакет `main`, в нём должна быть функция `main`.

`import "fmt"`

`import` подключает пакет с готовым кодом. `fmt` — часть стандартной библиотеки Go; в нём есть функции для форматированного ввода и вывода.

Компилятор не разрешит оставить импорт, который не используется. Это помогает поддерживать код чистым.

`func main()`

`func` объявляет функцию. `main` — точка входа: именно с неё начинается работа исполняемой программы.

```
func main() {
  // инструкции программы
}
```

Функция `main` не принимает аргументы и не возвращает результат. Ошибки в Go обычно обрабатывают внутри программы или возвращают из других функций как значение `error` — это разберём позднее.

`fmt.Println`

`fmt.Println` выводит переданные значения в стандартный вывод — обычно это окно терминала — и добавляет перевод строки в конце.

Попробуйте изменить программу:

```
func main() {
  fmt.Println("Привет, Go!")
  fmt.Println("Я создал первый модуль.")
}
```

Форматирование кода

Go придерживается единого формата кода. Примените его к текущей папке:

```
gofmt -w main.go
```

Редактор с поддержкой Go обычно запускает форматирование при сохранении. Не выравнивайте код вручную пробелами: это делает `gofmt`.

Сборка исполняемого файла

Чтобы собрать программу без запуска, используйте:

```
go build .
```

В папке появится исполняемый файл с именем проекта: `hello-go` в macOS/Linux или `hello-go.exe` в Windows. Его можно запустить из терминала:

```
./hello-go
```

В PowerShell для Windows:

```
.\hello-go.exe
```

> **Важно**
>
> Исполняемый файл создаётся для текущей ОС и архитектуры. В следующих главах он нам не нужен, поэтому после эксперимента его можно удалить вручную.

Частые ошибки

`go: cannot find main module`

Вы запустили `go run .` до команды `go mod init` или находитесь не в папке проекта. Перейдите в `hello-go` и создайте модуль.

`undefined: fmt`

Используется `fmt.Println`, но отсутствует `import "fmt"`.

`imported and not used`

Пакет импортирован, но ни одна его функция или тип не используется. Удалите импорт либо добавьте использование.

`package ... is not a main package`

Для запуска программы нужны `package main` и функция `func main()`.

Чек-лист

Вы готовы двигаться дальше, если можете:

- создать папку для проекта;
- выполнить `go mod init`;
- создать `main.go`;
- запустить проект командой `go run .`;
- объяснить назначение `package main`, `import` и `func main()`.

Самопроверка

Ответьте на вопросы после первой главы. Если ответ оказался неверным, вернитесь к соответствующему разделу и запустите пример ещё раз.

1. Какой командой можно проверить, что Go установлен и доступен в терминале?

A. `go version`

B. `go init`

C. `go start`

D. `go install`

2. Что создаёт команда `go mod init example.com/hello-go`?

A. Только файл `main.go`

B. Базу данных проекта

C. Файл go.mod для Go-модуля

D. Исполняемый файл программы

3. Какую команду стоит использовать для запуска всего пакета в текущей папке?

A. `go build main.go`

B. `go execute .`

C. `go fmt .`

D. `go run .`

4. Зачем в исполняемой Go-программе нужен `package main`?

A. Он создаёт новую переменную

B. Он включает сборщик мусора

C. Он подключает пакет `fmt`

D. Он обозначает пакет исполняемой программы

5. Что делает `import "fmt"`?

A. Подключает пакет стандартной библиотеки для форматированного ввода и вывода

B. Компилирует программу

C. Создаёт файл `go.mod`

D. Объявляет функцию `main`

6. Какая функция является точкой входа исполняемой программы на Go?

A. `func main()`

B. `func initModule()`

C. `func start()`

D. `func run()`

7. Для чего используется команда `gofmt -w main.go`?

A. Для запуска HTTP-сервера

B. Для загрузки зависимостей

C. Для удаления модуля

D. Для форматирования Go-файлов в текущей папке

Ответы к самопроверке

1. `go version`

2. Файл go.mod для Go-модуля

3. `go run .`

4. Он обозначает пакет исполняемой программы

5. Подключает пакет стандартной библиотеки для форматированного ввода и вывода

6. `func main()`

7. Для форматирования Go-файлов в текущей папке

Задания: первый модуль

Выполняйте задания по порядку. Сначала попробуйте самостоятельно, затем откройте подсказку. Решение смотрите только после собственной попытки.

Задание 1. Приветствие

Создайте модуль `example.com/hello-go`, добавьте файл `main.go` и выведите две строки:

```
Привет, Go!
Я учусь писать backend на Go.
```

Критерии проверки:

- в папке проекта есть `go.mod` и `main.go`;
- команда `go run .` завершается без ошибок;
- строки выводятся в правильном порядке.

Подсказка 1

Для создания модуля выполните `go mod init example.com/hello-go` внутри новой папки.

Подсказка 2

Для вывода строки понадобится пакет `fmt` и функция `fmt.Println`.

Решение

```
package main

import "fmt"

func main() {
  fmt.Println("Привет, Go!")
  fmt.Println("Я учусь писать backend на Go.")
}
```

Запустите:

```
go run .
```

Задание 2. Представьтесь

Измените программу так, чтобы она выводила ваше имя и город. Например:

```
Меня зовут Анна.
Я пишу код из Казани.
```

Не копируйте решение из предыдущего задания полностью: измените строки самостоятельно.

Критерии проверки:

- программа по-прежнему запускается через `go run .`;
- в выводе есть ваше имя и город;
- каждая фраза печатается с новой строки.

Подсказка

`fmt.Println` можно вызвать несколько раз внутри одной функции `main`.

Возможное решение

```
package main

import "fmt"

func main() {
  fmt.Println("Меня зовут Анна.")
  fmt.Println("Я пишу код из Казани.")
}
```

Замените имя и город на свои.

Задание 3. Соберите программу

Соберите текущий проект командой `go build .`, затем запустите получившийся файл. После проверки удалите файл сборки вручную.

Критерии проверки:

- `go build .` завершается без ошибок;
- в папке появляется `hello-go` или `hello-go.exe`;
- запуск файла выводит тот же текст, что и `go run .`.

Подсказка

В macOS/Linux файл запускается как `./hello-go`. В PowerShell используйте `.\hello-go.exe`.

Почему вывод совпадает?

`go run .` временно собирает и запускает программу. `go build .` сохраняет результат сборки в исполняемый файл, который можно запустить отдельно.

Мини-задача для будущего Task Manager API

Создайте отдельную папку `task-manager-api` и инициализируйте в ней модуль `example.com/task-manager-api`. Код писать пока не нужно: это будет основой сквозного проекта.

Критерий проверки: в папке `task-manager-api` есть файл `go.mod` с первой строкой:

```
module example.com/task-manager-api
```

Рекомендации по коду: старт с Go

- Устанавливайте Go только из официального источника и регулярно обновляйте версию.
- Создавайте `go.mod` в начале каждого нового проекта, даже если он состоит из одного файла.
- Запускайте проект командой `go run .`, а не отдельный файл: так команда останется корректной после добавления новых файлов.
- Форматируйте код через `gofmt` или настройте форматирование при сохранении в редакторе.
- Читайте сообщение компилятора до поиска ответа в интернете: Go обычно точно указывает файл, строку и причину ошибки.
- Не храните учебные и рабочие проекты в `GOPATH/src`: современный модульный Go этого не требует.

Быстрый чек-лист

Перед следующим уроком убедитесь, что `go version`, `go run .`, `gofmt -w main.go` и `go build .` выполняются в папке проекта без ошибок.

Переменные и константы

Цель урока

Научиться хранить данные в переменных, выбирать между `var` и `:=`, понимать нулевые значения и создавать понятные константы.

Что такое переменная

Переменная — это имя, по которому программа обращается к значению. Например, в переменной можно хранить название задачи, её приоритет или число выполненных действий.

У каждой переменной есть тип. Он определяет, какие значения допустимы и какие операции с ними возможны. В Go тип можно указать явно или поручить компилятору определить его по значению.

```
package main

import "fmt"

func main() {
  var taskTitle string = "Изучить Go"
  priority := 2

  fmt.Println(taskTitle, priority)
}
```

Объявление через `var`

Полная форма объявления выглядит так:

```
var name type = value
```

Пример:

```
var title string = "Купить продукты"
var priority int = 1
var completed bool = false
```

Если тип очевиден из значения, его можно не писать:

```
var title = "Купить продукты" // string
var priority = 1              // int
```

`var` используют, когда нужен явный тип, нужно объявить значение без начального значения или переменная объявляется за пределами функции.

Короткое объявление `:=`

Внутри функции чаще используют короткую форму:

```
title := "Купить продукты"
priority := 1
completed := false
```

Компилятор сам выводит тип из правой части. Короткое объявление работает **только внутри функции**.

```
package main

import "fmt"

func main() {
  title := "Подготовить отчёт"
  title = "Подготовить недельный отчёт"

  fmt.Println(title)
}
```

После объявления значение переменной можно менять, но новый тип присвоить нельзя:

```
priority := 1
priority = 2      // допустимо
// priority = "high" // ошибка: string нельзя присвоить int
```

> **Не путайте `:=` и `=`**
>
> `:=` создаёт хотя бы одну новую переменную. `=` присваивает новое значение уже существующей переменной.

Нулевые значения

Если объявить переменную через `var`, но не присвоить значение, Go даст ей нулевое значение соответствующего типа.

```
package main

import "fmt"

func main() {
  var priority int
  var completed bool
  var description string

  fmt.Printf("priority=%d, completed=%t, description=%q\n", priority, completed,
description)
}
```

Вывод:

```
priority=0, completed=false, description=""
```

Тип	Нулевое значение
Целые и дробные числа	`0`
`bool`	`false`
`string`	`""`

Нулевое значение — нормальная часть языка, а не «мусор» в памяти. Позднее появятся нулевые значения для более сложных типов.

Константы

Константа — значение, которое нельзя изменить после объявления. Константы объявляют ключевым словом `const`:

```
const maxTitleLength = 120
const defaultPriority = 2
```

Константы подходят для правил и значений, которые не должны меняться во время работы программы. Например, для допустимой длины заголовка или кодов приоритета.

```
package main

import "fmt"
```

```
const defaultPriority = 2func main() {  priority := defaultPriority
fmt.Println(priority)}
```

Попытка изменить константу — ошибка компиляции:

```
const maxTitleLength = 120
// maxTitleLength = 200 // нельзя изменить константу
```

Последовательные константы и `iota`

Внутри блока `const` идентификатор `iota` начинается с `0` и увеличивается на единицу в каждой следующей строке. Это удобно для фиксированных наборов значений.

```
const (
  priorityLow = iota + 1
  priorityMedium
  priorityHigh
)
```

Значения будут такими:

```
priorityLow    = 1
priorityMedium = 2
priorityHigh   = 3
```

В будущих уроках мы применим такой подход для статусов и приоритетов задач. Пока важно помнить: `iota` работает только в блоке `const`, а первая строка здесь начинается с `iota + 1`, поэтому отсчёт идёт с единицы.

Имена и область видимости

Имена должны объяснять назначение значения:

```
taskTitle := "Оплатить интернет"
maxTitleLength := 120
isCompleted := false
```

В Go используют `camelCase`. Имя с заглавной буквы экспортируется из пакета и становится доступно коду из других пакетов; эту тему разберём вместе с пакетами. Внутри одной функции сейчас используйте строчные имена, если нет специальной причины делать иначе.

Переменная существует в области, где объявлена. В этом уроке все переменные находятся внутри `main`, поэтому доступны только внутри её фигурных скобок.

Частые ошибки

Неиспользуемая переменная

Локальную переменную нельзя объявить и забыть использовать:

```
func main() {
  title := "Изучить Go"
  // ошибка: declared and not used: title
}
```

Добавьте использование, например `fmt.Println(title)`, либо удалите переменную.

`:=` вне функции

Такой код не скомпилируется:

```
// title := "Изучить Go"
```

За пределами функции используется `var` или `const`.

Повторное `:=` без новой переменной

```
title := "Первая задача"
// title := "Вторая задача" // ошибка: no new variables on left side of :=
title = "Вторая задача"
```

Дальше

Разберём числовые и логические типы, а также явное преобразование значений.

Базовые типы и преобразования

Цель урока

Научиться выбирать базовые типы для данных и явно преобразовывать совместимые значения. Строки и обработку текста разберём в следующей главе.

Целые числа

Для обычных целых чисел чаще всего используется `int`:

```
taskCount := 3
priority := 2
```

Размер `int` зависит от архитектуры компьютера: обычно это 64 бита на современных машинах, но полагаться на конкретный размер не стоит. Когда размер важен — например, формат данных или внешний API требует 64-битное число — используйте явный тип `int64`.

Тип	Когда нужен
`int`	Обычные целые числа: счётчики, индексы, приоритеты
`int8`, `int16`, `int32`, `int64`	Когда важны диапазон или размер числа
`uint8`, `uint16`, `uint32`, `uint64`	Беззнаковые значения, если они действительно нужны
`byte`	Псевдоним `uint8`, обычно для сырых байтов
`rune`	Псевдоним `int32`, обычно для Unicode-кода символа

Не выбирайте `uint` только потому, что число не бывает отрицательным. В прикладном коде смешивание знаковых и беззнаковых чисел нередко усложняет вычисления. Начинайте с `int`, пока нет конкретной причины выбрать другой тип.

Дробные числа

Для дробных значений обычно используют `float64`:

```
progress := 0.75
```

`float64` хранит приближённые значения. Поэтому для денег его обычно не используют: `0.1 + 0.2` в двоичной системе может быть представлено неточно. Для денежных сумм в реальном приложении выбирают подходящее целочисленное представление или специальную библиотеку; к этой задаче вернёмся в проектной части.

Логический тип `bool`

`bool` принимает только два значения: `true` и `false`.

```
completed := false
hasDescription := true
```

В Go нельзя использовать число вместо логического значения:

```
// if 1 { } // ошибка
```

Условия и операторы сравнения мы подробно разберём в следующей главе.

Явные преобразования типов

Go не выполняет неявные преобразования между числовыми типами. Тип нужно указывать явно:

```
priority := 2
priorityAsFloat := float64(priority)

fmt.Println(priorityAsFloat) // 2
```

Общий вид:

```
newValue := Type(value)
```

При преобразовании `float64` в `int` дробная часть отбрасывается, а не округляется:

```
progress := 3.9
completedSteps := int(progress)

fmt.Println(completedSteps) // 3
```

Преобразование может привести к потере данных. Например, большое число не поместится в маленький тип:

```
value := 300
smallValue := uint8(value)

fmt.Println(smallValue) // 44
```

Перед преобразованием убедитесь, что значение находится в допустимом диапазоне.

Частые ошибки

Неявное преобразование

```
var priority int = 2
// var score float64 = priority // ошибка
score := float64(priority)
```

Преобразование не равно валидации

`int(value)` не проверяет, что `value` подходит для бизнес-правил. Например, `int(-0.7)` вернёт `0`. Проверку допустимых значений мы добавим, когда разберём условия.

Дальше

Сейчас закрепите базовые типы в самопроверке и практике. После этого разберём строки, UTF-8 и полезные функции пакета `strings`.

Самопроверка

Ответьте на вопросы без подсказок. Если ответ вызывает сомнение, вернитесь к примеру, измените его и запустите программу.

1. Какой вариант создаёт новую переменную внутри функции и позволяет компилятору вывести её тип?

A. title := "Купить продукты"

B. const title := "Купить продукты"

C. string title = "Купить продукты"

D. title = "Купить продукты"

2. Где допустимо использовать короткое объявление :=?

A. В любой части Go-файла

B. Только внутри функции

C. Только внутри блока const

D. Только за пределами функции

3. Какое значение получит переменная var completed bool без инициализации?

A. Ошибка компиляции

B. true

C. false

D. nil

4. Что нужно использовать, если значение не должно меняться после объявления?

A. var

B. const

C. type

D. :=

5. Какое значение имеет первая строка с iota + 1 в блоке const?

A. Зависит от типа константы

B. 0

C. 1

D. iota нельзя использовать с арифметикой

6. Какой тип обычно выбирают для обычного счётчика задач?

A. rune

B. int

C. bool

D. string

7. Какое нулевое значение у переменной var title string?

A. Ошибка компиляции

B. nil

C. false

D. Пустая строка ""

8. Как правильно преобразовать int priority в float64?

A. var value float64 = priority

B. float64(priority)

C. priority.float64()

D. priority as float64

9. Что произойдёт при int(3.9)?

A. Получится 3

B. Получится 3.9

C. Будет ошибка компиляции

D. Получится 4

10. Что может произойти при преобразовании большого int в uint8?

A. Значение обязательно станет nil

B. Преобразование всегда округляет число

C. Часть значения может быть потеряна

D. Go всегда автоматически выберет uint64

Ответы к самопроверке

1. title := "Купить продукты"

2. Только внутри функции

3. false

4. const

5. 1

6. int

7. Пустая строка ""

8. float64(priority)

9. Получится 3

10. Часть значения может быть потеряна

Задания: значения для задачи

На этом этапе Task Manager API ещё не имеет структуры задачи или базы данных. Мы подготовим отдельные значения, из которых позднее соберём модель задачи.

Задание 1. Данные одной задачи

Создайте новый модуль `example.com/task-manager`, если не сделали этого в первой главе. В файле `main.go` объявите и выведите:

- название задачи: `Изучить переменные в Go`;
- описание: `Прочитать урок и выполнить практику`;
- приоритет: `2`;
- признак выполнения: `false`.

Используйте `var` хотя бы для одной переменной, а `:=` — хотя бы для одной другой.

Критерии проверки:

- проект запускается через `go run .`;
- используются `string`, `int` и `bool`;
- значения выводятся в терминал;
- в коде есть и `var`, и `:=`.

Подсказка 1

Для вывода нескольких значений можно вызвать `fmt.Println` несколько раз или передать несколько аргументов в один вызов.

Подсказка 2

Начните так:

```
var title string = "Изучить переменные в Go"
priority := 2
```

Решение

```
package main

import "fmt"

func main() {
  var title string = "Изучить переменные в Go"
  description := "Прочитать урок и выполнить практику"
  priority := 2
  completed := false

  fmt.Println("Название:", title)
  fmt.Println("Описание:", description)
  fmt.Println("Приоритет:", priority)
  fmt.Println("Выполнена:", completed)
}
```

Задание 2. Приоритеты как константы

Добавьте три константы приоритета: низкий, средний и высокий. Их значения должны быть `1`, `2` и `3`. Используйте `iota` и начните отсчёт с единицы.

Затем создайте переменную `taskPriority` со средним приоритетом и выведите её.

Критерии проверки:

- константы находятся в одном блоке `const`;
- для первого значения используется `iota + 1`;
- `taskPriority` получает значение средней константы;
- программа печатает `2`.

Подсказка

```
const (
  priorityLow = iota + 1
  // следующие значения продолжат последовательность
)
```

Решение

```
package main

import "fmt"

const (
  priorityLow = iota + 1
  priorityMedium
  priorityHigh
)

func main() {
  taskPriority := priorityMedium

  fmt.Println(taskPriority)
}
```

Задание 3. Преобразование прогресса

Пусть прогресс выполнения задачи хранится в `float64` и равен `3.8`. Преобразуйте его в `int` и выведите оба значения.

Критерии проверки:

- исходное значение имеет тип `float64`;
- для преобразования используется `int(progress)`;
- результат преобразования равен `3`;
- вы можете объяснить, почему это не округление.

Подсказка

При преобразовании `float64` в `int` дробная часть отбрасывается.

Решение

```
package main

import "fmt"

func main() {
  progress := 3.8
  completedSteps := int(progress)

  fmt.Println("Прогресс:", progress)
  fmt.Println("Завершённых шагов:", completedSteps)
}
```

Проверьте себя

Перед следующей главой убедитесь, что вы можете без подсказки:

- объяснить разницу между `var`, `:=` и `const`;
- назвать нулевые значения `int`, `bool` и `string`;
- выбрать `int`, `bool` или `string` для простых данных задачи;
- выполнить явное преобразование `float64` в `int`.

Рекомендации по коду: переменные и базовые типы

- Внутри функций по умолчанию используйте `:=`, если тип очевиден из значения; `var` оставляйте для нулевого значения и явного типа.
- Объявляйте переменную как можно ближе к месту первого использования.
- Давайте значениям имена по смыслу: `taskTitle`, `priority`, `completed`, а не `x`, `data` или `value` без контекста.
- Используйте `const` для неизменяемых правил и значений, а не для данных, которые меняются во время работы программы.
- Начинайте с `int`, пока формат данных или внешний API не требуют конкретный размер, например `int64`.
- Проверяйте диапазон до преобразования к более узкому числовому типу: преобразование само по себе не валидирует данные.

Чек-лист

В коде нет неиспользуемых переменных, `:=` не применяется за пределами функции, а каждая константа описывает устойчивое правило, а не временное состояние.

Строки и обработка текста

Цель урока

Понять устройство строк в Go, разницу между байтами и рунами, а также научиться очищать, искать, разделять и собирать текст через пакет `strings`.

Названия задач — пользовательский текст. Чтобы корректно считать символы, убирать лишние пробелы и готовить данные к валидации, нужно знать особенности строк и UTF-8.

Строка в Go

`string` — неизменяемая последовательность байтов. Строковые литералы обычно записывают в двойных кавычках:

```
title := "Купить продукты"
message := "Задача создана"
```

Строки можно соединять оператором `+`:

```
title := "Купить " + "продукты"
```

Исходную строку изменить нельзя:

```
title := "Го"
// title[0] = 'П' // ошибка компиляции
```

Функции пакета `strings` и операции конкатенации создают новое строковое значение.

Байты, руны и UTF-8

Go хранит текст в UTF-8. `len` для строки возвращает количество **байтов**, а не символов, которые видит человек:

```
package main

import "fmt"

func main() {
  title := "Go!"
  russianTitle := "Го"
```

```
  fmt.Println(len(title))        // 3 байта fmt.Println(len(russianTitle)) // 4
байта}
```

`byte` — псевдоним `uint8`; он подходит для отдельных байтов. `rune` — псевдоним `int32`, обычно представляющий Unicode-код символа.

Чтобы посчитать количество рун, используйте `unicode/utf8`:

```
package main

import (
  "fmt"
  "unicode/utf8"
)

func main() {
  title := "Задача"

  fmt.Println(len(title))                  // 12 байтов
  fmt.Println(utf8.RuneCountInString(title)) // 6 рун
}
```

Индексы строки относятся к байтам. Обход строки по Unicode-символам через `range` разберём в главе о циклах, после знакомства с `for`.

Очистка текста: `strings.TrimSpace`

Пользователь может добавить пробелы в начале или конце названия:

```
title := "  Купить молоко  "
cleanTitle := strings.TrimSpace(title)

fmt.Printf("%q\n", cleanTitle) // "Купить молоко"
```

`TrimSpace` удаляет пробелы, табуляции и переводы строк только с краёв. Внутренние пробелы сохраняются:

```
title := "Купить   молоко"
fmt.Println(strings.TrimSpace(title)) // Купить   молоко
```

Текст нормализуют до валидации. Тогда строка из одних пробелов станет пустой и не пройдёт проверку.

Поиск и изменение регистра

```
title := "Купить МОЛОКО"

fmt.Println(strings.ToLower(title)) // купить молоко
fmt.Println(strings.ToUpper(title)) // КУПИТЬ МОЛОКО
```

`strings.Contains` проверяет наличие подстроки:

```
title := "Купить молоко"
hasMilk := strings.Contains(strings.ToLower(title), "молоко")

fmt.Println(hasMilk) // true
```

Для поиска без учёта регистра приведите обе строки к одному регистру.

Функция	Назначение
`strings.TrimSpace(s)`	Удаляет пробелы по краям
`strings.ToLower(s)`	Переводит в нижний регистр
`strings.ToUpper(s)`	Переводит в верхний регистр
`strings.Contains(s, sub)`	Проверяет наличие подстроки
`strings.HasPrefix(s, prefix)`	Проверяет начало строки
`strings.HasSuffix(s, suffix)`	Проверяет конец строки

Частые ошибки

Ожидание числа символов от `len`

```
fmt.Println(len("Привет")) // 12, а не 6
```

Если нужна длина текста в рунах, используйте `utf8.RuneCountInString`.

Валидация до TrimSpace

```
if len(title) == 0 {
  return ErrEmptyTitle
}

title = strings.TrimSpace(title)
```

Строка из пробелов пройдёт первую проверку. Сначала нормализуйте текст, затем валидируйте.

Дальше

Закрепите строки на нормализации названия и поиске текста. Затем перейдём к операторам и выражениям.

Самопроверка

1. Что возвращает len("Привет")?

A. Ошибку компиляции

B. Количество байтов: 12

C. Количество рун: 6

D. Всегда 0 для кириллицы

2. Как посчитать количество рун в строке?

A. len(title)

B. strings.Count(title)

C. int(title)

D. utf8.RuneCountInString(title)

3. Что делает strings.TrimSpace?

A. Переводит строку в нижний регистр

B. Считает количество символов

C. Удаляет пробельные символы в начале и конце строки

D. Удаляет все пробелы внутри строки

4. Как выполнить поиск подстроки без учёта регистра?

A. Привести строку и подстроку к одному регистру через strings.ToLower

B. Преобразовать строку в int

C. Использовать только len

D. Вызвать time.Parse

5. Что делает strings.HasPrefix(title, "Купить")?

A. Проверяет, заканчивается ли строка указанным текстом

B. Проверяет, начинается ли строка с указанного текста

C. Удаляет указанный текст из начала строки

D. Считает число слов в строке

6. Что возвращает strings.ToUpper("задача")?

A. Число заглавных букв

B. Новую строку "ЗАДАЧА"

C. Слайс рун

D. Изменяет исходную строку на месте

7. Что нужно использовать, если требуется обойти строку по Unicode-символам?

A. Преобразование строки в bool

B. Обычный доступ title[index] для всех языков

C. Цикл for range, который будет разобран позже в главе о циклах

D. Только strings.TrimSpace

8. Можно ли изменить один символ строки через title[0] = 'П'?

A. Да, если строка содержит кириллицу

B. Да, после strings.TrimSpace

C. Нет, строки неизменяемы

D. Только с типом byte

9. Почему strings.TrimSpace(" ") полезен перед валидацией названия?

A. Он заменяет все внутренние пробелы на запятые

B. Он создаёт JSON-представление строки

C. Он автоматически добавляет название в Task

D. Он превращает строку из пробелов в пустую строку

10. Почему строку из пробелов нужно сначала обработать TrimSpace, а потом валидировать?

A. len не работает без TrimSpace

B. Иначе Go не скомпилирует строку

C. TrimSpace автоматически записывает строку в JSON

D. После TrimSpace она станет пустой и корректно не пройдёт проверку

Ответы к самопроверке

1. Количество байтов: 12

2. utf8.RuneCountInString(title)

3. Удаляет пробельные символы в начале и конце строки

4. Привести строку и подстроку к одному регистру через strings.ToLower

5. Проверяет, начинается ли строка с указанного текста

6. Новую строку "ЗАДАЧА"

7. Цикл for range, который будет разобран позже в главе о циклах

8. Нет, строки неизменяемы

9. Он превращает строку из пробелов в пустую строку

10. После TrimSpace она станет пустой и корректно не пройдёт проверку

Задания: нормализация и поиск названия

Название задачи — пользовательский ввод. Уберём лишние пробелы перед проверкой, выполним поиск без учёта регистра и разберём разницу между байтами и рунами.

Задание 1. Нормализуйте название

Создайте функцию `normalizeTitle(title string) string`, которая удаляет пробелы по краям. Передайте ей строку `" Купить продукты "` и выведите результат в кавычках.

Критерии проверки:

- используется `strings.TrimSpace`;
- функция возвращает новую строку;
- результат равен `"Купить продукты"`;
- исходная строка не изменяется.

Подсказка

```
func normalizeTitle(title string) string {
  return strings.TrimSpace(title)
}
```

Решение

```
package main

import (
  "fmt"
  "strings"
)

func normalizeTitle(title string) string {
  return strings.TrimSpace(title)
}

func main() {
  title := "  Купить продукты  "

  fmt.Printf("%q\n", normalizeTitle(title))
}
```

Задание 2. Проверьте слово в названии

Проверьте, содержит ли строка `Купить МОЛОКО` слово `молоко` без учёта регистра. Выведите результат типа `bool`.

Критерии проверки:

- используется `strings.Contains`;
- название и искомое слово приводятся к одному регистру;
- программа выводит `true`.

Подсказка

```
strings.Contains(strings.ToLower(title), strings.ToLower(query))
```

Решение

```
package main

import (
  "fmt"
  "strings"
)

func main() {
  title := "Купить МОЛОКО"
  query := "молоко"

  found := strings.Contains(strings.ToLower(title), strings.ToLower(query))
  fmt.Println(found)
}
```

Задание 3. Посчитайте символы названия

Для строки `"Купить молоко"` выведите количество байтов и количество рун. Используйте `len` и `utf8.RuneCountInString`.

Критерии проверки:

- импортирован `unicode/utf8`;
- байты получены через `len`;
- руны получены через `utf8.RuneCountInString`;
- результаты различаются из-за кириллицы.

Подсказка

```
fmt.Println("Байты:", len(title))
fmt.Println("Руны:", utf8.RuneCountInString(title))
```

Решение

```
package main

import (
  "fmt"
  "unicode/utf8"
)

func main() {
  title := "Купить молоко"

  fmt.Println("Байты:", len(title))
  fmt.Println("Руны:", utf8.RuneCountInString(title))
}
```

Проверьте себя

Перед следующей главой убедитесь, что вы можете:

- объяснить разницу между байтами и рунами;
- очистить пользовательский текст до валидации;
- выполнить поиск подстроки без учёта регистра;
- отличить количество байтов от количества рун.

Рекомендации по коду: строки и текст

- Нормализуйте пользовательский ввод через `strings.TrimSpace` до валидации и сохранения.
- Помните, что `len(string)` возвращает байты; для видимых Unicode-символов используйте `utf8.RuneCountInString`.
- Не изменяйте строку по индексу: строки неизменяемы. Создавайте новое значение.
- Для поиска без учёта регистра приводите обе стороны к одному регистру.
- Не используйте `strings.Split` как формат надёжного хранения структурированных данных — для этого подходит JSON.
- Не пытайтесь оптимизировать короткие конкатенации; `strings.Builder` нужен при построении большого текста в цикле.

Чек-лист

Строка из одних пробелов не проходит валидацию, поиск явно описывает чувствительность к регистру, а требования к длине текста различают байты и руны.

Операторы и выражения

Цель урока

Научиться выполнять вычисления, сравнивать значения, объединять условия в логические выражения и понимать, в каком порядке Go вычисляет части выражения.

Операторы понадобятся уже в Task Manager API: например, чтобы узнать длину заголовка, сравнить приоритет с допустимым диапазоном или проверить несколько требований к задаче одновременно.

Что такое выражение

Выражение — это фрагмент кода, который вычисляет значение.

```
priority + 1    // вычисляет число
priority >= 1   // вычисляет true или false
!completed      // вычисляет противоположное логическое значение
```

Оператор — это символ или ключевое слово, которое участвует в таком вычислении: `+`, `>=`, `&&`, `!` и другие.

Арифметические операторы

Для целых и дробных чисел доступны основные операции:

Оператор	Значение	Пример
`+`	сложение	`2 + 3` → `5`
`-`	вычитание	`5 - 2` → `3`
`*`	умножение	`4 * 3` → `12`
`/`	деление	`7 / 2` → `3` для `int`
`%`	остаток от деления	`7 % 2` → `1`

```
package main

import "fmt"

func main() {
  totalTasks := 7
  completedTasks := 3
  remainingTasks := totalTasks - completedTasks
```

```
  fmt.Println(remainingTasks) // 4  fmt.Println(totalTasks % 2) // 1}
```

Деление целых чисел

Если оба операнда имеют целочисленный тип, дробная часть результата отбрасывается:

```
totalTasks := 7
days := 2

fmt.Println(totalTasks / days) // 3
```

Если нужен дробный результат, преобразуйте хотя бы один операнд в `float64`:

```
averagePerDay := float64(totalTasks) / float64(days)
fmt.Println(averagePerDay) // 3.5
```

Остаток от деления

Оператор `%` полезен, когда нужно узнать, делится ли число без остатка:

```
taskCount := 8
isEven := taskCount%2 == 0

fmt.Println(isEven) // true
```

Операторы сравнения

Операторы сравнения всегда возвращают `bool`: `true` или `false`.

Оператор	Значение
`==`	равно
`!=`	не равно
`<`	меньше
`<=`	меньше или равно
`>`	больше
`>=`	больше или равно

```
package main

import "fmt"
```

```
func main() {   priority := 2   fmt.Println(priority == 2) // true
fmt.Println(priority != 1) // true  fmt.Println(priority >= 1) // true
fmt.Println(priority > 3)  // false}
```

Строки тоже можно сравнивать на равенство:

```
status := "new"
isNew := status == "new"

fmt.Println(isNew) // true
```

`=` и `==` — разные операторы

`=` присваивает значение переменной, а `==` сравнивает два значения.

```
priority = 2      // присваивание
isDefault := priority == 2 // сравнение
```

Логические операторы

Логические операторы работают со значениями типа `bool`.

Оператор	Значение	Результат
`&&`	И	`true`, если оба выражения истинны
`		`	`
`!`	НЕ	меняет `true` на `false` и наоборот

Для Task Manager API можно составить проверку допустимого приоритета без ветвления:

```
package main

import "fmt"

func main() {
  priority := 2
  isValidPriority := priority >= 1 && priority <= 3

  fmt.Println(isValidPriority) // true
}
```

Проверка названия задачи может состоять из нескольких требований:

```
titleLength := len("Купить молоко")
hasValidLength := titleLength > 0 && titleLength <= 120

fmt.Println(hasValidLength) // true
```

`&&` и `||` используют короткое замыкание: Go вычисляет левую часть первой и не вычисляет правую, если итог уже известен. Например, в `false && expression` правая часть не будет вычислена.

Присваивание и изменение значения

Обычное присваивание:

```
priority = 3
```

Сокращённые формы объединяют операцию и присваивание:

```
taskCount := 5

taskCount += 2 // 7
taskCount -= 1 // 6
taskCount *= 2 // 12
taskCount /= 3 // 4
```

Для увеличения или уменьшения целого числа на единицу есть `++` и `--`:

```
taskCount := 5
taskCount++ // 6
taskCount-- // 5
```

В Go `++` и `--` — отдельные инструкции, а не выражения. Их нельзя использовать внутри другого выражения:

```
// nextCount := taskCount++ // ошибка
taskCount++
nextCount := taskCount
```

Приоритет операций

Как в математике, умножение и деление выполняются раньше сложения и вычитания:

```
result := 2 + 3*4
fmt.Println(result) // 14
```

Круглые скобки делают порядок явным:

```
result := (2 + 3) * 4
fmt.Println(result) // 20
```

При сложных выражениях ставьте скобки, даже если помните приоритет. Так код проще проверять и поддерживать.

Полезное упрощённое правило:

1. сначала выполняются `*`, `/`, `%`;
2. затем `+` и `-`;
3. затем сравнения: `<`, `<=`, `>`, `>=`, `==`, `!=`;
4. затем `&&`;
5. затем `||`.

Переполнение чисел

Тип числа имеет ограниченный диапазон. При переполнении значения переменной результат может «перейти через границу» диапазона:

```
package main

import "fmt"

func main() {
  var priority int8 = 127
  priority++

  fmt.Println(priority) // -128
}
```

С переполнением не нужно экспериментировать в бизнес-логике. Для обычных счётчиков выбирайте `int`; при данных из внешнего источника проверяйте диапазон до преобразования к более узкому типу.

Частые ошибки

Ожидание дробного результата от `int / int`

```
fmt.Println(7 / 2) // 3, не 3.5
```

Используйте `float64`, если нужна дробная часть.

Использование одного `=` в сравнении

```
// isHigh := priority = 3 // ошибка
isHigh := priority == 3
```

Неверный порядок вычислений

```
limit := 2 + 3*4   // 14
expected := (2 + 3) * 4 // 20
```

Когда смысл зависит от порядка, ставьте скобки.

Дальше

Проверьте себя в коротком квизе и составьте несколько проверок для будущей задачи.

Самопроверка

1. Что вычислит выражение 7 / 2, если оба значения имеют тип int?

A. 3

B. 4

C. 3.5

D. Ошибка компиляции

2. Какой оператор возвращает остаток от деления?

A. /

B. &&

C. *

D. %

3. Какой оператор проверяет равенство двух значений?

A. =

B. ==

C. :=

D. !=

4. Какое значение вернёт выражение true && false?

A. false

B. true

C. Ошибка компиляции

D. 0

5. Что означает оператор ||?

A. Истина, только если истинны оба выражения

B. Изменение значения на противоположное

C. Целочисленное деление

D. Истина, если истинно хотя бы одно выражение

6. Какой результат у выражения 2 + 3 * 4?

A. 14

B. 9

C. 20

D. 24

7. Как записать увеличение taskCount на единицу?

A. taskCount + 1

B. ++taskCount

C. taskCount :=+ 1

D. taskCount++

8. Можно ли использовать taskCount++ как часть выражения?

A. Да, если taskCount имеет тип int

B. Только вместе с оператором :=

C. Нет, ++ — отдельная инструкция в Go

D. Да, только в функции main

9. Как проверить, что priority находится в диапазоне от 1 до 3 включительно?

A. priority >= 1 && priority <= 3

B. priority = 1 && 3

C. priority in 1..3

D. priority >= 1 || priority <= 3

10. Что произойдёт с int8, равным 127, после увеличения на единицу?

A. Станет -128 из-за переполнения

B. Станет 128

C. Программа всегда остановится с ошибкой

D. Станет 0

Ответы к самопроверке

1. 3

2. %

3. ==

4. false

5. Истина, если истинно хотя бы одно выражение

6. 14

7. taskCount++

8. Нет, ++ — отдельная инструкция в Go

9. priority >= 1 && priority <= 3

10. Станет -128 из-за переполнения

Задания: проверки задачи

Пока мы не используем `if`, поэтому программа не будет принимать решения и выводить разные сообщения. Вместо этого она будет вычислять логические значения: `true` или `false`. В следующей главе научимся использовать эти результаты в условиях.

Задание 1. Допустимый приоритет

В Task Manager API допустимый приоритет задачи находится в диапазоне от `1` до `3` включительно. Создайте переменную `priority` со значением `2`, вычислите `isValidPriority` и выведите результат.

Критерии проверки:

- `priority` имеет тип `int`;
- `isValidPriority` использует `>=`, `<=` и `&&`;
- программа выводит `true`;
- при изменении `priority` на `4` программа выводит `false`.

Подсказка

Проверка должна быть истинной одновременно для двух условий: значение не меньше `1` и не больше `3`.

Решение

```
package main

import "fmt"

func main() {
  priority := 2
  isValidPriority := priority >= 1 && priority <= 3

  fmt.Println(isValidPriority)
}
```

Задание 2. Допустимая длина заголовка

Заголовок задачи не должен быть пустым и не должен быть длиннее 120 байтов. Создайте строку `title := "Купить молоко"`, посчитайте её длину и сохраните результат проверки в `hasValidTitleLength`.

Критерии проверки:

- длина получена через `len(title)`;
- проверка использует `>` и `<=`;
- два условия объединены через `&&`;
- для указанного заголовка выводится `true`.

Подсказка

Пустая строка имеет длину `0`. Проверьте, что длина больше `0` и не превышает `120`.

Решение

```
package main

import "fmt"

func main() {
  title := "Купить молоко"
  titleLength := len(title)
  hasValidTitleLength := titleLength > 0 && titleLength <= 120

  fmt.Println(hasValidTitleLength)
}
```

Задание 3. Готовность задачи к созданию

Объедините две предыдущие проверки. Задача готова к созданию, если и приоритет, и длина заголовка допустимы.

Создайте переменную `isReadyToCreate` и выведите её. Затем измените заголовок на пустую строку и убедитесь, что результат стал `false`.

Критерии проверки:

- `isReadyToCreate` использует `hasValidTitleLength && isValidPriority`;
- с корректными исходными значениями программа выводит `true`;
- с пустым заголовком программа выводит `false`.

Подсказка

Результаты уже являются `bool`, поэтому их можно объединить напрямую: `firstCheck && secondCheck`.

Решение

```
package main

import "fmt"

func main() {
  title := "Купить молоко"
  priority := 2

  hasValidTitleLength := len(title) > 0 && len(title) <= 120
  isValidPriority := priority >= 1 && priority <= 3
  isReadyToCreate := hasValidTitleLength && isValidPriority

  fmt.Println(isReadyToCreate)
}
```

Задание 4. Среднее число задач

За 7 дней создано 25 задач. Выведите среднее количество задач в день как дробное число. Затем отдельно выведите число полных задач в день при целочисленном делении.

Критерии проверки:

- для дробного результата хотя бы один операнд преобразован к `float64`;
- дробный результат приблизительно равен `3.5714285714285716`;
- целочисленный результат равен `3`;
- вы можете объяснить разницу результатов.

Подсказка

Для дробного результата используйте `float64(totalTasks) / float64(days)`.

Решение

```
package main

import "fmt"

func main() {
  totalTasks := 25
  days := 7

  averageTasksPerDay := float64(totalTasks) / float64(days)
  wholeTasksPerDay := totalTasks / days

  fmt.Println(averageTasksPerDay)
  fmt.Println(wholeTasksPerDay)
}
```

Проверьте себя

Перед следующей главой убедитесь, что вы можете:

- отличить `=` от `==`;
- объяснить разницу между `&&`, `||` и `!`;
- рассчитать выражение с учётом приоритета операций;
- объяснить, почему `7 / 2` равно `3`;
- составить проверку допустимого значения без `if`.

Рекомендации по коду: операторы и выражения

- Используйте скобки, если порядок вычислений влияет на смысл или неочевиден при первом чтении.
- Не путайте присваивание `=` и сравнение `==`; давайте результатам проверок имена `is...`, `has...`, `can...`.
- Помните, что `int / int` отбрасывает дробную часть. Преобразуйте операнд к `float64`, только если нужен дробный результат.
- Не используйте `++` и `--` внутри сложных выражений: в Go это отдельные инструкции.
- Проверяйте граничные значения, особенно при вычислении приоритетов, индексов и счётчиков.
- Не применяйте переполнение как рабочую бизнес-логику.

Чек-лист

Условия читаются слева направо, сложные выражения разбиты на именованные промежуточные значения, а правила диапазонов выражены явно.

Условия: `if` и `switch`

Цель урока

Научиться выполнять код по условию, проверять несколько вариантов и выводить понятное сообщение при валидации будущей задачи.

В предыдущей главе мы вычисляли `true` и `false`. Теперь используем эти значения, чтобы программа могла выбрать нужное действие.

`if`

Конструкция `if` выполняет блок кода, только если условие имеет значение `true`.

```
package main

import "fmt"

func main() {
  priority := 3

  if priority == 3 {
    fmt.Println("Высокий приоритет")
  }
}
```

В Go условие записывается без круглых скобок, а фигурные скобки обязательны:

```
if priority > 0 {
  fmt.Println("Приоритет задан")
}
```

`if` и `else`

`else` задаёт альтернативный блок, когда условие ложно:

```
package main

import "fmt"

func main() {
```

```
  completed := false  if completed {    fmt.Println("Задача выполнена")  } else {
fmt.Println("Задача ещё в работе")  }}
```

Условие типа `bool` не нужно сравнивать с `true`:

```
if completed { // предпочтительно
  fmt.Println("Готово")
}

// if completed == true { ... } // работает, но избыточно
```

Несколько ветвей: `else if`

Когда вариантов больше двух, используйте `else if`:

```
package main

import "fmt"

func main() {
  priority := 2

  if priority == 1 {
    fmt.Println("Низкий приоритет")
  } else if priority == 2 {
    fmt.Println("Средний приоритет")
  } else if priority == 3 {
    fmt.Println("Высокий приоритет")
  } else {
    fmt.Println("Неизвестный приоритет")
  }
}
```

Выполняется только первая ветвь с истинным условием. Поэтому проверки обычно располагают от более специальной к более общей.

Валидация будущей задачи

Теперь можно превратить логические проверки в понятный результат для пользователя.

```
package main

import "fmt"
```

```
func main() {  title := "Купить молоко"  priority := 2  if len(title) == 0 {
fmt.Println("Название задачи не должно быть пустым") } else if len(title) > 120 {
fmt.Println("Название задачи длиннее 120 байтов")  } else if priority < 1 || priority
> 3 {    fmt.Println("Приоритет должен быть от 1 до 3") } else {   fmt.Println("Задачу
можно создать")  }}
```

Такая последовательность проверок — основа валидации. Сначала программа находит первую проблему, а если проблем нет — сообщает об успехе. Когда мы изучим функции, вынесем эту логику из `main`.

Короткое объявление внутри `if`

В `if` можно объявить переменную, доступную только в самом условии и его ветвях:

```
if titleLength := len(title); titleLength > 120 {
  fmt.Println("Название слишком длинное:", titleLength)
}
```

Здесь сначала выполняется `titleLength := len(title)`, затем проверяется `titleLength > 120`.

Переменная не существует за пределами конструкции `if`:

```
if titleLength := len(title); titleLength > 0 {
  fmt.Println("Название есть")
}

// fmt.Println(titleLength) // ошибка: переменная вне области видимости
```

Эта форма особенно полезна, когда промежуточное значение нужно только для одной проверки.

`switch`

`switch` удобно использовать, когда одно значение сравнивается с несколькими вариантами. Например, для преобразования числового приоритета в текст:

```
package main

import "fmt"

func main() {
  priority := 2

  switch priority {
  case 1:
```

```
    fmt.Println("Низкий приоритет")  case 2:    fmt.Println("Средний приоритет")
case 3:    fmt.Println("Высокий приоритет") default:   fmt.Println("Неизвестный
приоритет")  }}
```

В Go выполнение не «проваливается» в следующий `case` автоматически: после подходящей ветви `switch` завершается. Поэтому писать `break` после каждого `case` не нужно.

Несколько совпадающих значений можно перечислить через запятую:

```
switch priority {
case 1, 2:
  fmt.Println("Обычный приоритет")
case 3:
  fmt.Println("Срочная задача")
default:
  fmt.Println("Некорректный приоритет")
}
```

Когда выбрать `if`, а когда `switch`

Используйте `if`, когда:

- проверяете диапазон: `priority >= 1 && priority <= 3`;
- объединяете условия через `&&` и `||`;
- проверяете разнородные условия;
- важен порядок проверки ошибок.

Используйте `switch`, когда:

- сравниваете одно значение с несколькими конкретными вариантами;
- каждой ветви соответствует своё понятное действие;
- цепочка `else if` стала длинной и хуже читается.

Частые ошибки

Присваивание вместо сравнения

```
// if priority = 3 { } // ошибка
if priority == 3 {
  fmt.Println("Высокий приоритет")
}
```

Неверный порядок проверок

```
title := ""

if len(title) <= 120 {
  fmt.Println("Длина подходит") // сработает и для пустой строки
} else if len(title) == 0 {
  fmt.Println("Пустое название")
}
```

Проверку пустой строки нужно поставить первой либо объединить условия:

```
if len(title) == 0 {
  fmt.Println("Пустое название")
} else if len(title) <= 120 {
  fmt.Println("Длина подходит")
}
```

Лишний `break` в `switch`

```
switch priority {
case 1:
  fmt.Println("Низкий")
  // break не нужен
}
```

Дальше

Закрепите условия на валидации задачи. После этого перейдём к циклам, чтобы работать с повторяющимися действиями и наборами значений.

Самопроверка

1. Когда выполняется блок if?

A. Всегда

B. Когда условие равно false

C. Когда условие равно true

D. Только после switch

2. Как в Go записывают условие if?

A. if priority > 0 { }

B. if priority > 0 end

C. if (priority > 0) { }

D. if priority > 0 then { }

3. Что произойдёт в цепочке if / else if / else, если истинны два условия?

A. Будет ошибка компиляции

B. Выполнится только первая подходящая ветвь

C. Выполнятся обе подходящие ветви

D. Всегда выполнится else

4. Что проверяет условие len(title) == 0?

A. Что строка пустая

B. Что строка содержит нулевой байт

C. Что title имеет тип int

D. Что строка длиннее 120 байтов

5. Где доступна переменная titleLength, объявленная в if titleLength := len(title); ...?

A. Во всём пакете

B. Только внутри main до if

C. В условии и его ветвях

D. Только после if

6. Когда switch удобнее длинной цепочки else if?

A. Только для логических значений

B. Когда одно значение сравнивают с несколькими конкретными вариантами

C. Когда нужен цикл

D. Только для строк

7. Нужно ли писать break после каждого case в обычном switch Go?

A. Только в default

B. Да, иначе выполнится следующий case

C. Только если case содержит fmt.Println

D. Нет, switch завершается после подходящей ветви

8. Какая проверка корректно определяет допустимый приоритет от 1 до 3?

A. priority >= 1 || priority <= 3

B. priority >= 1 && priority <= 3

C. priority == 1 && 3

D. priority = 1..3

9. Какое сообщение выведет switch с default, если ни один case не подошёл?

A. Первый case

B. Ошибка компиляции

C. Ничего, default никогда не выполняется

D. Инструкции из ветви default

10. Почему проверку пустого названия стоит разместить перед проверкой максимальной длины?

A. switch не поддерживает пустые строки

B. Пустую строку нельзя передать в len

C. Пустая строка тоже короче 120 байтов

D. Иначе Go не скомпилирует программу

Ответы к самопроверке

1. Когда условие равно true

2. if priority > 0 { }

3. Выполнится только первая подходящая ветвь

4. Что строка пустая

5. В условии и его ветвях

6. Когда одно значение сравнивают с несколькими конкретными вариантами

7. Нет, switch завершается после подходящей ветви

8. priority >= 1 && priority <= 3

9. Инструкции из ветви default

10. Пустая строка тоже короче 120 байтов

Задания: валидация задачи

Продолжим Task Manager API. Теперь программа не только вычислит проверки, но и сообщит пользователю первую найденную проблему.

Задание 1. Проверка заголовка

Создайте программу с переменной `title`. Она должна выводить:

- `Название задачи не должно быть пустым`, если заголовок пустой;
- `Название задачи слишком длинное`, если длина заголовка больше 120 байтов;
- `Название задачи корректно` во всех остальных случаях.

Сначала проверьте программу с `title := "Купить молоко"`, а затем с `title := ""`.

Критерии проверки:

- используется `if / else if / else`;
- проверка пустой строки находится раньше проверки максимальной длины;
- для корректного заголовка выводится сообщение об успехе;
- для пустого заголовка выводится сообщение об ошибке.

Подсказка

Пустая строка: `len(title) == 0`. Слишком длинная строка: `len(title) > 120`.

Решение

```
package main

import "fmt"

func main() {
  title := "Купить молоко"

  if len(title) == 0 {
    fmt.Println("Название задачи не должно быть пустым")
  } else if len(title) > 120 {
    fmt.Println("Название задачи слишком длинное")
  } else {
    fmt.Println("Название задачи корректно")
  }
}
```

Задание 2. Проверка приоритета

Добавьте переменную `priority`. Она может принимать значения от `1` до `3`. Выведите сообщение `Приоритет должен быть от 1 до 3`, если значение находится вне диапазона. В остальных случаях выведите `Приоритет корректен`.

Проверьте программу для значений `2`, `0` и `4`.

Критерии проверки:

- проверка использует `priority < 1 || priority > 3`;
- значение `2` считается корректным;
- значения `0` и `4` считаются некорректными.

Подсказка

Некорректный приоритет — это значение меньше 1 **или** больше 3.

Решение

```
package main

import "fmt"

func main() {
  priority := 2

  if priority < 1 || priority > 3 {
    fmt.Println("Приоритет должен быть от 1 до 3")
  } else {
    fmt.Println("Приоритет корректен")
  }
}
```

Задание 3. Полная валидация

Объедините проверки из предыдущих заданий. Программа должна вывести только **первую** ошибку в таком порядке:

1. пустой заголовок;
2. слишком длинный заголовок;
3. некорректный приоритет;
4. `Задачу можно создать`, если ошибок нет.

Критерии проверки:

- используется одна цепочка `if / else if / else`;
- за один запуск выводится только одно сообщение;
- с `title := ""` и `priority := 5` первой выводится ошибка заголовка;
- с корректным заголовком и `priority := 2` выводится сообщение об успехе.

Подсказка

Расположите проверки в той же последовательности, что и в условии задания. `else` должен содержать сообщение об успехе.

Решение

```
package main

import "fmt"

func main() {
  title := "Купить молоко"
  priority := 2

  if len(title) == 0 {
    fmt.Println("Название задачи не должно быть пустым")
  } else if len(title) > 120 {
    fmt.Println("Название задачи слишком длинное")
  } else if priority < 1 || priority > 3 {
    fmt.Println("Приоритет должен быть от 1 до 3")
  } else {
    fmt.Println("Задачу можно создать")
  }
}
```

Задание 4. Текстовое описание приоритета

Используйте `switch`, чтобы вывести текстовое описание значения `priority`:

- `1` — `Низкий приоритет`;
- `2` — `Средний приоритет`;
- `3` — `Высокий приоритет`;
- любое другое значение — `Неизвестный приоритет`.

Критерии проверки:

- используется `switch priority`;
- есть три `case` и `default`;
- после `case` нет лишних `break`;
- для `priority := 2` выводится `Средний приоритет`.

Подсказка

```
switch priority {
case 1:
  // ...
default:
  // ...
}
```

Решение

```
package main

import "fmt"

func main() {
  priority := 2

  switch priority {
  case 1:
    fmt.Println("Низкий приоритет")
  case 2:
    fmt.Println("Средний приоритет")
  case 3:
    fmt.Println("Высокий приоритет")
  default:
    fmt.Println("Неизвестный приоритет")
  }
}
```

Проверьте себя

Перед следующей главой убедитесь, что вы можете:

- написать `if / else if / else` без круглых скобок вокруг условия;
- выбрать правильный порядок проверок;
- использовать `||` для проверки выхода за диапазон;
- объяснить область видимости переменной из короткого объявления в `if`;
- выбрать между `if` и `switch`.

Рекомендации по коду: условия

- Проверяйте ошибочные и граничные случаи раньше успешного пути: это уменьшает вложенность.
- Используйте ранний `return`, если после ошибки дальнейшая работа не имеет смысла.
- Выбирайте `if` для диапазонов и составных условий, а `switch` — для одного значения с несколькими конкретными вариантами.
- Не сравнивайте `bool` с `true` или `false`, если достаточно `if completed` или `if !completed`.
- Не добавляйте `break` в обычный `switch`: Go завершает подходящий `case` сам.
- Короткое объявление в `if` используйте только для значения, нужного именно этому условию.

Чек-лист

Валидация сообщает первую понятную проблему, ветви расположены от специальных условий к общим, а `else` не содержит большой основной сценарий после ошибки.

Циклы: `for` и `range`

Цель урока

Научиться повторять действия с помощью `for`, управлять выполнением через `break` и `continue`, а также обходить строку по Unicode-символам через `range`.

В Go есть только один ключевой цикл — `for`. Его формы покрывают обычные счётчики, циклы с условием и бесконечное повторение.

Цикл с условием

Самая простая форма повторяет код, пока условие истинно:

```
package main

import "fmt"

func main() {
  taskNumber := 1

  for taskNumber <= 3 {
    fmt.Printf("Задача №%d\n", taskNumber)
    taskNumber++
  }
}
```

В этом примере:

1. `taskNumber` получает значение `1`;
2. проверяется условие `taskNumber <= 3`;
3. выполняется тело цикла;
4. `taskNumber++` увеличивает счётчик;
5. проверка повторяется.

Если забыть изменить значение, от которого зависит условие, цикл может стать бесконечным.

Классический `for`

Когда у цикла есть счётчик, обычно используют компактную форму из трёх частей:

```
for начальное_значение; условие; изменение {
  // повторяемый код
```

```
}
```

Например, выведем номера первых пяти задач:

```
package main

import "fmt"

func main() {
  for taskNumber := 1; taskNumber <= 5; taskNumber++ {
    fmt.Printf("Задача №%d\n", taskNumber)
  }
}
```

Область видимости `taskNumber` ограничена циклом: после него переменная недоступна.

Цикл можно вести в обратную сторону:

```
for remaining := 3; remaining > 0; remaining-- {
  fmt.Println("Осталось задач:", remaining)
}
```

Бесконечный цикл

`for` без условия выполняется бесконечно:

```
for {
  // код будет повторяться всегда
}
```

Эта форма нужна, например, для постоянно работающего сервера или чтения событий. В учебных примерах она должна иметь понятный способ остановки, обычно через `break`.

```
attempt := 1

for {
  fmt.Println("Попытка", attempt)

  if attempt == 3 {
    break
  }
```

```
  attempt++}
```

`break`: остановить цикл

`break` немедленно завершает ближайший цикл:

```
package main

import "fmt"

func main() {
  for taskNumber := 1; taskNumber <= 10; taskNumber++ {
    if taskNumber == 4 {
      break
    }

    fmt.Printf("Задача №%d\n", taskNumber)
  }
}
```

Программа выведет только задачи с первой по третью. Когда счётчик станет равен `4`, код после `break` в теле цикла не выполнится.

`continue`: пропустить текущий шаг

`continue` прекращает текущую итерацию и сразу переходит к следующей:

```
package main

import "fmt"

func main() {
  for taskNumber := 1; taskNumber <= 5; taskNumber++ {
    if taskNumber == 3 {
      continue
    }

    fmt.Printf("Обработать задачу №%d\n", taskNumber)
  }
}
```

Третья задача не будет выведена, но цикл продолжится с четвёртой.

`continue` полезен, когда известно, что текущий элемент нужно пропустить. Не используйте его, если условие можно проще выразить через обычный `if`: читаемость важнее сокращения нескольких строк.

`range` для строк

`range` позволяет обходить строку по рунам — Unicode-символам. Это важно для русскоязычных названий задач.

```
package main

import "fmt"

func main() {
  title := "Go!"

  for byteIndex, symbol := range title {
    fmt.Printf("Байт %d: %c\n", byteIndex, symbol)
  }
}
```

У `range` для строки:

- первое значение — индекс **в байтах**;
- второе значение — текущая руна.

На кириллице это особенно заметно:

```
package main

import "fmt"

func main() {
  title := "Го"

  for byteIndex, symbol := range title {
    fmt.Printf("Байт %d: %c\n", byteIndex, symbol)
  }
}
```

Вывод:

```
Байт 0: Г
Байт 2: о
```

Если индекс не нужен, замените его на `_`:

```
for _, symbol := range title {
  fmt.Println(string(symbol))
}
```

`_` называют пустым идентификатором: он явно показывает, что значение намеренно не используется.

> **Почему пока только строки**
>
> `range` также работает со слайсами и map, но эти типы появятся в следующих главах. Сначала закрепим сам принцип цикла на строках и счётчиках.

Частые ошибки

Бесконечный цикл из-за неизменяемого счётчика

```
taskNumber := 1

for taskNumber <= 3 {
  fmt.Println(taskNumber)
  // taskNumber++ забыли
}
```

Условие всегда остаётся истинным. Добавьте изменение счётчика или используйте классическую форму `for`.

Ошибка на единицу

```
for taskNumber := 1; taskNumber < 3; taskNumber++ {
  fmt.Println(taskNumber)
}
```

Этот код выведет только `1` и `2`. Если нужно включить третью задачу, условие должно быть `taskNumber <= 3`.

Ожидание индекса руны от `range`

```
for index, symbol := range "Го" {
  fmt.Println(index, string(symbol))
}
```

`index` будет равен `0`, затем `2`, потому что это индекс байта в UTF-8, а не порядковый номер символа.

Дальше

Закрепите циклы на нумерации задач и обходе названия. Затем перейдём к функциям, чтобы выносить повторяющуюся логику из `main`.

Самопроверка

1. Какой ключевой цикл есть в Go?

A. loop

B. for

C. repeat

D. while

2. Сколько раз выполнится цикл for number := 1; number <= 3; number++?

A. 2 раза

B. 3 раза

C. Бесконечно

D. 4 раза

3. Что делает break внутри цикла?

A. Пропускает только текущую итерацию

B. Перезапускает программу

C. Завершает ближайший цикл

D. Увеличивает счётчик

4. Что делает continue внутри цикла?

A. Переходит к следующей итерации

B. Возвращает значение из функции

C. Полностью завершает цикл

D. Останавливает программу

5. Как записать бесконечный цикл в Go?

A. while true { }

B. for { }

C. for true; { }

D. loop { }

6. Почему цикл for taskNumber := 1; taskNumber < 3; taskNumber++ не выведет номер 3?

A. В Go нельзя использовать число 3 в циклах

B. for всегда заканчивается на 2

C. Оператор ++ пропускает последнюю итерацию

D. Условие taskNumber < 3 ложно при значении 3

7. Что возвращает range при обходе строки?

A. Номер слова и строку

B. Индекс в байтах и текущую руну

C. Только число байтов

D. Всегда индекс символа и byte

8. Почему в range для строки "Го" индексы равны 0 и 2?

A. Go считает индексы с нуля и двух

B. Строка содержит только одну руну

C. Кириллические буквы занимают по два байта в UTF-8

D. range пропускает каждую вторую букву

9. Как обозначить неиспользуемый индекс в for range?

A. for _, symbol := range title

B. for *, symbol := range title

C. for null, symbol := range title

D. for skip, symbol := range title

10. Что произойдёт, если в цикле с условием не изменять счётчик?

A. Цикл может стать бесконечным

B. Компилятор всегда исправит код

C. Go автоматически увеличит счётчик

D. break добавится автоматически

Ответы к самопроверке

1. for

2. 3 раза

3. Завершает ближайший цикл

4. Переходит к следующей итерации

5. for { }

6. Условие taskNumber < 3 ложно при значении 3

7. Индекс в байтах и текущую руну

8. Кириллические буквы занимают по два байта в UTF-8

9. for _, symbol := range title

10. Цикл может стать бесконечным

Задания: нумерация и название задачи

Пока Task Manager API не хранит список задач, но мы уже можем смоделировать нумерацию и обработать отдельное название задачи.

Задание 1. Номера задач

Выведите номера задач от `1` до `5` в формате:

```
Задача №1
Задача №2
...
Задача №5
```

Используйте классическую форму `for`.

Критерии проверки:

- счётчик начинается с `1`;
- последняя выведенная задача имеет номер `5`;
- цикл содержит начальное значение, условие и изменение счётчика;
- номера выводятся по порядку.

Подсказка

Начало цикла:

```
for taskNumber := 1; taskNumber <= 5; taskNumber++ {
  // вывод
}
```

Решение

```
package main

import "fmt"

func main() {
  for taskNumber := 1; taskNumber <= 5; taskNumber++ {
    fmt.Printf("Задача №%d\n", taskNumber)
  }
}
```

Задание 2. Пропустите служебную задачу

Выведите номера задач от `1` до `5`, но пропустите задачу № `3`. Используйте `continue`.

Критерии проверки:

- в цикле есть `continue`;
- задача № `3` не выводится;
- задачи № `1`, `2`, `4` и `5` выводятся;
- цикл не завершается раньше времени.

Подсказка

Перед выводом проверьте `if taskNumber == 3`, а внутри ветви используйте `continue`.

Решение

```
package main

import "fmt"

func main() {
  for taskNumber := 1; taskNumber <= 5; taskNumber++ {
    if taskNumber == 3 {
      continue
    }

    fmt.Printf("Задача №%d\n", taskNumber)
  }
}
```

Задание 3. Остановите обработку

Программа обрабатывает задачи с номерами от `1` до `10`, но должна остановиться перед задачей № `6`. Используйте `break`.

Критерии проверки:

- в цикле есть `break`;
- выводятся номера с `1` по `5`;
- номера `6` и больше не выводятся;
- `break` находится до команды вывода.

Подсказка

Сравните номер с `6`. Если числа равны, завершите цикл до `fmt.Println`.

Решение

```
package main

import "fmt"

func main() {
  for taskNumber := 1; taskNumber <= 10; taskNumber++ {
    if taskNumber == 6 {
      break
    }

    fmt.Printf("Обработана задача №%d\n", taskNumber)
  }
}
```

Задание 4. Символы названия

Для названия задачи `Купить молоко` выведите каждую руну с новой строки. Не выводите байтовый индекс.

Критерии проверки:

- используется `for range`;
- индекс заменён на `_`;
- каждая буква, пробел и другие символы выводятся отдельной строкой;
- кириллические буквы отображаются корректно.

Подсказка

Используйте форму `for _, symbol := range title`. Для вывода руны как символа подойдёт `fmt.Println(string(symbol))`.

Решение

```
package main

import "fmt"

func main() {
  title := "Купить молоко"

  for _, symbol := range title {
    fmt.Println(string(symbol))
  }
}
```

Проверьте себя

Перед следующей главой убедитесь, что вы можете:

- написать цикл с условием и классический `for`;
- объяснить разницу между `break` и `continue`;
- избежать ошибки на единицу в условии цикла;
- объяснить, почему индекс `range` для строки — это индекс байта;
- использовать `_`, когда часть результата цикла не нужна.

Рекомендации по коду: циклы

- Предпочитайте простую форму `for range`, если не нужен ручной счётчик или особый шаг.
- Чётко определяйте границу цикла: `<=` включает последнее значение, `<` — нет.
- Используйте `break` для прекращения поиска, а `continue` — только когда это делает код понятнее обычного `if`.
- Не пишите бесконечный `for` без очевидного условия выхода, контекста отмены или читаемого `break`.
- При запуске goroutine в цикле передавайте значение параметром функции, чтобы намерение было явным.
- Помните, что индекс `range` для строки измеряется в байтах UTF-8.

Чек-лист

Каждый цикл имеет понятное условие завершения, а тесты или примеры покрывают пустой набор, один элемент и границу диапазона.

Функции

Цель урока

Научиться выносить повторяющуюся логику из `main` в именованные функции, передавать им данные и получать один или несколько результатов.

В Task Manager API функции помогут отделить проверку задачи и получение текста приоритета от запуска программы. Позже этот подход позволит построить понятную структуру приложения.

Зачем нужны функции

Функция — именованный блок кода, который выполняет одну понятную задачу. Она помогает:

- не повторять одинаковый код;
- дать действию понятное имя;
- проверить часть программы отдельно;
- сделать `main` коротким и читаемым.

Например, вместо нескольких одинаковых строк вывода можно создать функцию:

```
package main

import "fmt"

func printWelcome() {
  fmt.Println("Добро пожаловать в Task Manager")
}

func main() {
  printWelcome()
}
```

`func` объявляет функцию, `printWelcome` — её имя, а круглые скобки содержат параметры. В этом примере параметров и возвращаемого результата нет.

Параметры

Параметры передают функции значения, с которыми она должна работать. Имя параметра записывают перед его типом:

```
func printTaskTitle(title string) {
  fmt.Println("Задача:", title)
```

```
}
```

Вызов передаёт аргумент:

```
func main() {
  printTaskTitle("Купить молоко")
}
```

Одна функция может принимать несколько параметров одного типа. Их можно записать через запятую:

```
func printTask(title string, priority int) {
  fmt.Println("Задача:", title)
  fmt.Println("Приоритет:", priority)
}
```

Количество, порядок и типы аргументов при вызове должны совпадать с объявлением:

```
printTask("Купить молоко", 2)
```

В Go аргументы передаются в функцию по значению: параметр получает собственную копию значения. Работа с указателями и изменением сложных данных появится в следующих главах.

Один возвращаемый результат

После списка параметров можно указать тип результата. Функция возвращает значение через `return`:

```
func priorityLabel(priority int) string {
  if priority == 1 {
    return "низкий"
  }

  if priority == 2 {
    return "средний"
  }

  if priority == 3 {
    return "высокий"
  }
```

```
  return "неизвестный"}
```

Результат вызова можно сохранить в переменную:

```
label := priorityLabel(2)
fmt.Println(label) // средний
```

Функция с объявленным результатом должна вернуть значение на каждом возможном пути выполнения. В примере последний `return` обрабатывает все значения, кроме `1`, `2` и `3`.

Ранний `return`

`return` завершает выполнение функции сразу. Это удобно для валидации: сначала возвращаем результат при первой найденной проблеме, а успешный путь оставляем в конце.

```
func validateTitle(title string) bool {
  if len(title) == 0 {
    return false
  }

  if len(title) > 120 {
    return false
  }

  return true
}
```

Такую проверку можно сделать короче одним выражением:

```
func validateTitle(title string) bool {
  return len(title) > 0 && len(title) <= 120
}
```

Оба варианта корректны. Первый удобнее, если позже понадобится разные сообщения для разных ошибок.

Несколько результатов

Go-функция может вернуть несколько значений. Типы результата указывают в скобках:

```
func validatePriority(priority int) (bool, string) {
  if priority < 1 || priority > 3 {
```

```
  } return true, ""}
```

При вызове каждому результату соответствует своя переменная:

```
isValid, message := validatePriority(4)

fmt.Println(isValid) // false
fmt.Println(message) // приоритет должен быть от 1 до 3
```

Если один результат сейчас не нужен, используйте `_`:

```
isValid, _ := validatePriority(2)
fmt.Println(isValid) // true
```

Не заменяйте `_` обычной переменной, которую не будете использовать: Go выдаст ошибку о неиспользуемой переменной.

Именованные результаты

Результат можно назвать в объявлении функции:

```
func validatePriority(priority int) (isValid bool, message string) {
  if priority < 1 || priority > 3 {
    return false, "приоритет должен быть от 1 до 3"
  }

  return true, ""
}
```

Имена делают сигнатуру понятнее, но не обязательны. В начале лучше использовать обычные результаты и явно указывать значения в `return`. Именованные результаты пригодятся, когда делают API функции понятнее, а не просто сокращают код.

Область видимости

Параметры и переменные внутри функции существуют только в этой функции:

```
func printTaskTitle(title string) {
  formattedTitle := "Задача: " + title
  fmt.Println(formattedTitle)
}
```

```
  printTaskTitle("Купить молоко")  // fmt.Println(formattedTitle) // ошибка:
переменная существует только внутри функции}
```

Функция может обращаться к импортированным пакетам, например `fmt`, но не видит локальные переменные другой функции.

Соберём валидацию задачи

Теперь вынесем проверку Task Manager API в отдельные функции:

```
package main

import "fmt"

func validateTitle(title string) (bool, string) {
  if len(title) == 0 {
    return false, "название задачи не должно быть пустым"
  }

  if len(title) > 120 {
    return false, "название задачи слишком длинное"
  }

  return true, ""
}

func validatePriority(priority int) (bool, string) {
  if priority < 1 || priority > 3 {
    return false, "приоритет должен быть от 1 до 3"
  }

  return true, ""
}

func main() {
  title := "Купить молоко"
  priority := 2

  if isValid, message := validateTitle(title); !isValid {
    fmt.Println(message)
    return
  }

  if isValid, message := validatePriority(priority); !isValid {
```

```
    fmt.Println(message)    return } fmt.Println("Задачу можно создать")}
```

`main` теперь управляет последовательностью действий, а каждая функция отвечает только за свою проверку. Пока вместо типа `error` мы возвращаем `bool` и текст. Обработку ошибок в идиоматичном Go подробно разберём в отдельной главе.

Частые ошибки

Вызов до объявления — не проблема

В Go функцию можно вызвать до её текстового объявления в файле. Важнее, чтобы имя и сигнатура совпадали:

```
func main() {
  fmt.Println(priorityLabel(2))
}

func priorityLabel(priority int) string {
  return "средний"
}
```

Результат не возвращён

```
func priorityLabel(priority int) string {
  if priority == 1 {
    return "низкий"
  }

  // ошибка: missing return
}
```

Добавьте результат для всех оставшихся случаев.

Неверное число аргументов

```
func printTask(title string, priority int) {
  // ...
}

// printTask("Купить молоко") // ошибка: не передан priority
```

Путаница `return` и `fmt.Println`

`fmt.Println` выводит данные в терминал, а `return` отдаёт результат вызывающему коду. Функция может делать одно, другое или оба действия, но для проверяемой логики обычно лучше вернуть результат, а вывести его в `main`.

Дальше

Закрепите функции на проверке и описании задачи. Затем перейдём к слайсам и массивам, чтобы работать с наборами значений.

Самопроверка

1. Каким ключевым словом объявляют функцию в Go?

A. method

B. func

C. def

D. function

2. Что делает вызов printWelcome()?

A. Удаляет функцию printWelcome

B. Автоматически выводит имя функции

C. Выполняет код функции printWelcome

D. Создаёт новую функцию

3. Как правильно объявить функцию, принимающую название задачи?

A. func printTitle(title) string { }

B. func printTitle(title string) { }

C. func printTitle(string title) { }

D. function printTitle(title: string) { }

4. Как в Go передаются аргументы функции?

A. Всегда по ссылке

B. Только через глобальные переменные

C. Только через указатели

D. По значению

5. Как объявить функцию priorityLabel, возвращающую string?

A. func priorityLabel(priority int): string { }

B. func priorityLabel(priority int) returns string { }

C. string func priorityLabel(priority int) { }

D. func priorityLabel(priority int) string { }

6. Что происходит после выполнения return в функции?

A. Программа всегда завершается

B. Функция сразу завершает выполнение

C. Цикл внутри функции повторяется

D. Go автоматически вызывает следующую функцию

7. Как получить два результата validatePriority?

A. validatePriority(2) -> isValid, message

B. isValid, message := validatePriority(2)

C. isValid = validatePriority(2, message)

D. isValid := message := validatePriority(2)

8. Как обозначить результат функции, который сейчас не нужен?

A. _

B. void

C. null

D. skip

9. Где доступна переменная, объявленная внутри функции?

A. Во всех файлах проекта

B. Только внутри этой функции или вложенного блока

C. Только внутри main

D. Во всех функциях пакета

10. Почему полезно вернуть результат проверки, а не печатать его прямо в функции?

A. Функция с return не принимает параметры

B. return работает только с bool

C. Функцию проще повторно использовать и проверить

D. fmt.Println нельзя вызывать в функциях

Ответы к самопроверке

1. func

2. Выполняет код функции printWelcome

3. func printTitle(title string) { }

4. По значению

5. func priorityLabel(priority int) string { }

6. Функция сразу завершает выполнение

7. isValid, message := validatePriority(2)

8. _

9. Только внутри этой функции или вложенного блока

10. Функцию проще повторно использовать и проверить

Задания: функции Task Manager API

Вынесем логику проверки будущей задачи из `main`. Пока задача представлена отдельными переменными, но уже сейчас каждая функция будет отвечать за одну конкретную обязанность.

Задание 1. Проверка заголовка

Напишите функцию `validateTitle(title string) bool`. Она должна вернуть `true`, если заголовок не пустой и не длиннее 120 байтов, иначе — `false`.

В `main` проверьте названия `Купить молоко` и пустую строку.

Критерии проверки:

- функция принимает параметр `title string`;
- функция возвращает `bool`;
- проверка использует `len(title) > 0 && len(title) <= 120`;
- корректный заголовок даёт `true`, пустой — `false`.

Подсказка

Тип результата записывают после круглых скобок с параметрами:

```
func validateTitle(title string) bool {
  // return ...
}
```

Решение

```
package main

import "fmt"

func validateTitle(title string) bool {
  return len(title) > 0 && len(title) <= 120
}

func main() {
  fmt.Println(validateTitle("Купить молоко"))
  fmt.Println(validateTitle(""))
}
```

Задание 2. Описание приоритета

Создайте функцию `priorityLabel(priority int) string`. Она должна возвращать:

- `низкий` для `1`;
- `средний` для `2`;
- `высокий` для `3`;
- `неизвестный` для остальных значений.

Используйте `switch`.

Критерии проверки:

- функция принимает `int` и возвращает `string`;
- используется `switch priority`;
- предусмотрен `default`;
- `priorityLabel(2)` возвращает `средний`.

Подсказка

Каждая ветвь `case` может сразу вернуть результат:

```
case 1:
  return "низкий"
```

Решение

```
package main

import "fmt"

func priorityLabel(priority int) string {
  switch priority {
  case 1:
    return "низкий"
  case 2:
    return "средний"
  case 3:
    return "высокий"
  default:
    return "неизвестный"
  }
}

func main() {
  fmt.Println(priorityLabel(2))
}
```

Задание 3. Проверка приоритета с сообщением

Напишите функцию `validatePriority(priority int) (bool, string)`. Для значения от `1` до `3` она должна вернуть `true, ""`. Для остальных значений — `false` и сообщение `приоритет должен быть от 1 до 3`.

В `main` сохраните оба результата в переменные и выведите их.

Критерии проверки:

- функция возвращает два значения: `bool` и `string`;
- в случае ошибки возвращается понятный текст;
- `validatePriority(2)` возвращает `true, ""`;
- `validatePriority(4)` возвращает `false` и текст ошибки.

Подсказка

Сигнатура функции:

```
func validatePriority(priority int) (bool, string) {
  // ...
}
```

Решение

```
package main

import "fmt"

func validatePriority(priority int) (bool, string) {
  if priority < 1 || priority > 3 {
    return false, "приоритет должен быть от 1 до 3"
  }

  return true, ""
}

func main() {
  isValid, message := validatePriority(4)

  fmt.Println(isValid)
  fmt.Println(message)
}
```

Задание 4. Полная проверка задачи

Используйте `validateTitle` и `validatePriority` для проверки одной задачи. Если заголовок некорректен, выведите его сообщение и завершите `main`. Если заголовок корректен, проверьте приоритет. При успешной проверке выведите:

```
Задачу "Купить молоко" можно создать. Приоритет: средний
```

Критерии проверки:

- `main` вызывает обе функции в правильном порядке;
- при первой ошибке выполняется `return` из `main`;
- при успехе используется `priorityLabel`;
- логика проверки не дублируется в `main`.

Подсказка

Проверку можно записать так:

```
if isValid, message := validateTitle(title); !isValid {
  fmt.Println(message)
  return
}
```

Решение

```
package main

import "fmt"

func validateTitle(title string) (bool, string) {
  if len(title) == 0 {
    return false, "название задачи не должно быть пустым"
  }

  if len(title) > 120 {
    return false, "название задачи слишком длинное"
  }

  return true, ""
}

func validatePriority(priority int) (bool, string) {
  if priority < 1 || priority > 3 {
    return false, "приоритет должен быть от 1 до 3"
  }

  return true, ""
}

func priorityLabel(priority int) string {
  switch priority {
  case 1:
    return "низкий"
  case 2:
    return "средний"
  case 3:
    return "высокий"
```

▼ Сведения

```
  default:   return "неизвестный"  }}func main() {   title := "Купить молоко"
priority := 2   if isValid, message := validateTitle(title); !isValid {
fmt.Println(message)    return } if isValid, message := validatePriority(priority);
!isValid {   fmt.Println(message)    return } fmt.Printf("Задачу %q можно создать.
Приоритет: %s\n", title, priorityLabel(priority))}
```

Проверьте себя

Перед следующей главой убедитесь, что вы можете:

- объявить и вызвать функцию;
- передать параметры нужных типов;
- вернуть один или несколько результатов;
- завершить функцию ранним `return`;
- объяснить, почему проверку задачи лучше вынести из `main`.

Рекомендации по коду: функции

- Пусть функция выполняет одну понятную задачу и имеет имя-глагол: `validateTitle`, `loadTasks`, `saveTasks`.
- Передавайте данные параметрами, а не через глобальные переменные.
- Возвращайте значение вместо `fmt.Println`, если логика должна быть переиспользуема или тестируема.
- Проверяйте ошибку сразу после вызова и используйте ранний `return`.
- Не используйте именованные результаты только ради сокращения кода; они полезны, когда действительно делают сигнатуру понятнее.
- Если функция возвращает несколько значений, используйте `_` только для результата, который осознанно не нужен.

Чек-лист

`main` управляет сценарием, а проверка, форматирование и преобразование данных вынесены в небольшие функции с понятными контрактами.

Массивы и слайсы

Цель урока

Научиться хранить несколько значений одного типа, различать массив и слайс, добавлять и удалять элементы, а также безопасно работать с индексами.

В Task Manager API первым набором данных станет список названий задач. Пока это `[]string`; позже мы заменим строки полноценными структурами задач.

Зачем нужны коллекции

До сих пор одна переменная хранила одно значение:

```
title := "Купить молоко"
```

Но приложению обычно нужно работать с несколькими задачами. Можно объявить много переменных, но это неудобно:

```
firstTitle := "Купить молоко"
secondTitle := "Оплатить интернет"
thirdTitle := "Прочитать книгу"
```

Для набора однотипных значений используют массивы и слайсы.

Массив

Массив хранит фиксированное число элементов одного типа. Размер указан в типе:

```
var priorities [3]int
```

В этом массиве ровно три целых числа. Его начальные значения — нулевые:

```
fmt.Println(priorities) // [0 0 0]
```

Массив можно заполнить сразу:

```
priorities := [3]int{1, 2, 3}
titles := [2]string{"Купить молоко", "Оплатить интернет"}
```

Размер входит в тип массива. `[2]string` и `[3]string` — разные типы, даже если оба содержат строки.

```
var shortList [2]string
var fullList [3]string

// shortList = fullList // ошибка: разные типы
```

Массивы полезны, когда размер действительно фиксирован. В прикладном Go чаще нужны слайсы, потому что количество задач обычно меняется.

Индексы

Элементы массива и слайса нумеруются с нуля:

```
titles := [3]string{"Купить молоко", "Оплатить интернет", "Прочитать книгу"}

fmt.Println(titles[0]) // Купить молоко
fmt.Println(titles[2]) // Прочитать книгу
```

Доступ к индексу за границей вызывает `panic` во время работы программы:

```
// fmt.Println(titles[3]) // panic: index out of range
```

Длина коллекции получается функцией `len`:

```
fmt.Println(len(titles)) // 3
```

Последний допустимый индекс — `len(titles) - 1`. Перед доступом по индексу из переменной всегда проверяйте границы:

```
index := 1

if index >= 0 && index < len(titles) {
  fmt.Println(titles[index])
}
```

Слайс

Слайс — последовательность значений, длину которой можно менять. Его тип записывают без размера в квадратных скобках:

```
var titles []string
```

Нулевое значение слайса — `nil`. Читать его длину и обходить в цикле безопасно:

```
var titles []string

fmt.Println(titles == nil) // true
fmt.Println(len(titles))   // 0
```

Создать непустой слайс можно литералом:

```
titles := []string{
  "Купить молоко",
  "Оплатить интернет",
  "Прочитать книгу",
}
```

В отличие от массива, количество элементов не является частью типа:

```
shortList := []string{"Первая задача"}
fullList := []string{"Первая задача", "Вторая задача"}

shortList = fullList // допустимо: оба значения имеют тип []string
```

`append`: добавить элемент

`append` добавляет один или несколько элементов в слайс и возвращает обновлённый слайс.

```
titles := []string{"Купить молоко"}

titles = append(titles, "Оплатить интернет")
titles = append(titles, "Прочитать книгу", "Позвонить врачу")

fmt.Println(titles)
```

Важно присваивать результат обратно. `append` может создать новый внутренний массив, поэтому старый заголовок слайса не всегда меняется сам:

```
titles := []string{"Купить молоко"}
```

```
// append(titles, "Оплатить интернет") // результат потерянtitles = append(titles,
"Оплатить интернет")
```

`append` работает и с `nil`-слайсом:

```
var titles []string
titles = append(titles, "Купить молоко")

fmt.Println(titles) // [Купить молоко]
```

`len` и `cap`

`len` — количество доступных элементов. `cap` — вместимость слайса до выделения нового внутреннего массива.

```
titles := make([]string, 2, 5)

fmt.Println(len(titles)) // 2
fmt.Println(cap(titles)) // 5
```

В начале в слайсе уже есть два элемента с нулевым значением `""`. Третий аргумент `make` необязателен:

```
titles := make([]string, 0, 5)

fmt.Println(len(titles)) // 0
fmt.Println(cap(titles)) // 5
```

Такой вариант полезен, если вы заранее знаете примерное число элементов и собираетесь добавлять их через `append`.

На первых этапах не пытайтесь вручную управлять вместимостью без необходимости. Обычно достаточно литерала и `append`.

Срез слайса

Из части слайса можно получить другой слайс выражением `from:to`. Начальная граница включается, конечная — нет:

```
titles := []string{
  "Купить молоко",
```

```
}importantTitles := titles[0:2]fmt.Println(importantTitles) // [Купить молоко
Оплатить интернет]
```

Сокращённые формы:

```
firstTwo := titles[:2] // от начала до индекса 2, не включая его
afterFirst := titles[1:] // от индекса 1 до конца
allTitles := titles[:] // все элементы
```

Границы должны находиться в диапазоне от `0` до `len(titles)`. В противном случае будет `panic`.

Удаление элемента по индексу

В Go нет встроенной команды «удалить элемент слайса». Обычно объединяют часть до элемента и часть после него:

```
titles := []string{
  "Купить молоко",
  "Оплатить интернет",
  "Прочитать книгу",
}

index := 1
titles = append(titles[:index], titles[index+1:]...)

fmt.Println(titles) // [Купить молоко Прочитать книгу]
```

Перед этим обязательно проверяйте индекс:

```
if index >= 0 && index < len(titles) {
  titles = append(titles[:index], titles[index+1:]...)
}
```

Три точки `...` передают элементы второго слайса в `append` по отдельности.

`copy`: независимая копия

Простое присваивание не создаёт независимую копию элементов:

```
titles := []string{"Купить молоко", "Оплатить интернет"}
backup := titles
```

```
backup[0] = "Изменённая задача"fmt.Println(titles[0]) // Изменённая задача
```

Если нужна отдельная копия, создайте слайс нужной длины и используйте `copy`:

```
titles := []string{"Купить молоко", "Оплатить интернет"}
backup := make([]string, len(titles))

copy(backup, titles)
backup[0] = "Изменённая задача"

fmt.Println(titles[0]) // Купить молоко
fmt.Println(backup[0]) // Изменённая задача
```

`copy` возвращает число скопированных элементов. Оно равно меньшей из длин исходного и целевого слайсов.

Обход слайса через `range`

Слайс удобно обходить циклом `range`:

```
titles := []string{"Купить молоко", "Оплатить интернет"}

for index, title := range titles {
  fmt.Printf("%d. %s\n", index+1, title)
}
```

Здесь `index` начинается с нуля, поэтому для пользовательской нумерации добавлено `1`.

Если индекс не нужен, используйте `_`:

```
for _, title := range titles {
  fmt.Println(title)
}
```

Частые ошибки

Использование `append` без присваивания

```
titles := []string{"Купить молоко"}
append(titles, "Оплатить интернет")

fmt.Println(titles) // [Купить молоко]
```

Сохраните возвращаемое значение: `titles = append(titles, "Оплатить интернет")`.

Индекс вне границ

```
titles := []string{"Купить молоко"}
// fmt.Println(titles[1]) // panic
```

У одного элемента допустим только индекс `0`.

`make` с ненулевой длиной вместо пустого списка

```
titles := make([]string, 3)
titles = append(titles, "Купить молоко")

fmt.Println(titles) // [   Купить молоко]
```

Если нужны три места для будущих добавлений, используйте `make([]string, 0, 3)`, а не `make([]string, 3)`.

Изменение копии range-значения

В `for _, title := range titles` переменная `title` — копия текущего значения. Изменение её не изменит элемент слайса. Для изменения элемента нужен индекс; эту тему подробнее разберём на практических задачах после структур.

Дальше

Закрепите работу со списком названий задач. Затем перейдём к map — коллекции, которая хранит данные по ключу.

Самопроверка

1. Чем массив отличается от слайса?

A. Массив хранит только числа, а слайс — только строки

B. Размер массива является частью его типа, а длина слайса может меняться

C. Слайс всегда содержит ровно три элемента

D. Массив нельзя обходить в цикле

2. Какой тип у литерала []string{"Купить молоко"}?

A. Слайс строк

B. Одна строка

C. Массив строк с одним элементом

D. Map строк

3. Какой индекс имеет первый элемент слайса?

A. -1

B. 1

C. Зависит от длины слайса

D. 0

4. Что возвращает len(titles)?

A. Последний индекс

B. Вместимость слайса

C. Размер одного элемента в байтах

D. Количество элементов слайса

5. Что нужно сделать с результатом append?

A. Присвоить его обратно слайсу

B. Передать в len

C. Ничего: append всегда меняет переменную сам

D. Удалить через delete

6. Как создать пустой слайс строк с вместимостью 5?

A. [5]string{}

B. make([]string, 5)

C. make(string, 0, 5)

D. make([]string, 0, 5)

7. Что означает titles[1:3]?

A. Элементы с индексами 1, 2 и 3

B. Только элемент с индексом 3

C. Удаление элементов 1 и 3

D. Элементы с индексами 1 и 2

8. Как безопасно проверить, существует ли index в слайсе titles?

A. titles[index] != nil

B. index <= len(titles)

C. index >= 0 && index < len(titles)

D. index > 0 && index <= cap(titles)

9. Что произойдёт при backup := titles; backup[0] = "Новая задача"?

A. Первый элемент titles тоже изменится

B. Изменится только длина titles

C. Создастся независимая копия

D. Будет ошибка компиляции

10. Какой результат даёт range по слайсу?

A. Индекс и значение элемента

B. Только вместимость и длину

C. Только число элементов

D. Ключ и значение map

Ответы к самопроверке

1. Размер массива является частью его типа, а длина слайса может меняться

2. Слайс строк

3. 0

4. Количество элементов слайса

5. Присвоить его обратно слайсу

6. make([]string, 0, 5)

7. Элементы с индексами 1 и 2

8. index >= 0 && index < len(titles)

9. Первый элемент titles тоже изменится

10. Индекс и значение элемента

Задания: список названий задач

Теперь Task Manager API умеет хранить несколько названий задач в слайсе. Мы пока работаем только со строками; полноценная задача со свойствами появится после изучения структур.

Задание 1. Выведите список

Создайте слайс из трёх названий задач и выведите их с нумерацией начиная с единицы:

```
1. Купить молоко
2. Оплатить интернет
3. Прочитать книгу
```

Используйте `range`.

Критерии проверки:

- используется литерал `[]string`;
- в `range` получаются индекс и название;
- к индексу прибавляется `1`;
- каждая задача выводится с новой строки.

Подсказка

Форматированный вывод:

```
fmt.Printf("%d. %s\n", index+1, title)
```

Решение

```
package main

import "fmt"

func main() {
  titles := []string{
    "Купить молоко",
    "Оплатить интернет",
    "Прочитать книгу",
  }

  for index, title := range titles {
    fmt.Printf("%d. %s\n", index+1, title)
  }
}
```

Задание 2. Добавьте задачу

Создайте пустой слайс названий с вместимостью `3`, затем добавьте в него три задачи через `append`. После каждого добавления не выводите промежуточный результат: выведите готовый список через `range`.

Критерии проверки:

- слайс создан через `make([]string, 0, 3)`;
- каждый вызов `append` присваивается обратно переменной;
- в готовом списке три названия;
- порядок добавления сохранён.

Подсказка

```
titles = append(titles, "Купить молоко")
```

Решение

```
package main

import "fmt"

func main() {
  titles := make([]string, 0, 3)

  titles = append(titles, "Купить молоко")
  titles = append(titles, "Оплатить интернет")
  titles = append(titles, "Прочитать книгу")

  for index, title := range titles {
    fmt.Printf("%d. %s\n", index+1, title)
  }
}
```

Задание 3. Удалите задачу по индексу

В списке из трёх задач удалите задачу с индексом `1` — вторую по счёту. Перед удалением проверьте, что индекс допустим. Выведите обновлённый список.

Критерии проверки:

- проверяется условие `index >= 0 && index < len(titles)`;
- используется `append(titles[:index], titles[index+1:]...)`;
- в результате остаются первая и третья задачи;
- программа не паникует при недопустимом индексе.

Подсказка

Слайс до элемента: `titles[:index]`. Слайс после элемента: `titles[index+1:]`.

Решение

```
package main

import "fmt"

func main() {
  titles := []string{
    "Купить молоко",
    "Оплатить интернет",
    "Прочитать книгу",
  }
  index := 1

  if index >= 0 && index < len(titles) {
    titles = append(titles[:index], titles[index+1:]...)
  }

  for itemIndex, title := range titles {
    fmt.Printf("%d. %s\n", itemIndex+1, title)
  }
}
```

Задание 4. Сделайте резервную копию

Создайте список из двух задач и его независимую резервную копию. Измените первый элемент резервной копии на `Отменённая задача`, затем выведите оба списка.

Критерии проверки:

- резервная копия создана через `make` и `copy`;
- изменение резервной копии не меняет исходный слайс;
- первый элемент исходного списка остаётся прежним;
- первый элемент копии меняется.

Подсказка

Создайте `backup := make([]string, len(titles))`, затем вызовите `copy(backup, titles)`.

Решение

```
package main

import "fmt"

func main() {
  titles := []string{"Купить молоко", "Оплатить интернет"}
  backup := make([]string, len(titles))

  copy(backup, titles)
  backup[0] = "Отменённая задача"

  fmt.Println("Исходный список:", titles)
  fmt.Println("Резервная копия:", backup)
}
```

Проверьте себя

Перед следующей главой убедитесь, что вы можете:

- объяснить разницу массива и слайса;
- добавить элемент через `append` без потери результата;
- проверить индекс до доступа к элементу;
- получить часть слайса выражением `from:to`;
- удалить элемент по индексу;
- создать независимую копию через `copy`.

Рекомендации по коду: массивы и слайсы

- В прикладном коде по умолчанию выбирайте слайс; массив используйте, только если фиксированный размер — часть модели.
- Всегда сохраняйте результат `append`: `items = append(items, value)`.
- Проверяйте индекс до доступа, удаления или среза, если он приходит из переменной или внешнего ввода.
- Создавайте `make([]T, 0, capacity)`, когда планируете добавлять элементы через `append`.
- Для независимой копии используйте `copy`, а не обычное присваивание слайса.
- Не удерживайте маленьким слайсом большой исходный массив без причины; при долгом хранении делайте копию нужной части.

Чек-лист

Код учитывает пустой слайс, результат `append` не теряется, а изменение копии не должно неожиданно менять исходные данные.

Map: ключ и значение

Цель урока

Научиться хранить значения по уникальному ключу, создавать и изменять map, безопасно искать элементы, удалять их и обходить все пары.

В Task Manager API map позволит обращаться к названию задачи по её ID. Например, по ключу `42` можно получить значение `"Купить молоко"`.

Что такое map

`map` хранит пары **ключ → значение**. Ключ уникален: одному ключу соответствует одно текущее значение.

```
taskTitles := map[int]string{
  1: "Купить молоко",
  2: "Оплатить интернет",
  3: "Прочитать книгу",
}
```

В этом примере:

- `int` — тип ключа, ID задачи;
- `string` — тип значения, название задачи;
- `1`, `2`, `3` — ключи;
- тексты задач — значения.

Общий вид типа:

```
map[типКлюча]типЗначения
```

Map подходит для поиска по идентификатору, кода статуса, имени настройки или другого уникального признака. Если важен порядок элементов, чаще нужен слайс.

Создание map

Map можно создать литералом:

```
priorityLabels := map[int]string{
  1: "низкий",
  2: "средний",
```

```
  3: "высокий",}
```

Пустую map создают через `make`:

```
taskTitles := make(map[int]string)
```

После этого в неё можно записывать значения:

```
taskTitles[1] = "Купить молоко"
taskTitles[2] = "Оплатить интернет"
```

Можно указать примерное число элементов — это необязательная подсказка для рантайма:

```
taskTitles := make(map[int]string, 10)
```

Чтение по ключу

Чтобы получить значение, укажите ключ в квадратных скобках:

```
taskTitles := map[int]string{
  1: "Купить молоко",
}

title := taskTitles[1]
fmt.Println(title) // Купить молоко
```

Если ключа нет, map вернёт нулевое значение типа значения:

```
title := taskTitles[999]
fmt.Printf("%q\n", title) // ""
```

Это не всегда достаточно для проверки: пустая строка может быть и настоящим значением. Поэтому при поиске обычно используют форму `value, ok`.

Проверка наличия ключа: `value, ok`

Во второй переменной Go возвращает `true`, если ключ существует, и `false`, если его нет:

```
taskTitles := map[int]string{
  1: "Купить молоко",
}
```

```
title, exists := taskTitles[1]fmt.Println(title, exists) // Купить молоко truetitle,
exists = taskTitles[999]fmt.Println(title, exists) //  false
```

Так можно безопасно найти задачу по ID:

```
taskID := 2

if title, exists := taskTitles[taskID]; exists {
  fmt.Println("Найдена задача:", title)
} else {
  fmt.Println("Задача не найдена")
}
```

Если само значение не нужно, используйте `_`:

```
if _, exists := taskTitles[taskID]; exists {
  fmt.Println("ID уже занят")
}
```

Добавление и обновление

Запись по новому ключу добавляет пару. Запись по существующему ключу заменяет значение:

```
taskTitles := make(map[int]string)

taskTitles[1] = "Купить молоко" // добавление
taskTitles[1] = "Купить продукты" // обновление

fmt.Println(taskTitles[1]) // Купить продукты
```

Map не хранит историю изменений ключа: после обновления старое значение недоступно, если вы не сохранили его отдельно.

Удаление

Для удаления пары используйте встроенную функцию `delete`:

```
taskTitles := map[int]string{
  1: "Купить молоко",
  2: "Оплатить интернет",
}
```

```
delete(taskTitles, 1)fmt.Println(taskTitles) // map[2:Оплатить интернет]
```

Удаление отсутствующего ключа безопасно: ничего не произойдёт.

```
delete(taskTitles, 999)
```

`delete` не возвращает результат. Если важно сообщить пользователю, была ли задача удалена, сначала проверьте наличие ключа через `value, ok`.

Длина map

`len` возвращает число пар:

```
taskTitles := map[int]string{
  1: "Купить молоко",
  2: "Оплатить интернет",
}

fmt.Println(len(taskTitles)) // 2
```

После добавления длина увеличится, после удаления — уменьшится.

Обход через `range`

`range` для map возвращает ключ и значение:

```
taskTitles := map[int]string{
  1: "Купить молоко",
  2: "Оплатить интернет",
}

for id, title := range taskTitles {
  fmt.Printf("%d. %s\n", id, title)
}
```

> **Порядок map не определён**
>
> Не рассчитывайте на порядок обхода map. Один запуск может вывести сначала ID `1`, затем `2`, а другой — в обратном порядке. Если пользователю нужен отсортированный список, позже мы соберём ключи в слайс и отсортируем его.

Если нужен только ключ или только значение:

```
for id := range taskTitles {
  fmt.Println(id)
}

for _, title := range taskTitles {
  fmt.Println(title)
}
```

Ключи map

Ключ должен быть значением, которое можно сравнить через `==`. Для начала используйте простые типы:

- `int` для ID;
- `string` для имён, кодов и настроек;
- `bool` в редких случаях.

Слайс нельзя использовать ключом:

```
// invalid := map[[]string]int{} // ошибка компиляции
```

Причину и более сложные правила сравнения типов разберём позднее. Пока выбирайте для ключа ID или строковый код.

Полная работа с задачей по ID

```
package main

import "fmt"

func main() {
  taskTitles := map[int]string{
    1: "Купить молоко",
    2: "Оплатить интернет",
  }

  taskTitles[3] = "Прочитать книгу"
  taskTitles[2] = "Оплатить коммунальные услуги"

  taskID := 2
  if title, exists := taskTitles[taskID]; exists {
    fmt.Printf("Задача %d: %s\n", taskID, title)
  } else {
```

```
    fmt.Println("Задача не найдена") } delete(taskTitles, 1) fmt.Println("Всего задач:", len(taskTitles))}
```

Частые ошибки

Попытка определить отсутствие ключа через нулевое значение

```
title := taskTitles[42]

if title == "" {
  // Неясно: ключа нет или в map хранится пустая строка?
}
```

Используйте `title, exists := taskTitles[42]`.

Ожидание определённого порядка

```
for id, title := range taskTitles {
  fmt.Println(id, title)
}
```

Не используйте такой обход, если первая выведенная задача должна быть «самой ранней» или «с наименьшим ID».

Запись в nil map

```
var taskTitles map[int]string
// taskTitles[1] = "Купить молоко" // panic: assignment to entry in nil map
```

Создайте map через `make` или литерал до записи:

```
taskTitles := make(map[int]string)
taskTitles[1] = "Купить молоко"
```

Дальше

Закрепите операции над задачами по ID. Затем перейдём к структурам: вместо одного названия задача получит несколько связанных полей.

Самопроверка

1. Для какой задачи лучше всего подходит map[int]string?

A. Хранить только одно значение

B. Хранить порядок символов в строке

C. Хранить элементы всегда в порядке добавления

D. Хранить название задачи по её числовому ID

2. Как создать пустую map с ключами int и строковыми значениями?

A. map(int, string)

B. make([]int, string)

C. make(map[int]string)

D. new(map[int]string, 0)

3. Как получить название задачи с ID 2?

A. taskTitles[2]

B. taskTitles.get(2)

C. taskTitles(2)

D. taskTitles.id(2)

4. Что вернёт map[int]string при чтении отсутствующего ключа без проверки?

A. Panic

B. Пустую строку — нулевое значение string

C. Ошибку компиляции

D. nil

5. Что означает второй результат exists в title, exists := taskTitles[id]?

A. Существует ли ключ в map

B. Количество задач

C. Можно ли удалить map

D. Является ли значение строкой

6. Что произойдёт при taskTitles[1] = "Новое название", если ключ 1 уже есть?

A. Значение для ключа 1 обновится

B. Будет ошибка компиляции

C. Добавится второй элемент с ключом 1

D. Map станет nil

7. Как удалить элемент с ключом id?

A. delete(taskTitles, id)

B. taskTitles[id] = nil

C. remove(taskTitles[id])

D. taskTitles.remove(id)

8. Можно ли полагаться на порядок обхода map через range?

A. Только для map[int]string

B. Да, это порядок добавления

C. Нет, порядок не определён

D. Да, ключи всегда идут по возрастанию

9. Какой тип нельзя использовать ключом map?

A. bool

B. int

C. string

D. []string

10. Почему запись в var taskTitles map[int]string вызывает panic?

A. В map нельзя добавлять строки

B. У map не может быть ключей int

C. Для записи нужно использовать copy

D. Map имеет значение nil и не была создана

Ответы к самопроверке

1. Хранить название задачи по её числовому ID

2. make(map[int]string)

3. taskTitles[2]

4. Пустую строку — нулевое значение string

5. Существует ли ключ в map

6. Значение для ключа 1 обновится

7. delete(taskTitles, id)

8. Нет, порядок не определён

9. []string

10. Map имеет значение nil и не была создана

Задания: задачи по ID

Теперь Task Manager API хранит названия задач по числовым идентификаторам. Map не гарантирует порядок задач, но позволяет быстро найти, обновить или удалить задачу по её ID.

Задание 1. Найдите задачу

Создайте map с тремя задачами: ID `1`, `2` и `3`. Затем создайте переменную `taskID := 2` и выведите название найденной задачи в формате:

```
Задача 2: Оплатить интернет
```

Если задачи нет, программа должна вывести `Задача не найдена`.

Критерии проверки:

- используется `map[int]string`;
- поиск выполняется через `title, exists := taskTitles[taskID]`;
- для ID `2` выводится название задачи;
- для отсутствующего ID программа не паникует.

Подсказка

Используйте короткое объявление внутри `if`:

```
if title, exists := taskTitles[taskID]; exists {
  // задача найдена
}
```

Решение

```
package main

import "fmt"

func main() {
  taskTitles := map[int]string{
    1: "Купить молоко",
    2: "Оплатить интернет",
    3: "Прочитать книгу",
  }
  taskID := 2

  if title, exists := taskTitles[taskID]; exists {
    fmt.Printf("Задача %d: %s\n", taskID, title)
  } else {
    fmt.Println("Задача не найдена")
  }
}
```

Задание 2. Добавьте и обновите задачу

Создайте пустую map, добавьте задачи с ID `1` и `2`, затем измените название задачи с ID `2` на `Оплатить коммунальные услуги`. Выведите обновлённое значение задачи с ID `2`.

Критерии проверки:

- map создана через `make(map[int]string)`;
- новые ключи добавлены через присваивание;
- повторное присваивание по ключу `2` обновляет значение;
- результат поиска по ID `2` содержит новое название.

Подсказка

Для map одно и то же выражение используется для добавления и обновления:

```
taskTitles[2] = "Новое название"
```

Решение

```
package main

import "fmt"

func main() {
  taskTitles := make(map[int]string)

  taskTitles[1] = "Купить молоко"
  taskTitles[2] = "Оплатить интернет"
  taskTitles[2] = "Оплатить коммунальные услуги"

  fmt.Println(taskTitles[2])
}
```

Задание 3. Удалите задачу

Создайте map из трёх задач и удалите задачу с ID `2`. До удаления проверьте, что такая задача существует. После удаления выведите количество оставшихся задач и попытайтесь найти удалённую задачу.

Критерии проверки:

- существование ключа проверяется через `_, exists`;
- удаление выполняется через `delete`;
- после удаления `len(taskTitles)` равно `2`;
- поиск ID `2` сообщает, что задача не найдена.

Подсказка

```
if _, exists := taskTitles[taskID]; exists {
  delete(taskTitles, taskID)
}
```

Решение

```
package main

import "fmt"

func main() {
  taskTitles := map[int]string{
    1: "Купить молоко",
    2: "Оплатить интернет",
    3: "Прочитать книгу",
  }
  taskID := 2

  if _, exists := taskTitles[taskID]; exists {
    delete(taskTitles, taskID)
  }

  fmt.Println("Всего задач:", len(taskTitles))

  if title, exists := taskTitles[taskID]; exists {
    fmt.Println(title)
  } else {
    fmt.Println("Задача не найдена")
  }
}
```

Задание 4. Выведите все задачи

Создайте map из трёх задач и выведите все пары в формате `ID: название`. Не требуйте от программы конкретный порядок строк.

Критерии проверки:

- используется `for id, title := range taskTitles`;
- каждая пара содержит ID и название;
- в проверке результата не предполагается порядок обхода map;
- программа выводит все три задачи.

Подсказка

```
for id, title := range taskTitles {
  fmt.Printf("%d: %s\n", id, title)
}
```

Решение

```
package main

import "fmt"

func main() {
  taskTitles := map[int]string{
    1: "Купить молоко",
    2: "Оплатить интернет",
    3: "Прочитать книгу",
  }

  for id, title := range taskTitles {
    fmt.Printf("%d: %s\n", id, title)
  }
}
```

Проверьте себя

Перед следующей главой убедитесь, что вы можете:

- создать map литералом и через `make`;
- безопасно найти значение через `value, ok`;
- добавить, обновить и удалить пару;
- объяснить, почему map не подходит для упорядоченного списка;
- обойти ключи и значения через `range`.

Рекомендации по коду: map

- Используйте map для поиска по уникальному ключу, а не для упорядоченного списка.
- Проверяйте наличие ключа через `value, ok`, если нулевое значение может быть настоящими данными.
- Инициализируйте map через литерал или `make` до первой записи.
- Не полагайтесь на порядок `range` по map; сортируйте ключи отдельно, если порядок нужен пользователю.

- Удаление отсутствующего ключа безопасно, но при необходимости сообщить результат сначала проверьте `ok`.
- Не изменяйте map одновременно из нескольких goroutine без синхронизации.

Чек-лист

Ключ соответствует смыслу данных, чтение отсутствующей записи обработано, а API не обещает порядок элементов там, где map его не гарантирует.

Структуры: модель задачи

Цель урока

Научиться объединять связанные данные в собственный тип, создавать и изменять структуры, а также хранить задачи в слайсе и map.

До этого Task Manager API хранил название, приоритет и признак выполнения в отдельных переменных. Но эти значения относятся к одной сущности — задаче. Структура поможет хранить их вместе.

Что такое структура

Структура (`struct`) — тип с именованными полями разных типов. Сначала объявим тип `Task`:

```
type Task struct {
  ID        int
  Title     string
  Priority  int
  Completed bool
}
```

Теперь `Task` — собственный тип. Он описывает одну задачу с четырьмя полями:

Поле	Тип	Назначение
`ID`	`int`	Уникальный идентификатор задачи
`Title`	`string`	Название задачи
`Priority`	`int`	Уровень приоритета от 1 до 3
`Completed`	`bool`	Признак выполнения

Названия полей начинаются с заглавной буквы. Позже, при изучении пакетов, мы узнаем, что такие поля доступны коду из других пакетов. Внутри текущего `main` это не влияет на работу, но это распространённый стиль для моделей, которые в будущем будут участвовать в API.

Создание структуры

Структуру можно создать литералом с именами полей:

```
task := Task{
  ID:        1,
  Title:     "Купить молоко",
  Priority:  2,
```

```
  Completed: false,}
```

Имена полей делают код понятным и позволяют не зависеть от их порядка. Это рекомендуемый способ создания структуры.

Можно указать не все поля. Остальные получат нулевые значения:

```
task := Task{
  ID:    1,
  Title: "Купить молоко",
}

fmt.Println(task.Priority)  // 0
fmt.Println(task.Completed) // false
```

Можно создать нулевое значение всей структуры:

```
var task Task

fmt.Println(task) // {0  0 false}
```

Нулевое значение структуры уже существует и безопасно для чтения её полей. Однако нулевые значения не всегда соответствуют правилам приложения: например, задача с пустым названием и ID `0` не должна считаться корректной.

Доступ и изменение полей

Для чтения и изменения поля используйте точку:

```
task := Task{
  ID:        1,
  Title:     "Купить молоко",
  Priority:  2,
  Completed: false,
}

fmt.Println(task.Title) // Купить молоко

task.Title = "Купить продукты"
task.Completed = true

fmt.Println(task)
```

Структура — значение. При присваивании одной переменной другой копируются все поля:

```
original := Task{
  ID:       1,
  Title:    "Купить молоко",
  Priority: 2,
}
copyOfTask := original

copyOfTask.Title = "Купить продукты"

fmt.Println(original.Title)   // Купить молоко
fmt.Println(copyOfTask.Title) // Купить продукты
```

Это удобно, когда нужна независимая копия, но важно помнить об этом при передаче структуры в функцию. Указатели и изменение структуры через функции разберём позднее.

Слайс задач

Слайс может хранить структуры так же, как строки или числа:

```
tasks := []Task{
  {
    ID:       1,
    Title:    "Купить молоко",
    Priority: 2,
  },
  {
    ID:       2,
    Title:    "Оплатить интернет",
    Priority: 3,
  },
}
```

Добавить задачу можно через `append`:

```
tasks = append(tasks, Task{
  ID:       3,
  Title:    "Прочитать книгу",
  Priority: 1,
})
```

Для вывода используйте `range`:

```
for _, task := range tasks {
  fmt.Printf("%d. %s — приоритет %d\n", task.ID, task.Title, task.Priority)
}
```

Переменная `task` в этом цикле — копия текущей структуры. Её изменение не меняет элемент слайса:

```
for _, task := range tasks {
  task.Completed = true
}

fmt.Println(tasks[0].Completed) // false
```

Чтобы изменить элемент слайса, обходите его по индексу:

```
for index := range tasks {
  tasks[index].Completed = true
}

fmt.Println(tasks[0].Completed) // true
```

Map задач по ID

Map может хранить `Task` как значение:

```
tasksByID := map[int]Task{
  1: {
    ID:       1,
    Title:    "Купить молоко",
    Priority: 2,
  },
}
```

Такой map удобен для поиска задачи по ID:

```
if task, exists := tasksByID[1]; exists {
  fmt.Println(task.Title)
}
```

Значение map нельзя изменить напрямую через поле:

```
// tasksByID[1].Completed = true // ошибка компиляции
```

Сначала получите структуру в переменную, измените её и сохраните обратно:

```
task := tasksByID[1]
task.Completed = true
tasksByID[1] = task
```

Это следствие того, что map возвращает копию значения. В будущей главе об указателях рассмотрим другой способ проектирования изменяемых моделей.

Полная модель задачи

```
package main

import "fmt"

type Task struct {
  ID        int
  Title     string
  Priority  int
  Completed bool
}

func main() {
  tasks := []Task{
    {
      ID:       1,
      Title:    "Купить молоко",
      Priority: 2,
    },
    {
      ID:       2,
      Title:    "Оплатить интернет",
      Priority: 3,
    },
  }

  tasks[0].Completed = true
```

```
  for _, task := range tasks {   fmt.Printf("%d. %s | приоритет: %d | выполнена:
%t\n",     task.ID,     task.Title,       task.Priority,      task.Completed,     )
}}
```

Частые ошибки

Литерал без имён полей

```
task := Task{1, "Купить молоко", 2, false}
```

Этот код компилируется, но хуже читается и зависит от порядка полей. Если порядок изменится, литерал придётся переписывать. Используйте имена:

```
task := Task{
  ID:       1,
  Title:    "Купить молоко",
  Priority: 2,
}
```

Изменение копии в `range`

```
for _, task := range tasks {
  task.Completed = true
}
```

Изменяется копия. Нужен индекс: `tasks[index].Completed = true`.

Прямое изменение поля структуры в map

```
// tasksByID[1].Completed = true // ошибка
```

Получите значение, измените его и присвойте обратно в map.

Дальше

Закрепите модель `Task` на практике. Следующая тема — методы и указатели: они позволят связать действия со структурой и изменить задачу через функцию без ручного копирования.

Самопроверка

1. Что описывает struct в Go?

A. Цикл с условием

B. Тип с именованными полями

C. Пакет стандартной библиотеки

D. Только список строк

2. Как правильно объявить тип Task с полем Title string?

A. type Task = { Title: string }

B. task Task struct { Title string }

C. struct Task { string Title }

D. type Task struct { Title string }

3. Какой способ создания структуры предпочтительнее?

A. new Task(1, "Купить молоко")

B. Task{ID: 1, Title: "Купить молоко", Priority: 2}

C. Task{1, "Купить молоко", 2, false}

D. Task("Купить молоко")

4. Как получить значение поля Title у переменной task?

A. task["Title"]

B. task->Title

C. Title(task)

D. task.Title

5. Какое нулевое значение у поля Completed типа bool?

A. Ошибка компиляции

B. false

C. nil

D. true

6. Что происходит при copyOfTask := original, если оба значения имеют тип Task?

A. Создаётся map

B. Копируются поля структуры

C. copyOfTask всегда указывает на original

D. original удаляется

7. Почему task.Completed = true внутри for _, task := range tasks не изменяет элементы слайса?

A. task — копия текущей структуры

B. Слайс всегда неизменяем

C. Поле Completed нельзя изменять

D. range работает только со строками

8. Как изменить Completed у всех задач слайса?

A. for index := range tasks { tasks[index].Completed = true }

B. append(tasks, Completed: true)

C. for _, task := range tasks { task.Completed = true }

D. tasks.Completed = true

9. Почему нельзя написать tasksByID[1].Completed = true для map[int]Task?

A. Map возвращает копию структуры, которую нельзя изменить через поле напрямую

B. Ключ 1 запрещён

C. Поле Completed доступно только в слайсах

D. Map не может хранить структуры

10. Как правильно обновить Completed у Task в map?

A. Вызвать delete, затем читать map

B. Получить Task, изменить поле и присвоить Task обратно в map

C. Изменить ключ структуры

D. Использовать append для map

Ответы к самопроверке

1. Тип с именованными полями

2. type Task struct { Title string }

3. Task{ID: 1, Title: "Купить молоко", Priority: 2}

4. task.Title

5. false

6. Копируются поля структуры

7. task — копия текущей структуры

8. for index := range tasks { tasks[index].Completed = true }

9. Map возвращает копию структуры, которую нельзя изменить через поле напрямую

10. Получить Task, изменить поле и присвоить Task обратно в map

Задания: модель Task

Теперь Task Manager API хранит задачу как одну структуру, а не как набор независимых переменных. Будем использовать минимальную модель: ID, название, приоритет и признак выполнения.

Задание 1. Создайте задачу

Объявите тип `Task` с полями `ID int`, `Title string`, `Priority int` и `Completed bool`. Создайте одну задачу с ID `1`, названием `Купить молоко` и приоритетом `2`. Выведите все её поля.

Критерии проверки:

- объявлен тип `Task struct`;
- литерал использует имена полей;
- поле `Completed` можно не указывать — оно должно получить `false`;
- значения полей читаются через точку.

Подсказка

```
type Task struct {
  ID        int
  Title     string
  Priority  int
  Completed bool
}
```

Решение

```
package main

import "fmt"

type Task struct {
  ID        int
  Title     string
  Priority  int
  Completed bool
}

func main() {
  task := Task{
    ID:       1,
    Title:    "Купить молоко",
    Priority: 2,
  }

  fmt.Println(task.ID)
  fmt.Println(task.Title)
  fmt.Println(task.Priority)
  fmt.Println(task.Completed)
}
```

Задание 2. Отметьте задачу выполненной

Создайте задачу с `Completed: false`, затем измените это поле на `true` и выведите результат. Также измените название задачи.

Критерии проверки:

- поля изменяются через `task.Completed = true` и `task.Title = ...`;
- итоговое значение `Completed` равно `true`;
- изменённое название выводится в терминал.

Подсказка

После создания структуры её поле изменяют как обычную переменную, используя точку.

Решение

```
package main

import "fmt"

type Task struct {
  ID        int
  Title     string
  Priority  int
  Completed bool
}

func main() {
  task := Task{
    ID:       1,
    Title:    "Купить молоко",
    Priority: 2,
  }

  task.Title = "Купить продукты"
  task.Completed = true

  fmt.Println(task.Title)
  fmt.Println(task.Completed)
}
```

Задание 3. Список задач

Создайте слайс из трёх `Task` и выведите каждую задачу в формате:

```
1. Купить молоко | приоритет: 2 | выполнена: false
```

После создания списка отметьте вторую задачу выполненной.

Критерии проверки:

- используется `[]Task`;
- каждая структура создаётся через именованные поля;
- вторая задача меняется через индекс слайса;
- список обходится через `range`;
- в выводе есть ID, название, приоритет и признак выполнения.

Подсказка

Вторая задача имеет индекс `1`:

```
tasks[1].Completed = true
```

Решение

```
package main

import "fmt"

type Task struct {
  ID        int
  Title     string
  Priority  int
  Completed bool
}

func main() {
  tasks := []Task{
    {ID: 1, Title: "Купить молоко", Priority: 2},
    {ID: 2, Title: "Оплатить интернет", Priority: 3},
    {ID: 3, Title: "Прочитать книгу", Priority: 1},
  }

  tasks[1].Completed = true

  for _, task := range tasks {
    fmt.Printf("%d. %s | приоритет: %d | выполнена: %t\n",
      task.ID,
      task.Title,
      task.Priority,
      task.Completed,
    )
  }
}
```

Задание 4. Map задач по ID

Создайте `map[int]Task` с двумя задачами. Получите задачу с ID `1`, отметьте её выполненной и сохраните обратно в map. Затем выведите обновлённое значение.

Критерии проверки:

- используется `map[int]Task`;
- задача извлекается в отдельную переменную;
- поле `Completed` меняется у переменной;
- изменённая структура присваивается обратно по тому же ID;
- в map сохранено значение с `Completed: true`.

Подсказка

```
task := tasksByID[1]
task.Completed = true
tasksByID[1] = task
```

Решение

```
package main

import "fmt"

type Task struct {
  ID        int
  Title     string
  Priority  int
  Completed bool
}

func main() {
  tasksByID := map[int]Task{
    1: {ID: 1, Title: "Купить молоко", Priority: 2},
    2: {ID: 2, Title: "Оплатить интернет", Priority: 3},
  }

  task := tasksByID[1]
  task.Completed = true
  tasksByID[1] = task

  fmt.Println(tasksByID[1])
}
```

Проверьте себя

Перед следующей главой убедитесь, что вы можете:

- объявить собственный тип структуры;
- создать структуру с именованными полями;
- прочитать и изменить поле через точку;
- хранить структуры в слайсе и менять элемент по индексу;
- обновить структуру, которая хранится значением в map.

Рекомендации по коду: структуры

- Создавайте литералы структур с именами полей: это устойчиво к изменению порядка и проще читается.
- Группируйте в структуре данные одной сущности, например свойства одной задачи.
- Не используйте структуру как неограниченный контейнер: если поля относятся к разным обязанностям, выделите отдельную модель.
- Помните, что структура копируется при присваивании и передаче по значению.
- Изменяйте элементы `[]Task` по индексу, а не через переменную `range`, которая является копией.
- При хранении `Task` значением в map получите копию, измените её и присвойте обратно.

Чек-лист

У модели есть ясная ответственность, её поля валидируются, а код осознанно различает копию структуры и исходное значение.

Методы

Цель урока

Научиться объявлять методы для собственного типа, вызывать их у структуры и размещать поведение задачи рядом с её данными.

В предыдущей главе `Task` описывал данные задачи. Теперь добавим действия, которые естественно относятся к задаче: получить текстовое представление и проверить, что поля допустимы.

Функция и метод

Обычная функция получает значения параметрами:

```
func isValidTask(task Task) bool {
  return len(task.Title) > 0 && task.Priority >= 1 && task.Priority <= 3
}
```

Метод тоже является функцией, но связан с конкретным типом. Получатель (receiver) записывается между `func` и именем метода:

```
func (task Task) IsValid() bool {
  return len(task.Title) > 0 && task.Priority >= 1 && task.Priority <= 3
}
```

Теперь метод вызывается через точку у значения типа `Task`:

```
task := Task{
  ID:       1,
  Title:    "Купить молоко",
  Priority: 2,
}

fmt.Println(task.IsValid()) // true
```

`task` в `(task Task)` — имя получателя. Это обычная переменная внутри метода, через которую можно читать поля структуры.

Модель задачи

Во всех примерах главы используем уже знакомый тип:

```
type Task struct {
  ID        int
  Title     string
  Priority  int
  Completed bool
}
```

Методы можно объявлять в том же пакете, что и тип. В маленьких примерах это один файл `main.go`; в настоящем приложении тип и методы часто находятся в отдельных файлах одного пакета.

Метод без результата

Метод может выполнять действие и ничего не возвращать:

```
func (task Task) Print() {
  fmt.Printf("%d. %s | приоритет: %d | выполнена: %t\n",
    task.ID,
    task.Title,
    task.Priority,
    task.Completed,
  )
}
```

Вызов:

```
task.Print()
```

Такой метод подходит для простого учебного вывода. Но бизнес-логику обычно удобнее писать без прямого вывода в терминал: тогда её легче переиспользовать и тестировать.

Метод с результатом

Пусть задача возвращает текстовое описание вместо того, чтобы сразу печатать его:

```
func (task Task) Summary() string {
  return fmt.Sprintf(
    "%d. %s | приоритет: %d | выполнена: %t",
    task.ID,
    task.Title,
```

```
    task.Priority,    task.Completed,   )}
```

`fmt.Sprintf` работает как `fmt.Printf`, но возвращает готовую строку. Теперь вызывающий код сам решает, что с ней делать:

```
fmt.Println(task.Summary())
```

Проверка данных методом

Метод `IsValid` может проверить простые правила задачи:

```
func (task Task) IsValid() bool {
  return task.ID > 0 &&
    len(task.Title) > 0 &&
    len(task.Title) <= 120 &&
    task.Priority >= 1 &&
    task.Priority <= 3
}
```

Использование:

```
if task.IsValid() {
  fmt.Println("Задачу можно создать")
} else {
  fmt.Println("Задача содержит некорректные данные")
}
```

Метод не обязан знать, как именно будет показано сообщение. Его обязанность — вернуть результат проверки.

Получатель по значению

Во всех примерах `Task` передаётся в метод как значение:

```
func (task Task) Summary() string {
  // task — копия структуры, доступная только в этом методе
}
```

Это подходит для методов, которые только читают данные. Если изменить поле получателя, изменится лишь копия:

```
func (task Task) MarkCompleted() {
  task.Completed = true
}

func main() {
  task := Task{Title: "Купить молоко"}
  task.MarkCompleted()

  fmt.Println(task.Completed) // false
}
```

Для изменения исходной структуры нужен получатель-указатель. Это следующая глава — пока не используйте методы, которые создают впечатление изменения, но на самом деле меняют копию.

Методы в слайсе задач

Методы удобно вызывать у каждого элемента слайса:

```
tasks := []Task{
  {ID: 1, Title: "Купить молоко", Priority: 2},
  {ID: 2, Title: "Оплатить интернет", Priority: 3},
}

for _, task := range tasks {
  fmt.Println(task.Summary())
}
```

Здесь `task` — копия элемента, но это не мешает методам `Summary` и `IsValid`, потому что они только читают данные.

Полный пример

```
package main

import "fmt"

type Task struct {
  ID        int
  Title     string
  Priority  int
  Completed bool
}
```

```
func (task Task) Summary() string {   return fmt.Sprintf(   "%d. %s | приоритет: %d | выполнена: %t",     task.ID,   task.Title,    task.Priority,    task.Completed,  )}func (task Task) IsValid() bool {   return task.ID > 0 &&   len(task.Title) > 0 && len(task.Title) <= 120 &&    task.Priority >= 1 &&   task.Priority <= 3}func main() {  task := Task{     ID:       1,    Title:    "Купить молоко",   Priority: 2, } if !task.IsValid() {   fmt.Println("Задача содержит некорректные данные")    return } fmt.Println(task.Summary())}
```

Частые ошибки

Метод объявлен без получателя

```
func Summary() string {
  // Это функция, а не метод: у неё нет доступа к конкретной Task.
  return ""
}
```

Добавьте получатель `(task Task)`:

```
func (task Task) Summary() string {
  return task.Title
}
```

Вызов метода как функции

```
// Summary(task) // ошибка: Summary — метод
fmt.Println(task.Summary())
```

Ожидание изменения исходной задачи

```
func (task Task) MarkCompleted() {
  task.Completed = true
}
```

Получатель по значению копируется. Не используйте такой метод для изменения задачи; решение появится после изучения указателей.

Дальше

Следующая глава посвящена указателям. Она объяснит, как работать с адресом значения и как писать методы, которые действительно изменяют исходную структуру.

Самопроверка

1. Чем метод отличается от обычной функции?

A. Метод всегда изменяет структуру

B. Метод не может принимать параметры

C. Метод нельзя вызвать из main

D. У метода есть получатель определённого типа

2. Как правильно объявить метод IsValid для типа Task?

A. func Task.IsValid() bool { }

B. func IsValid(task Task) method bool { }

C. func (task Task) IsValid() bool { }

D. method Task.IsValid() bool { }

3. Как вызвать метод Summary у переменной task?

A. task->Summary()

B. Task.Summary(task)

C. task.Summary()

D. Summary(task)

4. Какую функцию fmt стоит использовать, когда нужно вернуть отформатированную строку?

A. fmt.Println

B. fmt.Sprintf

C. fmt.Errorf

D. fmt.Scan

5. Для каких методов подходит получатель по значению Task?

A. Ни для каких: всегда нужен указатель

B. Только для работы с map

C. Для методов, которые только читают данные задачи

D. Только для методов, меняющих задачу

6. Что произойдёт после вызова MarkCompleted, если он объявлен как func (task Task) MarkCompleted()?

A. Исходная задача всегда станет выполненной

B. Изменится только копия task внутри метода

C. Программа не скомпилируется

D. Изменятся все задачи в слайсе

7. Может ли метод вернуть bool?

A. Только вместе с fmt.Println

B. Да, например IsValid() bool

C. Только если получатель — указатель

D. Нет, методы всегда ничего не возвращают

8. Где должны быть объявлены тип Task и его методы?

A. Только внутри main

B. В разных модулях

C. В одном пакете

D. Обязательно в одном файле

9. Что делает метод IsValid для Task?

A. Автоматически сохраняет задачу в базу данных

B. Удаляет некорректные поля

C. Всегда печатает задачу в терминал

D. Проверяет данные и возвращает результат проверки

10. Почему полезно разместить Summary рядом с типом Task?

A. Нельзя писать функции вне структуры

B. Поведение задачи находится рядом с её данными

C. Так Go перестаёт копировать значения

D. Метод автоматически становится JSON

Ответы к самопроверке

1. У метода есть получатель определённого типа

2. func (task Task) IsValid() bool { }

3. task.Summary()

4. fmt.Sprintf

5. Для методов, которые только читают данные задачи

6. Изменится только копия task внутри метода

7. Да, например IsValid() bool

8. В одном пакете

9. Проверяет данные и возвращает результат проверки

10. Поведение задачи находится рядом с её данными

Задания: поведение Task

В этой практике методы только читают данные задачи и возвращают результат. Изменение задачи через методы разберём в следующей главе об указателях.

Задание 1. Краткое описание задачи

Добавьте к типу `Task` метод `Summary() string`. Он должен возвращать строку вида:

```
1. Купить молоко | приоритет: 2 | выполнена: false
```

Создайте задачу и выведите результат метода.

Критерии проверки:

- метод объявлен с получателем `(task Task)`;
- метод возвращает `string`;
- для форматирования используется `fmt.Sprintf`;
- `main` выводит `task.Summary()`.

Подсказка

Сигнатура:

```
func (task Task) Summary() string {
  // return fmt.Sprintf(...)
}
```

Решение

```
package main

import "fmt"

type Task struct {
  ID        int
  Title     string
  Priority  int
  Completed bool
}

func (task Task) Summary() string {
  return fmt.Sprintf(
    "%d. %s | приоритет: %d | выполнена: %t",
    task.ID,
    task.Title,
    task.Priority,
    task.Completed,
  )
}

func main() {
  task := Task{ID: 1, Title: "Купить молоко", Priority: 2}

  fmt.Println(task.Summary())
}
```

Задание 2. Проверка задачи

Добавьте метод `IsValid() bool`. Задача корректна, если:

- ID больше `0`;
- название не пустое и не длиннее 120 байтов;
- приоритет находится в диапазоне от `1` до `3`.

Проверьте одну корректную и одну некорректную задачу.

Критерии проверки:

- метод возвращает `bool`;
- все три правила проверяются через `&&`;
- корректная задача даёт `true`;
- задача с пустым названием или приоритетом `4` даёт `false`.

Подсказка

Все проверки можно вернуть одним выражением из `return`.

Решение

```
package main

import "fmt"

type Task struct {
  ID        int
  Title     string
  Priority  int
  Completed bool
}

func (task Task) IsValid() bool {
  return task.ID > 0 &&
    len(task.Title) > 0 &&
    len(task.Title) <= 120 &&
    task.Priority >= 1 &&
    task.Priority <= 3
}

func main() {
  validTask := Task{ID: 1, Title: "Купить молоко", Priority: 2}
  invalidTask := Task{ID: 2, Title: "", Priority: 4}

  fmt.Println(validTask.IsValid())
  fmt.Println(invalidTask.IsValid())
}
```

Задание 3. Список корректных задач

Создайте слайс из трёх задач, одна из которых некорректна. Выведите `Summary()` только для корректных задач. Используйте `range`, `IsValid()` и `continue`.

Критерии проверки:

- используется `[]Task`;
- каждая задача проверяется через `task.IsValid()`;
- некорректная задача пропускается через `continue`;
- для остальных выводится `task.Summary()`.

Подсказка

```
for _, task := range tasks {
  if !task.IsValid() {
    continue
  }

  fmt.Println(task.Summary())
}
```

Решение

```
package main

import "fmt"

type Task struct {
  ID        int
  Title     string
  Priority  int
  Completed bool
}

func (task Task) Summary() string {
  return fmt.Sprintf("%d. %s | приоритет: %d | выполнена: %t", task.ID, task.Title,
task.Priority, task.Completed)
}

func (task Task) IsValid() bool {
  return task.ID > 0 && len(task.Title) > 0 && len(task.Title) <= 120 &&
task.Priority >= 1 && task.Priority <= 3
}

func main() {
  tasks := []Task{
    {ID: 1, Title: "Купить молоко", Priority: 2},
    {ID: 2, Title: "", Priority: 4},
    {ID: 3, Title: "Прочитать книгу", Priority: 1},
  }

  for _, task := range tasks {
    if !task.IsValid() {
      continue
```

▼ Сведения

```
    }   fmt.Println(task.Summary())  }}
```

Проверьте себя

Перед следующей главой убедитесь, что вы можете:

- отличить метод от обычной функции;
- объявить и вызвать метод с получателем `Task`;
- вернуть строку через `fmt.Sprintf`;
- написать метод, который проверяет данные, но не изменяет структуру;
- объяснить, почему получатель по значению не подходит для изменения исходной задачи.

Рекомендации по коду: методы

- Размещайте поведение сущности рядом с её типом: `Task.Summary`, `Task.IsValid`.
- Выбирайте value receiver для небольших неизменяемых операций чтения.
- Не создавайте метод с value receiver, если его имя обещает изменение исходной структуры.
- Сохраняйте единообразие receiver для одного типа; исключения должны быть понятны и обоснованы.
- Пусть методы бизнес-логики возвращают значение или ошибку, а вывод в терминал остаётся на уровне `main` или handler.
- Не добавляйте метод только потому, что он технически возможен: иногда обычная функция лучше выражает независимую операцию.

Чек-лист

Методы читаются как действия модели, не смешивают вывод и бизнес-логику, а их receiver соответствует тому, меняют ли они данные.

Указатели

Цель урока

Понять, что хранит указатель, как безопасно работать с `nil` и как с помощью pointer receiver менять исходную задачу методами `Complete` и `Rename`.

Указатели в Go не нужны для каждого значения. Они полезны, когда функция или метод должны изменить исходное значение, когда копирование большой структуры нежелательно или когда нужно выразить отсутствие значения через `nil`.

Значение и адрес

Обычная переменная хранит значение:

```
priority := 2
```

У каждой переменной есть адрес в памяти. Оператор `&` получает этот адрес:

```
priority := 2
priorityAddress := &priority

fmt.Println(priorityAddress)
```

Тип `priorityAddress` — `*int`: «указатель на `int`». Адрес при каждом запуске может выглядеть по-разному; не используйте его как данные программы.

```
var priorityAddress *int
```

Звёздочка в типе означает указатель на значение указанного типа.

Разыменование: `*`

Оператор `*` перед указателем даёт значение по адресу:

```
package main

import "fmt"

func main() {
  priority := 2
```

```
  priorityAddress := &priority fmt.Println(*priorityAddress) // 2  *priorityAddress =
3 fmt.Println(priority) // 3}
```

`*priorityAddress = 3` меняет не копию, а исходную переменную `priority`.

> **Не путайте роли `*`**
>
> В `*Task` звёздочка означает тип «указатель на Task». В `*task` она означает разыменование: доступ к значению по адресу.

`nil`

Указатель может не указывать ни на какое значение. Это состояние называется `nil`:

```
var task *Task

fmt.Println(task == nil) // true
```

Разыменовывать `nil` нельзя: программа завершится с panic.

```
var priorityAddress *int

// fmt.Println(*priorityAddress) // panic
```

Перед доступом через указатель проверяйте его, если `nil` возможен:

```
if priorityAddress != nil {
  fmt.Println(*priorityAddress)
}
```

`new`

Встроенная функция `new` создаёт нулевое значение указанного типа и возвращает указатель на него:

```
task := new(Task)

fmt.Println(task.ID)        // 0
fmt.Println(task.Completed) // false
```

`task` имеет тип `*Task`. Go автоматически разыменовывает указатель при обращении к полю, поэтому можно написать `task.ID`, а не `(*task).ID`.

Для создания заполненной структуры чаще используют литерал с `&`:

```
task := &Task{
  ID:       1,
  Title:    "Купить молоко",
  Priority: 2,
}
```

Здесь `task` тоже имеет тип `*Task`.

Аргументы передаются по значению

Go всегда передаёт аргументы функции по значению. Если передать структуру, функция получит её копию:

```
func completeCopy(task Task) {
  task.Completed = true
}

func main() {
  task := Task{Title: "Купить молоко"}
  completeCopy(task)

  fmt.Println(task.Completed) // false
}
```

Чтобы функция изменила исходную задачу, передайте указатель:

```
func completeTask(task *Task) {
  if task == nil {
    return
  }

  task.Completed = true
}

func main() {
  task := Task{Title: "Купить молоко"}
  completeTask(&task)

  fmt.Println(task.Completed) // true
}
```

`&task` передаёт адрес исходной структуры. Внутри функции `task.Completed` изменяет поле по этому адресу.

Pointer receiver

Метод с получателем по значению читает копию структуры:

```
func (task Task) Summary() string {
  return task.Title
}
```

Если метод должен изменить исходную структуру, получатель должен быть указателем:

```
func (task *Task) Complete() {
  if task == nil {
    return
  }

  task.Completed = true
}
```

Вызов выглядит привычно:

```
task := Task{Title: "Купить молоко"}
task.Complete()

fmt.Println(task.Completed) // true
```

Компилятор сам берёт адрес `&task`, потому что переменная `task` адресуемая. Это сокращение работает для переменных, но не для временных значений и не для значений структуры внутри map.

Методы `Complete` и `Rename`

Добавим поведение к модели Task Manager API:

```
type Task struct {
  ID        int
  Title     string
  Priority  int
  Completed bool
}
```

```
func (task *Task) Complete() bool {   if task == nil {    return false }
task.Completed = true  return true}func (task *Task) Rename(title string) bool { if
task == nil || len(title) == 0 || len(title) > 120 {    return false } task.Title =
title  return true}
```

Методы возвращают `bool`, чтобы сообщить, удалось ли выполнить действие. В будущей главе об ошибках заменим такой результат на более информативный `error`.

Использование:

```
task := Task{
  ID:       1,
  Title:    "Купить молоко",
  Priority: 2,
}

task.Complete()
task.Rename("Купить продукты")

fmt.Println(task.Title)     // Купить продукты
fmt.Println(task.Completed) // true
```

Значение и указатель: когда что выбрать

Получатель по значению `Task` подходит, когда метод:

- только читает поля;
- не должен менять исходную задачу;
- должен работать одинаково с копией и оригиналом.

```
func (task Task) IsValid() bool { /* ... */ }
func (task Task) Summary() string { /* ... */ }
```

Получатель `*Task` нужен, когда метод:

- меняет исходную задачу;
- должен отличать отсутствие задачи (`nil`);
- работает с большой структурой и копирование нежелательно.

```
func (task *Task) Complete() bool { /* ... */ }
func (task *Task) Rename(title string) bool { /* ... */ }
```

Для одного типа не смешивайте получатели без причины. В нашей модели методы чтения пока остаются значимыми, а методы изменения используют указатель — это понятное различие для учебного примера.

Указатель и map

Структура, хранящаяся значением в map, не адресуемая:

```
tasksByID := map[int]Task{
  1: {ID: 1, Title: "Купить молоко", Priority: 2},
}

// tasksByID[1].Complete() // ошибка: нельзя вызвать pointer method у значения map
```

Получите копию, измените её и сохраните обратно:

```
task := tasksByID[1]
task.Complete()
tasksByID[1] = task
```

Позднее обсудим, когда map хранит `Task`, а когда `*Task`. Сейчас важнее понимать, что map в этом примере возвращает копию структуры.

Частые ошибки

Разыменование `nil`

```
var task *Task
// task.Completed = true // panic
```

Проверьте указатель или создайте значение:

```
task = &Task{Title: "Купить молоко"}
task.Completed = true
```

Метод по значению для изменения структуры

```
func (task Task) Complete() {
  task.Completed = true
}
```

Этот метод меняет копию. Используйте получатель `*Task`.

Передача структуры вместо адреса

```
func completeTask(task *Task) { /* ... */ }

task := Task{}
// completeTask(task) // ошибка: нужен *Task
completeTask(&task)
```

Лишние указатели

Не используйте указатель к `int` или `string` только потому, что «так производительнее». Для маленьких значений это часто усложняет код без пользы. Сначала определите: нужно ли менять исходное значение или выразить отсутствие значения?

Дальше

Закрепите `Complete` и `Rename` на практике. Затем перейдём к интерфейсам — способу описывать нужное поведение без привязки к конкретному типу.

Самопроверка

1. Что хранит указатель типа *Task?

A. Адрес значения типа Task

B. Копию всех задач

C. Размер структуры Task

D. Только название задачи

2. Как получить адрес переменной task?

A. &task

B. address(task)

C. *task

D. task&

3. Что делает *taskPointer, если taskPointer имеет тип *Task?

A. Получает адрес указателя

B. Удаляет указатель

C. Даёт доступ к значению Task по адресу

D. Создаёт новую Task

4. Какое нулевое значение у указателя?

A. 0

B. ""

C. false

D. nil

5. Что произойдёт при попытке разыменовать nil-указатель?

A. Вернётся нулевое значение

B. Указатель станет пустой структурой

C. Будет panic во время выполнения

D. Компилятор всегда автоматически создаст значение

6. Что возвращает new(Task)?

A. Map задач

B. Task с заполненными полями

C. Слайс Task

D. Указатель на Task с нулевыми полями

7. Как передать исходную переменную task в функцию, которая принимает *Task?

A. Передать *task

B. Передать task без изменений

C. Передать new(task)

D. Передать &task

8. Какой получатель нужен методу Complete, который меняет поле Completed исходной задачи?

A. func (*Task Complete)

B. func (task Task) Complete()

C. func (task *Task) Complete()

D. func Complete(task Task)

9. Почему вызов task.Complete() работает, если Complete имеет получатель *Task?

A. Go копирует task в map

B. Task всегда является указателем

C. Компилятор берёт адрес адресуемой переменной task автоматически

D. Методы не используют получателей

10. Как обновить Task, который хранится значением в map[int]Task?

A. Получить Task, изменить его и присвоить обратно в map

B. Разыменовать ключ map

C. Вызвать tasksByID[id].Complete() напрямую

D. Использовать append для map

Ответы к самопроверке

1. Адрес значения типа Task

2. &task

3. Даёт доступ к значению Task по адресу

4. nil

5. Будет panic во время выполнения

6. Указатель на Task с нулевыми полями

7. Передать &task

8. func (task *Task) Complete()

9. Компилятор берёт адрес адресуемой переменной task автоматически

10. Получить Task, изменить его и присвоить обратно в map

Задания: изменение Task

Теперь Task Manager API может менять существующую задачу через методы. Методы с pointer receiver работают с исходной структурой, а не с её копией.

Задание 1. Выполните задачу

Добавьте к `Task` метод `Complete() bool`. Если получатель не `nil`, метод должен установить `Completed` в `true` и вернуть `true`. Для `nil` метод не должен паниковать и должен вернуть `false`.

Критерии проверки:

- используется получатель `*Task`;
- обычная задача после `Complete()` имеет `Completed: true`;
- метод возвращает `true` при успешном выполнении;
- вызов для `nil`-указателя не вызывает panic и возвращает `false`.

Подсказка

В начале метода проверьте получатель:

```
if task == nil {
  return false
}
```

Решение

```
package main

import "fmt"

type Task struct {
  ID        int
  Title     string
  Priority  int
  Completed bool
}

func (task *Task) Complete() bool {
  if task == nil {
    return false
  }

  task.Completed = true
  return true
}

func main() {
  task := Task{ID: 1, Title: "Купить молоко", Priority: 2}

  fmt.Println(task.Complete())
  fmt.Println(task.Completed)

  var missingTask *Task
  fmt.Println(missingTask.Complete())
}
```

Задание 2. Переименуйте задачу

Добавьте метод `Rename(title string) bool`. Он должен изменить название, если:

- получатель не `nil`;
- новое название не пустое;
- длина названия не превышает 120 байтов.

Во всех остальных случаях задача не должна измениться, а метод должен вернуть `false`.

Критерии проверки:

- используется получатель `*Task`;
- корректное название меняет поле `Title`;
- пустая строка не меняет поле;
- слишком длинное название не меняет поле;
- результат метода показывает успех или неудачу.

Подсказка

Все условия отказа можно объединить в один `if`:

```
if task == nil || len(title) == 0 || len(title) > 120 {
  return false
}
```

Решение

```
package main

import "fmt"

type Task struct {
  ID        int
  Title     string
  Priority  int
  Completed bool
}

func (task *Task) Rename(title string) bool {
  if task == nil || len(title) == 0 || len(title) > 120 {
    return false
  }

  task.Title = title
  return true
}

func main() {
  task := Task{ID: 1, Title: "Купить молоко", Priority: 2}

  fmt.Println(task.Rename("Купить продукты"))
  fmt.Println(task.Title)

  fmt.Println(task.Rename(""))
  fmt.Println(task.Title)
}
```

Задание 3. Измените задачу из map

Создайте `map[int]Task` с одной задачей. Получите её по ID, вызовите `Complete` и `Rename`, затем сохраните изменённую структуру обратно в map. Выведите итоговую задачу.

Критерии проверки:

- map имеет тип `map[int]Task`;
- задача извлекается в локальную переменную;
- методы вызываются у этой переменной;
- переменная присваивается обратно в map;
- в map сохранены новое название и `Completed: true`.

Подсказка

```
task := tasksByID[1]
task.Complete()
task.Rename("Купить продукты")
tasksByID[1] = task
```

Решение

```
package main

import "fmt"

type Task struct {
  ID        int
  Title     string
  Priority  int
  Completed bool
}

func (task *Task) Complete() bool {
  if task == nil {
    return false
  }

  task.Completed = true
  return true
}

func (task *Task) Rename(title string) bool {
  if task == nil || len(title) == 0 || len(title) > 120 {
    return false
  }

  task.Title = title
  return true
}

func main() {
  tasksByID := map[int]Task{
    1: {ID: 1, Title: "Купить молоко", Priority: 2},
  }
```

▼ Сведения

```
   task := tasksByID[1]  task.Complete()   task.Rename("Купить продукты") tasksByID[1]
= task fmt.Println(tasksByID[1])}
```

Проверьте себя

Перед следующей главой убедитесь, что вы можете:

- получить адрес переменной через `&`;
- объяснить разницу между `*Task` и `*task`;
- проверить указатель на `nil`;
- объявить метод с pointer receiver;
- изменить `Task` через `Complete` и `Rename`;
- обновить структуру, хранящуюся значением в map.

Рекомендации по коду: указатели

- Используйте указатель, когда нужно изменить исходное значение, выразить отсутствие значения или избежать копирования действительно большой структуры.
- Всегда рассматривайте `nil` как возможное состояние указателя, если он приходит извне или хранится в поле.
- Выбирайте pointer receiver для методов, которые меняют структуру.
- Не используйте `*int`, `*string` или указатель на слайс/map без ясной причины — это усложняет API.
- Не разыменовывайте указатель до проверки `nil`, если nil допустим по контракту.
- Учитывайте, что значение структуры в map не адресуемо: получите копию, измените и присвойте обратно.

Чек-лист

У каждого указателя есть понятная причина, методы изменения используют `*T`, а сценарии с `nil` не приводят к panic.

Интерфейсы

Цель урока

Научиться описывать нужное поведение небольшим интерфейсом, понимать неявную реализацию в Go и писать функции, которые зависят от возможностей значения, а не от конкретного типа.

В Task Manager API задача уже умеет формировать краткое описание через `Summary() string`. Теперь опишем это умение интерфейсом `TaskFormatter` и напишем код, который работает с любым значением, умеющим возвращать summary.

Что такое интерфейс

Интерфейс определяет набор методов. Он не хранит реализацию этих методов и не описывает поля структуры.

```
type TaskFormatter interface {
  Summary() string
}
```

`TaskFormatter` означает: «любое значение, у которого есть метод `Summary() string`».

Интерфейс не требует слова `implements`. Тип реализует интерфейс автоматически, если его методы совпадают по имени, параметрам и результатам.

Task реализует TaskFormatter

```
type Task struct {
  ID        int
  Title     string
  Priority  int
  Completed bool
}

func (task Task) Summary() string {
  return fmt.Sprintf(
    "%d. %s | приоритет: %d | выполнена: %t",
    task.ID,
    task.Title,
    task.Priority,
    task.Completed,
```

```
  )}
```

У `Task` есть метод с точной сигнатурой `Summary() string`, поэтому `Task` реализует `TaskFormatter` автоматически.

Проверить это на этапе компиляции можно присваиванием:

```
var formatter TaskFormatter = Task{
  ID:       1,
  Title:    "Купить молоко",
  Priority: 2,
}

fmt.Println(formatter.Summary())
```

Если сигнатура не совпадает, код не скомпилируется:

```
// func (task Task) Summary() { }
// var formatter TaskFormatter = Task{} // ошибка: Summary не возвращает string
```

Функция зависит от поведения

Вместо функции, которая принимает только `Task`, можно принять интерфейс:

```
func printSummary(formatter TaskFormatter) {
  fmt.Println(formatter.Summary())
}
```

Теперь ей можно передать любую реализацию `TaskFormatter`:

```
task := Task{
  ID:       1,
  Title:    "Купить молоко",
  Priority: 2,
}

printSummary(task)
```

Функции не важно, есть ли у значения поля `ID`, `Title` или `Completed`. Ей важен только метод `Summary`.

Ещё одна реализация

Создадим тип для краткого уведомления. Он не является задачей, но тоже может вернуть строковое описание:

```
type Notification struct {
  Message string
}

func (notification Notification) Summary() string {
  return "Уведомление: " + notification.Message
}
```

Он автоматически реализует тот же интерфейс:

```
task := Task{ID: 1, Title: "Купить молоко", Priority: 2}
notification := Notification{Message: "Список задач обновлён"}

printSummary(task)
printSummary(notification)
```

Это и есть полиморфизм через интерфейсы: один и тот же код работает с разными типами, если они предоставляют нужное поведение.

Небольшие интерфейсы

В Go предпочтительны маленькие интерфейсы с одним-двумя методами. Хороший интерфейс описывает конкретную потребность вызывающего кода:

```
type TaskFormatter interface {
  Summary() string
}
```

Не объединяйте в один интерфейс несвязанные возможности заранее:

```
// Неудачная абстракция для текущего этапа:
// type Everything interface {
//     Summary() string
//     IsValid() bool
//     Complete() bool
//     Rename(string) bool
// }
```

Если функции нужно только напечатать описание, ей не нужны методы проверки, изменения или сохранения. Интерфейс лучше объявлять там, где он используется, а не заранее «на всякий случай».

Методы по значению и по указателю

Способ объявления метода влияет на то, кто реализует интерфейс.

Метод с получателем по значению доступен и для `Task`, и для `*Task`:

```
func (task Task) Summary() string {
  return task.Title
}

var first TaskFormatter = Task{Title: "Купить молоко"}
var second TaskFormatter = &Task{Title: "Оплатить интернет"}

fmt.Println(first.Summary())
fmt.Println(second.Summary())
```

Метод с pointer receiver доступен только для `*Task`:

```
func (task *Task) Complete() bool {
  if task == nil {
    return false
  }

  task.Completed = true
  return true
}

type Completer interface {
  Complete() bool
}

// var first Completer = Task{} // ошибка: Task не реализует Completer
var second Completer = &Task{Title: "Купить молоко"}
```

Это важно: если интерфейс требует метод с pointer receiver, в него нужно передать указатель.

Интерфейсное значение и nil

Интерфейсное значение содержит тип и конкретное значение. Поэтому есть важная разница:

```
var formatter TaskFormatter
fmt.Println(formatter == nil) // true
```

Здесь интерфейс пуст: нет ни типа, ни значения.

Но если положить в интерфейс nil-указатель конкретного типа, сам интерфейс не равен `nil`:

```
var task *Task
var formatter TaskFormatter = task

fmt.Println(formatter == nil) // false
```

В этой главе достаточно запомнить правило: не передавайте nil-указатель в интерфейс без необходимости. Если метод может быть вызван у nil-получателя, он должен сам корректно обработать это состояние.

Полный пример

```
package main

import "fmt"

type TaskFormatter interface {
  Summary() string
}

type Task struct {
  ID        int
  Title     string
  Priority  int
  Completed bool
}

func (task Task) Summary() string {
  return fmt.Sprintf(
    "%d. %s | приоритет: %d | выполнена: %t",
    task.ID,
    task.Title,
    task.Priority,
    task.Completed,
  )
}
```

```
type Notification struct { Message string}func (notification Notification) Summary() string { return "Уведомление: " + notification.Message}func printSummary(formatter TaskFormatter) {  fmt.Println(formatter.Summary())}func main() { task := Task{ID: 1, Title: "Купить молоко", Priority: 2}  notification := Notification{Message: "Список задач обновлён"}  printSummary(task)  printSummary(notification)}
```

Частые ошибки

Ожидание явного `implements`

В Go его нет. Не нужно изменять `Task`, чтобы «подключить» интерфейс. Достаточно реализовать нужный метод с точной сигнатурой.

Слишком большой интерфейс

```
// type TaskManager interface {
//     Summary() string
//     IsValid() bool
//     Complete() bool
//     Rename(string) bool
// }
```

Не добавляйте методы, которые не нужны конкретной функции. Маленький `TaskFormatter` проще реализовать и тестировать.

Значение вместо указателя

Если интерфейс требует только методы с pointer receiver, передавайте `&task`, а не `task`:

```
type Completer interface {
  Complete() bool
}

task := Task{Title: "Купить молоко"}
var completer Completer = &task
```

Дальше

Закрепите интерфейс `TaskFormatter` на практике. Затем перейдём к ошибкам: вместо `bool` методы и функции смогут возвращать причину неуспешного действия.

Самопроверка

1. Что определяет интерфейс в Go?

A. Набор полей структуры

B. Набор методов, который должен быть доступен значению

C. Порядок выполнения goroutine

D. Размер массива

2. Как тип реализует интерфейс в Go?

A. Через ключевое слово implements

B. Автоматически, если имеет методы с нужными сигнатурами

C. Только через наследование

D. После регистрации в main

3. Как правильно объявить интерфейс TaskFormatter?

A. type TaskFormatter implements Summary() string

B. type TaskFormatter struct { Summary() string }

C. interface TaskFormatter { Summary() string }

D. type TaskFormatter interface { Summary() string }

4. Почему Task реализует TaskFormatter?

A. Task объявлен в main

B. Task хранится в слайсе

C. Task обязательно содержит поле Title

D. У Task есть метод Summary() string

5. Что может принять функция printSummary(formatter TaskFormatter)?

A. Только string

B. Только Task

C. Только *Task

D. Любое значение с методом Summary() string

6. Почему небольшие интерфейсы предпочтительнее?

A. Они всегда работают быстрее

B. Большие интерфейсы не компилируются

C. Интерфейс должен содержать ровно один метод по правилам Go

D. Они описывают конкретную потребность кода и проще реализуются

7. Кто реализует интерфейс с методом Complete() bool, если Complete объявлен для *Task?

A. *Task

B. Ни Task, ни *Task

C. Только Task

D. Task и *Task

8. Кто реализует интерфейс с методом Summary() string, если Summary объявлен для Task?

A. Только *Task

B. Ни Task, ни *Task

C. Task и *Task

D. Только Task

9. Чему равно неинициализированное интерфейсное значение?

A. false

B. Пустой Task

C. nil

D. Пустая строка

10. Почему интерфейс с nil-указателем *Task внутри может не быть равен nil?

A. Go автоматически создаёт Task

B. Интерфейс хранит конкретный тип *Task, даже если его значение nil

C. Все интерфейсы всегда не равны nil

D. Указатели нельзя хранить в интерфейсах

Ответы к самопроверке

1. Набор методов, который должен быть доступен значению

2. Автоматически, если имеет методы с нужными сигнатурами

3. type TaskFormatter interface { Summary() string }

4. У Task есть метод Summary() string

5. Любое значение с методом Summary() string

6. Они описывают конкретную потребность кода и проще реализуются

7. *Task

8. Task и *Task

9. nil

10. Интерфейс хранит конкретный тип *Task, даже если его значение nil

Задания: TaskFormatter

В этой практике функция будет работать не с конкретной структурой `Task`, а с любым значением, которое умеет формировать краткое описание.

Задание 1. Опишите форматируемую задачу

Создайте интерфейс `TaskFormatter` с методом `Summary() string`. Объявите `Task` и добавьте ему метод `Summary`. Затем присвойте задачу переменной типа `TaskFormatter` и выведите summary.

Критерии проверки:

- интерфейс содержит только `Summary() string`;
- `Task` имеет метод с точной сигнатурой;
- значение `Task` присваивается переменной типа `TaskFormatter`;
- вызов `formatter.Summary()` возвращает описание задачи.

Подсказка

```
type TaskFormatter interface {
  Summary() string
}
```

Решение

```
package main

import "fmt"

type TaskFormatter interface {
  Summary() string
}

type Task struct {
  ID       int
  Title    string
  Priority int
}

func (task Task) Summary() string {
  return fmt.Sprintf("%d. %s | приоритет: %d", task.ID, task.Title, task.Priority)
}

func main() {
  var formatter TaskFormatter = Task{
    ID:       1,
    Title:    "Купить молоко",
    Priority: 2,
  }

  fmt.Println(formatter.Summary())
}
```

Задание 2. Вынесите печать в функцию

Напишите функцию `printSummary(formatter TaskFormatter)`. Она должна вывести результат `formatter.Summary()`. Передайте ей обычную задачу.

Критерии проверки:

- параметр функции имеет тип `TaskFormatter`;
- функция не зависит от конкретного типа `Task`;
- внутри вызывается `formatter.Summary()`;
- задача передаётся в функцию без дополнительного преобразования.

Подсказка

```
func printSummary(formatter TaskFormatter) {
  fmt.Println(formatter.Summary())
}
```

Решение

```
package main

import "fmt"

type TaskFormatter interface {
  Summary() string
}

type Task struct {
  ID       int
  Title    string
  Priority int
}

func (task Task) Summary() string {
  return fmt.Sprintf("%d. %s | приоритет: %d", task.ID, task.Title, task.Priority)
}

func printSummary(formatter TaskFormatter) {
  fmt.Println(formatter.Summary())
}

func main() {
  task := Task{ID: 1, Title: "Купить молоко", Priority: 2}

  printSummary(task)
}
```

Задание 3. Добавьте другой форматируемый тип

Создайте тип `Notification` с полем `Message string`. Добавьте ему `Summary() string`, которая возвращает строку `Уведомление: <сообщение>`. Передайте и `Task`, и `Notification` в одну функцию `printSummary`.

Критерии проверки:

- `Notification` не содержит полей `Task`;
- у `Notification` есть метод `Summary() string`;
- обе структуры передаются в `printSummary`;
- функция не меняется после добавления нового типа.

Подсказка

Интерфейс важен именно тем, что `printSummary` уже не нужно менять. Добавьте метод `Summary` новому типу.

Решение

```
package main

import "fmt"

type TaskFormatter interface {
  Summary() string
}

type Task struct {
  ID       int
  Title    string
  Priority int
}

func (task Task) Summary() string {
  return fmt.Sprintf("%d. %s | приоритет: %d", task.ID, task.Title, task.Priority)
}

type Notification struct {
  Message string
}

func (notification Notification) Summary() string {
  return "Уведомление: " + notification.Message
}

func printSummary(formatter TaskFormatter) {
  fmt.Println(formatter.Summary())
}

func main() {
  task := Task{ID: 1, Title: "Купить молоко", Priority: 2}
  notification := Notification{Message: "Список задач обновлён"}

  printSummary(task)
  printSummary(notification)
}
```

Проверьте себя

Перед следующей главой убедитесь, что вы можете:

- объявить маленький интерфейс;
- объяснить, почему Task реализует TaskFormatter без `implements`;
- написать функцию, которая принимает интерфейс;
- добавить новый тип с тем же поведением без изменения этой функции;
- определить, нужен ли интерфейсу метод с value receiver или pointer receiver.

Рекомендации по коду: интерфейсы

- Объявляйте интерфейс у потребителя поведения, а не заранее рядом с типом «на всякий случай».
- Делайте интерфейсы маленькими: один-два метода обычно лучше большого «универсального» контракта.
- Используйте интерфейс, когда реально нужны несколько возможных реализаций или тестируемая граница зависимости.
- Не создавайте интерфейс для каждого типа автоматически; конкретный тип часто проще и понятнее.
- Помните, что методы с pointer receiver реализуются только указателем `*T`.
- Не передавайте nil-указатель в интерфейс, если вызывающий код ожидает обычный nil.

Чек-лист

Интерфейс выражает конкретную потребность функции, его можно реализовать без лишних методов, а выбор между `T` и `*T` осознан.

Обработка ошибок

Цель урока

Научиться возвращать и обрабатывать ошибки через тип `error`, добавлять к ним контекст и проверять причину ошибки с помощью `errors.Is`.

В Task Manager API метод `Rename` больше не будет возвращать только `true` или `false`. Вместо этого он сообщит вызывающему коду, почему новое название не подходит.

Ошибки — обычные значения

В Go ошибка обычно возвращается как последнее значение функции. Тип `error` — это интерфейс с методом `Error() string`.

```
func findTask(id int) (Task, error) {
  // ...
}
```

Если операция успешна, ошибка равна `nil`. Если возникла проблема, функция возвращает значение `error`, а результат операции часто имеет нулевое значение.

```
task, err := findTask(42)
if err != nil {
  fmt.Println("Не удалось найти задачу:", err)
  return
}

fmt.Println(task.Title)
```

Такой стиль делает возможные ошибки видимыми в сигнатуре функции и заставляет вызывающий код осознанно решить, что с ними делать.

Создание ошибки: `errors.New`

Пакет `errors` создаёт простую ошибку с текстом:

```
import "errors"

var ErrEmptyTitle = errors.New("название задачи не должно быть пустым")
```

Ошибки, которые обозначают важные повторяющиеся причины, удобно объявлять как переменные на уровне пакета:

```
var (
  ErrTaskIsNil   = errors.New("задача не существует")
  ErrEmptyTitle  = errors.New("название задачи не должно быть пустым")
  ErrTitleTooLong = errors.New("название задачи слишком длинное")
)
```

Имена таких переменных начинаются с `Err`. Позднее, когда появятся пакеты, экспортируемые ошибки позволят другому пакету проверить конкретную причину без сравнения текстов.

Не создавайте новую ошибку в каждом месте, если вызывающему коду важно распознать именно её причину:

```
// Менее удобно для последующей проверки причины:
// return errors.New("название задачи не должно быть пустым")

return ErrEmptyTitle
```

Обработка: `if err != nil`

Стандартный паттерн — проверить ошибку сразу после вызова и выйти по раннему `return`:

```
func printTaskTitle(task *Task) {
  if err := task.Rename("Купить продукты"); err != nil {
    fmt.Println("Не удалось переименовать задачу:", err)
    return
  }

  fmt.Println("Новое название:", task.Title)
}
```

Так успешный путь остаётся без лишней вложенности:

```
// Менее читаемо, если успешный код становится большим:
if err == nil {
  // успешный путь
} else {
  // обработка ошибки
}
```

Метод `Rename` возвращает error

Перепишем метод Task Manager API. Метод `Complete` по-прежнему меняет признак выполнения и не выполняет валидацию. А `Rename` может не выполниться из-за некорректного входного значения, поэтому он возвращает ошибку.

```
package main

import (
  "errors"
  "fmt"
)

var (
  ErrTaskIsNil    = errors.New("задача не существует")
  ErrEmptyTitle   = errors.New("название задачи не должно быть пустым")
  ErrTitleTooLong = errors.New("название задачи слишком длинное")
)

type Task struct {
  ID        int
  Title     string
  Priority  int
  Completed bool
}

func (task *Task) Complete() {
  task.Completed = true
}

func (task *Task) Rename(title string) error {
  if task == nil {
    return ErrTaskIsNil
  }

  if len(title) == 0 {
    return ErrEmptyTitle
  }

  if len(title) > 120 {
    return ErrTitleTooLong
  }

  task.Title = title
```

```
  return nil}func main() {   task := Task{ID: 1, Title: "Купить молоко", Priority: 2}
if err := task.Rename("Купить продукты"); err != nil {    fmt.Println("Ошибка:", err)
return } task.Complete()   fmt.Println(task.Title, task.Completed)}
```

Возврат `nil` означает, что операция выполнена успешно.

> **Не игнорируйте error**
>
> Если функция возвращает ошибку, не присваивайте её в `_` только ради того, чтобы скомпилировать код. Сначала решите, что должно произойти при неудаче: вернуть ошибку выше, показать сообщение, повторить операцию или выбрать другое значение.

Контекст ошибки: `fmt.Errorf`

Иногда одного сообщения причины недостаточно. Добавьте контекст через `fmt.Errorf`:

```
func renameTask(task *Task, title string) error {
  if task == nil {
    return fmt.Errorf("не удалось обновить задачу: %w", ErrTaskIsNil)
  }

  if err := task.Rename(title); err != nil {
    return fmt.Errorf("не удалось обновить задачу %d: %w", task.ID, err)
  }

  return nil
}
```

`%w` оборачивает исходную ошибку. Получившийся текст будет содержать и контекст, и причину:

```
не удалось обновить задачу 1: название задачи не должно быть пустым
```

Используйте `%w`, когда вышестоящему коду может понадобиться узнать исходную причину. Обычный `%v` только вставляет текст ошибки и не сохраняет связь для `errors.Is`.

Проверка `task == nil` находится перед обращением к `task.ID`: это позволяет безопасно обернуть и такую ошибку.

Проверка причины: `errors.Is`

Не сравнивайте текст ошибки: он предназначен для человека и может измениться. Если ошибка обёрнута, не всегда сработает и `err == ErrEmptyTitle`.

Используйте `errors.Is`:

```
err := renameTask(&Task{ID: 1}, "")

if errors.Is(err, ErrEmptyTitle) {
  fmt.Println("Попросите пользователя указать название")
}
```

`errors.Is` ищет указанную ошибку внутри цепочки, которую создал `%w`.

Полный пример:

```
func renameTask(task *Task, title string) error {
  if task == nil {
    return fmt.Errorf("не удалось обновить задачу: %w", ErrTaskIsNil)
  }

  if err := task.Rename(title); err != nil {
    return fmt.Errorf("не удалось обновить задачу %d: %w", task.ID, err)
  }

  return nil
}

func main() {
  task := &Task{ID: 1, Title: "Купить молоко", Priority: 2}

  err := renameTask(task, "")
  if err == nil {
    return
  }

  if errors.Is(err, ErrEmptyTitle) {
    fmt.Println("Введите название задачи")
    return
  }

  fmt.Println("Неожиданная ошибка:", err)
}
```

Когда возвращать error

Возвращайте `error`, когда вызывающий код должен узнать, что операция не выполнилась, и может на это повлиять. Например:

- входные данные не прошли валидацию;
- задача не найдена по ID;
- не удалось прочитать файл;
- внешний сервис или база данных вернули проблему.

Не используйте `error` для обычной ветви успешной логики. Например, метод `Summary() string` не должен возвращать ошибку, если он всегда умеет собрать строку из полей структуры.

Частые ошибки

Сообщение вместо проверки ошибки

```
if err.Error() == "название задачи не должно быть пустым" {
  // Хрупко: текст может измениться.
}
```

Для известных причин используйте `errors.Is(err, ErrEmptyTitle)`.

`%v` вместо `%w` при оборачивании

```
// return fmt.Errorf("не удалось переименовать: %v", err)
return fmt.Errorf("не удалось переименовать: %w", err)
```

`%w` сохраняет исходную ошибку для `errors.Is`.

Продолжение после ошибки

```
if err := task.Rename(""); err != nil {
  fmt.Println(err)
}

fmt.Println(task.Title) // Код продолжится, хотя операция не выполнилась.
```

Если дальнейший код зависит от успеха операции, после обработки ошибки сделайте `return`, `continue` или другой явный переход.

Panic вместо ошибки в обычной валидации

Пустое название — ожидаемая ошибка пользовательского ввода, а не авария программы. Возвращайте `error`; `panic` оставьте для действительно невозможных состояний или внутренних ошибок библиотек. `panic`, `defer` и `recover` разберём отдельно, когда появится практический контекст.

Дальше

Закрепите `Rename` с ошибками на практике. Затем перейдём к пакетам и модулям, чтобы разнести Task Manager API по отдельным файлам и пакетам.

Самопроверка

1. Какой тип обычно возвращают последним результатом функции, которая может завершиться неудачей?

A. bool всегда

B. string всегда

C. panic

D. error

2. Какое значение error означает успешное выполнение операции?

A. ""

B. 0

C. nil

D. true

3. Как создать простую ошибку с постоянным сообщением?

A. new(error, "сообщение")

B. errors.New("сообщение")

C. error.New("сообщение")

D. panic("сообщение")

4. Какой паттерн используют сразу после вызова функции, возвращающей error?

A. if error { ... }

B. switch nil { ... }

C. if err != nil { ... }

D. if err == false { ... }

5. Что делает return внутри ветви обработки ошибки?

A. Удаляет ошибку

B. Всегда завершает всю программу

C. Повторяет вызов функции

D. Завершает текущую функцию и не даёт продолжить успешный путь

6. Как добавить контекст к ошибке и сохранить её для errors.Is?

A. fmt.Errorf("контекст: %v", err)

B. errors.New("контекст: %w")

C. fmt.Errorf("контекст: %w", err)

D. fmt.Printf("контекст: %w", err)

7. Зачем используют errors.Is(err, ErrEmptyTitle)?

A. Чтобы создать новую ошибку

B. Чтобы вывести текст ошибки в терминал

C. Чтобы превратить error в string

D. Чтобы проверить известную причину, включая обёрнутую ошибку

8. Почему не стоит сравнивать err.Error() с текстом?

A. errors.Is работает только с числами

B. Текст предназначен для человека и может измениться

C. У error нет метода Error

D. Строки нельзя сравнивать в Go

9. Что должен вернуть Rename при успешном переименовании?

A. nil

B. ErrEmptyTitle

C. false

D. panic

10. Почему пустое название задачи не стоит обрабатывать через panic?

A. panic нельзя вызывать в Go

B. Пустые строки автоматически становятся корректными

C. Это ожидаемая ошибка ввода, которую вызывающий код может обработать

D. panic возвращает nil

Ответы к самопроверке

1. error

2. nil

3. errors.New("сообщение")

4. if err != nil { ... }

5. Завершает текущую функцию и не даёт продолжить успешный путь

6. fmt.Errorf("контекст: %w", err)

7. Чтобы проверить известную причину, включая обёрнутую ошибку

8. Текст предназначен для человека и может измениться

9. nil

10. Это ожидаемая ошибка ввода, которую вызывающий код может обработать

Задания: ошибки при переименовании задачи

В этой практике `Rename` возвращает `error` вместо `bool`. Вызывающий код получает не только факт неудачи, но и её причину.

Задание 1. Ошибки в Rename

Объявите ошибки `ErrTaskIsNil`, `ErrEmptyTitle` и `ErrTitleTooLong`. Напишите метод `Rename(title string) error` для `*Task`.

Метод должен вернуть:

- `ErrTaskIsNil`, если получатель равен `nil`;
- `ErrEmptyTitle`, если строка пуста;
- `ErrTitleTooLong`, если длина превышает 120 байтов;
- `nil`, если название успешно изменено.

Критерии проверки:

- ошибки создаются через `errors.New`;
- метод использует pointer receiver;
- исходное название меняется только при `nil`-ошибке;
- успешный вызов возвращает `nil`;
- некорректный вызов не вызывает panic.

Подсказка

Сигнатура метода:

```
func (task *Task) Rename(title string) error {
  // ...
}
```

Решение

```
package main

import (
  "errors"
  "fmt"
)

var (
  ErrTaskIsNil    = errors.New("задача не существует")
  ErrEmptyTitle   = errors.New("название задачи не должно быть пустым")
  ErrTitleTooLong = errors.New("название задачи слишком длинное")
)

type Task struct {
  Title string
}

func (task *Task) Rename(title string) error {
  if task == nil {
    return ErrTaskIsNil
  }

  if len(title) == 0 {
    return ErrEmptyTitle
  }

  if len(title) > 120 {
    return ErrTitleTooLong
  }

  task.Title = title
  return nil
}
```

▼ Сведения

```
func main() {   task := Task{Title: "Купить молоко"}  if err := task.Rename("Купить продукты"); err != nil {     fmt.Println(err)    return } fmt.Println(task.Title)}
```

Задание 2. Добавьте контекст

Напишите функцию `renameTask(task *Task, title string) error`. Она должна вызывать `task.Rename(title)`. При ошибке верните её с контекстом `не удалось обновить задачу <ID>` через `%w`.

Для задания добавьте в `Task` поле `ID int`.

Критерии проверки:

- вызывается `task.Rename(title)`;
- при ошибке используется `fmt.Errorf` и `%w`;
- при успехе возвращается `nil`;
- текст ошибки содержит ID задачи и первоначальную причину.

Подсказка

```
if err := task.Rename(title); err != nil {
  return fmt.Errorf("не удалось обновить задачу %d: %w", task.ID, err)
}
```

Решение

```
func renameTask(task *Task, title string) error {
  if task == nil {
    return fmt.Errorf("не удалось обновить задачу: %w", ErrTaskIsNil)
  }

  if err := task.Rename(title); err != nil {
    return fmt.Errorf("не удалось обновить задачу %d: %w", task.ID, err)
  }

  return nil
}
```

Эту функцию нужно добавить к коду из первого задания, где уже импортирован `fmt`.

Задание 3. Обработайте известную причину

Создайте задачу с ID `1` и попытайтесь переименовать её в пустую строку через `renameTask`. Если причина — `ErrEmptyTitle`, выведите:

```
Введите название задачи
```

Для других ошибок выведите исходный текст ошибки.

Критерии проверки:

- результат `renameTask` сохраняется в `err`;
- известная причина проверяется через `errors.Is`;
- не используется сравнение `err.Error()` со строкой;
- при пустом названии выводится нужное сообщение.

Подсказка

```
if errors.Is(err, ErrEmptyTitle) {
  fmt.Println("Введите название задачи")
}
```

Решение

```
package main

import (
  "errors"
  "fmt"
)

var ErrEmptyTitle = errors.New("название задачи не должно быть пустым")

type Task struct {
  ID    int
  Title string
}

func (task *Task) Rename(title string) error {
  if len(title) == 0 {
    return ErrEmptyTitle
  }

  task.Title = title
  return nil
}

func renameTask(task *Task, title string) error {
  if err := task.Rename(title); err != nil {
    return fmt.Errorf("не удалось обновить задачу %d: %w", task.ID, err)
  }

  return nil
}

func main() {
  task := &Task{ID: 1, Title: "Купить молоко"}
  err := renameTask(task, "")
```

▼ Сведения

```
  if errors.Is(err, ErrEmptyTitle) {    fmt.Println("Введите название задачи")
return } if err != nil {     fmt.Println(err)  }}
```

Задание 4. Сохраните задачу без ошибки

Добавьте к предыдущему коду успешный сценарий: вызовите `renameTask(task, "Купить продукты")`. Если ошибки нет, выведите новое название. Убедитесь, что проверка на ошибку выполняется до доступа к результату операции.

Критерии проверки:

- успешный вызов возвращает `nil`;
- новое название сохранено в `task.Title`;
- ошибка проверяется через `if err != nil`;
- успешный путь не вложен в большой `else`.

Подсказка

```
if err := renameTask(task, "Купить продукты"); err != nil {
  fmt.Println(err)
  return
}

fmt.Println(task.Title)
```

Проверьте себя

Перед следующей главой убедитесь, что вы можете:

- вернуть `error` как последний результат функции или метода;
- вернуть `nil` при успехе;
- создать повторно используемую ошибку через `errors.New`;
- обработать ошибку через ранний `return`;
- добавить контекст через `fmt.Errorf` и `%w`;
- проверить причину через `errors.Is`.

Рекомендации по коду: ошибки

- Возвращайте `error` последним результатом функции, если операция может не выполниться.
- Проверяйте ошибку сразу и используйте ранний `return`, чтобы не строить глубокие вложенности.
- Создавайте известные причины как переменные `Err...`, если вызывающему коду нужно их распознавать.
- Добавляйте контекст через `fmt.Errorf` и `%w`, не теряя исходную причину.
- Проверяйте причины через `errors.Is`, а не сравнением текста `err.Error()`.
- Не используйте `panic` для обычных ошибок валидации, файловой системы или сети.

Чек-лист

Ошибка содержит полезный контекст, известные причины доступны для `errors.Is`, а после неудачи код не продолжает успешный сценарий по ошибке.

`defer`, `panic` и `recover`

Цель урока

Научиться гарантированно освобождать ресурсы через `defer`, понимать разницу между ожидаемой ошибкой и `panic`, а также применять `recover` только на чёткой границе аварийного сценария.

Главное практическое применение `defer` в Task Manager API — закрытие файлов. `panic` и `recover` нужны гораздо реже: они не заменяют обычную обработку `error`.

`defer`: выполнить перед выходом из функции

`defer` откладывает вызов до момента, когда текущая функция завершится — обычным `return` или из-за panic:

```
package main

import "fmt"

func main() {
  defer fmt.Println("Выполнится последним")

  fmt.Println("Выполнится первым")
}
```

Вывод:

```
Выполнится первым
Выполнится последним
```

`defer` полезен для действий, которые должны произойти независимо от того, по какому пути функция завершится:

- закрыть файл;
- разблокировать mutex;
- вызвать `cancel` у context;
- вернуть соединение в пул;
- записать завершающий лог.

Закрытие файла через `defer`

После успешного открытия файла сразу запланируйте закрытие:

```
func readExport(path string) error {
  file, err := os.Open(path)
  if err != nil {
    return fmt.Errorf("не удалось открыть экспорт: %w", err)
  }
  defer file.Close()

  content, err := io.ReadAll(file)
  if err != nil {
    return fmt.Errorf("не удалось прочитать экспорт: %w", err)
  }

  fmt.Print(string(content))
  return nil
}
```

Если `io.ReadAll` вернёт ошибку, `return` завершит функцию, а затем Go вызовет `file.Close()`.

Важно ставить `defer file.Close()` **только после** проверки ошибки `os.Open`: до успешного открытия `file` может быть `nil`.

Значения аргументов вычисляются сразу

Аргументы отложенного вызова вычисляются в момент `defer`, а не в момент выполнения:

```
func main() {
  name := "Купить молоко"
  defer fmt.Println(name)

  name = "Купить продукты"
}
```

Вывод будет:

```
Купить молоко
```

Если нужен актуальный результат на момент выхода, используйте отложенную анонимную функцию:

```
defer func() {
  fmt.Println(name)
}()
```

Теперь будет выведено `Купить продукты`, потому что переменная читается внутри функции во время её запуска.

Несколько `defer`: порядок LIFO

Несколько отложенных вызовов выполняются в обратном порядке — последний добавленный выполняется первым:

```
func main() {
  defer fmt.Println("Первый defer")
  defer fmt.Println("Второй defer")
  defer fmt.Println("Третий defer")
}
```

Вывод:

```
Третий defer
Второй defer
Первый defer
```

Это правило называют LIFO: last in, first out. Оно естественно освобождает ресурсы в обратном порядке их захвата.

Ошибка `Close`

Для чтения ошибка при закрытии файла встречается редко, поэтому в простом коде часто достаточно `defer file.Close()`. При записи `Close` может вернуть важную ошибку: например, операционная система может обнаружить проблему при финальной записи буферов.

Если ошибку закрытия нужно вернуть, используйте именованный результат и отложенную функцию:

```
func writeExport(path, content string) (err error) {
  file, err := os.Create(path)
  if err != nil {
    return fmt.Errorf("не удалось создать экспорт: %w", err)
  }
```

```
  defer func() {    closeErr := file.Close()     if err == nil && closeErr != nil {
err = fmt.Errorf("не удалось закрыть экспорт: %w", closeErr)    } }()  if _, err :=
file.WriteString(content); err != nil {   return fmt.Errorf("не удалось записать
экспорт: %w", err)  } return nil}
```

Логика важна:

1. если запись уже завершилась ошибкой, она остаётся основной причиной;
2. если запись успешна, но `Close` не удался, возвращается ошибка закрытия;
3. файл закрывается при любом `return` после `defer`.

Для небольшого экспорта через `os.WriteFile` закрытие управляется стандартной библиотекой, поэтому вручную применять этот паттерн не нужно.

`panic`

`panic` означает аварийную ситуацию: текущая функция прекращает нормальное выполнение, запускаются отложенные вызовы, затем panic поднимается вверх по стеку. Если её никто не восстановил, программа завершается со stack trace.

```
func main() {
  panic("неожиданное состояние")
}
```

Не используйте `panic` для ожидаемых ситуаций:

- пользователь передал пустое название;
- файл не существует;
- сеть недоступна;
- задача не найдена.

Во всех этих случаях вызывающий код может принять решение, поэтому возвращайте `error`.

Panic допустим только для действительно невозможного состояния программы или нарушения внутреннего инварианта, которое нельзя разумно обработать на текущем уровне. В прикладном коде Task Manager API такие случаи должны быть редкими.

`recover`

`recover` перехватывает активную panic, но работает только внутри функции, вызванной через `defer` в той же goroutine:

```
func safeRun() {
  defer func() {
    if recovered := recover(); recovered != nil {
      fmt.Println("Перехвачена panic:", recovered)
```

```
    } }()  panic("ошибка демонстрации")}
```

После `recover` функция, в которой произошла panic, не продолжает выполнение с места panic. Управление возвращается из неё после выполнения отложенных функций.

> **Не превращайте recover в обычный error handling**
>
> `recover` не должен окружать каждую функцию и скрывать ошибки. Он полезен на ограниченной границе: например, в верхнем уровне сервера, в изолированной фоновой задаче или в библиотеке, которая защищает внешний API. Причину panic нужно залогировать и расследовать.

Panic и goroutine

Panic в одной goroutine не может быть перехвачена `recover` из другой goroutine:

```
func main() {
  defer func() {
    recover() // Не перехватит panic из go func().
  }()

  go func() {
    panic("авария в goroutine")
  }()
}
```

Если отдельная goroutine действительно должна быть защищена от panic, `recover` нужно разместить внутри неё. Но сначала исправьте причину panic: recover — последний защитный слой, а не замена корректному коду.

Task Manager API: безопасная запись экспорта

Для записи небольшого `tasks.txt` удобнее `os.WriteFile`. Но при работе с открытым `*os.File` функция Task Manager API должна гарантированно закрывать его:

```
func writeTextExport(path, content string) (err error) {
  file, err := os.Create(path)
  if err != nil {
    return fmt.Errorf("не удалось создать текстовый экспорт: %w", err)
  }

  defer func() {
    closeErr := file.Close()
    if err == nil && closeErr != nil {
```

```
      err = fmt.Errorf("не удалось закрыть текстовый экспорт: %w", closeErr)   } }()
if _, err := file.WriteString(content); err != nil {   return fmt.Errorf("не удалось
записать текстовый экспорт: %w", err) } return nil}
```

Эта функция не использует panic: ошибки файловой системы ожидаемы и должны вернуться вызывающему коду как `error`.

Частые ошибки

`defer` до проверки ошибки

```
file, err := os.Open(path)
defer file.Close() // file может быть nil

if err != nil {
  return err
}
```

Правильно сначала проверить `err`, затем отложить закрытие.

Забытый `Unlock`

```
store.mu.Lock()
if invalid {
  return // мьютекс остался заблокирован
}
store.mu.Unlock()
```

Сразу после успешного `Lock` используйте `defer store.mu.Unlock()`.

`recover` вне defer

```
recovered := recover() // ничего не перехватит
```

`recover` работает только в отложенной функции во время раскручивания panic.

Panic для ошибки валидации

```
if title == "" {
  panic("пустое название")
}
```

Верните `ErrEmptyTitle`: пользовательский ввод — ожидаемая ошибка, а не авария процесса.

Дальше

Закрепите безопасное закрытие файлов и границы применения panic. Затем перейдём к пакетам и модулям, чтобы продолжить организацию Task Manager API.

Самопроверка

1. Когда выполняется отложенный вызов defer?

A. После завершения всей программы

B. Сразу в момент объявления defer

C. Перед завершением текущей функции

D. Только при panic

2. Когда стоит написать defer file.Close()?

A. До вызова os.Open

B. Только после io.ReadAll

C. Файл никогда не нужно закрывать

D. Сразу после успешной проверки ошибки открытия файла

3. В каком порядке выполнятся три defer, объявленные один за другим?

A. Только первый defer

B. В случайном порядке

C. В том же порядке объявления

D. В обратном порядке объявления

4. Когда вычисляется аргумент defer fmt.Println(name)?

A. После закрытия всех файлов

B. Только после изменения name

C. Никогда, defer не принимает аргументы

D. В момент объявления defer

5. Что такое panic?

A. Альтернатива return nil

B. Обычная ошибка валидации

C. Аварийное завершение нормального выполнения с раскруткой стека

D. Способ создать goroutine

6. Что следует вернуть при пустом пользовательском названии задачи?

A. panic

B. Обычный error, например ErrEmptyTitle

C. Закрытый канал

D. recover

7. Где должен находиться recover, чтобы перехватить panic?

A. В любом месте main после panic

B. В другой goroutine

C. В блоке if err != nil

D. В функции, вызванной через defer в той же goroutine

8. Перехватит ли recover в main panic из другой goroutine?

A. Только если panic содержит error

B. Нет, recover работает только в той же goroutine

C. Да, всегда

D. Только при использовании WaitGroup

9. Почему при записи файла иногда важно обработать ошибку Close?

A. Close создаёт JSON

B. os.Create не возвращает файл до Close

C. Ошибка может возникнуть при финальной записи данных в файл

D. Close всегда удаляет файл

10. Какой паттерн корректен после mu.Lock()?

A. defer mu.Lock()

B. close(mu)

C. recover(mu)

D. defer mu.Unlock()

Ответы к самопроверке

1. Перед завершением текущей функции

2. Сразу после успешной проверки ошибки открытия файла

3. В обратном порядке объявления

4. В момент объявления defer

5. Аварийное завершение нормального выполнения с раскруткой стека

6. Обычный error, например ErrEmptyTitle

7. В функции, вызванной через defer в той же goroutine

8. Нет, recover работает только в той же goroutine

9. Ошибка может возникнуть при финальной записи данных в файл

10. defer mu.Unlock()

Задания: безопасная работа с файлами

В этой практике используем `defer` для гарантированного закрытия файлов. Обычные проблемы файловой системы по-прежнему возвращаем как `error`, а не через `panic`.

Задание 1. Закройте файл после чтения

Напишите функцию:

```
func readTextExport(path string) error
```

Она должна открыть `tasks.txt`, отложить `Close`, прочитать данные через `io.ReadAll` и вывести их в терминал.

Критерии проверки:

- `os.Open` проверяется на ошибку;
- `defer file.Close()` находится после проверки ошибки;
- `io.ReadAll(file)` проверяется на ошибку;
- ошибки получают контекст через `fmt.Errorf` и `%w`;
- файл закрывается при успешном чтении и при раннем `return`.

Подсказка

```
file, err := os.Open(path)
if err != nil {
  return fmt.Errorf("не удалось открыть экспорт: %w", err)
}
defer file.Close()
```

Решение

```
package main

import (
  "fmt"
  "io"
  "os"
)

func readTextExport(path string) error {
  file, err := os.Open(path)
  if err != nil {
    return fmt.Errorf("не удалось открыть экспорт: %w", err)
  }
  defer file.Close()

  content, err := io.ReadAll(file)
  if err != nil {
    return fmt.Errorf("не удалось прочитать экспорт: %w", err)
  }

  fmt.Print(string(content))
  return nil
}
```

Задание 2. Верните ошибку закрытия

Напишите функцию:

```
func writeTextExport(path, content string) (err error)
```

Она создаёт файл, записывает текст и корректно возвращает ошибку `Close`, если запись прошла успешно, но закрыть файл не удалось.

Критерии проверки:

- используется именованный результат `err error`;
- файл создаётся через `os.Create`;
- `Close` вызывается в отложенной анонимной функции;
- ошибка `Close` заменяет результат только если `err == nil`;
- ошибка записи получает контекст и не теряется.

Подсказка

```
defer func() {
  closeErr := file.Close()
  if err == nil && closeErr != nil {
    err = fmt.Errorf("не удалось закрыть экспорт: %w", closeErr)
  }
}()
```

Решение

```
func writeTextExport(path, content string) (err error) {
  file, err := os.Create(path)
  if err != nil {
    return fmt.Errorf("не удалось создать экспорт: %w", err)
  }

  defer func() {
    closeErr := file.Close()
    if err == nil && closeErr != nil {
      err = fmt.Errorf("не удалось закрыть экспорт: %w", closeErr)
    }
  }()

  if _, err := file.WriteString(content); err != nil {
    return fmt.Errorf("не удалось записать экспорт: %w", err)
  }

  return nil
}
```

Задание 3. Используйте defer в TaskStore

Вспомните `TaskStore`. Добавьте или перепишите метод `Complete`, чтобы он использовал `defer store.mu.Unlock()` сразу после `store.mu.Lock()`.

Критерии проверки:

- мьютекс блокируется до доступа к map;
- `defer store.mu.Unlock()` расположен сразу после `Lock`;
- отсутствующая задача возвращает ошибку без ручного `Unlock`;
- успешное изменение также освобождает мьютекс.

Подсказка

```
store.mu.Lock()
defer store.mu.Unlock()
```

Решение

```
func (store *TaskStore) Complete(id int) error {
  store.mu.Lock()
  defer store.mu.Unlock()

  currentTask, exists := store.tasks[id]
  if !exists {
    return ErrTaskNotFound
  }

  currentTask.Complete()
  store.tasks[id] = currentTask
  return nil
}
```

Задание 4. Локально перехватите panic

Напишите функцию `safeDemo()`, которая намеренно вызывает `panic("ошибка демонстрации")`, но перехватывает её в отложенной функции и выводит сообщение `Перехвачена panic: ошибка демонстрации`.

Это демонстрация механики `recover`. Не применяйте этот подход для `Rename`, файловых ошибок или валидации задач.

Критерии проверки:

- `recover` находится внутри `defer func() { ... }()`;
- вызов `panic` находится в той же функции;
- программа продолжает работу после `safeDemo()`;
- `recover` не используется для обычного `error`.

Подсказка

```
defer func() {
  if recovered := recover(); recovered != nil {
    fmt.Println("Перехвачена panic:", recovered)
  }
}()
```

Проверьте себя

Перед следующей главой убедитесь, что вы можете:

- отложить закрытие ресурса через `defer`;
- объяснить порядок нескольких defer;
- не потерять важную ошибку `Close`;
- отличить ожидаемый `error` от аварийной panic;
- использовать `recover` только внутри deferred-функции той же goroutine;
- безопасно освобождать mutex через `defer Unlock`.

Рекомендации по коду: `defer`, `panic` и `recover`

- Ставьте `defer Close`, `defer Unlock` и `defer cancel` сразу после успешного захвата ресурса.
- Не откладывайте вызов до проверки ошибки создания ресурса: указатель может быть `nil`.
- Помните порядок LIFO, если несколько defer освобождают зависимые ресурсы.
- Возвращайте `error` для ожидаемых сбоев: некорректного ввода, отсутствия файла, сетевой или SQL-ошибки.
- Используйте `panic` только для нарушенного внутреннего инварианта или ситуации, которую текущий уровень не может корректно обработать.
- Размещайте `recover` только на явной границе и обязательно логируйте или возвращайте информацию о причине panic.

Чек-лист

Все открытые ресурсы освобождаются, panic не используется для валидации, а recover не скрывает ошибку, которую нужно исправить в коде.

Пакеты и модули

Цель урока

Научиться разделять Go-проект на пакеты, понимать роль `go.mod`, импортировать собственный код и вынести модель Task Manager API из `main` в пакет `internal/task`.

До этого все примеры помещались в один файл `main.go`. Такой подход удобен для обучения, но реальное приложение быстро становится сложным. Пакеты позволяют разделить код по ответственности.

Пакет и модуль — разные уровни

Пакет — набор Go-файлов одной папки с одинаковым объявлением `package`. Пакет объединяет связанную логику.

```
package task
```

Модуль — проект или часть проекта с файлом `go.mod`. Он задаёт module path и управляет зависимостями.

```
task-manager/
├── go.mod
├── main.go
└── internal/
    └── task/
        └── task.go
```

В этой структуре:

- `task-manager` — корень модуля;
- `main.go` — пакет `main`, точка входа программы;
- `internal/task` — отдельный пакет `task` с моделью задачи;
- `go.mod` описывает весь модуль.

Один модуль может содержать много пакетов. Один пакет может состоять из нескольких `.go`-файлов одной папки.

Файл `go.mod`

Создать модуль можно командой:

```
go mod init example.com/task-manager
```

Она создаст `go.mod`:

```
module example.com/task-manager

go 1.26.0
```

Строка версии зависит от установленной версии Go и может отличаться. Не редактируйте её без причины.

Module path нужен инструментам Go для определения импортов внутри проекта и для поиска внешних модулей. Для учебного проекта допустимо `example.com/task-manager`. Если репозиторий будет опубликован на GitHub, путь обычно совпадает с его URL, например `github.com/username/task-manager`.

Пакет `main`

Исполняемая программа должна иметь пакет `main` и функцию `main`:

```
package main

func main() {
  // Точка входа приложения.
}
```

Остальные пакеты обычно содержат модели, бизнес-логику, работу с хранением данных или HTTP-обработчики. Они не запускаются самостоятельно, а импортируются из `main` или других пакетов.

Создание пакета `internal/task`

Создадим такую структуру в папке проекта:

```
task-manager/
├── go.mod
├── main.go
└── internal/
    └── task/
        └── task.go
```

В `internal/task/task.go` объявим пакет и перенесём в него модель задачи:

```
package task

import (
  "errors"
  "fmt"
)

var (
  ErrTaskIsNil    = errors.New("задача не существует")
  ErrEmptyTitle   = errors.New("название задачи не должно быть пустым")
  ErrTitleTooLong = errors.New("название задачи слишком длинное")
)

type Task struct {
  ID        int
  Title     string
  Priority  int
  Completed bool
}

func (task Task) Summary() string {
  return fmt.Sprintf(
    "%d. %s | приоритет: %d | выполнена: %t",
    task.ID,
    task.Title,
    task.Priority,
    task.Completed,
  )
}

func (task *Task) Rename(title string) error {
  if task == nil {
    return ErrTaskIsNil
  }

  if len(title) == 0 {
    return ErrEmptyTitle
  }

  if len(title) > 120 {
    return ErrTitleTooLong
  }
```

```
  task.Title = title  return nil}
```

Имя папки `task` и объявление `package task` совпадают. Это распространённое и понятное соглашение. Технически имя пакета и папки могут отличаться, но без веской причины так делать не стоит.

Импорт собственного пакета

В корневом `main.go` импортируйте пакет по module path и относительному пути до его папки:

```
package main

import (
  "fmt"

  "example.com/task-manager/internal/task"
)

func main() {
  currentTask := task.Task{
    ID:       1,
    Title:    "Купить молоко",
    Priority: 2,
  }

  if err := currentTask.Rename("Купить продукты"); err != nil {
    fmt.Println("Ошибка:", err)
    return
  }

  fmt.Println(currentTask.Summary())
}
```

У импортированного пакета используются его имя и точка:

```
currentTask := task.Task{}
fmt.Println(task.ErrEmptyTitle)
```

Путь импорта не начинается с `./` или `../`. Инструменты Go используют module path из `go.mod`.

Экспортируемые имена

Код другого пакета видит только имена с заглавной буквы:

```
type Task struct {}      // доступен как task.Task
var ErrEmptyTitle error  // доступна как task.ErrEmptyTitle

func NewTask() Task {}   // доступна как task.NewTask
```

Имена со строчной буквы доступны только внутри своего пакета:

```
func validateTitle(title string) error {
  return nil
}
```

В `main.go` нельзя вызвать `task.validateTitle`. Это не ошибка стиля, а правило видимости языка.

Экспортируйте только то, что действительно нужно потребителям пакета. Например, `Task`, её методы и известные ошибки нужны `main`, а вспомогательная внутренняя проверка — нет.

Зачем папка `internal`

Папка `internal` ограничивает импорт её пакетов. Пакет `example.com/task-manager/internal/task` можно импортировать из кода внутри `example.com/task-manager`, но нельзя из другого независимого модуля.

Это хороший выбор для логики конкретного приложения: Task Manager API использует пакет сам, но не обещает его как публичную библиотеку.

```
example.com/task-manager/main.go                ← может импортировать internal/task
example.com/task-manager/cmd/server/main.go     ← может импортировать internal/task
another.example/app/main.go                      ← не может импортировать
internal/task
```

Несколько файлов одного пакета

Необязательно держать весь пакет в `task.go`. Когда код вырастет, его можно разделить:

```
internal/task/
├── task.go       # Task и конструкторы
├── validation.go # проверки и ошибки
└── methods.go    # методы Task
```

Во всех файлах первой строкой будет:

```
package task
```

Файлы одного пакета компилируются вместе. Не нужно импортировать один файл из другого — импортируют пакет, а не файл.

Полный пример структуры

`go.mod`:

```
module example.com/task-manager
```

`internal/task/task.go`:

```
package task

import (
  "errors"
  "fmt"
)

var ErrEmptyTitle = errors.New("название задачи не должно быть пустым")

type Task struct {
  ID        int
  Title     string
  Priority  int
  Completed bool
}

func (task Task) Summary() string {
  return fmt.Sprintf("%d. %s | приоритет: %d", task.ID, task.Title, task.Priority)
}

func (task *Task) Rename(title string) error {
  if len(title) == 0 {
    return ErrEmptyTitle
  }

  task.Title = title
  return nil
}
```

`main.go`:

```
package main

import (
  "errors"
  "fmt"

  "example.com/task-manager/internal/task"
)

func main() {
  currentTask := task.Task{ID: 1, Title: "Купить молоко", Priority: 2}

  if err := currentTask.Rename(""); err != nil {
    if errors.Is(err, task.ErrEmptyTitle) {
      fmt.Println("Введите название задачи")
      return
    }

    fmt.Println("Неожиданная ошибка:", err)
    return
  }

  fmt.Println(currentTask.Summary())
}
```

Запуск выполняется из корня модуля:

```
go run .
```

Частые ошибки

Неверный путь импорта

```
// import "./internal/task" // не используйте относительный путь
import "example.com/task-manager/internal/task"
```

Первая часть пути должна совпадать со строкой `module` в `go.mod`.

Импорт не используется

```
import "example.com/task-manager/internal/task"

func main() {}
```

Go не разрешит неиспользуемый импорт. Удалите его или используйте `task.Task`, `task.ErrEmptyTitle` и другие экспортируемые имена.

Имя начинается со строчной буквы

```
// task.validateTitle("Купить молоко") // недоступно из main
```

Если это часть публичного API пакета, назовите её `ValidateTitle`. Но не экспортируйте вспомогательную функцию только ради того, чтобы обойти правило видимости.

Разные package в одной папке

```
// internal/task/task.go
package task

// internal/task/validation.go
// package validation // ошибка: в одной папке должен быть один пакет
```

Оба файла должны использовать `package task`.

Дальше

Закрепите разделение Task Manager API на `main` и `internal/task`. Затем начнём работать с файлами, чтобы сохранять задачи между запусками программы.

Самопроверка

1. Что такое Go-модуль?

A. Только папка internal

B. Один Go-файл

C. Проект или часть проекта с файлом go.mod

D. Синоним функции main

2. Что объединяет Go-пакет?

A. Только внешние зависимости

B. Все Go-файлы компьютера

C. Go-файлы одной папки с одинаковым package

D. Только один файл task.go

3. Какая команда создаёт файл go.mod?

A. go build mod

B. go run init

C. go package init

D. go mod init example.com/task-manager

4. Какой package нужен исполняемой Go-программе?

A. program

B. app

C. internal

D. main

5. Как правильно импортировать internal/task из модуля example.com/task-manager?

A. "task.go"

B. "../internal/task"

C. "example.com/task-manager/internal/task"

D. "./internal/task"

6. Как обратиться к экспортируемому типу Task пакета task?

A. Task.task

B. task->Task

C. import.Task

D. task.Task

7. Какое имя доступно другому пакету?

A. ErrEmptyTitle

B. taskTitle

C. validateTitle

D. renameTask

8. Зачем используют папку internal?

A. Чтобы запретить импорт пакета из независимых внешних модулей

B. Чтобы автоматически экспортировать все имена

C. Чтобы ускорить компиляцию любого кода

D. Чтобы хранить только тесты

9. Можно ли держать Task и её методы в разных файлах папки internal/task?

A. Нет, тип и методы всегда должны быть в одном файле

B. Только с внешней зависимостью

C. Да, если во всех файлах указано package task

D. Только если используется map

10. Почему Go сообщает об ошибке imported and not used?

A. Импортированный пакет не используется в файле

B. Имя пакета начинается со строчной буквы

C. Модуль не содержит go.sum

D. В пакете нет функции main

Ответы к самопроверке

1. Проект или часть проекта с файлом go.mod

2. Go-файлы одной папки с одинаковым package

3. go mod init example.com/task-manager

4. main

5. "example.com/task-manager/internal/task"

6. task.Task

7. ErrEmptyTitle

8. Чтобы запретить импорт пакета из независимых внешних модулей

9. Да, если во всех файлах указано package task

10. Импортированный пакет не используется в файле

Задания: пакет internal/task

Разделим Task Manager API на точку входа и внутренний пакет модели. Все команды выполняйте из корня модуля `task-manager`.

Задание 1. Создайте структуру проекта

Создайте новый модуль `example.com/task-manager` с файлами:

```
task-manager/
├── go.mod
├── main.go
└── internal/
    └── task/
        └── task.go
```

В `internal/task/task.go` напишите `package task`. В `main.go` оставьте `package main`.

Критерии проверки:

- `go.mod` содержит `module example.com/task-manager`;
- папка `internal/task` существует;
- `task.go` и `main.go` имеют разные package, потому что находятся в разных папках;
- `go run .` из корня модуля запускается без ошибки, даже если `main` пока пустая.

Подсказка

В терминале из корня проекта:

```
mkdir -p internal/task
touch main.go internal/task/task.go
```

В Windows создайте папки и файлы через проводник или используйте команды PowerShell.

Минимальное решение

`main.go`:

```
package main

func main() {}
```

`internal/task/task.go`:

```
package task
```

Задание 2. Вынесите Task в пакет

Перенесите тип `Task` в `internal/task/task.go`. Добавьте метод `Summary() string` и ошибку `ErrEmptyTitle`.

Критерии проверки:

- тип называется `Task` и экспортируется;
- `Summary` экспортируется и использует `fmt.Sprintf`;
- `ErrEmptyTitle` экспортируется и создана через `errors.New`;
- в `main.go` больше нет объявления `Task`.

Подсказка

Понадобятся импорты:

```
import (
  "errors"
  "fmt"
)
```

Решение

`internal/task/task.go`:

```
package task

import (
  "errors"
  "fmt"
)

var ErrEmptyTitle = errors.New("название задачи не должно быть пустым")

type Task struct {
  ID        int
  Title     string
  Priority  int
  Completed bool
}

func (task Task) Summary() string {
  return fmt.Sprintf("%d. %s | приоритет: %d", task.ID, task.Title, task.Priority)
}
```

Задание 3. Импортируйте внутренний пакет

В `main.go` импортируйте `example.com/task-manager/internal/task`, создайте `task.Task` и выведите его `Summary()`.

Критерии проверки:

- путь импорта начинается с module path из `go.mod`;
- задача создаётся как `task.Task{...}`;
- `main.go` не дублирует тип `Task`;
- `go run .` выводит summary.

Подсказка

```
import (
  "fmt"

  "example.com/task-manager/internal/task"
)
```

Решение

`main.go`:

```
package main

import (
  "fmt"

  "example.com/task-manager/internal/task"
)

func main() {
  currentTask := task.Task{
    ID:       1,
    Title:    "Купить молоко",
    Priority: 2,
  }

  fmt.Println(currentTask.Summary())
}
```

Задание 4. Экспортируйте только нужное

Добавьте в пакет `task` неэкспортируемую функцию `validateTitle(title string) error`, которая возвращает `ErrEmptyTitle` для пустой строки. Затем добавьте экспортируемый метод `Rename(title string) error`, использующий эту функцию.

В `main.go` переименуйте задачу и обработайте `task.ErrEmptyTitle` через `errors.Is`. Не пытайтесь вызвать `task.validateTitle` из `main`.

Критерии проверки:

- `validateTitle` начинается со строчной буквы;
- `Rename` начинается с заглавной буквы и доступен из `main`;
- `main` может проверить `task.ErrEmptyTitle`;
- внутренняя проверка не вызывается напрямую из `main`.

Подсказка

В методе пакета:

```
if err := validateTitle(title); err != nil {
  return err
}
```

Возможное решение

Добавьте в `internal/task/task.go`:

```
func validateTitle(title string) error {
  if len(title) == 0 {
    return ErrEmptyTitle
  }

  return nil
}

func (task *Task) Rename(title string) error {
  if err := validateTitle(title); err != nil {
    return err
  }

  task.Title = title
  return nil
}
```

В `main.go`:

```
if err := currentTask.Rename(""); err != nil {
  if errors.Is(err, task.ErrEmptyTitle) {
    fmt.Println("Введите название задачи")
    return
  }

  fmt.Println(err)
}
```

Проверьте себя

Перед следующей главой убедитесь, что вы можете:

- объяснить разницу между модулем и пакетом;
- найти module path в `go.mod`;
- создать импорт собственного пакета;
- отличить экспортируемое имя от неэкспортируемого;
- объяснить назначение папки `internal`;
- разделить модель и запуск приложения по разным пакетам.

Рекомендации по коду: пакеты и модули

- Создавайте один `go.mod` на независимое приложение или библиотеку.
- Пусть package name коротко описывает ответственность и обычно совпадает с именем папки.
- Импортируйте пакет по module path, а не через `./` или `../`.
- Экспортируйте только API, который нужен другим пакетам; вспомогательные детали оставляйте со строчной буквы.
- Используйте `internal` для кода, который является деталью конкретного приложения, а не публичной библиотекой.
- Не дробите проект на много пакетов преждевременно: одна связная область логики может жить в одном пакете.

Чек-лист

Зависимости пакетов направлены понятно, `main` не содержит модель и бизнес-логику, а внутренние детали не становятся экспортируемым API без причины.

Тестирование

Цель урока

Научиться писать unit-тесты с пакетом `testing`, запускать их через `go test`, использовать table-driven tests и subtests, а также проверять известные ошибки через `errors.Is`.

До этой главы Task Manager API проверяли вручную запуском `go run .`. Это полезно, но не гарантирует, что изменение кода не сломало уже работающие правила. Тесты фиксируют ожидаемое поведение и запускаются автоматически.

Где живут тесты

Тестовый файл заканчивается на `_test.go` и обычно находится рядом с кодом, который проверяет:

```
internal/
├── task/
│   ├── task.go
│   └── task_test.go
└── store/
    ├── store.go
    └── store_test.go
```

Файлы с суффиксом `_test.go` не входят в обычную сборку приложения, но используются командой `go test`.

Первый тест

Тест — функция с именем `Test...`, принимающая `*testing.T`:

```
package task

import "testing"

func TestTaskIsValid(t *testing.T) {
  currentTask := Task{
    ID:       1,
    Title:    "Купить молоко",
    Priority: 2,
  }
```

```
  if !currentTask.IsValid() {    t.Fatal("ожидалась корректная задача")  }}
```

Запуск тестов текущего пакета:

```
go test
```

Запуск тестов всего модуля:

```
go test ./...
```

`./...` означает текущий пакет и все вложенные пакеты модуля. Это основной вариант для проверки проекта целиком.

`t.Error`, `t.Fatal` и их варианты

`testing.T` предоставляет несколько способов сообщить о проблеме:

Метод	Поведение
`t.Error(...)`	Помечает тест неуспешным и продолжает его выполнение
`t.Errorf(...)`	То же, но с форматированием
`t.Fatal(...)`	Помечает тест неуспешным и завершает текущий тест сразу
`t.Fatalf(...)`	То же, но с форматированием

Используйте `t.Fatal` для условия, без которого дальнейшие проверки бессмысленны:

```
currentTask, exists := store.Get(1)
if !exists {
  t.Fatal("задача должна существовать")
}

if !currentTask.Completed {
  t.Error("задача должна быть выполненной")
}
```

Если задача не найдена, доступ к её полям уже не имеет смысла. Если поле просто имеет неверное значение, тест может сообщить и о других проблемах той же проверки.

Проверка результата

Хороший тест содержит три логические части:

1. **Arrange** — подготовить данные;
2. **Act** — вызвать тестируемую операцию;

3. **Assert** — проверить результат.

```
func TestTaskRename(t *testing.T) {
  // Arrange
  currentTask := Task{ID: 1, Title: "Купить молоко", Priority: 2}

  // Act
  err := currentTask.Rename("Купить продукты")

  // Assert
  if err != nil {
    t.Fatalf("Rename вернул ошибку: %v", err)
  }

  if currentTask.Title != "Купить продукты" {
    t.Errorf("Title = %q, ожидалось %q", currentTask.Title, "Купить продукты")
  }
}
```

Сообщение об ошибке должно показывать фактическое и ожидаемое значение. Это ускоряет диагностику, когда тест не проходит.

Table-driven tests

Когда одна функция должна корректно обработать несколько похожих сценариев, используйте таблицу тестовых случаев:

```
func TestTaskRename(t *testing.T) {
  testCases := []struct {
    name      string
    title     string
    wantTitle string
    wantErr   error
  }{
    {
      name:      "корректное название",
      title:     "Купить продукты",
      wantTitle: "Купить продукты",
    },
    {
      name:      "пустое название",
      title:     "",
      wantTitle: "Купить молоко",
      wantErr:   ErrEmptyTitle,
```

```
    },  } for _, testCase := range testCases {    t.Run(testCase.name, func(t
*testing.T) {       currentTask := Task{ID: 1, Title: "Купить молоко", Priority: 2}
err := currentTask.Rename(testCase.title)     if !errors.Is(err, testCase.wantErr) {
t.Fatalf("Rename error = %v, ожидалось %v", err, testCase.wantErr)     }     if
currentTask.Title != testCase.wantTitle {        t.Errorf("Title = %q, ожидалось %q",
currentTask.Title, testCase.wantTitle)      }   })  }}
```

Таблица делает набор сценариев компактным: добавление нового случая не требует копировать весь тест.

Subtests: `t.Run`

`t.Run` создаёт вложенный тест с понятным именем:

```
t.Run("пустое название", func(t *testing.T) {
  // Проверка конкретного сценария.
})
```

Вывод команды покажет, какой именно сценарий сломался:

```
--- FAIL: TestTaskRename (0.00s)
    --- FAIL: TestTaskRename/пустое_название (0.00s)
```

Можно запустить один тест или один subtest:

```
go test -run '^TestTaskRename$'
go test -run 'TestTaskRename/пустое_название'
```

Проверка ошибок через `errors.Is`

Не сравнивайте текст ошибки и не используйте `err == wantErr`, если ошибка может быть обёрнута:

```
if !errors.Is(err, ErrEmptyTitle) {
  t.Fatalf("ожидалась ErrEmptyTitle, получено: %v", err)
}
```

`errors.Is` работает и с исходной ошибкой, и с ошибкой внутри цепочки `%w`.

Для успешного случая `wantErr` равен `nil`; `errors.Is(nil, nil)` возвращает `true`, поэтому один и тот же шаблон подходит для обоих вариантов.

Тестирование Task

В `internal/task/task_test.go` можно проверить правила модели:

```
package task

import (
  "errors"
  "testing"
)

func TestTaskRename(t *testing.T) {
  currentTask := Task{ID: 1, Title: "Купить молоко", Priority: 2}

  err := currentTask.Rename("   ")
  if !errors.Is(err, ErrEmptyTitle) {
    t.Fatalf("Rename error = %v, ожидалась ErrEmptyTitle", err)
  }

  if currentTask.Title != "Купить молоко" {
    t.Errorf("Title изменился после ошибки: %q", currentTask.Title)
  }
}
```

Тест проверяет не только ошибку, но и важное свойство: неудачное переименование не должно менять исходную задачу.

Простой Store для unit-тестов

До главы о mutex используем простой Store без конкурентного доступа:

```
package store

import (
  "errors"

  "example.com/task-manager/internal/task"
)

var (
  ErrTaskNotFound      = errors.New("задача не найдена")
  ErrTaskAlreadyExists = errors.New("задача с таким ID уже существует")
)
```

```
type Store struct { tasks map[int]task.Task}func New() *Store {  return &Store{tasks:
make(map[int]task.Task)}}func (store *Store) Add(currentTask task.Task) error { if _,
exists := store.tasks[currentTask.ID]; exists {    return ErrTaskAlreadyExists  }
store.tasks[currentTask.ID] = currentTask  return nil}func (store *Store) Get(id int)
(task.Task, bool) { currentTask, exists := store.tasks[id]  return currentTask,
exists}
```

Этот Store предназначен для последовательного кода. В главе о mutex он будет заменён или расширен потокобезопасным `TaskStore`.

Тестирование Store

Проверим добавление и получение задачи:

```
func TestStoreAddAndGet(t *testing.T) {
  store := New()
  wantTask := task.Task{ID: 1, Title: "Купить молоко", Priority: 2}

  if err := store.Add(wantTask); err != nil {
    t.Fatalf("Add вернул ошибку: %v", err)
  }

  gotTask, exists := store.Get(1)
  if !exists {
    t.Fatal("задача не найдена после Add")
  }

  if gotTask != wantTask {
    t.Errorf("Get = %+v, ожидалось %+v", gotTask, wantTask)
  }
}
```

Структура `Task` в текущей версии содержит сравнимые поля, поэтому её можно сравнить целиком через `!=`. Когда в модели появится несравнимое поле, например слайс тегов, сравнение придётся сделать по отдельным полям или использовать `reflect.DeepEqual` для простого теста. Более специализированные инструменты сравнения можно добавить позже отдельным решением о зависимости.

Покрытие

Покрытие показывает, какие строки выполнялись во время тестов:

```
go test -cover ./...
```

Покрытие — подсказка, а не цель. Высокий процент не гарантирует проверку граничных условий, а низкий процент не означает автоматически плохой код. Важнее тестировать:

- правила бизнеса;
- ошибки и границы;
- код, который легко сломать при изменении;
- уже найденные ранее баги.

Race detector

Текущий простой Store не должен использоваться несколькими goroutine одновременно. Когда появится потокобезопасный `TaskStore`, запускайте весь набор с детектором гонок:

```
go test -race ./...
```

Эта команда особенно важна для тестов конкурентного кода, но полезна и для всего приложения.

Частые ошибки

Тест без проверки

```
func TestTaskRename(t *testing.T) {
  currentTask := Task{Title: "Купить молоко"}
  currentTask.Rename("Купить продукты")
}
```

Такой тест всегда успешен, если код не вызывает panic. Проверяйте ошибку и итоговое состояние.

Общие изменяемые данные между сценариями

```
store := New()

for _, testCase := range testCases {
  // Все случаи меняют один Store и влияют друг на друга.
}
```

Создавайте новое состояние внутри каждого `t.Run`, если сценарии должны быть независимыми.

Сравнение текста ошибки

```
if err.Error() != "название задачи не должно быть пустым" {
  // Хрупкая проверка.
```

```
}
```

Проверяйте причину через `errors.Is`.

Тестирование деталей вместо поведения

Тест не обязан проверять каждую строку реализации. Проверяйте контракт: что метод вернул, как изменил состояние, какую ошибку сообщил и какие побочные эффекты допустил.

Дальше

Закрепите unit-тесты модели и Store. Следующая глава добавит файловый экспорт и покажет `t.TempDir` для безопасных тестов операций с файлами.

Самопроверка

1. Как должен называться файл с тестами Go?

A. Он должен заканчиваться на _test.go

B. Он должен находиться только в папке tests

C. Он должен заканчиваться на .spec.go

D. Он должен иметь расширение .test

2. Какой вид имеет обычная тестовая функция?

A. func TestSomething(t *testing.T)

B. test TestSomething(t)

C. func testSomething()

D. func TestingSomething()

3. Как запустить все тесты текущего модуля и вложенных пакетов?

A. go build test

B. go run ./...

C. go test ./...

D. go test all

4. Когда стоит использовать t.Fatal?

A. Только для проверки строк

B. Чтобы запустить следующую goroutine

C. Чтобы отметить любой успешный тест

D. Когда без успешной проверки нельзя продолжать текущий тест

5. Что делает t.Run?

A. Запускает весь модуль повторно

B. Собирает приложение в бинарник

C. Запускает именованный вложенный тест

D. Включает race detector

6. Зачем нужны table-driven tests?

A. Чтобы не использовать t.Run

B. Чтобы автоматически покрыть весь код

C. Чтобы хранить тесты только в map

D. Чтобы проверить много похожих сценариев без дублирования теста

7. Как проверить, что Rename вернул ErrEmptyTitle, в том числе при оборачивании ошибки?

A. len(err) == 0

B. err.Error() == "название задачи не должно быть пустым"

C. err == true

D. errors.Is(err, ErrEmptyTitle)

8. Зачем в subtest обычно создают новое хранилище Store?

A. Store нельзя создать больше одного раза

B. Чтобы t.Run работал быстрее

C. Чтобы сценарии не влияли на состояние друг друга

D. Чтобы не проверять ошибки

9. Что показывает команда go test -cover ./...?

A. Список импортов проекта

B. Что все бизнес-правила точно проверены

C. Что в коде нет гонок данных

D. Какая часть кода выполнялась во время тестов

10. Какую команду следует использовать для тестов конкурентного кода?

A. go run -mutex .

B. go build -race

C. go test -parallel false

D. go test -race ./...

Ответы к самопроверке

1. Он должен заканчиваться на _test.go

2. func TestSomething(t *testing.T)

3. go test ./...

4. Когда без успешной проверки нельзя продолжать текущий тест

5. Запускает именованный вложенный тест

6. Чтобы проверить много похожих сценариев без дублирования теста

7. errors.Is(err, ErrEmptyTitle)

8. Чтобы сценарии не влияли на состояние друг друга

9. Какая часть кода выполнялась во время тестов

10. go test -race ./...

Задания: тесты Task и Store

В этой практике напишем первые unit-тесты для модели задачи и простого Store. Все тесты запускайте из корня модуля командой `go test ./...`.

Задание 1. Тест Rename

В `internal/task/task_test.go` напишите `TestTaskRename`. Проверьте, что корректное название меняет `Title`, а метод возвращает `nil`.

Критерии проверки:

- тест находится в файле с суффиксом `_test.go`;
- функция называется `TestTaskRename`;
- задача создаётся внутри теста;
- проверяется отсутствие ошибки;
- проверяется новое значение `Title`.

Подсказка

```
if err := currentTask.Rename("Купить продукты"); err != nil {
  t.Fatalf("Rename вернул ошибку: %v", err)
}
```

Решение

```
package task

import "testing"

func TestTaskRename(t *testing.T) {
  currentTask := Task{ID: 1, Title: "Купить молоко", Priority: 2}

  if err := currentTask.Rename("Купить продукты"); err != nil {
    t.Fatalf("Rename вернул ошибку: %v", err)
  }

  if currentTask.Title != "Купить продукты" {
    t.Errorf("Title = %q, ожидалось %q", currentTask.Title, "Купить продукты")
  }
}
```

Задание 2. Table-driven test для Rename

Расширьте тест Rename таблицей сценариев:

- корректное название;
- пустая строка;
- строка только из пробелов;
- слишком длинное название.

Для ошибочных случаев проверьте, что исходное название не меняется и возвращается нужная ошибка.

Критерии проверки:

- используется слайс test cases;
- каждый сценарий запускается через `t.Run`;
- известная ошибка проверяется через `errors.Is`;
- состояние задачи создаётся заново в каждом subtest;
- неуспешное переименование не меняет `Title`.

Подсказка

Добавьте в структуру test case поля `name`, `title`, `wantTitle` и `wantErr`.

Фрагмент решения

```
for _, testCase := range testCases {
  t.Run(testCase.name, func(t *testing.T) {
    currentTask := Task{ID: 1, Title: "Купить молоко", Priority: 2}

    err := currentTask.Rename(testCase.title)
    if !errors.Is(err, testCase.wantErr) {
      t.Fatalf("Rename error = %v, ожидалось %v", err, testCase.wantErr)
    }

    if currentTask.Title != testCase.wantTitle {
      t.Errorf("Title = %q, ожидалось %q", currentTask.Title, testCase.wantTitle)
    }
  })
}
```

Задание 3. Тест IsValid

Напишите table-driven test для `Task.IsValid()`. Проверьте корректную задачу, нулевой ID, пустое название и приоритет вне диапазона.

Критерии проверки:

- тест содержит минимум четыре сценария;
- каждый сценарий создаёт свой `Task`;
- ожидаемый результат имеет тип `bool`;
- сообщение теста показывает фактическое и ожидаемое значения.

Подсказка

```
if got := currentTask.IsValid(); got != testCase.want {
  t.Errorf("IsValid() = %t, ожидалось %t", got, testCase.want)
}
```

Задание 4. Тесты Store

Создайте пакет `internal/store` с простым `Store` из главы. Напишите тесты для:

- успешного `Add` и `Get`;
- повторного добавления того же ID;
- отсутствующей задачи;
- `Rename` и `Complete`.

Критерии проверки:

- Store создаётся заново в каждом тесте или subtest;
- повторный ID проверяется через `errors.Is(err, ErrTaskAlreadyExists)`;
- отсутствующая задача проверяется через `exists == false` или `ErrTaskNotFound`;
- Rename и Complete проверяют итоговую задачу через `Get`;
- все тесты проходят через `go test ./...`.

Подсказка

Для начала напишите небольшой тест добавления и получения:

```
store := New()
wantTask := task.Task{ID: 1, Title: "Купить молоко", Priority: 2}

if err := store.Add(wantTask); err != nil {
  t.Fatalf("Add вернул ошибку: %v", err)
}
```

Решение для Add и Get

```
func TestStoreAddAndGet(t *testing.T) {
  store := New()
  wantTask := task.Task{ID: 1, Title: "Купить молоко", Priority: 2}

  if err := store.Add(wantTask); err != nil {
    t.Fatalf("Add вернул ошибку: %v", err)
  }

  gotTask, exists := store.Get(1)
  if !exists {
    t.Fatal("задача не найдена после Add")
  }

  if gotTask != wantTask {
    t.Errorf("Get = %+v, ожидалось %+v", gotTask, wantTask)
  }
}
```

Команды проверки

После выполнения заданий запустите:

```
go test ./...
go test -cover ./...
```

Позже, после перехода на потокобезопасный `TaskStore`, добавьте:

```
go test -race ./...
```

Проверьте себя

Перед следующей главой убедитесь, что вы можете:

- создать файл `_test.go`;
- написать `Test...` с `*testing.T`;
- выбрать между `t.Error` и `t.Fatal`;
- оформить набор сценариев таблицей и `t.Run`;
- проверить ошибку через `errors.Is`;
- создать независимое состояние для каждого теста;
- запустить весь набор тестов и посмотреть покрытие.

Рекомендации по коду: тестирование

- Тестируйте публичное поведение и бизнес-правила, а не случайные детали реализации.
- Называйте тесты по проверяемому сценарию: `TestTaskRename`, `TestStoreAddAndGet`.
- Для похожих случаев используйте table-driven tests и `t.Run` с понятными именами.
- Создавайте новое изменяемое состояние в каждом тесте или subtest.
- Проверяйте ошибку через `errors.Is`, если важна причина, а не текст сообщения.
- Делайте сообщения тестов диагностическими: показывайте фактический и ожидаемый результат.
- Запускайте `go test ./...` перед изменением или публикацией кода; конкурентные тесты — с `-race`.

Чек-лист

Тест проверяет результат и побочный эффект, независим от порядка запуска других тестов, покрывает успешный, ошибочный и граничный сценарии.

Файлы и ввод-вывод

Цель урока

Научиться записывать текст в файл, читать его обратно, безопасно открывать и закрывать файлы, строить пути через `filepath` и тестировать файловый экспорт во временной директории.

В Task Manager API мы пока не сохраняем сами структуры задач. Сначала сделаем текстовый экспорт их кратких описаний в файл `tasks.txt`. В следующей главе JSON позволит сохранять данные так, чтобы программа могла восстановить их как задачи.

Файлы и ошибки

Работа с файлом зависит от файловой системы: файл может отсутствовать, путь может быть неверным, а прав на запись может не хватать. Поэтому почти все функции из `os` возвращают `error`.

```
content, err := os.ReadFile("tasks.txt")
if err != nil {
  return fmt.Errorf("не удалось прочитать tasks.txt: %w", err)
}

fmt.Println(string(content))
```

`os.ReadFile` возвращает содержимое как `[]byte`. Для текста его обычно преобразуют в `string`.

Быстрая запись: `os.WriteFile`

Для небольшого текста проще всего использовать `os.WriteFile`:

```
package main

import (
  "fmt"
  "os"
)

func main() {
  content := "Купить молоко\nОплатить интернет\n"

  err := os.WriteFile("tasks.txt", []byte(content), 0o644)
  if err != nil {
```

```
    fmt.Println("Не удалось записать файл:", err)    return } fmt.Println("Файл сохранён")}
```

Аргументы `os.WriteFile`:

1. путь к файлу;
2. данные в байтах;
3. права доступа для **нового** файла.

Запись создаёт файл, если его нет. Если файл уже существует, его предыдущее содержимое будет заменено. На Unix-подобных системах `0o644` обычно означает: владелец может читать и писать, остальные — только читать. В Windows этот аргумент не влияет на права так же, но его всё равно передают для переносимости Go-кода.

> **WriteFile не добавляет текст в конец**
>
> Повторный вызов `os.WriteFile` по тому же пути перезапишет файл. Для добавления записей в конец потребуется открыть файл с особыми флагами; этот сценарий рассмотрим позднее, когда появится логирование.

Чтение файла: `os.ReadFile`

Прочитаем созданный файл:

```
package main

import (
  "fmt"
  "os"
)

func main() {
  content, err := os.ReadFile("tasks.txt")
  if err != nil {
    fmt.Println("Не удалось прочитать файл:", err)
    return
  }

  fmt.Print(string(content))
}
```

`os.ReadFile` подходит для небольших файлов конфигурации, экспортов и учебных данных. Не используйте его для очень больших файлов: он загружает всё содержимое в память.

Если файл не существует, обычно вернётся ошибка. Её можно показать пользователю или обработать отдельно через `errors.Is(err, os.ErrNotExist)`:

```
content, err := os.ReadFile("tasks.txt")
if errors.Is(err, os.ErrNotExist) {
  fmt.Println("Экспорт ещё не создан")
  return
}

if err != nil {
  fmt.Println("Не удалось прочитать файл:", err)
  return
}

fmt.Print(string(content))
```

Открытие файла и `defer Close`

`os.ReadFile` сам открывает и закрывает файл. Но иногда нужно управлять файловым объектом вручную: например, читать данные через `io.Reader` или записывать по частям.

```
file, err := os.Open("tasks.txt")
if err != nil {
  return fmt.Errorf("не удалось открыть tasks.txt: %w", err)
}
defer file.Close()
```

После успешного `os.Open` файл нужно закрыть. `defer` откладывает вызов `file.Close()` до завершения текущей функции — даже если мы выйдем через `return` дальше по коду.

```
func printExport(path string) error {
  file, err := os.Open(path)
  if err != nil {
    return fmt.Errorf("не удалось открыть экспорт: %w", err)
  }
  defer file.Close()

  content, err := io.ReadAll(file)
  if err != nil {
    return fmt.Errorf("не удалось прочитать экспорт: %w", err)
  }
```

```
  fmt.Print(string(content)) return nil}
```

В этом примере:

- `os.Open` возвращает `*os.File`;
- `defer file.Close()` освобождает ресурс;
- `io.ReadAll` читает всё из уже открытого файла.

Для простого чтения используйте `os.ReadFile`. Пример с `os.Open` и `io.ReadAll` нужен, чтобы понять общий паттерн владения ресурсом: **открыл — запланируй закрытие сразу после успешной проверки ошибки**.

Создание файла: `os.Create`

`os.Create` создаёт файл для записи и возвращает `*os.File`:

```
file, err := os.Create("tasks.txt")
if err != nil {
  return fmt.Errorf("не удалось создать файл: %w", err)
}
defer file.Close()

if _, err := file.WriteString("Купить молоко\n"); err != nil {
  return fmt.Errorf("не удалось записать экспорт: %w", err)
}
```

Если файл уже существует, `os.Create` очистит его содержимое. Для единой короткой записи по-прежнему предпочитайте `os.WriteFile`: там меньше кода и меньше риска забыть закрыть файл.

Пути к файлам

Относительный путь считается от текущей рабочей директории. Если программу запускают из корня Task Manager API командой:

```
go run .
```

то путь `tasks.txt` означает файл рядом с `go.mod` и `main.go`:

```
task-manager/
├── go.mod
├── main.go
└── tasks.txt
```

Не полагайтесь на текущую директорию в большом приложении без необходимости. Когда появятся конфигурация и HTTP-сервер, путь к хранилищу будет задаваться явно. Пока достаточно запускать программу из корня модуля.

`path/filepath`

Не собирайте путь через строковую конкатенацию:

```
// exportPath := directory + "/tasks.txt"
```

Разделитель папок отличается между операционными системами. Используйте `filepath.Join`:

```
exportPath := filepath.Join("data", "tasks.txt")
```

`filepath.Join` выберет корректный разделитель для текущей ОС и очистит лишние разделители в пути.

Если папка для экспорта может отсутствовать, создайте её до записи:

```
directory := "data"
if err := os.MkdirAll(directory, 0o755); err != nil {
  return fmt.Errorf("не удалось создать папку экспорта: %w", err)
}

exportPath := filepath.Join(directory, "tasks.txt")
```

`os.MkdirAll` не возвращает ошибку, если папка уже существует. Не вызывайте его для каждого файла без необходимости: обычно папку создают на старте приложения или перед первой записью в хранилище.

Экспорт Task Manager API

Сохраним краткие описания задач в текстовый файл. Функция получает слайс задач, формирует текст и записывает его:

```
func exportTasks(path string, tasks []task.Task) error {
  var builder strings.Builder

  for _, currentTask := range tasks {
    builder.WriteString(currentTask.Summary())
    builder.WriteString("\n")
  }
```

```
  if err := os.WriteFile(path, []byte(builder.String()), 0o644); err != nil {
return fmt.Errorf("не удалось сохранить экспорт задач: %w", err)   } return nil}
```

`strings.Builder` собирает большой текст без создания новой строки на каждом шаге цикла. Добавьте импорт `strings` в файл с этой функцией.

Функция получает путь параметром, а не создаёт его внутри. Так вызывающий код управляет местом хранения, а тест может передать безопасный временный путь.

Полный пример с пакетом `internal/task`:

```
package main

import (
  "errors"
  "fmt"
  "os"
  "strings"

  "example.com/task-manager/internal/task"
)

func exportTasks(path string, tasks []task.Task) error {
  var builder strings.Builder

  for _, currentTask := range tasks {
    builder.WriteString(currentTask.Summary())
    builder.WriteString("\n")
  }

  if err := os.WriteFile(path, []byte(builder.String()), 0o644); err != nil {
    return fmt.Errorf("не удалось сохранить экспорт задач: %w", err)
  }

  return nil
}

func printExport(path string) error {
  content, err := os.ReadFile(path)
  if errors.Is(err, os.ErrNotExist) {
    return fmt.Errorf("файл экспорта ещё не создан: %w", err)
  }
  if err != nil {
    return fmt.Errorf("не удалось прочитать экспорт: %w", err)
  }
```

```
  fmt.Print(string(content)) return nil}func main() {   tasks := []task.Task{    {ID:
1, Title: "Купить молоко", Priority: 2},    {ID: 2, Title: "Оплатить интернет",
Priority: 3},   } const exportPath = "tasks.txt" if err := exportTasks(exportPath,
tasks); err != nil {    fmt.Println("Ошибка экспорта:", err)    return } if err :=
printExport(exportPath); err != nil {   fmt.Println("Ошибка чтения:", err)  }}
```

Тестирование файлового кода: `t.TempDir`

Тест не должен создавать `tasks.txt` в папке исходного проекта, зависеть от уже существующего файла или оставлять мусор после запуска. `testing.T.TempDir` создаёт уникальную временную папку для теста и автоматически удаляет её после завершения.

```
func TestExportTasks(t *testing.T) {
  path := filepath.Join(t.TempDir(), "tasks.txt")
  tasks := []task.Task{
    {ID: 1, Title: "Купить молоко", Priority: 2},
  }

  if err := exportTasks(path, tasks); err != nil {
    t.Fatalf("exportTasks вернул ошибку: %v", err)
  }

  content, err := os.ReadFile(path)
  if err != nil {
    t.Fatalf("не удалось прочитать экспорт: %v", err)
  }

  want := "1. Купить молоко | приоритет: 2 | выполнена: false\n"
  if string(content) != want {
    t.Errorf("экспорт = %q, ожидалось %q", content, want)
  }
}
```

Тест проверяет внешний контракт `exportTasks`: файл создан по указанному пути и содержит ожидаемый текст. Он не обязан проверять, использовала ли функция `strings.Builder` или другой внутренний механизм.

Частые ошибки

Игнорирование ошибки записи

```
os.WriteFile("tasks.txt", []byte("данные"), 0o644)
```

Go позволяет отбросить возвращаемую ошибку, но так сбой записи останется незамеченным. Обрабатывайте её явно:

```
if err := os.WriteFile("tasks.txt", []byte("данные"), 0o644); err != nil {
  return err
}
```

Забытый `Close`

```
file, err := os.Open("tasks.txt")
if err != nil {
  return err
}

// Файл не закрыт.
```

Сразу после успешного открытия добавьте `defer file.Close()`.

Ожидание, что файл добавит новые данные

```
os.WriteFile("tasks.txt", []byte("Новая строка\n"), 0o644)
```

Этот вызов заменяет старое содержимое. Не используйте его для журнала операций.

Чтение несуществующего файла без обработки

Первый запуск программы может произойти до создания экспорта. Для такой ситуации можно показать понятное сообщение, проверив `errors.Is(err, os.ErrNotExist)`.

Файл теста в папке проекта

```
// exportTasks("tasks.txt", tasks)
```

Так тест зависит от рабочей директории и может перезаписать реальный экспорт. Используйте `filepath.Join(t.TempDir(), "tasks.txt")`.

Дальше

Закрепите текстовый экспорт задач и его тест во временной директории. Следующая глава — JSON и сериализация: она позволит сохранять структуру данных, а не только готовый текст для человека.

Самопроверка

1. Какая функция подходит для записи небольшого текста в файл одной командой?

A. io.ReadAll

B. fmt.Println

C. os.Open

D. os.WriteFile

2. Какой тип данных возвращает os.ReadFile?

A. string и bool

B. *os.File и error

C. []byte и error

D. только error

3. Что обычно делают с []byte, полученным из os.ReadFile, чтобы вывести текст?

A. Сравнивают с nil без чтения

B. Преобразуют через string(content)

C. Разыменовывают через *content

D. Передают в append

4. Что происходит, если os.WriteFile вызывают для существующего файла?

A. Старое содержимое заменяется новыми данными

B. Файл автоматически удаляется

C. Go всегда возвращает os.ErrExist

D. Новые данные всегда добавляются в конец

5. Какую функцию используют для открытия файла, когда нужно работать с *os.File?

A. fmt.Open

B. os.ReadFile

C. os.Open

D. io.Open

6. Зачем сразу после успешного os.Open пишут defer file.Close()?

A. Чтобы удалить файл после чтения

B. Чтобы преобразовать файл в строку

C. Чтобы закрыть файл при выходе из текущей функции

D. Чтобы открыть файл во второй раз

7. Что делает os.Create с уже существующим файлом?

A. Очищает его и открывает для записи

B. Всегда возвращает ошибку

C. Открывает его только для чтения

D. Добавляет запись в конец

8. Как проверить, что ошибка чтения возникла именно потому, что файла нет?

A. err == false

B. len(err) == 0

C. err.Error() == "file not found"

D. errors.Is(err, os.ErrNotExist)

9. От чего считается относительный путь tasks.txt?

A. От папки пакета os

B. Всегда от домашней папки пользователя

C. Всегда от папки установки Go

D. От текущей рабочей директории программы

10. Почему os.ReadFile не подходит для очень большого файла?

A. Он удаляет файл после чтения

B. Он загружает всё содержимое в память

C. Он работает только с JSON

D. Он не возвращает error

11. Как безопасно собрать путь к tasks.txt внутри папки data?

A. os.Path("data", "tasks.txt")

B. filepath.Join("data", "tasks.txt")

C. strings.Split("data/tasks.txt", "/")

D. "data" + "/" + "tasks.txt"

12. Зачем в тесте файлового экспорта используют t.TempDir()?

A. Чтобы отключить ошибки os.WriteFile

B. Чтобы заменить filepath.Join

C. Чтобы создать изолированную временную папку, которую Go удалит после теста

D. Чтобы навсегда сохранить результаты теста

Ответы к самопроверке

1. os.WriteFile

2. []byte и error

3. Преобразуют через string(content)

4. Старое содержимое заменяется новыми данными

5. os.Open

6. Чтобы закрыть файл при выходе из текущей функции

7. Очищает его и открывает для записи

8. errors.Is(err, os.ErrNotExist)

9. От текущей рабочей директории программы

10. Он загружает всё содержимое в память

11. filepath.Join("data", "tasks.txt")

12. Чтобы создать изолированную временную папку, которую Go удалит после теста

Задания: текстовый экспорт задач

В этой практике Task Manager API создаёт текстовый файл `tasks.txt` со списком задач, а затем читает его обратно. Это экспорт для человека; в следующей главе JSON позволит восстановить сами структуры Task.

Задание 1. Запишите простой экспорт

Создайте переменную `content` со строкой из двух названий задач, каждое с новой строки. Запишите её в `tasks.txt` через `os.WriteFile`. Сообщите об ошибке, если запись не удалась.

Критерии проверки:

- импортирован пакет `os`;
- строка преобразована в `[]byte`;
- используется `os.WriteFile("tasks.txt", ..., 0o644)`;
- ошибка проверяется через `if err != nil`;
- после запуска рядом с программой появляется файл `tasks.txt`.

Подсказка

```
content := "Купить молоко\nОплатить интернет\n"
err := os.WriteFile("tasks.txt", []byte(content), 0o644)
```

Задание 5. Протестируйте текстовый экспорт

Создайте `export_test.go` рядом с файлом, где находится `exportTasks`. Напишите `TestExportTasks`: передайте путь внутри `t.TempDir()`, экспортируйте одну или две задачи, затем прочитайте файл через `os.ReadFile` и сравните текст с ожидаемым.

Критерии проверки:

- тестовый файл заканчивается на `_test.go`;
- путь собирается через `filepath.Join(t.TempDir(), "tasks.txt")`;
- тест не создаёт файл в рабочей папке проекта;
- ошибка `exportTasks` проверяется через `t.Fatal` или `t.Fatalf`;
- содержимое файла проверяется целиком;
- `go test ./...` проходит успешно.

Подсказка

```
path := filepath.Join(t.TempDir(), "tasks.txt")

if err := exportTasks(path, tasks); err != nil {
  t.Fatalf("exportTasks вернул ошибку: %v", err)
}
```

Решение

```
func TestExportTasks(t *testing.T) {
  path := filepath.Join(t.TempDir(), "tasks.txt")
  tasks := []task.Task{
    {ID: 1, Title: "Купить молоко", Priority: 2},
  }

  if err := exportTasks(path, tasks); err != nil {
    t.Fatalf("exportTasks вернул ошибку: %v", err)
  }

  content, err := os.ReadFile(path)
  if err != nil {
    t.Fatalf("не удалось прочитать экспорт: %v", err)
  }

  want := "1. Купить молоко | приоритет: 2 | выполнена: false\n"
  if string(content) != want {
    t.Errorf("экспорт = %q, ожидалось %q", content, want)
  }
}
```

Для теста нужны импорты `os`, `path/filepath`, `testing` и пакет с `Task`.

Решение

```
package main

import (
  "fmt"
  "os"
)
```

▼ Сведения

```
func main() {   content := "Купить молоко\nОплатить интернет\n"  if err := os.WriteFile("tasks.txt", []byte(content), 0o644); err != nil {    fmt.Println("Не удалось записать файл:", err)    return } fmt.Println("Экспорт сохранён")}
```

Задание 2. Прочитайте экспорт

Прочитайте `tasks.txt` через `os.ReadFile` и выведите содержимое. Если файл отсутствует, сообщите `Экспорт ещё не создан`. Другие ошибки выведите как есть.

Критерии проверки:

- импортированы `errors`, `fmt` и `os`;
- используется `os.ReadFile`;
- отсутствие файла проверяется через `errors.Is(err, os.ErrNotExist)`;
- успешное содержимое выводится через `fmt.Print(string(content))`.

Подсказка

Сначала обработайте отсутствие файла, затем остальные ошибки:

```
if errors.Is(err, os.ErrNotExist) {
  fmt.Println("Экспорт ещё не создан")
  return
}

if err != nil {
  // ...
}
```

Решение

```
package main

import (
  "errors"
  "fmt"
  "os"
)

func main() {
  content, err := os.ReadFile("tasks.txt")
  if errors.Is(err, os.ErrNotExist) {
    fmt.Println("Экспорт ещё не создан")
    return
  }
  if err != nil {
    fmt.Println("Не удалось прочитать файл:", err)
    return
  }

  fmt.Print(string(content))
}
```

Задание 3. Откройте и прочитайте файл вручную

Напишите функцию `readExport(path string) error`. Она должна открыть файл через `os.Open`, отложить `Close`, прочитать содержимое через `io.ReadAll` и вывести его.

Критерии проверки:

- функция возвращает `error`;
- `os.Open` проверяется на ошибку;
- после успешного открытия сразу вызывается `defer file.Close()`;
- `io.ReadAll(file)` проверяется на ошибку;
- ошибки получают контекст через `fmt.Errorf` и `%w`.

Подсказка

```
file, err := os.Open(path)
if err != nil {
  return fmt.Errorf("не удалось открыть экспорт: %w", err)
}
defer file.Close()
```

Решение

```
func readExport(path string) error {
  file, err := os.Open(path)
  if err != nil {
    return fmt.Errorf("не удалось открыть экспорт: %w", err)
  }
  defer file.Close()

  content, err := io.ReadAll(file)
  if err != nil {
    return fmt.Errorf("не удалось прочитать экспорт: %w", err)
  }

  fmt.Print(string(content))
  return nil
}
```

Для этого решения нужны импорты `fmt`, `io` и `os`.

Задание 4. Экспортируйте Task

В проекте из прошлой главы создайте в `main.go` функцию:

```
func exportTasks(path string, tasks []task.Task) error
```

Она должна сохранить `Summary()` каждой задачи в отдельной строке файла `tasks.txt`. После успешной записи прочитайте файл и выведите экспорт.

Критерии проверки:

- функция принимает `[]task.Task`;
- каждая задача добавляет в экспорт `currentTask.Summary() + "\n"`;
- используется `strings.Builder`;
- `os.WriteFile` возвращает ошибку с контекстом;
- `main` не продолжает чтение, если экспорт не записался;
- файл содержит строку summary для каждой задачи.

Подсказка

Вспомните импорт из предыдущей главы:

```
"example.com/task-manager/internal/task"
```

Для формирования текста используйте `strings.Builder`: добавляйте summary каждой задачи через `WriteString`.

Возможное решение

```
func exportTasks(path string, tasks []task.Task) error {
  var builder strings.Builder

  for _, currentTask := range tasks {
    builder.WriteString(currentTask.Summary())
    builder.WriteString("\n")
  }

  if err := os.WriteFile(path, []byte(builder.String()), 0o644); err != nil {
    return fmt.Errorf("не удалось сохранить экспорт задач: %w", err)
  }

  return nil
}
```

Проверьте себя

Перед следующей главой убедитесь, что вы можете:

- записать небольшой текст через `os.WriteFile`;
- прочитать его через `os.ReadFile`;
- обработать `os.ErrNotExist`;
- объяснить роль `defer file.Close()`;
- открыть файл через `os.Open` и прочитать его через `io.ReadAll`;
- экспортировать summary задач в текстовый файл.
- проверить экспорт через `t.TempDir` без создания файлов в проекте.

Рекомендации по коду: файлы и ввод-вывод

- Для небольшого файла используйте `os.ReadFile` и `os.WriteFile`; не усложняйте код без необходимости.
- После успешного `os.Open` или `os.Create` сразу добавляйте `defer file.Close()`.

- Оборачивайте ошибки файловой системы контекстом: путь и операция важны для диагностики.
- Различайте отсутствие ожидаемого файла через `errors.Is(err, os.ErrNotExist)`.
- Не держите файл открытым дольше необходимого.
- Не используйте `os.ReadFile` для большого или бесконечного потока данных.
- Собирайте пути через `filepath.Join`, а не строковой конкатенацией.
- Тестируйте файловые операции через `t.TempDir`, не создавая файлы в рабочей папке проекта.

Чек-лист

Каждый открытый файл закрывается, путь задаётся явно и собирается переносимо, ошибки чтения и записи не игнорируются, а файловые тесты изолированы во временной директории.

JSON и сериализация

Цель урока

Научиться преобразовывать структуры Go в JSON и обратно, управлять именами полей через теги, вынести JSON-хранилище в отдельный пакет и проверить его во временной директории.

Текстовый экспорт из прошлой главы удобен для чтения человеком, но программа не может надёжно восстановить из него структуры `Task`. JSON хранит данные в стандартном структурированном формате, понятном Go, HTTP API и многим другим системам.

Что такое JSON

JSON (JavaScript Object Notation) — текстовый формат обмена данными. Объекты записывают в фигурных скобках, массивы — в квадратных:

```
{
  "id": 1,
  "title": "Купить молоко",
  "priority": 2,
  "completed": false
}
```

В Go JSON обрабатывает стандартный пакет `encoding/json`.

Сериализация и десериализация

Сериализация превращает значение Go в последовательность байтов JSON:

```
data, err := json.Marshal(task)
```

Десериализация восстанавливает значение Go из JSON-байтов:

```
var task Task
err := json.Unmarshal(data, &task)
```

`json.Marshal` возвращает `[]byte`, потому что JSON — текстовые данные в байтовом виде. Для вывода в терминал преобразуйте результат в строку:

```
fmt.Println(string(data))
```

Сериализация структуры

Для начала возьмём знакомую структуру:

```
type Task struct {
  ID        int
  Title     string
  Priority  int
  Completed bool
}
```

Поля с заглавной буквы экспортируются, поэтому `encoding/json` может прочитать и записать их:

```
package main

import (
  "encoding/json"
  "fmt"
)

type Task struct {
  ID        int
  Title     string
  Priority  int
  Completed bool
}

func main() {
  task := Task{ID: 1, Title: "Купить молоко", Priority: 2}

  data, err := json.Marshal(task)
  if err != nil {
    fmt.Println("Не удалось сериализовать задачу:", err)
    return
  }

  fmt.Println(string(data))
}
```

Вывод будет похож на:

```
{"ID":1,"Title":"Купить молоко","Priority":2,"Completed":false}
```

Имена ключей сейчас совпадают с именами Go-полей. Обычно для API и файлов используют нижний регистр, поэтому добавляют JSON-теги.

JSON-теги

Тег указывается после типа поля в обратных кавычках:

```
type Task struct {
  ID        int    `json:"id"`
  Title     string `json:"title"`
  Priority  int    `json:"priority"`
  Completed bool   `json:"completed"`
}
```

Теперь JSON станет таким:

```
{"id":1,"title":"Купить молоко","priority":2,"completed":false}
```

Тег меняет имя поля только во внешнем представлении JSON. В Go оно по-прежнему используется как `task.Title`.

```
task.Title = "Купить продукты"
```

> **Поля должны быть экспортируемыми**
>
> `encoding/json` игнорирует поля со строчной буквы. Например, поле `title string` не попадёт в JSON автоматически. Для модели, которую нужно сохранять или отдавать через API, используйте экспортируемые поля и JSON-теги.

Красивый JSON: `json.MarshalIndent`

`json.Marshal` создаёт компактный JSON в одну строку. Для файла, который читает человек, удобнее `json.MarshalIndent`:

```
data, err := json.MarshalIndent(tasks, "", "  ")
```

Первый строковый аргумент — префикс каждой строки, второй — отступ одного уровня. Обычно используют пустой префикс и два пробела.

Пример результата:

```
[
  {
    "id": 1,
    "title": "Купить молоко",
    "priority": 2,
    "completed": false
  }
]
```

Десериализация: `json.Unmarshal`

`json.Unmarshal` получает JSON-байты и указатель на место, куда нужно записать результат:

```
data := []byte(`{"id":1,"title":"Купить молоко","priority":2,"completed":false}`)

var task Task
if err := json.Unmarshal(data, &task); err != nil {
  fmt.Println("Некорректный JSON:", err)
  return
}

fmt.Println(task.Title) // Купить молоко
```

Почему нужен `&task`? `Unmarshal` должен изменить переменную `task`, а значит получает её адрес.

Для массива JSON создайте слайс:

```
var tasks []Task
if err := json.Unmarshal(data, &tasks); err != nil {
  return err
}
```

Что делает Unmarshal с данными

- отсутствующее в JSON поле остаётся с нулевым значением;
- неизвестные поля JSON по умолчанию игнорируются;
- несовместимый тип вызывает ошибку, например строка вместо числа для `priority`.

JSON-декодирование проверяет формат, но не бизнес-правила. JSON с `"priority": 10` корректен синтаксически, однако для нашей модели приоритет всё ещё недопустим. После загрузки задачи нужно валидировать теми же правилами, что и данные из другого источника.

Пустой и nil-слайс

Есть тонкое различие:

```
var nilTasks []Task
emptyTasks := []Task{}

nilJSON, _ := json.Marshal(nilTasks)
emptyJSON, _ := json.Marshal(emptyTasks)

fmt.Println(string(nilJSON))   // null
fmt.Println(string(emptyJSON)) // []
```

Для списка задач в JSON обычно ожидают `[]`, а не `null`. Если экспортируете пустой список, создайте пустой слайс: `tasks := []task.Task{}`.

Сохранение задач в JSON-файл

Объединим JSON и файловый ввод-вывод. Сначала сериализуем слайс, затем записываем байты:

```
func SaveTasks(path string, tasks []task.Task) error {
  data, err := json.MarshalIndent(tasks, "", "  ")
  if err != nil {
    return fmt.Errorf("не удалось сериализовать задачи: %w", err)
  }

  if err := os.WriteFile(path, data, 0o644); err != nil {
    return fmt.Errorf("не удалось сохранить задачи: %w", err)
  }

  return nil
}
```

Ошибки оборачиваются отдельно: так вызывающий код понимает, проблема возникла на этапе JSON или записи файла.

Загрузка задач из JSON-файла

Сначала прочитаем байты, затем десериализуем их в слайс:

```
func LoadTasks(path string) ([]task.Task, error) {
  data, err := os.ReadFile(path)
  if err != nil {
```

```
    return nil, fmt.Errorf("не удалось прочитать задачи: %w", err)  } var tasks
[]task.Task  if err := json.Unmarshal(data, &tasks); err != nil {   return nil,
fmt.Errorf("не удалось разобрать JSON задач: %w", err)  } return tasks, nil}
```

Функция возвращает сразу два результата: список и ошибку. Если чтение или JSON-декодирование не удалось, список равен `nil`, а ошибка объясняет причину.

Пакет `internal/storage`

Не держите JSON-хранилище в `main.go`. `main` должен собирать зависимости и запускать сценарий, а операции сохранения и загрузки — жить в отдельном пакете:

```
task-manager/
├── main.go
└── internal/
    ├── task/
    │   └── task.go
    └── storage/
        ├── json.go
        └── json_test.go
```

В `internal/storage/json.go`:

```
package storage

import (
  "encoding/json"
  "fmt"
  "os"

  "example.com/task-manager/internal/task"
)

func SaveTasks(path string, tasks []task.Task) error {
  data, err := json.MarshalIndent(tasks, "", "  ")
  if err != nil {
    return fmt.Errorf("не удалось сериализовать задачи: %w", err)
  }

  if err := os.WriteFile(path, data, 0o644); err != nil {
    return fmt.Errorf("не удалось сохранить задачи: %w", err)
  }

  return nil
```

```
}func LoadTasks(path string) ([]task.Task, error) {  data, err := os.ReadFile(path)
if err != nil {     return nil, fmt.Errorf("не удалось прочитать задачи: %w", err)  }
var tasks []task.Task if err := json.Unmarshal(data, &tasks); err != nil {   return
nil, fmt.Errorf("не удалось разобрать JSON задач: %w", err) } return tasks, nil}
```

`SaveTasks` и `LoadTasks` экспортируются, потому что их вызывает `main`. Детали JSON и файловой системы остаются внутри `storage`.

Тестирование JSON-хранилища

Файловое хранилище тестируют через `t.TempDir`, чтобы не создавать `tasks.json` в исходном проекте:

```
func TestSaveAndLoadTasks(t *testing.T) {
  path := filepath.Join(t.TempDir(), "tasks.json")
  wantTasks := []task.Task{
    {ID: 1, Title: "Купить молоко", Priority: 2},
  }

  if err := SaveTasks(path, wantTasks); err != nil {
    t.Fatalf("SaveTasks вернул ошибку: %v", err)
  }

  gotTasks, err := LoadTasks(path)
  if err != nil {
    t.Fatalf("LoadTasks вернул ошибку: %v", err)
  }

  if !reflect.DeepEqual(gotTasks, wantTasks) {
    t.Errorf("LoadTasks = %#v, ожидалось %#v", gotTasks, wantTasks)
  }
}
```

В тесте используются `path/filepath`, `reflect`, `testing` и пакет `task`. `reflect.DeepEqual` подходит для сравнения слайсов структур в учебном примере. Позже можно выбрать более удобный инструмент сравнения, если модели станут сложнее.

Полезны и отдельные сценарии:

- сохранение пустого слайса как `[]`, а не `null`;
- загрузка несуществующего файла с проверкой `errors.Is(err, os.ErrNotExist)`;
- загрузка некорректного JSON;
- проверка имён ключей `id`, `title`, `priority`, `completed` в записанном файле.

Полный пример Task Manager API

Предположим, что тип `Task` с JSON-тегами расположен в `internal/task/task.go`:

```
package task

type Task struct {
  ID        int    `json:"id"`
  Title     string `json:"title"`
  Priority  int    `json:"priority"`
  Completed bool   `json:"completed"`
}
```

В `main.go`:

```
package main

import (
  "fmt"

  "example.com/task-manager/internal/storage"
  "example.com/task-manager/internal/task"
)

func main() {
  tasks := []task.Task{
    {ID: 1, Title: "Купить молоко", Priority: 2},
    {ID: 2, Title: "Оплатить интернет", Priority: 3},
  }

  const storagePath = "tasks.json"

  if err := storage.SaveTasks(storagePath, tasks); err != nil {
    fmt.Println("Ошибка сохранения:", err)
    return
  }

  loadedTasks, err := storage.LoadTasks(storagePath)
  if err != nil {
    fmt.Println("Ошибка загрузки:", err)
    return
  }

  for _, currentTask := range loadedTasks {
    fmt.Println(currentTask.Summary())
```

```
  }}
```

Частые ошибки

Передать значение вместо адреса в Unmarshal

```
var tasks []Task
// json.Unmarshal(data, tasks) // ошибка: нужен указатель
json.Unmarshal(data, &tasks)
```

`Unmarshal` должен записать загруженные данные в переменную.

Игнорирование ошибки JSON

```
data, _ := json.Marshal(tasks)
```

Не игнорируйте ошибку. Текущая структура может сериализоваться без проблем, но позже в неё могут появиться поля, которые JSON не поддерживает.

Ожидание валидации от Unmarshal

```
{"id":1,"title":"","priority":10,"completed":false}
```

Это корректный JSON, но некорректная задача. После загрузки применяйте `IsValid()` или другую бизнес-валидацию.

Запись `nil`-слайса, когда нужен `[]`

`json.Marshal(nilTasks)` даст `null`. Для пустого списка создайте `[]task.Task{}`.

Тест с настоящим `tasks.json`

Не запускайте тест с путём `"tasks.json"` из рабочей папки: он может зависеть от старого файла или перезаписать реальные данные. В тестах используйте `filepath.Join(t.TempDir(), "tasks.json")`.

Дальше

Закрепите пакет JSON-хранилища и его тесты. После этого перейдём к времени и дедлайнам задач.

Самопроверка

1. Какой пакет стандартной библиотеки работает с JSON?

A. fmt/json

B. encoding/json

C. io/encoding

D. os/json

2. Что делает json.Marshal?

A. Создаёт Go-структуру из JSON

B. Читает JSON-файл

C. Удаляет JSON-теги

D. Преобразует Go-значение в JSON-байты

3. Что делает json.Unmarshal?

A. Преобразует string в []byte без ошибок

B. Красиво форматирует JSON

C. Записывает JSON-данные в переданное Go-значение

D. Открывает файл для записи

4. Почему вторым аргументом json.Unmarshal передают &tasks?

A. Это синтаксис JSON-тега

B. JSON работает только с указателями на map

C. Функции нужен адрес, чтобы изменить переменную tasks

D. Так создаётся пустой слайс

5. Что задаёт тег json:"title" у поля Title?

A. Тип поля string

B. Максимальную длину строки

C. Новое имя Go-поля

D. Имя ключа title в JSON

6. Какие поля структура должна иметь для автоматической сериализации encoding/json?

A. Только поля типа string

B. Только неэкспортируемые поля

C. Поля без JSON-тегов

D. Экспортируемые поля с заглавной буквы

7. Чем json.MarshalIndent отличается от json.Marshal?

A. Работает только с одним объектом

B. Не возвращает ошибку

C. Автоматически записывает файл

D. Добавляет отступы для читаемого JSON

8. Что вернёт json.Marshal для nil-слайса?

A. {}

B. null

C. []

D. Ошибка всегда

9. Проверит ли json.Unmarshal, что priority находится в диапазоне от 1 до 3?

A. Только для nil-слайса

B. Нет, бизнес-валидацию нужно выполнить отдельно

C. Только если поле имеет тег json

D. Да, JSON всегда знает правила Task

10. Какой порядок действий нужен для загрузки задач из tasks.json?

A. json.Marshal, затем os.ReadFile

B. os.WriteFile, затем json.MarshalIndent

C. os.ReadFile, затем json.Unmarshal

D. fmt.Println, затем copy

11. В каком пакете стоит разместить SaveTasks и LoadTasks?

A. Обязательно внутри типа Task

B. Только в package main

C. В пакете fmt

D. В отдельном внутреннем пакете storage

12. Какой путь нужно передать JSON-тесту, чтобы не создавать tasks.json в проекте?

A. filepath.Join(t.TempDir(), "tasks.json")

B. go.mod

C. os.TempDir() + "/tasks.json"

D. "tasks.json"

Ответы к самопроверке

1. encoding/json

2. Преобразует Go-значение в JSON-байты

3. Записывает JSON-данные в переданное Go-значение

4. Функции нужен адрес, чтобы изменить переменную tasks

5. Имя ключа title в JSON

6. Экспортируемые поля с заглавной буквы

7. Добавляет отступы для читаемого JSON

8. null

9. Нет, бизнес-валидацию нужно выполнить отдельно

10. os.ReadFile, затем json.Unmarshal

11. В отдельном внутреннем пакете storage

12. filepath.Join(t.TempDir(), "tasks.json")

Задания: сохранение задач в JSON

Теперь Task Manager API сохраняет не только текстовое представление, но и сами структуры задач. После загрузки данные снова доступны как `[]task.Task`.

Задание 1. Добавьте JSON-теги

В пакете `internal/task` добавьте JSON-теги полям `Task`:

```
ID        int    `json:"id"`
Title     string `json:"title"`
Priority  int    `json:"priority"`
Completed bool   `json:"completed"`
```

Создайте задачу в `main.go`, преобразуйте её через `json.MarshalIndent` и выведите JSON.

Критерии проверки:

- у всех четырёх полей есть JSON-теги в нижнем регистре;
- используется `json.MarshalIndent`;
- ошибка сериализации проверяется;
- вывод содержит ключи `id`, `title`, `priority`, `completed`.

Подсказка

```
data, err := json.MarshalIndent(currentTask, "", "  ")
```

Решение

`internal/task/task.go`:

```
type Task struct {
  ID        int    `json:"id"`
  Title     string `json:"title"`
  Priority  int    `json:"priority"`
  Completed bool   `json:"completed"`
}
```

`main.go`:

```
data, err := json.MarshalIndent(currentTask, "", "  ")
if err != nil {
  fmt.Println("Не удалось сериализовать задачу:", err)
  return
}

fmt.Println(string(data))
```

Задание 2. Сохраните список задач

В пакете `internal/storage` напишите функцию:

```
func SaveTasks(path string, tasks []task.Task) error
```

Она должна сериализовать список через `json.MarshalIndent` и записать его в `tasks.json` через `os.WriteFile`.

Критерии проверки:

- функция экспортируется и находится в пакете `storage`;
- функция принимает `[]task.Task` и возвращает `error`;
- JSON имеет отступы;
- ошибки сериализации и записи получают разный контекст через `%w`;
- файл `tasks.json` создаётся после успешного вызова;
- JSON содержит массив задач.

Подсказка

Сначала выполните `json.MarshalIndent`, а затем передайте полученные `data` в `os.WriteFile`.

Решение

```
func SaveTasks(path string, tasks []task.Task) error {
  data, err := json.MarshalIndent(tasks, "", "  ")
  if err != nil {
    return fmt.Errorf("не удалось сериализовать задачи: %w", err)
  }

  if err := os.WriteFile(path, data, 0o644); err != nil {
    return fmt.Errorf("не удалось сохранить задачи: %w", err)
  }

  return nil
}
```

Задание 3. Загрузите список задач

В том же пакете `internal/storage` напишите функцию:

```
func LoadTasks(path string) ([]task.Task, error)
```

Она должна прочитать `tasks.json` через `os.ReadFile`, распарсить JSON в `[]task.Task` и вернуть загруженные задачи.

Критерии проверки:

- функция экспортируется и находится в пакете `storage`;
- функция читает файл до вызова `json.Unmarshal`;
- `json.Unmarshal` получает `&tasks`;
- ошибка чтения и ошибка JSON получают разные сообщения;
- при ошибке возвращается `nil` и ненулевой `error`;
- успешный результат имеет тип `[]task.Task`.

Подсказка

```
var tasks []task.Task
if err := json.Unmarshal(data, &tasks); err != nil {
  // ...
}
```

Решение

```
func LoadTasks(path string) ([]task.Task, error) {
  data, err := os.ReadFile(path)
  if err != nil {
    return nil, fmt.Errorf("не удалось прочитать задачи: %w", err)
  }

  var tasks []task.Task
  if err := json.Unmarshal(data, &tasks); err != nil {
    return nil, fmt.Errorf("не удалось разобрать JSON задач: %w", err)
  }

  return tasks, nil
}
```

Задание 4. Проверьте полный цикл

Создайте две задачи, сохраните их в `tasks.json`, загрузите обратно в новую переменную `loadedTasks` и выведите `Summary()` каждой загруженной задачи.

Критерии проверки:

- сохранение выполняется до загрузки;
- каждая операция проверяет ошибку и завершает `main` при неудаче;
- `loadedTasks` получен через `storage.LoadTasks`;
- число и данные загруженных задач совпадают с сохранёнными;
- программа работает после удаления старого `tasks.json`.

Подсказка

```
if err := storage.SaveTasks("tasks.json", tasks); err != nil {
  fmt.Println(err)
  return
}

loadedTasks, err := storage.LoadTasks("tasks.json")
if err != nil {
  fmt.Println(err)
  return
}
```

Задание 5. Протестируйте JSON-хранилище

В `internal/storage/json_test.go` напишите тест полного цикла: сохраните список задач в путь внутри `t.TempDir()`, загрузите его через `LoadTasks` и сравните с исходным списком.

Затем добавьте отдельные subtests для:

- пустого слайса — в файле должен быть `[]`;
- несуществующего файла — ошибка распознаётся через `errors.Is(err, os.ErrNotExist)`;
- некорректного JSON — `LoadTasks` возвращает ошибку;
- JSON-тегов — файл содержит ключи `"id"` и `"title"`, а не `"ID"` и `"Title"`.

Критерии проверки:

- путь строится через `filepath.Join(t.TempDir(), "tasks.json")`;
- тест не создаёт файлы в рабочей папке проекта;
- успешный результат сравнивается с исходными задачами;
- для слайсов используется `reflect.DeepEqual` или сравнение по элементам;
- отсутствующий файл проверяется через `errors.Is`;
- `go test ./...` проходит успешно.

Подсказка

```
path := filepath.Join(t.TempDir(), "tasks.json")

if err := SaveTasks(path, wantTasks); err != nil {
  t.Fatalf("SaveTasks вернул ошибку: %v", err)
}

gotTasks, err := LoadTasks(path)
```

Решение: сохранение и загрузка

```
func TestSaveAndLoadTasks(t *testing.T) {
  path := filepath.Join(t.TempDir(), "tasks.json")
  wantTasks := []task.Task{
    {ID: 1, Title: "Купить молоко", Priority: 2},
  }

  if err := SaveTasks(path, wantTasks); err != nil {
    t.Fatalf("SaveTasks вернул ошибку: %v", err)
  }

  gotTasks, err := LoadTasks(path)
  if err != nil {
    t.Fatalf("LoadTasks вернул ошибку: %v", err)
  }

  if !reflect.DeepEqual(gotTasks, wantTasks) {
    t.Errorf("LoadTasks = %#v, ожидалось %#v", gotTasks, wantTasks)
  }
}
```

Для теста нужны импорты `path/filepath`, `reflect`, `testing` и пакет `task`.

Проверьте себя

Перед следующей главой убедитесь, что вы можете:

- добавить JSON-тег к полю структуры;
- преобразовать значение в JSON через `json.MarshalIndent`;
- восстановить слайс через `json.Unmarshal`;
- объяснить, почему `Unmarshal` получает указатель;
- сохранить и загрузить `[]task.Task` через пакет `storage`;

- отличить синтаксическую JSON-ошибку от бизнес-валидации задачи.
- протестировать JSON-хранилище во временной директории.

Рекомендации по коду: JSON

- Используйте JSON-теги для стабильного внешнего API, а не полагайтесь на имена Go-полей.
- Модели для API и внутренние модели разделяйте, если их правила и поля начинают различаться.
- Проверяйте ошибку и у `Marshal`, и у `Unmarshal`.
- Не считайте успешный `Unmarshal` бизнес-валидацией: проверяйте допустимость полей отдельно.
- Для пустого списка выбирайте `[]T{}`, если API должен вернуть `[]`, а не `null`.
- Добавляйте контекст к ошибкам чтения файла, сериализации и десериализации отдельно.
- Выносите файловый JSON API в отдельный пакет `internal/storage`, а не оставляйте его в `main`.
- Тестируйте сохранение и загрузку через `t.TempDir`, включая пустой список, неверный JSON и отсутствующий файл.

Чек-лист

Внешний JSON имеет предсказуемые имена полей, ошибки декодирования понятны, хранилище отделено от точки входа, а загруженные данные проходят бизнес-валидацию до использования.

Время и дедлайны

Цель урока

Научиться работать с датами, длительностями и UTC через пакет `time`, а также добавить задаче дедлайн и проверку срока выполнения.

Пакет `time` нужен не только для отображения дат. Он задаёт таймауты в concurrent-коде, помогает ограничивать дедлайны операций и позволяет хранить срок выполнения задачи.

Время: `time.Time`

`time.Time` представляет конкретный момент времени:

```
now := time.Now()
fmt.Println(now)
```

Для задачи добавим необязательное поле дедлайна:

```
type Task struct {
  ID        int       `json:"id"`
  Title     string    `json:"title"`
  Priority  int       `json:"priority"`
  Completed bool      `json:"completed"`
  DueAt     time.Time `json:"due_at"`
}
```

Для этой модели пакет `internal/task` должен импортировать `time`:

```
import "time"
```

Нулевое значение `time.Time` означает отсутствие установленной даты:

```
var dueAt time.Time
fmt.Println(dueAt.IsZero()) // true
```

Длительности: `time.Duration`

`time.Duration` хранит интервал времени:

```
duration := 2*time.Hour + 30*time.Minute
fmt.Println(duration) // 2h30m0s
```

Частые единицы:

```
time.Nanosecond
time.Microsecond
time.Millisecond
time.Second
time.Minute
time.Hour
```

Добавить длительность к моменту времени можно через `Add`:

```
tomorrow := time.Now().Add(24 * time.Hour)
```

Разница двух моментов — тоже `Duration`:

```
remaining := dueAt.Sub(time.Now())
```

Для проверки срока используйте `After`:

```
if time.Now().After(dueAt) {
  fmt.Println("Срок задачи истёк")
}
```

Форматирование времени

В Go нет шаблонов вроде `YYYY-MM-DD`. Формат задаёт эталонная дата:

```
const layout = "2006-01-02 15:04"

fmt.Println(time.Now().Format(layout))
```

Эталон — именно:

```
Mon Jan 2 15:04:05 MST 2006
```

Layout	Пример результата

`"2006-01-02"`	`2026-10-12`
`"2006-01-02 15:04"`	`2026-10-12 14:30`
`time.RFC3339`	`2026-10-12T14:30:00Z`

Для обмена данными и логов используйте `time.RFC3339`. Для интерфейса выбирайте понятный пользователю layout.

Разбор даты: `time.Parse`

`time.Parse` создаёт `time.Time` из строки и layout:

```
const layout = "2006-01-02 15:04"

dueAt, err := time.Parse(layout, "2026-12-31 18:00")
if err != nil {
  return fmt.Errorf("некорректный дедлайн: %w", err)
}
```

Без отдельного location `time.Parse` интерпретирует время в UTC. Если пользователь вводит время конкретного региона, используйте `time.ParseInLocation` с явно выбранной локацией. В первой версии backend-API проще хранить и передавать даты в UTC.

UTC

UTC — единый мировой стандарт времени. В API и JSON обычно передают момент времени в UTC:

```
now := time.Now().UTC()
fmt.Println(now.Format(time.RFC3339))
```

Преобразование к UTC не меняет сам момент времени, а только его представление и часовую зону.

`time.Time` автоматически сериализуется в JSON как строка RFC3339. Поле `DueAt` будет сохраняться вместе с задачей без специального кода.

Дедлайн Task

Добавим методы модели:

```
func (task Task) IsOverdue(now time.Time) bool {
  return !task.DueAt.IsZero() && now.After(task.DueAt)
}

func (task Task) Remaining(now time.Time) time.Duration {
```

```
  if task.DueAt.IsZero() {     return 0 } return task.DueAt.Sub(now)}
```

Внешний код передаёт `now` параметром. Это делает методы предсказуемыми и удобными для тестирования.

```
now := time.Now().UTC()

if currentTask.IsOverdue(now) {
  fmt.Println("Срок задачи истёк")
} else if !currentTask.DueAt.IsZero() {
  fmt.Println("Осталось:", currentTask.Remaining(now).Round(time.Minute))
}
```

Тестирование дедлайнов с фиксированным временем

Не вызывайте `time.Now()` прямо в unit-тесте проверки дедлайна. Реальное время постоянно меняется, поэтому такой тест может стать нестабильным. Создайте фиксированное значение `now` и передайте его в методы:

```
now := time.Date(2026, time.January, 1, 12, 0, 0, 0, time.UTC)
```

Табличный тест может проверить основные сценарии:

```
func TestTaskIsOverdue(t *testing.T) {
  now := time.Date(2026, time.January, 1, 12, 0, 0, 0, time.UTC)

  testCases := []struct {
    name    string
    dueAt   time.Time
    want    bool
  }{
    {
      name: "дедлайн не установлен",
    },
    {
      name:  "дедлайн в прошлом",
      dueAt:  now.Add(-time.Minute),
      want:   true,
    },
    {
      name:  "дедлайн равен текущему времени",
      dueAt:  now,
      want:   false,
```

```
    },    {     name:  "дедлайн в будущем",      dueAt:  now.Add(time.Hour),
want:   false,    },  } for _, testCase := range testCases {    t.Run(testCase.name,
func(t *testing.T) {       currentTask := Task{DueAt: testCase.dueAt}       if got :=
currentTask.IsOverdue(now); got != testCase.want {       t.Errorf("IsOverdue() = %t,
ожидалось %t", got, testCase.want)      }   })  }}
```

В текущем контракте дедлайн считается просроченным, только если `now` **строго позже** `DueAt`. Поэтому при точном равенстве `IsOverdue` возвращает `false`. Это правило должно быть явно зафиксировано тестом, чтобы его случайно не изменить при рефакторинге.

Проверьте `Remaining` отдельными сценариями:

- дедлайн не установлен → `0`;
- дедлайн через час → `time.Hour`;
- дедлайн в прошлом → отрицательная длительность, например `-time.Minute`.

Отрицательное значение `Remaining` само по себе не ошибка: его можно использовать для отображения просрочки. Если вызывающему коду нужна только оставшаяся положительная длительность, он должен отдельно проверить `IsOverdue`.

JSON round-trip для `DueAt`

`time.Time` автоматически кодируется в JSON строкой RFC3339. При сохранении и загрузке задачи момент времени должен остаться тем же:

```
func TestTaskDueAtJSONRoundTrip(t *testing.T) {
  wantDueAt := time.Date(2026, time.December, 31, 18, 0, 0, 0, time.UTC)
  wantTask := Task{
    ID:       1,
    Title:    "Купить молоко",
    Priority: 2,
    DueAt:    wantDueAt,
  }

  data, err := json.Marshal(wantTask)
  if err != nil {
    t.Fatalf("Marshal вернул ошибку: %v", err)
  }

  var gotTask Task
  if err := json.Unmarshal(data, &gotTask); err != nil {
    t.Fatalf("Unmarshal вернул ошибку: %v", err)
  }

  if !gotTask.DueAt.Equal(wantDueAt) {
```

```
    t.Errorf("DueAt = %s, ожидалось %s", gotTask.DueAt, wantDueAt)  }}
```

Для сравнения времени используйте `Equal`, а не `==`: JSON round-trip меняет внутреннее представление `time.Time`, но не сам момент времени.

Частые ошибки

Неверный layout даты

```
// time.Now().Format("YYYY-MM-DD") // это не формат Go
time.Now().Format("2006-01-02")
```

Сравнение времени через `==`

Для логики дат используйте методы:

```
first.Before(second)
first.After(second)
first.Equal(second)
```

`24 * time.Hour` как «завтра в то же локальное время»

Для UTC-дедлайнов это обычно нормально. При переходах летнего времени для календарных вычислений используйте `time.AddDate` и явную `Location`.

Тест с `time.Now()`

```
if task.IsOverdue(time.Now()) {
  // Такой тест зависит от момента запуска.
}
```

В unit-тесте создавайте фиксированный `now` через `time.Date` и передавайте его в метод.

Дальше

Закрепите дедлайн задач. Затем перейдём к goroutine: `time.Duration` и `time.Second` понадобятся для ожидания и таймаутов.

Самопроверка

1. Какой тип представляет конкретный момент времени?

A. time.Time

B. time.Context

C. time.String

D. time.Duration

2. Как проверить, что DueAt не установлен?

A. task.DueAt.Empty()

B. task.DueAt.IsZero()

C. len(task.DueAt) == 0

D. task.DueAt == nil

3. Что хранит time.Duration?

A. Только дату без времени

B. Часовой пояс пользователя

C. JSON-тег

D. Интервал времени

4. Как добавить один час к моменту now?

A. now.Add(time.Hour)

B. time.Add(now, 1)

C. now.Append(time.Hour)

D. now + time.Hour

5. Какой layout правильно отформатирует дату как 2026-10-12?

A. "DD.MM.YYYY"

B. "year-month-day"

C. "2006-01-02"

D. "YYYY-MM-DD"

6. Какой метод корректно проверяет, что дедлайн уже прошёл?

A. task.DueAt.Duration() < 0

B. len(task.DueAt) > 0

C. time.Now().After(task.DueAt)

D. time.Now() == task.DueAt

7. Что делает time.Parse без явно заданной Location?

A. Интерпретирует разобранное время в UTC

B. Возвращает только Duration

C. Всегда использует локальную зону компьютера

D. Автоматически определяет город пользователя

8. Зачем в API обычно используют UTC?

A. Чтобы автоматически показать время в городе пользователя

B. Чтобы передавать один однозначный момент времени между системами

C. Чтобы не хранить секунды

D. Чтобы time.Parse перестал возвращать error

9. Как time.Time сериализуется в JSON по умолчанию?

A. Никак, всегда возникает error

B. Как строка в формате RFC3339

C. Как map с полями времени

D. Как число миллисекунд

10. Почему метод IsOverdue принимает now параметром?

A. Так результат предсказуем и метод проще тестировать

B. time.Now нельзя вызвать внутри метода

C. Так DueAt автоматически становится UTC

D. Это обязательно для JSON

11. Какой результат должен дать IsOverdue(now), если DueAt точно равен now в текущем контракте?

A. false, потому что now ещё не позже дедлайна

B. nil

C. true, любой равный момент уже просрочен

D. panic

12. Как сравнить момент времени после JSON round-trip с исходным?

A. strings.Compare(gotDueAt, wantDueAt)

B. len(gotDueAt) == len(wantDueAt)

C. gotDueAt.Equal(wantDueAt)

D. gotDueAt == wantDueAt всегда

Ответы к самопроверке

1. time.Time

2. task.DueAt.IsZero()

3. Интервал времени

4. now.Add(time.Hour)

5. "2006-01-02"

6. time.Now().After(task.DueAt)

7. Интерпретирует разобранное время в UTC

8. Чтобы передавать один однозначный момент времени между системами

9. Как строка в формате RFC3339

10. Так результат предсказуем и метод проще тестировать

11. false, потому что now ещё не позже дедлайна

12. gotDueAt.Equal(wantDueAt)

Задания: дедлайн задачи

Task Manager API получает срок выполнения задачи. Добавим `DueAt`, разберём его из строки и определим, просрочена ли задача.

Задание 1. Добавьте дедлайн

Добавьте в `Task` поле:

```
DueAt time.Time `json:"due_at"`
```

В `main` разберите строку `2026-12-31 18:00` через `time.Parse`, создайте задачу с этим дедлайном и выведите его в формате `31.12.2026 18:00`.

Критерии проверки:

- пакет `time` импортирован там, где объявлено поле `DueAt`;
- используется layout `"2006-01-02 15:04"` для Parse;
- ошибка Parse обработана;
- для Format используется layout `"02.01.2006 15:04"`;
- дедлайн не равен нулевому времени.

Подсказка

```
const inputLayout = "2006-01-02 15:04"
dueAt, err := time.Parse(inputLayout, "2026-12-31 18:00")
```

Решение

```
const inputLayout = "2006-01-02 15:04"
const outputLayout = "02.01.2006 15:04"

dueAt, err := time.Parse(inputLayout, "2026-12-31 18:00")
if err != nil {
  fmt.Println("Некорректный дедлайн:", err)
  return
}

currentTask := task.Task{
  ID:       1,
  Title:    "Купить молоко",
  Priority: 2,
  DueAt:    dueAt,
}

fmt.Println(currentTask.DueAt.Format(outputLayout))
```

Задание 2. Проверьте срок задачи

Добавьте к `Task` методы:

```
func (task Task) IsOverdue(now time.Time) bool
func (task Task) Remaining(now time.Time) time.Duration
```

Задача без дедлайна не должна считаться просроченной, а `Remaining` для неё должен вернуть `0`. Для задачи с дедлайном в будущем выведите оставшееся время, округлённое до минуты.

Критерии проверки:

- `IsOverdue` проверяет `DueAt.IsZero()`;
- для сравнения используется `now.After(task.DueAt)`;
- `Remaining` использует `task.DueAt.Sub(now)`;
- методы принимают `now` параметром;
- результат выводится через `Round(time.Minute)`.

Подсказка

```
if task.DueAt.IsZero() {
  return false
}
```

Решение

```
func (task Task) IsOverdue(now time.Time) bool {
  return !task.DueAt.IsZero() && now.After(task.DueAt)
}

func (task Task) Remaining(now time.Time) time.Duration {
  if task.DueAt.IsZero() {
    return 0
  }

  return task.DueAt.Sub(now)
}
```

Задание 3. Отобразите статус дедлайна

Создайте задачу без дедлайна, задачу с прошлым сроком и задачу со сроком через 90 минут. Для каждой выведите один из статусов:

- `Дедлайн не установлен`;
- `Срок задачи истёк`;
- `Осталось: <длительность>`.

Критерии проверки:

- сначала проверяется `DueAt.IsZero()`;
- затем используется `IsOverdue(now)`;
- для будущего дедлайна вызывается `Remaining(now).Round(time.Minute)`;
- `now` создаётся один раз и передаётся всем проверкам.

Подсказка

```
if currentTask.DueAt.IsZero() {
  // ...
} else if currentTask.IsOverdue(now) {
  // ...
} else {
  // ...
}
```

Задание 4. Протестируйте дедлайн

В `internal/task/task_test.go` добавьте table-driven tests для `IsOverdue` и `Remaining`. Создайте фиксированное значение:

```
now := time.Date(2026, time.January, 1, 12, 0, 0, 0, time.UTC)
```

Проверьте сценарии:

- дедлайн отсутствует;
- дедлайн на минуту раньше `now`;
- дедлайн равен `now`;
- дедлайн через час после `now`.

Критерии проверки:

- тест не вызывает `time.Now()`;
- каждый сценарий запускается через `t.Run`;
- для `IsOverdue` проверяется `bool`;
- для `Remaining` проверяется ожидаемая `time.Duration`;
- равный `now` дедлайн в текущем контракте не считается просроченным;
- запуск `go test ./...` проходит успешно.

Подсказка

```
testCases := []struct {
  name          string
  dueAt         time.Time
  wantOverdue   bool
  wantRemaining time.Duration
}{
  // сценарии
}
```

Решение

```
func TestTaskDeadline(t *testing.T) {
  now := time.Date(2026, time.January, 1, 12, 0, 0, 0, time.UTC)

  testCases := []struct {
    name          string
    dueAt         time.Time
    wantOverdue   bool
    wantRemaining time.Duration
  }{
    {
      name: "дедлайн не установлен",
    },
    {
      name:          "дедлайн в прошлом",
      dueAt:         now.Add(-time.Minute),
      wantOverdue:   true,
      wantRemaining: -time.Minute,
    },
    {
      name: "дедлайн равен now",
      dueAt: now,
    },
    {
      name:          "дедлайн через час",
      dueAt:         now.Add(time.Hour),
      wantRemaining: time.Hour,
    },
  }

  for _, testCase := range testCases {
```

▼ Сведения

```
    t.Run(testCase.name, func(t *testing.T) {     currentTask := Task{DueAt:
testCase.dueAt}       if got := currentTask.IsOverdue(now); got !=
testCase.wantOverdue {         t.Errorf("IsOverdue() = %t, ожидалось %t", got,
testCase.wantOverdue)     }      if got := currentTask.Remaining(now); got !=
testCase.wantRemaining {         t.Errorf("Remaining() = %s, ожидалось %s", got,
testCase.wantRemaining)     }   })  }}
```

Задание 5. Проверьте JSON round-trip дедлайна

Создайте задачу с `DueAt` в UTC, сериализуйте её через `json.Marshal`, восстановите через `json.Unmarshal` и проверьте, что загруженный дедлайн описывает тот же момент времени.

Критерии проверки:

- дедлайн создаётся через `time.Date` в `time.UTC`;
- проверяются ошибки `Marshal` и `Unmarshal`;
- момент времени сравнивается через `gotTask.DueAt.Equal(wantDueAt)`;
- не используется `gotTask.DueAt == wantDueAt`;
- тест является независимым от текущего времени компьютера.

Подсказка

```
wantDueAt := time.Date(2026, time.December, 31, 18, 0, 0, 0, time.UTC)
```

Проверьте себя

Перед следующей главой убедитесь, что вы можете:

- создать и отформатировать `time.Time`;
- разобрать дату через `time.Parse`;
- проверить дедлайн и вычислить оставшееся время;
- хранить `time.Time` в JSON-модели задачи;
- объяснить, зачем API использует UTC.
- написать детерминированный тест дедлайна с фиксированным `now`.

Рекомендации по коду: время и дедлайны

- Храните и передавайте моменты времени в UTC, особенно в JSON и API.
- Используйте `time.RFC3339` для машинного обмена, а пользовательский layout — только для отображения.
- Не сравнивайте `time.Time` через `==` для бизнес-логики; используйте `Before`, `After` и `Equal`.
- Передавайте `now` параметром в методы дедлайна: код станет детерминированным и тестируемым.
- Явно обрабатывайте отсутствие дедлайна через `IsZero`.
- Различайте интервал `time.Duration` и календарные операции: для календарных дней и месяцев нужен `AddDate`.
- Проверяйте дедлайны с фиксированным `now`, созданным через `time.Date`, а не через `time.Now()`.
- После JSON round-trip сравнивайте моменты через `time.Equal`.

Чек-лист

В API нет неявной локальной зоны, дата парсится по явному layout, дедлайн без значения не ошибочно считается просроченным, а тесты времени не зависят от момента запуска.

Goroutine

Цель урока

Научиться запускать независимую работу параллельно через `go`, ждать её завершения с `sync.WaitGroup`, тестировать фоновых работников без `time.Sleep` и понимать, почему порядок выполнения goroutine нельзя считать предсказуемым.

В Task Manager API goroutine пригодятся для независимых операций: например, можно одновременно подготовить текстовый экспорт и JSON-файл задач. Каналы появятся в следующей главе; пока для ожидания завершения достаточно `WaitGroup`.

Что такое goroutine

Goroutine — лёгкая функция, которую Go runtime запускает конкурентно с остальным кодом. Чтобы запустить вызов в новой goroutine, поставьте перед ним `go`:

```
go printTask("Купить молоко")
```

Для сравнения:

```
printTask("Купить молоко")    // обычный последовательный вызов
go printTask("Оплатить интернет") // конкурентный запуск
```

Goroutine не означает, что работа обязательно будет выполняться на другом ядре процессора. Главное для начала: несколько задач могут продвигаться независимо, а Go runtime распределяет их выполнение.

Порядок выполнения не определён

Рассмотрим пример:

```
package main

import "fmt"

func printMessage(message string) {
  fmt.Println(message)
}

func main() {
```

```
  go printMessage("Первая goroutine")   go printMessage("Вторая goroutine")
fmt.Println("main завершает работу")}
```

Программа может вывести строки в разном порядке или не вывести сообщения goroutine вовсе: когда `main` завершается, завершается вся программа вместе с незавершёнными goroutine.

> **Не используйте time.Sleep для синхронизации**
>
> `time.Sleep` иногда используют в короткой демонстрации, чтобы «дать goroutine время». Это ненадёжно: на медленном компьютере времени может не хватить, а на быстром программа просто будет ждать зря. Используйте синхронизацию — в этой главе `sync.WaitGroup`.

Ожидание через `sync.WaitGroup`

`sync.WaitGroup` хранит число незавершённых работ:

- `Add(n)` увеличивает число ожидаемых goroutine на `n`;
- `Done()` уменьшает его на единицу;
- `Wait()` блокирует текущую goroutine, пока счётчик не станет равен нулю.

```
package main

import (
  "fmt"
  "sync"
)

func printTask(task string, wg *sync.WaitGroup) {
  defer wg.Done()
  fmt.Println(task)
}

func main() {
  var wg sync.WaitGroup

  wg.Add(2)
  go printTask("Купить молоко", &wg)
  go printTask("Оплатить интернет", &wg)

  wg.Wait()
  fmt.Println("Все задачи обработаны")
}
```

`defer wg.Done()` ставят в начало goroutine: это гарантирует уменьшение счётчика при любом обычном выходе из функции, включая ранний `return`.

Правила работы с WaitGroup

Вызывайте `Add` до запуска goroutine

Правильно:

```
wg.Add(1)
go work(&wg)
```

Неправильно:

```
go work(&wg)
wg.Add(1)
```

Во втором варианте goroutine может вызвать `Done` раньше `Add`, а программа завершится с panic.

Один `Add` — один `Done`

Если добавили две работы, каждая должна вызвать `Done` ровно один раз:

```
wg.Add(2)
go first(&wg)  // first вызывает Done
go second(&wg) // second вызывает Done
wg.Wait()
```

Забытый `Done` приведёт к вечному ожиданию в `Wait`. Лишний `Done` приведёт к panic из-за отрицательного счётчика.

Передавайте WaitGroup указателем

`sync.WaitGroup` нельзя копировать после первого использования. Передавайте `*sync.WaitGroup`:

```
func work(wg *sync.WaitGroup) {
  defer wg.Done()
}
```

Анонимная goroutine

Для короткой работы можно использовать анонимную функцию:

```
var wg sync.WaitGroup

wg.Add(1)
go func() {
  defer wg.Done()
  fmt.Println("Экспорт подготовлен")
}()

wg.Wait()
```

Круглые скобки `()` после закрывающей фигурной скобки немедленно вызывают анонимную функцию. Без них вы только объявили бы функцию, но не запустили её.

Goroutine в цикле

Чтобы запустить работу для каждого значения, передавайте значение параметром в анонимную функцию:

```
tasks := []string{"Купить молоко", "Оплатить интернет"}

var wg sync.WaitGroup

for _, title := range tasks {
  wg.Add(1)

  go func(currentTitle string) {
    defer wg.Done()
    fmt.Println("Обработка:", currentTitle)
  }(title)
}

wg.Wait()
```

Явный параметр `currentTitle` фиксирует значение для конкретного запуска и делает намерение кода очевидным. В современных версиях Go переменная каждой итерации `range` создаётся заново, но передача параметра остаётся понятным и безопасным стилем, особенно при более сложной логике.

Независимый параллельный экспорт

Текстовый и JSON-экспорт используют один и тот же неизменяемый список задач, но записывают разные файлы. Это хороший первый случай для goroutine: операции независимы.

```
func saveExports(tasks []task.Task, textPath, jsonPath string) (error, error) {
  var wg sync.WaitGroup
  var textErr error
  var jsonErr error

  wg.Add(2)

  go func() {
    defer wg.Done()
    textErr = exportText(textPath, tasks)
  }()

  go func() {
    defer wg.Done()
    jsonErr = storage.SaveTasks(jsonPath, tasks)
  }()

  wg.Wait()
  return textErr, jsonErr
}
```

Каждая goroutine записывает только свою переменную ошибки, а вызывающий код читает результаты только после `wg.Wait()`. Пути передаются параметрами: это позволяет не создавать файлы в рабочей папке при тестировании.

Использование:

```
textErr, jsonErr := saveExports(tasks, "tasks.txt", "tasks.json")

if textErr != nil {
  fmt.Println("Ошибка текстового экспорта:", textErr)
}
if jsonErr != nil {
  fmt.Println("Ошибка JSON-экспорта:", jsonErr)
}
```

Тестирование goroutine без `time.Sleep`

Тест не должен ждать «достаточно долго» в надежде, что работники уже завершились. Вместо этого он должен получить явный сигнал завершения. Для проверки параллельного экспорта удобно использовать временную директорию и канал результатов.

Каналы подробно появятся в следующей главе. Здесь достаточно использовать один буферизированный канал как тестовый контракт: каждый worker отправляет результат, а отдельная goroutine закрывает канал после `wg.Wait()`.

```
type exportResult struct {
  Name string
  Err  error
}

func startExports(tasks []task.Task, textPath, jsonPath string) <-chan exportResult {
  results := make(chan exportResult, 2)
  var wg sync.WaitGroup
  wg.Add(2)

  go func() {
    defer wg.Done()
    result := exportResult{
      Name: "text",
      Err:  exportText(textPath, tasks),
    }
    results <- result
  }()

  go func() {
    defer wg.Done()
    result := exportResult{
      Name: "json",
      Err:  storage.SaveTasks(jsonPath, tasks),
    }
    results <- result
  }()

  go func() {
    wg.Wait()
    close(results)
  }()

  return results
}
```

Тест читает канал до закрытия. Это означает, что оба worker завершились, и никаких искусственных пауз не нужно:

```
func TestStartExports(t *testing.T) {
  directory := t.TempDir()
  tasks := []task.Task{{ID: 1, Title: "Купить молоко", Priority: 2}}

  results := startExports(
    tasks,
    filepath.Join(directory, "tasks.txt"),
    filepath.Join(directory, "tasks.json"),
  )

  for result := range results {
    if result.Err != nil {
      t.Errorf("%s export returned error: %v", result.Name, result.Err)
    }
  }
}
```

В следующей главе разберём, почему отправка и получение через канал работают именно так, как закрывать канал при нескольких отправителях и как выражать направление канала в API.

Что пока не делать

Не запускайте goroutine просто «для ускорения». Конкурентный код становится сложнее: появляются гонки данных, ошибки синхронизации, утечки goroutine и неопределённый порядок.

Пока избегайте общего изменяемого состояния:

```
// Плохой пример: несколько goroutine меняют одну переменную.
// total++
```

Без синхронизации это создаёт гонку данных. `Mutex`, атомарные операции и `go test -race` разберём позже, после каналов и контекста.

Частые ошибки

`main` завершилась раньше goroutine

```
go exportText("tasks.txt", tasks)
fmt.Println("Готово")
```

Сообщение «Готово» не означает, что экспорт действительно завершился. Используйте `WaitGroup`.

Забытый `Done`

```
func export(wg *sync.WaitGroup) {
  // wg.Done() забыли
}
```

`wg.Wait()` будет ждать бесконечно. Добавляйте `defer wg.Done()` первой строкой тела goroutine.

Копирование WaitGroup

```
func export(wg sync.WaitGroup) {
  defer wg.Done()
}
```

Так функция получает копию счётчика, а исходный `WaitGroup` не изменяется. Передавайте `*sync.WaitGroup`.

Ожидание фиксированное время

```
go exportText("tasks.txt", tasks)
time.Sleep(time.Second)
```

Время не доказывает завершение работы. Синхронизация должна отражать событие завершения, а не предположение о скорости компьютера.

Тест без ожидания завершения

```
go saveExports(tasks, textPath, jsonPath)
// Сразу проверяется файл: worker мог ещё не успеть его создать.
```

В тесте дождитесь `WaitGroup` или прочитайте все результаты из канала, который закрывается после завершения worker.

Дальше

Закрепите параллельный экспорт задач. Затем перейдём к каналам: они позволят goroutine обмениваться результатами и сигналами без ручных переменных ошибок.

Самопроверка

1. Как запустить вызов функции в новой goroutine?

A. parallel functionCall()

B. async functionCall()

C. goroutine functionCall()

D. go functionCall()

2. Что происходит с незавершёнными goroutine при завершении main?

A. Go автоматически ждёт каждую из них

B. Они превращаются в каналы

C. Они завершаются вместе с программой

D. Они всегда продолжают работу в фоне

3. Для чего нужен sync.WaitGroup?

A. Чтобы дождаться завершения группы goroutine

B. Чтобы отсортировать слайс

C. Чтобы заменить все каналы

D. Чтобы записать JSON в файл

4. Что делает wg.Add(2)?

A. Ждёт две секунды

B. Создаёт две goroutine

C. Уменьшает счётчик на 2

D. Увеличивает число ожидаемых завершений на 2

5. Где обычно размещают defer wg.Done()?

A. Перед wg.Add()

B. В начале тела goroutine

C. После wg.Wait() в main

D. Только в импортах

6. Когда нужно вызвать wg.Add(1) относительно запуска go work(&wg)?

A. Порядок не важен

B. После wg.Wait()

C. Только внутри work

D. До запуска goroutine

7. Почему WaitGroup передают как *sync.WaitGroup?

A. Потому что WaitGroup содержит JSON

B. Потому что значения нельзя передавать в функции

C. Чтобы все goroutine работали с одним счётчиком, а не с копией

D. Чтобы автоматически избежать всех гонок данных

8. Почему time.Sleep не подходит для ожидания goroutine?

A. Его нельзя использовать в main

B. Он не гарантирует, что работа завершилась

C. time.Sleep всегда вызывает panic

D. Он создаёт канал

9. Какой порядок вывода нескольких goroutine гарантирован без дополнительной синхронизации?

A. По алфавиту имени функции

B. Никакой: порядок не определён

C. Строго в обратном порядке

D. Строго в порядке их запуска

10. Что опасно делать нескольким goroutine с одной переменной без синхронизации?

A. Выводить её через fmt.Println

B. Одновременно изменять её — это может вызвать гонку данных

C. Читать её после wg.Wait()

D. Передавать её функции как параметр

11. Как в тесте корректно дождаться завершения фоновых экспортов?

A. Через time.Sleep с большим запасом

B. Через WaitGroup или чтение канала, закрываемого после завершения worker

C. Через fmt.Println в main

D. Сразу проверить файл после запуска goroutine

12. Почему пути экспорта лучше передавать параметрами в saveExports?

A. JSON нельзя записывать по фиксированному пути

B. Go не разрешает строковые литералы в функции

C. Так WaitGroup автоматически закрывается

D. Тест может передать изолированные пути внутри t.TempDir

Ответы к самопроверке

1. go functionCall()

2. Они завершаются вместе с программой

3. Чтобы дождаться завершения группы goroutine

4. Увеличивает число ожидаемых завершений на 2

5. В начале тела goroutine

6. До запуска goroutine

7. Чтобы все goroutine работали с одним счётчиком, а не с копией

8. Он не гарантирует, что работа завершилась

9. Никакой: порядок не определён

10. Одновременно изменять её — это может вызвать гонку данных

11. Через WaitGroup или чтение канала, закрываемого после завершения worker

12. Тест может передать изолированные пути внутри t.TempDir

Задания: параллельный экспорт

Task Manager API уже умеет создавать текстовый и JSON-экспорт. Теперь запустим эти независимые операции параллельно и дождёмся завершения обеих.

Задание 1. Дождитесь двух работ

Напишите программу с двумя goroutine. Первая выводит `Текстовый экспорт готов`, вторая — `JSON-экспорт готов`. Используйте `sync.WaitGroup`, чтобы строка `Все экспорты готовы` появилась только после обеих.

Критерии проверки:

- импортирован пакет `sync`;
- до запуска есть `wg.Add(2)`;
- каждая goroutine вызывает `defer wg.Done()`;
- `wg.Wait()` находится перед финальным сообщением;
- порядок первых двух сообщений может отличаться.

Подсказка

```
var wg sync.WaitGroup
wg.Add(2)

go func() {
  defer wg.Done()
  // первая работа
}()
```

Решение

```
package main

import (
  "fmt"
  "sync"
)

func main() {
  var wg sync.WaitGroup
  wg.Add(2)

  go func() {
    defer wg.Done()
    fmt.Println("Текстовый экспорт готов")
  }()

  go func() {
    defer wg.Done()
    fmt.Println("JSON-экспорт готов")
  }()

  wg.Wait()
  fmt.Println("Все экспорты готовы")
}
```

Задание 2. Вынесите экспорт в функции

Создайте две функции:

```
func exportText(path string, tasks []task.Task) error
func exportJSON(path string, tasks []task.Task) error
```

Первая записывает summary задач в текстовый файл, вторая сохраняет список через JSON. Можно взять их реализацию из предыдущих глав.

Критерии проверки:

- обе функции возвращают `error`;
- текстовый экспорт пишет `tasks.txt`;
- JSON-экспорт пишет `tasks.json`;
- функции не запускают goroutine самостоятельно;
- обе функции можно вызвать последовательно для проверки.

Подсказка

Goroutine — ответственность вызывающего кода. Функция экспорта должна просто выполнить работу и вернуть ошибку.

Задание 3. Запустите оба экспорта параллельно

Напишите функцию:

```
func saveExports(tasks []task.Task, textPath, jsonPath string) (error, error)
```

Она должна параллельно вызвать текстовый и JSON-экспорт по переданным путям, дождаться обеих операций и вернуть две ошибки.

Критерии проверки:

- используется один `sync.WaitGroup`;
- до запуска есть `wg.Add(2)`;
- каждая goroutine записывает ошибку только в свою переменную;
- `wg.Wait()` вызывается до возврата ошибок;
- текстовый и JSON-файлы имеют разные пути.

Подсказка

```
var textErr error
var jsonErr error

go func() {
  defer wg.Done()
  textErr = exportText("tasks.txt", tasks)
}()
```

Решение

```
func saveExports(tasks []task.Task) (error, error) {
  var wg sync.WaitGroup
  var textErr error
  var jsonErr error

  wg.Add(2)

  go func() {
    defer wg.Done()
    textErr = exportText(textPath, tasks)
  }()

  go func() {
    defer wg.Done()
    jsonErr = storage.SaveTasks(jsonPath, tasks)
  }()

  wg.Wait()
  return textErr, jsonErr
}
```

Задание 4. Протестируйте параллельных работников

Напишите `startExports`, которая запускает текстовый и JSON-экспорт в goroutine, возвращает канал `<-chan exportResult` и закрывает его после завершения обоих worker через `WaitGroup`. Напишите тест во временной директории: он должен прочитать все результаты из канала и убедиться, что `tasks.txt` и `tasks.json` созданы.

Это первое точечное применение канала. Детально устройство каналов будет разобрано в следующей главе.

Критерии проверки:

- `WaitGroup` ожидает ровно две goroutine;
- каждая goroutine отправляет один `exportResult`;
- отдельная goroutine закрывает канал только после `wg.Wait()`;
- тест строит пути через `filepath.Join(t.TempDir(), ...)`;
- тест не использует `time.Sleep`;
- после чтения закрытого канала существуют оба экспортных файла.

Подсказка

```
go func() {
  wg.Wait()
  close(results)
}()

for result := range results {
  if result.Err != nil {
    t.Errorf("%s export returned error: %v", result.Name, result.Err)
  }
}
```

Фрагмент теста

```
func TestStartExports(t *testing.T) {
  directory := t.TempDir()
  tasks := []task.Task{{ID: 1, Title: "Купить молоко", Priority: 2}}

  results := startExports(
    tasks,
    filepath.Join(directory, "tasks.txt"),
    filepath.Join(directory, "tasks.json"),
  )

  for result := range results {
    if result.Err != nil {
      t.Errorf("%s export returned error: %v", result.Name, result.Err)
    }
  }
}
```

Задание 5. Обработайте результаты

Создайте две задачи, вызовите `saveExports` с двумя путями внутри временной папки и обработайте каждую ошибку отдельно. Если обе ошибки равны `nil`, выведите `Все экспорты готовы`.

Критерии проверки:

- обе ошибки сохраняются в отдельные переменные;
- ошибки выводятся с понятным контекстом;
- сообщение об успехе выводится только при `textErr == nil && jsonErr == nil`;
- `main` не использует `time.Sleep`.

Подсказка

```
directory := t.TempDir()
textErr, jsonErr := saveExports(
  tasks,
  filepath.Join(directory, "tasks.txt"),
  filepath.Join(directory, "tasks.json"),
)

if textErr != nil {
  fmt.Println("Ошибка текстового экспорта:", textErr)
}
if jsonErr != nil {
  fmt.Println("Ошибка JSON-экспорта:", jsonErr)
}
```

Проверьте себя

Перед следующей главой убедитесь, что вы можете:

- запустить функцию через `go`;
- объяснить, почему `main` не ждёт goroutine автоматически;
- правильно использовать `Add`, `Done` и `Wait`;
- объяснить, почему `defer wg.Done()` стоит в начале goroutine;
- дождаться двух независимых экспортов без `time.Sleep`;
- протестировать фоновых worker без ожидания фиксированное время;
- не полагаться на порядок выполнения goroutine.

Рекомендации по коду: goroutine

- Запускайте goroutine только для независимой работы с ясным способом завершения.
- Не позволяйте `main` завершиться раньше нужных goroutine: используйте `WaitGroup`, канал или context.
- Вызывайте `wg.Add` до запуска goroutine, а `defer wg.Done()` ставьте в начале её тела.
- Не передавайте `WaitGroup` по значению после начала работы.
- Не используйте `time.Sleep` как синхронизацию.
- Не полагайтесь на порядок выполнения или вывода goroutine.
- Передавайте пути, зависимости и входные данные параметрами, чтобы worker можно было изолированно тестировать.
- В тестах ждите явный сигнал завершения через `WaitGroup` или канал, а не фиксированное время.

Чек-лист

Для каждой goroutine понятно, кто её ждёт, как она завершится и что произойдёт при ошибке или отмене; тесты не зависят от скорости планировщика.

Каналы

Цель урока

Научиться передавать значения между goroutine через каналы, понимать блокировку операций, использовать буфер, закрывать канал и получать результаты параллельных экспортов Task Manager API.

Канал (`chan`) — типизированный путь передачи значений между goroutine. В отличие от общих переменных, канал помогает явно показать, кто отправляет результат и кто его получает.

Создание канала

Канал создают через `make` и указывают тип передаваемых значений:

```
messages := make(chan string)
results := make(chan error)
```

`chan string` передаёт строки, а `chan error` — ошибки. Нулевое значение канала — `nil`, с ним нельзя выполнять обычную отправку и получение: такая операция заблокируется навсегда.

Отправка и получение

Оператор `<-` используют с двух сторон:

```
messages <- "Экспорт готов" // отправить в канал
message := <-messages        // получить из канала
```

Простейший рабочий пример требует двух goroutine:

```
package main

import "fmt"

func main() {
  messages := make(chan string)

  go func() {
    messages <- "Текстовый экспорт готов"
  }()
```

```
  message := <-messages fmt.Println(message)}
```

Здесь `main` ждёт результат через `<-messages`, а анонимная goroutine отправляет строку. После получения обе goroutine могут завершиться.

Небуферизированный канал

Канал без размера — небуферизированный:

```
messages := make(chan string)
```

Отправка в него ждёт, пока другая goroutine будет готова получить значение. Получение ждёт, пока кто-то отправит значение. Это называют синхронной передачей.

Такой код завершится deadlock:

```
messages := make(chan string)
messages <- "сообщение" // main ждёт получателя, но получить некому
```

Нельзя сначала отправить, а затем получить в той же goroutine через небуферизированный канал. Нужна другая goroutine или буфер.

Буферизированный канал

Размер канала указывают вторым аргументом `make`:

```
messages := make(chan string, 2)
```

Такой канал хранит до двух сообщений без немедленного получателя:

```
messages := make(chan string, 2)

messages <- "Текстовый экспорт готов"
messages <- "JSON-экспорт готов"

fmt.Println(<-messages)
fmt.Println(<-messages)
```

Третья отправка будет ждать, пока освободится место. Буфер не отменяет необходимость продумать синхронизацию: он только допускает ограниченное число отправок без ожидания.

Функции `len` и `cap` показывают текущее число сообщений и вместимость канала:

```
messages := make(chan string, 2)
messages <- "готово"

fmt.Println(len(messages)) // 1
fmt.Println(cap(messages)) // 2
```

В прикладном коде не принимайте важные решения только по `len(channel)`: состояние может измениться другой goroutine сразу после проверки.

Закрытие канала

Отправитель закрывает канал, когда новых значений больше не будет:

```
close(messages)
```

Получать уже отправленные значения из закрытого канала можно. После опустошения получение вернёт нулевое значение типа и `false` во втором результате:

```
message, open := <-messages
if !open {
  fmt.Println("Канал закрыт")
}
```

Для `chan string` нулевое значение — пустая строка. Поэтому проверка второго результата нужна, если пустая строка может быть настоящим сообщением.

```
message, open := <-messages
fmt.Println(message, open)
```

Закрывать канал должен тот, кто отправляет значения. Получатель не знает, будут ли ещё сообщения, поэтому обычно не вызывает `close`.

> **Ошибочные операции с закрытым каналом**
>
> Повторный `close` и отправка в закрытый канал вызывают panic. Получение из закрытого канала безопасно.

`range` по каналу

`range` получает значения из канала до его закрытия и опустошения:

```
package main

import "fmt"

func main() {
  messages := make(chan string)

  go func() {
    defer close(messages)

    messages <- "Текстовый экспорт готов"
    messages <- "JSON-экспорт готов"
  }()

  for message := range messages {
    fmt.Println(message)
  }
}
```

Если канал не закрыть, `range` будет ждать следующее сообщение вечно. Поэтому `defer close(messages)` удобно ставить в goroutine, которая владеет отправкой всех сообщений.

Направление канала в параметрах

Функция может принимать канал только для отправки или только для получения:

```
func sendStatus(statuses chan<- string, message string) {
  statuses <- message
}

func printStatuses(statuses <-chan string) {
  for message := range statuses {
    fmt.Println(message)
  }
}
```

- `chan<- string` — канал только для отправки;
- `<-chan string` — канал только для получения;
- `chan string` — двунаправленный канал.

Направление защищает API функции: компилятор не даст получить из `chan<- string` или отправить в `<-chan string`.

Результаты параллельных экспортов

В прошлой главе `WaitGroup` ожидал два экспорта, а ошибки записывались в отдельные переменные. Теперь каждая goroutine передаёт свой результат в канал. Так получатель может обработать результаты по мере готовности и узнать момент завершения всех отправителей через закрытие канала.

```
type exportResult struct {
  Name string
  Err  error
}

func startExports(tasks []task.Task, textPath, jsonPath string) <-chan exportResult {
  results := make(chan exportResult, 2)
  var wg sync.WaitGroup
  wg.Add(2)

  go func() {
    defer wg.Done()
    err := exportText(textPath, tasks)
    results <- exportResult{Name: "текстовый экспорт", Err: err}
  }()

  go func() {
    defer wg.Done()
    err := storage.SaveTasks(jsonPath, tasks)
    results <- exportResult{Name: "JSON-экспорт", Err: err}
  }()

  go func() {
    wg.Wait()
    close(results)
  }()

  return results
}
```

У канала буфер размером `2`: обе goroutine могут отправить результат, даже если получатель начнёт читать их чуть позже. `WaitGroup` нужен только координатору, который закрывает канал после завершения обоих отправителей. Возвращаемое значение `<-chan exportResult` сообщает вызывающему коду: он может только получать результаты.

Получатель может использовать `range` и не знать количество worker заранее:

```
results := startExports(tasks, "tasks.txt", "tasks.json")

for result := range results {

  if result.Err != nil {
    fmt.Printf("Ошибка %s: %v\n", result.Name, result.Err)
    continue
  }

  fmt.Printf("%s готов\n", result.Name)
}
```

Закрытие `results` — часть контракта `startExports`: после `range` вызывающий код знает, что все экспортные worker завершились. Получатель не закрывает канал, потому что не владеет отправителями.

Тестирование жизненного цикла канала

Тест worker не должен использовать `time.Sleep` или получать «ровно два» результата по счётчику. Чтение `range` до закрытия канала одновременно проверяет три свойства:

1. каждый worker отправил результат;
2. координатор дождался worker через `WaitGroup`;
3. канал корректно закрыт.

```
func TestStartExports(t *testing.T) {
  directory := t.TempDir()

  results := startExports(
    []task.Task{{ID: 1, Title: "Купить молоко", Priority: 2}},
    filepath.Join(directory, "tasks.txt"),
    filepath.Join(directory, "tasks.json"),
  )

  resultCount := 0
  for result := range results {
    resultCount++
    if result.Err != nil {
      t.Errorf("%s export returned error: %v", result.Name, result.Err)
    }
  }

  if resultCount != 2 {
```

```
    t.Errorf("получено результатов: %d, ожидалось 2", resultCount)  }}
```

После `range` отдельно проверьте существование обоих файлов через `os.Stat`. Пути из `t.TempDir` изолируют тест от рабочей папки проекта.

Частые ошибки

Отправка без получателя в небуферизированный канал

```
results := make(chan error)
results <- nil // deadlock, если получить некому
```

Запустите отправку в другой goroutine либо используйте канал с буфером, если это соответствует сценарию.

`range` по незакрытому каналу

```
for result := range results {
  fmt.Println(result)
}
```

Этот цикл закончится только после `close(results)`. Если канал никогда не закрывается, программа зависнет.

Закрытие канала получателем

Получатель не должен закрывать канал, если другой код ещё может отправить значение. Это может вызвать `panic: send on closed channel`.

Несколько отправителей закрывают один канал

```
go func() {
  results <- textResult
  close(results)
}()

go func() {
  results <- jsonResult // может отправить в уже закрытый канал
}()
```

Когда отправителей несколько, отдельная координирующая goroutine должна дождаться их через `WaitGroup` и только затем вызвать `close(results)`.

Использование канала как мьютекса без понимания

Канал не является универсальной заменой `Mutex`, `WaitGroup` или нормальной структуры программы. Сначала определите, передаёте ли вы данные между goroutine или защищаете общее состояние. `Mutex` подробно разберём позднее.

Дальше

Закрепите передачу результатов экспортов через каналы. В следующей главе `select` позволит ждать один из нескольких каналов, добавлять таймауты и не блокировать работу, если результата пока нет.

Самопроверка

1. Как создать небуферизированный канал строк?

A. new(chan string, 1)

B. make(chan string)

C. chan string{}

D. make([]string)

2. Как отправить значение message в канал messages?

A. messages.send(message)

B. messages => message

C. messages <- message

D. message <- messages

3. Как получить значение из канала messages?

A. <-message = messages

B. message := messages<-

C. message := messages.receive()

D. message := <-messages

4. Что происходит при отправке в небуферизированный канал без готового получателя?

A. Отправка блокируется

B. Значение удаляется

C. Канал автоматически закрывается

D. Значение автоматически сохраняется навсегда

5. Что означает make(chan string, 2)?

A. Два небуферизированных канала

B. Канал, который можно читать только два раза

C. Канал строк с буфером на два значения

D. Слайс из двух каналов

6. Кто обычно закрывает канал?

A. Получатель сразу после первого чтения

B. WaitGroup автоматически

C. Любая goroutine без согласования

D. Отправитель, когда новых значений больше не будет

7. Что произойдёт при отправке в закрытый канал?

A. Канал снова откроется

B. Вернётся false

C. Будет panic

D. Значение будет проигнорировано

8. Когда завершается for value := range values?

A. Сразу после запуска goroutine

B. После первого значения

C. Когда канал заполняет буфер

D. Когда канал закрыт и все отправленные значения получены

9. Что означает параметр типа <-chan error?

A. Слайс ошибок

B. Канал ошибок только для получения

C. Канал ошибок только для отправки

D. Указатель на канал ошибок

10. Почему в saveExports удобно использовать буфер results размером 2?

A. Буфер заменяет все проверки error

B. Канал автоматически проверяет ошибки

C. Две goroutine могут отправить результаты без немедленного получателя

D. Это гарантирует порядок результатов

11. Как корректно закрыть results, если в него отправляют несколько goroutine?

A. Пусть каждый отправитель вызывает close(results)

B. Дождаться всех отправителей через WaitGroup и закрыть канал в отдельной goroutine

C. Пусть получатель закрывает канал после первого результата

D. Не нужно ждать отправителей перед close

12. Что доказывает завершение for result := range results в тесте экспортов?

A. Результаты пришли строго в ожидаемом порядке

B. WaitGroup можно передавать по значению

C. Файлы автоматически удалены

D. Канал закрыт и все отправленные результаты получены

Ответы к самопроверке

1. make(chan string)

2. messages <- message

3. message := <-messages

4. Отправка блокируется

5. Канал строк с буфером на два значения

6. Отправитель, когда новых значений больше не будет

7. Будет panic

8. Когда канал закрыт и все отправленные значения получены

9. Канал ошибок только для получения

10. Две goroutine могут отправить результаты без немедленного получателя

11. Дождаться всех отправителей через WaitGroup и закрыть канал в отдельной goroutine

12. Канал закрыт и все отправленные результаты получены

Задания: результаты экспортов через канал

В прошлой главе `WaitGroup` только ожидал завершения экспортов. Теперь каждая goroutine передаёт в `main` собственный результат через канал.

Задание 1. Передайте одно сообщение

Создайте `chan string`. Запустите goroutine, которая отправит в него `Экспорт готов`. В `main` получите и выведите сообщение.

Критерии проверки:

- канал создан через `make(chan string)`;
- отправка находится внутри goroutine;
- `main` получает строку через `<-`;
- программа завершается без `time.Sleep`.

Подсказка

```
messages := make(chan string)

go func() {
  messages <- "Экспорт готов"
}()
```

Решение

```
package main

import "fmt"

func main() {
  messages := make(chan string)

  go func() {
    messages <- "Экспорт готов"
  }()

  fmt.Println(<-messages)
}
```

Задание 2. Передайте два статуса

Создайте буферизированный канал строк на два значения. В двух goroutine отправьте статусы текстового и JSON-экспортов. Получите и выведите оба статуса.

Критерии проверки:

- используется `make(chan string, 2)`;
- запущены две goroutine;
- каждая отправляет один статус;
- получены два значения;
- код не предполагает фиксированный порядок первых двух строк.

Подсказка

Получите сообщения дважды:

```
fmt.Println(<-statuses)
fmt.Println(<-statuses)
```

Решение

```
package main

import "fmt"

func main() {
  statuses := make(chan string, 2)

  go func() {
    statuses <- "Текстовый экспорт готов"
  }()

  go func() {
    statuses <- "JSON-экспорт готов"
  }()

  fmt.Println(<-statuses)
  fmt.Println(<-statuses)
}
```

Задание 3. Верните результаты экспортов

Создайте тип:

```
type exportResult struct {
  Name string
  Err  error
}
```

Напишите функцию:

```
func startExports(tasks []task.Task, textPath, jsonPath string) <-chan exportResult
```

Она запускает текстовый и JSON-экспорт в отдельных goroutine, возвращает канал с результатами и закрывает его после завершения обоих worker.

Критерии проверки:

- возвращаемый тип — `<-chan exportResult`;
- внутри создан `make(chan exportResult, 2)`;
- каждая goroutine отправляет `Name` и ошибку своей операции;
- `WaitGroup` ожидает завершения обоих отправителей;
- отдельная goroutine закрывает results после `wg.Wait()`;
- обе операции получают одинаковый слайс задач только для чтения.

Подсказка

```
go func() {
  wg.Wait()
  close(results)
}()
```

Решение

```
type exportResult struct {
  Name string
  Err  error
}

func startExports(tasks []task.Task, textPath, jsonPath string) <-chan exportResult {
  results := make(chan exportResult, 2)
  var wg sync.WaitGroup
  wg.Add(2)

  go func() {
    defer wg.Done()
    err := exportText(textPath, tasks)
    results <- exportResult{Name: "текстовый экспорт", Err: err}
  }()

  go func() {
    defer wg.Done()
    err := storage.SaveTasks(jsonPath, tasks)
    results <- exportResult{Name: "JSON-экспорт", Err: err}
  }()

  go func() {
    wg.Wait()
    close(results)
  }()

  return results
}
```

Задание 4. Обработайте оба результата

В `main` создайте две задачи, вызовите `startExports` с путями из временной директории и прочитайте результаты через `range`. Для каждого успешного результата выведите `<название> готов`, для ошибки — `Ошибка <название>: <ошибка>`.

Критерии проверки:

- результаты получаются через `range` до закрытия канала;
- успех и ошибка обрабатываются отдельно;
- нет `time.Sleep`;
- `main` не предполагает порядок текстового и JSON-результата;
- при успешных операциях создаются `tasks.txt` и `tasks.json`.

Подсказка

Канал закрывается координатором после завершения worker, поэтому можно читать его через `range`:

```
for result := range results {
  // обработка
}
```

Возможное решение

```
results := startExports(tasks, textPath, jsonPath)

for result := range results {

  if result.Err != nil {
    fmt.Printf("Ошибка %s: %v\n", result.Name, result.Err)
    continue
  }

  fmt.Printf("%s готов\n", result.Name)
}
```

Задание 5. Протестируйте результаты и закрытие

Напишите `TestStartExports`. Передайте пути из `t.TempDir`, прочитайте канал через `range`, проверьте, что получены два результата без ошибок, а оба файла существуют после завершения цикла.

Критерии проверки:

- тест не использует `time.Sleep`;
- `range` завершился, значит канал был закрыт;
- получено ровно два результата;
- каждый `exportResult.Err` равен `nil`;
- `tasks.txt` и `tasks.json` существуют в временной директории;
- тест запускается с `go test -race ./...`.

Подсказка

Проверьте себя

Перед следующей главой убедитесь, что вы можете:

- создать канал нужного типа;
- отправить и получить значение через `<-`;
- объяснить блокировку небуферизированного канала;
- выбрать буфер для двух известных результатов;
- объяснить, кто и когда должен закрывать канал;
- закрыть канал после нескольких отправителей через `WaitGroup`;
- протестировать результаты и закрытие канала без `time.Sleep`;
- вернуть канал только для чтения из функции.

Рекомендации по коду: каналы

- Используйте канал для передачи значений или сигналов между goroutine, а не как универсальный инструмент синхронизации.
- Выбирайте размер буфера по конкретной модели нагрузки, а не для случайного устранения deadlock.
- Закрывает канал только сторона, которая знает, что новых отправок больше не будет.
- Не отправляйте в закрытый канал и не закрывайте его дважды.
- Для `range` по каналу всегда должен быть понятный момент его закрытия.
- Используйте направленные каналы в сигнатурах, если функция только читает или только пишет.
- При нескольких отправителях используйте отдельную координирующую goroutine: она вызывает `close` только после `WaitGroup.Wait`.
- Тестируйте жизненный цикл канала чтением через `range` до закрытия, а не `time.Sleep`.

Чек-лист

У канала понятны владелец, отправители, получатели, правило закрытия и поведение, если получатель больше не ждёт результат; тесты проверяют все результаты и завершение `range`.

Select и таймауты

Цель урока

Научиться ждать несколько операций через `select`, выполнять неблокирующую проверку с `default`, ограничивать ожидание через `time.After`, корректно обрабатывать закрытые каналы и тестировать таймаут без реальной паузы.

В Task Manager API текстовый и JSON-экспорты уже отправляют результаты в канал. Теперь ограничим общее время ожидания, чтобы программа не зависала бесконечно, если одна операция не завершилась.

Что делает `select`

`select` ждёт, пока станет готова одна из операций с каналом. Его синтаксис похож на `switch`, но `case` содержит отправку или получение из канала:

```
select {
case message := <-messages:
  fmt.Println(message)
case err := <-errors:
  fmt.Println(err)
}
```

Если готовы несколько `case`, Go выбирает один из них случайно. Если ни один пока не готов, `select` блокируется, пока хотя бы один не станет готов.

Ожидание нескольких каналов

Пусть две goroutine отправляют сообщения в разные каналы:

```
package main

import "fmt"

func main() {
  textExport := make(chan string)
  jsonExport := make(chan string)

  go func() {
    textExport <- "Текстовый экспорт готов"
  }()
```

```
  go func() {     jsonExport <- "JSON-экспорт готов"  }()  for range 2 {     select {
case message := <-textExport:       fmt.Println(message)     case message := <-
jsonExport:       fmt.Println(message)     } }}
```

Порядок первых двух сообщений не гарантирован: `select` обработает тот канал, который готов первым.

`default`: не блокироваться

Ветка `default` выполняется сразу, если остальные операции не готовы:

```
select {
case message := <-messages:
  fmt.Println("Получено:", message)
default:
  fmt.Println("Сообщений пока нет")
}
```

Это полезно для проверки «есть ли результат прямо сейчас». Но `default` легко создать активный бесконечный цикл, который постоянно использует CPU:

```
// Не делайте так без паузы или другой логики:
for {
  select {
  case message := <-messages:
    fmt.Println(message)
  default:
  }
}
```

В большинстве задач лучше дождаться результата или использовать таймаут.

Таймаут через `time.After`

`time.After` возвращает канал, в который придёт значение времени после указанной задержки:

```
select {
case message := <-messages:
  fmt.Println("Получено:", message)
case <-time.After(2 * time.Second):
  fmt.Println("Время ожидания истекло")
}
```

В этом `select` будет обработано одно из двух событий:

- пришло сообщение;
- прошло две секунды.

`time.After` удобно для одного простого таймаута. Если таймер нужно останавливать, сбрасывать или использовать много раз, применяют `time.NewTimer`; это более продвинутый случай, который появится позже.

Общий таймаут для нескольких результатов

Создадим один таймер перед циклом. Тогда у всех результатов будет общий лимит времени, а не новый таймаут для каждой итерации:

```
timeout := time.After(3 * time.Second)

for received := 0; received < 2; {
  select {
  case result := <-results:
    fmt.Println(result)
    received++
  case <-timeout:
    fmt.Println("Не все экспорты завершились вовремя")
    return
  }
}
```

Если создать `time.After` внутри цикла, каждая итерация получит новый лимит, и общее время ожидания может стать больше задуманного.

Закрытый канал в select

Получение из закрытого канала готово немедленно. Поэтому важно проверять второй результат:

```
select {
case message, open := <-messages:
  if !open {
    fmt.Println("Канал закрыт")
    return
  }

  fmt.Println(message)
}
```

Если канал закрыт, но оставить `case message := <-messages` в цикле, этот `case` будет постоянно готов и может мешать обработке других событий. Обычно закрытый канал отключают, присваивая переменной `nil`:

```
for messages != nil {
  select {
  case message, open := <-messages:
    if !open {
      messages = nil
      continue
    }

    fmt.Println(message)
  }
}
```

Операции с `nil`-каналом никогда не готовы, поэтому соответствующий `case` временно исключается из `select`.

Таймаут экспортов Task Manager API

Из предыдущей главы у нас есть канал результатов:

```
type exportResult struct {
  Name string
  Err  error
}

func startExports(tasks []task.Task, textPath, jsonPath string) <-chan exportResult {
  // запускает два экспорта и возвращает канал с двумя результатами
}
```

Разделим ожидание и создание таймера. В production-коде обёртка использует `time.After`:

```
func waitForExports(results <-chan exportResult, timeout time.Duration) error {
  return waitForExportsUntil(results, time.After(timeout))
}

func waitForExportsUntil(results <-chan exportResult, timeout <-chan time.Time) error
{
  for {
    select {
    case result, open := <-results:
```

```
      if !open {        return nil     }     if result.Err != nil {         return
fmt.Errorf("%s завершился с ошибкой: %w", result.Name, result.Err)     }
fmt.Printf("%s готов\n", result.Name)   case <-timeout:       return
fmt.Errorf("экспорты не завершились вовремя")    } }}
```

`waitForExportsUntil` принимает готовый канал таймаута. Это делает её детерминированной в тесте: вместо реальных секунд тест может передать уже закрытый канал.

Вызов из `main`:

```
results := startExports(tasks, "tasks.txt", "tasks.json")

if err := waitForExports(results, 3*time.Second); err != nil {
  fmt.Println("Ошибка экспорта:", err)
  return
}
```

Функция не предполагает порядок результатов: текстовый и JSON-экспорт могут завершаться в любом порядке.

Тестирование таймаута без ожидания

Тест не должен ждать три реальные секунды. Создайте канал `chan time.Time`, закройте его до вызова функции и передайте в `waitForExportsUntil`:

```
func TestWaitForExportsUntilTimeout(t *testing.T) {
  results := make(chan exportResult)
  timeout := make(chan time.Time)
  close(timeout)

  err := waitForExportsUntil(results, timeout)
  if err == nil {
    t.Fatal("ожидалась ошибка таймаута")
  }
}
```

Закрытый канал готов к получению немедленно, поэтому `select` выбирает ветвь таймаута без паузы. Тест не зависит от скорости машины или планировщика.

Проверьте отдельно успешное завершение: создайте буферизированный канал, отправьте в него результаты, закройте и убедитесь, что `waitForExportsUntil` возвращает `nil`.

Почему таймаут не отменяет работу

`time.After` прекращает **ожидание результата** в текущей goroutine, но не останавливает экспортные goroutine автоматически. Они могут завершиться позже и попытаться отправить результат.

В нашем примере используется буферизированный канал на два результата, поэтому поздняя отправка не заблокирует goroutine. Для настоящей отмены работы используют `context`, который появится в отдельной главе.

Частые ошибки

Таймер внутри цикла вместо общего лимита

```
for range 2 {
  select {
  case result := <-results:
    fmt.Println(result)
  case <-time.After(3 * time.Second):
    return
  }
}
```

Это даёт до трёх секунд на каждый результат. Создайте один `timeout := time.After(...)` перед циклом, если нужен общий лимит.

Ожидание закрытого канала без `ok`

```
case result := <-results:
  // После закрытия result будет нулевым значением и case всегда готов.
```

Получайте второй результат `open` и завершайте или отключайте закрытый канал.

`default` вместо ожидания

```
select {
case result := <-results:
  fmt.Println(result)
default:
  fmt.Println("Нет результата")
}
```

Такой код не ждёт экспорт. Если результат должен прийти, используйте обычный `select` или таймаут.

Реальный `time.After` в unit-тесте

```
// Тест будет ждать реальную секунду.
err := waitForExports(results, time.Second)
```

Тестируйте `waitForExportsUntil` и передавайте готовый timeout-канал. Обёртка с `time.After` остаётся короткой и не требует отдельной задержки в тесте.

Закрытие чужого канала

Функция `waitForExports` только получает результаты, поэтому не должна закрывать `results`. Закрывать канал имеет право код, который владеет всеми отправителями.

Дальше

Закрепите ожидание экспортов с общим таймаутом. Затем перейдём к `Mutex` и синхронизации общего состояния: это дополнит каналы в задачах, где данные нужно не передавать, а безопасно изменять из нескольких goroutine.

Самопроверка

1. Что ждёт select без default?

A. Ровно одну секунду

B. Только первый case по порядку

C. Строго все case одновременно

D. Пока хотя бы одна операция с каналом не станет готова

2. Что произойдёт, если в select одновременно готовы несколько case?

A. Будет panic

B. Go выберет один из готовых case

C. Всегда выполнится первый case в коде

D. Выполнятся все готовые case

3. Когда выполняется default в select?

A. После каждого case

B. Когда ни один другой case не готов сразу

C. Только после закрытия канала

D. Через заданный таймаут

4. Что возвращает time.After(2 * time.Second)?

A. WaitGroup

B. Канал, в который придёт время примерно через две секунды

C. Число миллисекунд

D. Закрытый канал строк

5. Где создать time.After, если два результата должны уложиться в общий таймаут?

A. После цикла

B. Внутри каждого case с результатом

C. Внутри exportText

D. Перед циклом получения результатов

6. Что вернёт получение из закрытого и опустошённого канала с формой value, open := <-channel?

A. nil и ошибку

B. Только panic

C. Последнее значение и true всегда

D. Нулевое значение и false

7. Зачем присваивать закрытому каналу nil внутри select?

A. Чтобы снова открыть канал

B. Чтобы его case больше не был готов и не мешал другим case

C. Чтобы ускорить JSON-сериализацию

D. Чтобы удалить все сообщения из канала

8. Останавливает ли time.After экспортные goroutine автоматически?

A. Да, все goroutine завершаются сразу

B. Да, но только JSON-экспорт

C. Нет, он прекращает только ожидание в текущем select

D. Только если канал закрыт

9. Кто должен закрывать канал результатов exportResult?

A. Любой получатель после первого результата

B. Код, который владеет всеми отправителями результатов

C. Функция time.After

D. fmt.Println

10. Почему default в бесконечном цикле без паузы опасен?

A. Go перестаёт выполнять goroutine

B. Цикл может постоянно использовать CPU, не блокируясь

C. Он удаляет результаты из канала

D. default автоматически закрывает канал

11. Зачем waitForExportsUntil принимает timeout <-chan time.Time вместо готовой duration?

A. Чтобы отправлять JSON в канал

B. Чтобы тест мог передать готовый канал таймаута без реальной паузы

C. Чтобы time.After перестал работать

D. Чтобы автоматически отменить goroutine

12. Как в тесте мгновенно имитировать истечение таймаута?

A. Создать chan time.Time и закрыть его до вызова функции

B. Передать nil вместо timeout

C. Вызвать time.Sleep(time.Second)

D. Закрыть results до запуска select

Ответы к самопроверке

1. Пока хотя бы одна операция с каналом не станет готова

2. Go выберет один из готовых case

3. Когда ни один другой case не готов сразу

4. Канал, в который придёт время примерно через две секунды

5. Перед циклом получения результатов

6. Нулевое значение и false

7. Чтобы его case больше не был готов и не мешал другим case

8. Нет, он прекращает только ожидание в текущем select

9. Код, который владеет всеми отправителями результатов

10. Цикл может постоянно использовать CPU, не блокируясь

11. Чтобы тест мог передать готовый канал таймаута без реальной паузы

12. Создать chan time.Time и закрыть его до вызова функции

Задания: таймаут экспортов

Task Manager API получает результаты экспортов через канал. Теперь добавим общий лимит ожидания и понятную ошибку, если не все операции завершились вовремя.

Задание 1. Получите первый готовый результат

Создайте два канала строк. В двух goroutine отправьте сообщения `Текстовый экспорт готов` и `JSON-экспорт готов`. Используйте `select`, чтобы вывести то сообщение, которое пришло первым.

Критерии проверки:

- созданы два канала;
- каждая goroutine отправляет сообщение в свой канал;
- `select` содержит два case получения;
- программа выводит одно из двух сообщений;
- порядок готовности не предполагается.

Подсказка

```
select {
case message := <-textExport:
  fmt.Println(message)
case message := <-jsonExport:
  fmt.Println(message)
}
```

Решение

```
package main

import "fmt"

func main() {
  textExport := make(chan string)
  jsonExport := make(chan string)

  go func() {
    textExport <- "Текстовый экспорт готов"
  }()

  go func() {
    jsonExport <- "JSON-экспорт готов"
  }()

  select {
  case message := <-textExport:
    fmt.Println(message)
  case message := <-jsonExport:
    fmt.Println(message)
  }
}
```

Задание 2. Добавьте таймаут одного результата

Измените предыдущее задание: если ни один экспорт не прислал результат за одну секунду, выведите `Время ожидания истекло`.

Критерии проверки:

- импортирован пакет `time`;
- в `select` есть `case <-time.After(time.Second)`;
- таймаут и результат — альтернативные ветви одного `select`;
- нет `time.Sleep` в `main`.

Подсказка

```
case <-time.After(time.Second):
  fmt.Println("Время ожидания истекло")
```

Решение

```
select {
case message := <-textExport:
  fmt.Println(message)
case message := <-jsonExport:
  fmt.Println(message)
case <-time.After(time.Second):
  fmt.Println("Время ожидания истекло")
}
```

Задание 3. Ожидайте оба экспорта с общим лимитом

Напишите функцию:

```
func waitForExportsUntil(results <-chan exportResult, timeout <-chan time.Time) error
```

Она должна получать успешные или ошибочные результаты до закрытия канала, но вернуть ошибку, если общий таймаут закончился раньше.

Критерии проверки:

- функция получает готовый канал таймаута;
- цикл читает результаты до закрытия канала;
- результат с `Err != nil` приводит к ошибке с названием экспорта;
- истечение таймаута приводит к ошибке;
- успешное закрытие канала после результатов возвращает `nil`.

Подсказка

```
for {
  select {
  case result, open := <-results:
    if !open {
      return nil
    }
    // обработка result
  case <-timeout:
    return fmt.Errorf("экспорты не завершились вовремя")
  }
}
```

Решение

```
func waitForExportsUntil(results <-chan exportResult, timeout <-chan time.Time) error
{
  for {
    select {
    case result, open := <-results:
      if !open {
        return nil
      }

      if result.Err != nil {
        return fmt.Errorf("%s завершился с ошибкой: %w", result.Name, result.Err)
      }

      fmt.Printf("%s готов\n", result.Name)
    case <-timeout:
      return fmt.Errorf("экспорты не завершились вовремя")
    }
  }

}
```

Задание 4. Используйте таймаут в main

Создайте две задачи, получите `results := startExports(tasks, textPath, jsonPath)` из предыдущей главы и вызовите обёртку:

```
waitForExports(results, 3*time.Second)
```

При ошибке выведите `Ошибка экспорта: <ошибка>` и завершите программу. При успехе выведите `Все экспорты готовы`.

Критерии проверки:

- общий лимит равен трём секундам;
- ошибка проверяется через `if err != nil`;
- сообщение об успехе выводится только после успешного ожидания;
- в коде нет `time.Sleep`.

Подсказка

```
if err := waitForExports(results, 3*time.Second); err != nil {
  fmt.Println("Ошибка экспорта:", err)
  return
}

fmt.Println("Все экспорты готовы")
```

Задание 5. Протестируйте таймаут мгновенно

Напишите тест для `waitForExportsUntil`, который проверяет таймаут без `time.Sleep` и без реального `time.After`. Создайте канал `timeout`, закройте его до вызова функции и убедитесь, что получена ошибка.

Затем добавьте успешный сценарий: отправьте один или несколько `exportResult` в буферизированный канал, закройте его и проверьте, что функция вернула `nil`.

Критерии проверки:

- тест не ждёт реальное время;
- закрытый `timeout`-канал используется как мгновенный сигнал таймаута;
- успешный results-канал закрывается после отправки результатов;
- ошибочный результат `exportResult{Err: ...}` возвращает ошибку;
- все сценарии запускаются через `t.Run`;
- `go test ./...` проходит быстро и стабильно.

Подсказка

```
timeout := make(chan time.Time)
close(timeout)

err := waitForExportsUntil(make(chan exportResult), timeout)
if err == nil {
  t.Fatal("ожидалась ошибка таймаута")
}
```

Фрагмент успешного сценария

```
results := make(chan exportResult, 1)
results <- exportResult{Name: "текстовый экспорт"}
close(results)

timeout := make(chan time.Time)
if err := waitForExportsUntil(results, timeout); err != nil {
  t.Fatalf("waitForExportsUntil вернул ошибку: %v", err)
}
```

Проверьте себя

Перед следующей главой убедитесь, что вы можете:

- написать `select` для двух каналов;
- объяснить назначение `default`;
- добавить таймаут через `time.After`;
- отличить общий таймаут от таймаута каждой итерации;
- обработать закрытый канал через второй результат получения;
- протестировать таймаут через готовый канал без `time.Sleep`;
- объяснить, почему таймаут не отменяет работающую goroutine.

Рекомендации по коду: select и таймауты

- Создавайте общий таймер до цикла, если дедлайн относится ко всей операции, а не к каждой итерации.
- Используйте `default` только для осознанной неблокирующей проверки; не создавайте активный бесконечный цикл.
- Проверяйте второй результат получения из канала, если канал может закрыться.
- Отключайте закрытый канал через присваивание `nil`, если продолжаете ждать другие каналы в `select`.

- Помните: `time.After` прекращает ожидание, но не отменяет работу goroutine.
- Для настоящей отмены передавайте `context` в выполняемую операцию.
- Выносите создание таймера в тонкую обёртку, если основную логику ожидания нужно тестировать без реальных задержек.
- В unit-тестах передавайте уже готовый или закрытый timeout-канал вместо `time.After`.

Чек-лист

У таймаута есть ясная область действия, закрытые каналы обработаны, завершение ожидания не оставляет заблокированных отправителей, а тесты не зависят от реального времени.

Mutex и синхронизация

Цель урока

Научиться распознавать гонки данных, защищать общее изменяемое состояние через `sync.Mutex` и `sync.RWMutex`, создать безопасное хранилище задач `TaskStore` и проверить его через `go test -race`.

Каналы подходят для передачи данных между goroutine. Но иногда несколько goroutine должны обращаться к одной структуре данных — например, к map задач. В этом случае доступ нужно синхронизировать мьютексом.

Что такое гонка данных

Гонка данных (data race) возникает, когда несколько goroutine одновременно обращаются к одной переменной, хотя бы одна из них пишет, и доступ не синхронизирован.

```
package main

import (
  "fmt"
  "sync"
)

func main() {
  var wg sync.WaitGroup
  completedExports := 0

  for range 2 {
    wg.Add(1)

    go func() {
      defer wg.Done()
      completedExports++
    }()
  }

  wg.Wait()
  fmt.Println(completedExports)
}
```

Иногда этот пример напечатает `2`, но это не делает код корректным: две goroutine могут одновременно прочитать старое значение и записать одинаковый результат.

Проверяйте конкурентный код детектором гонок:

```
go run -race .
go test -race ./...
```

Детектор не доказывает отсутствие всех проблем, но часто находит реальные ошибки во время выполнения тестов или программы.

`sync.Mutex`

`sync.Mutex` допускает только одну goroutine в критическую секцию одновременно:

```
var mu sync.Mutex

mu.Lock()
// Работа с общими изменяемыми данными.
mu.Unlock()
```

Исправим счётчик:

```
package main

import (
  "fmt"
  "sync"
)

func main() {
  var wg sync.WaitGroup
  var mu sync.Mutex
  completedExports := 0

  for range 2 {
    wg.Add(1)

    go func() {
      defer wg.Done()

      mu.Lock()
      completedExports++
```

```
        mu.Unlock()     }()  } wg.Wait()  fmt.Println(completedExports) // 2}
```

Мьютекс не хранит данные сам. Он защищает данные, с которыми договорились работать только между `Lock` и `Unlock`.

`defer Unlock`

После успешного `Lock` обычно сразу пишут `defer Unlock`:

```
mu.Lock()
defer mu.Unlock()

// Критическая секция.
```

Это защищает от забытого `Unlock`, если функция выйдет раньше через `return`:

```
func updateTitle(mu *sync.Mutex, titles map[int]string, id int, title string) bool {
  mu.Lock()
  defer mu.Unlock()

  if _, exists := titles[id]; !exists {
    return false
  }

  titles[id] = title
  return true
}
```

Не вызывайте `defer mu.Unlock()` до `mu.Lock()`: будет попытка разблокировать незаблокированный мьютекс и panic.

TaskStore: общее хранилище задач

Создадим структуру, которая хранит map задач и защищает её мьютексом:

```
type TaskStore struct {
  mu    sync.RWMutex
  tasks map[int]task.Task
}
```

Мьютекс делают полем той же структуры, что и защищаемые данные. Так легче увидеть правило: поле `tasks` читается и меняется только через методы `TaskStore`.

Конструктор создаёт готовое к работе хранилище:

```
func NewTaskStore() *TaskStore {
  return &TaskStore{
    tasks: make(map[int]task.Task),
  }
}
```

Возвращаем указатель на `TaskStore`: мьютекс нельзя копировать после первого использования, а всем goroutine нужно обращаться к одному хранилищу.

Запись: `Lock`

Метод `Add` меняет map, поэтому использует `Lock`:

```
var ErrTaskAlreadyExists = errors.New("задача с таким ID уже существует")

func (store *TaskStore) Add(currentTask task.Task) error {
  store.mu.Lock()
  defer store.mu.Unlock()

  if _, exists := store.tasks[currentTask.ID]; exists {
    return ErrTaskAlreadyExists
  }

  store.tasks[currentTask.ID] = currentTask
  return nil
}
```

Проверка наличия ключа и добавление выполняются под одним мьютексом. Если разблокировать map между этими действиями, другая goroutine сможет добавить задачу с тем же ID.

Чтение: `RLock`

`sync.RWMutex` расширяет обычный мьютекс:

- `Lock` / `Unlock` — эксклюзивный доступ для записи;
- `RLock` / `RUnlock` — совместный доступ для чтения несколькими goroutine.

Метод `Get` только читает map:

```
func (store *TaskStore) Get(id int) (task.Task, bool) {
  store.mu.RLock()
  defer store.mu.RUnlock()
```

```
  currentTask, exists := store.tasks[id]  return currentTask, exists}
```

Несколько читателей могут работать одновременно. Но если идёт запись через `Lock`, новые читатели и писатели подождут.

Для небольшого проекта можно начать с `sync.Mutex` для всех операций. `RWMutex` полезен, когда чтений заметно больше, чем записей, и только после измерений или понятной причины. Здесь он нужен для знакомства с разделением чтения и записи.

Изменение задачи в TaskStore

Задача хранится в map значением. Поэтому получаем копию, меняем её и сохраняем обратно — всё под одним `Lock`:

```
var ErrTaskNotFound = errors.New("задача не найдена")

func (store *TaskStore) Rename(id int, title string) error {
  store.mu.Lock()
  defer store.mu.Unlock()

  currentTask, exists := store.tasks[id]
  if !exists {
    return ErrTaskNotFound
  }

  if err := currentTask.Rename(title); err != nil {
    return err
  }

  store.tasks[id] = currentTask
  return nil
}
```

Метод `Complete` устроен так же:

```
func (store *TaskStore) Complete(id int) error {
  store.mu.Lock()
  defer store.mu.Unlock()

  currentTask, exists := store.tasks[id]
  if !exists {
    return ErrTaskNotFound
  }
```

```
  currentTask.Complete()   store.tasks[id] = currentTask   return nil}
```

Полный пример

```
package main

import (
  "errors"
  "fmt"
  "sync"

  "example.com/task-manager/internal/task"
)

var (
  ErrTaskNotFound      = errors.New("задача не найдена")
  ErrTaskAlreadyExists = errors.New("задача с таким ID уже существует")
)

type TaskStore struct {
  mu    sync.RWMutex
  tasks map[int]task.Task
}

func NewTaskStore() *TaskStore {
  return &TaskStore{tasks: make(map[int]task.Task)}
}

func (store *TaskStore) Add(currentTask task.Task) error {
  store.mu.Lock()
  defer store.mu.Unlock()

  if _, exists := store.tasks[currentTask.ID]; exists {
    return ErrTaskAlreadyExists
  }

  store.tasks[currentTask.ID] = currentTask
  return nil
}

func (store *TaskStore) Get(id int) (task.Task, bool) {
  store.mu.RLock()
  defer store.mu.RUnlock()
```

```
  currentTask, exists := store.tasks[id]  return currentTask, exists}func (store
*TaskStore) Complete(id int) error {  store.mu.Lock()   defer store.mu.Unlock()
currentTask, exists := store.tasks[id]  if !exists {    return ErrTaskNotFound   }
currentTask.Complete()  store.tasks[id] = currentTask  return nil}func main() {
store := NewTaskStore()  _ = store.Add(task.Task{ID: 1, Title: "Купить молоко",
Priority: 2})  var wg sync.WaitGroup wg.Add(2)  go func() {    defer wg.Done()     _
= store.Complete(1) }()  go func() {    defer wg.Done()     _, _ = store.Get(1) }()
wg.Wait()  if currentTask, exists := store.Get(1); exists {
fmt.Println(currentTask.Summary())  }}
```

Для краткости в `main` ошибки методов игнорируются через `_`. В реальном приложении их нужно обрабатывать; в практических заданиях ниже это будет сделано явно.

Тестирование TaskStore

У `TaskStore` есть два уровня проверки:

1. последовательные unit-тесты контракта `Add`, `Get`, `Rename` и `Complete`;
2. конкурентные тесты, которые запускают операции из нескольких goroutine и проверяются с `-race`.

Последовательные операции

Table-driven tests помогают проверить нормальные и ошибочные сценарии без дублирования setup:

```
func TestTaskStoreAdd(t *testing.T) {
  testCases := []struct {
    name    string
    setup   func(*TaskStore)
    current task.Task
    wantErr error
  }{
    {
      name:    "новая задача",
      current: task.Task{ID: 1, Title: "Купить молоко", Priority: 2},
    },
    {
      name: "повторный ID",
      setup: func(store *TaskStore) {
        _ = store.Add(task.Task{ID: 1, Title: "Первая задача", Priority: 1})
      },
      current: task.Task{ID: 1, Title: "Вторая задача", Priority: 2},
      wantErr: ErrTaskAlreadyExists,
    },
```

```
  } for _, testCase := range testCases {    t.Run(testCase.name, func(t *testing.T) {
store := NewTaskStore()      if testCase.setup != nil {       testCase.setup(store)
}     err := store.Add(testCase.current)      if !errors.Is(err, testCase.wantErr) {
t.Fatalf("Add error = %v, ожидалось %v", err, testCase.wantErr)      }    })  }}
```

Каждый subtest создаёт новый `TaskStore`: сценарии не влияют на состояние друг друга.

Конкурентный сценарий

Тест должен проверять итоговое состояние, а не порядок выполнения goroutine. Например, `Complete` и `Rename` меняют разные поля задачи, поэтому после ожидания обе операции должны быть видны:

```
func TestTaskStoreConcurrentUpdate(t *testing.T) {
  store := NewTaskStore()
  if err := store.Add(task.Task{ID: 1, Title: "Купить молоко", Priority: 2}); err !=
nil {
    t.Fatalf("Add вернул ошибку: %v", err)
  }

  var wg sync.WaitGroup
  wg.Add(2)

  go func() {
    defer wg.Done()
    if err := store.Complete(1); err != nil {
      t.Errorf("Complete вернул ошибку: %v", err)
    }
  }()

  go func() {
    defer wg.Done()
    if err := store.Rename(1, "Купить продукты"); err != nil {
      t.Errorf("Rename вернул ошибку: %v", err)
    }
  }()

  wg.Wait()

  currentTask, exists := store.Get(1)
  if !exists {
    t.Fatal("задача не найдена")
  }

  if !currentTask.Completed {
```

```
    t.Error("задача должна быть выполненной") } if currentTask.Title != "Купить
продукты" {    t.Errorf("Title = %q, ожидалось %q", currentTask.Title, "Купить
продукты") }}
```

Запускайте такой тест с детектором гонок:

```
go test -race ./...
```

Обычный `go test` подтверждает ожидаемый результат этого запуска. `-race` дополнительно следит за одновременным небезопасным доступом к данным во время выполнения теста.

Конкурентное добавление с одинаковым ID

Одновременная попытка добавить один ID — полезная проверка границы `Add`. Итог должен быть предсказуем: один вызов успешен, остальные возвращают `ErrTaskAlreadyExists`.

Не требуйте, чтобы успешной была конкретная goroutine: её порядок не определён. Проверяйте количество успехов, количество ожидаемых ошибок и наличие ровно одной задачи в Store.

WaitGroup и Mutex решают разные задачи

Эти инструменты часто используются вместе, но отвечают на разные вопросы:

Инструмент	Вопрос
`sync.WaitGroup`	«Когда все goroutine закончили?»
`sync.Mutex` / `sync.RWMutex`	«Кто сейчас может обратиться к общим изменяемым данным?»
Канал	«Как передать значение или сигнал между goroutine?»

`WaitGroup` не делает map безопасной, а мьютекс не ждёт завершения goroutine.

Не копируйте Mutex и RWMutex

Не копируйте `TaskStore` после того, как он начал использоваться:

```
store := NewTaskStore()
// copiedStore := *store // нельзя: копируется уже используемый RWMutex
```

Передавайте `*TaskStore` в функции и храните один экземпляр хранилища. Правило аналогично `WaitGroup`: объекты синхронизации используют через указатель.

Не держите мьютекс во время медленной работы

Критическая секция должна быть короткой. Не выполняйте запись файла, HTTP-запрос или долгие вычисления под `Lock`:

```
// Плохо: остальные goroutine не смогут работать с store,
// пока выполняется медленный экспорт.
// store.mu.Lock()
// exportJSON("tasks.json", tasks)
// store.mu.Unlock()
```

Сначала под блокировкой получите нужные данные, затем разблокируйте хранилище и выполняйте медленную работу отдельно. Создание безопасной копии списка задач разберём, когда TaskStore получит метод `List`.

Частые ошибки

Открыть без закрытия

```
store.mu.Lock()

if task.ID <= 0 {
  return errors.New("некорректный ID") // Unlock не вызван
}
```

После `Lock` сразу используйте `defer store.mu.Unlock()`.

Использовать `RLock` для записи

```
store.mu.RLock()
store.tasks[id] = currentTask // ошибка логики: RLock только для чтения
store.mu.RUnlock()
```

Любое изменение map требует `Lock`.

Записать в map из нескольких goroutine без мьютекса

```
// store.tasks[id] = currentTask // опасно без Lock
```

Map не поддерживает конкурентное чтение и запись без синхронизации. Используйте методы `TaskStore` и проверяйте программу с `-race`.

Копировать Mutex

Не передавайте `sync.Mutex`, `sync.RWMutex` и `sync.WaitGroup` по значению после первого использования. Передавайте указатель на структуру, которая их содержит.

Тест без `-race`

Успешный `go test` не гарантирует отсутствие гонок. Конкурентные тесты запускайте также с `go test -race ./...`.

Дальше

Закрепите безопасный `TaskStore` на практике. Затем перейдём к `context`: он позволит отменять долгие операции и передавать дедлайны по цепочке вызовов.

Самопроверка

1. Когда возникает гонка данных?

A. Map содержит больше одного элемента

B. Одна функция вызывает другую

C. Канал имеет буфер

D. Несколько goroutine обращаются к одной переменной, и хотя бы одна пишет без синхронизации

2. Как запустить детектор гонок для программы?

A. go run --mutex .

B. go test --channel .

C. go run -race .

D. go build -lock .

3. Что делает mu.Lock()?

A. Запускает новую goroutine

B. Закрывает канал

C. Ждёт WaitGroup

D. Получает эксклюзивный доступ к критической секции

4. Почему после Lock обычно сразу пишут defer Unlock?

A. Чтобы Lock выполнился дважды

B. Чтобы мьютекс освободился и при раннем return

C. Чтобы автоматически создать map

D. Чтобы запретить чтение из map навсегда

5. Какой метод RWMutex используют для чтения map без изменения?

A. Lock и RUnlock

B. Wait и Done

C. Read и Close

D. RLock и RUnlock

6. Какой метод RWMutex нужен, чтобы изменить map?

A. Lock и Unlock

B. RLock и RUnlock

C. Send и Receive

D. Add и Done

7. Что решает sync.WaitGroup?

A. Передачу значений между goroutine

B. Ожидание завершения goroutine

C. Защиту map от одновременной записи

D. Сериализацию JSON

8. Почему TaskStore обычно передают как *TaskStore?

A. Чтобы не копировать используемый мьютекс и работать с одним хранилищем

B. Потому что все методы Go требуют указатель

C. Чтобы отключить -race

D. Потому что map нельзя хранить в структуре

9. Почему проверка наличия ID и добавление задачи должны быть под одним Lock?

A. Map не поддерживает len

B. Другая goroutine иначе может добавить тот же ID между проверкой и записью

C. errors.Is требует Lock

D. RLock автоматически удаляет элементы

10. Почему не стоит выполнять долгую запись файла под Lock TaskStore?

A. Файл автоматически удалит Mutex

B. Другие goroutine будут слишком долго ждать доступ к хранилищу

C. JSON всегда вызывает panic под Lock

D. os.WriteFile нельзя вызвать из goroutine

11. Почему каждый subtest TaskStore должен создавать новый Store?

A. Store нельзя использовать дважды

B. Чтобы отключить мьютекс

C. Чтобы ошибки стали nil

D. Чтобы сценарии не влияли на состояние друг друга

12. Что нужно проверить при конкурентном Add одного ID?

A. Только итоговый текст fmt.Println

B. Ровно один успех, ожидаемые ошибки дубликата и одну сохранённую задачу

C. Что первой всегда побеждает первая goroutine

D. Что все goroutine успешно добавляют одинаковый ID

Ответы к самопроверке

1. Несколько goroutine обращаются к одной переменной, и хотя бы одна пишет без синхронизации

2. go run -race .

3. Получает эксклюзивный доступ к критической секции

4. Чтобы мьютекс освободился и при раннем return

5. RLock и RUnlock

6. Lock и Unlock

7. Ожидание завершения goroutine

8. Чтобы не копировать используемый мьютекс и работать с одним хранилищем

9. Другая goroutine иначе может добавить тот же ID между проверкой и записью

10. Другие goroutine будут слишком долго ждать доступ к хранилищу

11. Чтобы сценарии не влияли на состояние друг друга

12. Ровно один успех, ожидаемые ошибки дубликата и одну сохранённую задачу

Задания: безопасный TaskStore

Task Manager API теперь хранит задачи в общей map, к которой могут одновременно обращаться несколько goroutine. Обернём map в `TaskStore` и защитим операции `Add`, `Get`, `Rename` и `Complete`.

Задание 1. Создайте TaskStore

Создайте структуру `TaskStore` с полями:

```
mu    sync.RWMutex
tasks map[int]task.Task
```

Добавьте конструктор `NewTaskStore() *TaskStore`, который создаёт пустую map.

Критерии проверки:

- импортирован `sync`;
- `tasks` имеет тип `map[int]task.Task`;
- используется `sync.RWMutex`;
- конструктор возвращает указатель;
- map инициализируется через `make`.

Подсказка

```
func NewTaskStore() *TaskStore {
  return &TaskStore{
    tasks: make(map[int]task.Task),
  }
}
```

Задание 2. Добавьте Add и Get

Добавьте метод `Add(currentTask task.Task) error`, который не допускает повторный ID, и метод `Get(id int) (task.Task, bool)`.

Критерии проверки:

- `Add` использует `Lock` и `defer Unlock`;
- `Get` использует `RLock` и `defer RUnlock`;
- повторный ID возвращает `ErrTaskAlreadyExists`;
- `Get` возвращает структуру и признак наличия;
- map нигде не читается и не меняется без мьютекса.

Подсказка

```
store.mu.Lock()
defer store.mu.Unlock()
```

Для чтения замените `Lock` на `RLock`.

Решение

```
var ErrTaskAlreadyExists = errors.New("задача с таким ID уже существует")

func (store *TaskStore) Add(currentTask task.Task) error {
  store.mu.Lock()
  defer store.mu.Unlock()

  if _, exists := store.tasks[currentTask.ID]; exists {
    return ErrTaskAlreadyExists
  }

  store.tasks[currentTask.ID] = currentTask
  return nil
}

func (store *TaskStore) Get(id int) (task.Task, bool) {
  store.mu.RLock()
  defer store.mu.RUnlock()

  currentTask, exists := store.tasks[id]
  return currentTask, exists
}
```

Задание 3. Добавьте Rename и Complete

Добавьте методы `Rename(id int, title string) error` и `Complete(id int) error`. Если ID не найден, верните `ErrTaskNotFound`. Полученную из map задачу измените и сохраните обратно.

Критерии проверки:

- оба метода используют `Lock`, а не `RLock`;
- ошибка отсутствующей задачи возвращается до изменения;
- `Rename` использует `currentTask.Rename(title)`;
- `Complete` использует `currentTask.Complete()`;
- изменённая задача сохраняется как `store.tasks[id] = currentTask`.

Подсказка

```
currentTask, exists := store.tasks[id]
if !exists {
  return ErrTaskNotFound
}
```

Решение

```
var ErrTaskNotFound = errors.New("задача не найдена")

func (store *TaskStore) Rename(id int, title string) error {
  store.mu.Lock()
  defer store.mu.Unlock()

  currentTask, exists := store.tasks[id]
  if !exists {
    return ErrTaskNotFound
  }

  if err := currentTask.Rename(title); err != nil {
    return err
  }

  store.tasks[id] = currentTask
  return nil
}

func (store *TaskStore) Complete(id int) error {
  store.mu.Lock()
  defer store.mu.Unlock()

  currentTask, exists := store.tasks[id]
  if !exists {
    return ErrTaskNotFound
  }

  currentTask.Complete()
  store.tasks[id] = currentTask
  return nil
}
```

Задание 4. Напишите последовательные тесты TaskStore

Создайте `internal/store/store_test.go` рядом с `TaskStore`. Напишите table-driven tests для:

- успешного добавления задачи;
- повторного ID и `ErrTaskAlreadyExists`;
- `Get` существующей и отсутствующей задачи;
- `Rename` и `Complete` отсутствующей задачи с `ErrTaskNotFound`.

Критерии проверки:

- каждый subtest создаёт новый `TaskStore`;
- известные ошибки проверяются через `errors.Is`;
- сценарии не используют одну общую map или Store;
- успешный Add проверяется последующим Get;
- `go test ./...` проходит успешно.

Подсказка

```
t.Run("повторный ID", func(t *testing.T) {
  store := NewTaskStore()
  _ = store.Add(task.Task{ID: 1, Title: "Первая задача", Priority: 1})

  err := store.Add(task.Task{ID: 1, Title: "Вторая задача", Priority: 2})
  if !errors.Is(err, ErrTaskAlreadyExists) {
    t.Fatalf("Add error = %v", err)
  }
})
```

Задание 5. Проверьте конкурентный доступ

Создайте `TaskStore`, добавьте задачу с ID `1`, затем запустите две goroutine:

- первая вызывает `store.Complete(1)`;
- вторая вызывает `store.Rename(1, "Купить продукты")`.

Дождитесь обеих через `WaitGroup`, получите задачу через `Get` и выведите её. Затем запустите программу с `go run -race .`.

Критерии проверки:

- используется один `TaskStore`;
- `WaitGroup` ждёт обе goroutine;
- ошибки `Rename` и `Complete` не игнорируются: их можно вывести из goroutine;
- итоговая задача имеет `Completed: true` и новое название;
- `go test -race ./...` не сообщает о гонке данных.

Подсказка

Не передавайте `TaskStore` по значению. Обе goroutine должны замкнуть одну переменную `store`, которая содержит указатель.

Возможное решение

```
store := NewTaskStore()

if err := store.Add(task.Task{ID: 1, Title: "Купить молоко", Priority: 2}); err !=
nil {
  fmt.Println(err)
  return
}

var wg sync.WaitGroup
wg.Add(2)

go func() {
  defer wg.Done()
  if err := store.Complete(1); err != nil {
    fmt.Println("Не удалось выполнить задачу:", err)
  }
}()

go func() {
  defer wg.Done()
  if err := store.Rename(1, "Купить продукты"); err != nil {
    fmt.Println("Не удалось переименовать задачу:", err)
  }
}()

wg.Wait()

if currentTask, exists := store.Get(1); exists {
  fmt.Println(currentTask.Summary())
}
```

Задание 6. Одновременный Add

Запустите несколько goroutine, каждая из которых пытается добавить задачу с ID `1`. Дождитесь их через `WaitGroup`. После завершения проверьте:

- в Store есть одна задача с ID `1`;
- ровно один вызов `Add` завершился успешно;
- все остальные ошибки распознаются как `ErrTaskAlreadyExists`;
- тест проходит с `go test -race ./...`.

Критерии проверки:

- используется один `TaskStore`;
- успешный вызов не привязан к конкретной goroutine;
- доступ к счётчикам результатов теста синхронизирован или собирается после worker через канал;
- ошибки проверяются через `errors.Is`;
- нет `time.Sleep`.

Подсказка

Поскольку глава о каналах уже пройдена, удобно отправлять результат каждого `Add` в буферизированный `chan error` и читать его после `wg.Wait()`.

Проверьте себя

Перед следующей главой убедитесь, что вы можете:

- объяснить, что такое гонка данных;
- запустить программу с `go run -race .`;
- защитить запись через `Lock` и `defer Unlock`;
- защитить чтение через `RLock` и `defer RUnlock`;
- объяснить разницу задач `WaitGroup`, мьютекса и канала;
- не копировать структуру с уже используемым мьютексом.
- тестировать последовательный и конкурентный контракт TaskStore;
- запускать хранилище с `go test -race ./...`.

Рекомендации по коду: mutex и синхронизация

- Защищайте одним мьютексом все связанные операции над общим состоянием, а не отдельные строки кода случайно.
- После `Lock` сразу пишите `defer Unlock`; после `RLock` — `defer RUnlock`.
- Используйте `RLock` только для чтения. Любое изменение требует обычного `Lock`.
- Держите критическую секцию короткой: не выполняйте под мьютексом запись файла, сеть или долгие вычисления.
- Не копируйте используемые `Mutex`, `RWMutex` и структуры, которые их содержат.
- Регулярно запускайте concurrent-код с `go test -race ./...` или `go run -race .`.
- В конкурентных тестах проверяйте детерминированный итог и число ожидаемых ошибок, а не порядок goroutine.
- Создавайте новый Store в каждом subtest и не делите изменяемое состояние между сценариями.

Чек-лист

Каждое общее изменяемое поле имеет ясный механизм синхронизации, все пути после блокировки освобождают мьютекс, а последовательные и конкурентные тесты запускаются с race detector.

Context: отмена и дедлайны

Цель урока

Научиться передавать сигнал отмены через `context.Context`, ограничивать операцию по времени, проверять `Done()` и корректно завершать параллельные экспорты Task Manager API.

`context` не выполняет работу за программу и не останавливает goroutine принудительно. Он передаёт сигнал: «работа больше не нужна» или «время истекло». Код, который выполняет работу, должен сам слушать этот сигнал и завершаться.

Когда нужен context

Контекст полезен, когда работа может стать неактуальной:

- пользователь отменил запрос;
- вышестоящая операция завершилась с ошибкой;
- истёк дедлайн HTTP-запроса;
- нужно остановить несколько связанных goroutine.

В Go контекст обычно передают первым параметром функции:

```
func exportTasks(ctx context.Context, tasks []task.Task) error {
  // ...
}
```

Контекст не хранят в структуре и не передают как поле `Task`. Он описывает время жизни **операции**, а не данные задачи.

Базовый контекст

`context.Background()` создаёт пустой корневой контекст:

```
ctx := context.Background()
```

У него нет отмены, дедлайна и значений. Его обычно используют в `main`, тестах или там, где начинается новая цепочка работы.

Отмена: `context.WithCancel`

`context.WithCancel` возвращает дочерний контекст и функцию отмены:

```
ctx, cancel := context.WithCancel(context.Background())
defer cancel()
```

Вызов `cancel()` закрывает канал `ctx.Done()`. Все goroutine, которые слушают этот канал, получают сигнал завершить работу.

```
package main

import (
  "context"
  "fmt"
)

func work(ctx context.Context, done chan<- struct{}) {
  defer close(done)

  select {
  case <-ctx.Done():
    fmt.Println("Работа отменена:", ctx.Err())
  }
}

func main() {
  ctx, cancel := context.WithCancel(context.Background())
  defer cancel()

  done := make(chan struct{})
  go work(ctx, done)
  cancel()
  <-done
}
```

Канал `done` не даёт `main` завершиться раньше worker: после `cancel()` программа ждёт, пока goroutine заметит отмену и напечатает сообщение. `defer cancel()` нужен даже если ожидается естественное завершение: он освобождает ресурсы дочернего контекста, когда текущая функция завершается раньше срока.

Дедлайн: `context.WithTimeout`

`context.WithTimeout` отменяет контекст автоматически по истечении указанного времени:

```
ctx, cancel := context.WithTimeout(context.Background(), 3*time.Second)
defer cancel()
```

Контекст также может быть отменён раньше вручную вызовом `cancel()`.

```
package main

import (
  "context"
  "fmt"
  "time"
)

func waitForWork(ctx context.Context) {
  select {
  case <-time.After(5 * time.Second):
    fmt.Println("Работа завершена")
  case <-ctx.Done():
    fmt.Println("Работа остановлена:", ctx.Err())
  }
}

func main() {
  ctx, cancel := context.WithTimeout(context.Background(), time.Second)
  defer cancel()

  waitForWork(ctx)
}
```

В этом примере сработает ветвь `ctx.Done()`, потому что дедлайн в одну секунду наступает раньше условной работы в пять секунд.

`Done` и `Err`

У контекста есть два главных метода:

- `Done() <-chan struct{}` — канал, который закрывается при отмене или истечении дедлайна;
- `Err() error` — причина завершения.

Возможные причины:

```
context.Canceled         // вызвали cancel()
context.DeadlineExceeded // истёк timeout или deadline
```

Обычный паттерн внутри долгой операции:

```
func processTasks(ctx context.Context, tasks []task.Task) error {
  for _, currentTask := range tasks {
    select {
    case <-ctx.Done():
      return ctx.Err()
    default:
    }

    fmt.Println("Обработка:", currentTask.Title)
  }

  return nil
}
```

`default` здесь не создаёт активный цикл: он только быстро проверяет отмену на каждой итерации существующего конечного цикла.

Контекст и блокирующая операция

Если goroutine ждёт канал, добавьте `ctx.Done()` в тот же `select`:

```
select {
case result := <-results:
  return result.Err
case <-ctx.Done():
  return ctx.Err()
}
```

Так ожидание результата прекратится, если операция была отменена.

Важно: передача контекста не делает любую функцию отменяемой автоматически. Например, `os.WriteFile` не принимает `context.Context`; отмена не может гарантированно прервать уже начавшийся системный вызов записи. Но код может проверить контекст **до** записи и между отдельными шагами работы.

Экспорты с отменой

Сделаем экспорты кооперативно отменяемыми. Каждый экспорт проверяет контекст перед началом и перед записью файла:

```
func exportText(ctx context.Context, path string, tasks []task.Task) error {
  if err := ctx.Err(); err != nil {
    return err
  }

  var builder strings.Builder
  for _, currentTask := range tasks {
    select {
    case <-ctx.Done():
      return ctx.Err()
    default:
    }

    builder.WriteString(currentTask.Summary())
    builder.WriteString("\n")
  }

  if err := ctx.Err(); err != nil {
    return err
  }

  return os.WriteFile(path, []byte(builder.String()), 0o644)
}
```

Для этой версии экспорта добавьте импорт `strings`.

JSON-экспорт устроен аналогично:

```
func exportJSON(ctx context.Context, path string, tasks []task.Task) error {
  if err := ctx.Err(); err != nil {
    return err
  }

  data, err := json.MarshalIndent(tasks, "", "  ")
  if err != nil {
    return fmt.Errorf("не удалось сериализовать задачи: %w", err)
  }

  if err := ctx.Err(); err != nil {
```

```
  return os.WriteFile(path, data, 0o644)}
```

Канал результатов с закрытием

В главе о каналах `saveExports` возвращала буферизированный канал с двумя известными результатами. Теперь функция принимает контекст и закрывает канал, когда оба отправителя завершились:

```
type exportResult struct {
  Name string
  Err  error
}

func startExports(
  ctx context.Context,
  tasks []task.Task,
  textPath string,
  jsonPath string,
) <-chan exportResult {
  results := make(chan exportResult, 2)
  var wg sync.WaitGroup
  wg.Add(2)

  go func() {
    defer wg.Done()
    result := exportResult{
      Name: "текстовый экспорт",
      Err:  exportText(ctx, textPath, tasks),
    }

    select {
    case results <- result:
    case <-ctx.Done():
    }
  }()

  go func() {
    defer wg.Done()
    result := exportResult{
      Name: "JSON-экспорт",
      Err:  exportJSON(ctx, jsonPath, tasks),
    }

    select {
```

```
    case results <- result:    case <-ctx.Done():    } }()  go func() {    wg.Wait()
close(results)  }()  return results}
```

Каждый экспорт работает в собственной goroutine. Отдельная координирующая goroutine ждёт обе через `WaitGroup` и затем безопасно закрывает `results`: закрывать канал можно только после завершения всех отправителей. `select` при отправке не даёт exporter-goroutine зависнуть, если контекст уже отменён и получатель прекратил ожидание.

Ожидание результатов с context

Теперь вместо отдельного `time.After` принимаем контекст с дедлайном:

```
func waitForExports(ctx context.Context, results <-chan exportResult) error {
  for {
    select {
    case result, open := <-results:
      if !open {
        return nil
      }

      if result.Err != nil {
        return fmt.Errorf("%s завершился с ошибкой: %w", result.Name, result.Err)
      }

      fmt.Printf("%s готов\n", result.Name)
    case <-ctx.Done():
      return ctx.Err()
    }
  }
}
```

В `main` создаём общий дедлайн для экспортов:

```
ctx, cancel := context.WithTimeout(context.Background(), 3*time.Second)
defer cancel()

results := startExports(ctx, tasks, "tasks.txt", "tasks.json")

if err := waitForExports(ctx, results); err != nil {
  if errors.Is(err, context.DeadlineExceeded) {
    fmt.Println("Экспорты не завершились вовремя")
    return
  }
```

```
  fmt.Println("Ошибка экспорта:", err)}
```

Контекст связывает ожидание и саму работу: после истечения дедлайна `waitForExports` прекращает ожидание, а экспорты замечают отмену при следующей проверке.

Тестирование отмены без `time.Sleep`

Ручную отмену можно тестировать детерминированно: worker сообщает о завершении через канал, а тест сам вызывает `cancel`. Не нужно ждать произвольное время.

```
func waitForCancel(ctx context.Context, done chan<- error) {
  defer close(done)

  <-ctx.Done()
  done <- ctx.Err()
}

func TestWaitForCancel(t *testing.T) {
  ctx, cancel := context.WithCancel(context.Background())
  defer cancel()

  done := make(chan error, 1)
  go waitForCancel(ctx, done)

  cancel()

  err, open := <-done
  if !open {
    t.Fatal("worker завершился без результата")
  }
  if !errors.Is(err, context.Canceled) {
    t.Fatalf("ошибка = %v, ожидалась context.Canceled", err)
  }

  if _, open := <-done; open {
    t.Fatal("канал done должен быть закрыт")
  }
}
```

Буфер `done` размером один позволяет worker отправить ошибку и завершиться, даже если планировщик ещё не переключился на тест. Закрытие `done` — явный сигнал, что worker больше не будет отправлять результаты.

Для теста отменяемого экспорта заранее отмените контекст **до** вызова функции:

```
ctx, cancel := context.WithCancel(context.Background())
cancel()

err := exportText(ctx, path, tasks)
if !errors.Is(err, context.Canceled) {
  t.Fatalf("exportText error = %v, ожидалась context.Canceled", err)
}
```

Так проверяется важный контракт: отменённая операция не начинает запись. Путь создавайте через `filepath.Join(t.TempDir(), "tasks.txt")` и после вызова убедитесь, что файла нет.

Не используйте context для данных

Контекст не предназначен для передачи названия задачи, пользователя, `TaskStore` или необязательных параметров функции:

```
// Неудачно:
// ctx = context.WithValue(ctx, "task", currentTask)
```

Для обычных данных используйте параметры. `WithValue` предназначен для данных, связанных с жизненным циклом запроса, и будет рассмотрен только при появлении HTTP-сервера.

Частые ошибки

Не вызвать cancel

```
ctx, _ := context.WithTimeout(context.Background(), time.Second)
```

Сохраните функцию отмены и вызовите её через `defer`:

```
ctx, cancel := context.WithTimeout(context.Background(), time.Second)
defer cancel()
```

Не слушать `ctx.Done()`

Передать `ctx` в функцию недостаточно:

```
func work(ctx context.Context) {
  for {
    // Работа никогда не проверяет ctx.Done().
```

```
  }}
```

Проверяйте `ctx.Err()` или добавляйте `<-ctx.Done()` в `select` в точках, где операция может закончиться.

Создать новый Background вместо передачи родителя

```
func exportText(ctx context.Context, tasks []task.Task) error {
  ctx = context.Background() // теряет отмену и дедлайн вызывающего кода
  // ...
}
```

Создавайте дочерний контекст от полученного `ctx`, только если это действительно нужно. Обычно достаточно передать исходный дальше.

Использовать timeout как единственную защиту от утечки

Таймаут завершает ожидание, но не исправляет goroutine, которая игнорирует `ctx.Done()` или заблокирована на небуферизированной отправке. Проектируйте отправку и завершение goroutine отдельно.

Дальше

Закрепите отменяемый экспорт задач. Далее перейдём к HTTP-клиенту и серверу: Task Manager API начнёт принимать и возвращать JSON по сети.

Самопроверка

1. Для чего обычно используют context.Context?

A. Для хранения всех задач приложения

B. Для замены любого параметра функции

C. Для передачи отмены и дедлайна по цепочке операций

D. Для сериализации JSON

2. Где по соглашению передают context.Context в сигнатуре функции?

A. Первым параметром

B. Внутри структуры Task

C. Последним параметром

D. Только в возвращаемом значении

3. Что возвращает context.WithCancel?

A. WaitGroup и Mutex

B. Канал строк

C. Только error

D. Дочерний context и функцию cancel

4. Почему после WithTimeout обычно пишут defer cancel()?

A. Чтобы превратить context в канал

B. Чтобы освободить ресурсы дочернего контекста при выходе из функции

C. Чтобы таймаут никогда не сработал

D. Чтобы автоматически остановить все goroutine без проверок

5. Что происходит с ctx.Done() после отмены контекста?

A. Канал закрывается

B. Контекст удаляет все файлы

C. В него всегда отправляется строка

D. Он становится map

6. Что вернёт ctx.Err() после истечения WithTimeout?

A. context.DeadlineExceeded

B. os.ErrNotExist

C. nil

D. context.Canceled всегда

7. Останавливает ли вызов cancel() goroutine принудительно?

A. Да, но только goroutine с WaitGroup

B. Только если goroutine пишет JSON

C. Нет, goroutine должна сама проверить ctx.Done() или ctx.Err()

D. Да, runtime всегда немедленно завершает её

8. Как вернуть ошибку отмены из функции после получения сигнала Done?

A. return <-ctx.Done()

B. return ctx.Err()

C. return nil

D. return context.Background()

9. Почему os.WriteFile не становится отменяемым сам после передачи context в функцию?

A. Контекст работает только с map

B. JSON отключает отмену

C. Файлы нельзя использовать с goroutine

D. os.WriteFile не принимает context и не может автоматически слушать его отмену

10. Что не стоит передавать через context?

A. Сигнал отмены

B. Обычные данные задачи, которые должны быть параметрами функции

C. Причину истечения таймаута через ctx.Err()

D. Дедлайн операции

11. Как в тесте детерминированно проверить ручную отмену worker?

A. Использовать time.Sleep с большим запасом

B. Запустить worker, вручную вызвать cancel и дождаться результата через канал

C. Вызвать panic в worker

D. Проверить только fmt.Println

12. Что должно произойти при вызове exportText с уже отменённым context?

A. Функция возвращает context.Canceled до записи файла

B. Файл обязательно создаётся, затем возвращается nil

C. Всегда возникает panic

D. Контекст автоматически перезапускается

Ответы к самопроверке

1. Для передачи отмены и дедлайна по цепочке операций

2. Первым параметром

3. Дочерний context и функцию cancel

4. Чтобы освободить ресурсы дочернего контекста при выходе из функции

5. Канал закрывается

6. context.DeadlineExceeded

7. Нет, goroutine должна сама проверить ctx.Done() или ctx.Err()

8. return ctx.Err()

9. os.WriteFile не принимает context и не может автоматически слушать его отмену

10. Обычные данные задачи, которые должны быть параметрами функции

11. Запустить worker, вручную вызвать cancel и дождаться результата через канал

12. Функция возвращает context.Canceled до записи файла

Задания: отменяемый экспорт

Теперь Task Manager API умеет прекращать ожидание и подготовку экспортов, если работа больше не нужна или истёк общий дедлайн.

Задание 1. Обработайте ручную отмену

Напишите функцию:

```
func waitForCancel(ctx context.Context) error
```

Она должна ждать `ctx.Done()` и вернуть `ctx.Err()`. В `main` создайте контекст через `context.WithCancel`, запустите `waitForCancel` в goroutine, вызовите `cancel()` и выведите полученную ошибку.

Критерии проверки:

- `context.Context` — первый параметр функции;
- функция ждёт `<-ctx.Done()`;
- функция возвращает `ctx.Err()`;
- `cancel()` вызывается явно;
- итоговая ошибка равна `context.Canceled`.

Подсказка

```
func waitForCancel(ctx context.Context) error {
  <-ctx.Done()
  return ctx.Err()
}
```

Решение

```
package main

import (
  "context"
  "fmt"
)

func waitForCancel(ctx context.Context) error {
  <-ctx.Done()
  return ctx.Err()
}

func main() {
  ctx, cancel := context.WithCancel(context.Background())
  defer cancel()

  errors := make(chan error, 1)
  go func() {
    errors <- waitForCancel(ctx)
  }()

  cancel()
  fmt.Println(<-errors)
}
```

Задание 2. Добавьте дедлайн

Напишите функцию `waitForWork(ctx context.Context) error`. Внутри неё через `select` ждите либо условную работу в течение двух секунд, либо отмену контекста. В `main` создайте контекст с таймаутом в одну секунду.

Критерии проверки:

- используется `context.WithTimeout`;
- после создания контекста есть `defer cancel()`;
- `select` содержит `time.After(2 * time.Second)` и `ctx.Done()`;
- функция возвращает `context.DeadlineExceeded`;
- в `main` проверяется причина через `errors.Is`.

Подсказка

```
case <-ctx.Done():
  return ctx.Err()
```

Решение

```
func waitForWork(ctx context.Context) error {
  select {
  case <-time.After(2 * time.Second):
    return nil
  case <-ctx.Done():
    return ctx.Err()
  }
}

func main() {
  ctx, cancel := context.WithTimeout(context.Background(), time.Second)
  defer cancel()

  err := waitForWork(ctx)
  if errors.Is(err, context.DeadlineExceeded) {
    fmt.Println("Время ожидания истекло")
  }
}
```

Задание 3. Сделайте текстовый экспорт отменяемым

Измените функцию текстового экспорта:

```
func exportText(ctx context.Context, path string, tasks []task.Task) error
```

Проверьте `ctx.Done()` на каждой итерации формирования текста и перед записью файла. При отмене верните `ctx.Err()`.

Критерии проверки:

- `ctx` — первый параметр;
- цикл по задачам проверяет отмену через `select`;
- перед `os.WriteFile` проверяется `ctx.Err()`;
- отменённый контекст не приводит к записи нового файла;
- успешный контекст сохраняет экспорт.

Подсказка

```
select {
case <-ctx.Done():
  return ctx.Err()
default:
}
```

Решение

```
func exportText(ctx context.Context, path string, tasks []task.Task) error {
  if err := ctx.Err(); err != nil {
    return err
  }

  var content string
  for _, currentTask := range tasks {
    select {
    case <-ctx.Done():
      return ctx.Err()
    default:
    }

    content += currentTask.Summary() + "\n"
  }

  if err := ctx.Err(); err != nil {
    return err
  }

  return os.WriteFile(path, []byte(content), 0o644)
}
```

Задание 4. Добавьте context к экспортам

Измените `startExports` и `waitForExports` так, чтобы они принимали `context.Context`. В `main` создайте общий `WithTimeout` на три секунды, запустите экспорты и обработайте `context.DeadlineExceeded` отдельным сообщением.

Критерии проверки:

- один контекст передаётся в `startExports`, `exportText`, `exportJSON` и `waitForExports`;
- `startExports` закрывает канал результатов после отправки обоих результатов;
- `waitForExports` слушает и `results`, и `ctx.Done()`;
- при дедлайне `main` сообщает, что экспорты не завершились вовремя;
- в каждом `WithTimeout` есть `defer cancel()`.

Подсказка

```
ctx, cancel := context.WithTimeout(context.Background(), 3*time.Second)
defer cancel()

results := startExports(ctx, tasks, textPath, jsonPath)
err := waitForExports(ctx, results)
```

Возможное решение

```
ctx, cancel := context.WithTimeout(context.Background(), 3*time.Second)
defer cancel()

results := startExports(ctx, tasks, textPath, jsonPath)

if err := waitForExports(ctx, results); err != nil {
  if errors.Is(err, context.DeadlineExceeded) {
    fmt.Println("Экспорты не завершились вовремя")
    return
  }

  fmt.Println("Ошибка экспорта:", err)
  return
}

fmt.Println("Все экспорты готовы")
```

Задание 5. Протестируйте worker с отменой

Напишите тест, в котором worker получает context, ждёт `ctx.Done()`, отправляет `ctx.Err()` в буферизированный канал и закрывает канал после результата. Тест должен вызвать `cancel` вручную и проверить `context.Canceled` без `time.Sleep`.

Критерии проверки:

- worker получает context параметром;
- worker не использует фиксированную задержку;
- тест вызывает `cancel()` после запуска worker;
- результат проверяется через `errors.Is`;
- тест проверяет закрытие канала после результата;
- `go test ./...` и `go test -race ./...` проходят успешно.

Подсказка

Буферизированный канал размера `1` позволит worker отправить ошибку, даже если тест ещё не начал чтение:

```
done := make(chan error, 1)
```

Решение

```
func waitForCancel(ctx context.Context, done chan<- error) {
  defer close(done)

  <-ctx.Done()
  done <- ctx.Err()
}

func TestWaitForCancel(t *testing.T) {
  ctx, cancel := context.WithCancel(context.Background())
  defer cancel()

  done := make(chan error, 1)
  go waitForCancel(ctx, done)

  cancel()

  err, open := <-done
  if !open {
    t.Fatal("worker завершился без результата")
  }
  if !errors.Is(err, context.Canceled) {
    t.Fatalf("ошибка = %v, ожидалась context.Canceled", err)
  }

  if _, open := <-done; open {
    t.Fatal("канал done должен быть закрыт")
  }
}
```

Задание 6. Отмените экспорт до записи

Создайте отменённый context до вызова `exportText`. Путь к файлу постройте через `filepath.Join(t.TempDir(), "tasks.txt")`. Проверьте, что функция возвращает `context.Canceled`, а файл не создаётся.

Критерии проверки:

- используется `t.TempDir`;
- context отменён до вызова экспорта;
- ошибка проверяется через `errors.Is`;
- проверяется отсутствие файла через `os.Stat` и `os.IsNotExist`;
- тест не использует `time.Sleep`.

Подсказка

```
ctx, cancel := context.WithCancel(context.Background())
cancel()

err := exportText(ctx, path, tasks)
```

Проверьте себя

Перед следующей главой убедитесь, что вы можете:

- создать `Background`, `WithCancel` и `WithTimeout`;
- вызвать `cancel` через `defer`;
- слушать `ctx.Done()` внутри `select`;
- вернуть `ctx.Err()` из отменяемой функции;
- отличить `context.Canceled` от `context.DeadlineExceeded`;
- передать общий context по цепочке экспортов.
- протестировать отмену worker без `time.Sleep`;
- проверить, что отменённый экспорт не создаёт файл.

Рекомендации по коду: context

- Передавайте `context.Context` первым параметром функции, которая выполняет отменяемую работу.
- Создавайте дочерний context от полученного родителя, а не от нового `Background` внутри операции.
- Всегда сохраняйте и вызывайте `cancel`, обычно через `defer cancel()`.
- Проверяйте `ctx.Done()` в блокирующих `select` и `ctx.Err()` между шагами долгой работы.
- Возвращайте `ctx.Err()` как причину отмены или истечения дедлайна.
- Не храните context в структуре и не используйте его вместо обычных параметров.
- Тестируйте ручную отмену явным вызовом `cancel`, а не реальными задержками.
- Пусть worker отправляет явный результат и закрывает принадлежащий ему канал, чтобы тест мог дождаться завершения.

Чек-лист

Отмена проходит по всей цепочке вызовов, goroutine умеют завершаться по `Done`, дедлайн не маскирует утечку работы, которая не слушает context, а тесты отмены не зависят от времени.

Логирование и отладка

Цель урока

Научиться писать структурированные логи через `log/slog`, выбирать уровень сообщения, добавлять полезный контекст к операциям Task Manager API и применять основные инструменты диагностики Go.

Лог — это сообщение для разработчика или оператора приложения. Он отвечает на вопросы: что произошло, когда, с какими данными и почему операция не выполнилась. Лог не заменяет возврат `error`: ошибка сообщает о проблеме вызывающему коду, а лог помогает наблюдать работу приложения.

Пакет `log/slog`

`log/slog` — стандартный пакет структурированного логирования Go. Вместо свободного текста он записывает сообщение вместе с полями:

```
logger.Info("задача добавлена", "task_id", 1, "priority", 2)
```

Текстовый вывод может выглядеть так:

```
time=2026-10-13T12:00:00.000Z level=INFO msg="задача добавлена" task_id=1 priority=2
```

Отдельные поля позволяют искать и фильтровать логи по `task_id`, уровню или другому атрибуту. Не собирайте такую информацию вручную в длинную строку:

```
// Менее удобно для поиска и обработки:
// logger.Info(fmt.Sprintf("задача %d добавлена с приоритетом %d", id, priority))
```

Уровни логов

Основные уровни `slog`:

Уровень	Когда использовать
`Debug`	Детали, полезные при локальной диагностике
`Info`	Нормальные значимые события приложения
`Warn`	Необычная ситуация, с которой приложение продолжает работу
`Error`	Операция не выполнилась и требует внимания

Примеры:

```
logger.Debug("начало сохранения задач", "count", len(tasks))
logger.Info("задача выполнена", "task_id", id)
logger.Warn("дедлайн задачи истёк", "task_id", id)
logger.Error("не удалось сохранить задачи", "err", err)
```

Не записывайте успешное событие на уровне `Error` и не превращайте каждое действие в `Info`: слишком шумные логи перестают быть полезными.

Создание logger

`slog.Default()` доступен без настройки:

```
slog.Info("приложение запущено")
```

Для явной настройки создайте logger и handler. Text handler удобен локально:

```
logger := slog.New(
  slog.NewTextHandler(os.Stderr, &slog.HandlerOptions{
    Level: slog.LevelDebug,
  }),
)
```

JSON handler подходит для production-среды, где логи собирает платформа или внешний сервис:

```
logger := slog.New(
  slog.NewJSONHandler(os.Stdout, &slog.HandlerOptions{
    Level: slog.LevelInfo,
  }),
)
```

Вывод JSON-лога:

```
{"time":"2026-10-13T12:00:00Z","level":"INFO","msg":"задача добавлена","task_id":1}
```

Пишем логи в `stderr`, а основной полезный вывод CLI оставляем в `stdout`. Это помогает отделить данные программы от диагностики.

Поля и группы

Ключи полей должны быть стабильными и понятными:

```
logger.Info(
  "задача переименована",
  "task_id", currentTask.ID,
  "old_title", oldTitle,
  "new_title", currentTask.Title,
)
```

Связанный контекст можно один раз добавить к дочернему logger:

```
taskLogger := logger.With("task_id", currentTask.ID)

taskLogger.Info("задача выполнена")
taskLogger.Info("экспорт задачи подготовлен")
```

Не используйте в логах чувствительные данные: пароли, токены доступа, содержимое авторизационных заголовков, персональные данные без необходимости.

Logger в TaskStore

Передавайте logger как зависимость, а не создавайте его внутри каждого метода. Это даёт приложению контроль над форматом и уровнем логов, а тестам — возможность отключить вывод.

```
type TaskStore struct {
  mu     sync.RWMutex
  tasks  map[int]task.Task
  logger *slog.Logger
}

func NewTaskStore(logger *slog.Logger) *TaskStore {
  if logger == nil {
    logger = slog.Default()
  }

  return &TaskStore{
    tasks:  make(map[int]task.Task),
    logger: logger,
  }
}
```

Метод `Add` логирует успех и ошибку дубликата:

```
func (store *TaskStore) Add(currentTask task.Task) error {
  store.mu.Lock()
  defer store.mu.Unlock()

  if _, exists := store.tasks[currentTask.ID]; exists {
    store.logger.Warn("задача уже существует", "task_id", currentTask.ID)
    return ErrTaskAlreadyExists
  }

  store.tasks[currentTask.ID] = currentTask
  store.logger.Info(
    "задача добавлена",
    "task_id", currentTask.ID,
    "priority", currentTask.Priority,
  )
  return nil
}
```

Лог находится рядом с событием, но метод по-прежнему возвращает `error`: вызывающий код сам решает, как завершить сценарий.

Логирование ошибок экспорта

Для измерения длительности операции используйте `time.Since`:

```
func saveTasks(logger *slog.Logger, path string, tasks []task.Task) error {
  startedAt := time.Now()

  if err := storage.SaveTasks(path, tasks); err != nil {
    logger.Error(
      "не удалось сохранить JSON-экспорт",
      "path", path,
      "task_count", len(tasks),
      "duration", time.Since(startedAt),
      "err", err,
    )
    return err
  }

  logger.Info(
    "JSON-экспорт сохранён",
    "path", path,
    "task_count", len(tasks),
```

```
    "duration", time.Since(startedAt),  ) return nil}
```

Логируйте операционный контекст: путь, число задач, длительность. Не нужно одновременно выводить ту же ошибку в каждом нижнем уровне приложения: это создаёт дублирующиеся записи. Обычно ошибка логируется на границе, где её окончательно обрабатывают — например, в `main`, HTTP handler или фоновом worker.

Тестирование кода с logger

Тесты не должны засорять вывод. Передайте logger с handler, который пишет в `io.Discard`:

```
func newTestLogger() *slog.Logger {
  return slog.New(slog.NewTextHandler(io.Discard, nil))
}

func TestTaskStoreAdd(t *testing.T) {
  store := NewTaskStore(newTestLogger())

  if err := store.Add(task.Task{ID: 1, Title: "Купить молоко", Priority: 2}); err !=
nil {
    t.Fatalf("Add вернул ошибку: %v", err)
  }
}
```

В unit-тесте обычно проверяйте поведение Store, а не текст лога. Логи стоит проверять отдельно только если они являются частью явного контракта системы, например для аудита или интеграции с мониторингом.

`go vet`

`go vet` анализирует распространённые подозрительные конструкции, которые компилятор может не считать ошибкой:

```
go vet ./...
```

Запускайте его вместе с тестами:

```
go vet ./...
go test ./...
go test -race ./...
```

`go vet` не заменяет тесты и code review, но должен стать обычной частью локальной проверки и будущего CI.

Адресный запуск тестов

Во время отладки не обязательно запускать весь набор:

```
go test ./internal/task -run '^TestTaskRename$'
go test ./internal/store -run 'TestTaskStoreConcurrentUpdate'
```

Флаг `-v` показывает имена тестов и subtests:

```
go test -v ./internal/task
```

Для повторного запуска теста без кэша используйте `-count=1`:

```
go test -count=1 ./...
```

Это особенно полезно для отладки конкурентного или файлового кода.

Stack trace и panic

Если panic не перехвачена, Go выводит stack trace: последовательность вызовов, приведших к проблеме.

```
panic: runtime error: index out of range

goroutine 1 [running]:
example.com/task-manager/internal/task.(*TaskStore).Get(...)
    /project/internal/task/store.go:42
main.main()
    /project/main.go:18
```

Начните чтение с первой строки вашего проекта: она обычно показывает место возникновения проблемы. Затем двигайтесь по стеку к вызывающему коду. Не скрывайте panic через `recover`, пока не поняли причину.

Отладчик IDE

Расширение Go для VS Code и GoLand поддерживают Delve — стандартный отладчик Go. Минимальный workflow:

1. поставьте breakpoint слева от строки кода;
2. запустите Debug для пакета или теста;
3. посмотрите переменные и стек вызовов;

4. используйте Step Over и Step Into;
5. после понимания проблемы исправьте код и закрепите её тестом.

Отладчик полезен для сложных ветвей и состояния, но не заменяет логирование и автоматические тесты.

Первое знакомство с `pprof`

Профилирование нужно, когда измерения указывают на проблему производительности. Не добавляйте `pprof` «на всякий случай» и не оптимизируйте без данных.

Для CPU-профиля тестов:

```
go test -cpuprofile=cpu.out ./internal/store
go tool pprof cpu.out
```

Для memory-профиля:

```
go test -memprofile=mem.out ./internal/store
go tool pprof mem.out
```

В интерактивном `pprof` начните с команды `top`, чтобы увидеть функции с наибольшими затратами. Полноценные benchmarks и profiling будут расширены в продвинутом блоке.

Частые ошибки

Логировать ошибку и возвращать её на каждом уровне

```
func saveTasks(...) error {
  if err := storage.SaveTasks(...); err != nil {
    logger.Error("ошибка", "err", err)
    return err
  }
}
```

Если `main` или HTTP handler тоже залогирует эту ошибку, получится несколько одинаковых записей. Выберите границу окончательной обработки и логируйте ошибку там.

Логировать только текст

```
logger.Info("задача 42 добавлена")
```

Лучше добавить поле:

```
logger.Info("задача добавлена", "task_id", 42)
```

Проверять только логи вместо результата

Лог подтверждает наблюдаемость, но не заменяет проверку контракта метода. Тест `TaskStore.Add` должен проверять, что задача добавлена или вернулась ожидаемая ошибка.

Профилировать без гипотезы

Сначала зафиксируйте проблему: медленный запрос, большая аллокация или перегруженный CPU. Затем снимайте профиль, анализируйте его и измеряйте результат изменения.

Дальше

Закрепите `slog` и диагностический workflow. Далее Task Manager API станет HTTP-сервером, который будет принимать и возвращать JSON.

Самопроверка

1. Какой пакет стандартной библиотеки предоставляет структурированное логирование?

A. log/slog

B. fmt/log

C. sync/logger

D. os/debug

2. Как лучше записать ID задачи в структурированный лог?

A. logger.Info("задача добавлена", "task_id", task.ID)

B. fmt.Println(task.ID)

C. logger.Info(fmt.Sprintf("задача %d добавлена", task.ID))

D. panic(task.ID)

3. Какой уровень подходит для нормального значимого события приложения?

A. Warn всегда

B. Info

C. Error

D. Fatal

4. Какой уровень подходит для неуспешной операции сохранения?

A. Error

B. Debug

C. Info

D. Trace

5. Для чего используют logger.With("task_id", id)?

A. Чтобы закрыть logger

B. Чтобы добавить общий контекст task_id ко всем логам дочернего logger

C. Чтобы создать goroutine

D. Чтобы изменить ID задачи в Store

6. Как отключить вывод logger в unit-тесте?

A. Заменить logger на nil без проверки

B. Удалить все вызовы logger из кода

C. Вызвать panic перед логом

D. Создать handler, который пишет в io.Discard

7. Какую команду используют для статического анализа подозрительных конструкций Go?

A. go profile ./...

B. go vet ./...

C. go lint встроенно

D. go debug ./...

8. Как запустить только TestTaskRename в пакете internal/task?

A. go run TestTaskRename

B. go vet TestTaskRename

C. go test ./internal/task -run "^TestTaskRename$"

D. go test -only TaskRename

9. С чего начать чтение stack trace после panic?

A. С удаления всех defer

B. С перезапуска компьютера

C. С последней строки стандартной библиотеки

D. С первой строки, относящейся к коду проекта

10. Когда стоит начинать profiling через pprof?

A. В каждом новом проекте до написания кода

B. Когда измерения показывают реальную проблему производительности

C. Вместо unit-тестов

D. Только если приложение не компилируется

Ответы к самопроверке

1. log/slog

2. logger.Info("задача добавлена", "task_id", task.ID)

3. Info

4. Error

5. Чтобы добавить общий контекст task_id ко всем логам дочернего logger

6. Создать handler, который пишет в io.Discard

7. go vet ./...

8. go test ./internal/task -run "^TestTaskRename$"

9. С первой строки, относящейся к коду проекта

10. Когда измерения показывают реальную проблему производительности

Задания: наблюдаемый TaskStore

Добавим `slog.Logger` в TaskStore и научимся запускать адресные проверки кода. Логирование не заменяет ошибки: Store продолжает возвращать `error`, а logger добавляет контекст для диагностики.

Задание 1. Создайте logger

В `main` создайте text logger уровня `Debug`, который пишет в `os.Stderr`. Запишите события старта приложения и успешного завершения.

Критерии проверки:

- используется `slog.NewTextHandler`;
- output — `os.Stderr`;
- уровень задаётся через `slog.HandlerOptions`;
- есть минимум один `Info` и один `Debug` лог;
- основной вывод программы не смешивается с логами в `stdout`.

Подсказка

```
logger := slog.New(slog.NewTextHandler(os.Stderr, &slog.HandlerOptions{
  Level: slog.LevelDebug,
}))
```

Задание 2. Внедрите logger в TaskStore

Добавьте поле `logger *slog.Logger` в `TaskStore`. Измените конструктор на:

```
func NewTaskStore(logger *slog.Logger) *TaskStore
```

Если передан `nil`, используйте `slog.Default()`. В `Add` запишите `Info` для успешного добавления и `Warn` для повторного ID.

Критерии проверки:

- logger передаётся через конструктор;
- Store не создаёт новый logger в методах;
- в успешном логе есть `task_id` и `priority`;
- повторный ID логируется как `Warn`;
- метод по-прежнему возвращает `ErrTaskAlreadyExists`.

Подсказка

```
if logger == nil {
  logger = slog.Default()
}
```

Фрагмент решения

```
func (store *TaskStore) Add(currentTask task.Task) error {
  store.mu.Lock()
  defer store.mu.Unlock()

  if _, exists := store.tasks[currentTask.ID]; exists {
    store.logger.Warn("задача уже существует", "task_id", currentTask.ID)
    return ErrTaskAlreadyExists
  }

  store.tasks[currentTask.ID] = currentTask
  store.logger.Info("задача добавлена", "task_id", currentTask.ID, "priority",
currentTask.Priority)
  return nil
}
```

Задание 3. Сделайте logger тихим в тестах

Напишите helper `newTestLogger() *slog.Logger`, который создаёт logger с `io.Discard`. Используйте его во всех unit-тестах `TaskStore`.

Критерии проверки:

- импортирован `io`;
- handler получает `io.Discard`;
- тесты продолжают проверять состояние Store и ошибки;
- проверка текста логов не требуется;
- `go test ./...` не засоряется логами Store.

Решение

```
func newTestLogger() *slog.Logger {
  return slog.New(slog.NewTextHandler(io.Discard, nil))
}
```

Задание 4. Залогируйте экспорт

Напишите обёртку `saveTasksWithLog`, которая измеряет длительность `storage.SaveTasks`, логирует успешное сохранение на уровне `Info` и ошибку на уровне `Error`.

Критерии проверки:

- время начала сохраняется через `time.Now()`;
- длительность вычисляется через `time.Since`;
- успешный лог содержит `path`, `task_count` и `duration`;
- ошибочный лог содержит те же поля и `err`;
- функция возвращает исходную ошибку вызывающему коду.

Подсказка

```
startedAt := time.Now()
err := storage.SaveTasks(path, tasks)
duration := time.Since(startedAt)
```

Задание 5. Примените диагностический workflow

Выполните команды из корня Task Manager API:

```
go vet ./...
go test ./...
go test -race ./...
go test -v ./internal/store
go test -count=1 ./...
```

Затем запустите один тест Store через `-run`. Если доступен VS Code или GoLand, поставьте breakpoint внутри `TaskStore.Add`, запустите тест в Debug и посмотрите `currentTask` и `store.tasks`.

Критерии проверки:

- `go vet` выполняется без замечаний;
- обычные и race-тесты проходят;
- запуск одного теста использует `-run`;
- вы можете объяснить разницу между логом, тестом и breakpoint.

Проверьте себя

Перед следующей главой убедитесь, что вы можете:

- выбрать корректный уровень лога;
- добавить структурированные поля к событию;
- передать logger как зависимость;
- отключить вывод logger в unit-тестах;
- запустить `go vet`, отдельный тест и race detector;
- начать анализ stack trace и CPU-профиля с конкретной гипотезой.

Рекомендации по коду: логирование и отладка

- Логируйте значимые события и ошибки на понятной границе обработки, а не на каждом уровне вызовов.
- Предпочитайте структурированные поля стабильным ключам: `task_id`, `path`, `duration`, `err`.
- Не записывайте в логи секреты, токены, пароли и лишние персональные данные.
- Передавайте `*slog.Logger` как зависимость и используйте `slog.Default()` как безопасный fallback.
- В unit-тестах направляйте логи в `io.Discard`, если проверка логов не является контрактом теста.
- Регулярно запускайте `go vet ./...`, `go test ./...` и `go test -race ./...`.

- Профилируйте только после измерения реальной проблемы и проверяйте эффект оптимизации повторным измерением.

Чек-лист

Логи содержат достаточный контекст без чувствительных данных, ошибки не дублируются на всех уровнях, а диагностические инструменты входят в обычный workflow разработки.

HTTP-сервер и JSON API

Цель урока

Научиться создать первую HTTP-версию Task Manager API средствами стандартного пакета `net/http`: принять запрос, выбрать маршрут, проверить входной JSON, вызвать бизнес-логику, вернуть JSON-ответ и протестировать handler без запуска настоящего сервера.

В этой главе HTTP-слой намеренно строится без фреймворка. Так будет видно, из каких частей состоит API и что именно позже упростит Gin. Бизнес-логика задач не должна зависеть от конкретного способа доставки запроса: сегодня её вызывает HTTP handler, завтра — CLI или другой transport.

> **Требуется Go 1.22 или новее**
>
> Новые шаблоны маршрутов `ServeMux`, например `GET /tasks/{id}`, и `r.PathValue("id")` появились в Go 1.22. Проверьте установленную версию:
>
> ```
> go version
> ```

Что происходит при HTTP-запросе

Клиент отправляет request, а сервер формирует response:

```
клиент → HTTP request → ServeMux → handler → service → HTTP response → клиент
```

Request содержит метод, путь, заголовки и иногда тело. Response содержит статус, заголовки и тело.

```
POST /tasks HTTP/1.1
Content-Type: application/json

{"title":"Купить молоко","priority":2}
```

Успешный ответ на создание задачи:

```
HTTP/1.1 201 Created
Content-Type: application/json
```

```
{"id":1,"title":"Купить молоко","priority":2,"completed":false}
```

`net/http` и handler

HTTP handler — это код, который получает `http.ResponseWriter` и `*http.Request`:

```
func health(w http.ResponseWriter, r *http.Request) {
  w.Header().Set("Content-Type", "application/json")
  w.WriteHeader(http.StatusOK)
  w.Write([]byte(`{"status":"ok"}`))
}
```

- `r` содержит данные запроса: метод, URL, заголовки, body и `Context()`;
- `w` формирует ответ: заголовки, статус и body.

Метод `w.Write` неявно отправляет статус `200 OK`, если статус ещё не был записан. Обычно API удобнее создавать через `json.Encoder`, но порядок остаётся важным: сначала заголовки, затем `WriteHeader`, затем body.

Запуск минимального сервера

```
package main

import (
  "log"
  "net/http"
)

func health(w http.ResponseWriter, r *http.Request) {
  w.Header().Set("Content-Type", "application/json")
  w.WriteHeader(http.StatusOK)
  w.Write([]byte(`{"status":"ok"}`))
}

func main() {
  mux := http.NewServeMux()
  mux.HandleFunc("GET /healthz", health)

  log.Println("сервер слушает http://localhost:8080")
  if err := http.ListenAndServe(":8080", mux); err != nil {
    log.Fatal(err)
```

```
  }}
```

Запустите сервер:

```
go run .
```

В другом терминале выполните:

```
curl -i http://localhost:8080/healthz
```

`http.NewServeMux` сопоставляет запрос с маршрутом. Шаблон `GET /healthz` задаёт **и метод, и путь**. Если зарегистрирован только этот шаблон, `POST /healthz` не попадёт в handler.

Для учебного сервера подойдёт `http.ListenAndServe`. В главе о конфигурации и жизненном цикле заменим его на `http.Server`, добавим таймауты и graceful shutdown.

Маршруты Task Manager API

Сделаем небольшой контракт первой версии API:

Метод и путь	Назначение	Успешный статус
`GET /healthz`	Проверить, что приложение отвечает	`200 OK`
`GET /tasks`	Получить список задач	`200 OK`
`POST /tasks`	Создать задачу	`201 Created`
`POST /tasks/{id}/complete`	Отметить задачу выполненной	`200 OK`

Регистрация маршрутов остаётся в одном месте:

```
func (api *API) Router() http.Handler {
  mux := http.NewServeMux()
  mux.HandleFunc("GET /healthz", api.health)
  mux.HandleFunc("GET /tasks", api.listTasks)
  mux.HandleFunc("POST /tasks", api.createTask)
  mux.HandleFunc("POST /tasks/{id}/complete", api.completeTask)
  return mux
}
```

`{id}` — path-параметр. Извлечь его можно через `r.PathValue`:

```
id, err := strconv.Atoi(r.PathValue("id"))
if err != nil || id < 1 {
  writeError(w, http.StatusBadRequest, "идентификатор задачи должен быть
```

```
положительным числом")   return}
```

Не доверяйте параметрам URL: строка `abc`, `0` и `-1` не должны попадать в сервис как корректный ID.

Слои приложения

HTTP handler отвечает только за HTTP: чтение запроса, валидацию формы данных, выбор статуса и кодирование ответа. Правила предметной области и работа с хранилищем остаются ниже:

```
HTTP handler → service → repository / TaskStore
```

Для связи handler с приложением опишем небольшой интерфейс. Он зависит от потребностей handler, а не от конкретной реализации Store:

```
type TaskService interface {
  List(ctx context.Context) ([]task.Task, error)
  Create(ctx context.Context, input task.CreateInput) (task.Task, error)
  Complete(ctx context.Context, id int) (task.Task, error)
}

type API struct {
  service TaskService
  logger  *slog.Logger
}

func New(service TaskService, logger *slog.Logger) *API {
  if logger == nil {
    logger = slog.Default()
  }

  return &API{service: service, logger: logger}
}
```

Здесь `task.CreateInput` — отдельный тип входных данных для бизнес-логики:

```
type CreateInput struct {
  Title    string
  Priority int
}
```

Так HTTP-пакет не зависит от map, mutex или будущего PostgreSQL-репозитория. В тестах вместо настоящего сервиса можно передать маленький fake.

В текущей учебной реализации `TaskStore.Complete` уже возвращает `store.ErrTaskNotFound`. Когда сервис передаёт эту известную причину выше, HTTP-слой может преобразовать её в `404`. При появлении PostgreSQL service сохранит тот же внешний контракт и сам сопоставит ошибку репозитория с `store.ErrTaskNotFound` или отдельной доменной ошибкой.

JSON-ответы

Повторяющийся код JSON-ответа вынесем в helper:

```
func writeJSON(w http.ResponseWriter, status int, value any) {
  w.Header().Set("Content-Type", "application/json; charset=utf-8")
  w.WriteHeader(status)

  if err := json.NewEncoder(w).Encode(value); err != nil {
    // После WriteHeader нельзя безопасно изменить уже отправленный ответ.
    return
  }
}

type errorResponse struct {
  Error string `json:"error"`
}

func writeError(w http.ResponseWriter, status int, message string) {
  writeJSON(w, status, errorResponse{Error: message})
}
```

`json.Encoder.Encode` добавляет перевод строки в конец JSON — это нормально. `Content-Type` нужно установить **до** `WriteHeader`: после отправки заголовков изменить их уже нельзя.

Список задач возвращается очень коротко:

```
func (api *API) listTasks(w http.ResponseWriter, r *http.Request) {
  tasks, err := api.service.List(r.Context())
  if err != nil {
    api.logger.Error("не удалось получить задачи", "err", err)
    writeError(w, http.StatusInternalServerError, "внутренняя ошибка сервера")
    return
  }

  writeJSON(w, http.StatusOK, tasks)
}
```

`r.Context()` связан с жизненным циклом запроса. Передавайте его дальше в service, репозиторий и внешние вызовы, чтобы отмена клиента или будущий серверный дедлайн могли остановить ненужную работу.

Чтение и проверка JSON-запроса

Для создания задачи определим HTTP-представление тела:

```
type createTaskRequest struct {
  Title    string `json:"title"`
  Priority int    `json:"priority"`
}
```

Это не обязательно та же структура, что `task.Task`. Клиент не должен передавать `id` и `completed`: ID выдаёт приложение, а выполнение меняется отдельной операцией.

Создадим helper, который ограничивает размер body, запрещает неизвестные поля и принимает ровно одно JSON-значение:

```
func decodeJSON(w http.ResponseWriter, r *http.Request, dst any) bool {
  r.Body = http.MaxBytesReader(w, r.Body, 1<<20) // не больше 1 MiB

  decoder := json.NewDecoder(r.Body)
  decoder.DisallowUnknownFields()

  if err := decoder.Decode(dst); err != nil {
    writeError(w, http.StatusBadRequest, "некорректный JSON-запрос")
    return false
  }

  if err := decoder.Decode(&struct{}{}); err != io.EOF {
    writeError(w, http.StatusBadRequest, "в body должен быть один JSON-объект")
    return false
  }

  return true
}
```

`DisallowUnknownFields` защитит от опечатки клиента: поле `priorty` не будет молча проигнорировано. Проверка второго `Decode` не позволит передать два JSON-объекта подряд. Ограничение размера body — базовая защита от слишком больших запросов.

Валидация и создание задачи

Handler проверяет, что запрос имеет корректную форму, а сервис обязан сохранять бизнес-инварианты и для других входов, не только HTTP.

```
func (api *API) createTask(w http.ResponseWriter, r *http.Request) {
  var request createTaskRequest
  if !decodeJSON(w, r, &request) {
    return
  }

  request.Title = strings.TrimSpace(request.Title)
  if request.Title == "" {
    writeError(w, http.StatusBadRequest, "название задачи обязательно")
    return
  }
  if request.Priority < 1 || request.Priority > 5 {
    writeError(w, http.StatusBadRequest, "приоритет должен быть от 1 до 5")
    return
  }

  createdTask, err := api.service.Create(r.Context(), task.CreateInput{
    Title:    request.Title,
    Priority: request.Priority,
  })
  if err != nil {
    api.logger.Error("не удалось создать задачу", "err", err)
    writeError(w, http.StatusInternalServerError, "внутренняя ошибка сервера")
    return
  }

  api.logger.Info("задача создана", "task_id", createdTask.ID)
  writeJSON(w, http.StatusCreated, createdTask)
}
```

Для недопустимых входных данных возвращаем `400 Bad Request`, а не `500`: ошибка на стороне клиента. При успешном создании используем `201 Created`.

Изменение существующей задачи и `404`

Превратим известную ошибку сервиса «задача не найдена» в HTTP-статус `404 Not Found`:

```
func (api *API) completeTask(w http.ResponseWriter, r *http.Request) {
  id, err := strconv.Atoi(r.PathValue("id"))
  if err != nil || id < 1 {
```

```
    writeError(w, http.StatusBadRequest, "идентификатор задачи должен быть
положительным числом")    return } completedTask, err :=
api.service.Complete(r.Context(), id) if errors.Is(err, store.ErrTaskNotFound) {
writeError(w, http.StatusNotFound, "задача не найдена")    return } if err != nil {
api.logger.Error("не удалось завершить задачу", "task_id", id, "err", err)
writeError(w, http.StatusInternalServerError, "внутренняя ошибка сервера")    return }
writeJSON(w, http.StatusOK, completedTask)}
```

Сравнивайте причины через `errors.Is`, а не текст ошибки. Наружу не отдавайте технические детали: они полезны в структурированном логе, но не клиенту API.

Сборка сервера

`main` соединяет зависимости приложения:

```
logger := slog.New(slog.NewTextHandler(os.Stderr, nil))
store := store.NewTaskStore(logger)
service := taskservice.New(store)
api := httpapi.New(service, logger)

server := &http.Server{
  Addr:              ":8080",
  Handler:           api.Router(),
  ReadHeaderTimeout: 5 * time.Second,
}

logger.Info("HTTP-сервер запущен", "address", server.Addr)
if err := server.ListenAndServe(); err != nil && !errors.Is(err,
http.ErrServerClosed) {
  logger.Error("HTTP-сервер завершился с ошибкой", "err", err)
}
```

`http.Server` даёт место для настройки таймаутов. Сейчас сервер завершается обычным способом; корректную остановку через `signal.NotifyContext` и `server.Shutdown` добавим в главе о жизненном цикле.

Проверка API через `curl`

После запуска сервера выполните:

```
curl -i http://localhost:8080/healthz

curl -i http://localhost:8080/tasks
```

```
curl -i -X POST http://localhost:8080/tasks \  -H 'Content-Type: application/json' \
-d '{"title":"Купить молоко","priority":2}'curl -i -X POST
http://localhost:8080/tasks/1/complete
```

Проверьте также ошибочные случаи: пустое название, приоритет `0`, некорректный JSON и несуществующий ID. API должен вернуть понятный JSON с корректным статусом, а не panic и не HTML-страницу.

Тестирование handler без настоящего сервера

`httptest.NewRequest` создаёт тестовый request, а `httptest.NewRecorder` запоминает response. Сеть и порт не нужны:

```
func TestCreateTask(t *testing.T) {
  service := &fakeTaskService{
    createdTask: task.Task{ID: 1, Title: "Купить молоко", Priority: 2},
  }
  api := New(service, newTestLogger())

  request := httptest.NewRequest(
    http.MethodPost,
    "/tasks",
    strings.NewReader(`{"title":"Купить молоко","priority":2}`),
  )
  recorder := httptest.NewRecorder()

  api.Router().ServeHTTP(recorder, request)

  if recorder.Code != http.StatusCreated {
    t.Fatalf("status = %d, ожидался %d", recorder.Code, http.StatusCreated)
  }

  var got task.Task
  if err := json.NewDecoder(recorder.Body).Decode(&got); err != nil {
    t.Fatalf("не удалось прочитать JSON-ответ: %v", err)
  }
  if got != service.createdTask {
    t.Fatalf("ответ = %+v, ожидалось %+v", got, service.createdTask)
  }
  if service.createInput.Title != "Купить молоко" || service.createInput.Priority !=
2 {
    t.Fatalf("service получил %+v", service.createInput)
  }
}
```

Минимальный fake реализует интерфейс и сохраняет входные данные. Он не должен повторять логику настоящего сервиса:

```
type fakeTaskService struct {
  createdTask task.Task
  createInput task.CreateInput
  createErr   error
}

func (service *fakeTaskService) Create(_ context.Context, input task.CreateInput)
(task.Task, error) {
  service.createInput = input
  return service.createdTask, service.createErr
}

func (service *fakeTaskService) List(context.Context) ([]task.Task, error) {
  return nil, nil
}

func (service *fakeTaskService) Complete(context.Context, int) (task.Task, error) {
  return task.Task{}, nil
}
```

Тест handler проверяет HTTP-контракт: статус, JSON и то, какие данные переданы в service. Проверка реальной бизнес-логики остаётся в её unit-тестах.

Частые ошибки

Записывать body до статуса

```
w.Write([]byte(`{"error":"не найдено"}`))
w.WriteHeader(http.StatusNotFound) // Уже поздно: клиент получил 200.
```

Устанавливайте заголовки и статус до записи body.

Смешивать HTTP и бизнес-логику

```
func (api *API) createTask(w http.ResponseWriter, r *http.Request) {
  // Не храните задачи и не управляйте mutex прямо здесь.
}
```

Handler переводит HTTP в вызов service. Это позволит заменить `net/http` на Gin в следующей главе без переписывания логики задач.

Игнорировать ошибку JSON-декодирования

Не продолжайте работу с нулевыми значениями структуры, если JSON не прочитан. Верните `400` и завершите handler.

Возвращать клиенту техническую ошибку

```
writeError(w, http.StatusInternalServerError, err.Error())
```

Сообщение может раскрыть пути файлов, детали базы данных или внутренние правила. Логируйте исходную ошибку, а клиенту отдавайте стабильное безопасное сообщение.

Дальше

У вас есть первая рабочая HTTP-версия Task Manager API на стандартном `net/http`. В следующей главе реализуем тот же HTTP-слой на Gin, сравним объём кода и сохраним service и repository без изменений.

Самопроверка

1. За что отвечает http.ResponseWriter?

A. За хранение задач в map

B. За формирование HTTP-ответа: заголовков, статуса и body

C. За чтение переменных окружения

D. За запуск goroutine

2. В каком порядке нужно формировать JSON-ответ?

A. Сначала body, потом статус и заголовки

B. Порядок не имеет значения

C. Сначала закрыть соединение, затем body

D. Установить заголовки, записать статус, затем записать body

3. Как в Go 1.22 получить значение {id} из маршрута POST /tasks/{id}/complete?

A. w.PathValue("id")

B. json.Unmarshal(r.Body, &id)

C. r.PathValue("id")

D. r.Header.Get("id")

4. Какой статус нужно вернуть после успешного создания задачи?

A. 500 Internal Server Error

B. 201 Created

C. 200 OK всегда

D. 400 Bad Request

5. Что делает decoder.DisallowUnknownFields()?

A. Запрещает все строковые поля

B. Удаляет из ответа пустые поля

C. Шифрует JSON-body

D. Отклоняет JSON с полями, которых нет в структуре запроса

6. Зачем handler передаёт r.Context() в service?

A. Чтобы получить путь из URL

B. Чтобы сериализовать response в JSON

C. Чтобы отмена запроса и дедлайн могли распространяться дальше

D. Чтобы создать новый mutex

7. Кто должен хранить правила предметной области задачи?

A. Только http.ResponseWriter

B. Service или модель предметной области, а не HTTP handler

C. JSON-теги структуры

D. Только curl

8. Как преобразовать известную ошибку «задача не найдена» в API-ответ?

A. Вызвать panic

B. Проверить errors.Is и вернуть 404 Not Found

C. Игнорировать ошибку и вернуть пустую задачу

D. Вернуть 500 и err.Error()

9. Какие инструменты позволяют проверить handler без запуска сервера на порту?

A. os.Open

B. sync.WaitGroup

C. httptest.NewRequest и httptest.NewRecorder

D. time.Sleep и fmt.Println

10. Почему клиенту не стоит возвращать err.Error() для внутренней ошибки?

A. HTTP не поддерживает строки

B. errors.Is перестанет работать

C. Ошибка может раскрыть внутренние технические детали

D. JSON не умеет передавать ошибки

Ответы к самопроверке

1. За формирование HTTP-ответа: заголовков, статуса и body

2. Установить заголовки, записать статус, затем записать body

3. r.PathValue("id")

4. 201 Created

5. Отклоняет JSON с полями, которых нет в структуре запроса

6. Чтобы отмена запроса и дедлайн могли распространяться дальше

7. Service или модель предметной области, а не HTTP handler

8. Проверить errors.Is и вернуть 404 Not Found

9. httptest.NewRequest и httptest.NewRecorder

10. Ошибка может раскрыть внутренние технические детали

Задания: первый Task Manager API

Соберите HTTP-версию Task Manager API на стандартном `net/http`. Держите handler тонким: он читает HTTP, вызывает service и формирует HTTP-ответ, но не управляет хранилищем напрямую.

Задание 1. Добавьте health-check

Создайте `GET /healthz`. Он должен вернуть статус `200 OK` и JSON:

```
{"status":"ok"}
```

Критерии проверки:

- используется `http.NewServeMux`;
- маршрут зарегистрирован как `GET /healthz`;
- `Content-Type` содержит `application/json`;
- статус равен `http.StatusOK`;
- ответ формируется через общий `writeJSON`.

Подсказка

```
func (api *API) health(w http.ResponseWriter, r *http.Request) {
  writeJSON(w, http.StatusOK, map[string]string{"status": "ok"})
}
```

Решение

```
func (api *API) Router() http.Handler {
  mux := http.NewServeMux()
  mux.HandleFunc("GET /healthz", api.health)
  return mux
}

func (api *API) health(w http.ResponseWriter, r *http.Request) {
  writeJSON(w, http.StatusOK, map[string]string{"status": "ok"})
}
```

Задание 2. Реализуйте `GET /tasks`

Добавьте метод `List(ctx)` в интерфейс `TaskService` и маршрут `GET /tasks`. При успехе верните слайс задач и `200`, при ошибке service — безопасный JSON с `500`.

Критерии проверки:

- handler передаёт в service именно `r.Context()`;
- успешный ответ имеет `200 OK`;
- service-ошибка логируется с ключом `err`;
- клиент получает сообщение без технического текста исходной ошибки;
- HTTP handler не читает `map` Store напрямую.

Подсказка

```
tasks, err := api.service.List(r.Context())
if err != nil {
  api.logger.Error("не удалось получить задачи", "err", err)
  writeError(w, http.StatusInternalServerError, "внутренняя ошибка сервера")
  return
}
```

Решение

```
func (api *API) listTasks(w http.ResponseWriter, r *http.Request) {
  tasks, err := api.service.List(r.Context())
  if err != nil {
    api.logger.Error("не удалось получить задачи", "err", err)
    writeError(w, http.StatusInternalServerError, "внутренняя ошибка сервера")
    return
  }

  writeJSON(w, http.StatusOK, tasks)
}
```

Задание 3. Создайте задачу через JSON

Реализуйте `POST /tasks`. Примите `title` и `priority`, запретите неизвестные JSON-поля, удалите пробелы вокруг названия и проверьте приоритет от 1 до 5. После успеха вызовите `TaskService.Create` и верните созданную задачу с `201`.

Критерии проверки:

- размер body ограничен через `http.MaxBytesReader`;
- decoder использует `DisallowUnknownFields`;
- некорректный JSON, пустое название и неверный приоритет дают `400`;
- `id` и `completed` не принимаются из body;
- созданная задача возвращается с `201 Created`;
- service получает `task.CreateInput`, а не `http.Request`.

Подсказка

```
request.Title = strings.TrimSpace(request.Title)
if request.Title == "" {
  writeError(w, http.StatusBadRequest, "название задачи обязательно")
  return
}
```

Решение

```
func (api *API) createTask(w http.ResponseWriter, r *http.Request) {
  var request createTaskRequest
  if !decodeJSON(w, r, &request) {
    return
  }

  request.Title = strings.TrimSpace(request.Title)
  if request.Title == "" {
    writeError(w, http.StatusBadRequest, "название задачи обязательно")
    return
  }
  if request.Priority < 1 || request.Priority > 5 {
    writeError(w, http.StatusBadRequest, "приоритет должен быть от 1 до 5")
    return
  }

  createdTask, err := api.service.Create(r.Context(), task.CreateInput{
    Title: request.Title, Priority: request.Priority,
  })
  if err != nil {
    api.logger.Error("не удалось создать задачу", "err", err)
    writeError(w, http.StatusInternalServerError, "внутренняя ошибка сервера")
    return
  }

  writeJSON(w, http.StatusCreated, createdTask)
}
```

Задание 4. Завершите задачу по ID

Создайте маршрут `POST /tasks/{id}/complete`. Преобразуйте `id` через `strconv.Atoi`, отклоните неположительные значения, вызовите `TaskService.Complete` и преобразуйте `store.ErrTaskNotFound` в `404`.

Критерии проверки:

- ID извлекается через `r.PathValue("id")`;
- некорректный и неположительный ID дают `400`;
- используется `errors.Is`, а не сравнение текста ошибки;
- отсутствующая задача даёт JSON-ответ с `404`;
- успешный ответ содержит обновлённую задачу и `200`.

Подсказка

```
if errors.Is(err, store.ErrTaskNotFound) {
  writeError(w, http.StatusNotFound, "задача не найдена")
  return
}
```

Решение

```
func (api *API) completeTask(w http.ResponseWriter, r *http.Request) {
  id, err := strconv.Atoi(r.PathValue("id"))
  if err != nil || id < 1 {
    writeError(w, http.StatusBadRequest, "идентификатор задачи должен быть
положительным числом")
    return
  }

  completedTask, err := api.service.Complete(r.Context(), id)
  if errors.Is(err, store.ErrTaskNotFound) {
    writeError(w, http.StatusNotFound, "задача не найдена")
    return
  }
  if err != nil {
    api.logger.Error("не удалось завершить задачу", "task_id", id, "err", err)
    writeError(w, http.StatusInternalServerError, "внутренняя ошибка сервера")
    return
  }

  writeJSON(w, http.StatusOK, completedTask)
}
```

Задание 5. Протестируйте `POST /tasks` без сети

Создайте fake service, который сохраняет полученный `task.CreateInput`. Через `httptest.NewRequest` и `httptest.NewRecorder` проверьте успешный запрос и хотя бы один невалидный JSON-запрос.

Критерии проверки:

- тест не запускает `ListenAndServe` и не использует реальный порт;
- успешный сценарий проверяет статус `201`, JSON-ответ и входные данные service;
- невалидный JSON возвращает `400`;
- fake реализует интерфейс `TaskService`;
- logger в тесте пишет в `io.Discard`;
- отсутствуют `time.Sleep` и зависимость от порядка тестов.

Подсказка

```
request := httptest.NewRequest(http.MethodPost, "/tasks",
strings.NewReader(`{"title":"Купить молоко","priority":2}`))
recorder := httptest.NewRecorder()

api.Router().ServeHTTP(recorder, request)
```

Решение

```
type fakeTaskService struct {
  createdTask task.Task
  createInput task.CreateInput
}

func (service *fakeTaskService) Create(_ context.Context, input task.CreateInput)
(task.Task, error) {
  service.createInput = input
  return service.createdTask, nil
}

func (service *fakeTaskService) List(context.Context) ([]task.Task, error) {
  return nil, nil
}

func (service *fakeTaskService) Complete(context.Context, int) (task.Task, error) {
  return task.Task{}, nil
}

func newTestLogger() *slog.Logger {
  return slog.New(slog.NewTextHandler(io.Discard, nil))
}

func TestCreateTask(t *testing.T) {
  service := &fakeTaskService{
    createdTask: task.Task{ID: 1, Title: "Купить молоко", Priority: 2},
  }
  api := New(service, newTestLogger())
  request := httptest.NewRequest(
    http.MethodPost,
    "/tasks",
    strings.NewReader(`{"title":"Купить молоко","priority":2}`),
  )
```

▼ Сведения

```
  recorder := httptest.NewRecorder()  api.Router().ServeHTTP(recorder, request)  if
recorder.Code != http.StatusCreated {   t.Fatalf("status = %d, ожидался %d",
recorder.Code, http.StatusCreated)  } if contentType :=
recorder.Header().Get("Content-Type"); !strings.Contains(contentType,
"application/json") {   t.Fatalf("Content-Type = %q, ожидался JSON", contentType) }
var got task.Task if err := json.NewDecoder(recorder.Body).Decode(&got); err != nil {
t.Fatalf("не удалось прочитать JSON-ответ: %v", err)  } if got != service.createdTask
{   t.Fatalf("ответ = %+v, ожидалось %+v", got, service.createdTask)   } if
service.createInput != (task.CreateInput{Title: "Купить молоко", Priority: 2}) {
t.Fatalf("service получил %+v", service.createInput)  }}func
TestCreateTaskInvalidJSON(t *testing.T) { api := New(&fakeTaskService{},
newTestLogger())  request := httptest.NewRequest(http.MethodPost, "/tasks",
strings.NewReader(`{"title":`))  recorder := httptest.NewRecorder()
api.Router().ServeHTTP(recorder, request)  if recorder.Code != http.StatusBadRequest {
t.Fatalf("status = %d, ожидался %d", recorder.Code, http.StatusBadRequest)  }}
```

Проверьте себя

Перед переходом к Gin убедитесь, что вы можете:

- зарегистрировать маршрут в `ServeMux` с методом и path-параметром;
- вернуть JSON и корректный HTTP-статус;
- прочитать и проверить JSON-body без неявного игнорирования полей;
- отделить HTTP handler от service и хранилища;
- передать `r.Context()` в операцию;
- проверить HTTP-контракт через `httptest` без настоящей сети.

Рекомендации по коду: HTTP API на `net/http`

- Регистрируйте маршруты с конкретными HTTP-методами и держите их в одном месте.
- Сначала устанавливайте response headers, затем статус через `WriteHeader`, и только потом записывайте body.
- Отдавайте JSON через общий helper с `Content-Type: application/json; charset=utf-8`.
- Проверяйте path-параметры, размер body, синтаксис JSON и входные значения до вызова service.
- Для входных JSON используйте отдельные request-типы; не позволяйте клиенту задавать служебные поля модели, например ID.
- Передавайте `r.Context()` в service, repository и внешние операции.
- Не смешивайте маршрутизацию, HTTP-статусы, mutex и бизнес-правила в одном handler.
- Логируйте внутреннюю ошибку с контекстом, а клиенту возвращайте безопасное стабильное сообщение.
- Тестируйте handlers через `httptest`; проверяйте статус, заголовки, JSON и вызов service без запуска сервера на порту.

Чек-лист

Каждый маршрут имеет понятный HTTP-контракт, невалидный вход возвращает `400`, отсутствующий ресурс — `404`, неожиданные сбои — безопасный `500`, а HTTP-слой можно заменить на Gin без переписывания service и repository.

HTTP API на Gin

Цель урока

Перенести HTTP-слой Task Manager API со стандартного `net/http` на Gin, не изменяя service и repository. Научиться регистрировать маршруты, извлекать параметры, читать JSON, писать middleware для request ID и структурированного access log, восстанавливаться после panic и тестировать Gin handlers через `httptest`.

Gin — популярный HTTP-фреймворк Go. Он работает поверх стандартных типов `net/http`: сервер всё так же получает HTTP-запрос и отправляет HTTP-ответ. Gin добавляет более компактную маршрутизацию, `gin.Context`, binding JSON и удобный механизм middleware.

В предыдущей главе API работал через `http.ServeMux`. Теперь заменим **только transport-слой**:

```
было:  net/http handler → service → repository / TaskStore
стало: Gin handler      → service → repository / TaskStore
```

`TaskService`, правила создания задач, ошибки Store и способ хранения задач не должны меняться. Это главный результат разделения слоёв.

Установка Gin

В корне Task Manager API добавьте зависимость:

```
go get github.com/gin-gonic/gin
```

Go добавит Gin и его версии зависимостей в `go.mod` и `go.sum`. Не редактируйте эти файлы вручную. Проверить сборку можно так:

```
go test ./...
```

Первый router

Вместо `http.NewServeMux` создайте `gin.Engine`:

```
func (api *API) Router() *gin.Engine {
  router := gin.New()

  router.GET("/healthz", api.health)
  router.GET("/tasks", api.listTasks)
```

```
  router.POST("/tasks", api.createTask) router.POST("/tasks/:id/complete",
api.completeTask) return router}
```

`gin.Default()` автоматически добавляет встроенные middleware `gin.Logger()` и `gin.Recovery()`. В этой главе используем `gin.New()`, чтобы явно подключить middleware и писать access log через уже знакомый `log/slog`.

Сравнение маршрута:

`net/http`	Gin
`mux.HandleFunc("GET /tasks", api.listTasks)`	`router.GET("/tasks", api.listTasks)`
`mux.HandleFunc("POST /tasks/{id}/complete", api.completeTask)`	`router.POST("/tasks/:id/complete", api.completeTask)`
`r.PathValue("id")`	`c.Param("id")`

В Gin path-параметр записывают как `:id`, а не `{id}`.

`gin.Context` и `context.Context`

Handler Gin получает `*gin.Context`:

```
func (api *API) listTasks(c *gin.Context) {
  // ...
}
```

`*gin.Context` — объект Gin для текущего HTTP-обмена. Через него получают параметры, query, body и формируют ответ. Это **не** тот же тип, что стандартный `context.Context`.

Для service всё ещё нужен стандартный context из исходного HTTP-запроса:

```
tasks, err := api.service.List(c.Request.Context())
```

Не передавайте `*gin.Context` в service, repository или `database/sql`. Это привяжет бизнес-логику к Gin и усложнит тестирование или будущую замену фреймворка.

JSON-ответы

В `net/http` мы устанавливали заголовок, статус и вызывали `json.NewEncoder`. В Gin эту работу делает `c.JSON`:

```
func (api *API) health(c *gin.Context) {
  c.JSON(http.StatusOK, gin.H{"status": "ok"})
}
```

```
type errorResponse struct {  Error string `json:"error"`}func writeError(c
*gin.Context, status int, message string) {  c.JSON(status, errorResponse{Error:
message})}
```

Gin устанавливает JSON `Content-Type` и сериализует значение. `gin.H` — короткая запись для `map[string]any`; для стабильного публичного контракта ошибок лучше оставить отдельную структуру `errorResponse`.

Группы маршрутов

Когда API разрастётся, маршруты удобно объединить общим префиксом:

```
func (api *API) Router() *gin.Engine {
  router := gin.New()

  router.GET("/healthz", api.health)

  v1 := router.Group("/api/v1")
  v1.GET("/version", api.version)

  return router
}
```

Группа не создаёт новый сервер: она только добавляет общий путь и позволяет назначить middleware всем маршрутам внутри неё. Она пригодится, если позже появится версионирование, например `/api/v1`. При миграции в этой главе сохраняем публичные URL версии на `net/http`: цель — сравнить Gin со стандартной библиотекой, а не одновременно изменить контракт для клиентов.

Контракт маршрутов после миграции:

Метод и путь	Ответ при успехе
`GET /healthz`	`200 OK` и `{"status":"ok"}`
`GET /tasks`	`200 OK` и массив задач
`POST /tasks`	`201 Created` и новая задача
`POST /tasks/:id/complete`	`200 OK` и выполненная задача

Входной JSON и сохранение контракта

`ShouldBindJSON` удобно для простых form и JSON-запросов, но в предыдущей главе API уже зафиксировал более строгий контракт: body не больше 1 MiB, неизвестные поля запрещены, а после одного JSON-объекта не должно быть другого значения. Сохраним эти правила в небольшом helper на стандартном `json.Decoder`:

```
type createTaskRequest struct {
  Title    string `json:"title"`
  Priority int    `json:"priority"`
}

const maxRequestBodyBytes = 1 << 20 // 1 MiB

func decodeJSON(c *gin.Context, dst any) error {
  c.Request.Body = http.MaxBytesReader(c.Writer, c.Request.Body, maxRequestBodyBytes)

  decoder := json.NewDecoder(c.Request.Body)
  decoder.DisallowUnknownFields()

  if err := decoder.Decode(dst); err != nil {
    return err
  }
  if err := decoder.Decode(&struct{}{}); !errors.Is(err, io.EOF) {
    return errors.New("в body должен быть один JSON-объект")
  }

  return nil
}
```

Helper использует `http.MaxBytesReader`, поэтому превышение лимита завершается ошибкой, а не чтением всего запроса в память. `DisallowUnknownFields` отклоняет опечатки наподобие `priorty`, а второй `Decode` гарантирует, что body содержит ровно один JSON-объект. Для этого фрагмента нужны импорты `encoding/json`, `errors` и `io`.

Handler создания задачи:

```
func (api *API) createTask(c *gin.Context) {
  var request createTaskRequest
  if err := decodeJSON(c, &request); err != nil {
    writeError(c, http.StatusBadRequest, "некорректные данные задачи")
    return
  }

  request.Title = strings.TrimSpace(request.Title)
  if request.Title == "" {
    writeError(c, http.StatusBadRequest, "название задачи обязательно")
    return
  }
  if request.Priority < 1 || request.Priority > 5 {
```

```
    writeError(c, http.StatusBadRequest, "приоритет должен быть от 1 до 5")
return } createdTask, err := api.service.Create(c.Request.Context(),
task.CreateInput{   Title:    request.Title,     Priority: request.Priority,  })   if
err != nil {    api.logger.Error("не удалось создать задачу", "err", err)
writeError(c, http.StatusInternalServerError, "внутренняя ошибка сервера")    return }
api.logger.Info("задача создана", "task_id", createdTask.ID)
c.JSON(http.StatusCreated, createdTask)}
```

Передавать `id` и `completed` по-прежнему нельзя: `createTaskRequest` содержит только разрешённые поля. Валидация title и priority остаётся в handler, а service повторяет бизнес-инварианты для других входов помимо HTTP.

Параметры, статусы и доменные ошибки

Извлечём `id`, завершим задачу и сохраним прежний HTTP-контракт:

```
func (api *API) completeTask(c *gin.Context) {
  id, err := strconv.Atoi(c.Param("id"))
  if err != nil || id < 1 {
    writeError(c, http.StatusBadRequest, "идентификатор задачи должен быть
положительным числом")
    return
  }

  completedTask, err := api.service.Complete(c.Request.Context(), id)
  if errors.Is(err, store.ErrTaskNotFound) {
    writeError(c, http.StatusNotFound, "задача не найдена")
    return
  }
  if err != nil {
    api.logger.Error("не удалось завершить задачу", "task_id", id, "err", err)
    writeError(c, http.StatusInternalServerError, "внутренняя ошибка сервера")
    return
  }

  c.JSON(http.StatusOK, completedTask)
}
```

Gin делает извлечение маршрута короче, но ответственность не меняется: handler проверяет форму входа и переводит известные ошибки домена в HTTP-статусы; service хранит бизнес-логику.

Middleware: общая обвязка запроса

Middleware получает `*gin.Context`, выполняется до и/или после handler и вызывает `c.Next()`, чтобы передать управление дальше:

```
func exampleMiddleware() gin.HandlerFunc {
  return func(c *gin.Context) {
    // До handler.
    c.Next()
    // После handler.
  }
}
```

Порядок `router.Use` важен. Middleware, зарегистрированный раньше, начинает выполняться раньше и завершает часть «после handler» позже — как вложенные `defer`.

Request ID

Каждому запросу полезен идентификатор: по нему можно связать access log, application log и сообщение об ошибке. Клиент может прислать `X-Request-ID`; если заголовка нет, сервер генерирует случайное значение.

```
const (
  requestIDKey       = "request_id"
  maxRequestIDLength = 128
)
```

Чтобы не вводить дополнительную зависимость только ради ID, используем стандартные `crypto/rand` и `encoding/hex`:

```
func newRequestID() (string, error) {
  bytes := make([]byte, 16)
  if _, err := rand.Read(bytes); err != nil {
    return "", err
  }

  return hex.EncodeToString(bytes), nil
}
```

Полный middleware должен обработать редкую ошибку генератора и не продолжать запрос без ID:

```
func requestID() gin.HandlerFunc {
  return func(c *gin.Context) {
    id := c.GetHeader("X-Request-ID")
```

```
    if len(id) > maxRequestIDLength {       writeError(c, http.StatusBadRequest,
"слишком длинный X-Request-ID")       c.Abort()       return   }   if id == "" {
var err error       id, err = newRequestID()       if err != nil {
writeError(c, http.StatusInternalServerError, "не удалось подготовить запрос")
c.Abort()         return     }   }   c.Set(requestIDKey, id)   c.Header("X-Request-
ID", id)   c.Next() }}
```

`c.Abort()` останавливает оставшиеся handlers текущей цепочки. Не вызывайте `c.Next()` после него. Ограничение длины не позволяет передать в логи произвольно большой заголовок от клиента.

Структурированный access log

Используем `slog`, чтобы access log был пригоден для поиска и машинной обработки:

```
func accessLog(logger *slog.Logger) gin.HandlerFunc {
  return func(c *gin.Context) {
    startedAt := time.Now()
    c.Next()

    requestID, _ := c.Get(requestIDKey)
    logger.Info(
      "HTTP-запрос завершён",
      "request_id", requestID,
      "method", c.Request.Method,
      "path", c.Request.URL.Path,
      "status", c.Writer.Status(),
      "duration", time.Since(startedAt),
    )
  }
}
```

Лог пишется после `c.Next()`, поэтому в него попадает итоговый статус. Значение из `c.Get` имеет тип `any`: middleware не должен паниковать, если ID отсутствует по ошибке конфигурации. В production можно дополнить запись размером ответа и IP клиента, но не логируйте тело запроса с паролями, токенами или персональными данными.

Recovery

Panic не должен завершать весь HTTP-сервер. Gin содержит `gin.Recovery()` — middleware, который перехватывает panic, пишет её в `stderr` и отвечает `500`:

```
router.Use(gin.Recovery())
```

В production чаще нужен структурированный лог panic. Тогда можно применить `gin.CustomRecovery`:

```
func recovery(logger *slog.Logger) gin.HandlerFunc {
  return gin.CustomRecovery(func(c *gin.Context, recovered any) {
    logger.Error("panic в HTTP handler", "recovered", recovered)
    c.AbortWithStatusJSON(http.StatusInternalServerError, errorResponse{
      Error: "внутренняя ошибка сервера",
    })
  })
}
```

Не используйте recovery для ожидаемых ошибок валидации или доступа к данным. Он нужен как последняя защита от ошибок программирования; причину panic всё равно нужно исправить и закрепить тестом.

Собираем router

Создадим middleware один раз при построении router:

```
func (api *API) Router() *gin.Engine {
  router := gin.New()
  router.Use(requestID())
  router.Use(accessLog(api.logger))
  router.Use(recovery(api.logger))

  router.GET("/healthz", api.health)

  router.GET("/tasks", api.listTasks)
  router.POST("/tasks", api.createTask)
  router.POST("/tasks/:id/complete", api.completeTask)

  return router
}
```

Порядок выбран осознанно:

1. `requestID` добавляет ID в context и response header;
2. `accessLog` измеряет всю последующую обработку и видит итоговый статус;
3. `recovery` преобразует panic в ответ `500`;
4. handler выполняет бизнес-сценарий.

Запуск приложения

`*gin.Engine` реализует `http.Handler`, поэтому сборка `http.Server` почти не меняется:

```
server := &http.Server{
  Addr:              ":8080",
  Handler:           api.Router(),
  ReadHeaderTimeout: 5 * time.Second,
}

if err := server.ListenAndServe(); err != nil && !errors.Is(err,
http.ErrServerClosed) {
  logger.Error("HTTP-сервер завершился с ошибкой", "err", err)
}
```

Не используйте `router.Run()` в финальном `main`: он создаёт сервер сам и не даёт централизованно задать `http.Server`. Таймауты и graceful shutdown добавим в следующей главе о жизненном цикле.

Тестирование Gin handler

Gin также совместим с `httptest`: запуска сервера и реального порта не требуется. В тестах установите release mode, чтобы не видеть debug output Gin:

```
func TestCreateTask(t *testing.T) {
  gin.SetMode(gin.TestMode)

  service := &fakeTaskService{
    createdTask: task.Task{ID: 1, Title: "Купить молоко", Priority: 2},
  }
  api := New(service, newTestLogger())

  request := httptest.NewRequest(
    http.MethodPost,
    "/tasks",
    strings.NewReader(`{"title":"Купить молоко","priority":2}`),
  )
  request.Header.Set("X-Request-ID", "test-request-1")
  recorder := httptest.NewRecorder()

  api.Router().ServeHTTP(recorder, request)

  if recorder.Code != http.StatusCreated {
    t.Fatalf("status = %d, ожидался %d", recorder.Code, http.StatusCreated)
  }
```

```
  if got := recorder.Header().Get("X-Request-ID"); got != "test-request-1" {
t.Fatalf("X-Request-ID = %q", got)  } var got task.Task if err :=
json.NewDecoder(recorder.Body).Decode(&got); err != nil {   t.Fatalf("не удалось
прочитать JSON-ответ: %v", err)  } if got != service.createdTask {
t.Fatalf("ответ = %+v, ожидалось %+v", got, service.createdTask)   }}
```

Тест проверяет тот же HTTP-контракт, что и версия на `net/http`, а также поведение middleware. Fake service остаётся прежним: Gin не проник в бизнес-логику.

Сравнение с `net/http`

Задача	`net/http`	Gin
Router	`http.NewServeMux()`	`gin.New()`
Маршрут	`"POST /tasks/{id}/complete"`	`POST("/tasks/:id/complete", ...)`
Параметр пути	`r.PathValue("id")`	`c.Param("id")`
JSON-ответ	helper + `json.Encoder`	`c.JSON(status, value)`
JSON-body	`json.Decoder` + ручная проверка	`c.ShouldBindJSON(&request)` + теги
Middleware	собственная обёртка `http.Handler`	`router.Use(...)`
Тест	`httptest`	тот же `httptest`

Gin уменьшает объём транспортного кода, но не отменяет HTTP: по-прежнему нужно выбирать статусы, валидировать вход, передавать context и не раскрывать внутренние ошибки.

Частые ошибки

Передавать `*gin.Context` в service

```
// Плохо: service теперь зависит от Gin.
api.service.Create(c, input)
```

Передавайте `c.Request.Context()`.

Использовать `gin.Default()` вместе со своими logger и recovery

```
router := gin.Default()
router.Use(accessLog(logger), recovery(logger))
```

Так будут два logger и два recovery middleware. Выберите `gin.Default()` с настройками по умолчанию **или** `gin.New()` с явной цепочкой. В этом проекте используется второй вариант.

Отдавать panic клиенту

Не записывайте `fmt.Sprint(recovered)` в JSON. Логируйте подробность на сервере и возвращайте клиенту нейтральный `500`.

Пропустить строгую проверку JSON

Если API ранее отвергал неизвестные поля, включите `gin.EnableJsonDecoderDisallowUnknownFields()` и добавьте тест. Иначе опечатка клиента может стать тихой ошибкой.

Дальше

HTTP-слой Task Manager API теперь основан на Gin. В следующей главе изучим стандартный `http.Client` и научимся безопасно вызывать внешние HTTP API с context, таймаутами и проверкой статусов.

Самопроверка

1. Какую часть Task Manager API заменяет Gin в этой главе?

A. Service и все бизнес-правила

B. Только HTTP-слой: маршрутизацию, handlers и middleware

C. Модель Task и unit-тесты

D. Репозиторий и PostgreSQL

2. Как в Gin записывают path-параметр id в маршруте?

A. /tasks/{id}/complete

B. /tasks/<id>/complete

C. /tasks/?id/complete

D. /tasks/:id/complete

3. Как получить path-параметр id в Gin handler?

A. c.Param("id")

B. c.JSON("id")

C. c.PathValue("id")

D. c.Header("id")

4. Какой context нужно передавать из Gin handler в service?

A. context.Background() для каждого запроса

B. Никакой context не нужен

C. Сам *gin.Context

D. c.Request.Context()

5. Почему для создания задачи подходит c.ShouldBindJSON?

A. Он запускает HTTP-сервер

B. Он заменяет проверку бизнес-правил в service

C. Он связывает JSON со структурой и возвращает ошибку без автоматического ответа

D. Он автоматически сохраняет задачу в Store

6. Что нужно включить, чтобы Gin отклонял неизвестные JSON-поля?

A. c.Abort()

B. http.MaxBytesReader() без decoder

C. gin.EnableDebugPrintRouteFunc()

D. gin.EnableJsonDecoderDisallowUnknownFields()

7. Зачем request ID добавляют в response header?

A. Чтобы отключить middleware

B. Чтобы заменить HTTP-статус

C. Чтобы клиент мог связать свой запрос с серверными логами

D. Чтобы передать пароль пользователя

8. Что делает c.Abort() в middleware?

A. Останавливает выполнение оставшихся handlers текущей цепочки

B. Перезапускает service

C. Завершает весь HTTP-сервер

D. Удаляет request из логов

9. Почему в проекте выбирают gin.New(), а не gin.Default()?

A. gin.New() автоматически подключает базу данных

B. gin.Default() не поддерживает маршруты

C. Так Gin перестаёт использовать net/http

D. Чтобы явно подключить свой slog access log и recovery без дублирования встроенных middleware

10. Как тестировать Gin handler без запуска сервера на реальном порту?

A. Никак: Gin требует реальную сеть

B. Через httptest.NewRequest, httptest.NewRecorder и router.ServeHTTP

C. Только вручную через curl

D. Через time.Sleep после router.Run()

Ответы к самопроверке

1. Только HTTP-слой: маршрутизацию, handlers и middleware

2. /tasks/:id/complete

3. c.Param("id")

4. c.Request.Context()

5. Он связывает JSON со структурой и возвращает ошибку без автоматического ответа

6. gin.EnableJsonDecoderDisallowUnknownFields()

7. Чтобы клиент мог связать свой запрос с серверными логами

8. Останавливает выполнение оставшихся handlers текущей цепочки

9. Чтобы явно подключить свой slog access log и recovery без дублирования встроенных middleware

10. Через httptest.NewRequest, httptest.NewRecorder и router.ServeHTTP

Задания: миграция Task Manager API на Gin

Перенесите HTTP-слой из предыдущей главы на Gin. `TaskService`, TaskStore и правила работы с задачами не меняйте: новая глава должна заменить только router, handlers и общую HTTP-обвязку.

Задание 1. Подключите Gin и перенесите health-check

Установите Gin, создайте `gin.New()` и перенесите `GET /healthz`. Ответ должен остаться прежним: JSON `{"status":"ok"}` со статусом `200`.

Критерии проверки:

- Gin добавлен через `go get github.com/gin-gonic/gin`;
- router создаётся через `gin.New()`;
- маршрут регистрируется методом `router.GET`;
- handler получает `*gin.Context`;
- ответ формируется через `c.JSON`;
- `TaskService` для health-check не вызывается.

Подсказка

```
func (api *API) health(c *gin.Context) {
  c.JSON(http.StatusOK, gin.H{"status": "ok"})
}
```

Решение

```
func (api *API) Router() *gin.Engine {
  router := gin.New()
  router.GET("/healthz", api.health)
  return router
}
```

Задание 2. Перенесите маршруты задач без изменения URL

Перенесите `GET /tasks`, `POST /tasks` и `POST /tasks/:id/complete`, сохранив URL из версии на `net/http`. Во всех вызовах service передавайте `c.Request.Context()`. Для знакомства с группами отдельно создайте `router.Group("/api/v1")` и добавьте в него будущий маршрут-заглушку, например `GET /version`.

Критерии проверки:

- health-check остаётся вне группы;
- задачи остаются доступны по `/tasks`;
- ID в маршруте записан как `:id`;
- HTTP handler не получает прямой доступ к TaskStore;
- в service передаётся стандартный context запроса;
- service и repository не импортируют Gin.

Подсказка

```
v1 := router.Group("/api/v1")
v1.GET("/version", func(c *gin.Context) {
  c.JSON(http.StatusOK, gin.H{"version": "v1"})
})
```

Задание 3. Перенесите создание задачи на binding

Перепишите `POST /tasks` через `ShouldBindJSON`. Добавьте теги `binding` для обязательного названия и приоритета от 1 до 5, затем отдельно удалите внешние пробелы у названия.

Критерии проверки:

- используется `ShouldBindJSON`, а не `BindJSON`;
- структура запроса содержит `json` и `binding` теги;
- пустое или состоящее из пробелов название даёт `400`;
- `priority` вне диапазона 1–5 даёт `400`;
- успешный ответ имеет статус `201`;
- `id` и `completed` не принимаются из входного JSON.

Подсказка

```
type createTaskRequest struct {
  Title    string `json:"title" binding:"required"`
  Priority int    `json:"priority" binding:"required,gte=1,lte=5"`
}
```

Фрагмент решения

```
var request createTaskRequest
if err := c.ShouldBindJSON(&request); err != nil {
  writeError(c, http.StatusBadRequest, "некорректные данные задачи")
  return
}

request.Title = strings.TrimSpace(request.Title)
if request.Title == "" {
  writeError(c, http.StatusBadRequest, "название задачи обязательно")
  return
}
```

Задание 4. Добавьте middleware

Подключите middleware в порядке `requestID`, `accessLog`, `recovery`. Request ID берите из `X-Request-ID` либо генерируйте 16 криптографически случайных байт через `crypto/rand`, кодируйте их в hex и возвращайте в response header. Access log должен содержать ID, метод, путь, статус и длительность. Recovery должен вернуть безопасный JSON `500`.

Критерии проверки:

- используется `gin.New()`, а не `gin.Default()`;
- request ID записывается в `c.Set` и `X-Request-ID`;
- middleware вызывает `c.Next()` только при успешном продолжении;
- access log создаётся после `c.Next()`;
- access log использует `slog.Logger`;
- panic не завершает процесс и не раскрывается клиенту;
- recovery завершает цепочку через `AbortWithStatusJSON`;
- порядок middleware соответствует заданию.

Подсказка

```
router.Use(requestID())
router.Use(accessLog(api.logger))
router.Use(recovery(api.logger))
```

Фрагмент решения

```
func accessLog(logger *slog.Logger) gin.HandlerFunc {
  return func(c *gin.Context) {
    startedAt := time.Now()
    c.Next()

    requestID, _ := c.Get(requestIDKey)
    logger.Info("HTTP-запрос завершён",
      "request_id", requestID,
      "method", c.Request.Method,
      "path", c.Request.URL.Path,
      "status", c.Writer.Status(),
      "duration", time.Since(startedAt),
    )
  }
}
```

Задание 5. Сохраните строгую проверку JSON

Включите `gin.EnableJsonDecoderDisallowUnknownFields()` при запуске приложения. Добавьте тест: запрос на создание задачи с опечаткой `priorty` должен вернуть `400`, а service не должен быть вызван.

Критерии проверки:

- глобальная настройка Gin включается один раз;
- запрос с неизвестным полем возвращает `400`;
- fake service не получил `Create`;
- тест не зависит от сети, задержек или порядка запуска;
- после настройки успешный корректный JSON продолжает обрабатываться.

Подсказка

```
gin.EnableJsonDecoderDisallowUnknownFields()
```

Задание 6. Протестируйте маршрут и middleware

Через `httptest` проверьте успешный `POST /tasks`, передав `X-Request-ID: test-request-1`. Убедитесь, что ответ содержит `201`, JSON созданной задачи и тот же `X-Request-ID`. Затем проверьте handler с panic: response должен быть `500` с безопасным JSON.

Критерии проверки:

- перед тестами используется `gin.SetMode(gin.TestMode)`;
- сервер не запускается через `Run` или `ListenAndServe`;
- проверяются статус, JSON и request ID header;
- panic не попадает в response;
- logger пишет в `io.Discard`;
- `go test ./...` и `go test -race ./...` проходят.

Подсказка

```
request := httptest.NewRequest(http.MethodPost, "/tasks", body)
request.Header.Set("X-Request-ID", "test-request-1")
recorder := httptest.NewRecorder()

api.Router().ServeHTTP(recorder, request)
```

Проверьте себя

Перед следующей главой убедитесь, что вы можете:

- объяснить разницу между `*gin.Context` и `context.Context`;
- перенести маршруты `net/http` на Gin без изменения service;
- связать JSON с request DTO и проверить его;
- построить middleware для request ID, access log и recovery;
- проверить Gin router и middleware через `httptest`.

Рекомендации по коду: Gin

- Внедряйте Gin только в transport-слой; service, repository и модель не должны получать `*gin.Context`.
- Передавайте в нижние слои `c.Request.Context()`, а не новый `context.Background()`.
- Используйте `gin.New()` с явным набором middleware или `gin.Default()` — не подключайте оба набора logger/recovery одновременно.
- Сохраняйте стабильный JSON-контракт ошибок через отдельную структуру, а не случайные строки или технические `error`.
- Создавайте request DTO с разрешёнными полями; не принимайте из JSON серверные поля `id` и `completed`.
- Проверяйте JSON binding, значения после нормализации и известные ошибки service до выбора HTTP-статуса.
- Используйте request ID в response и структурированных логах, но не принимайте неконтролируемо длинные значения заголовка от клиента.
- Recovery должен логировать проблему на сервере и возвращать клиенту нейтральный `500`, а не текст panic.
- Тестируйте Gin router через `httptest` и fake service; не запускайте реальный порт в unit-тестах.

Чек-лист

Gin заменяет только HTTP-обвязку, service и repository остаются независимыми от фреймворка, каждый запрос имеет request ID, panic не завершает сервер, а публичный HTTP-контракт закреплён тестами.

HTTP-клиент: внешние API

Цель урока

Научиться вызывать внешний HTTP API через стандартный `http.Client`: строить request с `context.Context`, передавать query-параметры и заголовки, корректно закрывать response body, проверять HTTP-статус, декодировать JSON и тестировать клиент без реальной сети.

Task Manager API уже принимает запросы как HTTP-сервер. Backend часто выступает и клиентом: запрашивает данные у сервиса уведомлений, платежей, геокодирования или другого внутреннего API. В этой главе Task Manager обращается к условному внешнему сервису, чтобы получить подсказку приоритета для новой задачи.

`http.Client` и request

Быстрый вызов можно сделать через `http.Get`, но для backend-кода обычно нужен явный `http.Client` и request:

```
client := &http.Client{
  Timeout: 3 * time.Second,
}

request, err := http.NewRequestWithContext(
  ctx,
  http.MethodGet,
  "https://priority.example.com/v1/suggest?title=Купить+молоко",
  nil,
)
if err != nil {
  return err
}

response, err := client.Do(request)
if err != nil {
  return err
}
defer response.Body.Close()
```

`http.Client` можно безопасно переиспользовать между запросами и goroutine. Создавайте его один раз при сборке приложения и передавайте как зависимость. Создание нового `http.Client` на каждый вызов мешает переиспользованию соединений.

`http.NewRequestWithContext` связывает запрос со стандартным `context.Context`. Если входящий запрос Task Manager отменён, внешний вызов тоже должен завершиться:

```
hint, err := priorityClient.Suggest(c.Request.Context(), title)
```

Не создавайте внутри клиента новый `context.Background()`: так отмена и дедлайн потеряются.

Таймаут клиента и дедлайн context

У HTTP-вызова может быть два ограничения по времени:

```
client := &http.Client{Timeout: 3 * time.Second}

ctx, cancel := context.WithTimeout(parentCtx, 1*time.Second)
defer cancel()
```

- `ctx` задаёт дедлайн **конкретной операции** и наследует отмену родителя;
- `Client.Timeout` — предельное время всей HTTP-операции клиента: соединения, редиректы и чтения body.

В Task Manager передавайте входящий `r.Context()` и настройте общий ограниченный `Client.Timeout`. Если для конкретной зависимости нужен более короткий дедлайн, создайте дочерний context от полученного родителя. Всегда вызывайте `cancel`.

```
ctx, cancel := context.WithTimeout(parentCtx, 800*time.Millisecond)
defer cancel()

hint, err := priorityClient.Suggest(ctx, title)
```

Таймаут — не замена отмене: он ограничивает время, но код всё равно должен возвращать ошибку вызывающему слою. Не подбирайте таймауты «на глаз» для тестов; тесты этой главы не ждут реальные секунды.

Контракт внешнего сервиса

Пусть сервис подсказок поддерживает запрос:

```
GET /v1/suggest?
title=%D0%9A%D1%83%D0%BF%D0%B8%D1%82%D1%8C+%D0%BC%D0%BE%D0%BB%D0%BE%D0%BA%D0%BE
HTTP/1.1
Accept: application/json
User-Agent: task-manager-api/1.0
```

И отвечает:

```
{
  "priority": 3,
  "source": "title-rules"
}
```

Внешний контракт не принадлежит нашей модели `Task`, поэтому выделим отдельный тип:

```
type PriorityHint struct {
  Priority int    `json:"priority"`
  Source   string `json:"source"`
}
```

Не декодируйте ответ внешнего API напрямую в `task.Task`: внешний сервис не должен иметь возможности задавать ID, completed или другие поля нашей предметной модели.

Клиент как зависимость

Спрячем детали HTTP в пакете `internal/priorityclient`:

```
type Client struct {
  baseURL    string
  httpClient *http.Client
}

func New(baseURL string, httpClient *http.Client) *Client {
  if httpClient == nil {
    httpClient = &http.Client{Timeout: 3 * time.Second}
  }

  return &Client{
    baseURL:    strings.TrimRight(baseURL, "/"),
    httpClient: httpClient,
  }
}
```

`baseURL` и `httpClient` передаются снаружи: production использует адрес из конфигурации, а тест — адрес `httptest.Server`. Так тест не обращается к интернету.

Безопасная сборка URL

Не добавляйте пользовательский текст к URL конкатенацией строк:

```
// Плохо: пробелы, &, ? и кириллица могут сломать запрос.
// endpoint := baseURL + "/v1/suggest?title=" + title
```

Используйте `net/url`:

```
endpoint, err := url.Parse(client.baseURL + "/v1/suggest")
if err != nil {
  return PriorityHint{}, fmt.Errorf("разобрать URL сервиса приоритетов: %w", err)
}

query := endpoint.Query()
query.Set("title", title)
endpoint.RawQuery = query.Encode()
```

`query.Encode()` экранирует специальные символы. Например, название `Позвонить маме & купить чай` будет передано одним параметром `title`, а не превратится в несколько query-параметров.

Полная реализация `Suggest`

Опишем ошибку неуспешного HTTP-статуса отдельно. Вызывающий код может проверить её через `errors.As` и решить, стоит ли повторять запрос или вернуть пользователю другой ответ:

```
type HTTPStatusError struct {
  StatusCode int
  Status     string
}

func (err *HTTPStatusError) Error() string {
  return fmt.Sprintf("сервис приоритетов вернул %s", err.Status)
}
```

Реализация клиента:

```
const maxResponseBytes = 1 << 20 // 1 MiB

func (client *Client) Suggest(ctx context.Context, title string) (PriorityHint,
error) {
  endpoint, err := url.Parse(client.baseURL + "/v1/suggest")
  if err != nil {
    return PriorityHint{}, fmt.Errorf("разобрать URL сервиса приоритетов: %w", err)
  }
```

```
  query := endpoint.Query()  query.Set("title", title)  endpoint.RawQuery =
query.Encode()  request, err := http.NewRequestWithContext(ctx, http.MethodGet,
endpoint.String(), nil)  if err != nil {     return PriorityHint{},
fmt.Errorf("создать запрос подсказки: %w", err)  } request.Header.Set("Accept",
"application/json") request.Header.Set("User-Agent", "task-manager-api/1.0")
response, err := client.httpClient.Do(request) if err != nil {     return
PriorityHint{}, fmt.Errorf("выполнить запрос подсказки: %w", err)  } defer
response.Body.Close()  if response.StatusCode < http.StatusOK || response.StatusCode
>= http.StatusMultipleChoices {     return PriorityHint{}, &HTTPStatusError{
StatusCode: response.StatusCode,      Status:     response.Status,   } } var hint
PriorityHint  if err := json.NewDecoder(io.LimitReader(response.Body,
maxResponseBytes)).Decode(&hint); err != nil {   return PriorityHint{},
fmt.Errorf("прочитать JSON-подсказку: %w", err)  } if hint.Priority < 1 ||
hint.Priority > 5 {   return PriorityHint{}, fmt.Errorf("некорректный приоритет
внешнего сервиса: %d", hint.Priority) } return hint, nil}
```

Ключевой порядок действий:

1. `Do` вернул response — сразу планируем `defer response.Body.Close()`;
2. проверяем статус **до** разбора JSON успеха;
3. ограничиваем размер читаемого response body;
4. декодируем в отдельный внешний DTO;
5. проверяем данные внешней системы, даже если ответ имеет статус `2xx`.

`response.Body.Close()` важен даже при ошибочном статусе: он освобождает ресурс и позволяет клиенту переиспользовать соединения.

Проверка статусов и ошибки сети

HTTP-клиент не считает `404`, `429` или `500` ошибкой транспорта: в этих случаях `client.Do` обычно вернёт `response` и `nil` error. Поэтому статус нужно проверять самостоятельно.

```
if response.StatusCode < 200 || response.StatusCode >= 300 {
  return PriorityHint{}, &HTTPStatusError{...}
}
```

Транспортная ошибка — это, например, отмена context, проблема DNS или недоступное соединение. Она приходит из `Do`:

```
response, err := client.httpClient.Do(request)
if err != nil {
  if errors.Is(err, context.Canceled) {
    return PriorityHint{}, err
  }
```

```
  return PriorityHint{}, fmt.Errorf("выполнить запрос подсказки: %w", err)}
```

На границе HTTP handler не превращайте все ошибки клиента во внутренний `500` автоматически. Например, отменённый клиентский запрос обычно уже не ждёт ответ; ограничение внешнего сервиса может требовать понятной политики. В этой главе client только возвращает причины, а решение о публичном HTTP-статусе остаётся за service или handler.

Заголовки

Заголовки передают метаданные запроса:

```
request.Header.Set("Accept", "application/json")
request.Header.Set("Authorization", "Bearer "+token)
request.Header.Set("X-Request-ID", requestID)
```

- `Accept` сообщает ожидаемый формат ответа;
- `Authorization` используют только при необходимости и получают token из конфигурации, а не из исходного кода;
- `X-Request-ID` связывает логи нескольких сервисов.

Не записывайте `Authorization`, cookies и полный body в логи. Если копируете входящий request ID во внешний запрос, сначала извлеките и проверьте его по правилам middleware из предыдущей главы.

Где вызывать внешний клиент

HTTP handler не должен сам знать URL внешнего сервиса. Пусть service получает интерфейс:

```
type PrioritySuggester interface {
  Suggest(ctx context.Context, title string) (priorityclient.PriorityHint, error)
}

type TaskService struct {
  store     TaskRepository
  priorities PrioritySuggester
}
```

При создании задачи service может применить подсказку, если пользователь не выбрал приоритет сам. Политику нужно определить явно: внешний сервис недоступен — создание задачи завершается ошибкой или используется приоритет по умолчанию? Для первого учебного API выберите один вариант и закрепите его тестом. Не скрывайте ошибку молча.

Тестирование через `httptest.Server`

`httptest.Server` запускает настоящий HTTP-сервер только внутри теста и автоматически предоставляет адрес. Снаружи сеть не используется:

```
func TestSuggest(t *testing.T) {
  server := httptest.NewServer(http.HandlerFunc(func(w http.ResponseWriter, r
*http.Request) {
    if r.Method != http.MethodGet {
      t.Fatalf("method = %s, ожидался GET", r.Method)
    }
    if got := r.URL.Query().Get("title"); got != "Купить молоко" {
      t.Fatalf("title = %q", got)
    }
    if got := r.Header.Get("Accept"); got != "application/json" {
      t.Fatalf("Accept = %q", got)
    }

    w.Header().Set("Content-Type", "application/json")
    fmt.Fprint(w, `{"priority":3,"source":"title-rules"}`)
  }))
  defer server.Close()

  client := New(server.URL, server.Client())
  hint, err := client.Suggest(context.Background(), "Купить молоко")
  if err != nil {
    t.Fatalf("Suggest вернул ошибку: %v", err)
  }
  if hint != (PriorityHint{Priority: 3, Source: "title-rules"}) {
    t.Fatalf("hint = %+v", hint)
  }
}
```

Проверяйте контракт исходящего запроса: метод, путь, query и существенные заголовки. Затем проверяйте, как клиент обработал ответ.

Неуспешный статус

```
func TestSuggestStatusError(t *testing.T) {
  server := httptest.NewServer(http.HandlerFunc(func(w http.ResponseWriter, r
*http.Request) {
    w.WriteHeader(http.StatusTooManyRequests)
  }))
  defer server.Close()
```

```
  client := New(server.URL, server.Client())  _, err :=
client.Suggest(context.Background(), "Купить молоко")  var statusError
*HTTPStatusError  if !errors.As(err, &statusError) {   t.Fatalf("error = %v,
ожидалась HTTPStatusError", err) } if statusError.StatusCode !=
http.StatusTooManyRequests {   t.Fatalf("status = %d", statusError.StatusCode)  }}
```

Отмена без `time.Sleep`

Чтобы детерминированно проверить связь request и context, отмените context **до** вызова. Клиент не должен пытаться ждать ответ:

```
func TestSuggestCanceled(t *testing.T) {
  server := httptest.NewServer(http.NotFoundHandler())
  defer server.Close()

  ctx, cancel := context.WithCancel(context.Background())
  cancel()

  client := New(server.URL, server.Client())
  _, err := client.Suggest(ctx, "Купить молоко")
  if !errors.Is(err, context.Canceled) {
    t.Fatalf("error = %v, ожидалась context.Canceled", err)
  }
}
```

Тест не использует настоящую задержку и не зависит от доступности публичного сервиса.

Повторы запросов

Не добавляйте автоматические retry «на всякий случай». Повтор может создать дубликат для `POST`, усилить нагрузку на недоступный сервис и ухудшить ситуацию при `429`.

Для идемпотентного `GET` повтор иногда оправдан после временной сетевой ошибки, но только с ограничением числа попыток, backoff, учётом context и понятной политикой. Эти сценарии относятся к более продвинутой production-надёжности; базовый клиент этой главы делает одну прозрачную попытку.

Частые ошибки

Не закрывать response body

```
response, err := client.Do(request)
if err != nil {
  return err
```

```
}// Пропущен response.Body.Close().
```

Это может истощить ресурсы и лишить клиента переиспользования соединений. Сразу после успешного `Do` пишите `defer response.Body.Close()`.

Декодировать ошибочный ответ как успешный JSON

```
json.NewDecoder(response.Body).Decode(&hint)
```

Сначала проверяйте статус. Сервер может прислать JSON другого формата, HTML или пустое тело с `404` и `500`.

Создать новый context внутри клиента

```
request, _ := http.NewRequestWithContext(context.Background(), ...)
```

Так отмена исходной операции не дойдёт до внешнего запроса. Получайте `ctx` параметром.

Тестировать публичный сервис

Публичный сервис может быть недоступен, изменить контракт или ограничить число запросов. Для unit-тестов используйте `httptest.Server`; реальную интеграцию проверяйте отдельно и осознанно.

Дальше

Task Manager умеет безопасно вызывать внешние API. Далее настроим конфигурацию и жизненный цикл приложения: переменные окружения, проверку настроек, graceful shutdown и health-check.

Самопроверка

1. Почему http.Client обычно создают один раз и переиспользуют?

A. Он переиспользует соединения и безопасен для нескольких goroutine

B. Так JSON автоматически валидируется

C. Его нельзя передавать как зависимость

D. Он заменяет context.Context

2. Как связать исходящий HTTP request с отменой входящей операции?

A. Передать ctx только в заголовке

B. Вызвать time.Sleep перед Do

C. Создать его через http.NewRequestWithContext с полученным ctx

D. Всегда использовать context.Background()

3. Что нужно сделать сразу после успешного client.Do(request)?

A. Запланировать defer response.Body.Close()

B. Сразу вызвать panic

C. Создать ещё один http.Client

D. Игнорировать response и вернуть nil

4. Почему статус HTTP нужно проверять отдельно от ошибки client.Do?

A. client.Do всегда превращает 404 в panic

B. JSON Decoder проверяет статус сам

C. HTTP не имеет статусов

D. Не-2xx ответ обычно возвращает response и nil error транспорта

5. Как безопасно добавить пользовательское название задачи в query-параметр?

A. Через заголовок Content-Type

B. Через url.Parse, endpoint.Query и query.Encode

C. Конкатенацией строки URL и title

D. Через fmt.Println(title)

6. Зачем внешний JSON декодируют в отдельный PriorityHint?

A. Чтобы не использовать json-теги

B. Чтобы внешний сервис не задавал поля внутренней модели Task

C. Чтобы HTTP перестал использовать JSON

D. Чтобы body не нужно было закрывать

7. Что описывает Client.Timeout?

A. Только время JSON-декодирования

B. Предельное время всей HTTP-операции клиента

C. Максимальный размер response body

D. Количество retry

8. Какой инструмент подходит для unit-теста HTTP-клиента без внешней сети?

A. Публичный API в интернете

B. httptest.Server

C. go run сервера на фиксированном порту

D. time.Sleep

9. Как проверить ошибку неуспешного статуса, если она имеет тип *HTTPStatusError?

A. errors.As(err, &statusError)

B. Сравнить err.Error() с текстом

C. errors.Is(err, http.StatusTooManyRequests)

D. Игнорировать ошибку

10. Почему не стоит автоматически повторять любой HTTP-запрос?

A. Retry автоматически закрывает body

B. Повтор может создать дубликат, усилить нагрузку или нарушить политику внешнего сервиса

C. Go запрещает повторять запросы

D. GET никогда нельзя повторять

Ответы к самопроверке

1. Он переиспользует соединения и безопасен для нескольких goroutine

2. Создать его через http.NewRequestWithContext с полученным ctx

3. Запланировать defer response.Body.Close()

4. Не-2xx ответ обычно возвращает response и nil error транспорта

5. Через url.Parse, endpoint.Query и query.Encode

6. Чтобы внешний сервис не задавал поля внутренней модели Task

7. Предельное время всей HTTP-операции клиента

8. httptest.Server

9. errors.As(err, &statusError)

10. Повтор может создать дубликат, усилить нагрузку или нарушить политику внешнего сервиса

Задания: клиент подсказок приоритета

Добавьте Task Manager API клиента внешнего сервиса приоритетов. Внешний HTTP-код должен жить в отдельном пакете, а service должен зависеть от маленького интерфейса, а не от конкретного `http.Client`.

Задание 1. Создайте переиспользуемый клиент

Создайте пакет `internal/priorityclient` и тип `Client` с `baseURL` и `*http.Client`. Конструктор получает зависимости снаружи, удаляет завершающий `/` из base URL и создаёт клиент с таймаутом три секунды, если передан `nil`.

Критерии проверки:

- `Client` хранит один `*http.Client`;
- `baseURL` нормализуется через `strings.TrimRight`;
- `nil`-клиент заменяется на клиент с `Timeout`;
- новый `http.Client` не создаётся внутри каждого запроса;
- пакет не импортирует Gin.

Подсказка

```
func New(baseURL string, httpClient *http.Client) *Client {
  if httpClient == nil {
    httpClient = &http.Client{Timeout: 3 * time.Second}
  }

  return &Client{
    baseURL:    strings.TrimRight(baseURL, "/"),
    httpClient: httpClient,
  }
}
```

Задание 2. Реализуйте `Suggest`

Реализуйте метод:

```
func (client *Client) Suggest(ctx context.Context, title string) (PriorityHint,
error)
```

Он должен выполнить `GET /v1/suggest?title=...`, передать заголовок `Accept: application/json`, проверить любой `2xx` как успешный статус и декодировать `priority` и `source` из JSON.

Критерии проверки:

- request создаётся через `http.NewRequestWithContext`;
- query-параметр собирается через `net/url`;
- после успешного `Do` есть `defer response.Body.Close()`;
- response status проверяется до JSON-декодирования;
- JSON декодируется в `PriorityHint`, а не в `task.Task`;
- приоритет ответа проверяется в диапазоне 1–5.

Подсказка

```
query := endpoint.Query()
query.Set("title", title)
endpoint.RawQuery = query.Encode()
```

Задание 3. Обработайте ошибки внешнего API

Создайте тип `HTTPStatusError` с `StatusCode` и `Status`. Для любого статуса вне диапазона `200–299` возвращайте указатель на этот тип. Ошибки `Do` оборачивайте через `%w`.

Критерии проверки:

- не-2xx ответ не декодируется как успешный JSON;
- ошибка содержит HTTP-статус без сравнения текста;
- тест может извлечь ошибку через `errors.As`;
- ошибка context не теряется при оборачивании;
- response body закрывается и при не-2xx ответе.

Подсказка

```
if response.StatusCode < http.StatusOK || response.StatusCode >=
http.StatusMultipleChoices {
  return PriorityHint{}, &HTTPStatusError{
    StatusCode: response.StatusCode,
    Status:     response.Status,
  }
}
```

Задание 4. Внедрите интерфейс в service

Создайте интерфейс `PrioritySuggester` с методом `Suggest`. Передайте его в `TaskService`. Определите и задокументируйте правило: если пользователь не передал приоритет, service использует подсказку; если внешний сервис вернул ошибку, создание задачи завершается ошибкой **или** использует приоритет по умолчанию.

Критерии проверки:

- service зависит от интерфейса, а не от `priorityclient.Client`;
- в interface передаётся `context.Context`;
- выбранная политика ошибки явно описана;
- выбранная политика покрыта unit-тестом service;
- HTTP handler не знает base URL внешнего API.

Задание 5. Протестируйте успешный запрос через `httptest.Server`

Создайте `httptest.Server`, который проверяет `GET`, путь `/v1/suggest`, query `title` и `Accept`. В ответ отправьте JSON с приоритетом 3. Проверьте, что `Suggest` вернул ожидаемый `PriorityHint`.

Критерии проверки:

- адрес тестового сервера передаётся в `New`;
- используется `server.Client()`;
- проверяется исходящий метод, путь, query и заголовок;
- проверяется полученный DTO;
- тест не требует доступа в интернет.

Подсказка

```
server := httptest.NewServer(http.HandlerFunc(func(w http.ResponseWriter, r
*http.Request) {
  // Проверки request и JSON-ответ.
}))
defer server.Close()

client := New(server.URL, server.Client())
```

Задание 6. Протестируйте ошибки без задержек

Добавьте тесты для `429 Too Many Requests`, некорректного JSON и заранее отменённого context. Для статуса используйте `errors.As`, для отмены — `errors.Is(err, context.Canceled)`. Не используйте `time.Sleep`.

Критерии проверки:

- `429` возвращается как `*HTTPStatusError`;
- неверный JSON возвращает ошибку;
- отменённый до вызова context возвращает `context.Canceled`;
- каждый тест создаёт собственный `httptest.Server` при необходимости;
- отсутствуют публичные HTTP-вызовы и фиксированные задержки;
- `go test ./...` и `go test -race ./...` проходят.

Подсказка

```
ctx, cancel := context.WithCancel(context.Background())
cancel()

_, err := client.Suggest(ctx, "Купить молоко")
if !errors.Is(err, context.Canceled) {
  t.Fatalf("error = %v, ожидалась context.Canceled", err)
}
```

Проверьте себя

Перед следующей главой убедитесь, что вы можете:

- переиспользовать настроенный `http.Client`;
- создать request с родительским context;
- проверить HTTP-статус и закрыть response body;
- безопасно передать query и заголовки;
- отделить DTO внешнего API от модели Task;
- протестировать клиент через `httptest.Server` без сети и задержек.

Рекомендации по коду: HTTP-клиент

- Создавайте и переиспользуйте `http.Client` как зависимость; задавайте его общий `Timeout`.
- Стройте request через `http.NewRequestWithContext` и передавайте context исходной операции дальше.
- Сразу после успешного `Do` вызывайте `defer response.Body.Close()`.
- Проверяйте HTTP-статус до декодирования успешного JSON; не-2xx — это не обязательно ошибка транспорта.
- Собирайте URL и query-параметры через `net/url`, а не конкатенацией пользовательских строк.
- Ограничивайте размер внешнего response body и проверяйте полученные данные, даже при `2xx`.
- Отделяйте внешний DTO от внутренних моделей и зависите от небольшого интерфейса клиента в service.
- Не логируйте токены, cookies, авторизационные заголовки и полные body.
- Не включайте retry без явной политики идемпотентности, backoff, лимитов и учёта context.
- Тестируйте HTTP-клиент через `httptest.Server`; unit-тесты не должны зависеть от публичных API или `time.Sleep`.

Чек-лист

Внешний вызов ограничен по времени, отменяется вместе с родительской операцией, закрывает body, проверяет статус и JSON-контракт, а его поведение при успехе, ошибке статуса, плохом JSON и отмене закреплено детерминированными тестами.

HTTP-клиент на Resty

Цель урока

Переписать клиент подсказок приоритета на Resty и сравнить его со стандартным `http.Client`. Научиться настраивать base URL, заголовки, context, JSON-декодирование, обработку HTTP-статусов, ограниченный retry и тестировать Resty-клиент через `httptest.Server` без изменения интерфейса service.

Resty — сторонняя библиотека для HTTP-клиента. Она использует `net/http` под капотом, но сокращает повторяющийся код для query-параметров, заголовков, JSON и retry. Она не отменяет знания предыдущей главы: статусы, context, таймауты, безопасность данных и HTTP-контракт всё равно остаются ответственностью разработчика.

```
TaskService → PrioritySuggester → Resty → net/http → внешний API
```

Как и Gin, Resty должен оставаться на границе приложения. `TaskService` зависит от интерфейса `PrioritySuggester`, а не от Resty.

Установка

В корне Task Manager API выполните:

```
go get github.com/go-resty/resty/v2
```

Импорт пакета:

```
import "github.com/go-resty/resty/v2"
```

Go добавит прямую зависимость в `go.mod`, а Resty и требуемые ей модули — в `go.sum`. Не редактируйте их вручную. После добавления запустите:

```
go test ./...
```

Сравнение с `http.Client`

В предыдущей главе базовый вызов выглядел так:

```
request, err := http.NewRequestWithContext(ctx, http.MethodGet, endpoint.String(),
```

```
if err != nil {   return PriorityHint{}, err}request.Header.Set("Accept",
"application/json")response, err := httpClient.Do(request)if err != nil {   return
PriorityHint{}, err}defer response.Body.Close()if response.StatusCode < 200 ||
response.StatusCode >= 300 { return PriorityHint{}, &HTTPStatusError{...}}var hint
PriorityHintif err := json.NewDecoder(response.Body).Decode(&hint); err != nil {
return PriorityHint{}, err}
```

Resty делает тот же запрос компактнее:

```
var hint PriorityHint

response, err := client.R().
  SetContext(ctx).
  SetQueryParam("title", title).
  SetResult(&hint).
  Get("/v1/suggest")
if err != nil {
  return PriorityHint{}, err
}

if response.IsError() {
  return PriorityHint{}, &HTTPStatusError{
    StatusCode: response.StatusCode(),
    Status:     response.Status(),
  }
}
```

`SetResult` автоматически декодирует JSON или XML для успешного статуса `2xx`, если сервер указал подходящий `Content-Type`. Но Resty не выбирает за вас политику статусов, проверки значений или публичную обработку ошибок.

Настройка Resty-клиента

Как и `http.Client`, экземпляр Resty создают один раз и передают как зависимость. Настройте base URL, общий timeout и заголовки в конструкторе:

```
type Client struct {
  resty *resty.Client
}

func New(baseURL string, transport http.RoundTripper) *Client {
  client := resty.New().
    SetBaseURL(strings.TrimRight(baseURL, "/")).
    SetTimeout(3 * time.Second).
```

```
    SetHeader("Accept", "application/json").    SetHeader("User-Agent", "task-
manager-api/1.0") if transport != nil {    client.SetTransport(transport) } return
&Client{resty: client}}
```

В production чаще достаточно `resty.New()` и настроек Resty. Если программе нужен специально настроенный транспорт (proxy, TLS, собственный `RoundTripper`), передайте его явно и проверьте контракт тестами. `httptest.Server` предоставляет совместимый транспорт как `server.Client().Transport`.

> **Не смешивайте ответственность клиентов**
>
> `resty.Client.SetTimeout` задаёт общий таймаут Resty. Не нужно одновременно создавать новый `http.Client` с другим timeout на каждый запрос. Для отдельной операции используйте `SetContext` с дочерним context и `defer cancel()`.

Query-параметры и заголовки

Resty сам кодирует query-параметры:

```
response, err := client.R().
  SetContext(ctx).
  SetQueryParam("title", title).
  Get("/v1/suggest")
```

Для нескольких параметров используйте `SetQueryParams`:

```
SetQueryParams(map[string]string{
  "title":  title,
  "locale": "ru-RU",
})
```

Общие заголовки задают в конструкторе. Заголовок конкретного запроса добавляют через `SetHeader`:

```
response, err := client.R().
  SetContext(ctx).
  SetHeader("X-Request-ID", requestID).
  Get("/v1/suggest")
```

Токены и пароли берите из конфигурации. Никогда не логируйте `Authorization`, cookies и body запроса целиком.

Context и отмена

Resty не заменяет `context.Context`. Передайте context у конкретного запроса:

```
response, err := client.R().
  SetContext(ctx).
  SetQueryParam("title", title).
  Get("/v1/suggest")
```

В Gin handler исходный context приходит из `c.Request.Context()`. Service передаёт его в `PrioritySuggester`, а реализация Resty вызывает `SetContext(ctx)`. Так отмена клиента и дедлайн распространяются от HTTP handler к внешнему API.

```
c.Request.Context() → TaskService → PrioritySuggester.Suggest → Request.SetContext
```

Не используйте `context.Background()` внутри метода `Suggest`: это оборвёт цепочку отмены.

JSON request и response

Для `GET` подсказки нужен только query. Если внешний API принимает JSON, Resty сериализует структуру через `SetBody`:

```
request := struct {
  Title string `json:"title"`
}{Title: "Купить молоко"}

response, err := client.R().
  SetContext(ctx).
  SetHeader("Content-Type", "application/json").
  SetBody(request).
  SetResult(&hint).
  Post("/v1/suggest")
```

Для текущего GET-запроса ответ декодируется через `SetResult`:

```
var hint PriorityHint

response, err := client.R().
  SetContext(ctx).
  SetQueryParam("title", title).
```

```
  SetResult(&hint). Get("/v1/suggest")
```

Не декодируйте внешний ответ в `task.Task`. `PriorityHint` остаётся отдельным DTO, а service решает, как применить его к правилам создания задачи.

Полная реализация `Suggest`

Сохраним контракт предыдущей главы: service получает `PriorityHint`, а не знает, какая библиотека выполняет запрос.

```
type PriorityHint struct {
  Priority int    `json:"priority"`
  Source   string `json:"source"`
}

type HTTPStatusError struct {
  StatusCode int
  Status     string
}

func (err *HTTPStatusError) Error() string {
  return fmt.Sprintf("сервис приоритетов вернул %s", err.Status)
}

func (client *Client) Suggest(ctx context.Context, title string) (PriorityHint,
error) {
  var hint PriorityHint

  response, err := client.resty.R().
    SetContext(ctx).
    SetQueryParam("title", title).
    SetResult(&hint).
    Get("/v1/suggest")
  if err != nil {
    return PriorityHint{}, fmt.Errorf("выполнить запрос подсказки: %w", err)
  }

  if response.IsError() {
    return PriorityHint{}, &HTTPStatusError{
      StatusCode: response.StatusCode(),
      Status:     response.Status(),
    }
  }
  if !response.IsSuccess() {
```

```
    return PriorityHint{}, &HTTPStatusError{      StatusCode: response.StatusCode(),
Status:     response.Status(),   } } if hint.Priority < 1 || hint.Priority > 5 {
return PriorityHint{}, fmt.Errorf("некорректный приоритет внешнего сервиса: %d",
hint.Priority)  } return hint, nil}
```

`response.IsError()` покрывает ответы `400–599`. Дополнительная проверка `!response.IsSuccess()` защищает контракт «принимаем только 2xx»: например, `304 Not Modified` — не ошибка Resty, но не является JSON-подсказкой в нашем API.

Resty управляет response body внутри `Execute`, поэтому вручную писать `defer response.Body.Close()` здесь не нужно. Но это не отменяет правило из предыдущей главы: в чистом `net/http` body всегда закрывает вызывающий код.

Строгость JSON и размер ответа

`SetResult` удобен, но его стандартная JSON-десериализация не заменяет полный контроль предыдущей главы. Если внешний контракт требует запрета неизвестных полей, строгой проверки одного JSON-значения или своего лимита body, оставьте реализацию на `net/http` либо настройте отдельную обработку и покройте её тестами.

Для простой, доверенной интеграции `SetResult` делает код короче. Для критичной границы — платежи, безопасность, нестабильный внешний контракт — предпочтительнее явно читать и ограничивать body, как в главе про `http.Client`.

Ограниченный retry

Resty поддерживает retry с backoff. Это не повод повторять каждый запрос. В нашем случае допустим пример только для идемпотентного `GET`, когда сервер вернул временный `503 Service Unavailable`:

```
client := resty.New().
  SetBaseURL(baseURL).
  SetRetryCount(2).
  SetRetryWaitTime(100 * time.Millisecond).
  SetRetryMaxWaitTime(500 * time.Millisecond).
  AddRetryCondition(func(response *resty.Response, err error) bool {
    return err == nil && response.StatusCode() == http.StatusServiceUnavailable
  })
```

Это означает максимум три попытки: первоначальная и две повторные. Resty применяет backoff с jitter между попытками.

Не применяйте такой retry автоматически к `POST /tasks`: повтор может создать дубликат. Для безопасных повторов нужны идемпотентность, контекст, ограничение попыток, политика статусов

и наблюдаемость. В базовой реализации `Suggest` retry выключен; пример нужен, чтобы понимать настройку, а не включать её без причины.

Hooks и структурированное логирование

Resty позволяет добавить hook после получения ответа:

```
func addAccessLog(client *resty.Client, logger *slog.Logger) {
  client.OnAfterResponse(func(_ *resty.Client, response *resty.Response) error {
    logger.Info(
      "внешний HTTP-запрос завершён",
      "method", response.Request.Method,
      "url", response.Request.URL,
      "status", response.StatusCode(),
      "duration", response.Time(),
    )
    return nil
  })
}
```

Hook полезен для наблюдаемости, но не должен выводить `Authorization`, cookies или response body. Он также не заменяет обработку `err`: при сетевой ошибке response может отсутствовать, поэтому логирование таких случаев делают на границе вызова или через `OnError` с аккуратной фильтрацией данных.

Тестирование через `httptest.Server`

Resty-клиент проверяют так же, как версию на `http.Client`: тестовый сервер проверяет исходный request и отдаёт контролируемый ответ.

```
func TestSuggest(t *testing.T) {
  server := httptest.NewServer(http.HandlerFunc(func(w http.ResponseWriter, r
*http.Request) {
    if r.Method != http.MethodGet {
      t.Fatalf("method = %s, ожидался GET", r.Method)
    }
    if r.URL.Path != "/v1/suggest" {
      t.Fatalf("path = %q", r.URL.Path)
    }
    if got := r.URL.Query().Get("title"); got != "Купить молоко" {
      t.Fatalf("title = %q", got)
    }

    w.Header().Set("Content-Type", "application/json")
```

```
    fmt.Fprint(w, `{"priority":3,"source":"title-rules"}`) })) defer server.Close()
client := New(server.URL, server.Client().Transport) hint, err :=
client.Suggest(context.Background(), "Купить молоко")  if err != nil {
t.Fatalf("Suggest вернул ошибку: %v", err)  } if hint != (PriorityHint{Priority: 3,
Source: "title-rules"}) {    t.Fatalf("hint = %+v", hint) }}
```

Тесты также должны покрыть `429`, некорректный JSON и отменённый context. Для отмены отмените context до вызова — это детерминированно и не требует `time.Sleep`.

Когда выбрать `http.Client`, а когда Resty

Ситуация	Выбор
Небольшой клиент, нужен минимальный набор зависимостей и полный контроль body	`net/http`
Обычные JSON API, много одинаковых headers/query/response DTO	Resty
Строгий контроль лимитов, формата тела и потокового чтения	`net/http`
Нужны готовые retry hooks, base URL и компактный код	Resty с явной политикой retry

В итоговом Task Manager можно оставить реализацию на `http.Client` или заменить её на Resty: interface `PrioritySuggester` и service при этом не меняются. В этом курсе финальную реализацию клиента подсказок строим на Resty, а фундаментальная глава про стандартный клиент остаётся обязательной.

Частые ошибки

Считать `response.IsError()` единственной проверкой

`IsError()` не считает `3xx` ошибкой. Если контракт принимает только JSON-ответы `2xx`, после неё обязательно проверяйте `response.IsSuccess()`.

Включить retry для всех методов

```
client.SetRetryCount(3).AddRetryAfterErrorCondition()
```

Такая глобальная настройка может повторить опасные `POST`, `PATCH` и `DELETE`. Ограничивайте retry по методу и статусу либо назначайте его точечно конкретному GET-запросу.

Привязать service к Resty

```
type TaskService struct {
  priorities *resty.Client
}
```

Service должен получать интерфейс `PrioritySuggester`; Resty — деталь реализации инфраструктурного пакета.

Считать SetResult достаточной валидацией

JSON может быть синтаксически корректным, но содержать недопустимый priority. Всегда проверяйте данные внешнего сервиса по собственному контракту.

Дальше

Task Manager использует Resty для компактной реализации внешнего HTTP-клиента, сохраняя правила HTTP и независимость service. Далее настроим конфигурацию и жизненный цикл приложения: переменные окружения, проверку настроек, graceful shutdown и health-check.

Самопроверка

1. Что Resty использует под капотом для HTTP-запросов?

A. Gin router

B. Только WebSocket

C. database/sql

D. Стандартный net/http

2. Как связать запрос Resty с context операции?

A. Вызвать SetContext(ctx) у конкретного request

B. Заменить ctx на gin.Context

C. Передать ctx в SetHeader

D. Создать context.Background() в Suggest

3. Что делает SetResult(&hint) для успешного JSON-ответа?

A. Повторяет запрос при 500

B. Автоматически проверяет бизнес-диапазон priority

C. Закрывает HTTP-сервер

D. Декодирует ответ в переданный DTO при подходящем Content-Type

4. Почему после IsError нужно дополнительно проверить IsSuccess?

A. Иначе Resty удалит response body

B. IsError всегда возвращает true

C. 3xx не является ошибкой Resty, но не входит в наш контракт 2xx

D. IsSuccess проверяет context

5. Когда допустим retry из примера главы?

A. Для любого POST без ограничений

B. Для идемпотентного GET при явно определённой временной ошибке

C. Вместо проверки статуса

D. Только после panic

6. Почему TaskService не должен хранить *resty.Client?

A. Потому что Resty нельзя использовать в Go modules

B. Service должен зависеть от интерфейса, а Resty — остаться инфраструктурной деталью

C. Иначе JSON перестанет работать

D. Resty не поддерживает GET

7. Что нельзя логировать в hook внешнего HTTP-клиента?

A. Authorization, cookies и полное тело запроса или ответа

B. Длительность запроса

C. Статус ответа

D. HTTP-метод

8. Какой инструмент используют для тестирования Resty-клиента без внешней сети?

A. httptest.Server

B. Публичный API

C. time.Sleep

D. gin.Run на фиксированном порту

9. Что происходит с response body при обычном Resty Request.Execute?

A. Его нельзя читать

B. Он остаётся открытым навсегда

C. Resty управляет закрытием body, вручную defer Close не нужен

D. Его обязан закрыть service

10. Какой слой решает, применять ли подсказку приоритета при недоступном внешнем сервисе?

A. httptest.Server

B. Resty автоматически

C. Service по явно заданной бизнес-политике

D. JSON-тег PriorityHint

Ответы к самопроверке

1. Стандартный net/http

2. Вызвать SetContext(ctx) у конкретного request

3. Декодирует ответ в переданный DTO при подходящем Content-Type

4. 3xx не является ошибкой Resty, но не входит в наш контракт 2xx

5. Для идемпотентного GET при явно определённой временной ошибке

6. Service должен зависеть от интерфейса, а Resty — остаться инфраструктурной деталью

7. Authorization, cookies и полное тело запроса или ответа

8. httptest.Server

9. Resty управляет закрытием body, вручную defer Close не нужен

10. Service по явно заданной бизнес-политике

Задания: перенос клиента подсказок на Resty

Замените реализацию `internal/priorityclient` со стандартного `http.Client` на Resty. Внешний интерфейс `PrioritySuggester` и `TaskService` не меняйте: service не должен заметить, какую библиотеку использует инфраструктурный клиент.

Задание 1. Подключите Resty и создайте клиент

Добавьте зависимость `github.com/go-resty/resty/v2`. В `priorityclient.New` создайте один `resty.Client` с base URL, timeout три секунды и общими заголовками `Accept: application/json` и `User-Agent: task-manager-api/1.0`. При необходимости конструктор может принимать `http.RoundTripper` для настройки транспорта.

Критерии проверки:

- зависимость добавлена командой `go get`;
- Resty импортируется как `github.com/go-resty/resty/v2`;
- base URL не имеет завершающего `/`;
- timeout устанавливается через `SetTimeout`;
- общие headers не повторяются в каждом `Suggest`;
- `TaskService` не импортирует Resty.

Подсказка

```
client := resty.New().
  SetBaseURL(strings.TrimRight(baseURL, "/")).
  SetTimeout(3 * time.Second).
  SetHeader("Accept", "application/json")
```

Задание 2. Перенесите `Suggest`

Сохраните сигнатуру метода `Suggest(ctx, title)`. Через `client.R()` передайте context, query `title` и указатель на `PriorityHint` в `SetResult`, затем выполните GET `/v1/suggest`.

Критерии проверки:

- используется `SetContext(ctx)`;
- query задаётся через `SetQueryParam`;
- `SetResult` получает `*PriorityHint`;
- URL пути относительный к base URL;
- передаётся `GET /v1/suggest`;
- результат приоритета дополнительно проверяется от 1 до 5.

Подсказка

```
response, err := client.resty.R().
  SetContext(ctx).
  SetQueryParam("title", title).
  SetResult(&hint).
  Get("/v1/suggest")
```

Задание 3. Сохраните HTTP-контракт ошибок

Верните `*HTTPStatusError` для любого ответа, который не входит в диапазон `2xx`. Используйте `response.IsError()` и `response.IsSuccess()`. Ошибки Resty оборачивайте через `%w`.

Критерии проверки:

- `4xx` и `5xx` становятся `*HTTPStatusError`;
- `3xx` также не принимается как успешный ответ;
- transport error сохраняет исходную причину для `errors.Is`;
- `HTTPStatusError` содержит код и текст статуса;
- `PriorityHint` не возвращается при неуспешном статусе.

Подсказка

```
if response.IsError() || !response.IsSuccess() {
  return PriorityHint{}, &HTTPStatusError{
    StatusCode: response.StatusCode(),
    Status:     response.Status(),
  }
}
```

Задание 4. Добавьте безопасный hook логирования

Напишите функцию, которая получает `*resty.Client` и `*slog.Logger`, добавляет `OnAfterResponse` и записывает метод, URL, HTTP-статус и длительность. Не пишите токены, cookies и body.

Критерии проверки:

- используется `OnAfterResponse`;
- лог создаётся только после получения response;
- есть `method`, `url`, `status`, `duration`;
- hook возвращает `nil`;
- в лог не попадают `Authorization`, cookies и body;
- сетевые ошибки обрабатываются вне hook.

Подсказка

```
client.OnAfterResponse(func(_ *resty.Client, response *resty.Response) error {
  logger.Info("внешний HTTP-запрос завершён", "status", response.StatusCode())
  return nil
})
```

Задание 5. Добавьте retry только для временного GET

Создайте отдельный Resty-клиент или настройте отдельный request для получения подсказки. Ограничьте retry двумя повторами и разрешите его только для `GET` с ответом `503`. Добавьте краткое пояснение, почему это правило нельзя бездумно перенести на `POST /tasks`.

Критерии проверки:

- `SetRetryCount(2)`;
- есть ограниченные `SetRetryWaitTime` и `SetRetryMaxWaitTime`;
- condition проверяет метод GET и статус `503`;
- retry не включён глобально для запросов изменения данных;
- в документации или комментарии указана опасность повтора `POST`;
- context передаётся в каждый request.

Подсказка

```
AddRetryCondition(func(response *resty.Response, err error) bool {
  return err == nil &&
    response.Request.Method == http.MethodGet &&
    response.StatusCode() == http.StatusServiceUnavailable
})
```

Задание 6. Проверьте клиента через `httptest.Server`

Перенесите тесты из главы про `http.Client`: успешный ответ, `429`, некорректный JSON и отменённый context. Добавьте проверку, что `title` с символом `&` приходит одним query-параметром. Для retry добавьте отдельный handler с контролируемым счётчиком: он отвечает `503` первые две попытки и `200` на третью.

Критерии проверки:

- тесты не вызывают публичные сервисы;
- каждый `httptest.Server` закрывается;
- успешный тест проверяет метод, путь, заголовок и query;
- `429` проверяется через `errors.As`;
- отмена проверяется через `errors.Is(err, context.Canceled)` без `time.Sleep`;
- retry-тест детерминированно проверяет три попытки;
- `go test ./...` и `go test -race ./...` проходят.

Проверьте себя

Перед следующей главой убедитесь, что вы можете:

- объяснить, что Resty дополняет, а не заменяет `net/http`;
- передать context, query и DTO через Resty request;
- проверить только `2xx` как успешный контракт;
- безопасно логировать внешний вызов;
- ограничить retry идемпотентным запросом;
- сохранить независимость service от HTTP-библиотеки.

Рекомендации по коду: Resty

- Используйте Resty в инфраструктурном HTTP-клиенте, а не в service или доменной модели.
- Создавайте один настроенный `resty.Client` с base URL, timeout и безопасными общими заголовками.
- Передавайте `context.Context` в каждый request через `SetContext`.
- Принимайте только оговорённые успешные статусы: для этого API — `2xx`, а не просто «не ошибка Resty».
- Декодируйте внешний ответ в отдельный DTO и проверяйте его бизнес-допустимость.
- Не логируйте токены, cookies, headers авторизации и полные тела запросов или ответов.
- Включайте retry только для явно безопасных идемпотентных сценариев с лимитом, backoff, context и проверяемым условием.
- Для строгого контроля размера и структуры response body предпочитайте реализацию на стандартном `net/http`.
- Тестируйте Resty-клиент через `httptest.Server`, включая статусы, некорректный JSON и отмену context.

Чек-лист

Resty уменьшает повторяющийся transport-код, но client по-прежнему передаёт context, проверяет только разрешённые статусы и данные внешнего сервиса, не раскрывает секреты в логах, а service зависит только от интерфейса.

Конфигурация и жизненный цикл приложения

Цель урока

Научиться получать настройки Task Manager API из переменных окружения и локального `.env`, проверять конфигурацию при старте, запускать HTTP-сервер с таймаутами, корректно обрабатывать `SIGINT` и `SIGTERM`, выполнять graceful shutdown и закрывать зависимости приложения.

До этого момента значения вроде `:8080` и адреса внешнего API можно было писать прямо в примере. В настоящем приложении среда меняется: локальная разработка, тесты, Docker и production используют разные адреса, токены и пути. Такие значения становятся конфигурацией, а не частью исходного кода.

Двенадцатифакторный принцип

Конфигурация зависит от окружения и должна быть отделена от кода:

```
код:            как работает приложение
переменные env: где слушать порт, куда подключаться, какой timeout использовать
```

Не храните в Git секреты, production-адреса и пароли. Код читает значения из окружения, а платформа запуска — shell, Docker Compose, CI или hosting — передаёт их приложению.

Структура конфигурации

Соберём все настройки в один тип:

```
type Config struct {
  HTTPAddress        string
  ShutdownTimeout    time.Duration
  PriorityServiceURL string
}
```

Для текущей версии Task Manager нужны:

Переменная	Назначение	Значение по умолчанию
`HTTP_ADDRESS`	адрес HTTP-сервера	`:8080`
`SHUTDOWN_TIMEOUT`	максимум времени на graceful shutdown	`10s`
`PRIORITY_SERVICE_URL`	базовый URL внешнего сервиса приоритетов	обязательна

Позже сюда добавятся адрес PostgreSQL и учётные данные. Не создавайте глобальные переменные для отдельных значений: один `Config` легче валидировать и передавать при сборке зависимостей.

Переменные окружения

Стандартный пакет `os` предоставляет два основных способа чтения env:

```
value := os.Getenv("HTTP_ADDRESS")
```

`Getenv` возвращает пустую строку и для отсутствующей переменной, и для явно заданного пустого значения. Если нужно различать эти сценарии, используйте `LookupEnv`:

```
value, exists := os.LookupEnv("PRIORITY_SERVICE_URL")
if !exists || strings.TrimSpace(value) == "" {
  return Config{}, errors.New("PRIORITY_SERVICE_URL обязателен")
}
```

Для необязательных переменных удобно сделать helper:

```
func envOrDefault(key, fallback string) string {
  if value := strings.TrimSpace(os.Getenv(key)); value != "" {
    return value
  }

  return fallback
}
```

Пустое значение здесь означает «использовать fallback». Для обязательных секретов или URL используйте отдельную проверку, а не fallback, который скроет ошибку конфигурации.

Локальный `.env`

ОС и production-платформа не читают `.env` автоматически. Для локальной разработки подключим `github.com/joho/godotenv`:

```
go get github.com/joho/godotenv
```

В `main` загрузите файл до чтения конфигурации:

```
if err := godotenv.Load(); err != nil && !errors.Is(err, os.ErrNotExist) {
  return fmt.Errorf("загрузить .env: %w", err)
```

```
}
```

Если `.env` отсутствует, это не ошибка: переменные могут быть переданы из shell, Docker или CI. Если файл есть, но прочитать его не удалось, запуск нужно остановить.

Создайте в репозитории шаблон `.env.example`, но не добавляйте сам `.env` в Git:

```
# Локальный HTTP-сервер.
HTTP_ADDRESS=:8080

# Время на завершение активных HTTP-запросов после SIGINT/SIGTERM.
SHUTDOWN_TIMEOUT=10s

# Адрес тестового или локального внешнего сервиса.
PRIORITY_SERVICE_URL=http://localhost:8090
```

Добавьте в `.gitignore`:

```
.env
```

`godotenv.Load` не должен перезаписывать значения, которые уже передала среда. Поэтому shell и production-платформа имеют приоритет над локальным `.env`.

Разбор и валидация конфигурации

Значения env — всегда строки. Сразу преобразуйте и проверьте их при старте, а не внутри HTTP handler:

```
func Load() (Config, error) {
  shutdownTimeout, err := time.ParseDuration(envOrDefault("SHUTDOWN_TIMEOUT", "10s"))
  if err != nil {
    return Config{}, fmt.Errorf("SHUTDOWN_TIMEOUT: %w", err)
  }
  if shutdownTimeout <= 0 {
    return Config{}, errors.New("SHUTDOWN_TIMEOUT должен быть больше нуля")
  }

  priorityServiceURL := strings.TrimSpace(os.Getenv("PRIORITY_SERVICE_URL"))
  if priorityServiceURL == "" {
    return Config{}, errors.New("PRIORITY_SERVICE_URL обязателен")
  }
  endpoint, err := url.ParseRequestURI(priorityServiceURL)
  if err != nil || endpoint.Scheme == "" || endpoint.Host == "" {
```

```
    return Config{}, errors.New("PRIORITY_SERVICE_URL должен быть абсолютным URL")  }
return Config{    HTTPAddress:        envOrDefault("HTTP_ADDRESS", ":8080"),
ShutdownTimeout:    shutdownTimeout,    PriorityServiceURL:
strings.TrimRight(priorityServiceURL, "/"),  }, nil}
```

Проверка `url.ParseRequestURI` и полей `Scheme`/`Host` предотвращает ошибку, когда приложение запускается с `priority-service:8090` без `http://`. Не логируйте весь `Config` бездумно: будущие поля могут содержать пароль или token.

Тестирование `Load`

Конфигурация — обычная логика, поэтому её надо тестировать. Не меняйте настоящий env процесса в параллельных тестах. Передайте функцию чтения как зависимость или используйте `t.Setenv` в непараллельных тестах:

```
func TestLoadInvalidShutdownTimeout(t *testing.T) {
  t.Setenv("PRIORITY_SERVICE_URL", "https://priority.example.com")
  t.Setenv("SHUTDOWN_TIMEOUT", "пять секунд")

  _, err := Load()
  if err == nil {
    t.Fatal("Load не вернул ошибку")
  }
}
```

`t.Setenv` сам восстановит значение после теста. Не вызывайте `t.Parallel` в тестах, которые используют `t.Setenv`: переменные окружения принадлежат всему процессу.

Сборка зависимостей в `main`

`main` отвечает за порядок старта: загрузить локальный `.env`, прочитать config, собрать logger и зависимости, затем запустить сервер.

```
func run() error {
  if err := godotenv.Load(); err != nil && !errors.Is(err, os.ErrNotExist) {
    return fmt.Errorf("загрузить .env: %w", err)
  }

  config, err := config.Load()
  if err != nil {
    return fmt.Errorf("конфигурация: %w", err)
  }

  logger := slog.New(slog.NewJSONHandler(os.Stdout, nil))
```

```
  priorityClient := priorityclient.New(config.PriorityServiceURL, nil) store :=
store.NewTaskStore(logger) service := taskservice.New(store, priorityClient)  api :=
ginapi.New(service, logger)  server := newServer(config.HTTPAddress, api.Router())
return serve(server, config.ShutdownTimeout, logger)}func main() { if err := run();
err != nil {    slog.Error("приложение завершилось с ошибкой", "err", err)
os.Exit(1) }}
```

`main` — подходящее место для `os.Exit`, потому что это граница процесса. В library-коде возвращайте ошибки, а не завершайте процесс самостоятельно: так код можно тестировать и переиспользовать.

HTTP server и защитные таймауты

Не используйте `router.Run()` в production-сборке: он не даёт централизованно задать `http.Server`. Создайте сервер явно:

```
func newServer(address string, handler http.Handler) *http.Server {
  return &http.Server{
    Addr:              address,
    Handler:           handler,
    ReadHeaderTimeout: 5 * time.Second,
    ReadTimeout:       15 * time.Second,
    WriteTimeout:      15 * time.Second,
    IdleTimeout:       60 * time.Second,
  }
}
```

Таймаут	Защищает от
`ReadHeaderTimeout`	очень медленной передачи заголовков
`ReadTimeout`	слишком долгого чтения всего запроса
`WriteTimeout`	слишком долгой записи ответа
`IdleTimeout`	бесконечного удержания неактивного keep-alive соединения

Значения зависят от продукта: например, для долгого скачивания или streaming нужны другие правила. Эти безопасные стартовые значения не являются универсальной нормой.

Сигналы процесса

При Ctrl+C терминал обычно отправляет процессу `SIGINT`; Docker и оркестраторы обычно используют `SIGTERM`. В обоих случаях приложение должно перестать принимать новые запросы и завершить активные в ограниченное время.

`signal.NotifyContext` создаёт context, который отменяется при нужном сигнале:

```
rootCtx, stop := signal.NotifyContext(context.Background(), os.Interrupt,
syscall.SIGTERM)
defer stop()

<-rootCtx.Done()
```

Вызов `stop` важен: он освобождает ресурсы, связанные с получением сигналов, и возвращает стандартное поведение сигнала после выхода.

Graceful shutdown

`server.Shutdown(ctx)`:

1. закрывает listener — новые соединения и запросы больше не принимаются;
2. ждёт завершения активных handler;
3. возвращает ошибку, если context завершения истёк.

Не используйте `server.Close()` как обычный путь остановки: он немедленно обрывает активные соединения. `Close` может быть аварийным fallback после истечения timeout, но не заменяет `Shutdown`.

Запустим `ListenAndServe` в goroutine и дождёмся сигнала:

```
func serve(server *http.Server, shutdownTimeout time.Duration, logger *slog.Logger)
error {
  serverErrors := make(chan error, 1)
  go func() {
    serverErrors <- server.ListenAndServe()
  }()

  rootCtx, stop := signal.NotifyContext(context.Background(), os.Interrupt,
syscall.SIGTERM)
  defer stop()

  select {
  case err := <-serverErrors:
    if !errors.Is(err, http.ErrServerClosed) {
      return fmt.Errorf("HTTP-сервер: %w", err)
    }
    return nil
  case <-rootCtx.Done():
    logger.Info("начато корректное завершение", "signal", rootCtx.Err())
  }
```

```
  shutdownCtx, cancel := context.WithTimeout(context.Background(), shutdownTimeout)
defer cancel()  if err := server.Shutdown(shutdownCtx); err != nil {    return
fmt.Errorf("корректно завершить HTTP-сервер: %w", err)  } if err := <-serverErrors;
err != nil && !errors.Is(err, http.ErrServerClosed) {     return fmt.Errorf("HTTP-
сервер после shutdown: %w", err)  } logger.Info("HTTP-сервер остановлен") return nil}
```

Канал `serverErrors` имеет буфер `1`, поэтому goroutine может сообщить о раннем завершении сервера, даже если `serve` в этот момент ещё создаёт context сигналов. Контекст shutdown намеренно создаётся от `context.Background()`: root context уже отменён сигналом, а серверу нужно получить отдельное время, чтобы завершить текущую работу.

Закрытие зависимостей

HTTP-сервер — не единственный ресурс. Файлы, базы данных и сетевые клиенты тоже могут требовать закрытия. Опишем минимальный интерфейс:

```
type Closer interface {
  Close() error
}

func closeAll(closers ...Closer) error {
  var closeErr error
  for _, closer := range closers {
    if closer == nil {
      continue
    }

    if err := closer.Close(); err != nil {
      closeErr = errors.Join(closeErr, err)
    }
  }
  return closeErr
}
```

В следующей главе `*sql.DB` станет одной из таких зависимостей. Вызывайте `closeAll` после `server.Shutdown`, чтобы не закрыть ресурс, пока активный handler ещё выполняет запрос.

```
if err := closeAll(database); err != nil {
  return fmt.Errorf("закрыть зависимости: %w", err)
}
```

`errors.Join` сохраняет все ошибки закрытия. Не вызывайте `os.Exit` внутри `defer`: deferred-функции не выполнятся, а ресурсы могут остаться незакрытыми.

Health-check

Базовый endpoint уже существует:

```
router.GET("/healthz", api.health)
```

Пока у Task Manager нет базы данных, он может отвечать:

```
{"status":"ok"}
```

Health-check должен быть быстрым, не требовать авторизации и не выполнять тяжёлую работу. В Docker и CI он подтвердит, что HTTP-сервер начал отвечать. После подключения PostgreSQL отдельно определим readiness-check, который проверяет критические зависимости; не превращайте простой liveness endpoint в медленный запрос к каждой внешней системе.

Проверка вручную

Создайте `.env` на основе `.env.example`, затем запустите приложение:

```
go run .
```

В другом терминале:

```
curl -i http://localhost:8080/healthz
```

Вернитесь в первый терминал и нажмите Ctrl+C. В логах должны появиться начало и завершение graceful shutdown. Сервер больше не принимает новые запросы, а процесс завершится с кодом `0`, если ошибок не было.

Частые ошибки

Читать env внутри handler

```
func (api *API) createTask(c *gin.Context) {
  baseURL := os.Getenv("PRIORITY_SERVICE_URL")
  // ...
}
```

Это повторяет работу на каждом request, скрывает некорректную конфигурацию до первого вызова и усложняет тесты. Читайте config один раз при старте.

Коммитить `.env`

`.env` часто содержит токены и пароли. Коммитьте только `.env.example` с безопасными примерами и добавьте настоящий `.env` в `.gitignore`.

Игнорировать ошибку `ListenAndServe`

Если порт занят, сервер не запустится. Проверяйте ошибку и отличайте ожидаемый `http.ErrServerClosed` после shutdown от настоящей ошибки старта.

Использовать `context.Background()` для каждого вызова

В `main` background — корень жизненного цикла. Но внутри handler передавайте `r.Context()` или `c.Request.Context()`, иначе отмена клиента не остановит внешние запросы и SQL-операции.

Закрыть базу до HTTP-сервера

Активный handler может ещё работать с БД. Сначала вызовите `server.Shutdown`, затем закрывайте зависимости.

Дальше

Task Manager умеет получать проверенную конфигурацию и корректно останавливаться. Далее упакуем приложение и локальный PostgreSQL в Docker Compose, чтобы следующие SQL-упражнения работали в воспроизводимом окружении.

Самопроверка

1. Почему настройки окружения не стоит хранить в исходном коде?

A. Иначе JSON перестанет работать

B. Они отличаются между средами и могут содержать секреты

C. Их нельзя передать Docker

D. Go не поддерживает константы

2. Чем os.LookupEnv отличается от os.Getenv?

A. LookupEnv автоматически читает .env

B. Getenv возвращает time.Duration

C. LookupEnv сообщает, была ли переменная задана, отдельно от её значения

D. Ничем

3. Зачем нужен .env.example?

A. Чтобы показать требуемые переменные без публикации настоящих секретов

B. Чтобы Go автоматически загрузил env

C. Чтобы заменить production secrets

D. Чтобы не писать тесты конфигурации

4. Когда нужно проверять обязательный URL внешнего сервиса?

A. Только в браузере

B. Никогда: URL всегда корректен

C. Только после первого HTTP-запроса пользователя

D. При старте приложения, до запуска HTTP-сервера

5. Какие сигналы обычно запускают graceful shutdown?

A. Только HTTP 404

B. Только SIGKILL

C. SIGINT и SIGTERM

D. Сигнал JSON

6. Что делает server.Shutdown(ctx)?

A. Немедленно обрывает все соединения без ожидания

B. Перезагружает HTTP-сервер

C. Перестаёт принимать новые запросы и ждёт активные до завершения ctx

D. Удаляет Docker-контейнер

7. Почему shutdown context создают от context.Background(), а не от уже отменённого signal context?

A. Чтобы пропустить закрытие сервера

B. Чтобы отмена сигнала никогда не работала

C. Чтобы JSON стал валидным

D. Чтобы дать активным handler отдельное ограниченное время на завершение

8. В каком порядке закрывать сервер и базу данных?

A. Базу закрывать не нужно

B. Сначала базу, затем начать принимать новые запросы

C. Сначала server.Shutdown, затем закрыть базу и остальные зависимости

D. Одновременно через os.Exit

9. Как безопасно менять env в непараллельном unit-тесте?

A. Через t.Setenv

B. Через fmt.Println

C. Через commit .env в репозиторий

D. Через time.Sleep

10. Каким должен быть базовый health-check?

A. Запускающим миграции БД

B. Быстрым, без авторизации и тяжёлой работы

C. Доступным только через POST

D. Обязательным запросом ко всем внешним API

Ответы к самопроверке

1. Они отличаются между средами и могут содержать секреты

2. LookupEnv сообщает, была ли переменная задана, отдельно от её значения

3. Чтобы показать требуемые переменные без публикации настоящих секретов

4. При старте приложения, до запуска HTTP-сервера

5. SIGINT и SIGTERM

6. Перестаёт принимать новые запросы и ждёт активные до завершения ctx

7. Чтобы дать активным handler отдельное ограниченное время на завершение

8. Сначала server.Shutdown, затем закрыть базу и остальные зависимости

9. Через t.Setenv

10. Быстрым, без авторизации и тяжёлой работы

Задания: управляемый запуск Task Manager API

Сделайте запуск Task Manager API управляемым: настройки поступают из env и локального `.env`, сервер получает защитные таймауты, а при Ctrl+C или `SIGTERM` корректно завершает активную работу.

Задание 1. Создайте `Config` и `.env.example`

Создайте пакет `internal/config` со структурой `Config` для `HTTP_ADDRESS`, `SHUTDOWN_TIMEOUT` и `PRIORITY_SERVICE_URL`. Добавьте `.env.example` с безопасными локальными значениями и добавьте `.env` в `.gitignore`.

Критерии проверки:

- `Config` не содержит глобальных переменных;
- `.env.example` не содержит настоящих токенов и паролей;
- `.env` игнорируется Git;
- в примере есть все три переменные;
- имена переменных документированы рядом с конфигурацией или в README.

Подсказка

```
HTTP_ADDRESS=:8080
SHUTDOWN_TIMEOUT=10s
PRIORITY_SERVICE_URL=http://localhost:8090
```

Задание 2. Загрузите и проверьте env

Реализуйте `config.Load()`. `HTTP_ADDRESS` должен иметь fallback `:8080`, `SHUTDOWN_TIMEOUT` — `10s`; `PRIORITY_SERVICE_URL` обязателен и должен быть абсолютным HTTP(S)-URL. Ошибки разбора должны объяснять имя переменной.

Критерии проверки:

- длительность разбирается через `time.ParseDuration`;
- нулевая и отрицательная длительность отклоняются;
- обязательный URL не может быть пустым;
- URL содержит scheme и host;
- trailing slash URL нормализуется;
- неправильная конфигурация не позволяет запустить сервер.

Подсказка

```
endpoint, err := url.ParseRequestURI(priorityServiceURL)
if err != nil || endpoint.Scheme == "" || endpoint.Host == "" {
  return Config{}, errors.New("PRIORITY_SERVICE_URL должен быть абсолютным URL")
}
```

Фрагмент решения

```
shutdownTimeout, err := time.ParseDuration(envOrDefault("SHUTDOWN_TIMEOUT", "10s"))
if err != nil {
  return Config{}, fmt.Errorf("SHUTDOWN_TIMEOUT: %w", err)
}
if shutdownTimeout <= 0 {
  return Config{}, errors.New("SHUTDOWN_TIMEOUT должен быть больше нуля")
}
```

Задание 3. Подключите локальный `.env`

Добавьте `github.com/joho/godotenv`. В `run` вызовите `godotenv.Load()` перед `config.Load`. Отсутствующий `.env` не должен мешать запуску, другие ошибки чтения — должны.

Критерии проверки:

- зависимость добавлена через `go get github.com/joho/godotenv`;
- `.env` загружается до чтения env;
- `os.ErrNotExist` игнорируется;
- другие ошибки оборачиваются и возвращаются;
- production env может переопределить локальное значение `.env`;
- `.env` не коммитится.

Подсказка

```
if err := godotenv.Load(); err != nil && !errors.Is(err, os.ErrNotExist) {
  return fmt.Errorf("загрузить .env: %w", err)
}
```

Задание 4. Настройте HTTP-сервер

Вынесите создание `http.Server` в `newServer(address, handler)`. Настройте `ReadHeaderTimeout`, `ReadTimeout`, `WriteTimeout` и `IdleTimeout`. Gin router передавайте как `http.Handler`.

Критерии проверки:

- используется `http.Server`, а не `router.Run()`;
- адрес приходит из `Config`;
- настроены все четыре таймаута;
- `ReadHeaderTimeout` ненулевой;
- server получает Gin router через `Handler`;
- стартовая ошибка `ListenAndServe` не игнорируется.

Задание 5. Добавьте graceful shutdown

Напишите `serve(server, shutdownTimeout, logger)`. Он запускает `ListenAndServe` в goroutine, ждёт `SIGINT` или `SIGTERM` через `signal.NotifyContext`, затем вызывает `server.Shutdown` с новым timeout context от `context.Background()`.

Критерии проверки:

- обработаны `os.Interrupt` и `syscall.SIGTERM`;
- `stop()` вызывается через `defer`;
- канал ошибок сервера буферизирован размером `1`;
- `http.ErrServerClosed` не считается ошибкой;
- timeout context создан от `context.Background()`;
- успешный shutdown логируется;
- `server.Close()` не используется вместо обычного shutdown.

Подсказка

```
shutdownCtx, cancel := context.WithTimeout(context.Background(), shutdownTimeout)
defer cancel()

if err := server.Shutdown(shutdownCtx); err != nil {
  return fmt.Errorf("корректно завершить HTTP-сервер: %w", err)
}
```

Задание 6. Закройте зависимости и протестируйте конфигурацию

Добавьте `closeAll(closers ...Closer)` с `errors.Join`. Закрывайте зависимости после `server.Shutdown`. Напишите table-driven тесты `config.Load` для default-значений, неверной длительности, пустого URL и URL без scheme. Используйте `t.Setenv` и не запускайте эти тесты параллельно.

Критерии проверки:

- `closeAll` пытается закрыть все зависимости, даже если одна вернула ошибку;
- ошибки закрытия объединяются через `errors.Join`;
- nil dependency не вызывает panic;
- тесты конфигурации независимы;
- тесты не используют настоящий `.env`, сеть или `time.Sleep`;
- `go test ./...` и `go test -race ./...` проходят.

Подсказка

```
func TestLoadInvalidShutdownTimeout(t *testing.T) {
  t.Setenv("PRIORITY_SERVICE_URL", "https://priority.example.com")
  t.Setenv("SHUTDOWN_TIMEOUT", "пять секунд")

  _, err := Load()
  if err == nil {
    t.Fatal("Load не вернул ошибку")
  }
}
```

Проверьте себя

Перед Docker-главой убедитесь, что вы можете:

- передать настройки через env, не храня секреты в коде;
- дать локальному `.env` меньший приоритет, чем production env;
- отклонить неверную конфигурацию до старта HTTP-сервера;
- объяснить назначение таймаутов `http.Server`;
- корректно завершить приложение по `SIGINT` и `SIGTERM`;
- закрыть HTTP-сервер перед базой данных и другими зависимостями.

Рекомендации по коду: конфигурация и жизненный цикл

- Читайте и проверяйте конфигурацию один раз при старте; не обращайтесь к env из handlers и service.
- Обязательные значения валидируйте явно, а fallback используйте только там, где он безопасен и документирован.
- Храните локальные значения в `.env`, коммитьте только `.env.example`, а `.env` всегда добавляйте в `.gitignore`.
- Не выводите целиком config в логи: будущие поля могут содержать секреты.
- Используйте явный `http.Server` и выбирайте read/write/idle timeout по характеру API.
- Обрабатывайте `SIGINT` и `SIGTERM` через `signal.NotifyContext`; вызывайте `stop` для освобождения ресурсов.
- Для shutdown создавайте отдельный ограниченный context от `context.Background()`.
- Сначала выполняйте `server.Shutdown`, затем закрывайте базу данных, файлы и остальные зависимости.
- Не вызывайте `os.Exit` за пределами `main` и не используйте его в defer.
- Тестируйте разбор конфигурации через `t.Setenv` без параллельных тестов и реального `.env`.

Чек-лист

Приложение не запускается с неверной конфигурацией, секреты не попадают в Git и логи, HTTP-сервер имеет защитные таймауты, а `SIGINT` / `SIGTERM` останавливают приём новых запросов и дают активной работе ограниченное время на завершение.

Docker и локальное окружение

Цель урока

Научиться упаковывать Task Manager API в Docker-образ, запускать API и PostgreSQL через Docker Compose, сохранять данные базы в volume, проверять её готовность через health-check и подготавливать повторяемое локальное окружение для следующей главы про SQL.

До этой главы для запуска проекта нужны были Go, переменные окружения и локальные зависимости. Docker описывает окружение в файлах репозитория: другой разработчик может получить одинаковые API и PostgreSQL без ручной установки конкретных версий в систему.

> **Что будет сделано сейчас**
>
> PostgreSQL уже запускается в Compose, но Task Manager пока продолжает использовать in-memory `TaskStore`. Подключение через `database/sql`, таблицы и миграции появятся в следующей главе. Это разделение помогает отлаживать инфраструктуру и SQL отдельно.

Основные понятия

Термин	Значение
Image	неизменяемый шаблон файловой системы и команды запуска
Container	запущенный экземпляр image
Dockerfile	рецепт сборки image
Volume	хранилище данных вне жизненного цикла container
Port mapping	публикация порта container на компьютере разработчика
Docker Compose	описание нескольких связанных сервисов в одном YAML-файле

Образ не равен container: один image можно запускать много раз. Удаление container не обязательно удаляет image, а удаление container PostgreSQL без volume удалит и его данные.

Учебный проект главы

В репозитории создан самостоятельный минимальный пример:

```
examples/32/task-manager-api/
├── cmd/api/main.go
├── Dockerfile
├── .dockerignore
├── compose.yaml
```

```
├── .env.example└── go.mod
```

Это намеренно небольшой API с `GET /healthz`. Он позволяет проверить Docker-контракт до того, как в следующей главе появится настоящий PostgreSQL-репозиторий. Перед командами перейдите в эту директорию:

```
cd examples/32/task-manager-api
```

Проверка Docker

Проверьте, что Docker Desktop или Docker Engine запущен:

```
docker version
docker compose version
```

Если клиент Docker установлен, но демон недоступен, команды сборки покажут ошибку подключения. На macOS и Windows обычно нужно открыть Docker Desktop и дождаться статуса **Running**.

Multi-stage Dockerfile для Go

Файл `examples/32/task-manager-api/Dockerfile` использует две стадии:

```
# syntax=docker/dockerfile:1

FROM golang:1.24-alpine AS build

WORKDIR /src

COPY go.mod ./
RUN go mod download

COPY . .
RUN CGO_ENABLED=0 GOOS=linux go build -trimpath -ldflags="-s -w" -o /out/task-
manager-api ./cmd/api

FROM alpine:3.21

RUN addgroup -S app && adduser -S -G app app
USER app

COPY --from=build /out/task-manager-api /task-manager-api
```

```
EXPOSE 8080ENTRYPOINT ["/task-manager-api"]
```

Первая стадия содержит компилятор Go и зависимости, вторая — только скомпилированный бинарный файл и минимальную ОС. Поэтому конечный image не содержит исходный код, Go toolchain и кэш модулей.

Порядок `COPY` и кэш

Сначала копируются `go.mod` / `go.sum`, затем выполняется `go mod download`. Docker может использовать этот слой повторно, если зависимости не изменились. Только после этого копируется исходный код.

Когда в проекте появится `go.sum`, добавьте его в команду:

```
COPY go.mod go.sum ./
RUN go mod download
```

Почему `CGO_ENABLED=0`

Текущий пример не использует C-зависимости. Статический бинарник проще запустить в компактном final image. Не добавляйте эту настройку механически в любой проект: драйверы, библиотеки с CGO или системные сертификаты могут требовать другой образ и конфигурацию.

Непривилегированный пользователь

`USER app` не позволяет приложению работать от root внутри container. Это не заменяет другие меры безопасности, но снижает последствия ошибки в приложении.

`EXPOSE 8080` документирует порт приложения внутри image. Сам по себе `EXPOSE` не открывает порт на компьютере: публикацию задаёт Compose через `ports`.

`.dockerignore`

При `docker build .` Docker отправляет build context демону. В `.dockerignore` перечислено то, что не должно попасть в context:

```
.git
.env
.env.*
*.md
bin
tmp
coverage.out
```

Это ускоряет сборку и не даёт случайно скопировать локальные секреты в image. `.dockerignore` не заменяет `.gitignore`: первый управляет контекстом Docker, второй — индексом Git.

Сборка и запуск одного container

Из `examples/32/task-manager-api`:

```
docker build -t task-manager-api:local .

docker run --rm -p 8080:8080 task-manager-api:local
```

В другом терминале проверьте API:

```
curl -i http://localhost:8080/healthz
```

Ожидаемый результат:

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8

{"status":"ok"}
```

Остановить foreground-container можно через Ctrl+C. Флаг `--rm` удалит container после остановки, но image останется для следующих запусков.

Docker Compose

Task Manager состоит из нескольких сервисов. `examples/32/task-manager-api/compose.yaml` описывает API, PostgreSQL и volume:

```
services:
  api:
    build:
      context: .
      dockerfile: Dockerfile
    environment:
      DATABASE_URL: postgresql://${POSTGRES_USER:-task_manager}:${POSTGRES_PASSWORD:-
local-development-only}@postgres:5432/${POSTGRES_DB:-task_manager}?sslmode=disable
    ports:
      - "${API_PORT:-8080}:8080"
    depends_on:
      postgres:
```

```
        condition: service_healthy  postgres:    image: postgres:17-alpine
environment:      POSTGRES_DB: ${POSTGRES_DB:-task_manager}      POSTGRES_USER:
${POSTGRES_USER:-task_manager}      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:-local-
development-only}    volumes:      - postgres-data:/var/lib/postgresql/datavolumes:
postgres-data:
```

Compose создаёт внутреннюю сеть. Внутри неё API обращается к БД по hostname **`postgres`** — имени сервиса, а не `localhost`:

```
postgresql://task_manager:...@postgres:5432/task_manager?sslmode=disable
```

`localhost` внутри API-container означает сам API-container, а не компьютер разработчика и не PostgreSQL. Это одна из самых частых ошибок при переходе на Docker.

Локальные переменные Compose

Скопируйте шаблон:

```
cp .env.example .env
```

Файл `.env` служит Compose для подстановки значений `${...}`. Он содержит только учебные локальные значения и не должен попадать в Git. Для Docker Compose shell-переменные имеют более высокий приоритет, поэтому можно временно переопределить порт:

```
API_PORT=8081 docker compose up --build
```

В Docker и production передавайте реальные secrets через безопасный механизм платформы. Пароль `local-development-only` в примере существует исключительно для локальной учебной среды.

PostgreSQL, volume и health-check

PostgreSQL хранит каталог данных в `/var/lib/postgresql/data`. Named volume:

```
volumes:
  - postgres-data:/var/lib/postgresql/data
```

переживает пересоздание container. Это означает:

```
docker compose down
docker compose up -d
```

не удалит данные базы.

Полное удаление окружения **вместе с данными**:

```
docker compose down -v
```

Используйте `-v` осознанно: для локальной разработки это удобный способ начать с чистой БД, но команда удаляет volume без возможности восстановления.

Health-check ждёт, пока PostgreSQL начнёт принимать подключения:

```
healthcheck:
  test: ["CMD-SHELL", "pg_isready -U $$POSTGRES_USER -d $$POSTGRES_DB"]
  interval: 5s
  timeout: 3s
  retries: 10
```

Двойной `$` нужен, чтобы Compose передал `$POSTGRES_USER` внутрь container, а не подставил переменную на компьютере разработчика. `depends_on.condition: service_healthy` запускает API после успешного health-check БД. Это не заменяет корректную обработку ошибок подключения в самом приложении: в production зависимость может стать недоступной уже после запуска.

Порт PostgreSQL только для разработки

В Compose задано:

```
ports:
  - "127.0.0.1:${POSTGRES_PORT:-5432}:5432"
```

Это делает PostgreSQL доступным с вашего компьютера на `localhost:5432`, например для DBeaver в следующей главе, но не публикует его на внешние сетевые интерфейсы. API внутри Compose всё равно использует `postgres:5432` и не зависит от этой публикации порта.

В production БД обычно не публикуют наружу через `ports`; к ней обращаются только сервисы во внутренней сети.

Основные команды Compose

Запустить или пересобрать окружение в foreground:

```
docker compose up --build
```

Запустить в фоне:

```
docker compose up --build -d
```

Проверить статус и health:

```
docker compose ps
```

Посмотреть логи всех сервисов или только API:

```
docker compose logs -f
docker compose logs -f api
```

Проверить API:

```
curl -i http://localhost:8080/healthz
```

Выполнить `psql` внутри PostgreSQL-container:

```
docker compose exec postgres psql -U task_manager -d task_manager
```

Остановить и удалить containers и сеть, оставив volume:

```
docker compose down
```

Подготовка к SQL-главе

После `docker compose up -d` инфраструктура следующей главы уже известна:

Настройка приложения	Внутри Compose	На компьютере разработчика
Host	`postgres`	`localhost`
Port	`5432`	`${POSTGRES_PORT:-5432}`
Database	`task_manager`	`task_manager`
User	`task_manager`	`task_manager`

В следующей главе добавим PostgreSQL-драйвер, `database/sql`, `PingContext`, миграции и репозиторий. DBeaver будет подключаться к `localhost`, а API-container — к сервису `postgres`.

Проверка Docker-файлов без запуска

До запуска Docker можно проверить развёрнутый Compose-конфиг:

```
docker compose config
```

Команда подставит значения `.env` и defaults, покажет итоговые порты, volumes и переменные. Это помогает найти ошибку YAML или переменной до создания containers.

Частые ошибки

Писать `localhost` в DATABASE_URL API

```
postgresql://task_manager:password@localhost:5432/task_manager
```

Для API-container это неверно: `localhost` указывает на него самого. Используйте имя Compose-сервиса `postgres`.

Хранить данные только в файловой системе container

Без named volume данные исчезнут после пересоздания PostgreSQL-container. Используйте volume для каталога данных базы.

Добавить `.env` в image

```
COPY . .
```

без `.dockerignore` может скопировать локальные секреты в build context и image layer. Добавьте `.env` и `.env.*` в `.dockerignore`.

Считать `depends_on` полной гарантией

Без `condition: service_healthy` обычный `depends_on` задаёт только порядок старта. Даже с health-check приложение обязано обрабатывать ошибки БД: она может перезапуститься позже.

Использовать `docker compose down -v` без предупреждения

Команда полезна, когда нужна чистая база, но удаляет named volumes. Для обычной остановки используйте `docker compose down`.

Дальше

Локальное окружение теперь воспроизводимо: API и PostgreSQL поднимаются одной командой, а данные базы сохраняются в volume. Далее подключим API к этой базе через `database/sql`, создадим таблицы и проверим их в DBeaver.

Самопроверка

1. Чем container отличается от image?

A. Container существует только для PostgreSQL

B. Image — шаблон, container — его запущенный экземпляр

C. Image всегда хранит данные базы

D. Это два названия одного файла

2. Зачем в Go Dockerfile используют multi-stage build?

A. Чтобы оставить компилятор в build-стадии, а в final image скопировать только бинарник

B. Чтобы не использовать go.mod

C. Чтобы открыть PostgreSQL в интернете

D. Чтобы каждый запрос запускал две копии API

3. Что делает .dockerignore?

A. Останавливает container

B. Удаляет файлы из Git-репозитория

C. Исключает файлы из Docker build context

D. Шифрует secrets

4. Как API-container обращается к PostgreSQL внутри Compose?

A. По localhost:5432

B. По hostname postgres и порту 5432

C. Через порт API 8080

D. По IP Docker Desktop из браузера

5. Для чего нужен named volume postgres-data?

A. Чтобы пересылать HTTP-логи в браузер

B. Чтобы данные PostgreSQL переживали пересоздание container

C. Чтобы открыть порт 5432 наружу

D. Чтобы заменить Dockerfile

6. Что удалит docker compose down -v?

A. Только source code проекта

B. Containers, сеть и named volumes, включая данные локальной БД

C. Только логи API

D. Ничего: это команда просмотра

7. Почему в health-check PostgreSQL используют $$POSTGRES_USER?

A. Чтобы Compose передал переменную внутрь container, а не подставил её на хосте

B. Чтобы пароль стал сильнее

C. Чтобы Docker удалил volume

D. Это синтаксис Go

8. Для чего PostgreSQL-порт привязан к 127.0.0.1?

A. Чтобы API не мог подключиться к БД

B. Чтобы DBeaver на этом компьютере мог подключиться, но БД не слушала внешнюю сеть

C. Чтобы volume не создавался

D. Чтобы включить production mode

9. Какая команда показывает итоговую Compose-конфигурацию после подстановки env?

A. docker compose down -v

B. go mod tidy

C. curl /healthz

D. docker compose config

10. Подключается ли API к PostgreSQL в этой главе?

A. Да, здесь уже реализован полный CRUD через database/sql

B. Нет, PostgreSQL готовит инфраструктуру для следующей SQL-главы

C. Да, но только через DBeaver

D. PostgreSQL вообще не запускается

Ответы к самопроверке

1. Image — шаблон, container — его запущенный экземпляр

2. Чтобы оставить компилятор в build-стадии, а в final image скопировать только бинарник

3. Исключает файлы из Docker build context

4. По hostname postgres и порту 5432

5. Чтобы данные PostgreSQL переживали пересоздание container

6. Containers, сеть и named volumes, включая данные локальной БД

7. Чтобы Compose передал переменную внутрь container, а не подставил её на хосте

8. Чтобы DBeaver на этом компьютере мог подключиться, но БД не слушала внешнюю сеть

9. docker compose config

10. Нет, PostgreSQL готовит инфраструктуру для следующей SQL-главы

Задания: Docker-окружение Task Manager API

Используйте учебный проект из `examples/32/task-manager-api`. Сначала проверьте его локально через Go, затем соберите image и поднимите API вместе с PostgreSQL. База в этой главе — инфраструктура: не добавляйте `database/sql` до следующей главы.

Задание 1. Проверьте минимальный API без Docker

Перейдите в каталог примера, запустите API через Go и проверьте `GET /healthz` с `curl`.

```
cd examples/32/task-manager-api
go run ./cmd/api
```

Критерии проверки:

- `go vet ./...` проходит;
- API запускается на `:8080`;
- `/healthz` возвращает `200` и JSON `{"status":"ok"}`;
- вы можете остановить процесс через Ctrl+C;
- в этом шаге PostgreSQL не требуется.

Задание 2. Разберите и соберите Dockerfile

Откройте `Dockerfile`, найдите build и final стадии. Соберите image:

```
docker build -t task-manager-api:local .
```

Запустите его отдельно:

```
docker run --rm -p 8080:8080 task-manager-api:local
```

В другом терминале выполните `curl -i http://localhost:8080/healthz`.

Критерии проверки:

- Dockerfile содержит минимум две стадии `FROM`;
- бинарник копируется в final stage через `COPY --from=build`;
- final stage не запускается от root;
- API отвечает из container;
- container удаляется после остановки благодаря `--rm`.

Подсказка

Проверить размер и параметры image можно так:

```
docker image inspect task-manager-api:local
docker image ls task-manager-api
```

Задание 3. Проверьте `.dockerignore`

Создайте локальный `.env` из шаблона и убедитесь, что `.env` и `.env.*` есть в `.dockerignore`. Объясните разницу между `.gitignore` и `.dockerignore`.

```
cp .env.example .env
```

Критерии проверки:

- `.env` не отображается в `git status`;
- `.env` и `.env.*` перечислены в `.dockerignore`;
- `.git` не отправляется в build context;
- вы не добавляете локальный пароль в Dockerfile;
- изменения source code всё ещё попадают в image после `docker build`.

Задание 4. Поднимите API и PostgreSQL через Compose

Проверьте развёрнутую конфигурацию и запустите сервисы в фоне:

```
docker compose config
docker compose up --build -d
docker compose ps
```

Проверьте API и логи:

```
curl -i http://localhost:8080/healthz
docker compose logs -f api
```

Критерии проверки:

- сервисы `api` и `postgres` имеют статус running;
- PostgreSQL становится `healthy`;
- API стартует после health-check PostgreSQL;
- API доступен на порту из `API_PORT`;
- `DATABASE_URL` использует hostname `postgres`, а не `localhost`;
- API пока не обязан открывать подключение к БД.

Задание 5. Проверьте PostgreSQL и сохранность volume

Откройте `psql` в database-container:

```
docker compose exec postgres psql -U task_manager -d task_manager
```

В `psql` выполните:

```
CREATE TABLE docker_check (id integer PRIMARY KEY);
INSERT INTO docker_check (id) VALUES (1);
SELECT * FROM docker_check;
```

Выйдите через `\q`, остановите окружение без `-v`, затем поднимите снова и убедитесь, что строка сохранилась.

```
docker compose down
docker compose up -d
docker compose exec postgres psql -U task_manager -d task_manager -c 'SELECT * FROM
docker_check;'
```

Критерии проверки:

- таблица и строка переживают `docker compose down`;
- named volume подключён к `/var/lib/postgresql/data`;
- вы понимаете, что `docker compose down -v` удалит эту строку;
- порт PostgreSQL опубликован только на `127.0.0.1`;
- в репозиторий не попадают данные volume.

Задание 6. Подготовьте окружение для SQL-главы

Дополните `.env.example` комментариями о назначении `POSTGRES_DB`, `POSTGRES_USER`, `POSTGRES_PASSWORD`, `POSTGRES_PORT`. Выведите в README примера или в заметках две строки подключения: для API-container и для DBeaver на хосте.

Критерии проверки:

- внутри Compose host — `postgres`;
- для DBeaver host — `localhost`;
- порт, база и пользователь совпадают с `.env`;
- password не попадает в Git вне безопасного учебного примера;
- указано, что в production PostgreSQL-порт обычно не публикуют наружу.

Проверьте себя

Перед SQL-главой убедитесь, что вы можете:

- отличить image, container и volume;
- собрать и проверить multi-stage Go image;
- объяснить назначение `.dockerignore`;
- запустить API и PostgreSQL одной командой;
- объяснить разницу `postgres:5432` и `localhost:5432`;
- сохранить или осознанно удалить данные volume.

Рекомендации по коду: Docker и локальное окружение

- Собирайте Go-приложение через multi-stage Dockerfile и копируйте в final image только нужный бинарник.
- Не запускайте production container от root, если для этого нет явной причины.
- Держите Docker build context маленьким: `.git`, `.env`, документация, временные файлы и coverage не должны попадать в image.
- Разделяйте `.gitignore` и `.dockerignore`: они решают разные задачи и оба нужны.
- Передавайте настройки через env и Compose, а не записывайте адреса, пароли и токены в Dockerfile.
- Внутри Compose обращайтесь к сервисам по их именам, например `postgres`, а не через `localhost`.
- Сохраняйте данные PostgreSQL в named volume и используйте `down -v` только когда осознанно нужен чистый state.
- Публикуйте PostgreSQL на `127.0.0.1` только для локальных инструментов; production-БД не должна быть доступна из внешней сети по умолчанию.
- Используйте health-check как сигнал готовности зависимости, но всё равно обрабатывайте ошибки подключения в приложении.

- Проверяйте итоговый YAML через `docker compose config`, а API — через `/healthz` после запуска.

Чек-лист

Окружение поднимается одной командой, API и PostgreSQL используют предсказуемые версии, БД сохраняет данные в volume, локальные secrets не попадают в image и Git, а приложение готово подключиться к service `postgres` в следующей главе.

SQL и PostgreSQL

Цель урока

Подключить Task Manager API к PostgreSQL из Docker Compose через `database/sql` и `pgx`, хранить структуру БД в SQL-миграциях Goose, реализовать CRUD-репозиторий задач с параметризованными запросами, применять context в операциях БД и проверять всё интеграционными тестами против отдельной тестовой базы.

В предыдущей главе PostgreSQL уже запущен как сервис `postgres` в Docker Compose. Сейчас он перестаёт быть только инфраструктурой: API подключается к нему и заменяет in-memory `TaskStore` на PostgreSQL-репозиторий.

```
Gin handler → TaskService → TaskRepository → database/sql → pgx → PostgreSQL
```

Service по-прежнему зависит от интерфейса репозитория. Благодаря этому HTTP-слой Gin и бизнес-правила не должны знать SQL-запросы, а unit-тесты service могут использовать fake repository.

PostgreSQL и DBeaver

Поднимите окружение из прошлой главы:

```
cd examples/32/task-manager-api
docker compose up -d
docker compose ps
```

Внутри Docker Compose приложение подключается к host `postgres`. DBeaver запускается на вашем компьютере, поэтому для него указывайте опубликованный локальный адрес:

Поле DBeaver	Значение из `.env` по умолчанию
Host	`localhost`
Port	`5432` (`POSTGRES_PORT`)
Database	`task_manager` (`POSTGRES_DB`)
Username	`task_manager` (`POSTGRES_USER`)
Password	`local-development-only` (`POSTGRES_PASSWORD`)

В DBeaver выберите **New Database Connection → PostgreSQL**, заполните поля и нажмите **Test Connection**. Если драйвер PostgreSQL ещё не скачан, DBeaver предложит сделать это сам.

Для учебного Docker Compose TLS не настроен. Если DBeaver требует SSL, на вкладке **SSL** выберите режим `disable`. В production режим TLS определяется требованиями инфраструктуры, а не этой локальной настройкой.

После выполнения миграций раскройте соединение в Database Navigator:

```
Databases → task_manager → Schemas → public → Tables → tasks
```

Через DBeaver удобно просматривать таблицы, запускать `SELECT` и проверять результат миграций. Он не заменяет миграции: изменение таблицы вручную в UI не попадёт в Git и не повторится у другого разработчика.

> **Два разных hostname**
>
> Из API-container подключайтесь к `postgres:5432`. Из DBeaver на компьютере — к `localhost:5432`. `localhost` внутри container означает сам container, а не сервис PostgreSQL.

`database/sql` и pgx

`database/sql` — стандартный пакет Go с общим API для SQL-баз. Он управляет пулом соединений, `QueryContext`, `ExecContext`, `BeginTx` и транзакциями, но сам не умеет разговаривать с PostgreSQL. Для этого нужен драйвер.

Используем `pgx` через адаптер `stdlib`:

```
go get github.com/jackc/pgx/v5/stdlib
```

Зарегистрируйте драйвер blank-импортом один раз в пакете, где открываете базу:

```
import (
  "database/sql"
  _ "github.com/jackc/pgx/v5/stdlib"
)
```

Пустой идентификатор означает: пакет нужен из-за side effect — он регистрирует драйвер с именем `pgx`. Теперь можно вызвать:

```
db, err := sql.Open("pgx", databaseURL)
if err != nil {
  return nil, fmt.Errorf("открыть PostgreSQL: %w", err)
}
```

`sql.Open` не гарантирует, что соединение с базой уже установлено. Он создаёт `*sql.DB` — пул будущих соединений. Реальную доступность проверяет `PingContext`.

Открытие и настройка пула

Создайте функцию подключения:

```
func Open(ctx context.Context, databaseURL string) (*sql.DB, error) {
  db, err := sql.Open("pgx", databaseURL)
  if err != nil {
    return nil, fmt.Errorf("открыть PostgreSQL: %w", err)
  }

  db.SetMaxOpenConns(10)
  db.SetMaxIdleConns(5)
  db.SetConnMaxLifetime(30 * time.Minute)
  db.SetConnMaxIdleTime(5 * time.Minute)

  pingCtx, cancel := context.WithTimeout(ctx, 5*time.Second)
  defer cancel()

  if err := db.PingContext(pingCtx); err != nil {
    db.Close()
    return nil, fmt.Errorf("проверить PostgreSQL: %w", err)
  }

  return db, nil
}
```

Значения пула зависят от нагрузки и ограничений PostgreSQL. Эти числа — безопасный старт для учебного API, а не универсальный production-рецепт. Общее число подключений всех экземпляров API должно помещаться в лимит `max_connections` базы.

Если `PingContext` не удался, закрывайте уже созданный пул до возврата ошибки. В `main` после graceful shutdown передайте `db` в `closeAll`, который появился в прошлой главе:

```
defer db.Close()
```

В реальном `run` закрытие должно идти **после** `server.Shutdown`, чтобы активный handler не потерял базу во время запроса.

`DATABASE_URL`

Внутри Compose строка подключения имеет вид:

```
postgresql://task_manager:local-development-only@postgres:5432/task_manager?
sslmode=disable
```

Для запуска кода на компьютере и DBeaver hostname меняется на `localhost`:

```
postgresql://task_manager:local-development-only@localhost:5432/task_manager?
sslmode=disable
```

`sslmode=disable` допустим только для локального учебного окружения Docker. В production способ TLS-подключения выбирается требованиями инфраструктуры; не переносите это значение без проверки.

Добавьте в локальный `.env`:

```
DATABASE_URL=postgresql://task_manager:local-development-
only@postgres:5432/task_manager?sslmode=disable
```

Значение внутри API-container использует `postgres`. Не выводите полный `DATABASE_URL` в лог: в нём есть пароль.

Миграции Goose

Миграция — версия изменения схемы БД, которая хранится в Git и применяется одинаково у каждого разработчика и на CI. Не создавайте таблицы вручную только через DBeaver или `psql`: остальным окружениям это изменение не достанется.

Добавьте библиотеку и CLI Goose:

```
go get github.com/pressly/goose/v3
go install github.com/pressly/goose/v3/cmd/goose@latest
```

Создайте каталог `migrations` и первую миграцию:

```
mkdir migrations
goose -dir migrations create create_tasks_table sql
```

Файл будет иметь timestamp в имени. Например, `migrations/20261014120000_create_tasks_table.sql`:

```
-- +goose Up
CREATE TABLE tasks (
```

```
    id BIGSERIAL PRIMARY KEY,    title TEXT NOT NULL CHECK (char_length(btrim(title))
> 0),    priority SMALLINT NOT NULL CHECK (priority BETWEEN 1 AND 5),    completed
BOOLEAN NOT NULL DEFAULT FALSE,    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW());CREATE INDEX
tasks_completed_priority_idx ON tasks (completed, priority);-- +goose DownDROP TABLE
tasks;
```

В `Up` находятся действия применения, в `Down` — откат именно этой миграции. После того как миграция попала в общую среду, не переписывайте её историю. Создайте следующую миграцию для нового изменения схемы.

Примените и проверьте миграции внутри API-container или на хосте с корректным `DATABASE_URL`:

```
goose -dir migrations postgres "$DATABASE_URL" status
goose -dir migrations postgres "$DATABASE_URL" up
goose -dir migrations postgres "$DATABASE_URL" status
```

Goose создаёт таблицу `goose_db_version`. В DBeaver убедитесь, что есть и `tasks`, и `goose_db_version`.

Модель SQL и Go

Добавим время создания и обновления в модель, чтобы репозиторий мог читать их из PostgreSQL:

```
type Task struct {
  ID        int64      `json:"id"`
  Title     string     `json:"title"`
  Priority  int        `json:"priority"`
  Completed bool       `json:"completed"`
  CreatedAt time.Time  `json:"created_at"`
  UpdatedAt time.Time  `json:"updated_at"`
}
```

Выбирайте `BIGSERIAL` и `int64` согласованно. PostgreSQL возвращает время с временной зоной в `time.Time`. JSON-кодировщик отдаст его в формате RFC 3339.

Для `INSERT ... RETURNING` создайте задачу и получите значения, которые назначила база:

```
const createTaskQuery = `
INSERT INTO tasks (title, priority)
VALUES ($1, $2)
RETURNING id, title, priority, completed, created_at, updated_at`

func (repository *Repository) Create(ctx context.Context, input task.CreateInput)
```

```
(task.Task,error){  var created task.Task err := repository.db.QueryRowContext(ctx, createTaskQuery, input.Title, input.Priority).Scan(    &created.ID, &created.Title,    &created.Priority,    &created.Completed,   &created.CreatedAt, &created.UpdatedAt, ) if err != nil {    return task.Task{}, fmt.Errorf("создать задачу: %w", err) } return created, nil}
```

`QueryRowContext` подходит, когда ожидается ровно одна строка. `Scan` выполняет запрос и считывает результат. Порядок переменных должен совпадать с порядком столбцов в `RETURNING`.

Параметры и SQL-инъекции

Никогда не собирайте значения пользователя через `fmt.Sprintf`:

```
// Уязвимо и неверно:
// query := fmt.Sprintf("SELECT id FROM tasks WHERE title = '%s'", title)
```

Используйте placeholders PostgreSQL `$1`, `$2` и передавайте значения отдельными аргументами:

```
const getTaskQuery = `
SELECT id, title, priority, completed, created_at, updated_at
FROM tasks
WHERE id = $1`

row := repository.db.QueryRowContext(ctx, getTaskQuery, id)
```

Драйвер сам передаёт `id` как значение, а не как часть SQL-синтаксиса. Placeholders применяются к **данным**, но не к именам таблиц, столбцов и `ORDER BY`-направлениям. Если позже понадобится динамическая сортировка, выбирайте имя столбца из закрытого whitelist, а не из строки клиента.

`Get`, `List` и `sql.ErrNoRows`

`Get` преобразует отсутствие строки в доменную ошибку:

```
var ErrTaskNotFound = errors.New("задача не найдена")

func (repository *Repository) Get(ctx context.Context, id int64) (task.Task, error) {
  var current task.Task
  err := repository.db.QueryRowContext(ctx, getTaskQuery, id).Scan(
    &current.ID,
    &current.Title,
    &current.Priority,
    &current.Completed,
    &current.CreatedAt,
```

```
    &current.UpdatedAt, ) if errors.Is(err, sql.ErrNoRows) {    return task.Task{},
ErrTaskNotFound   } if err != nil {     return task.Task{}, fmt.Errorf("получить
задачу %d: %w", id, err)   } return current, nil}
```

HTTP layer затем преобразует `ErrTaskNotFound` в `404`, не зная о `sql.ErrNoRows`.

Для нескольких строк используйте `QueryContext`. Всегда закрывайте `rows` и проверяйте `rows.Err()` после цикла:

```
const listTasksQuery = `
SELECT id, title, priority, completed, created_at, updated_at
FROM tasks
ORDER BY id`

func (repository *Repository) List(ctx context.Context) ([]task.Task, error) {
  rows, err := repository.db.QueryContext(ctx, listTasksQuery)
  if err != nil {
    return nil, fmt.Errorf("получить список задач: %w", err)
  }
  defer rows.Close()

  tasks := make([]task.Task, 0)
  for rows.Next() {
    var current task.Task
    if err := rows.Scan(
      &current.ID,
      &current.Title,
      &current.Priority,
      &current.Completed,
      &current.CreatedAt,
      &current.UpdatedAt,
    ); err != nil {
      return nil, fmt.Errorf("прочитать задачу: %w", err)
    }
    tasks = append(tasks, current)
  }
  if err := rows.Err(); err != nil {
    return nil, fmt.Errorf("обойти список задач: %w", err)
  }

  return tasks, nil
}
```

Пустой список возвращается как `[]`, а не `null`, поскольку слайс создаётся через `make`.

Update и Delete

Завершим задачу через `UPDATE ... RETURNING`:

```
const completeTaskQuery = `
UPDATE tasks
SET completed = TRUE, updated_at = NOW()
WHERE id = $1
RETURNING id, title, priority, completed, created_at, updated_at`

func (repository *Repository) Complete(ctx context.Context, id int64) (task.Task,
error) {
  var completed task.Task
  err := repository.db.QueryRowContext(ctx, completeTaskQuery, id).Scan(
    &completed.ID,
    &completed.Title,
    &completed.Priority,
    &completed.Completed,
    &completed.CreatedAt,
    &completed.UpdatedAt,
  )
  if errors.Is(err, sql.ErrNoRows) {
    return task.Task{}, ErrTaskNotFound
  }
  if err != nil {
    return task.Task{}, fmt.Errorf("завершить задачу %d: %w", id, err)
  }

  return completed, nil
}
```

Для удаления обычно достаточно `ExecContext` и проверки числа затронутых строк:

```
func (repository *Repository) Delete(ctx context.Context, id int64) error {
  result, err := repository.db.ExecContext(ctx, `DELETE FROM tasks WHERE id = $1`,
id)
  if err != nil {
    return fmt.Errorf("удалить задачу %d: %w", id, err)
  }

  affected, err := result.RowsAffected()
  if err != nil {
    return fmt.Errorf("узнать число удалённых задач: %w", err)
```

```
  } if affected == 0 {    return ErrTaskNotFound  } return nil}
```

Context в SQL-операциях

Используйте методы с суффиксом `Context`: `PingContext`, `QueryContext`, `QueryRowContext`, `ExecContext`, `BeginTx`. Тогда отменённый HTTP-запрос или дедлайн не оставит бессмысленную SQL-операцию работать дальше.

```
createdTask, err := repository.Create(c.Request.Context(), input)
```

Не подменяйте полученный context на `context.Background()` внутри репозитория. Исключение — отдельный lifecycle context при штатном завершении процесса, но это задача `main`, а не CRUD-метода.

Транзакции

Транзакция нужна, когда несколько изменений должны пройти все вместе или ни одно. Например, создадим задачу и запись аудита:

```
func (repository *Repository) CreateWithAudit(ctx context.Context, input
task.CreateInput) (task.Task, error) {
  tx, err := repository.db.BeginTx(ctx, nil)
  if err != nil {
    return task.Task{}, fmt.Errorf("начать транзакцию: %w", err)
  }
  defer tx.Rollback() // после Commit вернёт sql.ErrTxDone и будет проигнорирован

  var created task.Task
  err = tx.QueryRowContext(ctx, createTaskQuery, input.Title, input.Priority).Scan(
    &created.ID, &created.Title, &created.Priority, &created.Completed,
    &created.CreatedAt, &created.UpdatedAt,
  )
  if err != nil {
    return task.Task{}, fmt.Errorf("создать задачу: %w", err)
  }

  if _, err := tx.ExecContext(ctx,
    `INSERT INTO task_audit (task_id, action) VALUES ($1, $2)`,
    created.ID, "created",
  ); err != nil {
    return task.Task{}, fmt.Errorf("создать аудит: %w", err)
  }
```

```
    return task.Task{}, fmt.Errorf("зафиксировать транзакцию: %w", err)  } return
created, nil}
```

Не начинайте транзакцию вокруг каждого одиночного `INSERT` или `UPDATE`: PostgreSQL и так выполняет одиночную SQL-команду атомарно. `task_audit` здесь появится только в отдельной миграции, если такая функция действительно нужна.

Интеграционные тесты через Docker Compose

Unit-тесты service остаются быстрыми и используют fake repository. PostgreSQL-репозиторий нужно также проверять против настоящей БД: SQL-синтаксис, типы, constraints и миграции нельзя надёжно подтвердить только mock-объектом.

Поднимите PostgreSQL из Compose и создайте отдельную тестовую БД один раз:

```
cd examples/32/task-manager-api
docker compose up -d postgres
docker compose exec postgres createdb -U task_manager task_manager_test
```

Задайте **только** тестовую строку подключения:

```
export TEST_DATABASE_URL='postgresql://task_manager:local-development-
only@localhost:5432/task_manager_test?sslmode=disable'
```

Тесты никогда не должны получать `DATABASE_URL` production или development-базы. Разбирайте URL и проверяйте **именно имя базы**, а не наличие `_test` во всей строке: эта подстрока может случайно оказаться в имени пользователя, пароле или query-параметре.

```
func testDatabaseURL(t *testing.T) string {
  t.Helper()

  databaseURL := os.Getenv("TEST_DATABASE_URL")
  if databaseURL == "" {
    t.Skip("TEST_DATABASE_URL не задан")
  }

  parsedURL, err := url.Parse(databaseURL)
  if err != nil {
    t.Fatalf("разобрать TEST_DATABASE_URL: %v", err)
  }

  databaseName := strings.TrimPrefix(parsedURL.Path, "/")
  if databaseName != "task_manager_test" {
```

```
    t.Fatalf("TEST_DATABASE_URL должен указывать на task_manager_test, получено %q",
databaseName)   } return databaseURL}
```

Для этого helper понадобятся импорты `net/url` и `strings`. Если в вашем проекте тестовая БД называется иначе, вынесите разрешённое имя в константу и измените её явно.

Примените к ней миграции в `TestMain` или до тестового запуска:

```
goose -dir migrations postgres "$TEST_DATABASE_URL" up
go test -tags=integration ./internal/postgresrepo/...
```

Используйте build tag, чтобы обычный `go test ./...` не требовал запущенную БД:

```
//go:build integration

package postgresrepo_test
```

Перед каждым тестом очищайте только тестовую таблицу:

```
func truncateTasks(t *testing.T, db *sql.DB) {
  t.Helper()
  if _, err := db.Exec(`TRUNCATE tasks RESTART IDENTITY`); err != nil {
    t.Fatalf("очистить tasks: %v", err)
  }
}
```

Тест `Create` проверяет реальную вставку и чтение:

```
func TestRepositoryCreateAndGet(t *testing.T) {
  db := openTestDatabase(t)
  t.Cleanup(func() { db.Close() })
  truncateTasks(t, db)

  repository := postgresrepo.New(db)
  created, err := repository.Create(context.Background(), task.CreateInput{
    Title: "Купить молоко", Priority: 2,
  })
  if err != nil {
    t.Fatalf("Create: %v", err)
  }
  if created.ID == 0 || created.CreatedAt.IsZero() {
    t.Fatalf("создана некорректная задача: %+v", created)
```

```
  } got, err := repository.Get(context.Background(), created.ID)  if err != nil {
t.Fatalf("Get: %v", err)  } if got.Title != created.Title || got.Priority !=
created.Priority {    t.Fatalf("Get = %+v, ожидалась %+v", got, created) }}
```

Не запускайте такие тесты с `t.Parallel`, если они используют одну БД и `TRUNCATE`. Для полной изоляции параллельных интеграционных тестов понадобятся отдельные схемы или базы — это отдельная задача.

Частые ошибки

Думать, что `sql.Open` уже подключился

`sql.Open` может успешно вернуть pool при недоступной БД. Проверяйте доступность через `PingContext` с timeout при старте.

Подставлять значения в SQL строкой

```
query := "DELETE FROM tasks WHERE id = " + id
```

Такой код уязвим и ломается на форматах данных. Используйте `$1` и аргументы `ExecContext`.

Не закрывать `rows`

Незакрытые `rows` удерживают соединение пула. После успешного `QueryContext` сразу пишите `defer rows.Close()` и после обхода проверяйте `rows.Err()`.

Запускать тесты против development-БД

`TRUNCATE` в тесте может уничтожить ваши локальные данные. Используйте отдельную `task_manager_test`, `TEST_DATABASE_URL` и защитную проверку имени базы.

Редактировать уже применённую миграцию

У другого разработчика или CI эта версия могла уже примениться. Создайте новую миграцию, которая изменяет схему вперёд.

Дальше

Task Manager хранит задачи в PostgreSQL, а схема воспроизводимо развивается через Goose. В следующей главе настроим CI/CD и деплой: тесты, проверку образа, secrets и health-check после доставки приложения.

Самопроверка

1. Что представляет собой *sql.DB?

A. Только конфигурационный файл PostgreSQL

B. Одну строку результата SELECT

C. Пул соединений и точку доступа к БД, а не одно готовое соединение

D. Docker-container PostgreSQL

2. Как проверить доступность PostgreSQL после sql.Open?

A. Открыть DBeaver

B. Вызвать PingContext с ограниченным context

C. Вызвать fmt.Println(db)

D. Считать успешный sql.Open достаточным

3. Какой драйвер используется в главе вместе с database/sql?

A. Goose как драйвер запросов

B. Gin

C. DBeaver JDBC внутри Go

D. github.com/jackc/pgx/v5/stdlib с именем драйвера pgx

4. Как безопасно передать ID в PostgreSQL-запрос?

A. Использовать placeholder $1 и передать id отдельным аргументом

B. Добавить id к SQL через +

C. Передать id в комментарии SQL

D. Собрать запрос через fmt.Sprintf

5. Что означает sql.ErrNoRows после QueryRowContext().Scan()?

A. PostgreSQL выключен

B. Транзакция уже зафиксирована

C. Запрос выполнился, но подходящей строки не найдено

D. В SQL обязательно есть инъекция

6. Что необходимо сделать после успешного QueryContext?

A. Отложить rows.Close и после цикла проверить rows.Err

B. Игнорировать rows, потому что это слайс

C. Сразу закрыть *sql.DB

D. Вызвать os.Exit

7. Для чего в Goose-миграции нужны разделы +goose Up и +goose Down?

A. Для генерации Go-структур

B. Для шифрования пароля БД

C. Для применения и отката конкретного изменения схемы

D. Для запуска Docker-container

8. Когда нужна транзакция?

A. Только при открытии DBeaver

B. Вместо context.Context

C. Перед каждым одиночным SELECT

D. Когда несколько изменений должны выполниться все вместе или ни одно

9. К какой базе должны обращаться интеграционные тесты?

A. К случайной публичной базе

B. К отдельной базе из TEST_DATABASE_URL, например task_manager_test

C. К базе DBeaver без URL

D. К production DATABASE_URL

10. Какой hostname использует API-container для PostgreSQL из Compose?

A. postgres

B. dbeaver

C. localhost

D. 127.0.0.1 хоста

Ответы к самопроверке

1. Пул соединений и точку доступа к БД, а не одно готовое соединение

2. Вызвать PingContext с ограниченным context

3. github.com/jackc/pgx/v5/stdlib с именем драйвера pgx

4. Использовать placeholder $1 и передать id отдельным аргументом

5. Запрос выполнился, но подходящей строки не найдено

6. Отложить rows.Close и после цикла проверить rows.Err

7. Для применения и отката конкретного изменения схемы

8. Когда несколько изменений должны выполниться все вместе или ни одно

9. К отдельной базе из TEST_DATABASE_URL, например task_manager_test

10. postgres

Задания: PostgreSQL-репозиторий Task Manager

Подключите Task Manager API к PostgreSQL из Docker Compose. Сохраняйте границы приложения: Gin handler вызывает service, service зависит от интерфейса, а SQL остаётся в `internal/postgresrepo`.

Задание 1. Подготовьте базу и DBeaver

Запустите PostgreSQL из Compose, затем подключитесь к нему через DBeaver по `localhost` и значениям `.env`.

```
cd examples/32/task-manager-api
docker compose up -d postgres
docker compose ps
```

Критерии проверки:

- PostgreSQL имеет статус `healthy`;
- DBeaver подключается к `localhost:${POSTGRES_PORT}`;
- выбраны база, пользователь и пароль из `.env`;
- API-container в будущей строке подключения использует `postgres:5432`;
- пароль остаётся только в локальном `.env` или безопасном хранилище secrets.

Задание 2. Подключите `database/sql` и pgx

Добавьте `github.com/jackc/pgx/v5/stdlib`. Реализуйте `Open(ctx, databaseURL)`, который открывает драйвер `pgx`, настраивает пул и выполняет `PingContext` с timeout.

Критерии проверки:

- driver импортирован через blank import;
- используется `sql.Open("pgx", databaseURL)`;
- настраиваются `SetMaxOpenConns`, `SetMaxIdleConns`, `SetConnMaxLifetime`;
- `PingContext` имеет отдельный ограниченный context;
- при ошибке ping pool закрывается;
- полный DATABASE_URL не выводится в лог.

Подсказка

```
pingCtx, cancel := context.WithTimeout(ctx, 5*time.Second)
defer cancel()

if err := db.PingContext(pingCtx); err != nil {
  db.Close()
  return nil, fmt.Errorf("проверить PostgreSQL: %w", err)
}
```

Задание 3. Создайте и примените миграцию Goose

Установите Goose, создайте SQL-миграцию `create_tasks_table` и добавьте таблицу `tasks` с ID, названием, приоритетом, completed и timestamps. Добавьте constraints: название не пустое после `btrim`, приоритет от 1 до 5.

Критерии проверки:

- миграция хранится в `migrations/` и коммитится в Git;
- у неё есть разделы `-- +goose Up` и `-- +goose Down`;
- `id` согласован с `int64` в Go;
- есть `NOT NULL` и `CHECK` constraints;
- `goose status` показывает состояние миграций;
- в DBeaver видны `tasks` и `goose_db_version`.

Подсказка

```
-- +goose Up
CREATE TABLE tasks (
    id BIGSERIAL PRIMARY KEY,
    title TEXT NOT NULL CHECK (char_length(btrim(title)) > 0),
    priority SMALLINT NOT NULL CHECK (priority BETWEEN 1 AND 5),
    completed BOOLEAN NOT NULL DEFAULT FALSE
);

-- +goose Down
DROP TABLE tasks;
```

Задание 4. Реализуйте CRUD-репозиторий

Создайте `internal/postgresrepo.Repository` с методами `Create`, `Get`, `List`, `Complete` и `Delete`. Все методы должны принимать `context.Context`; значения передавайте через `$1`, `$2`; отсутствие строки превратите из `sql.ErrNoRows` в `ErrTaskNotFound`.

Критерии проверки:

- `Create` и `Complete` используют `RETURNING`;
- `Get` использует `QueryRowContext` и `Scan`;
- `List` использует `QueryContext`, `defer rows.Close` и `rows.Err`;
- `Delete` проверяет `RowsAffected`;
- нет SQL, собранного через конкатенацию пользовательских данных;
- технические SQL-ошибки обёрнуты с контекстом.

Подсказка

```
row := repository.db.QueryRowContext(ctx, getTaskQuery, id)
if err := row.Scan(&current.ID, &current.Title, ...); errors.Is(err, sql.ErrNoRows) {
  return task.Task{}, ErrTaskNotFound
}
```

Задание 5. Добавьте одну транзакцию

Создайте отдельную миграцию для `task_audit`. Реализуйте `CreateWithAudit`: одна транзакция создаёт задачу и запись аудита. Если вставка аудита не удалась, задача не должна остаться в БД.

Критерии проверки:

- используется `BeginTx(ctx, nil)`;
- есть `defer tx.Rollback()`;
- оба SQL-вызова используют `tx`, а не `db`;
- `Commit` вызывается только после обоих успешных действий;
- ошибка любого шага откатывает изменение;
- одиночные CRUD-методы не обёрнуты в лишние транзакции.

Задание 6. Напишите интеграционные тесты

Создайте отдельную `task_manager_test` в уже запущенном PostgreSQL. Задайте `TEST_DATABASE_URL`, примените миграции и добавьте integration-тесты Create/Get, List, Complete, Delete и отсутствующей задачи. Запускайте их build tag `integration`.

```
docker compose exec postgres createdb -U task_manager task_manager_test
export TEST_DATABASE_URL='postgresql://task_manager:local-development-
only@localhost:5432/task_manager_test?sslmode=disable'
goose -dir migrations postgres "$TEST_DATABASE_URL" up
go test -tags=integration ./internal/postgresrepo/...
```

Критерии проверки:

- tests используют только `TEST_DATABASE_URL`;
- URL проверяется на отдельную test-БД;
- миграции применены до тестов;
- таблицы очищаются перед сценарием через `TRUNCATE ... RESTART IDENTITY`;
- tests не используют `t.Parallel`, публичную сеть и `time.Sleep`;
- обычный `go test ./...` не требует PostgreSQL;
- integration suite проходит с `-race`.

Проверьте себя

Перед главой CI/CD убедитесь, что вы можете:

- подключить PostgreSQL через `database/sql` и pgx;
- создать миграцию и проверить её через Goose и DBeaver;
- реализовать SQL CRUD только с параметрами и context;
- отличить `QueryRowContext`, `QueryContext` и `ExecContext`;
- выбрать транзакцию только для нескольких зависимых изменений;
- безопасно запускать интеграционные тесты на отдельной базе.

Рекомендации по коду: SQL и PostgreSQL

- Открывайте `*sql.DB` один раз при старте, настраивайте пул и проверяйте доступность через `PingContext`.
- Передавайте context в каждую операцию базы: `QueryContext`, `QueryRowContext`, `ExecContext` и `BeginTx`.
- Никогда не собирайте пользовательские значения конкатенацией в SQL; используйте placeholders `$1`, `$2`.
- Сразу после `QueryContext` откладывайте `rows.Close` и всегда проверяйте `rows.Err` после обхода.

- Преобразуйте `sql.ErrNoRows` в ошибку предметной области, а не раскрывайте SQL-детали HTTP-слою.
- Храните изменения схемы в неизменяемых SQL-миграциях Goose и применяйте их одинаково локально, в CI и production.
- Используйте транзакции только там, где несколько изменений должны быть атомарными.
- Не храните DATABASE_URL или пароли в Git и не записывайте их целиком в логи.
- Проверяйте репозиторий интеграционными тестами против отдельной test-БД; не запускайте destructive SQL против development или production базы.
- Используйте DBeaver для диагностики схемы и данных, но не для ручных изменений, которые должны быть миграциями.

Чек-лист

API подключается к PostgreSQL через настроенный пул, схема создаётся Goose-миграциями, CRUD передаёт context и использует параметры SQL, ошибки отсутствия ресурса становятся `404` на HTTP-границе, а реальный PostgreSQL-репозиторий проверен на отдельной тестовой базе.

CI/CD и деплой

Цель урока

Настроить для Task Manager API GitHub Actions: автоматически запускать проверку Go-кода и PostgreSQL integration-тесты, собирать Docker-образ, публиковать проверенный образ в GitHub Container Registry (GHCR) и безопасно доставлять его на VPS через Docker Compose с health-check и возможностью отката.

CI (continuous integration) отвечает на вопрос «можно ли объединить это изменение?»: собирает проект и запускает проверки. **CD** (continuous delivery/deployment) отвечает на вопрос «как доставить уже проверенный образ в окружение?». Сначала сделаем CI надёжным, затем отдельно подготовим delivery и deployment.

```
pull request / push
        ↓
GitHub Actions: vet → unit/race → PostgreSQL integration → Docker build
        ↓
GHCR: неизменяемый Docker image по SHA commit
        ↓
VPS: docker compose pull/up → /healthz → rollback при проблеме
```

Что должно быть до CI

Финальный Task Manager API должен содержать:

- `go.mod` и `go.sum`;
- unit-тесты, запускаемые через `go test ./...`;
- миграции Goose в `migrations/`;
- integration-тесты PostgreSQL с build tag `integration`;
- `Dockerfile`;
- endpoint `GET /healthz`;
- `compose.production.yaml` для VPS.

Не копируйте workflow в учебник как есть, если пути и имя модуля отличаются. Сначала запустите все команды локально — CI должен автоматизировать уже работающий workflow, а не заменять отладку.

CI: unit-тесты, race detector и Docker build

Создайте `.github/workflows/ci.yml` в репозитории **Task Manager API**:

```
name: CI

on:
  push:
    branches: [master]
  pull_request:

permissions:
  contents: read

jobs:
  unit:
    name: Unit tests and static checks
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-go@v5
        with:
          go-version-file: go.mod
          cache: true

      - name: Check module files
        run: |
          go mod tidy
          git diff --exit-code -- go.mod go.sum

      - name: Vet
        run: go vet ./...

      - name: Unit tests with race detector
        run: go test -race ./...

      - name: Build API
        run: go build ./cmd/api

  docker:
    name: Docker build
    runs-on: ubuntu-latest
    needs: unit
    steps:
      - uses: actions/checkout@v4
```

```
      - name: Validate Compose configuration        working-directory:
examples/32/task-manager-api        run: docker compose config      - name: Build
image        working-directory: examples/32/task-manager-api        run: docker build
--tag task-manager-api:ci .
```

`go mod tidy` **может менять** `go.mod` и `go.sum`, поэтому сразу после него выполняется `git diff --exit-code`. Так CI не только сообщает, что файлы модулей неактуальны, но и не оставляет незаметное изменение в runner.

Docker build зависит от `unit`: нет смысла тратить время на сборку образа, если код не прошёл статический анализ или тесты. В production workflow можно использовать cache BuildKit, но сначала добейтесь воспроизводимой простой сборки.

PostgreSQL integration-тесты в GitHub Actions

Для integration-тестов используем service container PostgreSQL. Он живёт только во время job и не требует Docker Compose или отдельной тестовой инфраструктуры:

```
  integration:
    name: PostgreSQL integration tests
    runs-on: ubuntu-latest
    needs: unit
    services:
      postgres:
        image: postgres:17-alpine
        env:
          POSTGRES_DB: task_manager_test
          POSTGRES_USER: task_manager
          POSTGRES_PASSWORD: test-password-only
        ports:
          - 5432:5432
        options: >-
          --health-cmd "pg_isready -U task_manager -d task_manager_test"
          --health-interval 5s
          --health-timeout 3s
          --health-retries 10

    env:
      TEST_DATABASE_URL: postgresql://task_manager:test-password-
only@localhost:5432/task_manager_test?sslmode=disable

    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-go@v5
```

```
        with:          go-version-file: go.mod          cache: true      - name:
Install Goose        run: go install github.com/pressly/goose/v3/cmd/goose@v3.29.0
- name: Apply migrations        run: $HOME/go/bin/goose -dir migrations postgres
"$TEST_DATABASE_URL" up      - name: Integration tests        run: go test -race -
count=1 -tags=integration ./internal/postgresrepo/...
```

Пароль в этом примере не секрет: это одноразовая тестовая БД внутри ephemeral runner. Настоящие пароли, токены и SSH-ключи никогда не записывают прямо в workflow — для них используются GitHub Actions Secrets.

Тесты должны обращаться только к `TEST_DATABASE_URL`. Проверка имени базы с суффиксом `_test` из прошлой главы остаётся дополнительной защитой от разрушительного `TRUNCATE` не той базы.

`-count=1` отключает тестовый кэш: integration-тест действительно выполняется против service container. Команда `go test -race` медленнее обычной, но для небольшого backend-проекта это оправданная проверка конкурентных ошибок.

Матрица проверок

Job	Что подтверждает	Чего не подтверждает
`unit`	компиляцию, unit-тесты, race detector, актуальность модулей	реальный SQL PostgreSQL
`integration`	миграции, SQL, constraints и PostgreSQL-репозиторий	production-конфигурацию VPS
`docker`	Dockerfile и Compose YAML	работоспособность приложения после запуска образа
deployment	доставку конкретного образа и `/healthz`	полную бизнес-нагрузку production

Каждый job нужен для отдельного риска. Не заменяйте PostgreSQL-тесты проверкой `docker build`, а health-check после деплоя — unit-тестами.

Публикация образа в GHCR

GitHub Container Registry хранит Docker-образы рядом с репозиторием. Публикуйте только образ, прошедший CI, и маркируйте его неизменяемым SHA commit. `latest` удобен для просмотра, но не годится как единственный production-тег: по нему нельзя понять, какой именно код работает и как откатиться.

Создайте `.github/workflows/publish-image.yml`:

```
name: Publish image

on:
  push:
```

```
    branches: [master]permissions:  contents: read  packages: writeenv:  # GitHub
image names должны быть lowercase.  IMAGE_NAME: ghcr.io/danilchugaev/task-manager-
apijobs:  publish:    runs-on: ubuntu-latest    steps:      - uses:
actions/checkout@v4      - uses: docker/setup-buildx-action@v3      - uses:
docker/login-action@v3        with:          registry: ghcr.io          username: ${{
github.actor }}          password: ${{ secrets.GITHUB_TOKEN }}      - id: meta
uses: docker/metadata-action@v5        with:          images: ${{ env.IMAGE_NAME }}
tags: |
            type=raw,value=latest
            type=sha,format=long      - uses: docker/build-push-action@v6
with:          context: .          push: true          tags: ${{
steps.meta.outputs.tags }}          labels: ${{ steps.meta.outputs.labels }}
```

Замените `danilchugaev/task-manager-api` на lowercase-имя владельца и образа вашего конечного API. Если registry package закрыт, VPS понадобится токен с разрешением `read:packages`; не используйте personal token с избыточными правами.

> **Не публикуйте образ до CI**
>
> Самостоятельный publish workflow запускается на `push`, но GitHub Actions не гарантирует порядок разных workflow. Для строгого правила «publish только после зелёного CI» объедините CI и publish в один workflow: job `publish` должен иметь `needs: [unit, integration, docker]`. Альтернатива — запускать publish только после защищённого merge в `master`, где branch protection требует успешный CI.

Практический вариант одного workflow:

```
  publish:
    needs: [unit, integration, docker]
    if: github.event_name == 'push' && github.ref == 'refs/heads/master'
    # login, metadata и build-push-action здесь
```

Это безопаснее для первого production-проекта, потому что зависимость job выражена явно.

Production Compose на VPS

На VPS создайте каталог, например `/opt/task-manager`, и поместите туда `compose.production.yaml` и закрытый `.env`:

```
services:
  api:
    image: ${IMAGE:?IMAGE is required}
    env_file: .env
    depends_on:
```

```
      postgres:        condition: service_healthy    ports:      # Reverse proxy на
том же сервере обращается к API локально.      - "127.0.0.1:8080:8080"    restart:
unless-stopped  postgres:    image: postgres:17-alpine    env_file: .env
environment:      POSTGRES_DB: ${POSTGRES_DB}      POSTGRES_USER: ${POSTGRES_USER}
POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}    volumes:      - postgres-
data:/var/lib/postgresql/data    healthcheck:      test: ["CMD-SHELL", "pg_isready -U
$$POSTGRES_USER -d $$POSTGRES_DB"]      interval: 5s      timeout: 3s      retries:
10    restart: unless-stoppedvolumes:  postgres-data:
```

В production у PostgreSQL **нет** `ports`: он доступен только в сети Compose. API опубликован лишь на `127.0.0.1`; внешний HTTPS-трафик принимает отдельный reverse proxy (например, Caddy или Nginx). TLS, домен и reverse proxy заслуживают отдельной настройки, поэтому не выдавайте HTTP-порт API напрямую в интернет как финальное решение.

Пример закрытого `/opt/task-manager/.env`:

```
POSTGRES_DB=task_manager
POSTGRES_USER=task_manager
POSTGRES_PASSWORD=replace-with-a-long-random-secret
DATABASE_URL=postgresql://task_manager:replace-with-a-long-random-
secret@postgres:5432/task_manager?sslmode=disable
PRIORITY_SERVICE_URL=https://priority.example.com
SHUTDOWN_TIMEOUT=10s
```

Файл принадлежит пользователю деплоя и имеет ограниченные права:

```
chmod 600 /opt/task-manager/.env
```

Не коммитьте production `.env`, не копируйте его в Docker image и не выводите командой `docker compose config` в публичные CI-логи: результат содержит подставленные secrets.

Первый деплой и проверка

На VPS сначала авторизуйтесь в GHCR. Для private image создайте отдельный токен только с `read:packages` и передайте его через stdin:

```
printf '%s' "$GHCR_READ_TOKEN" | docker login ghcr.io -u "$GHCR_USER" --password-
stdin
```

Затем разверните конкретный immutable image:

```
cd /opt/task-manager
export IMAGE=ghcr.io/danilchugaev/task-manager-api:sha-0123456789abcdef
```

```
docker compose -f compose.production.yaml pull apidocker compose -f
compose.production.yaml up -dcurl --fail --retry 10 --retry-connrefused --retry-delay
1 \  http://127.0.0.1:8080/healthz
```

`curl --fail` возвращает ненулевой код на `4xx`/`5xx`, поэтому команда подходит для проверки после delivery. Сохраняйте SHA-тег успешной версии в release notes или deploy-логе.

Откат

Откат должен использовать предыдущий известный рабочий immutable тег, а не `latest`:

```
cd /opt/task-manager
export IMAGE=ghcr.io/danilchugaev/task-manager-api:sha-previous-known-good
docker compose -f compose.production.yaml pull api
docker compose -f compose.production.yaml up -d --no-deps api

curl --fail http://127.0.0.1:8080/healthz
```

Откат образа не откатывает схему PostgreSQL. Поэтому миграции должны быть backward-compatible в течение времени, когда возможен rollback: например, сначала добавить nullable столбец, затем обновить приложение, а удалять старый столбец отдельным поздним релизом.

Автоматический deployment workflow

Когда ручной деплой понятен и проверен, его можно запускать из GitHub Actions. Не добавляйте ключ VPS в репозиторий. Создайте GitHub Secrets:

Secret	Назначение
`VPS_HOST`	hostname или IP сервера
`VPS_USER`	отдельный пользователь деплоя без root-логина
`VPS_SSH_PRIVATE_KEY`	приватный SSH-ключ только для deploy user
`VPS_KNOWN_HOSTS`	заранее проверенный fingerprint/known_hosts сервера

Фрагмент `.github/workflows/deploy.yml`:

```
  deploy:
    needs: publish
    runs-on: ubuntu-latest
    environment: production
    steps:
      - name: Configure SSH
        env:
          SSH_KEY: ${{ secrets.VPS_SSH_PRIVATE_KEY }}
          KNOWN_HOSTS: ${{ secrets.VPS_KNOWN_HOSTS }}
```

```
        run: |
          install -m 700 -d ~/.ssh
          printf '%s\n' "$SSH_KEY" > ~/.ssh/id_ed25519
          chmod 600 ~/.ssh/id_ed25519
          printf '%s\n' "$KNOWN_HOSTS" > ~/.ssh/known_hosts      - name: Deploy
immutable image        env:          HOST: ${{ secrets.VPS_HOST }}          USER: ${{
secrets.VPS_USER }}          IMAGE: ghcr.io/danilchugaev/task-manager-api:sha-${{
github.sha }}        run: |
          ssh "$USER@$HOST" "
            set -eu
            cd /opt/task-manager
            export IMAGE='$IMAGE'
            docker compose -f compose.production.yaml pull api
            docker compose -f compose.production.yaml up -d --no-deps api
            curl --fail --retry 10 --retry-connrefused --retry-delay 1
http://127.0.0.1:8080/healthz
          "
```

Environment `production` позволяет настроить required reviewers и отдельные environment secrets в GitHub. `VPS_KNOWN_HOSTS` важен: не заменяйте проверку хоста на `StrictHostKeyChecking=no`, иначе workflow становится уязвимым к подмене SSH-сервера.

Автоматический deploy не отменяет rollback-план. Перед включением workflow вручную проверьте: доступ VPS к GHCR, права deploy user на Docker, наличие `.env`, volume PostgreSQL, reverse proxy и health-check.

Частые ошибки

Заменить CI одним `go test ./...`

Так не проверятся `go vet`, гонки, миграции, SQL-привязка к PostgreSQL и Dockerfile. Разделите unit, integration и image build на явные jobs.

Передать production пароль в YAML workflow

```
env:
  DATABASE_URL: postgresql://user:real-password@host/database
```

Это секрет в истории Git. Используйте GitHub Secrets для CI/CD и закрытый `.env` с правами `600` на VPS.

Деплоить `latest`

`latest` меняется при каждом publish и не задаёт точную версию. Деплойте SHA-тег и сохраняйте предыдущий рабочий SHA для rollback.

Публиковать PostgreSQL наружу на VPS

```
ports:
  - "5432:5432"
```

Это увеличивает поверхность атаки. Для API и PostgreSQL достаточно внутренней Compose-сети; внешний доступ организуют только при осознанной необходимости и защите.

Откатывать код после несовместимой миграции

Предыдущий image может ожидать старую схему. Делайте миграции совместимыми в несколько этапов и планируйте их вместе с rollback.

Дальше

Task Manager теперь имеет CI, проверяемый image и понятный путь доставки на VPS. Последняя глава соберёт все части проекта вместе: структура репозитория, README, запуск, примеры API и итоговый чек-лист качества.

Самопроверка

1. За что отвечает CI в этой главе?

A. За хранение production-паролей в Git

B. Только за публикацию сайта на GitHub Pages

C. За автоматическую проверку кода, тестов, миграций и сборки образа

D. Только за открытие DBeaver

2. Зачем после go mod tidy в CI выполняют git diff --exit-code?

A. Чтобы выполнить миграции PostgreSQL

B. Чтобы опубликовать Docker image

C. Чтобы удалить go.mod

D. Чтобы проверить, что go.mod и go.sum уже актуальны в commit

3. Как в CI запускаются PostgreSQL integration-тесты?

A. С PostgreSQL service container и TEST_DATABASE_URL

B. Против production PostgreSQL

C. Без PostgreSQL и миграций

D. Через DBeaver на компьютере разработчика

4. Почему integration-тестам передают -count=1?

A. Чтобы не использовать кэш и действительно выполнить проверки против тестовой БД

B. Чтобы включить Docker daemon

C. Чтобы заменить race detector

D. Чтобы выполнить тест ровно один раз за весь проект

5. Какой тег образа лучше использовать для production-деплоя?

A. Тег без имени registry

B. Неизменяемый SHA commit

C. Только latest

D. Случайный локальный tag

6. Какое разрешение GitHub Actions требуется для публикации образа в GHCR?

A. actions: admin

B. packages: write

C. checks: none

D. issues: write

7. Почему PostgreSQL не публикуют через ports на production VPS по умолчанию?

A. Так невозможно использовать volumes

B. Базе достаточно внутренней сети Compose, а открытый порт увеличивает поверхность атаки

C. API не умеет подключаться по hostname postgres

D. PostgreSQL не поддерживает Docker

8. Что должен проверить deployment после docker compose up?

A. Health-check API, например curl --fail /healthz

B. Только наличие Dockerfile

C. Только git status на VPS

D. Только открытие веб-браузера

9. Почему для SSH workflow нужен заранее проверенный VPS_KNOWN_HOSTS?

A. Чтобы GitHub автоматически создал PostgreSQL

B. Чтобы Docker быстрее собирал image

C. Чтобы проверять подлинность VPS и не отключать защиту SSH

D. Чтобы скрыть health-check

10. Почему откат image не всегда откатывает приложение полностью?

A. GHCR не хранит историю образов

B. Docker не поддерживает старые теги

C. go test удаляет все images

D. Схема PostgreSQL могла измениться миграцией и остаться новой

Ответы к самопроверке

1. За автоматическую проверку кода, тестов, миграций и сборки образа

2. Чтобы проверить, что go.mod и go.sum уже актуальны в commit

3. С PostgreSQL service container и TEST_DATABASE_URL

4. Чтобы не использовать кэш и действительно выполнить проверки против тестовой БД

5. Неизменяемый SHA commit

6. packages: write

7. Базе достаточно внутренней сети Compose, а открытый порт увеличивает поверхность атаки

8. Health-check API, например curl --fail /healthz

9. Чтобы проверять подлинность VPS и не отключать защиту SSH

10. Схема PostgreSQL могла измениться миграцией и остаться новой

Задания: доставка Task Manager API

Настройте GitHub Actions и delivery-процесс для финального Task Manager API. Выполняйте задания в отдельном репозитории API, а не в репозитории учебника: этот сайт не содержит Go-модуль и не должен пытаться запускать Go CI.

Задание 1. Создайте базовый CI workflow

Создайте `.github/workflows/ci.yml`, который запускается на `push` в `master` и `pull_request`. Добавьте job с checkout, `setup-go`, `go mod tidy`, проверкой diff, `go vet`, `go test -race` и `go build ./cmd/api`.

Критерии проверки:

- workflow имеет минимальное `contents: read` разрешение;
- версия Go читается из `go.mod`;
- cache Go включён;
- `go mod tidy` не меняет незакоммиченные module-файлы;
- `go vet ./...` и `go test -race ./...` проходят;
- сборка API проверяется до merge.

Подсказка

```
- name: Check module files
  run: |
    go mod tidy
    git diff --exit-code -- go.mod go.sum
```

Задание 2. Добавьте PostgreSQL integration job

Добавьте service container `postgres:17-alpine`, тестовую БД `task_manager_test`, `TEST_DATABASE_URL`, установку Goose, применение миграций и запуск integration-тестов с `-race`, `-count=1` и `-tags=integration`.

Критерии проверки:

- service container имеет health-check через `pg_isready`;
- база называется `task_manager_test`;
- URL передаётся через `TEST_DATABASE_URL`, а не `DATABASE_URL`;
- Goose применяет миграции перед тестами;
- integration job не использует production secrets;
- тесты не запускаются против development-базы.

Подсказка

```
services:
  postgres:
    image: postgres:17-alpine
    env:
      POSTGRES_DB: task_manager_test
```

Задание 3. Проверяйте Docker-артефакты

Добавьте job `docker`, зависящий от unit job. Он запускает `docker compose config` и собирает image с тегом `task-manager-api:ci`.

Критерии проверки:

- job использует `needs: unit`;
- YAML Compose валидируется до сборки;
- Docker build использует production Dockerfile;
- image build не публикуется из pull request;
- failure Dockerfile делает CI красным.

Задание 4. Публикуйте проверенный image в GHCR

Добавьте publish job в тот же workflow после `unit`, `integration` и `docker`. Войдите в `ghcr.io` через `GITHUB_TOKEN`, публикуйте `latest` и SHA-тег, но для deployment используйте только SHA.

Критерии проверки:

- workflow имеет `packages: write` только там, где это необходимо;
- publish идёт только после успешных CI jobs;
- image name полностью в lowercase;
- `docker/login-action` не получает пароль в открытом тексте;
- SHA-tag записан в логи релиза или deployment;
- pull request не публикует production image.

Подсказка

```
publish:
  needs: [unit, integration, docker]
  if: github.event_name == 'push' && github.ref == 'refs/heads/master'
```

Задание 5. Подготовьте VPS Compose

На VPS создайте `/opt/task-manager` с `compose.production.yaml` и закрытым `.env`. API должен получать image через `${IMAGE}`, PostgreSQL — использовать named volume без published `ports`, а API — слушать только `127.0.0.1:8080` для reverse proxy.

Критерии проверки:

- production `.env` не находится в Git;
- права `.env` — `600`;
- database password отличается от учебного локального примера;
- PostgreSQL не имеет `ports`;
- API использует `restart: unless-stopped`;
- PostgreSQL имеет health-check;
- `docker compose config` на VPS проходит без ошибок.

Задание 6. Выполните deploy, проверку и rollback

Войдите на VPS в GHCR токеном только с `read:packages`. Разверните SHA-tag через `docker compose pull/up`, проверьте `/healthz`, затем выполните учебный rollback на предыдущий известный SHA-tag. После ручной проверки создайте deployment workflow с GitHub Secrets и `VPS_KNOWN_HOSTS`.

Критерии проверки:

- deploy использует immutable SHA-tag;
- `/healthz` проверяется через `curl --fail`;
- rollback не использует `latest`;
- команда rollback проверена до аварийной ситуации;
- SSH private key хранится только в GitHub Secret;
- проверка SSH host key включена;
- secrets не попадают в workflow output и shell history.

Проверьте себя

Перед итоговой главой убедитесь, что вы можете:

- объяснить разницу CI, delivery и deployment;
- запустить unit, race и PostgreSQL integration-проверки в GitHub Actions;
- собрать и опубликовать image в GHCR только после CI;
- безопасно передать secrets и проверить SSH-host;
- развернуть SHA-tag на VPS, проверить `/healthz` и откатиться;
- объяснить, почему migrations нужно планировать вместе с rollback.

Рекомендации по коду: CI/CD и деплой

- Автоматизируйте в CI уже работающие локально команды, а не заменяйте ими локальную диагностику.
- Разделяйте unit, integration и Docker build jobs: каждый подтверждает отдельный риск.
- Запускайте integration-тесты только на ephemeral service container и отдельной `*_test` базе.
- Давайте GitHub Actions минимально необходимые permissions: `contents: read` по умолчанию, `packages: write` только publish job.
- Публикуйте и деплойте immutable image по SHA; `latest` оставляйте только как удобную ссылку.
- Никогда не храните passwords, registry tokens и SSH keys в Git, Dockerfile или открытых workflow env.
- На VPS проверяйте SSH host key через known_hosts и используйте отдельного deploy user, а не root.
- Не публикуйте PostgreSQL наружу без отдельной обоснованной и защищённой сетевой конфигурации.

- Проверяйте deployment через быстрый `/healthz`; сохраняйте предыдущий рабочий SHA и регулярно проверяйте rollback.
- Делайте миграции backward-compatible, пока возможен rollback на предыдущий image.

Чек-лист

Каждый merge проходит статический анализ, unit/race и PostgreSQL integration-тесты, Dockerfile собирается, production получает конкретный SHA image из GHCR, secrets изолированы, deployment проверяется health-check, а rollback не зависит от `latest` и не ломается из-за несовместимой схемы.

Итоговая интеграция

Цель урока

Собрать все части курса в один завершённый Task Manager API: документировать структуру проекта и запуск, проверить PostgreSQL-миграции, выполнить HTTP-сценарии через Gin, убедиться в качестве кода, Docker-окружения и CI/CD, а затем составить осознанный план развития проекта.

Эта глава не добавляет новую библиотеку. Её цель — превратить набор отдельных тем в работающий backend-проект, который можно передать другому разработчику: он сможет клонировать репозиторий, поднять окружение, применить миграции, вызвать API и понять, как проверить изменения.

Что получилось

Итоговый Task Manager API умеет:

- принимать и возвращать JSON через Gin;
- создавать, получать, завершать и удалять задачи;
- проверять входные данные и возвращать предсказуемые HTTP-статусы;
- передавать request context в service, внешний HTTP-клиент и PostgreSQL;
- писать структурированные логи, request ID и access log;
- хранить задачи в PostgreSQL;
- применять схему БД через Goose-миграции;
- запускаться в Docker Compose;
- корректно завершаться по `SIGINT` / `SIGTERM`;
- проходить unit, race и PostgreSQL integration-тесты;
- собираться в Docker image и доставляться через CI/CD.

Итоговая архитектура

Разложите исходный код по обязанностям, а не по типам файлов. Один из возможных итоговых вариантов:

```
task-manager-api/
├── cmd/
│   └── api/
│       └── main.go                # сборка зависимостей и жизненный цикл
├── internal/
│   ├── config/                    # env и валидация Config
│   ├── task/                      # Task, CreateInput, доменные правила
│   ├── taskservice/               # бизнес-сценарии и интерфейсы зависимостей
```

```
│   ├── postgresrepo/              # SQL-реализация TaskRepository│   ├──
priorityclient/            # Resty-клиент внешнего API│   └── ginapi/
# Gin router, handlers, middleware├── migrations/                    # Goose SQL-
миграции├── Dockerfile├── compose.yaml                    # локальная разработка├──
compose.production.yaml        # VPS-окружение├── .env.example├── go.mod├──
go.sum└── README.md
```

Направление зависимостей должно оставаться таким:

```
Gin handler → TaskService → TaskRepository interface
                                  ↓
                          PostgreSQL repository

TaskService → PrioritySuggester interface
                         ↓
                  Resty client
```

`ginapi`, `postgresrepo` и `priorityclient` — детали доставки и инфраструктуры. Пакет `taskservice` не должен импортировать Gin, Resty, `database/sql` или конкретный PostgreSQL-драйвер. Благодаря этому unit-тесты service остаются быстрыми и не требуют HTTP-сервера либо БД.

Контракт HTTP API

Зафиксируйте в README текущий контракт. В учебной версии он может выглядеть так:

Метод	Путь	Описание	Успех
`GET`	`/healthz`	Быстрая проверка работы HTTP-сервера	`200`
`GET`	`/tasks`	Список задач	`200`
`POST`	`/tasks`	Создать задачу	`201`
`POST`	`/tasks/:id/complete`	Отметить задачу выполненной	`200`
`DELETE`	`/tasks/:id`	Удалить задачу	`204`

Единый формат ошибки помогает клиентам API:

```
{
  "error": "задача не найдена"
}
```

Базовое соответствие статусов:

Ситуация	Статус
Некорректный JSON, ID или входные данные	`400 Bad Request`

Задача не найдена	`404 Not Found`
Задача создана	`201 Created`
Успешное удаление без body	`204 No Content`
Неожиданная внутренняя ошибка	`500 Internal Server Error`

Не меняйте статусы или формат ошибки «по ходу» без обновления тестов и README: для внешнего клиента это публичный контракт.

README проекта

README — первая точка входа. Он должен позволять запустить проект без личных объяснений автора. Минимальная структура:

```
# Task Manager API

Краткое описание: JSON API для управления задачами на Go, Gin и PostgreSQL.

## Возможности

- создание, просмотр, завершение и удаление задач;
- PostgreSQL и Goose-миграции;
- Docker Compose для локального запуска;
- health-check и graceful shutdown.

## Требования

- Go версии из go.mod;
- Docker Desktop / Docker Engine и Docker Compose.

## Быстрый запуск

...

## Конфигурация

...

## Миграции

...

## API

...
```

```
## Проверки...
```

Укажите настоящие команды и реальные имена переменных окружения именно вашего репозитория. README с командами, которые не запускались после последнего изменения, вводит в заблуждение сильнее, чем отсутствующий README.

Быстрый локальный запуск

Ниже — рекомендуемый workflow для проекта с Docker Compose:

```
git clone <URL-вашего-репозитория>
cd task-manager-api

cp .env.example .env
docker compose up --build -d
docker compose ps
```

Проверьте API:

```
curl -i http://localhost:8080/healthz
```

Если в Compose API применяет миграции во время старта, после `healthy` можно сразу вызывать `/tasks`. Если миграции запускаются отдельной командой, явно задокументируйте её:

```
goose -dir migrations postgres "$DATABASE_URL" up
```

Для команды на компьютере разработчика `DATABASE_URL` использует `localhost`; внутри API-container — имя Compose-сервиса `postgres`. Не смешивайте эти две строки подключения.

Остановка окружения, сохраняющая данные БД:

```
docker compose down
```

Полный сброс local state:

```
docker compose down -v
```

Вторая команда удаляет named volume PostgreSQL. Укажите это предупреждение прямо в README.

Примеры HTTP-запросов

Health-check

```
curl -i http://localhost:8080/healthz
```

Ожидается `200 OK`:

```
{"status":"ok"}
```

Создание задачи

```
curl -i -X POST http://localhost:8080/tasks \
  -H 'Content-Type: application/json' \
  -H 'X-Request-ID: demo-create-1' \
  -d '{"title":"Подготовить README","priority":2}'
```

Ожидается `201 Created` и задача с ID, временными метками и `completed: false`:

```
{
  "id": 1,
  "title": "Подготовить README",
  "priority": 2,
  "completed": false,
  "created_at": "2026-10-14T12:00:00Z",
  "updated_at": "2026-10-14T12:00:00Z"
}
```

Сохраните ID в shell-переменную, чтобы не подставлять его вручную:

```
TASK_ID=1
```

Список задач

```
curl -i http://localhost:8080/tasks
```

Успешный пустой список — это JSON-массив, а не `null`:

```
[]
```

Завершение задачи

```
curl -i -X POST "http://localhost:8080/tasks/${TASK_ID}/complete"
```

Удаление задачи

```
curl -i -X DELETE "http://localhost:8080/tasks/${TASK_ID}"
```

При `204 No Content` body отсутствует — не пытайтесь декодировать JSON-ответ там, где контракт его не обещает.

Ошибочный запрос

```
curl -i -X POST http://localhost:8080/tasks \
  -H 'Content-Type: application/json' \
  -d '{"title":"   ","priority":0}'
```

Ожидается `400 Bad Request` и безопасное JSON-сообщение. Текст SQL-ошибки, DSN, stack trace или panic не должны попадать в response.

Миграции и DBeaver

После старта PostgreSQL примените миграции к development-базе и проверьте их состояние:

```
goose -dir migrations postgres "$DATABASE_URL" status
goose -dir migrations postgres "$DATABASE_URL" up
```

В DBeaver подключайтесь к PostgreSQL с хоста через `localhost:${POSTGRES_PORT}`. После миграции проверьте:

```
Databases → task_manager → Schemas → public → Tables
```

Там должны быть `tasks` и `goose_db_version`. Используйте DBeaver для `SELECT` и диагностики, а изменения схемы записывайте только новыми Goose-миграциями.

Проверки качества перед merge

Перед pull request выполните из корня API:

```
go mod tidy
git diff --exit-code -- go.mod go.sum
go vet ./...
go test ./...
```

```
go test -race ./...docker compose configdocker build -t task-manager-api:local .
```

Если поднята отдельная тестовая PostgreSQL-база:

```
export TEST_DATABASE_URL='postgresql://task_manager:local-
password@localhost:5432/task_manager_test?sslmode=disable'
goose -dir migrations postgres "$TEST_DATABASE_URL" up
go test -race -count=1 -tags=integration ./internal/postgresrepo/...
```

Проверьте, что URL действительно ведёт в `*_test` базу. Не запускайте destructive integration-тесты против development или production БД.

Итоговый checklist

Код и архитектура

- [ ] `main` собирает зависимости, а не содержит бизнес-логику.
- [ ] Gin используется только в HTTP-слое.
- [ ] Service зависит от интерфейсов repository и внешнего клиента.
- [ ] SQL находится в PostgreSQL-репозитории и использует `$1`, `$2`.
- [ ] Каждый внешний вызов и SQL-операция получает `context.Context`.
- [ ] Ошибки не сравниваются по тексту; используются `errors.Is` и `errors.As`.

HTTP API

- [ ] Маршруты, статусы и JSON-форматы описаны в README.
- [ ] Невалидный ввод возвращает `400`, отсутствующий ресурс — `404`.
- [ ] Внутренние ошибки логируются, но не раскрывают секреты и SQL-детали клиенту.
- [ ] Есть `/healthz`, request ID, access log и recovery middleware.

Данные и конфигурация

- [ ] Схема БД изменяется только Goose-миграциями.
- [ ] `DATABASE_URL`, пароли и токены не лежат в Git или логах.
- [ ] `.env.example` содержит безопасные примеры, а `.env` находится в `.gitignore`.
- [ ] PostgreSQL использует named volume локально и не опубликован наружу в production.

Проверки и доставка

- [ ] Unit, race и PostgreSQL integration-тесты проходят.
- [ ] Dockerfile собирается; `docker compose config` валиден.
- [ ] CI запускает обязательные проверки на pull request.
- [ ] Production получает immutable SHA image, а не только `latest`.
- [ ] После deploy проверяется `/healthz`, известен предыдущий рабочий SHA и способ rollback.

Демонстрация проекта

Перед показом проекта другому человеку начните с чистого окружения или отдельного Docker volume. Пройдите сценарий без ручных исправлений:

1. клонировать проект;
2. создать `.env` из `.env.example`;
3. поднять Compose;
4. применить миграции, если они не запускаются автоматически;
5. вызвать health-check;
6. создать, получить, завершить и удалить задачу;
7. посмотреть изменения в DBeaver;
8. запустить проверки качества;
9. остановить окружение, не удаляя volume.

Если один из шагов требует устной подсказки автора, добавьте её в README или автоматизируйте. Это лучший способ заметить недостающую документацию.

Куда развивать проект

Рабочий CRUD API — хорошая база, но не повод добавлять всё сразу. Выберите следующую задачу по реальной потребности и спроектируйте контракт, миграции, тесты и наблюдаемость до реализации.

Аутентификация и авторизация

Добавьте пользователей, безопасное хранение паролей, выдачу/проверку токенов и middleware авторизации. До реализации определите модель прав: кто видит и меняет какую задачу. Не ограничивайтесь проверкой, что заголовок `Authorization` просто существует.

Фильтрация, сортировка и пагинация

Для большого списка задач добавьте query-параметры, например `completed`, `priority`, `limit` и cursor. Все динамические поля сортировки выбирайте из whitelist; лимит ограничивайте сверху. Для стабильной пагинации определите порядок, например `id ASC`.

Метрики и tracing

Логи помогают расследовать событие, но не показывают общую картину. Следующий шаг — latency, число запросов, ошибки и состояние пула БД через metrics. Для запросов через несколько сервисов добавляют distributed tracing. Не записывайте в labels высококардинальные значения вроде ID задачи или request ID.

Rate limiting

Защитите API от чрезмерного числа запросов. Определите единицу лимита — IP, пользователь, API-key или endpoint — и ответ `429 Too Many Requests`. Лимитер в памяти не подойдёт для

нескольких экземпляров API без общей координации.

Другие возможные улучшения

- soft delete и восстановление задачи;
- теги, дедлайны и напоминания;
- фоновая обработка и outbox-паттерн;
- OpenAPI-спецификация и генерация клиента;
- backup и восстановление PostgreSQL;
- более строгие production политики образов и supply-chain security.

Итог курса

Вы прошли путь от первых переменных и функций Go до backend-приложения с HTTP API, PostgreSQL, Docker и CI/CD. Самое ценное не конкретный список библиотек, а инженерный цикл: сформулировать контракт, отделить слои, написать тесты, проверить реальную инфраструктуру, наблюдать работу и безопасно доставить изменение.

Дальше можно переходить к продвинутым темам Go или развивать Task Manager по одному осмысленному улучшению за раз.

Самопроверка

1. Какое направление зависимостей сохраняет итоговая архитектура?

A. DBeaver вызывает HTTP handler

B. Task напрямую импортирует Gin и Resty

C. PostgreSQL repository → Gin handler → service

D. Gin handler → service → интерфейс repository → PostgreSQL-реализация

2. Что должен содержать README, чтобы другой разработчик запустил API?

A. Только скриншот DBeaver

B. Только название проекта

C. Требования, конфигурацию, команды запуска, миграций, проверок и API-примеры

D. Пароли production-базы

3. Какой статус успешного DELETE без response body?

A. 201 Created

B. 500 Internal Server Error

C. 204 No Content

D. 400 Bad Request

4. Что нельзя делать после ответа 204 No Content?

A. Логировать факт удаления на сервере

B. Пытаться декодировать обещанный JSON-body, которого нет

C. Завершить curl-команду

D. Проверить HTTP-статус

5. Почему DBeaver не заменяет Goose-миграции?

A. Миграции нужны только в браузере

B. Goose не работает с SQL

C. Ручное изменение через UI не хранится в Git и не воспроизводится в других средах

D. DBeaver не умеет подключаться к PostgreSQL

6. Какой образ нужно передавать в production deployment?

A. Локальный image без registry

B. Image без тега

C. Immutable image с SHA commit

D. Только latest

7. Какую базу должны использовать destructive PostgreSQL integration-тесты?

A. Development DATABASE_URL

B. Production database

C. Базу, выбранную в DBeaver вручную

D. Отдельную test-БД из TEST_DATABASE_URL

8. Что должно произойти после deployment?

A. Удаление PostgreSQL volume

B. Отключение CI

C. Публикация .env в Git

D. Проверка /healthz и сохранение информации о рабочем SHA для rollback

9. Почему фильтр и сортировку нельзя без проверки подставлять в SQL?

A. Параметры $1 работают для имён столбцов

B. Gin автоматически делает это безопасным

C. PostgreSQL запрещает ORDER BY

D. Имена SQL-столбцов и направления сортировки нужно выбирать из whitelist

10. Какой подход лучше для дальнейшего развития Task Manager?

A. Заменить PostgreSQL на ручной JSON без причины

B. Добавить все возможные функции в одном commit

C. Выбирать одно обоснованное улучшение и сначала проектировать контракт, миграции, тесты и наблюдаемость

D. Сразу удалить unit-тесты

Ответы к самопроверке

1. Gin handler → service → интерфейс repository → PostgreSQL-реализация

2. Требования, конфигурацию, команды запуска, миграций, проверок и API-примеры

3. 204 No Content

4. Пытаться декодировать обещанный JSON-body, которого нет

5. Ручное изменение через UI не хранится в Git и не воспроизводится в других средах

6. Immutable image с SHA commit

7. Отдельную test-БД из TEST_DATABASE_URL

8. Проверка /healthz и сохранение информации о рабочем SHA для rollback

9. Имена SQL-столбцов и направления сортировки нужно выбирать из whitelist

10. Выбирать одно обоснованное улучшение и сначала проектировать контракт, миграции, тесты и наблюдаемость

Задания: выпуск Task Manager API

Завершите Task Manager API как самостоятельный backend-проект. Цель — не добавить новую технологию, а доказать, что проект воспроизводимо запускается, имеет проверяемый HTTP-контракт и готов к безопасному дальнейшему развитию.

Задание 1. Проведите ревизию структуры

Сопоставьте реальные каталоги проекта с итоговой архитектурой. Вынесите код из `main`, если там остались handlers, SQL-запросы или бизнес-правила. Убедитесь, что Gin, Resty и PostgreSQL не проникают в доменную модель и service.

Критерии проверки:

- `cmd/api/main.go` собирает зависимости и запускает приложение;
- Gin находится в HTTP-пакете;
- SQL находится в PostgreSQL-репозитории;
- service зависит от интерфейсов;
- доменные типы не импортируют Gin, Resty и `database/sql`;
- package names объясняют назначение кода.

Задание 2. Напишите README с нуля

Попросите коллегу или самого себя в чистом каталоге выполнить README без устных подсказок. Включите требования, `.env.example`, Docker Compose, миграции, тесты и таблицу API.

Критерии проверки:

- README объясняет назначение проекта одной-двумя фразами;
- перечислены минимальные требования;
- есть команды copy `.env.example`, `docker compose up`, migrations и health-check;
- описаны все переменные без настоящих секретов;
- есть API-таблица с методами, путями и статусами;
- есть отдельное предупреждение для `docker compose down -v`.

Подсказка

Проверьте README в новом shell: не используйте уже экспортированные переменные и локально запомненные шаги.

Задание 3. Закрепите HTTP-контракт

Добавьте или обновите handler-тесты для `GET /healthz`, `GET /tasks`, `POST /tasks`, `POST /tasks/:id/complete` и `DELETE /tasks/:id`. Проверьте успешные ответы, `400`, `404`, формат error JSON и `204` без body.

Критерии проверки:

- tests используют `httptest`, а не реальный порт;
- каждый маршрут проверяет корректный HTTP-статус;
- error response соответствует одной структуре;
- тест удаления проверяет `204` и пустое body;
- не найденная задача возвращает `404`;
- request ID middleware не ломает HTTP-контракт.

Подсказка

```
if recorder.Code != http.StatusNoContent {
  t.Fatalf("status = %d, ожидался %d", recorder.Code, http.StatusNoContent)
}
if recorder.Body.Len() != 0 {
  t.Fatalf("204 не должен содержать body: %q", recorder.Body.String())
}
```

Задание 4. Пройдите сценарий API вручную

Поднимите проект на чистом Docker volume, примените миграции и выполните сценарий: health-check → create → list → complete → delete. Посмотрите таблицу `tasks` и `goose_db_version` через DBeaver.

Критерии проверки:

- Docker Compose показывает healthy PostgreSQL;
- `curl` создаёт задачу с `201`;
- `GET /tasks` возвращает JSON-массив;
- complete меняет `completed` и `updated_at`;
- delete возвращает `204`;
- DBeaver показывает те же данные, что API;
- миграции не создавались вручную только через UI.

Задание 5. Выполните release-проверки

Запустите весь список проверок перед release. Исправьте все предупреждения, а не отключайте проверки.

```
go mod tidy
git diff --exit-code -- go.mod go.sum
go vet ./...
go test ./...
go test -race ./...
docker compose config
docker build -t task-manager-api:release .
```

При доступной test-БД добавьте integration suite.

Критерии проверки:

- module-файлы не меняются после tidy;
- unit и race tests проходят;
- integration tests используют только `TEST_DATABASE_URL`;
- Dockerfile собирается;
- Compose YAML валиден;
- GitHub Actions проходит для pull request;
- нет незакоммиченных `.env`, secrets, бинарников или локальных volume-данных.

Задание 6. Подготовьте первое улучшение

Выберите **одно** направление: авторизация, фильтрация/пагинация, метрики или rate limiting. До кода создайте короткий design note в issue, README или `docs/`:

1. проблема пользователя;
2. HTTP-контракт;
3. изменения модели и миграций;
4. правила доступа или ограничения;
5. unit, integration и handler-тесты;
6. логи, метрики и rollout/rollback-риски.

Критерии проверки:

- выбран ровно один небольшой scope;
- контракт описан до реализации;
- возможная миграция совместима с rollback;
- нет SQL-конкатенации для фильтров и сортировки;
- определено, какие данные нельзя логировать;
- есть понятный критерий готовности изменения.

Проверьте себя

Вы завершили основной курс, если можете:

- с чистого окружения запустить проект только по README;
- объяснить границы Gin, service, repository и внешнего клиента;
- выполнить и проверить HTTP CRUD-сценарий;
- работать с PostgreSQL через миграции, integration tests и DBeaver;
- собрать image, пройти CI и задеплоить SHA-версию;
- выбрать следующее улучшение и спроектировать его до кода.

Рекомендации по коду: итоговый проект

- Держите `main` тонким: он загружает config, собирает зависимости и управляет жизненным циклом.
- Сохраняйте направление зависимостей от HTTP и инфраструктуры к service и интерфейсам, а не наоборот.
- Считайте README частью продукта: обновляйте его в том же pull request, что и команды, env или HTTP-контракт.
- Закрепляйте публичные статусы и JSON-форматы handler-тестами.
- Меняйте схему PostgreSQL только миграциями, а не вручную через DBeaver.
- Перед merge запускайте vet, unit, race, integration tests, Compose validation и Docker build.
- Деплойте проверенный SHA image, проверяйте `/healthz` и храните известный рабочий тег для rollback.

- Не добавляйте новые функции без контракта, миграционного плана, тестов и наблюдаемости.
- Не усложняйте проект заранее: выбирайте одно улучшение, которое решает понятную пользовательскую проблему.

Чек-лист

Task Manager API запускается в чистом окружении по README, имеет разделённые слои, тестируемый HTTP-контракт, воспроизводимую PostgreSQL-схему, безопасное Docker/CI/CD-окружение и понятный план дальнейшего развития.

Продвинутый Go A: ошибки и управление ресурсами

Цель главы

Научиться проектировать ошибки как часть контракта программы, безопасно освобождать ресурсы, осознанно ограничивать `panic` и `recover`, а также переводить внутренние причины в стабильные ошибки HTTP API.

Этот блок рассчитан на читателя, который уже завершил Task Manager API. Мы не повторяем базовый паттерн `if err != nil`: теперь важны границы между слоями приложения и последствия решений для вызывающего кода, логов, тестов и публичного API.

Карта главы

1. **Эта страница** — цепочки ошибок, собственные типы, `errors.As` и `errors.Join`.
2. `defer` и ресурсы — порядок очистки, ошибки `Close`, транзакции и циклы.
3. `panic` и `recover` в библиотечном коде — когда такая граница оправдана, а когда она скрывает дефект.
4. Ошибки HTTP API — перевод доменных ошибок в HTTP-ответы, один осмысленный лог и защита публичного контракта.

Ошибка — это контракт, а не только текст

У ошибки обычно есть два получателя:

- **человек** читает `Error()` в логе или в диагностике;
- **код** выбирает действие: повторить запрос, показать поле формы, вернуть `404`, завершить процесс.

Поэтому полезно разделять три вещи:

```
контекст операции  → «сохранить задачу 42»
машинная причина   → sql.ErrNoRows, ErrTaskNotFound, *ValidationError
сообщение клиенту  → «задача не найдена»
```

Текст ошибки нужен для диагностики, но он не должен быть единственным API между пакетами. Если вызывающему коду важно распознать ситуацию, дайте ему стабильную причину для `errors.Is` или структурированные данные для `errors.As`.

Цепочка обёрнутых ошибок

Каждый слой добавляет только тот контекст, который ему известен:

```
func (repository *Repository) Get(ctx context.Context, id int64) (Task, error) {
  row := repository.db.QueryRowContext(ctx, getTaskQuery, id)

  var task Task
  if err := row.Scan(&task.ID, &task.Title); err != nil {
    return Task{}, fmt.Errorf("прочитать задачу %d: %w", id, err)
  }

  return task, nil
}

func (service *Service) Complete(ctx context.Context, id int64) (Task, error) {
  task, err := service.repository.Get(ctx, id)
  if err != nil {
    return Task{}, fmt.Errorf("получить задачу для завершения: %w", err)
  }

  // ...
  return task, nil
}
```

Если `Scan` вернул `sql.ErrNoRows`, итоговый текст будет содержать оба контекста. Главное — `%w` сохраняет связь между ошибками. `errors.Is` и `errors.As` пройдут по этой связи без ручного разбора текста.

```
task, err := service.Complete(ctx, 42)
if errors.Is(err, sql.ErrNoRows) {
  // Техническая причина найдена внутри цепочки.
}
```

В реальном приложении HTTP-слой не должен зависеть от `sql.ErrNoRows`; эту проблему разберём на странице об HTTP API. Пример выше показывает именно механику цепочки.

> **Оборачивать — значит раскрывать зависимость**
>
> Когда пакет возвращает `fmt.Errorf("...: %w", sql.ErrNoRows)`, его вызывающий код получает возможность проверять `sql.ErrNoRows`. Иногда это нужно, но чаще связывает верхний слой с деталями базы данных. На границе repository лучше преобразовать техническую причину в доменную ошибку, например `ErrTaskNotFound`.

`errors.Is`: известная причина

`errors.Is` отвечает на вопрос: «есть ли в цепочке именно эта причина?» Он подходит для ошибок-маркеров и стандартных значений:

```
var ErrTaskNotFound = errors.New("задача не найдена")

func (repository *Repository) Get(ctx context.Context, id int64) (Task, error) {
  var task Task

  // ...
  if errors.Is(err, sql.ErrNoRows) {
    return Task{}, ErrTaskNotFound
  }
  if err != nil {
    return Task{}, fmt.Errorf("получить задачу %d: %w", id, err)
  }

  return task, nil
}
```

Теперь service и HTTP handler могут проверять `ErrTaskNotFound`, не зная, PostgreSQL, файл или другой источник данных используется внутри.

Не создавайте новый `errors.New("задача не найдена")` при каждом возврате: это другое значение, и `errors.Is` его не узнает. Сравнение `err.Error()` со строкой ещё хрупче — текст можно уточнить, локализовать или дополнить контекстом.

Собственные типы ошибок

Ошибка-маркер сообщает только факт. Иногда вызывающему коду нужны данные: имя поля, нарушенное правило, лимит, задержка повторной попытки или идентификатор внешнего сервиса. Для этого создают тип ошибки.

Ниже `ValidationError` хранит детали некорректного поля и при этом оборачивает общую причину `ErrInvalidInput`:

```
var ErrInvalidInput = errors.New("некорректные входные данные")

type ValidationError struct {
  Field string
  Rule  string
  Value string
}

func (err *ValidationError) Error() string {
```

```
  return fmt.Sprintf("поле %q не прошло правило %q", err.Field, err.Rule)}func (err *ValidationError) Unwrap() error {  return ErrInvalidInput}func validateTitle(title string) error {   title = strings.TrimSpace(title)  if title == "" {    return &ValidationError{     Field: "title",       Rule:  "required",    } } if utf8.RuneCountInString(title) > 120 {   return &ValidationError{       Field: "title",     Rule:  "max_length",       Value: strconv.Itoa(utf8.RuneCountInString(title)),     } } return nil}
```

У типа есть три полезных свойства:

- `Error()` даёт человекочитаемую диагностику;
- поля дают коду структурированные данные, не требуя разбора строки;
- `Unwrap()` делает одновременно верными две проверки: конкретного типа и общей причины.

```
err := validateTitle("")

fmt.Println(errors.Is(err, ErrInvalidInput)) // true

var validationErr *ValidationError
if errors.As(err, &validationErr) {
  fmt.Println(validationErr.Field) // title
  fmt.Println(validationErr.Rule)  // required
}
```

`errors.As`: получить тип и его данные

`errors.As` ищет в цепочке ошибку, которую можно присвоить указанному типу. Второй аргумент — **указатель на переменную для результата**, поэтому для `*ValidationError` получается `**ValidationError`:

```
var validationErr *ValidationError
if errors.As(err, &validationErr) {
  // validationErr уже содержит найденную ошибку.
  return validationErr.Field
}
```

Можно искать и интерфейс. Например, стандартный `net.Error` сообщает, что сетевая ошибка временная или связана с timeout:

```
var networkErr net.Error
if errors.As(err, &networkErr) && networkErr.Timeout() {
```

```
  // Политика повторной попытки зависит от операции.}
```

Не используйте `errors.As` только потому, что тип существует. Если вызывающему коду достаточно вопроса «это отсутствие задачи?», проще и понятнее `errors.Is(err, ErrTaskNotFound)`. Тип вводят тогда, когда его поля действительно становятся частью контракта.

Указатель, значение и ловушка typed `nil`

В примере `ValidationError` возвращается указателем. Это позволяет не копировать структуру и делает стиль единообразным. Но интерфейс `error` состоит из динамического типа и значения. Интерфейс с типом `*ValidationError` и значением `nil` **не равен** `nil`:

```
func invalidReturn() error {
  var validationErr *ValidationError
  return validationErr // error содержит тип *ValidationError, поэтому он не nil
}

func main() {
  if err := invalidReturn(); err != nil {
    fmt.Println("неожиданная ошибка")
  }
}
```

При отсутствии ошибки всегда возвращайте явный `nil`:

```
func validatePriority(priority int) error {
  if priority < 1 || priority > 5 {
    return &ValidationError{Field: "priority", Rule: "range"}
  }

  return nil
}
```

Несколько причин: `errors.Join`

Начиная с Go 1.20, `errors.Join` объединяет независимые ошибки. Это полезно, когда операция выполнила несколько обязательных действий и нужно не потерять ни одну неудачу: например, основная запись не удалась, а закрытие ресурса тоже вернуло ошибку.

```
func validateCreateInput(title string, priority int) error {
  var errs []error

  if err := validateTitle(title); err != nil {
```

```
    errs = append(errs, err)   } if err := validatePriority(priority); err != nil {
errs = append(errs, err)   } return errors.Join(errs...)}
```

`errors.Join()` возвращает `nil`, если все аргументы равны `nil` или аргументов нет. Для объединённой ошибки работают `errors.Is` и `errors.As`:

```
err := validateCreateInput("", 10)

fmt.Println(errors.Is(err, ErrInvalidInput)) // true

var validationErr *ValidationError
fmt.Println(errors.As(err, &validationErr)) // true
```

Но первый найденный `ValidationError` — не полноценный список ошибок для JSON-ответа. Если API должно сообщать обо всех проблемных полях, храните их в явной структуре ответа или возвращайте отдельную доменную коллекцию. Не пытайтесь строить публичный контракт, перебирая внутреннюю цепочку через `errors.Unwrap`: у объединённой ошибки может быть несколько дочерних ошибок.

> **Не применяйте `errors.Join` вместо решения**
>
> Одна операция должна иметь ясную главную причину. Объединяйте ошибки, когда обе важны для диагностики или очистки ресурсов. Не добавляйте в `Join` повтор одного и того же контекста на каждом уровне — лог и итоговая строка станут шумными.

Контракт пакета: что можно проверять снаружи

Перед тем как вернуть или обернуть ошибку, ответьте на вопросы:

1. **Кто будет её обрабатывать?** Этот же пакет, service, HTTP handler, `main`?
2. **Какое действие доступно вызывающему коду?** Отдать `404`, попросить исправить поле, повторить операцию, завершить запуск?
3. **Нужно ли раскрывать внутреннюю причину?** Например, внешний пакет обычно не должен зависеть от драйвера БД.
4. **Какие поля стабильны?** Если это `ValidationError`, пользователь `Field` и `Rule` должны иметь документированные значения.

Практическое правило границ Task Manager:

Слой	Что делает с ошибкой
repository	Переводит технические ожидаемые причины в доменные; добавляет контекст к непредвиденным инфраструктурным сбоям.
service	Применяет бизнес-правила, при необходимости добавляет бизнес-контекст; не знает HTTP-статусы.

HTTP handler	Распознаёт доменные ошибки и строит безопасный публичный ответ.
`main` / worker	Принимает окончательное решение о завершении процесса и логирует необработанную ошибку.

Следующая страница посвящена другой стороне надёжности: действия, которые должны произойти при любом выходе из функции.

`defer` и жизненный цикл ресурсов

`defer` уже встречался при закрытии файлов, отмене context и снятии mutex. В production-коде важно не просто «поставить defer», а определить владельца ресурса, порядок его освобождения и судьбу ошибки очистки.

Ресурс должен иметь владельца

Ресурсом может быть файл, HTTP response body, строка результатов SQL, транзакция, соединение, lock или context с `cancel`. У каждого ресурса должно быть понятно:

1. кто его получил;
2. кто обязан его освободить;
3. когда это произойдёт;
4. что делать, если освобождение не удалось.

Обычное правило владения: **функция, которая успешно получила ресурс и не передала владение явно, освобождает его сама**.

```
func readConfig(path string) ([]byte, error) {
  file, err := os.Open(path)
  if err != nil {
    return nil, fmt.Errorf("открыть конфигурацию: %w", err)
  }
  defer file.Close()

  content, err := io.ReadAll(file)
  if err != nil {
    return nil, fmt.Errorf("прочитать конфигурацию: %w", err)
  }

  return content, nil
}
```

`file` не возвращается вызывающему коду, значит именно `readConfig` остаётся владельцем и гарантирует `Close`. Если функция возвращает `*os.File`, это должно быть явно отражено в

документации: обязанность закрыть файл переходит к вызывающему коду.

Что именно делает `defer`

Отложенный вызов выполняется при выходе из **текущей функции**: через `return` или при раскрутке стека из-за panic. Аргументы вызова вычисляются сразу, а несколько `defer` выполняются по правилу LIFO — последний зарегистрированный вызов сработает первым.

```
func example() {
  name := "до изменения"
  defer fmt.Println(name) // аргумент уже вычислен
  defer func() {
    fmt.Println(name) // переменная будет прочитана позже
  }()

  name = "после изменения"
}

// Вывод:
// после изменения
// до изменения
```

LIFO естественно освобождает вложенные ресурсы в обратном порядке захвата. Если сначала открыли файл, а поверх него создали `bufio.Writer`, сначала нужно выполнить `Flush`, а затем закрыть файл.

Ошибки `Close`: не теряйте значимую причину

Ошибка закрытия при чтении часто не меняет уже прочитанные данные, но не является невозможной. При записи она особенно важна: финальная запись буфера или синхронизация с файловой системой может произойти именно во время `Close`.

В функции с именованным результатом отложенная функция может дополнить итоговую ошибку. `errors.Join` сохраняет и исходную неудачу, и проблему очистки:

```
func writeReport(path, report string) (err error) {
  file, err := os.Create(path)
  if err != nil {
    return fmt.Errorf("создать отчёт: %w", err)
  }

  defer func() {
    if closeErr := file.Close(); closeErr != nil {
      err = errors.Join(err, fmt.Errorf("закрыть отчёт: %w", closeErr))
    }
```

```
  }() if _, err := io.WriteString(file, report); err != nil {     return
fmt.Errorf("записать отчёт: %w", err) } return nil}
```

Если запись успешна, функция вернёт ошибку `Close`, если она возникла. Если не удались и запись, и закрытие, обе причины останутся доступны через `errors.Is` / `errors.As`.

Не усложняйте каждый короткий пример этим шаблоном. Выбирайте политику по контракту операции: для критичного экспорта ошибка закрытия важна; для best-effort чтения временного файла может быть достаточно зафиксировать решение в комментарии или документации.

Зависимые ресурсы: буфер до файла

Порядок регистрации `defer` важнее порядка строк в итоговом выполнении:

```
func writeBufferedReport(path, report string) (err error) {
  file, err := os.Create(path)
  if err != nil {
    return fmt.Errorf("создать отчёт: %w", err)
  }
  defer func() {
    if closeErr := file.Close(); closeErr != nil {
      err = errors.Join(err, fmt.Errorf("закрыть отчёт: %w", closeErr))
    }
  }()

  writer := bufio.NewWriter(file)
  defer func() {
    if flushErr := writer.Flush(); flushErr != nil {
      err = errors.Join(err, fmt.Errorf("сбросить буфер отчёта: %w", flushErr))
    }
  }()

  if _, err := writer.WriteString(report); err != nil {
    return fmt.Errorf("записать отчёт в буфер: %w", err)
  }

  return nil
}
```

Последний `defer` для `Flush` сработает первым, поэтому данные попадут в файл до `Close`. Если поменять их местами, `Flush` будет писать в уже закрытый файл.

Мьютексы и короткие критические секции

Сразу после успешного `Lock` регистрируйте `Unlock`:

```
store.mu.Lock()
defer store.mu.Unlock()

if _, exists := store.tasks[id]; exists {
  return ErrTaskAlreadyExists
}
store.tasks[id] = task
return nil
```

Так ранний `return` не оставит mutex заблокированным. Но `defer` не оправдывает долгую критическую секцию: не держите lock во время HTTP-вызова, обращения к БД, записи файла или ожидания канала. Сначала скопируйте необходимые данные под lock, снимите его и только потом выполняйте медленную работу.

`cancel` у context

`context.WithCancel`, `WithTimeout` и `WithDeadline` возвращают функцию отмены. Она освобождает внутренние ресурсы раньше, чем истечёт дедлайн, поэтому её вызывают всегда:

```
func fetchPriority(parent context.Context, client *http.Client, url string) error {
  ctx, cancel := context.WithTimeout(parent, 2*time.Second)
  defer cancel()

  request, err := http.NewRequestWithContext(ctx, http.MethodGet, url, nil)
  if err != nil {
    return fmt.Errorf("создать запрос приоритета: %w", err)
  }

  response, err := client.Do(request)
  if err != nil {
    return fmt.Errorf("получить приоритет: %w", err)
  }
  defer response.Body.Close()

  return nil
}
```

У `http.Response.Body` тот же контракт владения: после успешного `Do` вызывающий обязан закрыть body, даже если код статуса неуспешный. Это позволяет transport вернуть соединение в пул. Для переиспользования HTTP-соединения обычно также нужно дочитать body до конца, если это оправдано контрактом клиента.

Опасность `defer` в долгом цикле

`defer` выполняется при выходе из функции, а не в конце итерации. Поэтому код ниже откроет все файлы до завершения `processAll` и может исчерпать файловые дескрипторы:

```
func processAll(paths []string) error {
  for _, path := range paths {
    file, err := os.Open(path)
    if err != nil {
      return err
    }
    defer file.Close() // Плохо: закроется только в конце processAll.
    // ...
  }
  return nil
}
```

Выберите один из безопасных вариантов.

Вариант 1: вынести одну итерацию в отдельную функцию. Тогда `defer` относится к короткой области видимости:

```
func processOne(path string) error {
  file, err := os.Open(path)
  if err != nil {
    return fmt.Errorf("открыть %q: %w", path, err)
  }
  defer file.Close()

  // Обработка одного файла.
  return nil
}

func processAll(paths []string) error {
  for _, path := range paths {
    if err := processOne(path); err != nil {
      return err
    }
  }
  return nil
}
```

Вариант 2: закрыть ресурс явно в конце каждой итерации. Здесь нужно аккуратно объединить ошибку обработки и ошибку закрытия. В большинстве случаев маленькая вспомогательная функция читается лучше.

Та же проблема бывает с `defer cancel()` внутри цикла. Создавайте context на одну итерацию во вложенной функции или вызовите `cancel()` явно сразу после работы.

Транзакция SQL: rollback по умолчанию

У транзакции есть два корректных финала: `Commit` при успехе и `Rollback` при любой ошибке. Удобный шаблон — сразу отложить rollback:

```
func (repository *Repository) Complete(ctx context.Context, id int64) error {
  tx, err := repository.db.BeginTx(ctx, nil)
  if err != nil {
    return fmt.Errorf("начать транзакцию: %w", err)
  }
  defer tx.Rollback()

  result, err := tx.ExecContext(ctx, completeTaskQuery, id)
  if err != nil {
    return fmt.Errorf("завершить задачу %d: %w", id, err)
  }

  updated, err := result.RowsAffected()
  if err != nil {
    return fmt.Errorf("получить число обновлённых задач: %w", err)
  }
  if updated == 0 {
    return ErrTaskNotFound
  }

  if err := tx.Commit(); err != nil {
    return fmt.Errorf("зафиксировать завершение задачи %d: %w", id, err)
  }

  return nil
}
```

После успешного `Commit` отложенный `Rollback` вернёт `sql.ErrTxDone`; его здесь безопасно игнорировать, потому что транзакция уже завершена. Если контракт требует диагностировать rollback или cleanup отдельно, используйте явную, протестированную политику — не добавляйте сложность автоматически.

`defer` не заменяет измерения

В современном Go стоимость `defer` в обычном коде обычно не повод для микрооптимизации. Сначала пишите понятный безопасный код, затем измеряйте benchmark и profile, если операция

действительно находится на горячем пути. В первую очередь избегайте не самого `defer`, а неверной области его действия: длинного цикла, долгого lock или неочевидного порядка очистки.

Итог

`defer` хорош, когда ресурс живёт до конца небольшой функции и его освобождение неизбежно. Следующая страница разберёт другое поведение, которое тоже запускает отложенные вызовы, — `panic`, и объяснит, почему библиотека почти никогда не должна скрывать его без строгого контракта.

`panic` и `recover` в библиотечном коде

Сначала правило по умолчанию

Библиотечный код должен возвращать `error` для ожидаемых неудач. Неверный ввод, недоступная БД, отменённый context, отсутствие файла и ответ внешнего HTTP-сервиса — это не повод для `panic`.

```
// Неверно: пользовательский ввод — ожидаемая ситуация.
func NewTask(title string, priority int) Task {
  if title == "" {
    panic("title is required")
  }
  return Task{Title: title, Priority: priority}
}

// Верно: вызывающий код выбирает, как сообщить о проблеме.
func NewTask(title string, priority int) (Task, error) {
  if err := validateCreateInput(title, priority); err != nil {
    return Task{}, err
  }
  return Task{Title: title, Priority: priority}, nil
}
```

`panic` прерывает обычный поток управления. Пока стек раскручивается, вызываются `defer`; если panic не перехватить, завершается весь процесс. Поэтому это инструмент для нарушенного внутреннего инварианта или строго ограниченной защитной границы, а не альтернативная форма `error`.

Что `recover` действительно может перехватить

`recover` срабатывает только если он вызван **непосредственно** из отложенной функции в той же goroutine, пока эта goroutine раскручивает panic:

```
func callSafely(fn func()) (panicked bool) {
  defer func() {
    if recovered := recover(); recovered != nil {
      panicked = true
    }
  }()

  fn()
  return false
}
```

После восстановления функция, в которой началась panic, не продолжает работу с точки panic. В примере выше `callSafely` завершится с `true`.

`recover` за пределами `defer` вернёт `nil`. Вызов из вспомогательной функции также не является надёжной заменой прямому вызову в отложенной функции. Делайте recovery-блок коротким и видимым рядом с границей, которую он защищает.

`recover` не пересекает goroutine

Такой код не защитит процесс:

```
func wrong() {
  defer func() {
    recover()
  }()

  go func() {
    panic("panic в фоновой задаче")
  }()
}
```

У каждой goroutine свой стек. Если вы запускаете изолированную пользовательскую callback-функцию или worker и по контракту не хотите дать ей завершить весь процесс, recovery должен находиться **внутри этой goroutine**:

```
func runWorker(logger *slog.Logger, work func()) {
  go func() {
    defer func() {
      if recovered := recover(); recovered != nil {
        logger.Error("panic в изолированном worker", "recovered", recovered)
      }
    }()
```

```
    work() }()}
```

Это последний барьер, а не доказательство, что worker безопасен. Нужно решить, что делать с частично записанными данными, состоянием очереди и возможной повторной задачей. Для критичных операций обычно важнее транзакция, идемпотентность и корректная обработка `error`.

Когда library boundary действительно оправдана

Иногда библиотека вызывает код, который ей не принадлежит: плагины, callbacks, пользовательские шаблоны, тестовые хуки. Если паника такого callback не должна разрушить основной процесс, библиотека может превратить её в отдельную, документированную ошибку.

```
type PanicError struct {
  Value any
}

func (err *PanicError) Error() string {
  return "callback завершился panic"
}

func RunCallback(callback func() error) (err error) {
  defer func() {
    if recovered := recover(); recovered != nil {
      err = &PanicError{Value: recovered}
    }
  }()

  return callback()
}
```

Пользователь API может различить нормальную ошибку callback и нарушенный контракт callback:

```
err := RunCallback(func() error {
  panic("не должен падать")
})

var panicErr *PanicError
if errors.As(err, &panicErr) {
  // Зафиксировать инцидент, остановить конкретную задачу,
  // не отправлять Value клиенту.
}
```

Этот дизайн оправдан только при явном обещании `RunCallback`: callback изолирован, а его panic становится ошибкой. Не оборачивайте так все функции пакета «на всякий случай»: это превращает дефект программы в обычную ветку, лишает stack trace и затрудняет расследование.

Не теряйте диагностику

В HTTP boundary или worker обычно нужно залогировать panic вместе со stack trace, но не вернуть его клиенту:

```
func recoverWorker(logger *slog.Logger) {
  if recovered := recover(); recovered != nil {
    logger.Error(
      "panic в worker",
      "recovered", recovered,
      "stack", string(debug.Stack()),
    )
  }
}

func runWorker(logger *slog.Logger, work func()) {
  go func() {
    defer recoverWorker(logger)
    work()
  }()
}
```

Stack trace может содержать внутренние пути, аргументы или иные чувствительные данные. Он предназначен для защищённого журнала, а не для JSON-ответа, метрики с неограниченной кардинальностью или сообщения пользователю. Проверьте политику хранения логов до добавления таких полей.

В библиотеке чаще лучше **вернуть** `PanicError` без stack trace. Верхняя граница приложения решает, нужно ли писать стек, в какой лог и как его защитить.

`panic` как намеренный контракт

Некоторые API стандартной библиотеки паникуют при нарушении программистского контракта: например, индекс вне границ слайса. Свой API может использовать такой подход только если одновременно выполняются условия:

- ошибка означает баг вызывающего программиста, а не входные данные пользователя или внешнюю среду;
- проверка возвращаемого `error` сделала бы API существенно хуже, а не просто короче;
- поведение ясно отражено в имени и документации;
- panic не оставляет систему в тихо повреждённом состоянии.

Честный вариант — отдельный helper с префиксом `Must`:

```
func MustParseDuration(value string) time.Duration {
  duration, err := time.ParseDuration(value)
  if err != nil {
    panic(fmt.Sprintf("некорректная duration %q: %v", value, err))
  }
  return duration
}
```

Такой helper уместен для константной конфигурации в `main` или тестах, когда ошибка означает дефект исходного кода. Он не подходит для переменной окружения или тела HTTP-запроса: там вместо него нужна функция с `error` и понятной диагностикой запуска либо ответа клиенту.

`panic`, `os.Exit` и cleanup

При panic выполняются отложенные вызовы текущего стека. При `os.Exit` — нет. Поэтому package-код не должен вызывать `os.Exit`: он не даёт вызывающему коду закрыть БД, дописать лог, снять lock или проверить ошибку в тесте.

```
// В библиотеке плохо:
if err != nil {
  os.Exit(1)
}

// Библиотека возвращает ошибку; main принимает решение о процессе.
if err != nil {
  return fmt.Errorf("запустить приложение: %w", err)
}
```

`main` — естественная граница, где допустимы окончательный лог и ненулевой код завершения. Даже там предпочтительнее дать `run()` вернуть ошибку, а не рассыпать `os.Exit` по зависимостям.

Чек-лист перед `recover`

Перед добавлением recovery-блока спросите себя:

1. Можно ли устранить причину и вернуть обычный `error`?
2. Какая конкретная граница изолируется: HTTP-запрос, callback, отдельный worker?
3. Что произойдёт с частично изменёнными данными?
4. Где будут записаны причина и stack trace?
5. Как гарантировать, что клиент не увидит внутреннюю информацию?
6. Как тест докажет, что recovery не скрывает нормальные ошибки и не завершает процесс?

В следующей странице эти решения будут применены к самой важной внешней границе Task Manager API — HTTP handler.

Ошибки HTTP API: контракт, логи и panic

HTTP handler — граница между внутренностями приложения и клиентом. Он получает детали repository, service и внешних клиентов, но не должен выдавать клиенту SQL-текст, адрес базы, stack trace или сообщение panic. Вместо этого handler переводит известные доменные причины в стабильный публичный контракт.

Один внутренний сбой — два разных представления

Пусть запрос на завершение несуществующей задачи прошёл путь:

```
PostgreSQL: sql.ErrNoRows
        ↓ repository переводит
domain: ErrTaskNotFound
        ↓ handler переводит
HTTP: 404 + {"code":"task_not_found","message":"Задача не найдена"}
```

Каждое представление предназначено для своего слоя. Нельзя возвращать `err.Error()` в JSON только потому, что это быстро:

```
// Небезопасно: текст может раскрыть SQL, URL, имя таблицы или внутренний контекст.
c.JSON(http.StatusInternalServerError, gin.H{"error": err.Error()})
```

Стабильная форма ответа

Сделайте код ошибки машинно-читаемым, а сообщение — понятным человеку. Например:

```
type errorResponse struct {
  Code    string            `json:"code"`
  Message string            `json:"message"`
  Fields  map[string]string `json:"fields,omitempty"`
}
```

Примеры контракта:

```
{"code":"task_not_found","message":"Задача не найдена"}
```

```
{
  "code":"validation_failed",
  "message":"Проверьте входные данные",
  "fields":{"title":"required"}
}
```

Клиентская программа должна ориентироваться на `code`, а не на русский текст `message`. Текст можно исправить или локализовать, не ломая интеграцию. Значения `code` и ключи `fields` документируют так же тщательно, как пути и HTTP-статусы.

> **Не меняйте текущий контракт незаметно**
>
> В основной версии Task Manager API из предыдущих глав ошибка имеет форму `{"error":"..."}`. Новый формат — пример развития API. Для существующих клиентов сохраните старую форму либо вводите новую версию маршрутов/контракта согласованно. «Улучшение» JSON без миграции — breaking change.

Центральная функция классификации

Не дублируйте длинный `switch` в каждом handler. Одна функция превращает доменную ошибку в HTTP-статус и безопасный payload:

```
func classifyError(err error) (int, errorResponse) {
  var validationErr *task.ValidationError

  switch {
  case errors.As(err, &validationErr):
    return http.StatusUnprocessableEntity, errorResponse{
      Code:    "validation_failed",
      Message: "Проверьте входные данные",
      Fields: map[string]string{
        validationErr.Field: validationErr.Rule,
      },
    }
  case errors.Is(err, task.ErrTaskNotFound):
    return http.StatusNotFound, errorResponse{
      Code:    "task_not_found",
      Message: "Задача не найдена",
    }
  case errors.Is(err, task.ErrTaskAlreadyCompleted):
    return http.StatusConflict, errorResponse{
      Code:    "task_already_completed",
      Message: "Задача уже выполнена",
```

```
    } default:   return http.StatusInternalServerError, errorResponse{      Code:
"internal_error",     Message: "Внутренняя ошибка сервера",   } }}
```

Статус `422 Unprocessable Entity` здесь означает: JSON синтаксически корректен, но не проходит правила предметной области. Многие API используют для той же ситуации `400 Bad Request`. Оба подхода возможны; важнее выбрать правило команды, документировать его и не менять выбор от handler к handler.

Для ошибок синтаксиса JSON, неизвестных полей и неверного path-параметра обычно подходит `400`. `401`, `403`, `404`, `409`, `422`, `429`, `503` и другие статусы должны отражать осмысленную, стабильную семантику, а не просто «любой не-500».

Handler: распознать, записать один ответ, завершиться

Handler вызывает service, классифицирует ошибку и после записи ответа делает `return`:

```
func (api *API) completeTask(c *gin.Context) {
  id, err := strconv.ParseInt(c.Param("id"), 10, 64)
  if err != nil || id < 1 {
    c.JSON(http.StatusBadRequest, errorResponse{
      Code:    "invalid_task_id",
      Message: "Идентификатор задачи должен быть положительным числом",
    })
    return
  }

  completedTask, err := api.service.Complete(c.Request.Context(), id)
  if err != nil {
    status, response := classifyError(err)
    if status >= http.StatusInternalServerError {
      api.logger.Error(
        "не удалось завершить задачу",
        "task_id", id,
        "request_id", c.GetString(requestIDKey),
        "err", err,
      )
    }
    c.JSON(status, response)
    return
  }

  c.JSON(http.StatusOK, completedTask)
}
```

`return` после `c.JSON` предотвращает попытку записать второй ответ. Не маскируйте ожидаемые клиентские ошибки (`404`, `422`) как `Error` в каждом слое: access log уже запишет статус, а дополнительный error-log для каждого неверного запроса создаст шум. Непредвиденный `500` нужно записать с внутренней ошибкой и безопасным операционным контекстом.

Где логировать ошибку

Ошибка и лог решают разные задачи:

Ситуация	Возврат `error`	Лог
repository не нашёл строку	вернуть `ErrTaskNotFound`	обычно нет
service отверг бизнес-правило	вернуть доменную ошибку	обычно нет
handler вернул ожидаемый `4xx`	обработать и вернуть JSON	access log со статусом; отдельный error-log обычно не нужен
handler вернул непредвиденный `5xx`	обработать и вернуть безопасный JSON	один `Error` на границе с request ID и внутренней причиной
`main` не смог запустить приложение	вернуть ошибку из `run`	один финальный `Error` перед выходом
middleware перехватил panic	завершить запрос `500`	один `Error` со stack trace в защищённом журнале

Логировать ошибку в repository, затем service, затем handler и `main` — значит получить несколько почти одинаковых записей. Добавляйте контекст в возвращаемую цепочку на нижних слоях; логируйте там, где ошибка окончательно обработана.

Context: отмена клиента и timeout

`c.Request.Context()` отменяется, когда клиент разорвал соединение или сервер прекращает запрос. Не превращайте каждую отмену в `500` и не пытайтесь писать клиенту ответ, который уже отключился.

Политика зависит от того, где установлен дедлайн:

- `context.Canceled` часто означает, что клиент уже ушёл; завершите работу и учитывайте это отдельно от ошибки сервера;
- `context.DeadlineExceeded` из дедлайна gateway или handler может соответствовать `504 Gateway Timeout`, но это решение архитектуры, а не автоматическое правило;
- timeout к внешнему сервису иногда правильнее отдать как `503 Service Unavailable`, иногда как `502 Bad Gateway` или `504`.

Сначала передайте context во все отменяемые операции (`QueryContext`, `http.NewRequestWithContext`), затем договоритесь о статусах и метриках на границе API. Не подменяйте отменённый context новым `context.Background()` ради продолжения работы: это нарушит shutdown и отмену запроса.

Recovery middleware — последний слой защиты

В Gin recovery должен располагаться в цепочке middleware и возвращать такой же безопасный контракт `500`:

```
func recovery(logger *slog.Logger) gin.HandlerFunc {
  return gin.CustomRecovery(func(c *gin.Context, recovered any) {
    logger.Error(
      "panic в HTTP handler",
      "request_id", c.GetString(requestIDKey),
      "recovered", recovered,
      "stack", string(debug.Stack()),
    )

    c.AbortWithStatusJSON(http.StatusInternalServerError, errorResponse{
      Code:    "internal_error",
      Message: "Внутренняя ошибка сервера",
    })
  })
}
```

Recovery не исправляет код и не отменяет уже выполненные side effects. После инцидента найдите причину по логу, добавьте воспроизводящий тест и исправьте дефект. Тело ответа никогда не должно содержать `recovered`, `debug.Stack()` или внутренний `err`.

Тестируйте контракт, а не строку лога

Минимальные тесты функции `classifyError` проверяют публичное поведение без Gin и базы:

```
func TestClassifyErrorTaskNotFound(t *testing.T) {
  status, response := classifyError(fmt.Errorf("завершить задачу: %w",
task.ErrTaskNotFound))

  if status != http.StatusNotFound {
    t.Fatalf("status = %d, want %d", status, http.StatusNotFound)
  }
  if response.Code != "task_not_found" {
    t.Fatalf("code = %q", response.Code)
  }
}
```

Для handler добавьте `httptest`: проверьте статус, `Content-Type`, JSON `code` и отсутствие внутреннего текста. Отдельно протестируйте, что middleware возвращает безопасный `500` при

panic. Не делайте stack trace частью тестируемого HTTP-ответа.

Итоговая граница

Хороший HTTP API не заставляет клиента знать устройство Go-пакетов, драйвер БД или текст panic. Он публикует документированные статусы и коды ошибок, сохраняет внутреннюю диагностику в один защищённый лог и оставляет `panic` последним сигналом о дефекте, а не регулярным механизмом управления программой.

Самопроверка: ошибки и управление ресурсами

1. Что сохраняет fmt.Errorf("операция: %w", err) в дополнение к тексту контекста?

A. Связь с исходной ошибкой для errors.Is и errors.As

B. HTTP-статус ошибки

C. Stack trace любой ошибки

D. Только строку err.Error() без связи с причиной

2. Когда errors.Is(err, target) подходит лучше errors.As?

A. Когда нужно узнать наличие известной причины-маркера без дополнительных данных

B. Когда нужно прочитать поля структурированной ошибки

C. Только когда err равен nil

D. Когда нужно сравнить тексты ошибок

3. Для чего в собственной ошибке ValidationError может понадобиться метод Unwrap() error?

A. Чтобы recover мог перехватить ошибку

B. Чтобы errors.Is мог распознать общую причину ErrInvalidInput

C. Чтобы удалить поля Field и Rule

D. Чтобы автоматически отправить ошибку клиенту как JSON

4. Почему для поиска *ValidationError используют var validationErr *ValidationError и errors.As(err, &validationErr)?

A. errors.As записывает найденную ошибку в переданную переменную

B. errors.As принимает строковое имя типа

C. Так errors.As создаёт новую ошибку

D. Иначе Go автоматически вызовет panic

5. Почему функция не должна возвращать типизированный nil-указатель как error?

A. Компилятор всегда запретит такой return

B. errors.Is всегда вернёт false

C. Указатель автоматически превратится в errors.New

D. Интерфейс error будет содержать динамический тип и окажется не равен nil

6. Что вернёт errors.Join(), если ему передать только nil-ошибки?

A. nil

B. PanicError

C. Пустую строку

D. sql.ErrNoRows

7. В каком порядке сработают несколько defer в одной функции?

A. В обратном порядке регистрации: LIFO

B. Только последний defer

C. В случайном порядке

D. В порядке регистрации: FIFO

8. Почему defer file.Close() опасен внутри большого цикла?

A. Close всегда вызывает panic в цикле

B. defer отключает обработку ошибок io.ReadAll

C. defer нельзя применять к файлам

D. Файлы закроются лишь при выходе из внешней функции и могут исчерпаться дескрипторы

9. Зачем после BeginTx обычно сразу вызывают defer tx.Rollback()?

A. Чтобы Commit сработал два раза

B. Чтобы незавершённая транзакция откатилась при любом раннем return

C. Чтобы не передавать context в SQL-запросы

D. Чтобы превратить sql.ErrNoRows в 404 автоматически

10. Где должен находиться recover, чтобы перехватить panic в goroutine worker?

A. В defer функции main в другой goroutine

B. В errorResponse HTTP handler

C. В defer внутри той же goroutine worker

D. В любом месте после оператора go

11. Что безопасно вернуть клиенту при непредвиденной внутренней ошибке HTTP API?

A. err.Error() вместе со строкой SQL

B. Значение recovered из panic

C. Стабильный код и общее сообщение без текста внутренней ошибки

D. debug.Stack() в поле stack

12. Где обычно нужно записывать непредвиденную ошибку 500?

A. Один раз на границе окончательной обработки, например в HTTP handler, с операционным контекстом

B. Нигде: достаточно вернуть клиенту err.Error()

C. Во всех слоях: repository, service, handler и main

D. Только внутри Error() собственной ошибки

Ответы к самопроверке

1. Связь с исходной ошибкой для errors.Is и errors.As

2. Когда нужно узнать наличие известной причины-маркера без дополнительных данных

3. Чтобы errors.Is мог распознать общую причину ErrInvalidInput

4. errors.As записывает найденную ошибку в переданную переменную

5. Интерфейс error будет содержать динамический тип и окажется не равен nil

6. nil

7. В обратном порядке регистрации: LIFO

8. Файлы закроются лишь при выходе из внешней функции и могут исчерпаться дескрипторы

9. Чтобы незавершённая транзакция откатилась при любом раннем return

10. В defer внутри той же goroutine worker

11. Стабильный код и общее сообщение без текста внутренней ошибки

12. Один раз на границе окончательной обработки, например в HTTP handler, с операционным контекстом

Задания: ошибки и управление ресурсами

Практика продолжает Task Manager API на уровне границ пакетов. Выполняйте задания по порядку: следующие используют типы и ошибки из предыдущих. Решения показывают один из вариантов; в production-коде зафиксируйте выбранные коды, статусы и правила в документации API.

Задание 1. Ошибка валидации с данными

Создайте пакет `task` с ошибкой-маркером `ErrInvalidInput` и типом `ValidationError` с полями `Field` и `Rule`.

Напишите функцию:

```
func ValidateCreateInput(title string, priority int) error
```

Правила:

- `title` после `strings.TrimSpace` не должен быть пустым — `Field: "title"`, `Rule: "required"`;
- `priority` должен быть от `1` до `5` — `Field: "priority"`, `Rule: "range"`;
- любая ошибка валидации должна быть доступна и через `errors.Is(err, ErrInvalidInput)`, и через `errors.As` к `*ValidationError`.

Критерии проверки:

- `ValidationError` реализует `error`;
- `Unwrap()` возвращает `ErrInvalidInput`;
- функция возвращает явный `nil`, когда оба поля корректны;
- нет сравнения текста ошибки;
- в ошибке не хранится весь пользовательский title: для этого правила достаточно поля и имени правила.

Подсказка

```
func (err *ValidationError) Unwrap() error {
  return ErrInvalidInput
}
```

Решение

```
package task

import (
  "errors"
  "fmt"
  "strings"
)

var ErrInvalidInput = errors.New("некорректные входные данные")

type ValidationError struct {
  Field string
  Rule  string
}

func (err *ValidationError) Error() string {
  return fmt.Sprintf("поле %q не прошло правило %q", err.Field, err.Rule)
}

func (err *ValidationError) Unwrap() error {
  return ErrInvalidInput
}

func ValidateCreateInput(title string, priority int) error {
  if strings.TrimSpace(title) == "" {
    return &ValidationError{Field: "title", Rule: "required"}
  }
  if priority < 1 || priority > 5 {
    return &ValidationError{Field: "priority", Rule: "range"}
  }

  return nil
}
```

Задание 2. Проверьте тип и общую причину

Напишите тест `TestValidateCreateInputEmptyTitle`. Он должен убедиться, что ошибка пустого title:

1. распознаётся через `errors.Is(err, ErrInvalidInput)`;
2. извлекается через `errors.As` в `*ValidationError`;
3. содержит `Field == "title"` и `Rule == "required"`.

Критерии проверки:

- тест не сравнивает `err.Error()`;
- `errors.As` получает указатель на переменную результата;
- тест завершается с понятным сообщением, если ошибка неожиданно равна `nil`;
- тест не зависит от порядка текста в `Error()`.

Подсказка

```
var validationErr *ValidationError
if !errors.As(err, &validationErr) {
  t.Fatal("ошибка не имеет тип ValidationError")
}
```

Решение

```
package task

import (
  "errors"
  "testing"
)

func TestValidateCreateInputEmptyTitle(t *testing.T) {
  err := ValidateCreateInput("   ", 3)
  if err == nil {
    t.Fatal("ValidateCreateInput не вернул ошибку")
  }
  if !errors.Is(err, ErrInvalidInput) {
    t.Fatalf("errors.Is(err, ErrInvalidInput) = false, err = %v", err)
  }

  var validationErr *ValidationError
  if !errors.As(err, &validationErr) {
    t.Fatalf("errors.As не нашёл ValidationError, err = %v", err)
  }
  if validationErr.Field != "title" {
    t.Fatalf("Field = %q, want title", validationErr.Field)
  }
  if validationErr.Rule != "required" {
    t.Fatalf("Rule = %q, want required", validationErr.Rule)
  }
}
```

Задание 3. Безопасная запись отчёта

Напишите `WriteReport(path, report string) error`, которая записывает отчёт в файл через `os.Create` и `io.WriteString`.

Если `Close` тоже вернул ошибку, она не должна потеряться: объедините её с возможной ошибкой записи через `errors.Join`. Используйте именованный результат и отложенную функцию.

Критерии проверки:

- `defer` ставится только после успешного `os.Create`;
- при записи и закрытии файл освобождается на любом пути;
- ошибка записи получает контекст `записать отчёт`;
- ошибка закрытия получает контекст `закрыть отчёт`;
- успешная операция возвращает `nil`.

Подсказка

В `defer` можно присвоить именованному `err` новое значение:

```
err = errors.Join(err, fmt.Errorf("закрыть отчёт: %w", closeErr))
```

Решение

```
package report

import (
  "errors"
  "fmt"
  "io"
  "os"
)

func WriteReport(path, report string) (err error) {
  file, err := os.Create(path)
  if err != nil {
    return fmt.Errorf("создать отчёт: %w", err)
  }

  defer func() {
    if closeErr := file.Close(); closeErr != nil {
      err = errors.Join(err, fmt.Errorf("закрыть отчёт: %w", closeErr))
    }
  }()

  if _, err := io.WriteString(file, report); err != nil {
    return fmt.Errorf("записать отчёт: %w", err)
  }

  return nil
}
```

Задание 4. Переведите доменную ошибку в HTTP-контракт

Для Gin API напишите функцию `classifyError`, которая преобразует:

- `task.ErrTaskNotFound` в `404` и код `task_not_found`;
- `*task.ValidationError` в `422` и код `validation_failed`, добавляя поле и правило в `fields`;
- всё остальное в `500` и код `internal_error`.

Не передавайте `err.Error()` в ответ для `500`. Напишите unit-тест хотя бы для обёрнутого `ErrTaskNotFound`.

Критерии проверки:

- используется `errors.Is` для ошибки-маркера;
- используется `errors.As` для `ValidationError`;
- функция не зависит от Gin, поэтому тестируется без `httptest`;
- код и сообщение `500` не содержат внутренних деталей;
- обёрнутая ошибка задачи тоже даёт `404`.

Подсказка

```
if errors.Is(err, task.ErrTaskNotFound) {
  return http.StatusNotFound, errorResponse{...}
}
```

Решение

```
package api

import (
  "errors"
  "net/http"

  "example.com/task-manager/internal/task"
)

type errorResponse struct {
  Code    string            `json:"code"`
  Message string            `json:"message"`
  Fields  map[string]string `json:"fields,omitempty"`
}

func classifyError(err error) (int, errorResponse) {
  var validationErr *task.ValidationError

  switch {
  case errors.As(err, &validationErr):
    return http.StatusUnprocessableEntity, errorResponse{
      Code:    "validation_failed",
      Message: "Проверьте входные данные",
      Fields: map[string]string{
        validationErr.Field: validationErr.Rule,
      },
    }
  case errors.Is(err, task.ErrTaskNotFound):
    return http.StatusNotFound, errorResponse{
      Code:    "task_not_found",
      Message: "Задача не найдена",
    }
  default:
    return http.StatusInternalServerError, errorResponse{
      Code:    "internal_error",
```

▼ Сведения

```
       Message: "Внутренняя ошибка сервера",   } }}
```

```
func TestClassifyErrorTaskNotFound(t *testing.T) {
  status, response := classifyError(fmt.Errorf("завершить задачу: %w",
task.ErrTaskNotFound))

  if status != http.StatusNotFound {
    t.Fatalf("status = %d, want %d", status, http.StatusNotFound)
  }
  if response.Code != "task_not_found" {
    t.Fatalf("Code = %q, want task_not_found", response.Code)
  }
}
```

Во втором фрагменте нужны импорты `fmt`, `net/http`, `testing` и пакет `task`.

Задание 5. Изолируйте callback с `recover`

Напишите функцию:

```
func RunCallback(callback func() error) (err error)
```

Если callback вызывает panic, функция должна вернуть `*PanicError` с полем `Value any`. Если callback обычным образом вернул ошибку, она должна остаться исходной. Добавьте тест для обоих сценариев.

Критерии проверки:

- `recover` находится в отложенной функции внутри `RunCallback`;
- обычная `error` не заменяется `PanicError`;
- panic распознаётся через `errors.As`;
- `PanicError.Value` не отправляется клиенту или в незащищённый лог;
- функция документирована как граница изоляции callback, а не применяется ко всему прикладному коду.

Подсказка

Именованный результат позволяет установить `err` внутри `defer`:

```
defer func() {
  if recovered := recover(); recovered != nil {
    err = &PanicError{Value: recovered}
  }
}()
```

Решение

```
package callback

import "errors"

type PanicError struct {
  Value any
}

func (err *PanicError) Error() string {
  return "callback завершился panic"
}

// RunCallback изолирует panic кода, переданного вызывающим пакетом.
func RunCallback(callback func() error) (err error) {
  defer func() {
    if recovered := recover(); recovered != nil {
      err = &PanicError{Value: recovered}
    }
  }()

  return callback()
}

func IsPanic(err error) bool {
  var panicErr *PanicError
  return errors.As(err, &panicErr)
}
```

Проверьте себя

Перед переходом к следующей продвинутой главе убедитесь, что вы можете:

- выбрать между ошибкой-маркером, собственным типом ошибки и `errors.Join`;
- объяснить, какую зависимость раскрывает `%w`;
- получить данные ошибки через `errors.As` без разбора текста;
- определить владельца файла, `rows`, response body, transaction, lock и `cancel`;
- не разместить `defer` в длинном цикле без отдельной области видимости;
- отличить recovery boundary от попытки скрыть дефект;
- вернуть наружу безопасный HTTP-контракт и оставить внутренние детали только в защищённой диагностике.

Рекомендации по коду: ошибки и ресурсы

Ошибки как контракт пакета

- Добавляйте контекст операции через `fmt.Errorf("...: %w", err)`, но осознанно: `%w` делает внутреннюю причину доступной вызывающему коду.
- Используйте `errors.Is` для известных причин-маркеров (`ErrTaskNotFound`, `context.Canceled`, `sql.ErrNoRows` на подходящей границе).
- Используйте `errors.As`, когда вызывающему коду нужны данные собственного типа ошибки, а не только факт причины.
- Документируйте экспортируемые ошибки, их поля и допустимые значения так же, как публичные функции.
- Не сравнивайте `err.Error()`; текст меняется и предназначен для диагностики, а не для ветвления.
- Не возвращайте typed `nil` как `error`: при успехе возвращайте именно `nil`.
- Объединяйте причины через `errors.Join` только когда несколько независимых ошибок действительно нужны для диагностики или cleanup.
- Не раскрывайте клиентам технические ошибки драйверов, сетевых библиотек и stack trace.

Владение и `defer`

- Сразу после успешного получения ресурса решайте, кто его освобождает; если владение не передано явно, закрывайте ресурс в той же функции.
- Регистрируйте `defer` сразу после успешных `Open`, `Lock`, `BeginTx`, `WithTimeout` и `client.Do`.
- Помните LIFO: зависимые ресурсы освобождаются в обратном порядке их захвата. Буфер нужно `Flush` до закрытия файла.
- Для операций записи выбирайте явную политику ошибки `Close`; если важны обе причины, не потеряйте их через `errors.Join`.
- Не откладывайте `Close`, `cancel` или `Unlock` в длинном цикле до конца внешней функции. Вынесите одну итерацию в helper или завершайте ресурс явно.
- Не удерживайте mutex во время сети, SQL, файлового ввода-вывода и других медленных операций.
- Для транзакции ставьте `defer tx.Rollback()` сразу после успешного `BeginTx`, а `Commit` делайте только после всех успешных операций.

`panic`, `recover` и границы процесса

- Возвращайте `error` для ожидаемых проблем пользователя, сети, файла, БД и конфигурации окружения.
- Применяйте `panic` только для нарушения программистского инварианта с ясно документированным контрактом; helper, который может паниковать, называйте с префиксом `Must`.
- `recover` размещайте только на конкретной изолирующей границе: в HTTP middleware, callback boundary или внутри отдельной goroutine worker.
- Помните: `recover` не перехватывает panic другой goroutine.
- После recovery логируйте причину и stack trace только в защищённый внутренний журнал; не отправляйте их клиенту.
- Не вызывайте `os.Exit` из library-кода: он не запускает `defer` и лишает вызывающий код возможности завершить cleanup. Возвращайте ошибку в `main`.

HTTP API и логи

- Переводите технические причины в доменные ошибки ниже HTTP-слоя; handler не должен знать детали PostgreSQL или конкретного HTTP-клиента.
- Централизуйте соответствие «доменная ошибка → HTTP status → публичный code/message», чтобы разные handlers не расходились в контракте.
- Пусть клиент ориентируется на стабильный `code`, а не на локализуемое `message`.
- После `c.JSON` или `c.AbortWithStatusJSON` делайте `return`, если handler не должен продолжать работу.
- Ожидаемые `4xx` обычно видны в access log и не требуют error-log на каждом слое. Непредвиденный `5xx` логируйте один раз на границе с request ID и внутренней причиной.
- Проверяйте тестами статус, JSON-код, безопасное сообщение и отсутствие внутренних деталей; не делайте текст логов частью случайного публичного контракта.

Итоговый чек-лист

Перед merge спросите:

- [ ] Может ли вызывающий код обработать ошибку без разбора её текста?
- [ ] Не раскрывает ли `%w` техническую зависимость за пределами нужного пакета?
- [ ] Закрывается ли каждый полученный ресурс на всех путях, включая ранний `return`?
- [ ] Не накапливает ли `defer` ресурсы внутри цикла?
- [ ] Сохраняется ли значимая ошибка `Close`, `Flush`, `Commit` или cleanup?
- [ ] Есть ли конкретная причина для `panic` или `recover`, а не желание «чтобы не падало»?
- [ ] Безопасен ли JSON-ответ при `500` и можно ли найти внутреннюю причину по одному логу?
- [ ] Зафиксированы ли тестом все документированные ошибки и HTTP-коды?

Продвинутый Go B: типы и generics

Цель главы

Научиться отличать defined type от type alias, писать и читать generic-функции и generic-типы, проектировать ограничения типов (`constraints`) и выбирать generics только там, где они действительно уменьшают дублирование без потери ясности.

Generics появились в Go 1.18. Они не заменяют интерфейсы, не делают любой код «универсальнее» автоматически и не обязательны для хорошего backend-сервиса. В Task Manager API большинство business-типов остаются обычными: `Task`, `CreateInput`, `TaskRepository`, `ValidationError`. Обобщение полезно, когда алгоритм одинаков для нескольких **семейств типов**, а различие между типами важно сохранить во время компиляции.

Карта главы

1. **Эта страница** — defined types, type alias, underlying type и методы.
2. `Параметры типов и generic-функции` — `[T any]`, вывод типа, `any` и `comparable`.
3. `Constraints`, type sets и `~` — интерфейсы как ограничения, объединения типов и базовый тип.
4. Generic-типы и выбор абстракции — `Set`, `Optional`, границы применимости и интеграция с Task Manager API.

Сначала: три похожих объявления

Пусть в приложении есть числовой идентификатор задачи:

```
type TaskID int64
```

Это **defined type** (определённый, или именованный, тип). Он имеет тот же underlying type — `int64`, но является отдельным типом для проверки компилятором.

Сравните с alias:

```
type LegacyTaskID = int64
```

Это **type alias**: новое имя для уже существующего типа. `LegacyTaskID` и `int64` — один и тот же тип, без нового слоя типобезопасности.

Объявление	Что создаёт	Совместимость с `int64` без преобразования	Можно объявлять методы

`type TaskID int64`	новый defined type	нет	да
`type LegacyTaskID = int64`	alias существующего типа	да	нет

Defined type: защита смысла значений

Обычные числа с одинаковым представлением легко перепутать:

```
func loadTask(taskID int64) {}
func loadUser(userID int64) {}

loadTask(42)
loadUser(42) // Компилятор не знает, что это другое значение предметной области.
```

Defined types помогают выразить намерение в сигнатурах:

```
type TaskID int64
type UserID int64

func loadTask(taskID TaskID) {}
func loadUser(userID UserID) {}

var taskID TaskID = 42
var userID UserID = 42

loadTask(taskID)
loadUser(userID)

// loadTask(userID) // Ошибка компиляции: UserID не TaskID.
```

Такой тип не подтверждает, что ID существует в БД, и не заменяет валидацию. Он предотвращает отдельный класс ошибок: случайную передачу идентификатора одной сущности туда, где ожидается другая.

При границе со стандартной библиотекой, JSON или драйвером БД иногда понадобится явное преобразование:

```
func taskPath(id TaskID) string {
  return strconv.FormatInt(int64(id), 10)
}

func taskIDFromPath(value string) (TaskID, error) {
  id, err := strconv.ParseInt(value, 10, 64)
  if err != nil {
```

```
    return 0, fmt.Errorf("разобрать ID задачи: %w", err) } if id < 1 {    return 0,
errors.New("ID задачи должен быть положительным")  } return TaskID(id), nil}
```

Явное `TaskID(id)` делает преобразование заметным в code review. Но преобразование не выполняет проверку диапазона: `TaskID(-1)` скомпилируется. Сначала валидируйте внешние данные, затем создавайте доменный тип.

Underlying type и операции

У defined type есть underlying type. Для `type TaskID int64` это `int64`; для более сложного типа — базовый тип в его объявлении. Благодаря этому над числовым defined type разрешены подходящие операции:

```
type Priority int

const (
  PriorityLow    Priority = 1
  PriorityNormal Priority = 2
  PriorityHigh   Priority = 3
)

func nextPriority(priority Priority) Priority {
  return priority + 1
}
```

Но это не означает неограниченную совместимость с `int`. Например, присваивание требует преобразования:

```
var priority Priority = PriorityHigh

// var level int = priority // Ошибка: разные defined types.
var level int = int(priority)
```

Для строковых статусов полезна та же идея:

```
type TaskStatus string

const (
  TaskStatusOpen      TaskStatus = "open"
  TaskStatusCompleted TaskStatus = "completed"
)
```

Константы с типом защищают присваивание в Go-коде. Но JSON всё равно может принести произвольную строку, поэтому HTTP input должен проходить валидацию, а не только преобразование `TaskStatus(request.Status)`.

Методы принадлежат defined type

Defined type можно дополнить методами. Это удобно, когда поведение относится к доменному значению:

```
type Priority int

func (priority Priority) Valid() bool {
  return priority >= PriorityLow && priority <= PriorityHigh
}

func (priority Priority) String() string {
  switch priority {
  case PriorityLow:
    return "low"
  case PriorityNormal:
    return "normal"
  case PriorityHigh:
    return "high"
  default:
    return "unknown"
  }
}
```

Метод нельзя объявить для alias не-локального типа:

```
type LegacyTaskID = int64

// func (id LegacyTaskID) String() string { ... }
// Ошибка: нельзя определять новые методы для non-local type int64.
```

Кроме того, метод должен объявляться в том же пакете, что и defined type. Нельзя добавить метод к `time.Time`, `http.Client` или defined type из другого пакета. Используйте обёртку, helper-функцию или композицию, а не пытайтесь расширить чужой тип.

Когда нужен type alias

Alias нужен главным образом для совместимости и постепенной миграции API, а не для моделирования предметной области.

Например, пакет раньше экспортировал `OldTask`, а теперь хочет назвать тип `Task`, не ломая всех потребителей сразу:

```
type Task struct {
  ID    int64
  Title string
}

// Deprecated: используйте Task.
type OldTask = Task
```

Код с `OldTask` и `Task` работает с одним типом. Методы `Task` доступны через alias, потому что alias не создаёт новый тип.

Другой распространённый случай — переэкспорт типа внешнего пакета в совместимом API. Но alias расширяет поверхность зависимости: пользователи начинают зависеть от точного типа. Применяйте его осознанно и документируйте deprecated-имена.

> **Alias не делает преобразование безопаснее**
>
> `type TaskID = int64` выглядит как доменное имя, но компилятор разрешит передать `UserID`, `int64` или любой другой совместимый `int64` туда, где ожидается `TaskID`. Если цель — исключить путаницу сущностей, нужен defined type без `=`.

Defined type, alias и интерфейсы

Интерфейс зависит от метода и его точной сигнатуры. Два defined type с одинаковым underlying type остаются разными для сигнатур:

```
type TaskID int64
type UserID int64

type TaskLoader interface {
  Load(TaskID) error
}

type UserStore struct{}

func (UserStore) Load(UserID) error {
  return nil
}

// var _ TaskLoader = UserStore{} // Ошибка: Load(UserID) не Load(TaskID).
```

Это полезная строгость: интерфейс проверяет не только имя метода, но и смысл его параметров, выраженный типами.

Не создавайте типы ради каждого поля

Defined type — инструмент, а не обязательный ритуал. Хорошие кандидаты:

- идентификаторы разных доменных сущностей (`TaskID`, `UserID`);
- ограниченный набор значений (`TaskStatus`, `Priority`);
- единицы, которые опасно смешивать (`Bytes`, `RetryCount`), если это улучшает код;
- значения с собственными методами валидации, форматирования или разбора.

Плохие кандидаты — типы, которые не меняют читаемость, не защищают от реальной путаницы и требуют постоянных бессмысленных преобразований. Например, `type TaskTitle string` может быть полезен, если у него есть единый конструктор и правила длины; иначе это может стать только шумом.

Следующая страница перейдёт от конкретных domain types к параметрам типов: как написать алгоритм один раз для нескольких типов, не теряя проверки компилятора.

Параметры типов и generic-функции

Что меняют generics

Обычная функция получает значения известных типов:

```
func containsTaskID(ids []TaskID, wanted TaskID) bool {
  for _, id := range ids {
    if id == wanted {
      return true
    }
  }
  return false
}
```

Та же операция — поиск значения в слайсе — подходит для строк, чисел и других сравнимых типов. До generics были три типичных варианта:

- написать отдельную функцию для каждого типа;
- принять `[]any` и проверять типы во время выполнения;
- использовать reflection.

Первый вариант часто создаёт дублирование, два следующих теряют часть безопасности компилятора. Generic-функция описывает переменную типа `T`, а constraint ограничивает допустимые подстановки:

```
func Contains[T comparable](values []T, wanted T) bool {
  for _, value := range values {
    if value == wanted {
      return true
    }
  }

  return false
}
```

`T` здесь не значение, а параметр типа. При каждом вызове компилятор подставляет конкретный тип и проверяет, что он удовлетворяет `comparable`.

```
fmt.Println(Contains([]string{"open", "completed"}, "open")) // true
fmt.Println(Contains([]TaskID{1, 2, 3}, TaskID(2)))             // true
```

Синтаксис параметров типа

Список параметров типа находится после имени функции и до обычных параметров:

```
func FunctionName[T Constraint](value T) T {
  return value
}
```

Параметров может быть несколько:

```
func Keys[K comparable, V any](values map[K]V) []K {
  keys := make([]K, 0, len(values))
  for key := range values {
    keys = append(keys, key)
  }
  return keys
}
```

Вызов:

```
statuses := map[TaskID]string{
  1: "open",
  2: "completed",
}
```

```
ids := Keys(statuses) // Компилятор выводит K = TaskID, V = string.
```

Имена `T`, `K`, `V`, `E` привычны для коротких универсальных helpers. Если параметр обозначает роль, а не просто место, лучше выбрать читаемое имя: `ID`, `Result`, `Payload`.

Вывод параметров типа

В большинстве вызовов Go выводит тип из аргументов, поэтому квадратные скобки не нужны:

```
numbers := []int{10, 20, 30}

found := Contains(numbers, 20) // T = int
```

Иногда компилятору не из чего сделать вывод или выбор важен для чтения. Тогда тип указывают явно:

```
emptyIDs := []TaskID{}

// Тип элемента пустого литерала был бы неочевиден без []TaskID.
fmt.Println(Contains[TaskID](emptyIDs, 1))
```

Явная подстановка полезна редко. Не добавляйте `[string]` к каждому вызову, если вывод типа уже делает код короче и понятнее.

`any`: разрешено любое значение

`any` — alias для `interface{}`. Он означает «любой тип»:

```
func First[T any](values []T) (T, bool) {
  if len(values) == 0 {
    var zero T
    return zero, false
  }

  return values[0], true
}
```

`First` не сравнивает, не складывает и не вызывает методы `T`, поэтому достаточно `any`. Функция сохраняет конкретный тип:

```
title, ok := First([]string{"Купить молоко"})
// title имеет тип string, а не any.
```

```
taskID, ok := First([]TaskID{42})// taskID имеет тип TaskID.
```

Не путайте параметр типа `T any` с обычным параметром `any`:

```
func printValue(value any) {
  fmt.Println(value)
}

func First[T any](values []T) (T, bool) {
  // Возвращает тот же конкретный T, что был в слайсе.
  // ...
}
```

В первом случае тип информации стирается внутри `any`; чтобы использовать конкретный тип, обычно нужен type assertion или type switch. Во втором тип известен компилятору для каждого конкретного вызова.

`comparable`: можно сравнивать `==` и `!=`

Constraint `comparable` позволяет применять к `T` операторы `==` и `!=`, а также использовать `T` как ключ `map`:

```
func IndexOf[T comparable](values []T, wanted T) int {
  for index, value := range values {
    if value == wanted {
      return index
    }
  }

  return -1
}

func Frequency[T comparable](values []T) map[T]int {
  counts := make(map[T]int)
  for _, value := range values {
    counts[value]++
  }
  return counts
}
```

Подходят числа, строки, `bool`, указатели, каналы, массивы сравнимых элементов и структуры, если все их поля сравнимы. Не подходят слайсы, map и функции:

```
type TaskFilter struct {
  Statuses []TaskStatus // Слайс делает всю структуру несравнимой.
}

// Contains([][]string{{"open"}}, []string{"open"})
// Ошибка: []string не удовлетворяет comparable.
```

Ограничение не случайно: Go не определяет оператор `==` для слайсов и map. Если алгоритму нужно сравнивать такие значения, его контракт должен принять функцию сравнения:

```
func ContainsFunc[T any](values []T, wanted T, equal func(T, T) bool) bool {
  for _, value := range values {
    if equal(value, wanted) {
      return true
    }
  }
  return false
}

filtersEqual := func(left, right TaskFilter) bool {
  return slices.Equal(left.Statuses, right.Statuses)
}
```

`ContainsFunc` не «хуже» generic-функции с `comparable`: он честно показывает, что именно означает равенство для сложного значения.

Нулевое значение параметра типа

У любого конкретного `T` есть нулевое значение, но generic-код не всегда может записать его литералом. Используйте объявление переменной:

```
func Zero[T any]() T {
  var zero T
  return zero
}
```

Для `int` это `0`, для `string` — `""`, для указателя — `nil`, для структуры — её нулевое значение. Нулевое значение не всегда означает «данных нет», поэтому API `First` выше возвращает второй результат `bool`.

Не используйте `nil` как универсальный нуль:

```
// func Zero[T any]() T {
//     return nil // Неверно: T может быть int или struct.
// }
```

In-place операции и слайсы

Generics не меняют правила значений и ссылочных типов. Например, функция может фильтровать слайс без новой аллокации, но должна ясно документировать, что меняет исходный массив:

```
func FilterInPlace[T any](values []T, keep func(T) bool) []T {
  result := values[:0]
  for _, value := range values {
    if keep(value) {
      result = append(result, value)
    }
  }
  return result
}
```

Такая функция может быть полезной на горячем пути, но её контракт требует внимательности: возвращённый слайс разделяет backing array с исходным. Для большинства прикладного кода более очевидный вариант с новым слайсом лучше, даже если он универсален.

Когда generic-функция оправдана

Хорошие признаки:

- алгоритм и инварианты одинаковы для разных типов;
- параметр типа участвует в сигнатуре и сохраняет типовую информацию;
- constraint точно описывает нужную операцию;
- вызов проще и безопаснее дублирующих функций или `any`/reflection.

Плохие признаки:

- у функции только один реальный тип использования;
- бизнес-правила начинают прятаться за `T`, `K`, `V`;
- читателю нужно долго восстанавливать конкретные типы из сложных ограничений;
- типовая параметризация создана «на будущее», но пока не устраняет реальное дублирование.

На следующей странице рассмотрим, как писать собственные constraints и почему `~int` — не то же самое, что `int`.

Constraints, type sets и оператор `~`

Constraint — это интерфейс для параметра типа

В generic-коде интерфейс может описывать не объект с методами, а множество типов, которые разрешено подставить вместо `T`. Такой интерфейс называют constraint.

Самый простой constraint уже знаком:

```
func Clone[T any](values []T) []T {
  return append([]T(nil), values...)
}
```

`any` разрешает все типы. `comparable` разрешает типы, для которых доступны `==` и `!=`. Свой constraint можно назвать и использовать повторно:

```
type Number interface {
  int | int64 | float64
}

func Sum[T Number](values []T) T {
  var total T
  for _, value := range values {
    total += value
  }
  return total
}
```

Вертикальная черта означает union: `T` может быть `int`, `int64` **или** `float64`. Компилятор разрешает в теле только операции, которые корректны для каждого типа type set. Для перечисленных числовых типов `+` допустим.

Методы и type terms нельзя смешивать произвольно

Обычный интерфейс значений задаёт методы:

```
type Stringer interface {
  String() string
}
```

A constraint может объединить требование метода с семейством типов:

```
type TaskCode interface {
  ~string
  String() string
}
```

Такой constraint означает: тип должен иметь underlying type `string` и метод `String() string`. Важно: интерфейсы с type terms (`~string`, `int | int64`) предназначены для параметров типа. Их нельзя использовать как обычный тип переменной:

```
// var code TaskCode // Ошибка: constraint interface нельзя использовать как value type.
```

Не пытайтесь заменить этим привычные маленькие интерфейсы зависимостей Task Manager API. Например, `TaskRepository` описывает поведение конкретного объекта и остаётся обычным интерфейсом, который передают в service как значение.

Точное имя типа: `int`

Constraint с одним типом без `~` разрешает только этот точный тип:

```
type IntOnly interface {
  int
}

func DoubleInt[T IntOnly](value T) T {
  return value * 2
}

type RetryCount int

fmt.Println(DoubleInt(5)) // 10
// DoubleInt(RetryCount(5)) // Ошибка: RetryCount — не точный int.
```

Несмотря на underlying type `int`, `RetryCount` — отдельный defined type. Это именно то, что мы хотели получить в первой странице главы.

Семейство типов: `~int`

Тильда `~` расширяет type term: разрешён любой defined type с указанным underlying type, а также сам базовый тип.

```
type Integer interface {
  ~int | ~int64
}

func Double[T Integer](value T) T {
  return value * 2
}

type RetryCount int
type TaskID int64

fmt.Println(Double(RetryCount(3))) // 6, тип результата RetryCount
fmt.Println(Double(TaskID(42)))    // 84, тип результата TaskID
```

`~int` не означает «все числа» и не выполняет преобразование. Он описывает типы, underlying type которых ровно `int`. Например, `int32`, `uint` и `float64` не подходят.

Это особенно важно, когда generic-helper должен сохранять defined type результата. Функция с `func Double[T ~int](value T) T` вернёт `RetryCount`, а не голый `int`.

Почему constraints должны быть минимальными

Constraint — контракт алгоритма. Запрашивайте только те операции, которые использует тело функции.

Если функция лишь копирует слайс, ей не нужен `comparable`:

```
func Clone[T any](values []T) []T {
  return append([]T(nil), values...)
}
```

Если функция строит map частот, ей нужен именно `comparable`:

```
func Frequency[T comparable](values []T) map[T]int {
  counts := make(map[T]int)
  for _, value := range values {
    counts[value]++
  }
  return counts
}
```

Если функция сортирует, одного перечисления числовых типов часто недостаточно: нужно определить, что означает порядок, как сравнивать строки, пользовательские enum и поля

структуры. Вместо слишком широкого самодельного `Ordered` constraint нередко яснее принять `less func(left, right T) bool`:

```
func SortBy[T any](values []T, less func(left, right T) bool) {
  slices.SortFunc(values, func(left, right T) int {
    switch {
    case less(left, right):
      return -1
    case less(right, left):
      return 1
    default:
      return 0
    }
  })
}
```

Этот API прямо просит вызывающий код определить порядок. Для `Task` это часто полезнее, чем делать `Task` «упорядочиваемым» типом: сортировка по приоритету, title и дате — разные бизнес-операции.

Нельзя получить поле или метод «из underlying type»

Type set не сообщает, какие поля есть у структуры. Даже если вы перечислите несколько структурных типов, нельзя написать общий `value.ID`, пока constraint не требует метод:

```
type HasTaskID interface {
  TaskID() TaskID
}

func CollectTaskIDs[T HasTaskID](values []T) []TaskID {
  ids := make([]TaskID, 0, len(values))
  for _, value := range values {
    ids = append(ids, value.TaskID())
  }
  return ids
}
```

Для domain-моделей часто лучше вовсе не создавать generic helper, а написать явную функцию `TaskIDs(tasks []Task) []TaskID`: она проще для поиска, её бизнес-смысл ясен, а переиспользование может никогда не понадобиться.

Встраивание constraints

Constraint может включать другой constraint. Это удобно, когда новый контракт уточняет уже существующий:

```
type Integer interface {
  ~int | ~int64
}

type PositiveInteger interface {
  Integer
}
```

Но `PositiveInteger` выше вводит в заблуждение: type set не может доказать, что **значение** положительно. Название обещает больше, чем выражает constraint. Проверка значений остаётся обычной логикой:

```
func IsPositive[T Integer](value T) bool {
  return value > 0
}
```

Хорошие имена constraints говорят о разрешённых типах или доступных операциях: `Integer`, `StringLike`, `TaskCode`. Не называйте их `ValidID`, `NonEmptyString` или `PositiveNumber`, если это свойства конкретных значений.

Type approximation и эволюция API

Выбор между `int` и `~int` меняет публичный API generic-функции:

- `int` подходит, когда алгоритм намеренно предназначен только для встроенного `int`;
- `~int` подходит, когда алгоритм корректен для всего семейства defined types на `int` и должен сохранить их тип.

Добавление `~` делает функцию более гибкой, но не всегда более понятной. Для внутреннего helper, который вызывается с `RetryCount`, это может быть верный выбор. Для маленького API, который всегда работает с `int`, лишняя абстракция ничего не приносит.

Проверяйте ограничения компилятором

Тесты generic-кода должны проверять поведение для нескольких репрезентативных подстановок: базового типа и defined type с тем же underlying type. Но тест не может проверить, что неподходящий тип **не компилируется**, не превратив проект в намеренно ломающуюся сборку.

Обычно достаточно:

- держать constraint коротким и читаемым;
- показывать запрещённый вызов в комментарии;

- запускать `go test ./...` и при необходимости `go vet ./...`;
- не расширять type set без реального использования и отдельного решения об API.

Следующая страница объединит параметры и constraints в generic-типы и поможет решить, когда такая абстракция уместна в приложении.

Generic-типы и выбор абстракции

Generic-функции обобщают операцию. Generic-типы обобщают структуру данных или контейнер, который хранит значения неизвестного заранее типа. Тип объявляет параметры в квадратных скобках после имени:

```
type Name[T Constraint] struct {
  Value T
}
```

Пример: множество `Set`

`map[T]struct{}` — обычный идиоматичный способ хранить множество сравнимых значений. Generic-тип даёт ему имя и небольшой API:

```
type Set[T comparable] map[T]struct{}

func NewSet[T comparable](values ...T) Set[T] {
  set := make(Set[T], len(values))
  for _, value := range values {
    set.Add(value)
  }
  return set
}

func (set Set[T]) Add(value T) {
  set[value] = struct{}{}
}

func (set Set[T]) Has(value T) bool {
  _, exists := set[value]
  return exists
}

func (set Set[T]) Delete(value T) {
  delete(set, value)
}
```

```
func (set Set[T]) Len() int {  return len(set)}
```

Использование сохраняет тип:

```
allowedStatuses := NewSet(TaskStatusOpen, TaskStatusCompleted)
fmt.Println(allowedStatuses.Has(TaskStatusOpen)) // true

taskIDs := NewSet(TaskID(1), TaskID(2))
taskIDs.Add(3) // Нет необходимости преобразовывать в int64.
```

`T comparable` нужен, потому что `T` становится ключом `map`. Ограничение у generic-типа проверяется при создании конкретного `Set[T]`:

```
// var filters Set[[]TaskStatus] // Ошибка: []TaskStatus не comparable.
```

Нулевое значение `Set`

Нулевое значение map можно читать, удалять из него и проверять ключи, но нельзя добавлять элементы:

```
var taskIDs Set[TaskID]

fmt.Println(taskIDs.Has(1)) // false
// taskIDs.Add(1) // Panic: assignment to entry in nil map.
```

Это часть контракта map, а не особенность generics. Есть два ясных варианта дизайна:

1. требовать создание через `NewSet`;
2. сделать тип структурой с внутренней map и инициализировать её в методе `Add`.

Второй вариант даёт удобное нулевое значение, но требует pointer receiver:

```
type SafeSet[T comparable] struct {
  values map[T]struct{}
}

func (set *SafeSet[T]) Add(value T) {
  if set.values == nil {
    set.values = make(map[T]struct{})
  }
  set.values[value] = struct{}{}
}
```

```
func (set SafeSet[T]) Has(value T) bool { _, exists := set.values[value] return
exists}
```

Нет универсально правильного выбора. `Set[T] map[T]struct{}` проще и прозрачно показывает свойства map; `SafeSet[T]` полезен, если API специально обещает работоспособное нулевое значение. Не скрывайте это решение от пользователей типа.

Порядок и преобразование в слайс

Map не гарантирует порядок обхода. Поэтому метод `Values()` не должен обещать стабильную последовательность:

```
func (set Set[T]) Values() []T {
  values := make([]T, 0, len(set))
  for value := range set {
    values = append(values, value)
  }
  return values
}
```

Если HTTP API отдаёт значения набора, определите порядок отдельно. Для string-подобных статусов можно отсортировать копию; для `Task` лучше использовать конкретное бизнес-правило сортировки. Не делайте вывод «generic Set сам умеет сортироваться»: `comparable` не означает ordered, а универсальный порядок для любого `T` отсутствует.

Пример: явное наличие значения

Иногда нужно вернуть результат, который может отсутствовать, но нулевое значение `T` допустимо. Например, `TaskID(0)` может быть нулём типа, а пустой title — пустой строкой. Generic-тип `Optional` делает отсутствие явным:

```
type Optional[T any] struct {
  value T
  ok    bool
}

func Some[T any](value T) Optional[T] {
  return Optional[T]{value: value, ok: true}
}

func None[T any]() Optional[T] {
  return Optional[T]{}
}
```

```
func (optional Optional[T]) Get() (T, bool) {  return optional.value, optional.ok}
```

Пример использования:

```
func findCachedTitle(cache map[TaskID]string, id TaskID) Optional[string] {
  title, exists := cache[id]
  if !exists {
    return None[string]()
  }
  return Some(title)
}
```

Однако в обычном Go-коде `(T, bool)` часто проще и привычнее:

```
func findCachedTitle(cache map[TaskID]string, id TaskID) (string, bool) {
  title, exists := cache[id]
  return title, exists
}
```

`Optional[T]` оправдан, только если он действительно добавляет полезные операции, должен храниться как поле или делает сложный контракт яснее. Не добавляйте его лишь потому, что аналог есть в другом языке.

Generic-тип и интерфейс — разные оси

Generics отвечают на вопрос: «какой тип значения хранит структура или принимает алгоритм?» Интерфейс отвечает: «какое поведение требуется?» Их можно комбинировать, но одно не заменяет другое.

```
type Repository[T any, ID comparable] interface {
  Get(ctx context.Context, id ID) (T, error)
  Save(ctx context.Context, value T) error
}
```

Такой интерфейс технически возможен. Но для Task Manager он может быть избыточным: `TaskRepository` с методами, названными в терминах задач, легче читать и он не обещает ложную универсальность для всех будущих сущностей.

```
type TaskRepository interface {
  Get(ctx context.Context, id TaskID) (Task, error)
  Create(ctx context.Context, input CreateInput) (Task, error)
```

```
  Complete(ctx context.Context, id TaskID) (Task, error)}
```

Второй интерфейс выражает реальные операции домена. Generic CRUD-репозиторий часто скрывает важные различия: поиск, авторизацию, транзакции, частичное обновление, уникальные ограничения и пагинацию.

Где generics уместны в Task Manager API

В приложении можно рассмотреть generics для технических, повторяющихся частей:

- `Set[TaskID]` для дедупликации ID;
- маленькие helpers `Contains`, `Clone`, `Filter` в общем internal-пакете — только после появления нескольких реальных использований;
- тестовые helpers, которые делают одинаковые проверки для разных типов;
- технические контейнеры вроде `Optional[T]`, если они действительно упрощают несколько контрактов.

Обычно не стоит обобщать:

- `Task`, `CreateInput`, `TaskService`, обработчики Gin и HTTP-ответы;
- доменные правила приоритета, выполнения и валидации;
- SQL-репозиторий только ради единого CRUD-интерфейса;
- ошибки, когда код и поля ошибки относятся к конкретной предметной области.

Правило: сначала сделайте ясный конкретный API. Обобщайте после второго или третьего похожего случая, когда можете назвать общую операцию без потери предметного смысла.

Не путайте `[]T`, `[]any` и `any`

Слайс конкретного типа нельзя присвоить `[]any`, даже если каждый элемент можно сохранить в `any`:

```
taskIDs := []TaskID{1, 2}

// var values []any = taskIDs // Ошибка: []TaskID и []any имеют разное представление.
```

Generic-функция принимает `[]T` без такой потери:

```
func PrintAll[T any](values []T) {
  for _, value := range values {
    fmt.Println(value)
  }
}

PrintAll(taskIDs)
```

Не создавайте `[]any`, чтобы «сделать всё generic». Он уместен на действительно динамической границе: лог с произвольными атрибутами, декодированный JSON, плагинная система. В остальном предпочтительнее конкретный тип или параметр `T`.

Стоимость и читаемость

Generics не работают как шаблоны C++ в пользовательской модели и не требуют runtime-reflection для каждого вызова. Но это не повод использовать их ради предполагаемой производительности. Сначала измеряйте `go test -bench` и profile. Главная цена неудачной generic-абстракции обычно не CPU, а сложность API, сообщений компилятора и поддержки.

Перед добавлением type parameter спросите:

1. Есть ли минимум два реальных типа с одинаковым алгоритмом?
2. Достаточно ли простого constraint, который соответствует операциям в теле?
3. Сохранит ли вызов предметный смысл для следующего читателя?
4. Не будет ли явная функция или callback проще?
5. Нужен ли новый тип вообще, или достаточно `(T, bool)`, `[]T`, map либо маленького интерфейса?

Если на несколько вопросов ответ «нет», конкретная реализация обычно лучше. Далее — самопроверка и практика, где все решения будут проверяться через обычный компилятор Go.

Самопроверка: типы и generics

1. Что создаёт объявление type TaskID int64?

A. Только второе имя встроенного int64

B. Интерфейс для всех числовых значений

C. Новый defined type с underlying type int64

D. Generic-тип без параметров

2. Чем type LegacyTaskID = int64 отличается от type TaskID int64?

A. Alias можно использовать только внутри generic-функций

B. Alias автоматически добавляет методы int64

C. Alias запрещает хранить отрицательные значения

D. Alias не создаёт новый тип и совместим с int64 без преобразования

3. Почему type TaskID int64 может быть полезен в доменном коде?

A. Компилятор не даст случайно передать UserID вместо TaskID

B. Он отменяет необходимость валидировать ID из HTTP-запроса

C. Он автоматически проверяет наличие задачи в PostgreSQL

D. Он превращает JSON-строку в число без ошибки

4. Какой constraint нужен функции Contains, которая сравнивает элементы через ==?

A. any

B. nil

C. error

D. comparable

5. Какой тип нельзя подставить вместо T comparable?

A. TaskID с underlying type int64

B. string

C. struct с полями int и bool

D. []string

6. Что сохраняет T any в функции First[T any]([]T) (T, bool) по сравнению с параметром any?

A. Автоматическое JSON-кодирование

B. Возможность сравнивать любые значения через ==

C. Обязательную проверку nil для любого T

D. Конкретный тип результата для каждого вызова

7. Что означает ~int в constraint?

A. Только встроенный int

B. Тип, который может быть nil

C. int и defined types с underlying type int

D. Любое целое и дробное число

8. Подойдёт ли type RetryCount int для функции Double[T ~int](value T) T?

A. Нет, потому что defined types нельзя передавать в generic-код

B. Да, потому что underlying type RetryCount — int

C. Только если RetryCount реализует error

D. Только после преобразования RetryCount в string

9. Почему interface с type term ~string нельзя объявить как обычную переменную?

A. Строки нельзя хранить в интерфейсах

B. Это constraint interface, предназначенный для параметра типа, а не value type

C. У него нет метода Error()

D. Его можно использовать только в package main

10. Как получить нулевое значение неизвестного параметра типа T?

A. Всегда вернуть 0

B. Вызвать new(any)

C. Объявить var zero T и вернуть zero

D. Всегда вернуть nil

11. Какое свойство имеет нулевое значение Set[T], объявленного как type Set[T comparable] map[T]struct{}?

A. Оно автоматически инициализируется при первом Add

B. Оно содержит один нулевой элемент T

C. Нельзя даже вызвать len или Has

D. Можно читать и проверять ключи, но Add вызовет panic из-за nil map

12. Когда generic-абстракция скорее избыточна?

A. Функция сохраняет конкретный тип результата и убирает дублирование

B. Алгоритм одинаков для нескольких типов и constraint короткий

C. Тип используется для технического Set[TaskID]

D. Есть только один конкретный случай, а параметр T скрывает бизнес-смысл

Ответы к самопроверке

1. Новый defined type с underlying type int64

2. Alias не создаёт новый тип и совместим с int64 без преобразования

3. Компилятор не даст случайно передать UserID вместо TaskID

4. comparable

5. []string

6. Конкретный тип результата для каждого вызова

7. int и defined types с underlying type int

8. Да, потому что underlying type RetryCount — int

9. Это constraint interface, предназначенный для параметра типа, а не value type

10. Объявить var zero T и вернуть zero

11. Можно читать и проверять ключи, но Add вызовет panic из-за nil map

12. Есть только один конкретный случай, а параметр T скрывает бизнес-смысл

Задания: типы и generics

Эта практика не требует менять production-код учебного Task Manager API. Создайте отдельный небольшой Go-модуль или каталог внутри своего учебного проекта, чтобы экспериментировать с типами и запускать `go test ./...`. Цель — освоить компилируемые контракты, а не заменить все конкретные типы generic-версиями.

Задание 1. Семантические идентификаторы

Объявите:

```
type TaskID int64
type UserID int64
```

Напишите `TaskPath(id TaskID) string`, которая возвращает путь вида `/tasks/42`, и `ParseTaskID(value string) (TaskID, error)`, которая принимает только положительный ID.

Критерии проверки:

- используются defined types без `=`;
- `TaskPath` принимает только `TaskID`;
- внутри `TaskPath` есть явное преобразование к `int64` перед `strconv.FormatInt`;
- `ParseTaskID` оборачивает ошибку разбора через `%w`;
- `0` и отрицательные значения возвращают ошибку;
- успешный разбор возвращает `TaskID`, а не `int64`.

Подсказка

```
id, err := strconv.ParseInt(value, 10, 64)
if err != nil {
  return 0, fmt.Errorf("разобрать ID задачи: %w", err)
}
```

Решение

```
package task

import (
  "errors"
  "fmt"
  "strconv"
)

type TaskID int64
type UserID int64

func TaskPath(id TaskID) string {
  return "/tasks/" + strconv.FormatInt(int64(id), 10)
}

func ParseTaskID(value string) (TaskID, error) {
  id, err := strconv.ParseInt(value, 10, 64)
  if err != nil {
    return 0, fmt.Errorf("разобрать ID задачи: %w", err)
  }
  if id < 1 {
    return 0, errors.New("ID задачи должен быть положительным")
  }

  return TaskID(id), nil
}
```

Этот код намеренно не позволит вызвать `TaskPath(UserID(42))`: это и есть защита смысла типов.

Задание 2. Generic-функция поиска

Напишите:

```
func Contains[T comparable](values []T, wanted T) bool
```

Проверьте её тестами для `[]string` и `[]TaskID`. Добавьте пример в комментарии, почему `[][]string` не скомпилируется с этим constraint.

Критерии проверки:

- параметр типа называется и ограничен `comparable`;
- поиск завершается сразу при первом совпадении;
- не применяется `any`, reflection или преобразование элементов в строку;
- тесты проверяют существующее и отсутствующее значение;
- вызов с `TaskID` не теряет defined type.

Подсказка

```
for _, value := range values {
  if value == wanted {
    return true
  }
}
```

Решение

```
package collection

type TaskID int64

func Contains[T comparable](values []T, wanted T) bool {
  for _, value := range values {
    if value == wanted {
      return true
    }
  }
  return false
}
```

```
func TestContains(t *testing.T) {
  if !Contains([]string{"open", "completed"}, "completed") {
    t.Fatal("Contains не нашёл строку")
  }
  if Contains([]TaskID{1, 2}, TaskID(3)) {
    t.Fatal("Contains нашёл отсутствующий TaskID")
  }
}

// Contains([][]string{{"open"}}, []string{"open"})
// не скомпилируется: []string не удовлетворяет comparable.
```

Задание 3. Constraint с `~`

Создайте constraint `Integer`, который принимает встроенные `int`, `int64`, а также defined types с такими underlying types. Напишите:

```
func Sum[T Integer](values []T) T
```

Проверьте, что функция работает для `[]int` и `[]RetryCount`, где `type RetryCount int`. Результат второго вызова должен иметь тип `RetryCount`.

Критерии проверки:

- constraint использует `~int | ~int64`;
- сумма начинается с нулевого значения `T`, а не с `int(0)`;
- не используется преобразование каждого элемента в `int64`;
- `RetryCount` проходит проверку компилятора без специального overload;
- комментарий объясняет, что constraint не проверяет положительность значений.

Подсказка

```
type Integer interface {
  ~int | ~int64
}
```

Решение

```
package metrics

type Integer interface {
  ~int | ~int64
}

type RetryCount int

func Sum[T Integer](values []T) T {
  var total T
  for _, value := range values {
    total += value
  }
  return total
}
```

```
func TestSum(t *testing.T) {
  if got := Sum([]int{1, 2, 3}); got != 6 {
    t.Fatalf("Sum(int) = %d, want 6", got)
  }

  got := Sum([]RetryCount{1, 2, 3})
  if got != RetryCount(6) {
    t.Fatalf("Sum(RetryCount) = %d, want 6", got)
  }
}
```

`Integer` ограничивает **типы**, а не значения: `Sum([]RetryCount{-1, 2})` разрешён. Проверка положительности — отдельное бизнес-правило.

Задание 4. Generic `Set` с явным контрактом

Реализуйте `Set[T comparable]` на базе `map[T]struct{}` с методами `Add`, `Has`, `Delete`, `Len` и функцией-конструктором `NewSet`.

Документируйте один из вариантов поведения нулевого значения:

- либо `Set` нужно создавать только через `NewSet`;
- либо реализуйте `SafeSet[T]` со структурой и ленивой инициализацией в `Add`.

Добавьте тест на дедупликацию двух одинаковых `TaskID`.

Критерии проверки:

- `T` ограничен `comparable`;
- повторный `Add` не увеличивает размер;
- `Has` возвращает `false` для отсутствующего значения;
- `Delete` не вызывает panic для отсутствующего значения;
- выбранное правило нулевого значения записано в doc comment или README;
- тест не рассчитывает на порядок обхода map.

Подсказка

```
type Set[T comparable] map[T]struct{}

func (set Set[T]) Add(value T) {
  set[value] = struct{}{}
}
```

Решение

```
package collection

type TaskID int64

// Set хранит уникальные значения. Создавайте Set через NewSet:
// нулевое значение Set нельзя изменять, потому что его map равна nil.
type Set[T comparable] map[T]struct{}

func NewSet[T comparable](values ...T) Set[T] {
  set := make(Set[T], len(values))
  for _, value := range values {
    set.Add(value)
  }
  return set
}

func (set Set[T]) Add(value T) {
  set[value] = struct{}{}
}

func (set Set[T]) Has(value T) bool {
  _, exists := set[value]
  return exists
}

func (set Set[T]) Delete(value T) {
  delete(set, value)
}

func (set Set[T]) Len() int {
  return len(set)
}
```

▼ Сведения

```
func TestSetDeduplicatesTaskIDs(t *testing.T) {
  set := NewSet(TaskID(1), TaskID(1), TaskID(2))

  if got := set.Len(); got != 2 {
    t.Fatalf("Len() = %d, want 2", got)
  }
  if !set.Has(TaskID(2)) {
    t.Fatal("Set не содержит TaskID(2)")
  }

  set.Delete(TaskID(2))
  if set.Has(TaskID(2)) {
    t.Fatal("Delete не удалил TaskID(2)")
  }
}
```

Задание 5. Откажитесь от лишнего generic API

В Task Manager API нужно сформировать для HTTP-ответа список заголовков задач, отсортированный сначала по приоритету, затем по ID. Коллега предлагает API:

```
type Entity[ID comparable] interface {
  ID() ID
}

func SortedTitles[T Entity[ID], ID comparable](values []T) []string
```

Напишите вместо него конкретную функцию:

```
func SortedTaskTitles(tasks []Task) []string
```

Она не должна менять исходный слайс. Добавьте короткое объяснение в комментарии или README, почему конкретное решение предпочтительнее.

Критерии проверки:

- функция копирует слайс перед сортировкой;
- сортировка имеет ясное правило: приоритет по убыванию, затем ID по возрастанию;
- извлечение title происходит после сортировки;
- нет generic-интерфейса и искусственного метода `ID()` только ради абстракции;
- объяснение упоминает бизнес-правило сортировки задач.

Подсказка

```
ordered := append([]Task(nil), tasks...)
slices.SortFunc(ordered, func(left, right Task) int {
  // Верните отрицательное число, если left должен идти раньше right.
})
```

Решение

```
package task

import "slices"

type TaskID int64

type Task struct {
  ID       TaskID
  Title    string
  Priority int
}

// SortedTaskTitles выражает бизнес-правило Task Manager: сначала высокий
// приоритет, затем меньший ID. Generic Entity скрывал бы этот смысл.
func SortedTaskTitles(tasks []Task) []string {
  ordered := append([]Task(nil), tasks...)
  slices.SortFunc(ordered, func(left, right Task) int {
    if left.Priority != right.Priority {
      return right.Priority - left.Priority
    }
    if left.ID < right.ID {
      return -1
    }
    if left.ID > right.ID {
      return 1
    }
    return 0
  })

  titles := make([]string, 0, len(ordered))
  for _, task := range ordered {
    titles = append(titles, task.Title)
  }
```

▼ Сведения

```
    return titles}
```

В production-коде не вычитайте два `int`, если значения могут быть достаточно большими для переполнения. Здесь приоритет имеет небольшой ограниченный диапазон; для более общего числового поля сравните значения через `<` и `>` так же, как ID.

Проверьте себя

Перед следующей главой убедитесь, что вы можете:

- выбрать defined type для доменного значения и alias для миграции совместимости;
- объяснить отличие `T any` от обычного `any`;
- применить `comparable` только при необходимости `==`, `!=` или ключа map;
- написать constraint с `~`, который сохраняет defined type результата;
- спроектировать нулевое значение generic-типа и задокументировать его;
- оставить конкретную функцию конкретной, когда она выражает бизнес-операцию Task Manager API.

Рекомендации по коду: типы и generics

Defined types и alias

- Создавайте defined type через `type Name BaseType`, когда нужно выразить доменный смысл, предотвратить путаницу значений или добавить методы.
- Используйте type alias через `type Name = ExistingType` главным образом для совместимости, миграции или переэкспорта; alias не добавляет типобезопасность.
- Валидируйте внешние данные до преобразования в domain type: `TaskID(value)` не делает ID существующим или положительным.
- Не плодите типы для каждого поля. Новый тип должен улучшать читаемость, защищать реальную границу или нести собственное поведение.
- Не пытайтесь добавлять методы к типам из другого пакета; используйте обёртку, helper или композицию.

Generic-функции и constraints

- Обобщайте алгоритм, а не бизнес-сущность. `Contains`, `Clone` и `Set` могут быть универсальны; создание задачи и HTTP-контракт — обычно нет.
- Начинайте с самого слабого constraint: `any`, если операции над `T` не требуются; `comparable`, только если нужны `==`, `!=` или map key.
- Используйте `~int`, `~string` и другие approximation terms, когда алгоритм должен работать с defined types на том же underlying type и сохранять их в результате.
- Не называйте constraint по свойству значения (`PositiveInteger`, `ValidID`), если он описывает только множество типов. Валидируйте значения отдельной логикой.

- Для сложного равенства, сортировки или сравнения принимайте явную функцию (`equal`, `less`), а не пытайтесь расширить constraint до нечитаемого type set.
- Возвращайте нулевое значение unknown `T` через `var zero T`; не возвращайте `nil`, если `T` может быть числом или структурой.
- Не рассчитывайте, что слайс `[]T` можно присвоить `[]any`. Если нужен обобщённый код, принимайте `[]T`.

Generic-типы

- Документируйте нулевое значение generic-типа так же, как у обычного: например, можно ли вызвать `Add` у нулевого `Set`.
- Помните, что map не гарантирует порядок. API `Set.Values()` не должен неявно обещать сортировку.
- Не добавляйте `Optional[T]`, Result-типы и универсальные коллекции только потому, что они есть в другом языке. Сначала сравните их с привычными `(T, bool)`, `error`, `[]T` и `map`.
- Параметризованный repository или service вводите только при доказанной общей модели операций. Не скрывайте за CRUD-абстракцией авторизацию, транзакции, инварианты и разный жизненный цикл сущностей.

Проверка и эволюция

- Тестируйте generic-алгоритм на встроенном и defined type с тем же underlying type.
- Держите type set компактным: расширение `int | int64` до большого списка — изменение публичного контракта.
- Оценивайте generics сначала по понятности API и отсутствию дублирования, а производительность — только по benchmark и profile.
- Перед добавлением type parameter убедитесь, что есть несколько реальных применений и одно ясное имя общей операции.

Итоговый чек-лист

- [ ] Новый type — defined type, если нужна защита смысла, а alias — только для совместимости.
- [ ] Внешние строки, JSON и path-параметры проверяются до преобразования в domain type.
- [ ] Constraint соответствует операциям в теле функции и не шире необходимого.
- [ ] `~` добавлен осознанно, чтобы поддержать семейство defined types.
- [ ] Generic-контейнер документирует нулевое значение, изменяемость и порядок.
- [ ] Domain-операции Task Manager не спрятаны в искусственный `T`/`K`/`V` API.
- [ ] Тесты подтверждают поведение для нескольких конкретных подстановок типов.

Продвинутый Go C: конкурентные паттерны

Цель главы

Научиться строить ограниченную и отменяемую конкурентную работу: выбирать worker pool, fan-out/fan-in или pipeline, распространять `context` между goroutine, корректно останавливать группу работников и находить утечки goroutine.

Основные главы уже показали `go`, каналы, `WaitGroup`, `select`, `Mutex` и `context`. Здесь важна не новая синтаксическая конструкция, а проектирование полного жизненного цикла:

```
создать работу → ограничить параллелизм → передать результат
      ↓                  ↓                        ↓
принять отмену ← закрыть каналы владельцем ← дождаться завершения
```

Конкурентный код полезен только тогда, когда независимые операции действительно могут перекрываться: запросы к нескольким внешним сервисам, обработка списка файлов, отправка уведомлений. Он увеличивает число состояний, поэтому сначала нужен понятный последовательный вариант, измерение или явная причина для параллелизма.

Карта главы

1. **Эта страница** — worker pool, bounded parallelism и backpressure.
2. `Fan-out`, `fan-in` и pipeline — передача данных и владение каналами между этапами.
3. `errgroup`, отмена и graceful shutdown — группа задач с первой ошибкой и управляемая остановка работников.
4. Утечки goroutine и диагностика — типичные утечки, тестирование и профили goroutine.

Прежде чем запускать goroutine

Для каждой новой goroutine должен быть ответ на пять вопросов:

1. **Кто её запускает?** Где находится владелец жизненного цикла?
2. **Когда она заканчивает работу?** Закрытие канала, отмена context, конечное число задач, сигнал shutdown?
3. **Как она сообщает об ошибке или результате?** Канал, возвращаемая ошибка `errgroup`, callback, запись в защищённое хранилище?
4. **Что случится, если получатель больше не ждёт?** Отправитель не должен навечно заблокироваться.
5. **Кто и когда ждёт её завершения?** `WaitGroup`, `errgroup.Wait`, отдельный done-канал?

Если хотя бы один пункт неясен, добавление `go` почти наверняка создаст зависание, утечку или непредсказуемое завершение процесса.

Зачем ограничивать параллелизм

Наивный подход запускает goroutine для каждой задачи:

```
for _, id := range taskIDs {
  go refreshPriority(ctx, id)
}
```

Для пяти задач это может быть приемлемо. Для десяти тысяч он создаст тысячи одновременных запросов, соединений, таймеров и объектов в памяти. Внешний сервис может получить всплеск нагрузки, локальный HTTP transport — исчерпать соединения, а база — connection pool.

Больше goroutine не означает быстрее. Реальная верхняя граница определяется самым дефицитным ресурсом: лимитом API, числом соединений БД, CPU, памятью, файловыми дескрипторами или допустимой задержкой.

Worker pool

Worker pool создаёт фиксированное число worker-goroutine. Они читают задания из общего канала, пока producer не закроет его или общий context не будет отменён.

Для примера Task Manager обновляет подсказки приоритета задач через внешний клиент:

```
type PriorityRefresher interface {
  Refresh(ctx context.Context, taskID TaskID) error
}

func RefreshPriorities(
  ctx context.Context,
  refresher PriorityRefresher,
  taskIDs []TaskID,
  workerCount int,
) error {
  if workerCount < 1 {
    return errors.New("число worker должно быть больше нуля")
  }

  jobs := make(chan TaskID)
  var workers sync.WaitGroup
  var errorsMu sync.Mutex
  var refreshErrors []error

  worker := func() {
    defer workers.Done()
```

```
    for {     select {     case <-ctx.Done():         return     case taskID, open
:= <-jobs:       if !open {          return       }       if err :=
refresher.Refresh(ctx, taskID); err != nil {           errorsMu.Lock()
refreshErrors = append(refreshErrors,           fmt.Errorf("обновить приоритет
задачи %d: %w", taskID, err),          )          errorsMu.Unlock()       }     }
} } workers.Add(workerCount)  for range workerCount {   go worker()  } for _,
taskID := range taskIDs {   select {   case <-ctx.Done():      close(jobs)
workers.Wait()      return errors.Join(append(refreshErrors, ctx.Err())...)     case
jobs <- taskID:     } } close(jobs)  workers.Wait()  return
errors.Join(refreshErrors...)}
```

Этот вариант имеет политику **best-effort**: ошибка одной задачи сохраняется, но другие задачи продолжают выполняться; после завершения pool вызывающий получает `errors.Join`. Доступ к `refreshErrors` защищён mutex, а чтение происходит только после `workers.Wait`. Для политики fail-fast лучше применить `errgroup` на следующей странице.

Инварианты worker pool

- `workerCount` ограничивает число одновременно выполняемых `Refresh`;
- только producer закрывает `jobs`, потому что он единственный знает, что заданий больше не будет;
- worker не закрывает `jobs`: у него есть конкурирующие отправители/получатели и нет права решать за producer;
- `Wait` вызывается после закрытия/отмены входного потока, чтобы worker успели выйти;
- каждый блокирующий `send` и `receive` участвует в `select` с `ctx.Done()` там, где отмена должна его разблокировать.

Буфер jobs и backpressure

Небуферизированный `jobs` даёт немедленный backpressure: producer может передать следующее задание, только когда какой-то worker готов принять текущее.

```
producer ── job ──► свободный worker
    ▲                    │
    └──── ждёт, пока ────┘
```

Буфер разрешает producer опередить worker лишь на ограниченное число заданий:

```
jobs := make(chan TaskID, workerCount*2)
```

Это полезно, если получение следующего задания очень дёшево, а небольшая очередь сглаживает короткие колебания скорости. Но буфер не является «ускорителем» сам по себе и не решает бесконечный входной поток. Когда буфер заполнен, отправка снова блокируется — это и есть backpressure.

Выбирайте размер из ограничений системы, а не из случайного большого числа:

Сценарий	Типичное решение
Нужна строгая естественная обратная связь	небуферизированный канал или маленький буфер
Нужно кратко сгладить разную скорость этапов	ограниченный буфер с объяснённым размером
Вход быстрее обработки постоянно	ограничить/отклонить вход, сохранять в устойчивую очередь или изменить продуктовую политику
Нельзя потерять события	не заменять переполнение бесконечной памятью; использовать durable broker/БД и подтверждения

`len(jobs)` и `cap(jobs)` подходят для наблюдаемости или отладки, но не для решения «можно ли отправить»: между проверкой и send состояние меняется другой goroutine. Сам `select`/send — единственная надёжная синхронизация.

Ограничение параллелизма семафором на buffered channel

Если задачи уже запускаются в разных местах и не нужен общий job queue, ограничить одновременно активные операции можно каналом-семафором:

```
func RefreshAll(ctx context.Context, client PriorityRefresher, taskIDs []TaskID,
limit int) error {
  if limit < 1 {
    return errors.New("limit должен быть больше нуля")
  }

  semaphore := make(chan struct{}, limit)
  var group sync.WaitGroup
  errs := make(chan error, len(taskIDs))

  for _, taskID := range taskIDs {
    select {
    case semaphore <- struct{}{}:
    case <-ctx.Done():
      group.Wait()
      return ctx.Err()
    }

    group.Add(1)
    go func(id TaskID) {
      defer group.Done()
      defer func() { <-semaphore }()
```

```
      if err := client.Refresh(ctx, id); err != nil {        select {       case
errs <- fmt.Errorf("обновить приоритет задачи %d: %w", id, err):         case <-
ctx.Done():         }      }    }(taskID)  } group.Wait() close(errs)  var allErrors
[]error  for err := range errs {    allErrors = append(allErrors, err)  } return
errors.Join(allErrors...)}
```

Семафор содержит token для каждой активной операции. Отправка token перед запуском блокируется, когда достигнут `limit`; defer возвращает token при любом выходе worker. Канал ошибок имеет буфер длиной входа, поэтому worker не зависнет, если сбор ошибок начнётся только после `Wait`.

Однако эта модель всё ещё создаёт по goroutine на каждую задачу — просто часть из них ждёт token до запуска. Для большого или неограниченного потока лучше worker pool: он ограничивает и число активных запросов, и число живых goroutine. Для fail-fast политики пример с семафором также неудобнее: следующая страница покажет `errgroup.SetLimit`.

> **Не закрывайте семафор**
>
> Семафор — это не поток данных. Его обычно не закрывают: завершение работы выражают context, прекращением создания новых задач и ожиданием `WaitGroup` / `errgroup`. Закрытие может привести к успешному receive нулевого значения и запутать протокол.

Когда pool не нужен

- Операции зависят друг от друга: сначала нужно получить задачу, затем только для неё запросить приоритет. Это больше похоже на pipeline.
- Есть несколько заранее известных независимых операций: часто достаточно нескольких `errgroup.Go`.
- Операция CPU-bound, а лимит определяется числом ядер и измерениями, а не произвольным числом worker.
- Нужна очередь между процессами, повторная доставка после рестарта или приоритеты: канал в памяти не заменяет durable queue.

В следующей странице разберём, как направлять данные через несколько этапов и как безопасно объединять результаты многих источников.

`Fan-out`, `fan-in` и pipeline

Три разных формы работы

Эти термины часто используют вместе, но они решают разные задачи:

```
pipeline:  source → validate → enrich → persist
fan-out:   одна очередь → несколько одинаковых worker
```

```
fan-in:    несколько источников → один поток результатов
```

- **Pipeline** делит обработку на последовательные этапы. Каждый следующий этап зависит от результата предыдущего.
- **Fan-out** распределяет независимые единообразные задания между несколькими consumer.
- **Fan-in** объединяет несколько входных каналов в один.

Все три паттерна должны иметь владельца каналов и путь отмены. Иначе быстрый этап заблокируется на медленном, а завершившийся consumer оставит producer навечно ждать отправку.

Pipeline: один этап — одна ответственность

Пусть фоновая команда экспортирует задачи. Упрощённый pipeline может выглядеть так:

```
TaskID source → load Task → build ExportRow → write output
```

Каждая функция принимает входной канал и возвращает выходной только для получения. Она сама владеет своим выходным каналом и поэтому закрывает его, когда больше не будет отправок.

```
type ExportRow struct {
  ID       TaskID
  Title    string
  Priority int
}

func loadTasks(
  ctx context.Context,
  ids <-chan TaskID,
  repository TaskRepository,
) <-chan Task {
  out := make(chan Task)

  go func() {
    defer close(out)
    for {
      select {
      case <-ctx.Done():
        return
      case id, open := <-ids:
        if !open {
          return
        }
```

```
        task, err := repository.Get(ctx, id)        if err != nil {           //
Для краткости пример пропускает ошибку.          // В production она должна попасть в
общий error path.          continue       }        select {       case out <- task:
case <-ctx.Done():           return       }      }    } }()  return out}func
buildRows(ctx context.Context, tasks <-chan Task) <-chan ExportRow { out := make(chan
ExportRow) go func() {     defer close(out)    for {      select {      case <-
ctx.Done():         return     case task, open := <-tasks:         if !open {
return       }        row := ExportRow{          ID:       task.ID,           Title:
task.Title,           Priority: task.Priority,         }        select {       case
out <- row:         case <-ctx.Done():           return       }      }    } }()
return out}
```

Использование:

```
rows := buildRows(ctx, loadTasks(ctx, ids, repository))
for row := range rows {
  // Записать строку экспорта.
}
```

Этот пример прежде всего показывает ownership. В настоящем API нельзя молча пропустить ошибку `repository.Get`; следующая страница добавит единый путь ошибок через `errgroup`.

Правило закрытия каналов

Для каждого канала запишите одно предложение:

> **Кто отправляет все значения, тот закрывает канал после последней отправки.**

В pipeline `loadTasks` — единственный отправитель `out`, поэтому внутри этой функции находится `defer close(out)`. Consumer `buildRows` не закрывает вход `tasks`: он не владеет его producer. Входной канал `ids` закрывает исходный producer, который знает, что новых ID не будет.

Если у канала несколько sender, сами sender не могут независимо закрывать его. Нужна отдельная координирующая goroutine, которая дождётся всех sender и вызовет `close(out)` ровно один раз.

Fan-out: несколько worker читают одну очередь

Fan-out для обогащения задач создаёт одну очередь входа и несколько одинаковых worker:

```
func startEnrichers(
  ctx context.Context,
  workerCount int,
  in <-chan Task,
  client PriorityClient,
```

```
) <-chan Task {   out := make(chan Task)   var workers sync.WaitGroup
workers.Add(workerCount)   for range workerCount {    go func() {       defer
workers.Done()      for {        select {        case <-ctx.Done():           return
case task, open := <-in:           if !open {            return         }
priority, err := client.Suggest(ctx, task.Title)          if err != nil {
return // Ошибка должна быть передана владельцу pipeline.          }
task.Priority = priority           select {          case out <- task:          case
<-ctx.Done():              return         }        }      }    }() } go func() {
workers.Wait()    close(out) }()  return out}
```

Несколько worker совместно получают значения из `in`: каждую задачу получит ровно один из них. Координатор ждёт всех worker и закрывает `out`; закрыть его из каждого worker означало бы риск `send on closed channel` у остальных.

Параллельность меняет порядок: задачи могут прийти в `out` не в том же порядке, в каком они вошли. Если API требует порядок по ID или времени, восстановите его отдельным этапом: сохраните sequence number, соберите результаты и отсортируйте. Не полагайтесь на планировщик goroutine.

Fan-in: объединить несколько потоков

`fan-in` полезен, когда несколько producer дают одинаковый тип данных: например, события из ручного запуска и планировщика. Функция `merge` запускает по одной goroutine на input, пересылает значения в общий output и закрывает его после завершения всех пересылок.

```
func merge[T any](ctx context.Context, inputs ...<-chan T) <-chan T {
  out := make(chan T)
  var forwarders sync.WaitGroup

  forward := func(input <-chan T) {
    defer forwarders.Done()
    for {
      select {
      case <-ctx.Done():
        return
      case value, open := <-input:
        if !open {
          return
        }
        select {
        case out <- value:
        case <-ctx.Done():
          return
        }
      }
```

```
    } } forwarders.Add(len(inputs))  for _, input := range inputs {   go
forward(input)  } go func() {    forwarders.Wait()   close(out) }()  return out}
```

Важны два `select`:

- первый не даёт forwarder вечно ждать значение из input после отмены;
- второй не даёт ему застрять на отправке в `out`, если consumer закончил раньше.

`merge` не закрывает входные каналы: их владельцы — исходные producer. Нулевое количество inputs допустимо: `WaitGroup` сразу завершится, и `out` будет закрыт.

Cancellation должна идти против потока данных

Данные обычно текут вниз pipeline: source → transform → sink. Отмена должна быть доступна **каждому** этапу, в том числе source, который иначе может зависнуть на send.

```
data:    source ──────► transform ──────► sink
context:   └──────────── cancel ◄──────────┘
```

Следующий код небезопасен, если consumer прекращает чтение после первой ошибки:

```
out <- value // Может ждать вечно.
```

Для отменяемого pipeline отправка должна иметь альтернативу:

```
select {
case out <- value:
case <-ctx.Done():
  return
}
```

То же относится к producer, отправляющему jobs в worker pool, и к worker, сообщающему результат. Context не завершает goroutine принудительно — каждая блокирующая точка должна добровольно заметить отмену.

Pipeline и backpressure

Небуферизированные связи между этапами создают естественный backpressure: быстрый `loadTasks` не сможет бесконечно обогнать медленный writer. Буфер между этапами допускает ограниченный задел, но добавляет память, задержку остановки и больше задач, которые уже приняты, но больше не нужны.

Не добавляйте большие буферы, чтобы «исправить зависание». Если pipeline зависает, выясните, кто перестал читать, кто не закрыл канал и кто не слушает отмену. Буфер может скрыть ошибку до

первой большой нагрузки.

Когда не нужен канал

Не каждая последовательность функций — pipeline. Если данные помещаются в память и нужны только одной goroutine, обычные вызовы и слайсы проще:

```
tasks, err := repository.List(ctx)
if err != nil {
  return err
}

rows := make([]ExportRow, 0, len(tasks))
for _, task := range tasks {
  rows = append(rows, makeRow(task))
}
```

Каналы оправданы, когда есть конкурентные producer/consumer, streaming, backpressure или отменяемая передача между независимыми этапами. Следующая страница соединит эти идеи с единым управлением ошибками и graceful shutdown.

`errgroup`, отмена и graceful shutdown

Почему одного `WaitGroup` недостаточно

`sync.WaitGroup` хорошо отвечает на вопрос «все goroutine завершились?», но не передаёт ошибку и не отменяет остальные работы после первой неудачи. В результате появляется ручной код: канал ошибок, `sync.Once`, `cancel`, закрытие каналов и порядок ожидания.

Для набора связанных операций часто подходит `golang.org/x/sync/errgroup`:

```
go get golang.org/x/sync/errgroup
```

`errgroup.Group` запускает функции, ждёт их через `Wait()` и возвращает первую ненулевую ошибку. `errgroup.WithContext` создаёт context, отменяемый при первой ошибке группы. Это естественная модель для операции «всё должно выполниться, иначе остальное больше не нужно».

Несколько независимых операций

Task Manager может параллельно запросить настройки уведомлений и правила приоритета перед запуском batch-операции:

```
func loadDependencies(ctx context.Context, client *Client) (Rules, Settings, error) {
  group, groupCtx := errgroup.WithContext(ctx)

  var rules Rules
  var settings Settings

  group.Go(func() error {
    loadedRules, err := client.LoadRules(groupCtx)
    if err != nil {
      return fmt.Errorf("загрузить правила приоритета: %w", err)
    }
    rules = loadedRules
    return nil
  })

  group.Go(func() error {
    loadedSettings, err := client.LoadSettings(groupCtx)
    if err != nil {
      return fmt.Errorf("загрузить настройки уведомлений: %w", err)
    }
    settings = loadedSettings
    return nil
  })

  if err := group.Wait(); err != nil {
    return Rules{}, Settings{}, err
  }

  return rules, settings, nil
}
```

Записи `rules` и `settings` выполняют разные goroutine, а читаются только после `group.Wait()`, поэтому между ними нет гонки. Если одна операция вернула ошибку, `groupCtx` отменяется; другая операция должна передавать его в HTTP/SQL-вызовы или проверять `groupCtx.Done()`, чтобы завершиться быстро.

> **Первая ошибка не означает мгновенную остановку**
>
> `errgroup` вызывает cancel, но не убивает goroutine. `Wait()` вернётся только после завершения всех функций. Если функция игнорирует context или навечно ждёт канал, группа всё равно зависнет. Отмена должна доходить до каждого блокирующего места.

Ограничение `errgroup.SetLimit`

`SetLimit` ограничивает число одновременно активных функций группы. Это часто компактнее worker pool для конечного списка задач:

```
func refreshPriorities(
  ctx context.Context,
  client PriorityRefresher,
  taskIDs []TaskID,
  limit int,
) error {
  if limit < 1 {
    return errors.New("limit должен быть больше нуля")
  }

  group, groupCtx := errgroup.WithContext(ctx)
  group.SetLimit(limit)

  for _, taskID := range taskIDs {
    id := taskID
    group.Go(func() error {
      if err := client.Refresh(groupCtx, id); err != nil {
        return fmt.Errorf("обновить приоритет задачи %d: %w", id, err)
      }
      return nil
    })
  }

  return group.Wait()
}
```

`group.Go` блокируется, когда активных функций уже `limit`. Это создаёт backpressure в goroutine, которая добавляет работу. Поэтому проверяйте **`groupCtx`** в цикле, если producer должен перестать добавлять работу после замеченной ошибки другой функции:

```
for _, taskID := range taskIDs {
  if err := groupCtx.Err(); err != nil {
    break
  }

  id := taskID
  group.Go(func() error {
    return client.Refresh(groupCtx, id)
```

```
  })}
```

Проверка выполняется до `Go`, поэтому ошибка может всё равно прийти в момент, когда следующий `Go` уже ждёт свободное место. Это нормальная граница API `errgroup`: операции обязаны получать `groupCtx` и быстро завершаться, даже если одна из них успела стартовать после cancel. Если для входного потока нужно строго не принимать новые jobs после ошибки, используйте явный worker pool с `select` по каналу jobs и `groupCtx.Done()`.

`SetLimit` нельзя менять, пока в группе есть активные goroutine. Не вызывайте `Go` после начала `Wait`: жизненный цикл группы должен быть простым — создать, добавить работы, один раз дождаться.

Для сценария, где producer не должен блокироваться, есть `TryGo`: он возвращает `false`, если лимит достигнут. Но тогда вызывающий код обязан выбрать политику: поставить задачу в очередь, отклонить её, выполнить синхронно или записать для повтора. Игнорировать `false` означает тихо потерять работу.

Fail-fast или собрать все ошибки

`errgroup` возвращает первую ошибку. Это правильно, когда дальнейшая работа бессмысленна или опасна: миграция, единая batch-операция, построение ответа из обязательных источников.

Иногда нужно обработать все независимые задачи и показать полный отчёт: например, nightly export десятков проектов. Тогда одна ошибка не должна отменять остальные. Используйте `WaitGroup` и канал результатов/ошибок или собирайте ошибки под mutex, затем объединяйте через `errors.Join`.

```
func runAll(ctx context.Context, taskIDs []TaskID, run func(context.Context, TaskID)
error) error {
  errs := make(chan error, len(taskIDs))
  var workers sync.WaitGroup

  for _, taskID := range taskIDs {
    id := taskID
    workers.Add(1)
    go func() {
      defer workers.Done()
      if err := run(ctx, id); err != nil {
        errs <- fmt.Errorf("обработать задачу %d: %w", id, err)
      }
    }()
  }

  workers.Wait()
  close(errs)
```

```
  var allErrors []error for err := range errs {    allErrors = append(allErrors, err)
} return errors.Join(allErrors...)}
```

Этот вариант подходит только для ограниченного списка: он запускает goroutine на каждую задачу и буферизует все ошибки. Для больших объёмов комбинируйте worker pool с result collector. Всегда выбирайте и документируйте семантику: **fail-fast** или **best-effort with report**.

Graceful shutdown worker group

Graceful shutdown — это не «сразу выйти из `main`». Нужно перестать принимать новую работу, сообщить активным goroutine об остановке, дождаться их в ограниченное время и лишь затем закрыть зависимости.

Ниже приведён skeleton фонового processor. Он владеет `jobs`, поэтому только он может закрыть этот канал:

```
type Processor struct {
  jobs   chan TaskID
  cancel context.CancelFunc
  done   chan struct{}
}

func StartProcessor(parent context.Context, workerCount int, run
func(context.Context, TaskID) error) *Processor {
  ctx, cancel := context.WithCancel(parent)
  processor := &Processor{
    jobs:   make(chan TaskID, workerCount),
    cancel: cancel,
    done:   make(chan struct{}),
  }

  var workers sync.WaitGroup
  workers.Add(workerCount)
  for range workerCount {
    go func() {
      defer workers.Done()
      for {
        select {
        case <-ctx.Done():
          return
        case id, open := <-processor.jobs:
          if !open {
            return
          }
          _ = run(ctx, id) // Реальная политика ошибок определяется вызывающим
```

```
кодом.
        }     }   }() } go func() {    workers.Wait()   close(processor.done) }()
return processor}func (processor *Processor) Submit(ctx context.Context, id TaskID)
error {  select { case <-ctx.Done():    return ctx.Err()  case processor.jobs <- id:
return nil }}func (processor *Processor) Shutdown(ctx context.Context) error {
close(processor.jobs) // Больше не принимаем новые задания. processor.cancel()    //
Просим активные операции остановиться.  select { case <-processor.done:    return nil
case <-ctx.Done():    return ctx.Err()  }}
```

В production этот skeleton требует дополнительного контракта: `Submit` и `Shutdown` нельзя вызывать конкурентно без защиты состояния, иначе `Submit` может отправить в уже закрытый канал и вызвать panic. Возможные решения: отдельный mutex/флаг приёма, единственная goroutine-владелец очереди или архитектура, где caller гарантирует порядок. Этот пример специально показывает, что закрытие канала — часть API, а не локальная мелочь.

Также определите продуктовую политику для уже принятых jobs:

- **drain:** перестать принимать новые, закрыть `jobs`, дать worker обработать уже принятые задания до deadline;
- **cancel immediately:** отменить context, прервать всё, что поддерживает отмену;
- **durable handoff:** сохранить необработанные задания в БД/очереди для следующего процесса.

Нельзя обещать drain и сразу вызывать `cancel`: worker может оставить уже принятые задания. Для настоящего drain сначала закройте вход, дождитесь пустой очереди и worker, а отмену применяйте только после истечения внешнего deadline как аварийный fallback.

Shutdown HTTP-сервера и фоновой работы

Последовательность Task Manager API при `SIGTERM` обычно такая:

1. перестать принимать новые HTTP-запросы через `server.Shutdown`;
2. перестать принимать новые фоновые jobs;
3. дать активным handler и worker завершиться в `shutdownCtx`;
4. после ожидания закрыть БД, HTTP client transports и остальные зависимости;
5. если deadline истёк, записать ошибку и завершить процесс по политике платформы.

Сначала закрывать `db` или отменять root context, пока handler ещё обслуживают запросы, опасно: это создаёт лишние `500` и теряет управляемость. Один root context и единый ограниченный deadline помогают всем подсистемам завершаться согласованно.

Тестирование отмены и shutdown

Не проверяйте остановку через `time.Sleep`. Worker должен иметь тестируемую контрольную точку:

```
func TestWorkerStopsAfterCancel(t *testing.T) {
  ctx, cancel := context.WithCancel(context.Background())
  started := make(chan struct{})
  stopped := make(chan struct{})

  go func() {
    defer close(stopped)
    close(started)
    <-ctx.Done()
  }()

  <-started
  cancel()

  select {
  case <-stopped:
  case <-time.After(time.Second):
    t.Fatal("worker не завершился после отмены")
  }
}
```

Здесь timeout — не синхронизация успешного пути, а защитное ограничение теста при баге. События `started` и `stopped` создают детерминированный порядок. В следующей странице разберём, какие блокировки чаще всего оставляют goroutine навсегда жить.

Утечки goroutine и диагностика

Goroutine завершается только когда возвращается её функция. Если она навечно ждёт канал, lock, I/O без дедлайна или зависшую зависимость, runtime продолжает хранить её стек и связанные объекты. Одна утечка может быть незаметна; утечка на каждый HTTP-запрос постепенно исчерпает память, соединения и лимиты процесса.

Частые причины утечек

Отправитель ждёт получателя

```
func wrong() {
  results := make(chan string)
  go func() {
    results <- "готово" // Навсегда ждёт, если wrong вернулась.
  }()
```

```
  return}
```

Не «лечите» это случайным буфером: он только сдвинет проблему, если отправок станет больше буфера. Выберите один контракт:

- caller гарантированно читает все результаты;
- канал имеет доказанно достаточный буфер для фиксированного числа отправок;
- sender выбирает между отправкой и `<-ctx.Done()`.

```
select {
case results <- value:
case <-ctx.Done():
  return
}
```

Получатель ждёт канал, который никто не закроет

```
for result := range results {
  process(result)
}
```

`range` завершится только после `close(results)`. Если producer завершился без закрытия, consumer утечёт. При нескольких producer сделайте одного координатора: `WaitGroup` → `close(results)`.

Заблокированный `select` без отмены

```
result := <-results // Может ждать бесконечно.
```

Если операция имеет жизненный цикл запроса или shutdown, добавьте context:

```
select {
case result := <-results:
  return result
case <-ctx.Done():
  return Result{}, ctx.Err()
}
```

Сеть без context и timeout

HTTP-клиент без timeout/контекста может ждать сервер, DNS или тело ответа непредсказуемо долго. Передавайте `context` через `http.NewRequestWithContext`, настраивайте ограничения

`http.Client` по контракту и всегда закрывайте `response.Body` после успешного `Do`.

Неосвобождённый lock или token

Забытый `Unlock`, `wg.Done` или `<-semaphore` может заблокировать не одну goroutine, а всех следующих участников. Регистрируйте cleanup сразу после успешного захвата:

```
semaphore <- struct{}{}
defer func() { <-semaphore }()

mutex.Lock()
defer mutex.Unlock()
```

Consumer завершился раньше producer

Pipeline часто ломается так: sink увидел первую ошибку и вернулся, а upstream продолжает посылать значения в небуферизированный канал. Для этого и нужен общий `cancel`, который все sender проверяют при send.

Leak или просто ожидающая goroutine?

Не вся долго живущая goroutine — утечка. HTTP server, worker, scheduler или metrics collector могут жить всё время процесса, если у них есть владелец и чёткий путь shutdown. Утечка — goroutine, которая больше не выполняет полезную работу и не имеет гарантированного события завершения.

Полезная документация для компонента отвечает:

```
Start создаёт N worker.
Они завершаются, когда закрыт jobs или отменён ctx.
Stop/Shutdown ждёт их через done.
Владелец jobs — Processor; внешний код канал не закрывает.
```

Наблюдаемость: счётчик goroutine

Быстрый сигнал — число goroutine в процессе:

```
logger.Debug("runtime состояние", "goroutines", runtime.NumGoroutine())
```

Само число не доказывает утечку: оно меняется из-за HTTP transport, тестов, runtime и нормальных фоновых задач. Но устойчивый рост после одинаковой нагрузки — повод исследовать проблему. Не делайте `NumGoroutine` строгим production-SLO без базовой линии и контекста среды.

Профиль goroutine

`runtime/pprof` показывает, где goroutine остановлены. В тесте можно записать профиль:

```
func TestWriteGoroutineProfile(t *testing.T) {
  file, err := os.Create("goroutine.out")
  if err != nil {
    t.Fatal(err)
  }
  defer file.Close()

  if err := pprof.Lookup("goroutine").WriteTo(file, 2); err != nil {
    t.Fatal(err)
  }
}
```

Для работающего debug-сервера возможен endpoint `net/http/pprof`, но его нельзя открывать публично: профиль раскрывает пути, функции, внутреннюю структуру и иногда данные аргументов. Открывайте `pprof` только в защищённой внутренней сети или временно при расследовании.

При анализе ищите множество одинаковых stack trace, например горутины, остановленные на:

```
chan send
your/module/internal/export.sendResult(...)
```

Это указывает не на «медленную Go runtime», а на протокол: кто-то отправляет результат, который никто не читает или не отменяет.

Тесты жизненного цикла

Лучший способ не получить утечку — тестировать договорённость о завершении, а не только функциональный результат.

Barrier вместо `Sleep`

Используйте каналы, чтобы тест сам управлял фазами worker:

```
func TestRunStopsWhenContextCanceled(t *testing.T) {
  ctx, cancel := context.WithCancel(context.Background())
  defer cancel()

  started := make(chan struct{})
  stopped := make(chan struct{})

  go func() {
```

```
    defer close(stopped)    close(started)    <-ctx.Done() }() <-started  cancel()
select { case <-stopped:   case <-time.After(time.Second):    t.Fatal("goroutine не
завершилась")  }}
```

Тест ждёт конкретное событие `stopped`; timeout нужен только для того, чтобы test suite не завис навсегда при регрессии.

Повтор и race detector

Конкурентные проблемы часто проявляются не в каждом запуске:

```
go test -race ./...
go test -count=100 ./internal/export
go test -run TestProcessorShutdown -count=100 ./internal/worker
```

Race detector находит несинхронизированный доступ к памяти, но не доказывает отсутствие утечек и deadlock. Повтор тестов повышает шанс поймать ошибку расписания; детерминированные контрольные точки всё равно важнее.

Диагностика зависшего теста

У Go есть встроенный timeout пакета:

```
go test -timeout 10s ./internal/worker
```

Когда timeout истечёт, Go напечатает stack trace активных goroutine. Читайте его как карту ожиданий:

- `chan receive` — кто должен отправить или закрыть канал?
- `chan send` — кто должен прочитать либо отменить sender?
- `sync.(*WaitGroup).Wait` — все ли пути вызывают `Done`?
- `sync.(*Mutex).Lock` — кто держит lock и почему не освобождает?
- network I/O — передан ли context, есть ли timeout, закрыто ли response body?

Начинайте с goroutine вашего пакета, затем восстановите граф: producer → channel → consumer → shutdown signal. Обычно корень — не строка, на которой тест завис, а отсутствующее событие в другой goroutine.

Минимальная политика для Task Manager API

- Каждый HTTP handler использует `c.Request.Context()` для downstream SQL и HTTP-вызовов.
- У каждой фоновой группы есть `Start`/`Shutdown` или понятный владелец `errgroup.Wait`.
- У потоков результатов известен единственный код, который закрывает канал.
- У внешних операций есть timeout и cleanup (`cancel`, `Close`, `Rollback`).

- CI запускает `go test -race ./...`; для конкурентных пакетов есть тесты отмены и shutdown.
- При росте числа goroutine расследуются профили и протоколы каналов, а не добавляются бесконечные буферы.

Следующие страницы закрепят паттерны вопросами и заданиями. Главное правило всей главы: каждая созданная goroutine должна иметь проверяемый путь к завершению.

Самопроверка: конкурентные паттерны

1. Какое утверждение лучше всего описывает worker pool?

A. Это канал, который всегда имеет бесконечный буфер

B. Новая goroutine создаётся без лимита для каждого HTTP-запроса

C. Один mutex заменяет все каналы и context

D. Фиксированное число worker читает задания из общей очереди

2. Кто обычно закрывает канал jobs в worker pool?

A. Каждый worker после одного задания

B. Первый consumer, который получил задание

C. Producer, который знает, что новых заданий больше не будет

D. Любая goroutine после context cancellation

3. Что означает backpressure в канальном pipeline?

A. Worker всегда становится быстрее после увеличения буфера

B. Отмена context происходит при каждом send

C. Канал автоматически отбрасывает все старые задачи

D. Быстрый producer вынужден ждать, когда очередь заполнена или consumer не готов

4. Почему большой буфер не является универсальным решением зависания?

A. Буфер запрещает использовать context

B. Go runtime всегда игнорирует буфер больше одного элемента

C. Он лишь откладывает блокировку, увеличивает память и может скрыть ошибку протокола

D. Буферизированные каналы нельзя читать через range

5. Как semaphore на buffered channel ограничивает число активных операций?

A. Канал автоматически отменяет лишние goroutine через panic

B. Он гарантирует порядок выполнения worker

C. Он сортирует задания по приоритету

D. Каждая операция занимает token перед стартом и возвращает его при завершении

6. Чем fan-out отличается от fan-in?

A. Fan-out распределяет поток между worker, fan-in объединяет несколько потоков

B. Fan-in запрещает несколько producer

C. Это два названия одного и того же паттерна

D. Fan-out работает только с map, fan-in — только со слайсами

7. Почему sender в отменяемом pipeline отправляет значение через select с ctx.Done()?

A. Чтобы закрыть ctx.Done() вручную

B. Чтобы заменить WaitGroup

C. Чтобы отправить одно значение сразу во все каналы

D. Чтобы не остаться навечно на send, если downstream перестал читать

8. Что делает errgroup.WithContext при первой ненулевой ошибке функции группы?

A. Принудительно уничтожает все goroutine в процессе

B. Возвращает все ошибки через errors.Join

C. Автоматически закрывает все созданные каналы

D. Отменяет производный context; остальные функции должны сами заметить отмену

9. Когда errgroup с первой ошибкой хуже, чем сбор всех ошибок?

A. Когда две обязательные HTTP-операции можно запустить параллельно

B. Когда нужно отменить обязательную batch-операцию при первой ошибке

C. Когда не нужно ждать завершения ни одной goroutine

D. Когда независимые задачи нужно выполнить best-effort и выдать полный отчёт

10. Что означает graceful shutdown worker group в общем случае?

A. Немедленно вызвать os.Exit из первого worker

B. Остановить приём новой работы, сообщить об остановке, дождаться worker в пределах deadline

C. Закрыть все каналы из любого consumer

D. Сразу закрыть базу данных, пока handler ещё работают

11. Какой сценарий является типичной утечкой goroutine?

A. HTTP handler передал Request.Context в QueryContext

B. Worker завершился после закрытия jobs

C. Координатор закрыл output после WaitGroup.Wait

D. Goroutine навечно ждёт send в канал, который caller больше не читает

12. Для чего в тесте shutdown полезны каналы started и stopped?

A. Они автоматически находят все data race

B. Они заменяют context во всём приложении

C. Они задают детерминированные события вместо ожидания произвольного Sleep

D. Они позволяют не вызывать cancel

Ответы к самопроверке

1. Фиксированное число worker читает задания из общей очереди

2. Producer, который знает, что новых заданий больше не будет

3. Быстрый producer вынужден ждать, когда очередь заполнена или consumer не готов

4. Он лишь откладывает блокировку, увеличивает память и может скрыть ошибку протокола

5. Каждая операция занимает token перед стартом и возвращает его при завершении

6. Fan-out распределяет поток между worker, fan-in объединяет несколько потоков

7. Чтобы не остаться навечно на send, если downstream перестал читать

8. Отменяет производный context; остальные функции должны сами заметить отмену

9. Когда независимые задачи нужно выполнить best-effort и выдать полный отчёт

10. Остановить приём новой работы, сообщить об остановке, дождаться worker в пределах deadline

11. Goroutine навечно ждёт send в канал, который caller больше не читает

12. Они задают детерминированные события вместо ожидания произвольного Sleep

Задания: конкурентные паттерны

Практика рассчитана на отдельный каталог или модуль рядом с Task Manager API. Запускайте проверки несколько раз: конкурентный код должен проходить не только один удачный запуск.

```
go test -race ./...
go test -count=100 ./...
```

Не используйте `time.Sleep` как способ дождаться нормального завершения worker. В решениях timeout встречается только как аварийная защита теста от вечного зависания.

Задание 1. Worker pool с best-effort отчётом

Есть интерфейс:

```
type PriorityRefresher interface {
  Refresh(ctx context.Context, id TaskID) error
}
```

Напишите:

```
func RefreshAll(
  ctx context.Context,
  refresher PriorityRefresher,
  ids []TaskID,
  workerCount int,
) error
```

Функция запускает не более `workerCount` worker, передаёт им каждый ID через канал jobs, ждёт завершения и возвращает объединение всех ошибок через `errors.Join`. Ошибка одной задачи не должна останавливать остальные.

Критерии проверки:

- `workerCount < 1` возвращает ошибку;
- producer закрывает `jobs` после отправки всех ID;
- worker завершается по закрытию `jobs` или `ctx.Done()`;
- ошибки передаются в один result/error path, а не теряются;
- `WaitGroup` ждёт worker перед закрытием канала ошибок;
- при отмене отправка jobs не зависает;
- нет гонки данных при `go test -race`.

Подсказка

Пусть worker отправляет ошибку в буферизированный канал, а координатор закрывает его после `workers.Wait()`:

```
go func() {
  workers.Wait()
  close(errs)
}()
```

Решение

```
package priority

import (
  "context"
  "errors"
  "fmt"
  "sync"
)

type TaskID int64

type PriorityRefresher interface {
  Refresh(ctx context.Context, id TaskID) error
}

func RefreshAll(
  ctx context.Context,
  refresher PriorityRefresher,
  ids []TaskID,
  workerCount int,
) error {
  if workerCount < 1 {
    return errors.New("число worker должно быть больше нуля")
  }

  jobs := make(chan TaskID)
  errs := make(chan error, len(ids))
  var workers sync.WaitGroup

  worker := func() {
    defer workers.Done()
```

▼ Сведения

```
    for {      select {     case <-ctx.Done():         return     case id, open := <-
jobs:        if !open {          return       }        if err :=
refresher.Refresh(ctx, id); err != nil {          select {          case errs <-
fmt.Errorf("обновить задачу %d: %w", id, err):          case <-ctx.Done():
return         }        }      }    } } workers.Add(workerCount)   for range
workerCount {     go worker()   } for _, id := range ids {     select {   case jobs <-
id:    case <-ctx.Done():      close(jobs)       workers.Wait()      return ctx.Err()
} } close(jobs)   go func() {     workers.Wait()    close(errs)   }()  var allErrors
[]error   for err := range errs {    allErrors = append(allErrors, err)  } return
errors.Join(allErrors...)}
```

Здесь `errs` имеет буфер `len(ids)`, потому что каждая задача может дать максимум одну ошибку. Для неограниченного потока так делать нельзя: нужен одновременный collector или другая политика отчёта.

Задание 2. Fan-in с отменой

Напишите generic-функцию:

```
func Merge[T any](ctx context.Context, inputs ...<-chan T) <-chan T
```

Она должна пересылать значения из всех inputs в один output, завершаться при отмене и закрывать output после закрытия всех inputs. Входные каналы закрывать нельзя.

Критерии проверки:

- на каждый input запускается отдельный forwarder;
- forwarder слушает `ctx.Done()` и при receive, и при send;
- только координатор после `WaitGroup.Wait()` закрывает output;
- `Merge(ctx)` без inputs возвращает уже закрывающийся output;
- тест не предполагает порядок значений из разных inputs;
- отмена не оставляет forwarder заблокированным на output send.

Подсказка

```
select {
case value, open := <-input:
  // ...
case <-ctx.Done():
  return
}
```

Решение

```
func Merge[T any](ctx context.Context, inputs ...<-chan T) <-chan T {
  out := make(chan T)
  var forwarders sync.WaitGroup

  forward := func(input <-chan T) {
    defer forwarders.Done()
    for {
      select {
      case <-ctx.Done():
        return
      case value, open := <-input:
        if !open {
          return
        }
        select {
        case out <- value:
        case <-ctx.Done():
          return
        }
      }
    }
  }

  forwarders.Add(len(inputs))
  for _, input := range inputs {
    go forward(input)
  }

  go func() {
    forwarders.Wait()
    close(out)
  }()
```

▼ Сведения

```
    return out}
```

Задание 3. Fail-fast с `errgroup.SetLimit`

Используйте `golang.org/x/sync/errgroup` для функции:

```
func RefreshFailFast(
  ctx context.Context,
  refresher PriorityRefresher,
  ids []TaskID,
  limit int,
) error
```

При первой ошибке context группы должен отмениться, новые операции не должны добавляться после замеченной отмены, а функция возвращает ошибку `errgroup.Wait()`.

Критерии проверки:

- зависимость добавлена командой `go get`, а не ручным изменением `go.mod`;
- используется `errgroup.WithContext`;
- `SetLimit(limit)` вызывается до `Go`;
- каждый `Refresh` получает context группы, а не `context.Background()`;
- ID цикла фиксируется локальной переменной;
- `limit < 1` возвращает ошибку;
- не ожидается, что errgroup принудительно оборвёт игнорирующую context goroutine.

Подсказка

```
group, groupCtx := errgroup.WithContext(ctx)
group.SetLimit(limit)
```

Решение

```
func RefreshFailFast(
  ctx context.Context,
  refresher PriorityRefresher,
  ids []TaskID,
  limit int,
) error {
  if limit < 1 {
    return errors.New("limit должен быть больше нуля")
  }

  group, groupCtx := errgroup.WithContext(ctx)
  group.SetLimit(limit)

  for _, taskID := range ids {
    if err := groupCtx.Err(); err != nil {
      break
    }

    id := taskID
    group.Go(func() error {
      if err := refresher.Refresh(groupCtx, id); err != nil {
        return fmt.Errorf("обновить задачу %d: %w", id, err)
      }
      return nil
    })
  }

  return group.Wait()
}
```

Проверка `groupCtx.Err()` не исключает ситуацию, когда ошибка пришла, пока следующий `group.Go` уже ждал свободное место. Для строгого прекращения приёма jobs спроектируйте явный worker pool с producer, который выбирает между `jobs <- id` и `groupCtx.Done()`. Главное: первая ошибка должна быть частью явного контракта, а не случайной потерей результата worker.

Задание 4. Graceful drain processor

Спроектируйте `Processor` с методами:

```
func (processor *Processor) Submit(ctx context.Context, id TaskID) error
func (processor *Processor) Shutdown(ctx context.Context) error
```

Выберите и задокументируйте политику **drain**: после начала `Shutdown` новые `Submit` возвращают ошибку, ранее принятые jobs обрабатываются, затем `Shutdown` ждёт завершение worker до deadline из `ctx`.

Критерии проверки:

- `Submit` и `Shutdown` не могут привести к `send on closed channel` при конкурентном вызове;
- ровно один владелец закрывает jobs;
- контекст `Shutdown` ограничивает ожидание, но не используется как подмена рабочего context для немедленной отмены drain;
- job, успешно принятая `Submit` до shutdown, либо обработана, либо её судьба явно отражена в документированной политике;
- тест использует барьеры/каналы: подтверждает start job, инициирует shutdown и ждёт done;
- после `Shutdown` worker не остаются жить.

Подсказка

Для защиты приёма jobs можно использовать mutex, флаг `stopping` и `WaitGroup` активных `Submit`. Не держите mutex во время блокирующего вызова worker.

Направление решения

Это задание намеренно сложнее предыдущих: готовый «короткий» пример часто содержит гонку между `Submit` и `close(jobs)`. Возможная архитектура:

1. `Submit` под mutex проверяет `stopping`, увеличивает `submitters` и отпускает mutex.
2. Затем оно отправляет job через `select` с context caller.
3. После send или отмены уменьшает `submitters`.
4. `Shutdown` под mutex ставит `stopping = true`, ждёт `submitters`, затем один раз закрывает `jobs`.
5. Worker завершаются, когда дочитают закрытый jobs; `Shutdown` ждёт их через отдельный `done`-канал и `ctx.Done()`.

Нужно также решить, как отменённый caller влияет на job, уже переданную в канал: после успешного send работа уже принята processor и должна быть обработана согласно policy drain.

Задание 5. Тест отмены без `Sleep`

Напишите worker `Run(ctx context.Context, started chan<- struct{}, stopped chan<- struct{})`, который сообщает о запуске, ждёт отмену context и сообщает о завершении. Напишите тест, который:

1. ждёт `started`;
2. вызывает `cancel()`;
3. ждёт `stopped`;
4. имеет timeout только как защиту от вечного зависания.

Критерии проверки:

- успешный путь не содержит `time.Sleep`;
- `stopped` закрывается/отправляется через `defer`, поэтому работает при раннем выходе;
- test failure ясно сообщает о незавершившемся worker;
- `cancel` вызывается через `defer`, если тест завершится раньше;
- `go test -race` не сообщает о гонке.

Решение

```
func Run(ctx context.Context, started chan<- struct{}, stopped chan<- struct{}) {
  defer close(stopped)
  close(started)
  <-ctx.Done()
}

func TestRunStopsAfterCancel(t *testing.T) {
  ctx, cancel := context.WithCancel(context.Background())
  defer cancel()

  started := make(chan struct{})
  stopped := make(chan struct{})
  go Run(ctx, started, stopped)

  <-started
  cancel()

  select {
  case <-stopped:
  case <-time.After(time.Second):
    t.Fatal("worker не завершился после отмены")
  }
}
```

Проверьте себя

Перед следующей главой убедитесь, что вы можете:

- выбрать pool, pipeline, fan-out/fan-in или несколько `errgroup.Go` по форме задачи;
- назвать владельца каждого канала и его правило закрытия;
- ограничить и число активных операций, и число созданных goroutine;
- провести `context` до каждого потенциально блокирующего send, receive, SQL и HTTP-вызова;
- выбрать fail-fast или best-effort contract до написания worker;
- доказать тестом shutdown без ожидания «на глаз»;
- прочитать goroutine stack trace и найти отсутствующее событие в протоколе.

Рекомендации по коду: конкурентные паттерны

Жизненный цикл goroutine

- Создавайте goroutine только с явной причиной, владельцем и проверяемым путём завершения.
- Для каждой goroutine определите: кто запускает, кто ждёт, какой сигнал останавливает и что происходит с её ошибкой.
- Передавайте `context.Context` первым параметром в отменяемые операции; не создавайте `context.Background()` внутри worker вместо контекста операции.
- В каждом потенциально блокирующем send/receive pipeline добавляйте путь `<-ctx.Done()`, если downstream может закончить раньше.
- Не используйте `time.Sleep` для синхронизации. Каналы событий, `WaitGroup`, `errgroup.Wait` и `done` создают детерминированный контракт.

Каналы, pool и backpressure

- Закрывает канал только владелец всех отправок — после последней отправки. Consumer не закрывает входной канал.
- При нескольких sender используйте координатор: `WaitGroup.Wait()` → `close(out)` ровно один раз.
- Worker pool применяйте для ограниченного числа живых worker и долгого/неограниченного потока jobs.
- Semaphore на buffered channel подходит для ограничения активных операций, но не ограничивает число goroutine, ожидающих token.
- Буфер выбирайте как конечную очередь с объяснённым размером. Он сглаживает краткие всплески, но не исправляет постоянно медленный consumer.
- Не закрывайте channel-семафор: его жизненный цикл выражают отмена, прекращение создания работ и ожидание группы.
- Не полагайтесь на порядок результатов после fan-out; восстанавливайте его отдельным бизнес-этапом при необходимости.

Ошибки, отмена и shutdown

- Выберите семантику до реализации: fail-fast с `errgroup.WithContext` или best-effort с полным отчётом ошибок.
- `errgroup` отменяет context при первой ошибке, но не убивает goroutine: каждый downstream вызов обязан замечать отмену.
- Вызывайте `SetLimit` до запуска функций и не меняйте лимит, пока группа активна.
- Не запускайте новые `errgroup.Go` после начала `Wait`.
- При graceful shutdown сначала остановите приём новой работы, затем обработайте/отмените уже принятую по явной политике, дождитесь worker до deadline и только потом закройте зависимости.
- Не обещайте drain и не отменяйте рабочий context немедленно: это противоречивые семантики. Для незавершённых jobs в production определите durable handoff или retry policy.
- Не закрывайте БД и HTTP-клиент, пока активные handler/worker могут ими пользоваться.

Утечки и диагностика

- Считайте зависший send, receive, `Wait`, lock или I/O без deadline потенциальной утечкой, пока не доказан путь завершения.
- Закрывайте `response.Body`, вызывайте `cancel`, `Rollback`, `Unlock` и возврат semaphore token на всех путях через `defer` после успешного захвата ресурса.
- Запускайте `go test -race ./...` и повторяйте чувствительные тесты с `-count`; race detector не находит все deadlock и leaks.
- Применяйте `go test -timeout` для диагностики зависаний и читайте stack trace как граф producer, consumer и сигналов остановки.
- Исследуйте устойчивый рост `runtime.NumGoroutine()` через goroutine profile; не маскируйте проблему бесконечным буфером.
- Не открывайте `net/http/pprof` публично: профили содержат внутреннюю информацию приложения.

Итоговый чек-лист

- [ ] У каждой goroutine есть владелец, error path и условие завершения.
- [ ] Для каждого канала известны sender, consumer и единственный закрывающий код.
- [ ] Все блокирующие точки участвуют в отмене там, где операция может стать неактуальной.
- [ ] Число worker, активных операций и размер очереди имеют обоснованные лимиты.
- [ ] Семантика ошибок — fail-fast либо best-effort — отражена в API и тестах.
- [ ] Shutdown не допускает send в закрытый канал и не закрывает зависимости раньше worker.
- [ ] Конкурентные тесты проходят с `-race`, повтором и контролируемыми событиями без `Sleep`.

Продвинутый Go D: производительность и диагностика

Цель главы

Научиться измерять производительность Go-кода воспроизводимо: формулировать гипотезу, писать benchmark, сравнивать изменения через `benchstat`, снимать CPU и memory profiles, интерпретировать `pprof`, понимать аллокации и применять `strings.Builder` или `sync.Pool` только после подтверждённого измерениями выигрыша.

Эта глава не о том, как сделать любой код «максимально быстрым». Она о безопасном инженерном цикле:

```
симптом → метрика → воспроизводимый сценарий → профиль/benchmark
   ↓                                                    ↓
проверка после изменения ← минимальное понятное исправление
```

Для Task Manager API симптомом может быть медленный `GET /tasks`, высокий расход памяти при JSON-экспорте или рост CPU после появления фильтрации. До измерения нельзя уверенно утверждать, что проблема в строках, SQL, кодировании JSON, HTTP-клиенте, блокировках или инфраструктуре.

Карта главы

1. **Эта страница** — методика измерений, виды метрик и подготовка окружения.
2. Benchmarks и `benchstat` — `testing.B`, `go test -bench`, память и статистическое сравнение.
3. CPU, память и `pprof` — профили, команды `top` / `list` / `web`, profiling тестов и защищённый runtime endpoint.
4. Аллокации, escape analysis и оптимизации — `-gcflags=-m`, `strings.Builder`, `sync.Pool` и случаи, где оптимизация вредна.

Сначала измерение, потом оптимизация

Оптимизация без измерений обычно имеет один из трёх результатов:

- код становится сложнее, но не быстрее в реальном сценарии;
- улучшена редкая функция, а bottleneck остался в SQL, сети или JSON;
- microbenchmark показывает выигрыш, но production ухудшается из-за блокировок, памяти или более сложного протокола.

Вместо вопроса «как ускорить этот фрагмент?» начните с вопросов:

1. **Что именно плохо для пользователя или системы?** p95 latency, CPU, RSS, число аллокаций, стоимость инфраструктуры?
2. **Где это проявляется?** Один endpoint, CLI-экспорт, worker, тест, нагрузочный сценарий?
3. **Как воспроизвести сценарий?** Набор данных, настройки, запрос, версия Go, машина и команда.
4. **Какова гипотеза?** «Экспорт создаёт много промежуточных строк», а не «нужно добавить pool».
5. **Как будет подтверждён результат?** Те же benchmark/profile, функциональные тесты и, если нужно, нагрузочный тест.

> **Производительность — свойство сценария**
>
> `strings.Builder` может улучшить формирование большого текста, но ничего не изменит, если 95% времени занимает PostgreSQL. Увеличение числа worker может ускорить I/O до лимита внешнего API, а после него повысить ошибки и p99. Измеряйте реальный путь, а не только отдельную понравившуюся строку кода.

Что измеряет Go

`go test` и `pprof` показывают разные стороны проблемы:

Инструмент	Вопрос	Основные показатели
benchmark	Насколько быстро и с какими аллокациями работает конкретная операция?	`ns/op`, `B/op`, `allocs/op`
CPU profile	Где процесс тратит процессорное время?	flat/cumulative CPU time
heap profile	Какие места создают или удерживают память?	bytes/objects, allocated/in-use
block/mutex profile	Где goroutine ждут синхронизацию?	время блокировок/конфликтов
load test / метрики	Как система ведёт себя под реалистичной нагрузкой?	throughput, latency percentiles, errors, saturation

Ни один из них не заменяет остальные. Benchmark не показывает поведение базы и сети; CPU profile не доказывает, что endpoint удовлетворяет p99; heap profile не заменяет проверку отсутствия утечки.

Стабильное окружение

Микроизмерения шумны: на них влияют scheduler, фоновая нагрузка, thermal throttling, версия компилятора, частота CPU и соседние процессы. Сделайте сравнение честнее:

- измеряйте на одной машине и одной версии Go;
- не сравнивайте результаты ноутбука на батарее с CI на shared runner;
- закройте тяжёлые приложения и не запускайте параллельно большую сборку;
- фиксируйте входные данные и размер сценария;

- запускайте benchmark несколько раз (`-count`) и сравнивайте статистически;
- фиксируйте команду, commit и изменение конфигурации рядом с результатом.

Абсолютное число `ns/op` редко переносимо между машинами. Важнее сравнение до/после в сопоставимых условиях и понимание, что именно оно измеряет.

Сначала проверьте корректность

Оптимизация не оправдывает изменение контракта. Перед benchmark и после изменения запускайте обычные проверки:

```
go test ./...
go test -race ./...
go vet ./...
```

Для HTTP-изменений сохраните handler-тесты: быстрый JSON-ответ с другой семантикой фильтра, пропущенной ошибкой или гонкой данных не является улучшением.

Пример гипотезы: экспорт задач

Допустим, JSON-экспорт 100 000 задач стал медленным. Не начинайте с переписывания всего на ручную сериализацию. Сначала зафиксируйте:

```
Сценарий: сериализация 100 000 Task в JSON без файлового I/O.
Метрика: ns/op, B/op, allocs/op и CPU profile.
Гипотеза: значительную долю CPU или памяти создаёт конкретная подготовка данных,
а не json.MarshalIndent сам по себе.
Критерий: после изменения результат идентичен, benchmark стабильно улучшен,
а profile показывает уменьшение именно предполагаемого узкого места.
```

Отделите сериализацию от записи файла и сети. Если измерять всё вместе, результат может отражать filesystem cache, диск или docker volume вместо кода, который вы хотите изучить.

Измеряемая единица работы

Хороший benchmark имеет одну понятную операцию:

- построить текст экспорта из заранее созданных задач;
- отфильтровать слайс из N задач;
- кодировать один DTO в JSON;
- распарсить один запрос;
- выполнить один in-memory метод.

Плохой benchmark смешивает инициализацию, случайные данные, вывод в stdout, sleep, HTTP-сервер и проверку результата внутри основного цикла. Он измеряет случайный набор действий и

хуже объясняет причину числа.

Следующая страница покажет, как оформить измеряемую единицу как benchmark и не обмануть себя setup-кодом или оптимизациями компилятора.

Benchmarks и `benchstat`

Функция benchmark

Benchmark — функция в файле `*_test.go`, имя которой начинается с `Benchmark` и принимает `*testing.B`:

```
func BenchmarkBuildExport(b *testing.B) {
  // Подготовка данных.
  for b.Loop() {
    // Одна измеряемая операция.
  }
}
```

`b.Loop()` доступен начиная с Go 1.24 и сам управляет числом итераций. Он сообщает компилятору, что тело benchmark не должно быть «оптимизировано прочь». В существующем коде также часто встречается стиль для Go 1.23 и ниже:

```
for i := 0; i < b.N; i++ {
  // Одна измеряемая операция.
}
```

Используйте один стиль последовательно в проекте. Если поддерживаемая версия Go не имеет `b.Loop`, применяйте `b.N` и сохраняйте результат в package-level sink, чтобы компилятор не удалил вычисление.

Первый benchmark: текстовый экспорт

Сравним наивную конкатенацию и `strings.Builder`. Модель и входные данные создаются до измеряемого цикла:

```
type Task struct {
  ID       int64
  Title    string
  Priority int
}

func buildExportConcat(tasks []Task) string {
```

```
  result := "" for _, task := range tasks {   result += fmt.Sprintf("%d\t%s\t%d\n",
task.ID, task.Title, task.Priority)   } return result}func buildExportBuilder(tasks
[]Task) string {  var builder strings.Builder  for _, task := range tasks {
fmt.Fprintf(&builder, "%d\t%s\t%d\n", task.ID, task.Title, task.Priority)   } return
builder.String()}
```

```
var exportResult string

func BenchmarkBuildExportConcat(b *testing.B) {
  tasks := makeBenchmarkTasks(1_000)
  b.ReportAllocs()
  b.ResetTimer()

  for b.Loop() {
    exportResult = buildExportConcat(tasks)
  }
}

func BenchmarkBuildExportBuilder(b *testing.B) {
  tasks := makeBenchmarkTasks(1_000)
  b.ReportAllocs()
  b.ResetTimer()

  for b.Loop() {
    exportResult = buildExportBuilder(tasks)
  }
}
```

`makeBenchmarkTasks` не должен быть частью измерения, если гипотеза касается именно экспорта. `b.ResetTimer()` отделяет setup от основной работы. `exportResult` сохраняет возвращённую строку в package-level переменную: результат observable, и компилятор не может убрать вызов как бесполезный.

`fmt.Fprintf` сам может стать bottleneck. Это нормально для первого честного измерения: profile покажет, стоит ли менять именно форматирование. Не подменяйте его заранее ручными `strconv.AppendInt`, пока benchmark/profile не подтвердят необходимость.

Запуск benchmark

Базовая команда:

```
go test -run '^$' -bench '^BenchmarkBuildExport' -benchmem ./internal/export
```

- `-run '^$'` отключает обычные тесты, чтобы не смешивать их время с benchmark;
- `-bench` выбирает benchmark регулярным выражением;
- `-benchmem` добавляет `B/op` и `allocs/op`;
- путь ограничивает пакет, чтобы запуск был быстрым и точным.

Для всех benchmark в модуле:

```
go test -run '^$' -bench . -benchmem ./...
```

Пример формата:

```
BenchmarkBuildExportConcat-10       1200   1012000 ns/op   2100000 B/op   4000
allocs/op
BenchmarkBuildExportBuilder-10     12000    100000 ns/op    180000 B/op   1750
allocs/op
```

Суффикс `-10` — значение `GOMAXPROCS`, а не число ваших benchmark. Цифры — пример, они зависят от машины, Go и входных данных.

Как читать метрики

Метрика	Значение	Что не доказывает
`ns/op`	среднее время одной операции	p95/p99 latency production-запроса
`B/op`	сколько байт выделяется на операцию	сколько памяти остаётся жить после GC
`allocs/op`	число выделений на операцию	что именно нужно оптимизировать

Меньшее число не всегда лучше. Вариант может стать немного медленнее, но сильно проще, безопаснее или экономнее по памяти. Сначала определите ограничение: CPU, latency, allocation rate или retained heap.

Размер входа и sub-benchmarks

Алгоритм может вести себя по-разному на 10, 1 000 и 100 000 задачах. Используйте sub-benchmarks с понятными именами:

```
func BenchmarkBuildExport(b *testing.B) {
  for _, size := range []int{10, 1_000, 100_000} {
    b.Run(fmt.Sprintf("tasks=%d", size), func(b *testing.B) {
      tasks := makeBenchmarkTasks(size)
      b.ReportAllocs()
      b.ResetTimer()

      for b.Loop() {
```

```
        exportResult = buildExportBuilder(tasks)      }   })  }}
```

Не генерируйте random input внутри `b.Loop`, если вы не измеряете генератор. При необходимости seed random generator фиксированным значением и подготовьте данные заранее.

Варианты benchmark

Если у реализации несколько стратегий, оформляйте их отдельными именованными sub-benchmarks:

```
func BenchmarkFilterTasks(b *testing.B) {
  tasks := makeBenchmarkTasks(10_000)

  b.Run("append", func(b *testing.B) {
    for b.Loop() {
      result := filterWithAppend(tasks)
      benchmarkTasks = result
    }
  })

  b.Run("preallocated", func(b *testing.B) {
    for b.Loop() {
      result := filterPreallocated(tasks)
      benchmarkTasks = result
    }
  })
}

var benchmarkTasks []Task
```

Не меняйте входной слайс между итерациями, если следующая итерация предполагает исходные данные. Если операция намеренно изменяет вход, подготовьте или скопируйте его вне таймера и явно документируйте, что входит в измерение.

Сравнение до и после через `benchstat`

Один запуск benchmark может случайно оказаться быстрее из-за шума. `benchstat` сравнивает серии запусков статистически.

Установите инструмент:

```
go install golang.org/x/perf/cmd/benchstat@latest
```

Снимите baseline до изменения и результат после изменения в одинаковом окружении:

```
go test -run '^$' -bench '^BenchmarkBuildExport' -benchmem -count=10
./internal/export > old.txt
# Внесите одно изменение и убедитесь, что тесты проходят.
go test -run '^$' -bench '^BenchmarkBuildExport' -benchmem -count=10
./internal/export > new.txt
benchstat old.txt new.txt
```

Пример интерпретации:

```
name                          old time/op  new time/op  delta
BuildExportBuilder/tasks=1k   101µs ± 3%    91µs ± 2%  -9.9%  (p=0.000 n=10+10)

name                          old alloc/op new alloc/op delta
BuildExportBuilder/tasks=1k    180kB ± 0%    120kB ± 0%  -33.3%
```

`p` и разброс помогают отличить устойчивое изменение от шума, но не отвечают на продуктовый вопрос «достаточно ли это важно». Не принимайте изменение только по красивому проценту: проверьте профиль, корректность и сложность кода.

Типичные ошибки benchmark

Setup внутри измерения

```
for b.Loop() {
  tasks := makeBenchmarkTasks(10_000)
  buildExportBuilder(tasks)
}
```

Этот benchmark измеряет и генерацию задач. Подготовьте `tasks` до цикла, если не исследуете весь сценарий целиком.

Неиспользованный результат

```
for b.Loop() {
  buildExportBuilder(tasks)
}
```

Компилятор может удалить вычисление или оптимизировать его неожиданным образом. Присвойте результат sink-переменной либо используйте его так, чтобы результат был observable.

Один запуск и громкий вывод

Один результат не доказывает регрессию или ускорение. Не пишите в stdout внутри benchmark: I/O доминирует над измеряемой операцией и создаёт шум.

Benchmark вместо интеграционного измерения

Быстрая функция `filterTasks` не означает быстрый endpoint. После оптимизации hot helper проверьте endpoint нагрузочным тестом и метриками, если исходная проблема была на уровне сервиса.

Следующая страница углубит исследование: когда одного числа `ns/op` мало, CPU и heap profiles покажут, какие именно функции потребляют ресурсы.

CPU, память и `pprof`

Benchmark говорит, **насколько** изменилась одна операция. `pprof` помогает понять, **где** расходуются CPU и память. Профиль — статистическая выборка или снимок, а не абсолютная трассировка каждого вызова, поэтому интерпретируйте его вместе со сценарием и benchmark.

CPU profile из benchmark

Для CPU profile лучше выбрать один репрезентативный benchmark и достаточную длительность:

```
go test -run '^$' -bench '^BenchmarkBuildExportBuilder$' \
  -benchtime=3s -cpuprofile=cpu.out ./internal/export

go tool pprof cpu.out
```

`-benchtime=3s` даёт profiler больше выборок, чем очень короткий запуск. Не сравнивайте profile, снятые с разных входных данных или разных benchmark: сначала зафиксируйте сценарий.

В интерактивном режиме начните с:

```
(pprof) top
(pprof) top -cum
(pprof) list buildExportBuilder
```

- `top` показывает flat time: время непосредственно в функции;
- `top -cum` показывает cumulative time: функция плюс вызванный ею код;
- `list name` показывает строки исходника с затратами рядом.

Если большая доля находится в `runtime`, это не автоматически дефект runtime. Часто причина выше по стеку: много аллокаций создают GC work, блокировки дают scheduler overhead, а форматирование вызывает reflection/конверсии. Поднимайтесь по callers и проверяйте гипотезу.

Heap profile: выделено или удерживается?

Память можно смотреть с разными вопросами:

- **alloc space / alloc objects** — сколько всего выделила операция за период; полезно для allocation rate;
- **inuse space / inuse objects** — что остаётся живым в heap на момент профиля; полезно для роста памяти и утечек.

Снять heap profile из теста/benchmark:

```
go test -run '^$' -bench '^BenchmarkBuildExportBuilder$' \
  -benchtime=3s -memprofile=mem.out ./internal/export

go tool pprof -alloc_space mem.out
go tool pprof -inuse_space mem.out
```

Внутри `pprof` можно сменить sample type:

```
(pprof) sample_index=alloc_space
(pprof) top
(pprof) list buildExportBuilder
```

Высокий `alloc_space` не равен утечке: короткоживущие объекты могут активно выделяться и освобождаться. Высокий и растущий `inuse_space` после GC — повод искать удерживающие ссылки, кэш без ограничения, глобальную map или goroutine, которая держит данные.

Сравнение профилей

Снимите профиль до и после изменения в одинаковом сценарии. Для сравнения используйте `-base`:

```
go tool pprof -base old.mem new.mem
```

Положительные значения показывают рост относительно baseline, отрицательные — уменьшение. Это полезнее, чем смотреть два `top` глазами, но не отменяет проверку того, что оба файла получены одинаковыми командами и на сходном объёме работы.

Для CPU profiles также можно передать `-base`, однако различия профилей чувствительны к шуму. Сопроводите изменение benchmark-серией и `benchstat`.

Веб-интерфейс и граф вызовов

Из интерактивного `pprof`:

```
(pprof) web
```

Команда строит граф вызовов и обычно требует Graphviz (`dot`). Альтернатива — локальный web UI:

```
go tool pprof -http=:0 cpu.out
```

Инструмент выберет свободный порт и напечатает URL. В UI можно увидеть graph, flame graph, source и top. Профиль может содержать внутренние имена пакетов и пути, поэтому не публикуйте его в открытом issue или репозитории без проверки.

Runtime endpoint `net/http/pprof`

Для работающего процесса пакет `net/http/pprof` регистрирует debug endpoints в `DefaultServeMux` через blank import:

```
import _ "net/http/pprof"
```

В приложении с отдельным debug-сервером лучше не использовать основной public router. Ограничьте доступ loopback-адресом или внутренней сетью:

```
func startDebugServer() *http.Server {
  server := &http.Server{
    Addr:              "127.0.0.1:6060",
    Handler:           http.DefaultServeMux,
    ReadHeaderTimeout: 5 * time.Second,
  }

  go func() {
    if err := server.ListenAndServe(); err != nil && !errors.Is(err,
http.ErrServerClosed) {
      slog.Error("debug-сервер завершился", "err", err)
    }
  }()

  return server
}
```

Снять 30-секундный CPU profile:

```
go tool pprof http://127.0.0.1:6060/debug/pprof/profile?seconds=30
```

Посмотреть heap:

```
go tool pprof http://127.0.0.1:6060/debug/pprof/heap
```

Другие полезные endpoints: `/debug/pprof/goroutine`, `/debug/pprof/block`, `/debug/pprof/mutex`. Block/mutex profiles нужно включать и интерпретировать осторожно; они добавляют overhead и полезны только при гипотезе о contention.

> **Не открывайте pprof в публичный интернет**
>
> Pprof раскрывает структуру программы, package paths, stack traces и потенциально данные в аргументах. Не подключайте `net/http/pprof` к публичному Gin router, не публикуйте порт Docker без ограничения и не полагайтесь только на «сложный URL». Доступ должен ограничиваться сетью и политикой инфраструктуры.

CPU профиль реального HTTP endpoint

Для HTTP API есть два разных шага:

1. нагрузчик создаёт воспроизводимый поток запросов с тем же payload, auth и concurrency, что интересует вас;
2. pprof снимается во время стабильной нагрузки.

Не снимайте CPU profile с простаивающего API: он покажет idle runtime, а не стоимость endpoint. Также не делайте вывод по одному «самому медленному» запросу: warm-up, connection establishment и cache miss могут быть отдельными явлениями.

Полезные вопросы к profile

Вместо «какую строку оптимизировать?» спрашивайте:

- какая функция имеет большой cumulative cost в пути endpoint?
- почему она вызывается столько раз?
- это CPU, allocations/GC, блокировка или I/O wait?
- есть ли более высокий уровень, где можно сократить количество работы?
- изменится ли публичный контракт, порядок, memory ownership или error handling?

Например, profile может показать много времени в сортировке. Не обязательно менять алгоритм: возможно, endpoint сортирует 50 000 задач, хотя пользователю нужна пагинация и фильтр на уровне SQL. Производительная архитектура часто сокращает работу, а не ускоряет каждую локальную операцию.

Следующая страница разберёт аллокации и несколько часто предлагаемых оптимизаций — с условиями, при которых они действительно оправданы.

Аллокации, escape analysis и точечные оптимизации

Что такое allocation rate

Каждый объект, который должен пережить текущий stack frame или не может быть размещён на стеке безопасно, может оказаться в heap. Heap-объекты позже обрабатывает garbage collector. Небольшое число аллокаций обычно не проблема; высокий allocation rate на горячем пути может повысить CPU на GC, memory pressure и latency.

Начинайте с `-benchmem` и heap profile, а не со стремления получить `0 allocs/op`. Иногда одна понятная аллокация возвращаемой строки необходима и дешевле сложного ручного буфера.

Escape analysis: подсказка компилятора

Компилятор Go решает размещение объектов на стеке или heap. Его решение можно посмотреть так:

```
go test -gcflags='-m=2' ./internal/export
```

Или для package build:

```
go build -gcflags='-m=2' ./cmd/server
```

Фрагменты вывода могут выглядеть так:

```
internal/export/export.go:24:6: can inline buildExport
internal/export/export.go:31:12: moved to heap: builder
internal/export/export.go:35:18: task.Title escapes to heap
```

Это диагностика реализации текущего компилятора, а не публичный контракт языка. После небольшой перестройки кода, обновления Go или добавления вызова вывод может измениться. Не переписывайте программу только ради исчезновения слова `escapes`: сначала проверьте, отражается ли это на `allocs/op`, CPU или retained memory в нужном сценарии.

Частые причины escape

Объект может уйти в heap, когда:

- возвращается указатель на него;
- его адрес сохраняется в более долгоживущем объекте;

- значение передаётся в интерфейс или замыкание в ситуации, где компилятор не может доказать безопасность стека;
- размер объекта слишком велик для стека или его время жизни сложно определить.

Это не список ошибок. Возврат `*Task` может быть правильным API, а замыкание — самым понятным способом выразить callback. Escape analysis помогает объяснить наблюдаемый allocation rate, а не задаёт стиль кода самостоятельно.

`strings.Builder`: когда текст строится в цикле

Конкатенация строк в цикле создаёт новую строку при каждом `+` и часто даёт квадратичную работу по объёму копирования:

```
func buildTitlesSlow(tasks []Task) string {
  result := ""
  for _, task := range tasks {
    result += task.Title + "\n"
  }
  return result
}
```

Для большого последовательного текста используйте `strings.Builder`:

```
func buildTitles(tasks []Task) string {
  var builder strings.Builder
  for _, task := range tasks {
    builder.WriteString(task.Title)
    builder.WriteByte('\n')
  }
  return builder.String()
}
```

Если примерный размер известен, `Grow` может уменьшить перевыделения:

```
func buildTitles(tasks []Task) string {
  var builder strings.Builder
  builder.Grow(len(tasks) * 32) // Эвристика, а не обещание точного размера.

  for _, task := range tasks {
    builder.WriteString(task.Title)
    builder.WriteByte('\n')
  }
```

```
  return builder.String()}
```

Не вызывайте `Grow` с неограниченной длиной из HTTP body или внешнего поля: это превращает оценку размера в риск большой аллокации. Не копируйте `strings.Builder` после первого использования. Для короткой фиксированной конкатенации `title + ": " + status` часто проще и достаточно эффективна — подтвердите benchmark, если путь горячий.

Избегайте лишних преобразований, но не ломайте API

Иногда profile покажет лишние преобразования `[]byte` ↔ `string`, `fmt.Sprintf` в цикле или повторную сериализацию одинакового значения. Возможное улучшение:

```
// Вместо fmt.Sprintf для каждого ID на горячем пути:
buffer = strconv.AppendInt(buffer, id, 10)
```

Но `strconv.AppendInt` требует управления буфером и может ухудшить читаемость. Он оправдан, если profile и benchmark показывают заметную стоимость форматирования в критичном сценарии. Не меняйте публичные `string`-контракты на mutable `[]byte` без ясной необходимости и правил ownership.

`sync.Pool`: временный cache, а не ресурсный pool

`sync.Pool` хранит временные объекты, которые можно повторно использовать между goroutine. Runtime имеет право очистить pool при garbage collection; `Get` может вернуть `nil`, даже если вы раньше вызвали `Put`. Поэтому `sync.Pool` не подходит для обязательных ресурсов, connection pool, кэша данных, подсчёта объектов или передачи владения.

Пример для временного `bytes.Buffer` в доказанно горячем пути:

```
var bufferPool = sync.Pool{
  New: func() any {
    return new(bytes.Buffer)
  },
}

func encodeTask(task Task) []byte {
  buffer := bufferPool.Get().(*bytes.Buffer)
  buffer.Reset()
  defer func() {
    if buffer.Cap() <= maxPooledBufferCapacity {
      bufferPool.Put(buffer)
    }
  }()
```

```
  buffer.WriteString(task.Title) buffer.WriteByte('\n')  // Bytes ссылается на
память buffer. Возвращаем независимую копию,  // иначе caller увидит изменение после
Put/reuse. return bytes.Clone(buffer.Bytes())}
```

Эта функция остаётся корректной даже при немедленном reuse buffer, но `bytes.Clone` снова выделяет память. Pool может сократить временные внутренние allocation, а может не дать выигрыша в end-to-end метрике. Измерьте именно этот код с и без pool.

Риски `sync.Pool`

- забытый `Reset` может раскрыть данные предыдущего запроса;
- возврат `buffer.Bytes()` без копии создаёт use-after-Put логическую ошибку и data race;
- слишком большие буферы могут оставаться в pool и раздувать память;
- сложность ownership часто дороже небольшого сокращения `allocs/op`;
- pool не гарантирует, что конкретный объект будет возвращён тому же worker.

Здесь константу объявляют рядом с pool:

```
const maxPooledBufferCapacity = 64 << 10
```

Сначала `Reset`, затем решайте, возвращать ли буфер. Большой разовый запрос не должен навсегда превращать pool в набор мегабайтных буферов.

Оптимизация алгоритма важнее микрооптимизации

Ни `Builder`, ни `Pool` не компенсируют неверный объём работы. Для Task Manager обычно сильнее влияют:

- пагинация и лимит результата вместо чтения всех задач;
- фильтрация и сортировка в SQL с подходящим индексом вместо обработки всего набора в Go;
- устранение N+1 запросов;
- кеширование только при чёткой стратегии invalidation;
- batch-операции вместо одного сетевого вызова на элемент;
- разумные timeout и concurrency limit для внешних сервисов.

Но и эти изменения требуют измерений, тестов контракта и плана rollout. «Быстрее» не оправдывает устаревшие данные, неограниченный cache или потерю ошибок.

Проверка после изменения

После оптимизации выполните один и тот же protocol:

```
go test ./...
go test -race ./...
```

```
./internal/export > new.txt
benchstat old.txt new.txtgo tool pprof –base old.mem new.mem
```

Если benchmark улучшился, но profile не изменился так, как ожидалось, перепроверьте сценарий. Если profile лучше, но p99 сервиса не изменился, bottleneck может быть выше уровнем. Сохраняйте только изменения, которые улучшают важную метрику и не ухудшают корректность или сопровождение.

Итог

Хорошая performance-работа — не набор магических пакетов. Это короткий воспроизводимый цикл: измерить, понять, сделать минимальное изменение, подтвердить результат и остановиться. Далее закрепим этот цикл на вопросах и заданиях.

Самопроверка: производительность и диагностика

1. С чего нужно начать работу над предполагаемой проблемой производительности?

A. Сформулировать сценарий, важную метрику и воспроизводимую гипотезу

B. Заменить все строки на []byte

C. Сразу добавить sync.Pool ко всем временным объектам

D. Запустить goroutine для каждой операции

2. Какой файл содержит Go benchmark?

A. Файл с суффиксом _test.go и функцию BenchmarkXxx(*testing.B)

B. Только файл go.mod

C. Файл с суффиксом _profile.go

D. Любой .go с функцией main

3. Зачем вызывать b.ResetTimer() после подготовки benchmark-данных?

A. Чтобы сборщик мусора перестал работать

B. Чтобы включить race detector

C. Чтобы b.Loop стал бесконечным

D. Чтобы setup не попадал во время измеряемой операции

4. Зачем сохранять результат benchmark-функции в package-level sink?

A. Чтобы запретить любые аллокации

B. Чтобы вычисление было observable и компилятор не смог удалить его как ненужное

C. Чтобы тест всегда выполнялся в одной goroutine

D. Чтобы результат автоматически попал в pprof

5. Что добавляет флаг -benchmem к benchmark-выводу?

A. Список goroutine

B. Только покрытие тестами

C. B/op и allocs/op

D. HTTP latency percentiles

6. Какое утверждение о B/op верно?

A. Это точный размер binary-файла программы

B. Это число CPU-ядер

C. Это p99 latency endpoint

D. Это байты, выделенные на операцию; показатель не равен удерживаемой памяти после GC

7. Для чего используют benchstat?

A. Для запуска pprof endpoint в Gin

B. Для генерации SQL-миграций

C. Для автоматического исправления гонок данных

D. Для статистического сравнения серий benchmark до и после изменения

8. Какой профиль отвечает на вопрос, где программа проводит CPU-время?

A. Heap profile только inuse_space

B. go vet

C. CPU profile

D. Race detector

9. В чём разница между alloc_space и inuse_space heap profile?

A. alloc_space показывает всего выделенную память, inuse_space — память, остающуюся живой в heap

B. inuse_space запрещён в Go

C. Они всегда содержат идентичные данные

D. Первый измеряет CPU, второй HTTP-запросы

10. Что означает сообщение escape analysis "moved to heap"?

A. Компилятор выбрал размещение объекта в heap; это повод проверить метрики, а не автоматический баг

B. Файл удалён из проекта

C. В программе обязательно произошла утечка памяти

D. Нужно немедленно применить unsafe

11. Когда strings.Builder наиболее оправдан?

A. При построении большого текста последовательными добавлениями в цикле

B. Для хранения database/sql соединений

C. Для любого сложения двух коротких строк

D. Вместо JSON Encoder при любом HTTP-ответе

12. Почему sync.Pool нельзя использовать как обязательный ресурсный pool или постоянный cache?

A. sync.Pool нельзя применять с bytes.Buffer

B. Он гарантирует возврат объекта той же goroutine

C. Runtime может очистить pool, а Get может вернуть nil; pool предназначен для временного reuse

D. Он автоматически сохраняет объекты в PostgreSQL

Ответы к самопроверке

1. Сформулировать сценарий, важную метрику и воспроизводимую гипотезу

2. Файл с суффиксом _test.go и функцию BenchmarkXxx(*testing.B)

3. Чтобы setup не попадал во время измеряемой операции

4. Чтобы вычисление было observable и компилятор не смог удалить его как ненужное

5. B/op и allocs/op

6. Это байты, выделенные на операцию; показатель не равен удерживаемой памяти после GC

7. Для статистического сравнения серий benchmark до и после изменения

8. CPU profile

9. alloc_space показывает всего выделенную память, inuse_space — память, остающуюся живой в heap

10. Компилятор выбрал размещение объекта в heap; это повод проверить метрики, а не автоматический баг

11. При построении большого текста последовательными добавлениями в цикле

12. Runtime может очистить pool, а Get может вернуть nil; pool предназначен для временного reuse

Задания: производительность и диагностика

Практика строит измерительный workflow, а не требует «выиграть» заранее заданный процент. Числа на разных компьютерах различаются; важны воспроизводимость, сохранение поведения и аргументированное решение.

Для заданий создайте пакет, например `internal/export`, и файл `export_benchmark_test.go`. Сначала убедитесь, что обычные тесты проходят:

```
go test ./...
go test -race ./...
```

Задание 1. Benchmark экспорта с корректным setup

Есть модель:

```
type Task struct {
  ID       int64
  Title    string
  Priority int
}
```

Напишите `BuildExport(tasks []Task) string`, которая формирует строки `ID\tTitle\tPriority\n`. Затем создайте `BenchmarkBuildExport` для 1 000 заранее созданных задач.

Критерии проверки:

- benchmark находится в `*_test.go` и называется `BenchmarkBuildExport`;
- данные создаются до `b.ResetTimer()`;
- основной цикл использует `b.Loop()` или совместимый `for i := 0; i < b.N; i++`;
- результат сохраняется в sink-переменную или другим способом остаётся observable;
- включён `b.ReportAllocs()`;
- benchmark не пишет в stdout, не делает I/O и не генерирует данные в измеряемом цикле.

Подсказка

```
var benchmarkExport string

func BenchmarkBuildExport(b *testing.B) {
  tasks := makeTasks(1_000)
  b.ReportAllocs()
  b.ResetTimer()

  for b.Loop() {
    benchmarkExport = BuildExport(tasks)
  }
}
```

Решение

```
package export

import (
  "strconv"
  "strings"
  "testing"
)

type Task struct {
  ID       int64
  Title    string
  Priority int
}

func BuildExport(tasks []Task) string {
  var builder strings.Builder
  for _, task := range tasks {
    builder.WriteString(strconv.FormatInt(task.ID, 10))
    builder.WriteByte('\t')
    builder.WriteString(task.Title)
    builder.WriteByte('\t')
    builder.WriteString(strconv.Itoa(task.Priority))
    builder.WriteByte('\n')
  }
  return builder.String()
}
```

▼ Сведения

```
func makeTasks(count int) []Task {  tasks := make([]Task, count) for index := range
tasks {    tasks[index] = Task{       ID:       int64(index + 1),      Title:
"Задача",       Priority: index%5 + 1,     } } return tasks}var benchmarkExport
stringfunc BenchmarkBuildExport(b *testing.B) {  tasks := makeTasks(1_000)
b.ReportAllocs()  b.ResetTimer()  for b.Loop() {    benchmarkExport =
BuildExport(tasks)  }}
```

Запуск:

```
go test -run '^$' -bench '^BenchmarkBuildExport$' -benchmem ./internal/export
```

Задание 2. Сравните две реализации через `benchstat`

Реализуйте вторую версию `BuildExportConcat`, которая использует `result += ...` в цикле. Добавьте отдельный benchmark. Снимите по 10 измерений до/после изменения одной из версий и сравните их через `benchstat`.

Критерии проверки:

- у двух реализаций одинаковый публичный результат; это подтверждает unit-тест;
- benchmark-данные и команды до/после совпадают;
- запуск использует `-benchmem -count=10`;
- в выводе сравниваются время, байты и аллокации;
- вывод содержит краткое заключение: какой вариант выбран и почему;
- решение не основывается только на одном случайном запуске.

Подсказка

```
go test -run '^$' -bench '^BenchmarkBuildExport' -benchmem -count=10
./internal/export > old.txt
go test -run '^$' -bench '^BenchmarkBuildExport' -benchmem -count=10
./internal/export > new.txt
benchstat old.txt new.txt
```

Решение

```
func BuildExportConcat(tasks []Task) string {
  result := ""
  for _, task := range tasks {
    result += strconv.FormatInt(task.ID, 10) + "\t" + task.Title + "\t" +
strconv.Itoa(task.Priority) + "\n"
  }
  return result
}

func BenchmarkBuildExportConcat(b *testing.B) {
  tasks := makeTasks(1_000)
  b.ReportAllocs()
  b.ResetTimer()

  for b.Loop() {
    benchmarkExport = BuildExportConcat(tasks)
  }
}

func TestBuildExportImplementationsMatch(t *testing.T) {
  tasks := makeTasks(10)

  if got, want := BuildExportConcat(tasks), BuildExport(tasks); got != want {
    t.Fatalf("экспорты различаются\ngot:\n%s\nwant:\n%s", got, want)
  }
}
```

Обычно `strings.Builder` выигрывает на большом количестве строк, но не делайте это предположением теста. В commit или PR зафиксируйте фактические команды, входной размер и вывод `benchstat`.

Задание 3. CPU и heap profile benchmark

Для выбранной реализации снимите CPU и memory profiles. Найдите в `pprof` функцию с заметным cumulative cost и место с заметными allocation. Сравните `alloc_space` и `inuse_space` и объясните разницу для этого сценария.

Критерии проверки:

- benchmark запускается с фиксированным именем и `-benchtime=3s`;
- CPU profile открывается через `go tool pprof cpu.out`;
- memory profile рассматривается хотя бы в `alloc_space` и `inuse_space`;
- в заметке указаны команды `top`, `top -cum` и `list <function>`;
- не делается вывод об утечке только по высокому `alloc_space`;
- профили не добавляются в Git без осмысленной причины и проверки содержимого.

Подсказка

```
go test -run '^$' -bench '^BenchmarkBuildExport$' -benchtime=3s \
  -cpuprofile=cpu.out -memprofile=mem.out ./internal/export

go tool pprof cpu.out
go tool pprof -alloc_space mem.out
go tool pprof -inuse_space mem.out
```

Ожидаемый результат

Конкретные верхние функции зависят от версии Go и вашей реализации. В отчёте должно быть не «pprof открыт», а связка наблюдения и вывода, например:

```
CPU: buildExport занимает cumulative время вместе с strconv и strings.Builder.
Heap alloc_space: больше всего выделений связано с итоговой строкой/ростом буфера.
Heap inuse_space: после завершения benchmark эти временные объекты не удерживаются,
поэтому высокий alloc_space сам по себе не доказывает утечку.
```

Если profile показывает стоимость `fmt`, следующая гипотеза может касаться форматирования. Не меняйте его до benchmark-сравнения и проверки результата.

Задание 4. Проверьте гипотезу об аллокациях

Добавьте в `BuildExport` `builder.Grow` с консервативной оценкой, например `len(tasks) * 24`. Сравните версию до и после на размерах 10, 1 000 и 100 000 задач. Посмотрите `go test -gcflags='-m=2'` и объясните хотя бы одну строку escape analysis, не делая из неё автоматический вывод об ошибке.

Критерии проверки:

- есть sub-benchmarks с несколькими размерами входа;
- `Grow` не получает размер, вычисленный из неограниченного внешнего HTTP-ввода;
- до и после сравниваются в одинаковом окружении;
- в отчёте разделены наблюдаемые `allocs/op` и сообщения compiler diagnostics;
- итоговое решение сохраняет только устойчиво полезное изменение;
- тест подтверждает, что результат экспорта не изменился.

Подсказка

```
func BenchmarkBuildExportSizes(b *testing.B) {
  for _, size := range []int{10, 1_000, 100_000} {
    b.Run(fmt.Sprintf("tasks=%d", size), func(b *testing.B) {
      // setup, ResetTimer, b.Loop
    })
  }
}
```

Направление решения

`Grow` — это эвристика. Он может уменьшить число расширений буфера, но слишком большая оценка повышает пиковое потребление памяти, особенно для коротких title или малых входов. Поэтому корректный вывод может быть любым из трёх:

1. оставить `Grow`, если есть устойчивое улучшение важной метрики;
2. не использовать `Grow`, если выигрыш шумный или ухудшает малые сценарии;
3. применить его только в другом уровне API, где размер ограничен и известен лучше.

Сообщение `moved to heap` означает решение компилятора о размещении объекта в текущей версии и форме кода. Сверьте его с `allocs/op` и profile, прежде чем менять API или усложнять код.

Задание 5. Обоснованно откажитесь от `sync.Pool`

Коллега предлагает добавить `sync.Pool` для `bytes.Buffer` в `BuildExport`, чтобы «избавиться от всех аллокаций». Напишите короткий design note (в README пакета, issue или комментарии к PR) с решением: добавить pool или отказаться от него.

Если добавляете pool, реализуйте корректную функцию, которая не возвращает слайс, ссылающийся на buffer после `Put`, вызывает `Reset` и не кладёт обратно чрезмерно большой buffer. Сравните end-to-end benchmark с версией без pool.

Критерии проверки:

- note объясняет измеряемый сценарий и важную метрику;
- учитываются `Get == nil`, очистка pool runtime и отсутствие гарантии конкретного объекта;
- описано ownership возвращаемых `[]byte` / `string`;
- указаны риски утечки данных предыдущего запроса и удержания больших буферов;
- выбранное решение подтверждено benchmark и не меняет внешний контракт;
- pool не используется для `sql.DB`, HTTP client, каналов, постоянного cache или обязательного ресурса.

Подсказка

Безопасный возврат байтов после `Put` требует независимой копии:

```
result := bytes.Clone(buffer.Bytes())
bufferPool.Put(buffer)
return result
```

Подумайте, не отменяет ли эта копия ожидаемый выигрыш в вашей end-to-end метрике.

Пример обоснованного отказа

```
Сценарий: формирование экспорта из 1 000 задач.
Метрики: ns/op, B/op, allocs/op.
Наблюдение: strings.Builder уже даёт одну итоговую строку; добавление pool
не улучшило время за пределами шума и усложнило ownership буфера.
Решение: sync.Pool не добавляем. Если будущий CPU/heap profile покажет заметную
стоимость временных буферов в нагруженном endpoint, повторим измерение отдельно.
```

Отказ от оптимизации — хороший результат, если он основан на измерении и сохраняет простой корректный код.

Проверьте себя

Перед переходом к следующей главе убедитесь, что вы можете:

- сформулировать симптом, метрику, входные данные и гипотезу до изменения кода;
- написать benchmark без setup и I/O в измеряемом цикле;
- сравнить серии измерений через `benchstat`, а не один запуск;
- объяснить разницу CPU profile, `alloc_space` и `inuse_space`;
- использовать escape analysis как подсказку, а не как список обязательных исправлений;
- оценить `strings.Builder` и `sync.Pool` по реальному сценарию, ownership и сложности;
- остановиться после подтверждённого улучшения, не превращая код в коллекцию случайных микрооптимизаций.

Рекомендации по коду: производительность и диагностика

Методика

- Начинайте с наблюдаемого симптома и важной метрики: latency, throughput, CPU, allocation rate, retained heap или contention.

- Фиксируйте сценарий: размер и форму входа, конфигурацию, версию Go, команду, commit и окружение измерения.
- Формулируйте проверяемую гипотезу до изменения; не выбирайте `Builder`, `Pool` или concurrency как заранее известное лекарство.
- Сохраняйте функциональный контракт: до и после запускайте unit-тесты, `go vet` и `go test -race`, если изменения затрагивают конкурентный код.
- Предпочитайте сокращение ненужной работы — пагинацию, batching, правильный SQL и устранение N+1 — микрооптимизации локального кода.

Benchmarks

- Делайте одну понятную операцию на benchmark-итерацию; preparation, random input и I/O выносите из измеряемого цикла, если они не являются объектом измерения.
- Используйте `b.Loop()` в поддерживаемой версии Go либо корректный цикл по `b.N`; сохраняйте результат в sink, чтобы вычисление не исчезло.
- Включайте `b.ReportAllocs()` или запускайте с `-benchmem`, если исследуете память.
- Используйте sub-benchmarks для разных размеров входа и имена, которые объясняют сценарий.
- Не делайте вывод по одному запуску. Сравнивайте серии одинаковых запусков через `benchstat`.
- Не переносите абсолютные `ns/op` между разными машинами, версиями Go и режимами питания как универсальный факт.

Profiles и память

- CPU profile отвечает на вопрос о процессорном времени; heap profile различает всего выделенную (`alloc_space`) и удерживаемую (`inuse_space`) память.
- Начинайте исследование `pprof` с `top`, `top -cum` и `list`; смотрите callers, а не только строку с большим flat time.
- Снимайте profile при стабильной репрезентативной нагрузке. Idle profile почти ничего не говорит о стоимости endpoint.
- Сравнивайте до/после через `pprof -base` только при одинаковом сценарии и подтверждайте вывод benchmark-серией.
- Не считайте высокий allocation rate автоматической утечкой; ищите удерживаемые ссылки и рост `inuse_space` после GC.
- Держите runtime pprof вне публичного router, ограничивайте loopback/internal network и не публикуйте profiles без проверки содержимого.

Аллокации и точечные оптимизации

- Используйте `go test -gcflags='-m=2'` как диагностику решения компилятора, а не как требование устранить каждый escape.
- Применяйте `strings.Builder` для большого текста в цикле; для короткой простой конкатенации оставляйте читаемый код, пока benchmark не доказал проблему.
- `Grow` — ограниченная эвристика. Не рассчитывайте размер из недоверенного или неограниченного внешнего ввода.

- Не меняйте immutable `string` API на mutable `[]byte` без явных правил ownership и подтверждённой необходимости.
- Используйте `sync.Pool` только для временных объектов в доказанно горячем пути; `Get` может вернуть nil, а runtime может очистить pool.
- Перед `Put` очищайте объект, не возвращайте caller память после `Put` и не сохраняйте чрезмерно большие buffers в pool.
- Никогда не используйте `sync.Pool` как pool соединений, обязательный cache, очередь, канал передачи владения или механизм управления жизненным циклом.

Итоговый чек-лист

- [ ] Есть симптом, репрезентативный сценарий, baseline и измеряемый критерий успеха.
- [ ] Benchmark измеряет одну операцию, не включает случайный setup и сохраняет результат.
- [ ] Вывод основан на серии запусков и при необходимости подтверждён `benchstat`.
- [ ] Profile объясняет предполагаемый bottleneck, а не служит декоративным артефактом.
- [ ] `alloc_space` не спутан с retained heap или memory leak.
- [ ] Изменение не ухудшило контракт, тестируемость, безопасность данных и понятность кода.
- [ ] `Builder`, `Grow`, `sync.Pool` и другие оптимизации оставлены только при доказанной пользе.

Продвинутый Go E: reflection, `unsafe` и cgo

Цель главы

Понять назначение reflection, `unsafe` и cgo, безопасно читать их код в зависимостях, распознавать риски и выбирать обычные типы, интерфейсы, code generation или библиотечную интеграцию, когда эти инструменты не нужны.

Это не глава о том, как «обойти» систему типов Go. У всех трёх механизмов есть законные узкие области применения:

- **reflection** позволяет библиотеке исследовать типы и значения во время выполнения;
- **`unsafe`** открывает низкоуровневые операции с памятью, которые язык обычно запрещает;
- **cgo** вызывает C-код и C-библиотеки из Go.

Но в прикладном Task Manager API они почти никогда не нужны в handler, service, repository или доменной модели. Обычный статически типизированный Go-код лучше проверяется компилятором, проще рефакторится и безопаснее переносится между платформами.

Карта главы

1. **Эта страница** — границы применения, решения до выбора low-level инструмента.
2. Reflection: `reflect.Type` и `reflect.Value` — устройство API, теги, изменяемость и роль в JSON/ORM/framework.
3. `unsafe`: память и границы гарантий — `Sizeof`, `Pointer`, `uintptr`, GC и проверка `checkptr`.
4. cgo: вызов C-кода из Go — минимальный вызов, владение памятью, потоки, сборка и критерии отказа.

Модель принятия решения

Перед использованием reflection, `unsafe` или cgo пройдите по порядку:

1. **Можно ли написать конкретный код?** Для `Task` явный `task.Title` лучше динамического поиска поля по строке.
2. **Подойдёт ли маленький интерфейс?** Если нужен метод `Validate() error`, интерфейс выражает контракт на этапе компиляции.
3. **Можно ли применить generics?** Для алгоритма над `[]T` generic-функция сохраняет информацию о типе без runtime inspection.
4. **Нужна ли code generation?** Для стабильного schema/API protobuf, OpenAPI, SQLC или `go generate` дают проверяемый код заранее.
5. **Есть ли внешняя граница, где динамика неизбежна?** JSON schema, ORM metadata, plugin protocol или существующая C-библиотека могут обосновать специальный инструмент.
6. **Документированы ли контракт, тесты, производительность и ownership?** Если нет — инструмент выбран слишком рано.

```
конкретный тип → интерфейс/generics → генерация кода → reflection → unsafe/cgo
       проще и безопаснее                                  ↑ только при доказанной
необходимости
```

Это не абсолютная иерархия. `encoding/json` разумно использует reflection, потому что библиотека должна работать с произвольными пользовательскими структурами. Но приложение, которое знает свой `CreateTaskRequest`, не получает ту же выгоду от универсального runtime validator.

Цена динамики

Инструмент	Что даёт	Основная цена
Reflection	динамический доступ к типам, полям, тегам и методам	runtime panic, меньше проверок компилятором, медленнее и хрупче refactor
`unsafe`	работа с представлением памяти	возможные memory corruption, нарушение GC-инвариантов, platform dependence
cgo	доступ к C ABI и C-библиотекам	сложная сборка, ownership на двух рантаймах, blocking calls, portability и security surface

Если одна из этих цен не компенсирована конкретным требованием, предпочтительнее обычный Go.

Практические роли в экосистеме

JSON

`encoding/json` использует reflection, чтобы прочитать экспортируемые поля struct и их теги `json:"title"`. Благодаря этому библиотека не требует отдельной функции сериализации для каждой структуры.

Но reflection не заменяет контракт API:

- JSON decoder проверяет форму данных, но не бизнес-правила;
- неизвестные поля и `omitempty` — часть внешнего контракта, которую нужно тестировать;
- handler должен декодировать конкретный DTO, а не принимать `map[string]any` без необходимости;
- custom `MarshalJSON` / `UnmarshalJSON` нужны только для особого формата, не как способ «ускорить всё».

ORM и frameworks

ORM может по тегам построить SQL schema, web-framework — связать path/query/body со структурой, DI-container — собрать зависимости по типу. Это удобно на интеграционной границе, но скрывает часть поведения от компилятора. При подключении такой библиотеки проверяйте

SQL, миграции, ошибки валидации, производительность и поведение при rename полей интеграционными тестами.

`unsafe` и производительные библиотеки

Некоторые низкоуровневые библиотеки применяют `unsafe` для оптимизации, syscall или zero-copy преобразований. Пользователь библиотеки не должен копировать этот подход в application code. Сперва обновите библиотеку, прочитайте её contract и измерьте реальную проблему.

C-библиотеки

cgo оправдан, когда есть существующая библиотека с важной функцией: системный SDK, crypto/HSM integration, hardware driver или зрелый native engine. Создайте тонкую изолированную обёртку на границе пакета; не распространяйте `C.*` типы по service и handler.

Общие правила безопасности

- Держите код с reflection, `unsafe` и cgo маленьким, локальным и хорошо протестированным.
- Не передавайте недоверенный ввод в динамический доступ к полям/методам или в C API без строгой валидации.
- Не превращайте runtime panic в тихое игнорирование ошибки. Проверяйте `Kind`, `IsValid`, `CanSet`, ошибки C API и возвращайте обычный `error` на границе.
- Не делайте предположений о layout struct, размере указателя и порядке полей без явной документированной платформенной причины.
- Запускайте `go vet ./...`, unit/integration tests, `go test -race ./...` при релевантности и несколько target platform сборок для cgo-кода.
- Документируйте minimum Go version, platform/architecture limits, ownership памяти и план замены зависимости.

Следующая страница начнёт с самого безопасного из трёх механизмов — reflection, но и там будет много причин предпочесть явный код.

Reflection: `reflect.Type` и `reflect.Value`

Что отражает пакет `reflect`

В обычном Go компилятор знает тип заранее:

```
func taskTitle(task Task) string {
  return task.Title
}
```

Reflection нужен, когда библиотека получает `any` и должна исследовать неизвестный до запуска конкретный тип. Две основные точки входа:

```
value := Task{ID: 1, Title: "Купить молоко", Priority: 2}

typ := reflect.TypeOf(value)
reflectedValue := reflect.ValueOf(value)

fmt.Println(typ.Name())           // Task
fmt.Println(typ.Kind())           // struct
fmt.Println(reflectedValue.Kind()) // struct
```

- `reflect.Type` описывает тип: имя, `Kind`, поля, методы, теги;
- `reflect.Value` представляет конкретное значение: его можно читать, а в ограниченных условиях — менять или вызывать методы.

`Kind` — широкая категория (`struct`, `slice`, `map`, `ptr`, `string`), а не точный named type. У `Task`, `CreateTaskRequest` и любой другой struct будет `Kind() == reflect.Struct`; для различения конкретного типа используйте сам `Type` или явный type switch.

Сначала type switch, потом reflection

Если число вариантов известно, type switch безопаснее и понятнее:

```
func taskID(value any) (TaskID, bool) {
  switch current := value.(type) {
  case Task:
    return current.ID, true
  case *Task:
    if current == nil {
      return 0, false
    }
    return current.ID, true
  default:
    return 0, false
  }
}
```

Reflection оправдан, когда набор структур действительно открыт: библиотека metadata, serializer, framework integration. Не заменяйте type switch reflection только ради «универсальности» для двух известных типов.

Инспекция структуры

`Type.NumField` и `Type.Field` позволяют библиотеке обойти поля struct. Сначала проверьте `Kind`, иначе `NumField` вызовет panic:

```
func exportedFieldNames(value any) ([]string, error) {
  typ := reflect.TypeOf(value)
  if typ == nil {
    return nil, errors.New("значение не должно быть nil")
  }

  if typ.Kind() == reflect.Ptr {
    typ = typ.Elem()
  }
  if typ.Kind() != reflect.Struct {
    return nil, fmt.Errorf("ожидалась структура, получен %s", typ.Kind())
  }

  names := make([]string, 0, typ.NumField())
  for index := 0; index < typ.NumField(); index++ {
    field := typ.Field(index)
    if field.PkgPath != "" {
      continue // Неэкспортируемое поле.
    }
    names = append(names, field.Name)
  }
  return names, nil
}
```

Для поля `reflect.StructField.PkgPath == ""` означает, что оно экспортируемо. Это важно для библиотек, похожих на JSON encoder: private поля не часть внешнего contract и reflection не должен обходить правила видимости через unsafe.

Struct tags

Тег — строка metadata рядом с объявлением поля. `encoding/json` интерпретирует тег `json`, но пакет `reflect` только возвращает текст:

```
type TaskDTO struct {
  ID       int64  `json:"id" db:"id"`
  Title    string `json:"title" db:"title"`
  Internal string `json:"-"`
}

func jsonFieldName(field reflect.StructField) (string, bool) {
  tag := field.Tag.Get("json")
  if tag == "-" {
    return "", false
```

```
  } name, _, _ := strings.Cut(tag, ",")   if name == "" {     return field.Name, true
} return name, true}
```

`StructTag.Get` возвращает пустую строку и для отсутствующего тега, и для тега с пустым значением. Если это различие важно, используйте `Lookup`:

```
raw, exists := field.Tag.Lookup("json")
```

Не пишите свой JSON encoder, SQL mapper или validator только потому, что tags доступны. Форматы, embedded fields, `omitempty`, pointers, custom marshalers, time и null semantics быстро делают простой reflection-код неполной и хрупкой копией зрелой библиотеки.

`reflect.Value`: чтение и nil

`reflect.ValueOf(nil)` возвращает invalid Value. Методы вроде `Kind` на invalid value допустимы и вернут `Invalid`, но `Interface`, `Elem` и многие другие операции могут вызвать panic. Проверяйте `IsValid`:

```
func describe(value any) string {
  reflected := reflect.ValueOf(value)
  if !reflected.IsValid() {
    return "<nil>"
  }

  return reflected.Type().String()
}
```

Nil pointer и nil interface — разные ситуации:

```
var task *Task
var value any = task

reflected := reflect.ValueOf(value)
fmt.Println(reflected.Kind())  // ptr
fmt.Println(reflected.IsNil()) // true
```

`IsNil` разрешён только для подходящих Kind (`chan`, `func`, `map`, `pointer`, `interface`, `slice`). Вызов `IsNil` у struct или int вызовет panic. Поэтому reflection-код должен сначала проверять `Kind`.

Изменение значений: addressable и `CanSet`

Значение, переданное в `reflect.ValueOf`, обычно является копией и не settable:

```
task := Task{Title: "Купить молоко"}
value := reflect.ValueOf(task)

field := value.FieldByName("Title")
fmt.Println(field.CanSet()) // false
// field.SetString("Купить продукты") // panic
```

Чтобы менять исходную структуру, передайте non-nil pointer, вызовите `Elem` и проверьте всё до `Set`:

```
func setExportedString(target any, fieldName, newValue string) error {
  value := reflect.ValueOf(target)
  if !value.IsValid() || value.Kind() != reflect.Ptr || value.IsNil() {
    return errors.New("target должен быть non-nil указателем на структуру")
  }

  value = value.Elem()
  if value.Kind() != reflect.Struct {
    return errors.New("target должен указывать на структуру")
  }

  field := value.FieldByName(fieldName)
  if !field.IsValid() {
    return fmt.Errorf("поле %q не найдено", fieldName)
  }
  if field.Kind() != reflect.String || !field.CanSet() {
    return fmt.Errorf("поле %q должно быть доступной строкой", fieldName)
  }

  field.SetString(newValue)
  return nil
}
```

Этот helper учебный. В Task Manager явное `task.Title = newValue` лучше: имя поля проверяется компилятором, IDE находит usages, а рефакторинг не ломает строку `"Title"` во время запуска.

Методы через reflection

`Value.MethodByName` и `Call` позволяют вызвать метод по строке, но теряют проверку сигнатуры при компиляции:

```
method := reflect.ValueOf(task).MethodByName("Summary")
if !method.IsValid() {
  return errors.New("метод Summary не найден")
}

results := method.Call(nil)
summary, ok := results[0].Interface().(string)
if !ok {
  return errors.New("Summary должен вернуть string")
}
```

Неверное число аргументов, неправильный тип или вызов метода, который panics, тоже могут закончиться panic. Если API требует `Summary() string`, определите маленький интерфейс и вызовите метод напрямую. Reflection method call оставьте для framework/extension boundary с документированным динамическим contract.

Производительность и cache metadata

Reflection медленнее прямого доступа, но реальная стоимость зависит от сценария. В serializer/ORM повторный обход `Type` и tags для каждой записи может стать заметным. Библиотеки часто кэшируют metadata по `reflect.Type`:

```
type fieldMetadata struct {
  index int
  name  string
}

var metadataCache sync.Map // map[reflect.Type][]fieldMetadata
```

Такой cache сам создаёт ответственность: concurrency safety, bounded growth, корректность embedded fields и тесты. Не добавляйте его до profile. Для application-кода обычно выгоднее вообще не писать reflection loop и использовать `encoding/json`, конкретный mapper или generated code.

Проверяйте dynamic contract тестами

Для reflection-helper тестируйте минимум:

- ожидаемую структуру и exported/private поля;
- non-struct input;
- `nil` interface и typed nil pointer;
- отсутствующее поле, неверный Kind и невозможность `Set`;
- теги `json:"-"`, пустое имя и опции;

- изменения после rename поля, если строковые имена являются частью API.

Нижний слой с reflection должен возвращать обычный `error` для неправильной формы, а не полагаться на `recover` как на регулярную валидацию. Следующая страница покажет инструмент, который может вообще обойти эти проверки, — `unsafe`.

`unsafe`: память и границы гарантий

Пакет `unsafe` существует для runtime, syscall, cgo и небольшого числа системных библиотек. Он позволяет выразить операции, которые обычная система типов запрещает. Название буквально означает: компилятор больше не может гарантировать безопасность памяти.

Для Task Manager API `unsafe` не нужен. Не используйте его для «ускорения» JSON, преобразования строк, доступа к private полям или ручной упаковки сетевых данных без измерения, глубокого понимания lifetime и строгой необходимости.

Что остаётся безопасным: compile-time layout information

Некоторые операции полезны для диагностики низкоуровневых интеграций:

```
type header struct {
  Version uint16
  Flags   uint8
  Length  uint32
}

fmt.Println(unsafe.Sizeof(header{}))        // размер в байтах на текущей архитектуре
fmt.Println(unsafe.Alignof(header{}))       // требуемое выравнивание
fmt.Println(unsafe.Offsetof(header{}.Length)) // смещение поля
```

Эти значения зависят от архитектуры, выравнивания и layout компилятора. Они не делают Go struct бинарным network protocol: padding, endianness, версия протокола и проверка длины остаются отдельными задачами. Для внешнего формата обычно используйте явный `encoding/binary`.

```
func encodeHeader(version uint16, flags uint8, length uint32) []byte {
  data := make([]byte, 7)
  binary.BigEndian.PutUint16(data[0:2], version)
  data[2] = flags
  binary.BigEndian.PutUint32(data[3:7], length)
  return data
}
```

Этот вариант явно фиксирует формат независимо от padding и архитектуры.

`unsafe.Pointer` и `uintptr`

`unsafe.Pointer` — специальный тип указателя, который можно конвертировать в/из указателей других типов при соблюдении жёстких правил. `uintptr` — целое число, достаточно большое для представления адреса, но **не указатель для garbage collector**.

Главная опасность: если сохранить адрес только как `uintptr`, GC может считать объект недостижимым, переместить/освободить его в будущих реализациях или не сохранить требуемое время жизни. Между отдельными выражениями могут происходить safe points и вызовы runtime.

```
// Небезопасный анти-паттерн: не храните адрес как uintptr для будущего
использования.
address := uintptr(unsafe.Pointer(&task))
// ... произвольная работа, allocations, вызовы функций ...
taskPointer := (*Task)(unsafe.Pointer(address))
```

Не лечите это `runtime.KeepAlive` без точного понимания официальных правил: `KeepAlive` решает отдельные проблемы lifetime для корректно спроектированного low-level кода, а не делает произвольную арифметику адресов безопасной.

Указательная арифметика не для application code

`unsafe.Add` может сдвинуть pointer на число байтов. Для обычного слайса это почти всегда хуже индексирования:

```
// Ясно, проверяется bounds check и остаётся переносимым.
priority := priorities[index]
```

Вариант с указателями требует доказать границы, выравнивание и lifetime вручную. Ошибка может не вызвать panic там, где вы ожидаете, а тихо испортить память. Даже если microbenchmark показывает выигрыш, сначала оцените алгоритм, allocation, batch size и I/O: они почти всегда важнее.

Zero-copy преобразования — риск ownership

Строка в Go неизменяема, а `[]byte` изменяем. Опасные zero-copy преобразования могут позволить одному коду изменить байты, которые другой считает строкой, или сохранить ссылку на буфер после его reuse. Это приводит к data race, повреждённым данным и security bugs.

В обычном коде используйте обычные преобразования:

```
text := string(data) // создаёт независимую строку
data = []byte(text)  // создаёт независимый слайс
```

Если profile доказал, что копирование — bottleneck, сначала ищите API, который принимает нужный вид данных (`io.Reader`, `io.Writer`, streaming encoder), или меняйте границу ownership. Только изолированная низкоуровневая библиотека с чёткими lifetime guarantees может обсуждать unsafe zero-copy.

Нельзя обращаться к private полям через unsafe

Reflection не даёт устанавливать неэкспортируемые поля из другого пакета. Это защита инкапсуляции. Обход через `unsafe` нарушает контракт пакета и может сломаться при обновлении зависимости:

```
// Не делайте так: offset private поля — не public API пакета.
// pointer := unsafe.Add(unsafe.Pointer(value), offset)
```

Если библиотека не предоставляет нужную возможность, используйте её публичный API, расширение, fork с осознанным сопровождением или другую библиотеку. Не делайте внутренний layout скрытой зависимостью production-сервиса.

Проверки low-level кода

`go vet` содержит проверки подозрительного использования `unsafe.Pointer`, но не доказывает отсутствие всех ошибок. Для дополнительной диагностики указателей запускайте тесты с checkptr:

```
go test -gcflags=all=-d=checkptr=2 ./internal/lowlevel
go test -race ./internal/lowlevel
```

`checkptr` ловит часть неправильных преобразований и арифметики, но не заменяет review, тесты на целевых архитектурах и понимание ownership. Нельзя писать код, который работает «только без checkptr», и считать это оптимизацией.

Когда `unsafe` может быть оправдан

Все условия должны выполняться одновременно:

- есть конкретная измеренная или платформенная необходимость;
- стандартная библиотека, явный binary encoding, конкретный API и code generation не решают проблему;
- код локализован в маленьком internal package;

- в комментарии описаны layout, lifetime, target architectures и причина невозможности обычного решения;
- есть тесты, benchmarks, `vet`, checkptr и проверка на нужных OS/arch;
- есть владелец, который будет сопровождать код при обновлении Go и зависимости.

Для большинства backend-проектов хотя бы одно условие не выполняется. Это достаточная причина отказаться.

Итог

`unsafe` — не более быстрый Go. Это выход за гарантии языка, который переносит доказательство корректности с компилятора на автора кода. Следующая страница рассмотрит похожую границу — переход между Go и C через cgo.

cgo: вызов C-кода из Go

Что делает cgo

cgo связывает Go-пакет с C compiler и C ABI. Файл с `import "C"` может содержать C preamble в комментарии непосредственно перед этим импортом:

```
package native

/*
#include <stdlib.h>
#include <string.h>
*/
import "C"

import "unsafe"

func CLength(value string) int {
  cValue := C.CString(value)
  defer C.free(unsafe.Pointer(cValue))

  return int(C.strlen(cValue))
}
```

`C.CString` выделяет null-terminated строку в C heap. Её освобождает C-функция `free`, а не Go garbage collector. Именно поэтому `defer C.free(...)` ставится сразу после успешного выделения.

Пример специально прост: в реальной программе `len(value)` быстрее, проще и не требует C compiler. Его цель — показать ownership на границе двух рантаймов, а не рекомендовать `strlen`

через cgo.

Почему cgo — не просто импорт

Обычный Go-код компилируется toolchain в один привычный workflow. cgo добавляет:

- установленный C compiler и headers на машине сборки;
- linker flags и native libraries на build/runtime machine;
- platform-specific ABI и различия Linux/macOS/Windows;
- отдельные правила lifetime для Go и C памяти;
- сложность cross-compilation, Docker images и CI;
- дополнительную security surface в C-коде и его зависимостях.

Перед `go get` wrapper-библиотеки проверьте, требует ли она cgo и какие system packages нужны. README приложения и Dockerfile должны воспроизводимо устанавливать эти зависимости.

Владение памятью: Go heap и C heap

Есть два разных владельца:

Память	Кто выделяет	Кто освобождает
Go value / `make` / `new`	Go runtime	garbage collector, когда объект недостижим
`C.CString`, `C.malloc`	C runtime	соответствующий `C.free`
C buffer из library	C library	функция освобождения из этой library

Нельзя освобождать Go-память через `C.free` и нельзя ждать, что Go GC освободит результат `C.malloc`. Всегда читайте C API: иногда библиотека требует `library_free`, а не стандартный `free`.

Если C возвращает `char*`, преобразуйте его и вызовите правильный cleanup:

```
func readLibraryMessage() (string, error) {
  message := C.library_message()
  if message == nil {
    return "", errors.New("library_message вернул nil")
  }
  defer C.library_free(message)

  return C.GoString(message), nil
}
```

`C.GoString` копирует данные в Go string. Не возвращайте `*C.char` выше Go-обёртки: это распространяет C ownership в service/handler и почти гарантирует утечки.

Правила указателей Go ↔ C

Главное упрощённое правило: **C не должен хранить Go pointer после завершения cgo-вызова**. Go GC не управляет C heap, а C runtime не знает lifetime Go объектов. Передача pointer в C также ограничена, если Go-объект содержит другие Go pointers.

Плохая модель:

```
Go создаёт Task → C сохраняет указатель → Go возвращается из вызова
→ GC/Go-код меняет жизненный цикл → C позже читает устаревший pointer
```

Для асинхронного callback или долгоживущего состояния не передавайте прямой указатель на Go object. Используйте документированный механизм handles (`runtime/cgo.Handle`), копируйте данные в C-managed memory либо проектируйте callback boundary по правилам библиотеки. Это сложная интеграция, которую нужно изолировать и покрыть integration tests/race tests.

Не передавайте `&slice[0]` , если слайс пуст, и не позволяйте C писать за пределы переданного буфера. Входной размер и возвращаемый C размер проверяются до преобразования в Go slice/string.

Ошибки C API

C часто возвращает код статуса, `NULL` или устанавливает `errno` , а не Go `error` . Тонкая Go-обёртка должна сразу переводить это в idiomatic contract:

```
func encrypt(input []byte) ([]byte, error) {
  // 1. Проверить длину input и выделить output по правилам C API.
  // 2. Вызвать C.encrypt(...).
  // 3. Проверить status/NULL/errno.
  // 4. Скопировать успешный результат в Go-managed []byte.
  // 5. Освободить C resources на всех путях.
  return nil, errors.New("пример: реализация зависит от C API")
}
```

Не возвращайте «частично заполненный» output как успешный и не паникуйте на код статуса C. Добавьте контекст через `fmt.Errorf` , но не включайте секретные input/output в error или log.

Производительность и блокировки

cgo call имеет стоимость перехода между runtime. Для редкой тяжёлой операции она может быть несущественной; для миллиона коротких вызовов она может доминировать. Измеряйте end-to-end benchmark и старайтесь использовать batch API вместо вызова C для каждого байта/поля.

Когда C блокируется в системном вызове, Go runtime может создать дополнительный OS thread, но это не делает вызов автоматически отменяемым через `context.Context` . Если C API не поддерживает отмену/timeout, shutdown Task Manager API может ждать native call до конца.

Выбирайте библиотеку с timeout/cancel mechanism, изолируйте вызов и явно документируйте ограничение.

Не передавайте arbitrary Go callbacks в C, не блокируйте C callback на channel send и не вызывайте длительную C-функцию под mutex. Эти комбинации затрудняют shutdown и могут создавать deadlock на границе двух runtime.

Сборка и portability

Для cgo сборка должна происходить в окружении, совместимом с target:

```
CGO_ENABLED=1 go build ./cmd/server
```

Cross-compilation больше не сводится к `GOOS=... GOARCH=... go build`: нужен подходящий cross C compiler, headers и target libraries. Docker build должен включать compiler в builder stage и нужные shared libraries в runtime stage, либо использовать статическую сборку, если это поддерживает конкретная library и license.

В CI добавьте отдельную job для cgo-package и целевой OS/architecture. Pure Go тесты не доказывают, что native linker, dynamic loader и system library работают в production image.

Когда лучше не использовать cgo

Откажитесь от cgo, если:

- библиотека нужна только для функции, которую уже даёт Go standard library или mature pure-Go dependency;
- код нужен на разных OS/architecture, а нет ресурса поддерживать native builds;
- security policy не допускает C dependency или закрытый native binary;
- dependency не поддерживает timeout, обновления или прозрачную license;
- integration нельзя протестировать в Docker/CI близко к production;
- ожидаемая выгода не подтверждена benchmark/profile.

В backend-разработке простой pure-Go вариант почти всегда выигрывает по deployability и сопровождению. cgo — архитектурное решение, а не локальная оптимизация.

Чек-лист C boundary

Перед merge убедитесь:

- [ ] `import "C"` и C code изолированы в маленьком пакете;
- [ ] для каждого C allocation есть правильный C cleanup на всех путях;
- [ ] Go API не экспортирует `C.*` типы наружу;
- [ ] C не хранит Go pointers после вызова; callback/handle lifetime документирован;
- [ ] входные размеры, nil и status codes валидируются;
- [ ] есть integration tests на target environment и воспроизводимый Docker/CI build;
- [ ] debug logs не раскрывают секреты из native boundary;

- [ ] известны поддерживаемые OS/arch, license и план обновления C dependency.

Самопроверка: reflection, `unsafe` и cgo

1. Когда reflection обычно оправдан?

A. Чтобы отменить context другой goroutine

B. Когда библиотека должна работать с открытым набором неизвестных заранее типов и metadata

C. Когда нужно прочитать известное поле Task.Title

D. Чтобы заменить любую проверку типов компилятором

2. Что описывает reflect.Type в отличие от reflect.Value?

A. C-выделение памяти

B. Только текущее содержимое конкретной map

C. Информацию о типе: Kind, поля, методы и теги

D. Указатель, который всегда можно изменить через Set

3. Почему перед Type.NumField нужно проверить Kind?

A. Kind проверяет JSON-синтаксис

B. NumField работает только для map

C. Это нужно только для ускорения компиляции

D. NumField применим к struct; другой Kind может вызвать panic

4. Что вернёт reflect.ValueOf(nil)?

A. Панику до возврата значения

B. Invalid Value, который нужно проверить через IsValid

C. Settable pointer на nil

D. reflect.Struct

5. Что нужно передать, чтобы reflection мог менять поле исходной структуры?

A. Non-nil указатель на структуру и проверить CanSet у поля

B. Имя поля в JSON-теге без указателя

C. uintptr адрес строки

D. Копию структуры как значение

6. Почему прямой доступ task.Title предпочтительнее FieldByName("Title") в прикладном коде?

A. Task.Title автоматически делает поле приватным

B. Reflection всегда требует cgo

C. FieldByName нельзя использовать в Go

D. Компилятор и IDE проверяют имя поля, а refactor не оставит скрытую строковую ошибку

7. Что верно для uintptr, полученного из unsafe.Pointer?

A. Он всегда равен размеру объекта

B. Это безопасная долгоживущая ссылка, управляемая GC

C. Его следует передавать в JSON API вместо ID

D. Это число, а не сохраняемый для GC указатель; нельзя держать адрес как uintptr для будущего использования

8. Почему Go struct нельзя напрямую считать сетевым бинарным протоколом через unsafe?

A. encoding/binary использует reflection и поэтому запрещён

B. Go struct нельзя хранить в памяти

C. Padding, выравнивание и endianness не образуют стабильный внешний формат

D. unsafe.Sizeof всегда возвращает ноль

9. Какой инструмент дополнительно проверяет часть некорректных unsafe преобразований?

A. go fmt -unsafe

B. go test -gcflags=all=-d=checkptr=2

C. benchstat -checkptr

D. go mod unsafe

10. Кто освобождает строку, созданную через C.CString?

A. json.Unmarshal

B. context.CancelFunc

C. Go garbage collector автоматически

D. C runtime через C.free (или функцию cleanup конкретной C-библиотеки)

11. Какое правило важно для Go pointer, переданного в C?

A. C может всегда хранить Go pointer в глобальной переменной

B. Go pointers нельзя передавать в C даже на время синхронного вызова

C. C не должен сохранять Go pointer после завершения cgo-вызова без специального документированного механизма

D. Нужно преобразовать любой pointer в uintptr и сохранить навсегда

12. Почему cgo усложняет deploy и CI?

A. Нужны совместимые C compiler, headers, linker и native libraries для target environment

B. Он автоматически публикует pprof в интернет

C. C cgo нельзя использовать Docker

D. cgo отключает все Go-тесты

Ответы к самопроверке

1. Когда библиотека должна работать с открытым набором неизвестных заранее типов и metadata

2. Информацию о типе: Kind, поля, методы и теги

3. NumField применим к struct; другой Kind может вызвать panic

4. Invalid Value, который нужно проверить через IsValid

5. Non-nil указатель на структуру и проверить CanSet у поля

6. Компилятор и IDE проверяют имя поля, а refactor не оставит скрытую строковую ошибку

7. Это число, а не сохраняемый для GC указатель; нельзя держать адрес как uintptr для будущего использования

8. Padding, выравнивание и endianness не образуют стабильный внешний формат

9. go test -gcflags=all=-d=checkptr=2

10. C runtime через C.free (или функцию cleanup конкретной C-библиотеки)

11. C не должен сохранять Go pointer после завершения cgo-вызова без специального документированного механизма

12. Нужны совместимые C compiler, headers, linker и native libraries для target environment

Задания: reflection, `unsafe` и cgo

Цель практики — научиться строить узкие безопасные границы и так же уверенно отказываться от low-level решения. Не внедряйте эти фрагменты напрямую в production Task Manager API: создайте изолированный учебный пакет, например `internal/metadata` или `internal/native`.

Перед и после изменения запускайте:

```
go test ./...
go vet ./...
go test -race ./...
```

Для cgo выполняйте задания, только если на вашей системе установлен C compiler. Отсутствие cgo toolchain — не ошибка Go-кода и не повод отключать его глобально в основном проекте.

Задание 1. Безопасная инспекция JSON metadata

Напишите функцию:

```
func JSONFields(value any) ([]string, error)
```

Она принимает struct или non-nil pointer на struct и возвращает имена экспортируемых полей в JSON-представлении. Учитывайте `json:"name"`, пропускайте `json:"-"`; для поля без тега используйте Go-имя. Функция не должна паниковать на `nil`, typed nil pointer, число или слайс.

Критерии проверки:

- `reflect.TypeOf(nil)` обрабатывается до вызовов, требующих type;
- pointer разворачивается через `Elem`, но typed nil pointer отклоняется;
- non-struct возвращает обычный `error`;
- private поля не попадают в ответ;
- `StructTag.Lookup` либо обоснованно `Get` корректно обрабатывает теги;
- тесты покрывают DTO с `json:"-"`, tag options (`json:"title,omitempty"`) и невалидный input.

Подсказка

```
if typ.Kind() == reflect.Ptr {
  typ = typ.Elem()
}
if typ.Kind() != reflect.Struct {
  return nil, errors.New("ожидалась структура")
}
```

Решение

```
package metadata

import (
  "errors"
  "reflect"
  "strings"
)

func JSONFields(value any) ([]string, error) {
  if value == nil {
    return nil, errors.New("значение не должно быть nil")
  }

  reflected := reflect.ValueOf(value)
  typ := reflected.Type()
  if typ.Kind() == reflect.Ptr {
    if reflected.IsNil() {
      return nil, errors.New("указатель не должен быть nil")
    }
    typ = typ.Elem()
  }
  if typ.Kind() != reflect.Struct {
    return nil, errors.New("ожидалась структура или указатель на структуру")
  }

  fields := make([]string, 0, typ.NumField())
  for index := 0; index < typ.NumField(); index++ {
    field := typ.Field(index)
    if field.PkgPath != "" {
      continue
    }
```

▼ Сведения

```
    tag, exists := field.Tag.Lookup("json")   if exists && tag == "-" {
continue   }   name, _, _ := strings.Cut(tag, ",")    if name == "" {       name =
field.Name   }   fields = append(fields, name)  } return fields, nil}
```

Этот helper не повторяет все правила `encoding/json`: embedded fields, conflicts, custom marshalers и другие случаи требуют отдельного контракта. Не используйте его как замену JSON encoder.

Задание 2. Reflection setter с явным контрактом

Напишите:

```
func SetExportedString(target any, fieldName, value string) error
```

Функция меняет только экспортируемое строковое поле у non-nil pointer на struct. Для отсутствующего поля, private поля, поля другого типа, неуказателя и typed nil верните error без panic.

Критерии проверки:

- проверяются `IsValid`, `Kind`, `IsNil` и `CanSet` до `SetString`;
- helper не использует `unsafe` для private field;
- в unit-тесте успешный вызов меняет исходную структуру;
- ошибки проверяются для `Task{}` вместо `&Task{}`, поля `Priority`, несуществующего и private поля;
- README/комментарий объясняет, почему Task Manager не использует функцию для обычного обновления title.

Подсказка

```
field := value.Elem().FieldByName(fieldName)
if !field.IsValid() || field.Kind() != reflect.String || !field.CanSet() {
  return errors.New("поле недоступно для изменения")
}
```

Решение

```
func SetExportedString(target any, fieldName, newValue string) error {
  value := reflect.ValueOf(target)
  if !value.IsValid() || value.Kind() != reflect.Ptr || value.IsNil() {
    return errors.New("target должен быть non-nil указателем на структуру")
  }

  value = value.Elem()
  if value.Kind() != reflect.Struct {
    return errors.New("target должен указывать на структуру")
  }

  field := value.FieldByName(fieldName)
  if !field.IsValid() || field.Kind() != reflect.String || !field.CanSet() {
    return fmt.Errorf("поле %q недоступно для изменения", fieldName)
  }

  field.SetString(newValue)
  return nil
}
```

Для решения нужны импорты `errors`, `fmt` и `reflect`. В прикладном коде предпочтительнее `task.Title = newValue`: его контракт проверяется компилятором.

Задание 3. Бинарный формат без `unsafe`

Нужно передать заголовок протокола: `version uint16`, `flags uint8`, `length uint32` в network byte order. Напишите `EncodeHeader` и `DecodeHeader` через `encoding/binary`, без преобразования Go struct к `[]byte` через `unsafe`.

Критерии проверки:

- wire format всегда ровно 7 байт;
- используется `binary.BigEndian`;
- `DecodeHeader` проверяет длину до slice expressions;
- round-trip test подтверждает все три поля;
- test с коротким input возвращает error, а не panic;
- комментарий объясняет, почему `unsafe.Sizeof(Header{})` не определяет network format.

Подсказка

```
data := make([]byte, 7)
binary.BigEndian.PutUint16(data[0:2], version)
data[2] = flags
binary.BigEndian.PutUint32(data[3:7], length)
```

Решение

```
package protocol

import (
  "encoding/binary"
  "errors"
)

const headerSize = 7

type Header struct {
  Version uint16
  Flags   uint8
  Length  uint32
}

func EncodeHeader(header Header) []byte {
  data := make([]byte, headerSize)
  binary.BigEndian.PutUint16(data[0:2], header.Version)
  data[2] = header.Flags
  binary.BigEndian.PutUint32(data[3:7], header.Length)
  return data
}

func DecodeHeader(data []byte) (Header, error) {
  if len(data) < headerSize {
    return Header{}, errors.New("недостаточно байтов для заголовка")
  }

  return Header{
    Version: binary.BigEndian.Uint16(data[0:2]),
    Flags:   data[2],
    Length:  binary.BigEndian.Uint32(data[3:7]),
```

▼ Сведения

```
    }, nil}
```

`unsafe.Sizeof(Header{})` зависит от padding и выравнивания на архитектуре; network protocol должен задаваться явно, поэтому wire format здесь фиксирован константой `headerSize`.

Задание 4. Проверьте low-level пакет

Создайте небольшой изолированный пакет с одним допустимым применением `unsafe.Sizeof` или `unsafe.Offsetof` для диагностического теста layout. Добавьте комментарий о target architecture и о том, что значение не используется как внешний формат. Запустите:

```
go vet ./...
go test -gcflags=all=-d=checkptr=2 ./internal/lowlevel
go test -race ./internal/lowlevel
```

Критерии проверки:

- пакет не содержит преобразование `uintptr` с хранением адреса для будущего использования;
- нет доступа к private полям другого пакета;
- нет zero-copy `string` / `[]byte` helper без задокументированного ownership;
- тесты проходят с `checkptr` и race detector;
- проверка layout не заявляет portability без фактических target OS/arch тестов;
- в review описано, почему обычный Go-код не решал именно эту узкую задачу.

Направление решения

В большинстве прикладных проектов верный ответ — не создавать такой пакет вовсе. Упражнение нужно, чтобы отличать относительно безопасную compile-time диагностику layout от указательной арифметики и обхода GC. Если нет конкретной платформенной интеграции, зафиксируйте в отчёте отказ от `unsafe`.

Задание 5. Тонкая cgo-обёртка или обоснованный отказ

Если есть C compiler, реализуйте `CLength(value string) int` из теории в изолированном package `internal/native`. Добавьте тесты для пустой строки и UTF-8 текста. Запустите integration-команду:

```
CGO_ENABLED=1 go test ./internal/native
```

Если C compiler нет или у проекта нет реальной native-зависимости, вместо кода подготовьте design note с отказом от cgo: укажите, что `len(value)` решает учебный пример без native toolchain, C heap ownership и platform-specific сборки.

Критерии проверки для cgo-варианта:

- C preamble находится непосредственно перед `import "C"`;
- `C.CString` освобождается через `C.free` на каждом пути;
- Go API возвращает `int`, а не `*C.char` или другой `C.*` тип;
- вызов изолирован от domain/service/handler пакетов;
- README описывает нужный compiler, target OS/arch и команду CI/Docker build;
- тесты не предполагают, что каждый символ UTF-8 занимает один byte: C `strlen` и Go `len` считают байты.

Решение

```
package native

/*
#include <stdlib.h>
#include <string.h>
*/
import "C"

import "unsafe"

func CLength(value string) int {
  cValue := C.CString(value)
  defer C.free(unsafe.Pointer(cValue))

  return int(C.strlen(cValue))
}
```

```
func TestCLength(t *testing.T) {
  for _, value := range []string{"", "Go", "Привет"} {
    if got, want := CLength(value), len(value); got != want {
      t.Fatalf("CLength(%q) = %d, want %d", value, got, want)
    }
  }
}
```

Этот пример демонстрационный. В production `len(value)` всегда предпочтительнее: cgo не даёт здесь ни функции, ни performance-выгоды.

Проверьте себя

Перед следующей главой убедитесь, что вы можете:

- выбрать явный код, интерфейс, generic-функцию или generation вместо runtime reflection, когда тип известен;
- проверить `Kind`, `IsValid`, `IsNil` и `CanSet` до потенциально опасных reflection-операций;
- объяснить, почему tags — metadata, а не бесплатная реализация JSON/ORM/validation;
- не использовать `unsafe` для network format, private fields или сомнительного zero-copy;
- объяснить разницу Go heap и C heap и ownership C allocation;
- изолировать cgo boundary и обосновать native-зависимость в CI/Docker/production;
- признать отказ от reflection, `unsafe` или cgo корректным инженерным решением.

Рекомендации по коду: reflection, `unsafe` и cgo

Выбор инструмента

- Начинайте с конкретных типов и прямого доступа к полям; затем рассмотрите маленький интерфейс, generics или code generation.
- Используйте reflection только на реальной dynamic boundary: serializer, metadata framework, ORM integration, schema tooling или plugin contract с открытым набором типов.
- Не заменяйте пару известных вариантов reflection-кодом: type switch или явная функция лучше проверяются и рефакторятся.
- Изолируйте reflection, `unsafe` и cgo в маленьком пакете. Не распространяйте их типы и детали в handler/service/domain API.

Reflection

- До `Elem`, `NumField`, `IsNil`, `Set` и `Call` проверяйте `IsValid`, `Kind`, nil и `CanSet` в соответствии с операцией.
- Считайте строковые имена полей и методов частью dynamic contract: документируйте и тестируйте их, потому что rename не проверяется компилятором.
- Уважайте экспортируемость полей. Не обходите private state чужого пакета через unsafe.
- Разбирайте tags полностью по контракту либо используйте зрелую библиотеку; `StructTag.Get` не реализует правила JSON, ORM или validation сам по себе.
- Кэшируйте reflection metadata только после profile и с продуманными concurrency/size semantics.
- Возвращайте обычный `error` для неверной формы входа; не используйте `recover` как штатную validation strategy.

`unsafe`

- Рассматривайте `unsafe` как исключение с измеренной/платформенной причиной, а не как оптимизацию «на всякий случай».
- Не храните pointer как `uintptr` между выражениями, не делайте pointer arithmetic в application code и не предполагайте layout чужих типов.
- Не используйте Go struct как wire format: фиксируйте endianness, размеры и bounds через `encoding/binary` или protocol library.
- Не делайте unsafe zero-copy между `string` и `[]byte`, пока не доказаны необходимость, ownership и lifetime; обычные преобразования безопаснее.
- Проверяйте изолированный low-level код через `go vet`, `-d=checkptr=2`, race tests и target OS/arch CI.
- Документируйте lifetime, layout, архитектурные ограничения, benchmark и владельца поддержки рядом с каждым unsafe-блоком.

cgo

- Выбирайте cgo только при реальной ценности существующей C-библиотеки, которую не даёт standard library или поддерживаемый pure-Go вариант.
- Держите `import "C"` и C types в тонком boundary package; наружу возвращайте idiomatic Go values и `error`.
- Для каждого C allocation сразу определяйте правильную C cleanup-функцию и вызывайте её на всех путях.
- C не должен сохранять Go pointer после вызова без документированного handle/callback design; проверяйте размеры и nil до передачи буферов.
- Не блокируйте длительные C-вызовы под mutex и не считайте context автоматической отменой native call.
- Воспроизводите C compiler, headers, linker flags и native libraries в CI и multi-stage Docker build; тестируйте target OS/arch.
- Проверяйте security updates, license, timeout/cancellation semantics и план обновления native dependency до merge.

Итоговый чек-лист

- [ ] Обычные типы, интерфейсы, generics и generation оценены до low-level решения.
- [ ] Dynamic/low-level код локализован, документирован и покрыт отрицательными тестами.
- [ ] Reflection не может panic на невалидном/неподходящем input.
- [ ] Нет unsafe зависимости от private layout, внешнего binary format или долгоживущего `uintptr`.
- [ ] C и Go ownership памяти разделены, C resources освобождаются на всех путях.
- [ ] Native build и tests воспроизводимы в CI/Docker на production-like target.
- [ ] Внешний HTTP API, ошибки, секреты и domain-слой не зависят от деталей reflection/unsafe/cgo.

Продвинутый Go F: архитектура production-сервиса

Цель главы

Научиться принимать архитектурные решения для production-сервиса: удерживать направление зависимостей, вводить интерфейсы на реальных границах, управлять конфигурацией и секретами, строить наблюдаемость, readiness/liveness, retries, idempotency, rate limiting и защиту HTTP API.

Task Manager API после основного маршрута уже имеет Gin, PostgreSQL, Docker, CI/CD, request ID и graceful shutdown. Эта глава не предлагает переписать его под «идеальную архитектуру». Она помогает развивать сервис по мере появления настоящих требований — внешних клиентов, нескольких экземпляров API, фоновых задач, SLA, атакующего трафика и изменений команды.

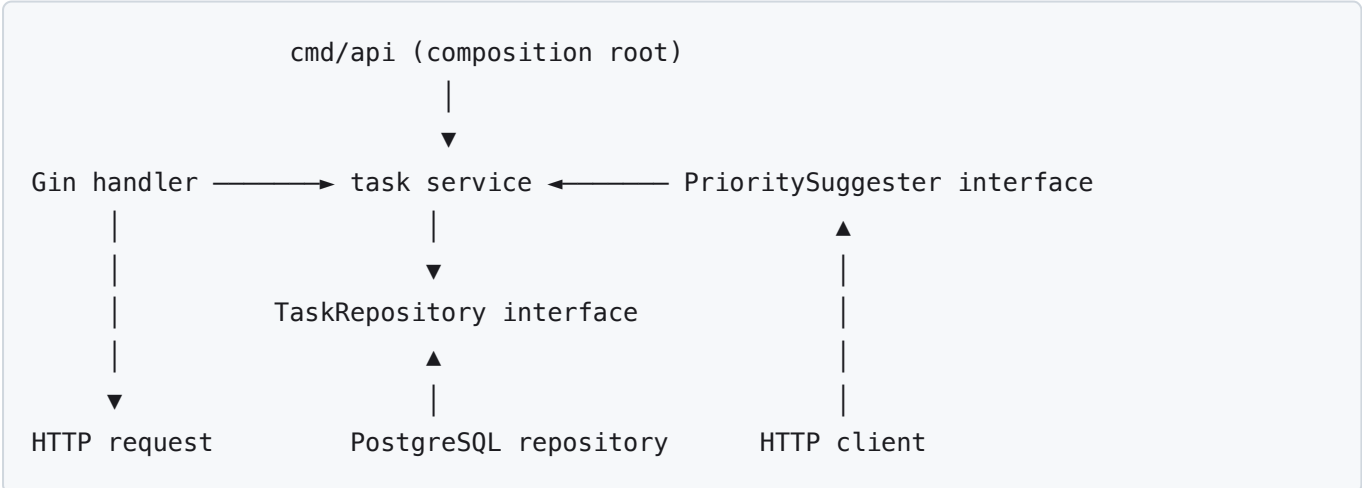
Карта главы

1. **Эта страница** — зависимости, слои и интерфейсы без overengineering.
2. Конфигурация, секреты и health-checks — startup validation, liveness, readiness и lifecycle.
3. Observability: логи, метрики и tracing — три сигнала и безопасный контекст операций.
4. Timeouts, retries, idempotency и rate limiting — надёжное поведение при сбоях и повторной доставке.
5. Безопасность API и code review — аутентификация, авторизация, input limits, HTTP hardening и checklist.

Архитектура — это направление зависимостей

Слои нужны не ради количества папок. Они отвечают на вопрос: какой код знает о деталях инфраструктуры, а какой — только о правилах предметной области.

Для Task Manager полезна следующая зависимость:



Стрелка означает «знает тип и вызывает его». HTTP handler может знать Gin; PostgreSQL-repository может знать `database/sql`; Resty-клиент может знать Resty. Но `taskservice` не должен импортировать Gin, `database/sql`, pgx, Resty, переменные окружения или конкретный logger transport.

Responsibilities важнее названий папок

Одна из понятных структур:

```
task-manager-api/
├── cmd/api/main.go                # composition root и lifecycle процесса
├── internal/
│   ├── task/                      # Task, TaskID, CreateInput, доменные ошибки
│   ├── taskservice/               # сценарии Create, Complete, List
│   ├── postgresrepo/              # SQL и *sql.DB
│   ├── priorityclient/            # HTTP-клиент внешнего сервиса
│   ├── ginapi/                    # router, handlers, HTTP DTO, middleware
│   ├── config/                    # Config, загрузка и startup validation
│   └── observability/             # создание logger/meter/tracer при необходимости
├── migrations/
└── compose.yaml
```

Это пример, не догма. Не создавайте package на каждую структуру или один огромный `internal/common`. Пакет должен иметь связную причину существования: доменные правила задач, PostgreSQL persistence, HTTP transport или сборка приложения.

Domain не должен зависеть от delivery

Модель и бизнес-правила не должны знать, пришёл ли запрос по HTTP, из CLI, очереди или cron-job:

```
package task

type CreateInput struct {
  Title    string
  Priority Priority
}

func (input CreateInput) Validate() error {
  if strings.TrimSpace(input.Title) == "" {
    return &ValidationError{Field: "title", Rule: "required"}
  }
  if !input.Priority.Valid() {
    return &ValidationError{Field: "priority", Rule: "range"}
```

```
  } return nil}
```

Gin handler отвечает за HTTP-специфику: прочитать JSON, ограничить body, распарсить path, выбрать status и сериализовать response. Service отвечает за use case: проверить правило, сохранить задачу, при необходимости запросить подсказку приоритета. Repository отвечает за persistence contract. Такое разделение даёт быстрые unit-тесты service и отдельные integration-тесты SQL.

Интерфейс объявляет потребитель

Интерфейс полезен не потому, что «всё нужно mock-ать», а потому, что потребителю нужна небольшая возможность. Service может определить её рядом с use case:

```
package taskservice

type Repository interface {
  Create(ctx context.Context, input task.CreateInput) (task.Task, error)
  Get(ctx context.Context, id task.TaskID) (task.Task, error)
  Complete(ctx context.Context, id task.TaskID) (task.Task, error)
}

type PrioritySuggester interface {
  Suggest(ctx context.Context, title string) (task.Priority, error)
}

type Service struct {
  repository Repository
  priority   PrioritySuggester
}
```

`postgresrepo.Repository` реализует `Repository` неявно, а `priorityclient.Client` — `PrioritySuggester`. При этом service не зависит от пакетов с реализациями. Тест передаёт маленький fake, который моделирует только нужное поведение.

```
type fakeRepository struct {
  create func(context.Context, task.CreateInput) (task.Task, error)
}

func (fake *fakeRepository) Create(ctx context.Context, input task.CreateInput)
(task.Task, error) {
  return fake.create(ctx, input)
}
```

Не обязательно создавать интерфейс заранее

Если service использует только конкретную чистую функцию или value object, интерфейс не нужен. Если существует одна реализация и нет тестовой границы, сначала оставьте конкретный тип. Интерфейс должен выражать потребность потребителя, а не предсказывать все возможные будущие реализации.

Тонкий handler, конкретный service, изолированный repository

Хороший handler коротко переводит transport data в use case:

```
func (api *API) createTask(c *gin.Context) {
  var request createTaskRequest
  if err := c.ShouldBindJSON(&request); err != nil {
    writeError(c, http.StatusBadRequest, "invalid_request", "Некорректный JSON")
    return
  }

  created, err := api.service.Create(c.Request.Context(), task.CreateInput{
    Title:    request.Title,
    Priority: task.Priority(request.Priority),
  })
  if err != nil {
    api.writeServiceError(c, err)
    return
  }

  c.JSON(http.StatusCreated, created)
}
```

Что **не** должен делать handler:

- собирать SQL и вызывать `db.QueryContext`;
- читать env на каждом запросе;
- запускать goroutine без ownership и shutdown contract;
- превращать `err.Error()` в публичный JSON;
- передавать `*gin.Context` в service/repository.

Что **не** должен делать service:

- выбирать HTTP status или создавать `gin.H`;
- открывать подключение к БД на каждый метод;
- читать токен из `os.Getenv`;
- скрывать бизнес-правило за общим CRUD, если операция имеет отдельную семантику.

Repository не обязан быть «тупым»: он переводит ожидаемые технические причины (`sql.ErrNoRows`) в domain errors, использует context и защищает SQL parameters. Но authorization обычно не прячут в случайных SQL predicate без явного уровня policy: сервису и review должно быть видно, кто имеет право завершить задачу.

Composition root: `main` собирает graph зависимостей

Только entrypoint знает конкретные реализации:

```
func run(ctx context.Context) error {
  config, err := config.Load()
  if err != nil {
    return fmt.Errorf("загрузить конфигурацию: %w", err)
  }

  logger := observability.NewLogger(config.LogLevel)
  database, err := postgresrepo.Open(ctx, config.DatabaseURL)
  if err != nil {
    return fmt.Errorf("подключить PostgreSQL: %w", err)
  }

  repository := postgresrepo.New(database)
  priorityClient := priorityclient.New(config.PriorityServiceURL,
config.HTTPClientTimeout)
  service := taskservice.New(repository, priorityClient)
  api := ginapi.New(service, logger)

  server := newServer(config.HTTPAddress, api.Router())
  return serve(ctx, server, database, logger)
}
```

Здесь допустимо видеть все packages: это composition root. Внутренние слои остаются независимыми. Такая «ручная dependency injection» проста, проверяется компилятором и обычно лучше контейнера, который собирает зависимости reflection-магией.

Границы бывают и у данных

Не передавайте database/HTTP модели прямо наружу:

```
// Внутренняя доменная модель.
type Task struct {
  ID        TaskID
  Title     string
  Priority  Priority
```

```
  Completed bool  CreatedAt time.Time}// HTTP response может развиваться
отдельно.type taskResponse struct {   ID        int64     `json:"id"`   Title
string    `json:"title"`   Priority  int       `json:"priority"` Completed bool
`json:"completed"`  CreatedAt time.Time `json:"created_at"`}
```

Явный mapping кажется лишним для маленького API, но защищает от случайной публикации нового поля, изменения внутреннего типа или утечки metadata БД. Создавайте отдельные DTO там, где внешний contract действительно отличается. Не копируйте структуры механически, если модель уже является сознательно спроектированным публичным contract.

Как добавлять новую функцию

Новая возможность проходит вопросы по слоям:

1. **Contract:** какой HTTP/CLI/queue вход и результат? Кто может выполнить действие?
2. **Domain:** какие инварианты и errors появляются?
3. **Use case:** какие зависимости нужны, какие side effects допустимы, нужна ли transaction?
4. **Persistence/integration:** какая миграция, SQL, внешний API, timeout и retry policy?
5. **Operations:** какие logs, metrics, traces, health/readiness изменения и dashboards/alerts потребуются?
6. **Delivery:** совместима ли миграция с rollback, есть ли tests, config и deploy plan?

Архитектура production-сервиса — это способность отвечать на эти вопросы перед тем, как изменение попадёт в `main` и production. Следующая страница продолжит с конфигурации и состояния готовности процесса.

Конфигурация, секреты и health-checks

Конфигурация — входной контракт процесса

Конфигурация не должна быть набором разрозненных `os.Getenv` в handlers. Она приходит на границе процесса, разбирается один раз, валидируется до открытия listener и передаётся зависимостям как типизированные значения.

```
type Config struct {
  HTTPAddress         string
  DatabaseURL         string
  PriorityServiceURL  string
  HTTPClientTimeout   time.Duration
  ShutdownTimeout     time.Duration
  LogLevel            slog.Level
  ReadinessTimeout    time.Duration
}
```

Ошибочная конфигурация должна завершить startup с ясной ошибкой. Лучше не поднять pod с неверным DSN, чем принять первый запрос и ответить `500` из глубины repository.

Разделяйте config и secrets

Не каждое env-значение секретно. Полезно разделить данные по риску:

Категория	Примеры	Можно ли безопасно логировать
Обычная config	HTTP address, log level, feature flag name	обычно да, после проверки
Sensitive config	внутренний URL, tenant identifier	зависит от инфраструктуры
Secret	password, API token, private key, signing secret	нет

`DATABASE_URL` почти всегда содержит пароль, поэтому не логируйте целиком `Config` и не печатайте подставленный Compose config в публичные CI logs. Логируйте лишь безопасную часть: например, `db_host` после аккуратного разбора и только если это допускает модель угроз.

```
logger.Info(
  "конфигурация загружена",
  "http_address", config.HTTPAddress,
  "log_level", config.LogLevel.String(),
  // Без DatabaseURL и tokens.
)
```

Источники и приоритет

Для 12-factor приложения production source of truth — environment или secret manager платформы. Локальный `.env` удобен только для разработки:

```
production env / secret manager  → наивысший приоритет
локальный .env                   → только fallback для разработки
кодовые defaults                 → только для безопасных необязательных настроек
```

Не задавайте default для обязательного production secret. Пустой token или случайный development password должны приводить к понятной startup error, а не к небезопасному режиму. `.env.example` содержит имена и безопасные placeholders, но не рабочие production credentials.

Ротация secrets

Секрет однажды будет заменён: сотрудник уходит, token скомпрометирован, срок действия сертификата истекает. Поэтому заранее определите:

- где хранится source of truth (GitHub Actions Secrets, VPS secret file, cloud secret manager);
- кто имеет доступ и как он аудируется;

- как новый secret попадает в окружение;
- нужен ли restart/rollout для его применения;
- как отозвать старый secret и проверить отсутствие его в logs/history.

Не пытайтесь «горячо» менять env внутри процесса без чёткого design. Большинство сервисов безопаснее перезапустить rolling deployment после обновления секрета. Если требуется zero-downtime rotation ключей, API часто должен временно принимать два ключа (current + previous) с явным сроком жизни и audit trail.

Liveness, readiness и startup — разные состояния

Один `/healthz` не всегда должен отвечать на все operational вопросы.

Probe	Вопрос платформы	Что проверять	Типичная реакция на failure
Startup	«Процесс успел инициализироваться?»	миграции/обязательная initialisation, если они блокируют старт	подождать или не направлять трафик
Liveness	«Процесс жив и способен обрабатывать event loop?»	лёгкая проверка процесса, без внешней сети/БД	перезапустить застрявший процесс
Readiness	«Можно ли направлять новый трафик?»	критические локальные зависимости и состояние draining	убрать instance из balancing, не обязательно рестартовать

Называть endpoints можно по-разному: `/livez`, `/readyz`, `/healthz`. Важнее документировать contract для вашего orchestrator. Не добавляйте external HTTP API в liveness: временная авария соседнего сервиса не должна заставлять platform перезапускать все экземпляры Task Manager одновременно.

Liveness: дешёвая проверка процесса

```
func (api *API) live(c *gin.Context) {
  c.JSON(http.StatusOK, gin.H{"status": "ok"})
}
```

Liveness должен быть быстрым, не требовать auth, не читать БД и не выполнять долгие операции. Если process deadlock или event loop не отвечает, request всё равно не дойдёт до handler — platform увидит timeout. Не превращайте liveness в глубокий dependency audit.

Readiness: можно ли принимать новый трафик

Readiness может учитывать состояние shutdown и критическую БД. Дайте probe маленький собственный timeout, чтобы он не занял все connections pool:

```
type ReadinessChecker interface {
  PingContext(context.Context) error
}

type API struct {
  database ReadinessChecker
  draining atomic.Bool
}

func (api *API) ready(c *gin.Context) {
  if api.draining.Load() {
    c.JSON(http.StatusServiceUnavailable, gin.H{"status": "draining"})
    return
  }

  ctx, cancel := context.WithTimeout(c.Request.Context(), time.Second)
  defer cancel()

  if err := api.database.PingContext(ctx); err != nil {
    api.logger.Warn("readiness PostgreSQL не пройдена", "err", err)
    c.JSON(http.StatusServiceUnavailable, gin.H{"status": "not_ready"})
    return
  }

  c.JSON(http.StatusOK, gin.H{"status": "ready"})
}
```

`PingContext` на каждом readiness request может быть лишней нагрузкой в конкретной платформе. Альтернативы: периодический background check с cached state, отдельный pool или readiness, основанная только на собственном состоянии процесса. Выберите policy по SLA и поведению orchestrator; не копируйте probe механически.

> **Не раскрывайте internals probe-ответом**
>
> Клиенту и публичному интернету не нужны DSN, имена таблиц, полные ошибки PostgreSQL или состояние всех internal dependencies. Status + короткий code достаточно; полная причина остаётся в защищённом логе/метрике.

Readiness во время graceful shutdown

При `SIGTERM` сервис должен перестать принимать новый трафик до того, как истечёт shutdown deadline. Один из вариантов:

1. выставить `draining = true` — `/readyz` начинает возвращать `503`;
2. дать load balancer время увидеть состояние и перестать направлять новые requests;
3. вызвать `server.Shutdown(shutdownCtx)`;
4. дождаться handler и worker;
5. закрыть DB и остальные dependencies.

Точный grace period зависит от Kubernetes, reverse proxy и deployment platform. Не заменяйте этот протокол `time.Sleep`: платформа должна иметь documented readiness semantics, а сервер — ограниченный `Shutdown` context.

Feature flags и динамическая config

Feature flag — тоже configuration contract. Для каждого flag определите:

- владелец и бизнес-цель;
- default и поведение при недоступности flag service;
- scope (global, tenant, user, request);
- observability: как видно, какая ветка сработала;
- дата удаления flag и старого кода.

Не используйте flags, чтобы скрыть отсутствие миграционного или authorization design. Они полезны для постепенного включения, rollback и эксперимента, но превращаются в долг при неограниченном накоплении.

Конфигурация и deploy

Config shape — часть delivery. Изменение обязательной env-переменной требует в одном PR:

- parser + validation;
- `.env.example` и README;
- Compose/Helm/Terraform/secret manager config;
- CI/deploy environment;
- migration/rollback plan, если поведение меняет данные;
- tests для missing/invalid/valid configuration.

Следующая страница добавит observability: как после успешного запуска понять, что service действительно здоров, быстр и правильно обрабатывает трафик.

Observability: логи, метрики и tracing

Observability — способность понять состояние системы по внешним сигналам без входа на сервер и догадок. Она не равна «писать больше логов». Для production обычно сочетают три сигнала:

```
logs    → что случилось с конкретной операцией?
metrics → насколько часто и насколько плохо это происходит?
```

```
traces  → где проходит время одного запроса через несколько компонентов?
```

Логи: контекст расследования

Структурированный `slog` уже даёт фундамент. В production лог на границе ошибки должен содержать безопасный operational context:

```
logger.Error(
  "создание задачи не выполнено",
  "request_id", requestID,
  "method", request.Method,
  "route", "/tasks",
  "task_id", taskID,
  "err", err,
)
```

Выбирайте стабильные ключи: `request_id`, `trace_id`, `method`, `route`, `status`, `duration`, `error_code`. Для router metrics/logs предпочтительнее route template (`/tasks/:id`), а не raw path `/tasks/918273`: так не растёт cardinality и легче агрегировать запросы.

Не записывайте в лог по умолчанию:

- `Authorization`, cookies, password, token, DSN;
- полное request/response body;
- персональные данные без документированной цели и retention policy;
- неограниченные строки от клиента как label/ключ metric.

Метрики: состояние системы во времени

Метрика — числовой ряд с ограниченным набором labels. В Task Manager полезны RED-сигналы для HTTP:

Сигнал	Пример	Зачем
Rate	число requests	замечать трафик и падение нагрузки
Errors	`5xx` / failure count	видеть ошибки, которые требуют действия
Duration	histogram latency	видеть latency и p50/p95/p99

Дополните их ресурсами и зависимостями: in-flight requests, DB pool stats, число retry, rate-limit rejects, queue depth, worker count. Названия и exact instrumentation зависят от выбранной системы (Prometheus/OpenTelemetry/managed platform), но контракт важнее SDK.

Cardinality: самая частая ловушка

Каждая уникальная комбинация labels образует отдельный time series. Нельзя использовать как labels:

```
task_id=123
request_id=...
user_email=...
raw_path=/tasks/123
error_message="..."
```

Это создаёт практически неограниченную cardinality, дорогую память и медленные queries. Вместо этого:

```
http_requests_total{method="POST",route="/tasks",status="201"}
http_request_duration_seconds{method="POST",route="/tasks"}
task_service_errors_total{operation="create",code="validation_failed"}
```

Конкретный ID остаётся в log/trace, а aggregate metric хранит только конечный известный набор значений.

Измеряйте семантику, а не реализацию

Хорошая метрика описывает user-visible или operational событие:

```
task_create_total{outcome="success"}
task_create_total{outcome="validation_failed"}
priority_suggest_requests_total{outcome="timeout"}
```

Менее полезно делать metric на каждый private helper или на каждую строку кода. Она сломается при рефакторинге и не отвечает на вопрос оператора. Введите metrics там, где есть contract: endpoint, use case, внешний dependency, background worker или rate limiter.

Tracing: путь одного запроса

Distributed trace объединяет spans одной операции. HTTP server создаёт root span; service, SQL и HTTP client создают дочерние:

```
POST /tasks [trace_id=abc]
├── taskservice.Create
│   ├── priorityclient.Suggest
│   └── postgresrepo.Create
└── response 201
```

Trace полезен, когда latency проходит через несколько сервисов. Он отвечает: «в каком downstream и на каком шаге потеряны 800 ms?» Лог обычно отвечает «какая ошибка у request ID?», а metric — «как часто p95 выше SLO?»

Если используете OpenTelemetry, извлекайте trace context из входного HTTP request и передавайте `c.Request.Context()` вниз. Не создавайте новый `context.Background()` в service/client: это разорвёт cancellation и trace propagation.

```
func (service *Service) Create(ctx context.Context, input task.CreateInput)
(task.Task, error) {
  ctx, span := service.tracer.Start(ctx, "taskservice.Create")
  defer span.End()

  created, err := service.repository.Create(ctx, input)
  if err != nil {
    span.RecordError(err)
    return task.Task{}, err
  }

  return created, nil
}
```

Не добавляйте title, token, SQL DSN и личные данные в span attributes. Trace — не частный debug buffer: он может экспортироваться внешнему vendor, sampled частично и храниться дольше одного запроса.

Request ID и trace ID

Request ID остаётся полезным и без tracing. Он может быть заголовком API (`X-Request-ID`) и связывать access log/response. Trace ID формируется/распространяется tracing system и связывает spans между сервисами. Они могут сосуществовать:

```
request_id → поиск одного HTTP-обмена в собственных logs
trace_id   → путь операции через API, worker и downstream services
```

Не делайте request ID единственным metric label. В лог удобно добавить оба идентификатора безопасными полями.

Ошибки, уровни и sampling

Не каждый `4xx` — server error. Validation failure может быть ожидаемым исходом и попадать в metric с `outcome="validation_failed"`, но не создавать alert и не засорять `ERROR` level. `5xx`, dependency timeout и panic требуют отдельной observability.

Трафик может сделать logs/traces слишком дорогими. Возможные политики:

- сохранять все errors и медленные requests;
- sampling успешных traces;

- ограничивать объём body и число attributes;
- redaction до отправки события.

Sampling нельзя использовать, чтобы скрыть security/audit requirements: такие события должны иметь отдельный, специально спроектированный безопасный audit pipeline.

От метрик к SLO и alert

Alert должен быть действием, а не уведомлением «что-то есть». Вместо alert на каждый единичный `500` подумайте о user-impact window:

```
Условие: 5xx rate выше 2% в течение 10 минут
Контекст: dashboard с request rate, route, dependency errors и deploy version
Действие: проверить recent deploy, PostgreSQL readiness, priority-service и traces
```

SLO и error budget зависят от продукта. Для учебного Task Manager не нужно имитировать enterprise monitoring. Но даже маленький сервис выигрывает от понятного dashboard: request rate, error rate, latency, version, readiness и DB pool saturation.

Тестируйте observability contract

Не проверяйте в каждом unit-тесте полный текст лога. Проверяйте бизнес-результат. Отдельно тестируйте то, что действительно является contract:

- middleware добавляет request ID в response;
- `5xx` инкрементирует ожидаемый bounded error code;
- metric не содержит task ID/request ID;
- readiness не раскрывает внутреннюю ошибку;
- trace context передаётся в HTTP/SQL client, если tracing включён.

Следующая страница разберёт поведение в момент реальных сбоев: timeout, retry, повторная доставка запроса и злоупотребление API.

Timeouts, retries, idempotency и rate limiting

Production-сервис живёт в частично ненадёжной среде: сеть задерживает пакеты, dependency может вернуть `503`, клиент повторит request после timeout, а бот отправит тысячи запросов. Надёжность — не «повторить всё три раза», а явный контракт для каждой операции.

Timeouts: бюджет, а не случайное число

У HTTP request есть конечный latency budget. Если API обещает ответ за 2 секунды, нельзя дать database 2 секунды, внешнему client ещё 2 секунды и retry ещё 2 секунды. Дедлайны должны укладываться друг в друга:

```
входной request: 2 s
├── validation:      почти мгновенно
├── PostgreSQL:       до 300 ms
└── priority service: до 500 ms, включая возможный retry
```

Передавайте `c.Request.Context()` в service, `QueryContext` и `http.NewRequestWithContext`. Если отдельный downstream budget нужен, создавайте дочерний context с меньшим timeout:

```
func (client *Client) Suggest(ctx context.Context, title string) (task.Priority,
error) {
  ctx, cancel := context.WithTimeout(ctx, 500*time.Millisecond)
  defer cancel()

  request, err := http.NewRequestWithContext(ctx, http.MethodGet, client.endpoint,
nil)
  if err != nil {
    return 0, fmt.Errorf("создать запрос приоритета: %w", err)
  }

  response, err := client.httpClient.Do(request)
  if err != nil {
    return 0, fmt.Errorf("запросить приоритет: %w", err)
  }
  defer response.Body.Close()

  // Проверка статуса и decode response.
  return 0, nil
}
```

Не создавайте `context.Background()` внутри client, иначе внешний вызов переживёт отмену клиента и shutdown. Не подменяйте timeout бесконечным retry loop.

Retry: только при обоснованной временной ошибке

Retry имеет цену: он повышает latency, умножает нагрузку на уже повреждённую dependency и может создать дубликат side effect. Перед retry ответьте:

1. Операция безопасна для повторения?
2. Ошибка действительно временная: network timeout, `429`, `502`, `503`, `504`?
3. Есть ли оставшийся deadline budget?
4. Есть ли лимит attempts, backoff и jitter?
5. Наблюдаемы ли attempts и итоговый outcome?

Для простого idempotent `GET` допустима контролируемая политика:

```
func retryableStatus(status int) bool {
  return status == http.StatusTooManyRequests ||
    status == http.StatusBadGateway ||
    status == http.StatusServiceUnavailable ||
    status == http.StatusGatewayTimeout
}

func backoff(attempt int) time.Duration {
  base := 50 * time.Millisecond
  maximum := time.Second
  delay := base * (1 << attempt)
  if delay > maximum {
    delay = maximum
  }
  // Для нескольких клиентов добавьте bounded random jitter.
  return delay
}
```

Production реализация должна ждать через timer/select с `ctx.Done()`, уважать `Retry-After` при `429` и не делать retry после deadline. Не повторяйте автоматически `POST /tasks`, если server мог создать задачу, но response потерялся: сначала нужна idempotency semantics.

> **Retry не лечит постоянную проблему**
>
> Retry полезен для короткого transient failure. При долгой аварии нужны timeout, circuit breaker/ограничение нагрузки, fallback или явная ошибка клиенту. Бесконечные retries превращают outage в очереди goroutine, connection exhaustion и cascade failure.

Idempotency: повтор не должен менять результат повторно

Операция идемпотентна, если несколько одинаковых применений дают тот же конечный эффект, что и одно. `GET` обычно идемпотентен. `PUT` / `DELETE` могут быть идемпотентны по contract. `POST` создания — нет автоматически.

Клиент может не получить response из-за network failure и отправить тот же create request повторно. Без защиты API создаст две задачи.

Idempotency key для создания

Один распространённый protocol:

1. client генерирует случайный `Idempotency-Key` для логической операции;
2. API валидирует format/length ключа и привязывает его к authenticated actor;

3. в transaction сохраняет key, request fingerprint и итог response/status;
4. повтор с тем же key и тем же request возвращает первоначальный response;
5. тот же key с другим request возвращает `409 Conflict`;
6. record имеет TTL/cleanup policy.

Схематичная таблица:

```
CREATE TABLE idempotency_keys (
    actor_id BIGINT NOT NULL,
    key TEXT NOT NULL,
    request_hash BYTEA NOT NULL,
    status_code SMALLINT NOT NULL,
    response_body JSONB NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    PRIMARY KEY (actor_id, key)
);
```

Главное — атомарность с созданием Task. Если сначала создать задачу, а key сохранить после отдельным запросом, crash между операциями снова даст дубликат. Используйте одну transaction, правильный unique constraint и продумайте поведение concurrent duplicate requests.

Не храните в key table необработанные credentials или неограниченное body. Hash request и ограниченный JSON response уменьшают риск, но retention и PII policy всё равно обязательны.

Idempotency не равна exactly-once

В распределённой системе «ровно один раз» редко гарантируется автоматически. Idempotency key уменьшает дублирование для конкретного endpoint. Для side effects вроде email, payment или очереди нужны отдельные patterns: outbox, unique business key, provider idempotency key, reconciliation job. Не обещайте exactly-once в README, если не можете доказать его для всей цепочки.

Rate limiting: защита ограниченного ресурса

Rate limiting отвечает: сколько работы конкретный actor может создать за период. До реализации определите:

- ключ: IP, authenticated user, API key, tenant, route или комбинация;
- scope: весь API, отдельный дорогой endpoint, login/create/export;
- policy: fixed window, sliding window, token bucket, concurrency limit;
- response: `429 Too Many Requests`, возможно `Retry-After`;
- storage: in-memory или shared distributed store;
- observability и abuse response.

Token bucket в одном экземпляре

Для одного процесса token bucket допускает короткий burst и ограничивает среднюю скорость. Но in-memory limiter не глобален: при трёх replicas клиент фактически получает примерно тройной лимит. Это допустимо только при явно documented best-effort policy.

Gin middleware должен не доверять безусловно `X-Forwarded-For`: этот заголовок можно подделать, если reverse proxy не перезаписывает/не ограничивает его. Настройте trusted proxies и используйте authenticated identity, если лимит относится к пользователю.

```
func rateLimit(limiter *rate.Limiter) gin.HandlerFunc {
  return func(c *gin.Context) {
    if !limiter.Allow() {
      c.Header("Retry-After", "1")
      c.AbortWithStatusJSON(http.StatusTooManyRequests, errorResponse{
        Code:    "rate_limited",
        Message: "Слишком много запросов. Повторите позже.",
      })
      return
    }
    c.Next()
  }
}
```

Этот пример — глобальный limiter одного instance, не per-user policy. Для production многорепличного API рассмотрите rate limiting на gateway/reverse proxy или shared Redis/database store, оцените availability этого dependency и не создавайте безграничную map лимитеров по user-controlled IP.

Concurrency limit дополняет rate limit

Rate ограничивает поток request за секунду. Concurrency limit ограничивает число одновременно дорогих операций: export, report, external API. Для медленного endpoint одного rate limit может быть недостаточно — 10 разрешённых долгих requests могут занять все DB connections. Выбирайте bounded queue, immediate `429/503` или async job model по product contract.

Деградация и dependency failure

Когда optional priority service недоступен, API может:

- отказать в создании задачи (`503`), если priority обязательна;
- создать задачу с default priority и записать событие для повторного enrichment;
- вернуть cached suggestion с явной freshness policy;
- принять async job и дать клиенту статус обработки.

Это product decision. Не прячьте его в catch-all `recover` или silent fallback. Для каждого пути определите HTTP contract, audit/log/metric, retry policy и data consistency.

Тесты надёжности

Проверяйте contract, а не случайные sleeps:

- client прекращает retry при отменённом context;
- retry не запускается на non-retryable `400`;
- retry ограничен attempts и укладывается в deadline;
- одинаковый idempotency key возвращает сохранённый ответ;
- тот же key с другим request возвращает documented conflict;
- rate limiter возвращает `429` и не вызывает downstream service;
- метрики имеют bounded labels и фиксируют retry/limit outcome.

Следующая страница добавит security boundary и code review, который помогает эти решения не потерять в следующем изменении.

Безопасность API и code review

Безопасность production API — не один middleware с JWT. Это последовательность границ: кто вызывает API, что ему разрешено, какой input принимается, сколько ресурсов он может занять, что увидит в error response и как изменения проверяются до deploy.

Authentication и authorization — разные проверки

Authentication отвечает: «кто делает запрос?». Примеры: session, OAuth/OIDC access token, API key, mTLS identity.

Authorization отвечает: «разрешено ли этой identity выполнить действие над этим ресурсом?». У authenticated пользователя всё равно может не быть права читать/завершать чужую задачу.

Плохой дизайн:

```
JWT валиден → GET /tasks/42 разрешён
```

Нужен policy check с данными ресурса и actor:

```
func (service *Service) Complete(ctx context.Context, actor Actor, id task.TaskID)
(task.Task, error) {
  current, err := service.repository.Get(ctx, id)
  if err != nil {
    return task.Task{}, err
  }
  if current.OwnerID != actor.ID && !actor.HasRole("admin") {
    return task.Task{}, ErrForbidden
  }
```

```
  return service.repository.Complete(ctx, id)}
```

Это пример направления, не готовая policy. Важно, что authorization видна в use case и тестируется для owner, другого user, admin и отсутствующего actor. Не полагайтесь только на hidden frontend button или ID из path клиента.

Token validation на границе

Middleware authentication должен:

1. извлечь credential из ожидаемого места;
2. проверить signature/issuer/audience/expiry и algorithm согласно provider contract;
3. ограничить размер заголовка и не логировать credential;
4. создать минимальный immutable actor context;
5. передать actor в нижний слой безопасным способом;
6. вернуть нейтральный `401` для неаутентифицированного запроса.

Не принимайте algorithm из JWT header как доверенный выбор и не отключайте проверку audience/issuer ради «совместимости». Для OAuth/OIDC используйте поддерживаемую библиотеку и проверяйте ключи/rotation по документации провайдера.

Input validation и resource limits

JSON binding проверяет только часть формы. Production handler должен ограничивать ресурсы до decoding:

```
const maxRequestBodyBytes = 1 << 20 // 1 MiB; выберите по contract endpoint.

func limitBody(c *gin.Context) {
  c.Request.Body = http.MaxBytesReader(c.Writer, c.Request.Body, maxRequestBodyBytes)
}
```

Затем decoder/binding возвращает `413` или безопасный `400` согласно вашему contract. Также ограничивайте:

- длину title, число элементов массива, глубину/сложность filter;
- `limit` пагинации и допустимые sort fields whitelist;
- время и число результатов экспортов;
- upload size/type, если uploads появятся;
- входные значения rate-limit/idempotency key.

Не собирайте SQL из client strings. Values передаются placeholders, а identifiers/sort directions выбираются из закрытого whitelist. Не отражайте input в HTML или log без escaping/redaction.

HTTP hardening

Базовые server timeouts уже защищают от медленных клиентов. Дополнительно рассмотрите по конкретному deployment:

- TLS завершает reverse proxy или сам Go server; plaintext HTTP не должен быть публичным;
- `http.Server` имеет `ReadHeaderTimeout`, `ReadTimeout`, `WriteTimeout`, `IdleTimeout`;
- reverse proxy ограничивает request body, header size, connection rate и trusted forwarded headers;
- API выставляет только необходимые CORS origins/methods/headers, а не `*` вместе с credentials;
- security headers полезны в зависимости от типа response; JSON API и web UI имеют разные needs;
- debug endpoints (`pprof`, metrics admin) доступны только в internal network/authenticated admin boundary.

Не копируйте browser security headers в JSON API без понимания. Например, CSP важна для HTML, но не решает authorization API. Security controls должны быть проверяемы на уровне реального ingress, а не только в Gin unit-test.

Ошибки и information disclosure

Публичный error response должен быть стабильным и безопасным:

```
{"code":"internal_error","message":"Внутренняя ошибка сервера"}
```

Лог получает внутренний `err`, request/trace ID и безопасный context; клиент — code/message без SQL, file paths, panic, DSN, stack trace или деталей авторизации. Для `401`/`403` выбирайте policy, которая не помогает enumeration ресурсов или пользователей.

Не путайте «не раскрыть лишнее» с «скрыть incident»: internal logs, metrics, traces и audit events должны позволять расследование с нужными access controls.

Dependency и supply-chain security

Production code включает dependencies, Docker base image, GitHub Actions, OS packages и workflow permissions. Минимальный процесс:

- обновлять Go и dependencies осознанно, читать release/security notes;
- использовать lock/`go.sum` и проверять `go mod tidy` diff;
- закреплять GitHub Actions по trusted version/commit согласно security policy;
- запускать dependency updates через CI и integration tests;
- минимизировать Docker image, не запускать процесс root без необходимости;
- не класть secrets в image layers, source, build args и CI output;
- знать owner и response plan для critical CVE.

Автоматический dependency bot создаёт повод для review, но не является заменой review. Особенно внимательно проверяйте auth, crypto, SQL drivers, serialization и network packages.

Code review: проверяйте risk, а не стиль только глазами

Pull request должен отвечать: что изменилось, какой contract меняется, какие риски и как это проверено. Для больших изменений полезен короткий design note до реализации.

Checklist автора и reviewer

Архитектура

- [ ] Новая зависимость направлена внутрь только через нужную boundary; service не импортирует Gin/SQL/client implementation.
- [ ] Интерфейс появился по реальной потребности consumer, а не как абстракция «на будущее».
- [ ] Конкретный use case не спрятан за generic CRUD, если важны правила, transaction или authorization.
- [ ] Изменения config, migrations и deploy отражены в README/Compose/CI.

Корректность и данные

- [ ] Контракт request/response, status и error codes задокументирован и закреплён тестом.
- [ ] SQL параметризован; migration проверена против тестовой БД и совместима с rollback plan.
- [ ] Context передан в SQL/HTTP/worker и deadline не превышает request budget.
- [ ] Retry, idempotency и failure semantics определены для external side effect.

Security

- [ ] Authentication и authorization проверяются отдельно; есть тесты доступа к чужому ресурсу.
- [ ] Input/body/pagination limits и rate limiting соответствуют endpoint risk.
- [ ] Secrets, PII, token, DSN и stack trace не попадают в response/log/metric/trace.
- [ ] Debug/admin endpoints не опубликованы через public ingress.
- [ ] Dependencies, Docker and workflow permissions проверены для нового risk.

Operations

- [ ] Логи содержат нужный bounded context, metrics — bounded labels, traces — безопасные attributes.
- [ ] Liveness/readiness отражают documented platform semantics и shutdown draining.
- [ ] Alert/dashboard/operational runbook обновлены, если меняется failure mode.
- [ ] Unit, integration, race, API contract, Compose and deploy checks соответствуют изменению.

После merge: verify in production

Зелёный CI подтверждает известные scenarios, но не заменяет rollout observation. После deploy проверьте:

1. image/version и config revision;
2. readiness/liveness и отсутствие restart loop;

3. ключевой HTTP scenario с безопасным test actor;
4. error rate, latency, saturation и dependency metrics;
5. logs/traces без новых sensitive fields;
6. rollback condition и known-good version.

Новые high-risk изменения включайте постепенно: feature flag, canary, ограниченный tenant или staged rollout. Если baseline и rollback неизвестны, deployment ещё не готов.

Итог

Production architecture не означает максимум паттернов, метрик и middleware. Это набор ясных контрактов: зависимости направлены правильно; конфигурация и secrets управляемы; операции наблюдаемы; повторные запросы и failures имеют policy; public API защищён; a review проверяет изменения по их риску.

Самопроверка: архитектура production-сервиса

1. Какое направление зависимостей предпочтительно для Task Manager API?

A. Repository выбирает HTTP-статус и JSON response

B. Domain package импортирует Gin и database/sql

C. Gin handler → service → интерфейс repository; PostgreSQL реализует интерфейс снаружи

D. Каждый handler открывает своё подключение к PostgreSQL

2. Где обычно объявляют небольшой интерфейс зависимости?

A. В общем пакете common до появления использования

B. В main вместе со всеми env-переменными

C. Только в пакете конкретной PostgreSQL-реализации

D. Рядом с потребителем, которому нужна конкретная возможность

3. Что является главной ролью composition root в main?

A. Загрузить config, создать конкретные реализации, связать зависимости и управлять lifecycle

B. Хранить SQL всех repository

C. Заменять unit-тесты интеграцией

D. Проверять JSON body каждого запроса

4. Почему нельзя логировать Config целиком?

A. slog не умеет логировать структуры

B. Это нарушает правила компилятора Go

C. В нём могут быть DSN, пароли и tokens; логируют только безопасные поля

D. Config всегда содержит только числа

5. Что должна отвечать liveness probe?

A. Сколько задач в PostgreSQL

B. Доступен ли каждый downstream API в интернете

C. Жив ли процесс; проверка должна быть лёгкой и не зависеть от внешних сервисов

D. Какая версия JWT у текущего клиента

6. Как readiness должна вести себя в начале graceful shutdown?

A. Сразу закрыть базу данных

B. Перестать принимать новый трафик, например вернуть 503 со статусом draining

C. Отправить клиентам stack trace

D. Продолжать отвечать ready до os.Exit

7. Почему task_id нельзя добавлять как label HTTP-метрики?

A. TaskID нельзя использовать в Go

B. Метрики всегда должны содержать полный request body

C. Числовые labels запрещены Prometheus

D. Он создаёт неограниченную cardinality time series

8. Что нужно передавать в service и downstream client для сохранения отмены и trace context?

A. c.Request.Context()

B. Глобальный mutable context

C. Новый context.Background() в каждом методе

D. Только X-Request-ID как string

9. Когда retry обычно безопаснее применять?

A. Бесконечно, пока dependency не ответит

B. Для ограниченной idempotent операции и только при временной ошибке в пределах deadline

C. Для любого POST без idempotency policy

D. После любого 400 validation error

10. Как idempotency key помогает POST создания?

A. Автоматически авторизует пользователя

B. Заменяет transaction в PostgreSQL

C. Шифрует database password

D. Повтор того же логического запроса возвращает первоначальный результат вместо создания дубликата

11. Почему in-memory rate limiter не даёт строгий общий лимит при нескольких replicas?

A. Redis не поддерживает rate limiting

B. Rate limit всегда должен быть равен нулю

C. Gin запрещает limiter при Docker Compose

D. У каждого процесса собственное состояние и клиент может распределить запросы между ними

12. Чем authentication отличается от authorization?

A. Это два названия JWT signature verification

B. Authentication устанавливает identity, authorization проверяет право на конкретное действие/ресурс

C. Authorization выполняется только на frontend

D. Authentication заменяет проверку owner задачи

Ответы к самопроверке

1. Gin handler → service → интерфейс repository; PostgreSQL реализует интерфейс снаружи

2. Рядом с потребителем, которому нужна конкретная возможность

3. Загрузить config, создать конкретные реализации, связать зависимости и управлять lifecycle

4. В нём могут быть DSN, пароли и tokens; логируют только безопасные поля

5. Жив ли процесс; проверка должна быть лёгкой и не зависеть от внешних сервисов

6. Перестать принимать новый трафик, например вернуть 503 со статусом draining

7. Он создаёт неограниченную cardinality time series

8. c.Request.Context()

9. Для ограниченной idempotent операции и только при временной ошибке в пределах deadline

10. Повтор того же логического запроса возвращает первоначальный результат вместо создания дубликата

11. У каждого процесса собственное состояние и клиент может распределить запросы между ними

12. Authentication устанавливает identity, authorization проверяет право на конкретное действие/ресурс

Задания: архитектура production-сервиса

Эти задания развивают Task Manager API по одному production-требованию. Не пытайтесь реализовать все middleware, metrics и policies одновременно. Для каждого изменения сначала опишите contract, затем добавьте код, тесты и operational documentation.

Задание 1. Выделите service boundary и fake repository

Опишите use case `Complete(ctx, actor, taskID)` в пакете `taskservice`. Service должен зависеть от маленького repository interface, а не от `*sql.DB` или Gin. Введите `Actor` и проверку: owner может завершить свою задачу, admin — любую, другой user получает `ErrForbidden`.

Критерии проверки:

- package `taskservice` не импортирует Gin, `database/sql` или PostgreSQL driver;
- interface объявлен рядом с service и содержит только нужные методы;
- `context.Context` передаётся в repository;
- unit-тест использует fake repository, не настоящую БД;
- тесты покрывают owner, admin, другой user и отсутствующую задачу;
- HTTP handler переводит `ErrForbidden` в документированный status без раскрытия repository details.

Подсказка

```
type Repository interface {
  Get(ctx context.Context, id task.TaskID) (task.Task, error)
  Complete(ctx context.Context, id task.TaskID) (task.Task, error)
}
```

Направление решения

Не передавайте `*gin.Context` в `Complete`. Authentication middleware может создать минимальный `Actor`, handler передаст его в use case, a service выполнит policy check на domain data. PostgreSQL repository остаётся отдельной реализацией interface и проверяется integration-тестами.

Задание 2. Config и безопасный startup log

Добавьте `DATABASE_URL`, `HTTP_CLIENT_TIMEOUT`, `SHUTDOWN_TIMEOUT` и `LOG_LEVEL` в `Config`. Все значения должны быть разобраны и проверены при старте. Напишите функцию безопасного логирования startup configuration, которая не выводит DSN, password и token.

Критерии проверки:

- обязательный `DATABASE_URL` без default вызывает startup error;
- durations парсятся и должны быть положительными;
- URL внешнего сервиса проверяется как absolute URL;
- `.env.example` и README содержат новые переменные без production secrets;
- unit-тесты используют `t.Setenv` без `t.Parallel`;
- log содержит HTTP address/log level, но не `DATABASE_URL` и credentials.

Подсказка

Создайте отдельную функцию, например `LogStartup(logger, config)`, и тестируйте output через `slog` handler в `bytes.Buffer` либо проверяйте список разрешённых fields в собственном adapter.

Задание 3. Разделите `/livez` и `/readyz`

Добавьте два endpoint:

- `GET /livez` всегда быстро возвращает `200` для живого процесса;
- `GET /readyz` возвращает `200`, только когда service не находится в draining state и `PingContext` критической БД проходит с коротким timeout; иначе — безопасный `503`.

Добавьте `draining` state, который выставляется до `server.Shutdown`.

Критерии проверки:

- liveness не делает SQL/HTTP call;
- readiness получает dependency через маленький `ReadinessChecker` interface;
- timeout readiness дочерний от request context и всегда отменяется;
- `503` не раскрывает DSN или текст SQL error;
- handler-тесты покрывают ready, DB failure и draining;
- README/Compose/оркестратор documentation описывают назначение обоих endpoints.

Подсказка

```
if api.draining.Load() {
  c.JSON(http.StatusServiceUnavailable, gin.H{"status": "draining"})
  return
}
```

Задание 4. Контракт retry и idempotency

Спроектируйте два независимых контракта:

1. priority HTTP client делает максимум две попытки только для retryable `GET`, учитывает context и фиксирует metric/log outcome;
2. `POST /tasks` принимает ограниченный `Idempotency-Key`, сохраняет key + request hash + response атомарно с созданием задачи и возвращает тот же ответ на повтор.

В первой итерации достаточно design note, SQL migration и тестов contract — не требуется писать полноценную distributed очередь.

Критерии проверки:

- retry не срабатывает на `400` и не продолжается после cancel/deadline;
- delay ограничен, имеет backoff/jitter design и входит в request budget;
- idempotency key scoped к authenticated actor, имеет проверку длины/формата;
- same key + same request возвращает первоначальный status/body;
- same key + different request возвращает documented conflict;
- key persistence и создание Task находятся в одной transaction/unique constraint protocol;
- retention/cleanup и PII policy для key records описаны.

Подсказка

В таблице idempotency key используйте `(actor_id, key)` как unique identity и hash нормализованного request, а не raw credentials или неограниченный body.

Задание 5. Rate limit и API security review

Добавьте middleware или gateway policy для дорогого `POST /tasks` / export endpoint. Опишите identity key, limit/burst, поведение `429`, `Retry-After` и multi-replica semantics. Проведите security review существующего API по checklist этой главы.

Критерии проверки:

- untrusted `X-Forwarded-For` не считается надёжным identity без настройки trusted proxy;
- limiter не создаёт безграничную map состояний по client-controlled ключам;
- при отказе возвращается `429` с безопасным JSON contract и downstream service не вызывается;
- body имеет лимит до decoding; pagination/filter/sort имеют bounds и whitelist;
- authentication и authorization проверяются отдельными тестами;
- debug/pprof/admin endpoints не опубликованы наружу;
- review содержит action items с owner/приоритетом, а не только список общих слов.

Направление решения

In-memory `rate.Limiter` может быть достаточен для single-instance internal service с documented best-effort policy. Для публичного многорепличного API выберите gateway или shared-store design и отдельно оцените availability/cost этой инфраструктуры. Rate limiting не заменяет authorization, WAF, body limits или observability.

Проверьте себя

Перед выпуском production-изменения убедитесь, что вы можете:

- показать направление зависимостей от HTTP/infrastructure к service/domain;
- назвать owner каждого secret и доказать, что он не попадает в response/log/config dump;
- объяснить semantics startup, liveness и readiness во время drain;
- выбрать bounded labels для metrics и передать trace/cancellation context вниз;
- обосновать timeout budget, retry policy и idempotency на create side effect;
- определить scope и storage rate limiter для числа replicas;
- отделить authentication от authorization и проверить доступ к чужому ресурсу;
- показать reviewer тесты, migration/deploy plan, dashboard/alert/runbook для изменённого failure mode.

Рекомендации по коду: архитектура production-сервиса

Слои и зависимости

- Держите направление зависимостей от transport/infrastructure к service/domain; `main` остаётся явным composition root.
- Пусть Gin живёт в HTTP-слое, `database/sql` — в repository, а конкретный HTTP client — в integration package.
- Объявляйте маленькие interfaces рядом с consumer; не создавайте abstractions до реальной потребности и не mock-айте чистый конкретный код.
- Отделяйте request/response DTO от внутренних моделей, если их contract/lifecycle различаются.
- Передавайте `context.Context` вниз во все SQL, HTTP и cancellable worker operations; не создавайте `Background` внутри use case.

Config, secrets и lifecycle

- Читайте и валидируйте configuration один раз на startup; обязательный secret/config не должен иметь небезопасный default.
- Логируйте только явно разрешённые config fields. DSN, password, token, key, cookie и raw body не являются безопасным startup output.

- Обновляйте parser, `.env.example`, README, CI, deploy config и rotation/rollback plan одним изменением.
- Разделяйте startup, liveness и readiness. Liveness лёгкий; readiness отражает возможность принимать новый трафик и draining.
- При shutdown сначала снимайте readiness/останавливайте приём, затем ждите handler/worker в deadline и только после этого закрывайте dependencies.

Observability

- Логируйте структурированный безопасный context на окончательной error boundary: request/trace ID, route, operation, bounded error code и внутренний error.
- Используйте metrics для rate, errors, duration, saturation и dependency outcome; labels должны быть конечным низкокардинальным набором.
- Не используйте request ID, task ID, email, raw path, error text или token как metric label.
- Передавайте incoming context для cancellation и trace propagation; trace attributes подчиняются той же privacy policy, что logs.
- Связывайте alerts с user impact, dashboard и конкретным действием, а не с единичным шумным событием.

Reliability

- Планируйте timeout как вложенный budget всей операции, а не как независимые большие числа в каждом layer.
- Retry только idempotent/защищённые операции, только transient errors, с limit, backoff, jitter, deadline awareness и observability.
- Для create/side effect определяйте idempotency contract до автоматических retries; делайте key persistence атомарной с effect.
- Различайте in-memory best-effort limiter и distributed shared policy; лимитируйте rate и concurrency по отдельности, если ресурс дорогой.
- Явно выбирайте degradation policy optional dependency: fail, default, cache, async job — с contract и observability.

Security и review

- Проверяйте authentication и authorization отдельно; ownership/role policy должна быть видна в use case и тестах.
- Ограничивайте body, pagination, filter/sort, upload и expensive operations до того, как они заняли ресурсы.
- Параметризуйте SQL; выбирайте dynamic identifiers из whitelist; не возвращайте internal errors, stack trace или SQL клиенту.
- Ограничивайте CORS, trusted proxies, TLS ingress и доступ к debug/admin endpoints по реальному deployment, а не только Gin router.
- Проверяйте dependencies, Docker image, workflow permissions, secrets и deployment artifacts как часть security surface.
- В code review требуйте contract tests, migration/rollback, configuration changes, operational signals и rollout plan соразмерно риску.

Итоговый чек-лист

- [ ] У каждого нового use case есть явный owner, dependency boundary, errors и context propagation.
- [ ] Config и secrets валидируются, не логируются и воспроизводимы во всех environments.
- [ ] Liveness/readiness/draining соответствуют поведению deployment platform.
- [ ] Logs, metrics и traces дают диагностику без sensitive data и high cardinality.
- [ ] Retry, idempotency, rate/concurrency limits определены для side effects и dependency failure.
- [ ] Authorization, input limits, error disclosure и debug endpoints проверены на уровне endpoint/ingress.
- [ ] Tests, migrations, CI, Docker, observability, rollout и rollback обновлены вместе с production-риском.

СПРАВОЧНИК

Базовые понятия

Данный раздел — это всесторонний справочник (глоссарий) по базовым понятиям и концепциям Go. Он организован по категориям для удобства, с алфавитным указателем в конце. Каждое понятие объяснено просто, с примерами кода (где применимо), аналогиями и ссылками на связанные концепции.

> **Цель**
>
> Чтобы при встрече нового термина в коде или документации ты мог быстро вернуться сюда и понять суть.

В основе спецификация Go (The Go Programming Language Specification) и актуальные версии (1.23+ на 2025 год).

Справочник не охватывает advanced темы (например, unsafe или generics в деталях), но фокусируется на базовых.

Для практики: Скопируй примеры в `main.go` и запусти `go run main.go`.

1. Основы языка и структура программы

Package (Пакет)

> Группа файлов с кодом, объединённая по функциональности.

Каждый Go-файл начинается с `package <name>`. `package main` — для исполняемых программ (с функцией `main()`). Другие пакеты — для библиотек (импортируются).

Аналогия: Пакет — как модуль в Python или namespace в C++.

Пример:

```
package main  // Это основной пакет
```

Связанные: Import, Module.

Import (Импорт)

> Подключение внешних пакетов (библиотек) для использования их функций/типов.

Используй `import "package/name"`. Стандартные пакеты (fmt, os) в GOROOT; пользовательские — в GOPATH или modules.

Аналогия: Как `import math` в Python.

Пример:

```
import "fmt"  // Для Println
import (
    "os"
    "time"
)  // Многострочный импорт
```

Связанные: Package, Module.

Подводный камень: Unused imports — ошибка компиляции.

Main function (Функция main)

> Точка входа в программу.

Должна быть в `package main`. Не принимает аргументов, ничего не возвращает.

Аналогия: Как `main()` в C или точка старта приложения.

Пример:

```
func main() {
    // Код начинается здесь
}
```

Связанные: Function, os.Args (для аргументов командной строки).

Module (Модуль)

> Система управления зависимостями (с Go 1.11+).

Файл `go.mod` определяет версию Go и зависимости. Инициализируй `go mod init <name>`.

Аналогия: Как package.json в Node.js.

Пример

В терминале:

```
go mod init example.com/myapp
go get github.com/some/lib # добавляем зависимость
```

Связанные: Import, go.sum (checksums для безопасности).

Compilation (Компиляция)

> Процесс преобразования кода в бинарный исполняемый файл.

Команды: `go build` (создаёт бинарник), `go run` (компилирует и запускает временно). Go — статически типизированный, так что ошибки типов ловятся на этапе компиляции.

Аналогия: Как javac в Java, но быстрее и с статической линковкой (все зависимости в одном файле).

Связанные: Build tags (для conditional compilation, как //go:build linux).

2. Переменные и типы данных

Variable (Переменная)

> Именованное место в памяти для хранения данных.

Объявление: `var name type = value` (явное) или `name := value` (короткое, авто-тип). Переменные должны использоваться (unused — ошибка).

Аналогия: Коробка с ярлыком, где лежит значение.

Пример:

```
var age int = 25  // Явное
name := "Alice"   // Короткое (string)
```

Связанные: Type, Constant.

Type (Тип)

> Определяет, какие данные может хранить переменная и операции над ними.

Базовые: int (целые, 32/64-bit), uint (беззнаковые), float32/float64 (дробные), bool (true/false), string (строки, immutable), byte (uint8, для байтов), rune (int32, для Unicode-символов).

Аналогия: Форма коробки — круглая для шаров, квадратная для кубов.

Пример: `var pi float64 = 3.14`.

Связанные: Composite types (array, slice, map, struct), Type conversion (e.g., int(float64Var)).

Constant (Константа)

> Неизменяемое значение, объявляется `const name = value`.

Могут быть типизированными или untyped (авто).

Аналогия: Зафиксированное правило, как Pi в математике.

Пример:

```
const Pi = 3.14
const DaysInWeek int = 7
```

Связанные: Iota (для последовательных констант, как enum: const (Mon iota; Tue)).

Zero value (Нулевое значение)

> Значение по умолчанию для неинициализированных переменных.

Для int — 0, string — "", bool — false, pointer/slice/map/chan — nil.

Аналогия: Пустая коробка, когда ничего не положили.

Пример: `var s string // s == ""`.

Связанные: Nil (специальное значение для "ничего").

3. Коллекции и составные типы

Array (Массив)

> Фиксированная последовательность элементов одного типа.

Размер — часть типа: [3]int. Передаётся по значению (копируется).

Аналогия: Фиксированная полка с ячейками.

Пример:

```
var arr [2]string = [2]string{"Go", "Lang"}
fmt.Println(arr[0])  // Go
```

Связанные: Slice (динамическая версия).

Slice (Слайс)

> Динамическая "ссылка" на часть массива.

Структура: указатель на данные + len (длина) + cap (ёмкость). Передаётся по ссылке.

Аналогия: Вид на полку, который можно расширять.

Пример:

```
s := []int{1, 2, 3}
s = append(s, 4)  // [1 2 3 4]
fmt.Println(len(s), cap(s))  // 4 4 (или больше)
```

Связанные: Make (make([]int, len, cap)), Range.

Map (Мапа)

> Ассоциативный массив (ключ-значение), как hash-таблица.

Ключи уникальны, порядок не гарантирован.

Аналогия: Словарь, где по слову находишь определение.

Пример:

```
m := map[string]int{"age": 25}
m["height"] = 180
delete(m, "age")
val, ok := m["height"]  // 180, true
```

Связанные: Make (make(map[K]V)).

Struct (Структура)

> Составной тип для группировки данных (как record или class без методов).

Поля: имя + тип.

Аналогия: Форма с полями, как анкета.

Пример:

```
type Person struct {
    Name string
    Age  int
```

```
}p := Person{Name: "Bob", Age: 30}fmt.Println(p.Name)  // Bob
```

Связанные: Embedding (встраивание структур), Method.

Pointer (Указатель)

> Переменная, хранящая адрес другой переменной.

Синтаксис: *T (указатель на T), &var (адрес).

Аналогия: Указатель на дом, а не сам дом.

Пример:

```
var ptr *int
i := 42
ptr = &i
*ptr = 43  // i теперь 43
```

Связанные: Nil, Dereferencing (*ptr). (Подробно отложено, но базово: для изменений по ссылке).

4. Функции и методы

Function (Функция)

> Блок кода для повторного использования.

`func name(params) returns { ... }`. Может возвращать несколько значений.

Аналогия: Рецепт: ингредиенты (params), результат (returns).

Пример:

```
func add(a, b int) (int, error) {
    return a + b, nil
}
```

Связанные: Defer, Variadic (...args).

Method (Метод)

> Функция, прикреплённая к типу (обычно struct).

Receiver: (r ReceiverType) или (r *ReceiverType).

Аналогия: Действие объекта, как "car.move()".

Пример:

```
func (p *Person) Birthday() {
    p.Age++
}
```

Связанные: Pointer receiver (для изменений), Value receiver (для копий).

Defer

> Откладывает вызов функции до конца enclosing функции (после return).

Полезно для cleanup (close files).

Аналогия: "Сделай это перед уходом".

Пример:

```
defer fmt.Println("Done")  // Выполнится последним
```

Связанные: Panic/recover (defer ловит panic).

Variadic function (Вариадическая функция)

> Принимает переменное число аргументов (...T). Внутри — как slice.

Аналогия: Функция для "любого количества гостей".

Пример:

```
func sum(nums ...int) int {
    total := 0
    for _, n := range nums {
        total += n
    }
    return total
}
sum(1, 2, 3)  // 6
```

5. Контроль потока

If/else

> Условное выполнение.

Нет скобок для условия. Может инициализировать переменные.

Пример:

```
if x > 0 { /*...*/ } else { /*...*/ }
```

Связанные: Short if (if err := func(); err != nil { ... }).

For

> Единственный цикл.

Варианты: for init; cond; post {}, for cond {}, for range, infinite for {}. Нет while.

Пример:

```
for i := 0; i < 5; i++ { ... }
for _, v := range slice { ... }
```

Связанные: Break, Continue, Range.

Switch

> Многоуровневое условие.

Case на выражениях, fallthrough для продолжения.

Пример:

```
switch os := runtime.GOOS; os {
case "darwin":
    fmt.Println("macOS")
default:
    fmt.Println("Other")
}
```

Связанные: Type switch (switch v := i.(type) { ... }).

Range

> Итератор для коллекций (slice, map, chan, string).

Возвращает index/key и value.

Пример:

```
for i, v := range arr { /*...*/ } // (i — индекс, v — значение)
```

Связанные: For, Slice.

6. Интерфейсы и типы

Interface (Интерфейс)

> Контракт — набор методов.

Тип реализует интерфейс implicitly (duck typing).

Аналогия: Розетка: любой прибор с подходящей вилкой работает.

Пример:

```
type Shaper interface {
    Area() float64
}
```

Связанные: Empty interface (interface{} — any type), Type assertion (i.(Type)).

Embedding (Встраивание)

> В struct/interface — поля/методы "промотируются" от встроенного типа.

Аналогия: Наследование без классов.

Пример:

```
// Employee имеет поля/методы Person
type Employee struct {
    Person
}
```

Связанные: Struct, Interface.

7. Обработка ошибок

Error (Ошибка)

> Встроенный интерфейс `type error interface { Error() string }`.

Функции возвращают error как второе значение.

Аналогия: Флаг "что-то пошло не так".

Пример:

```
_, err := someFunc()
if err != nil {
    fmt.Println(err.Error())
}
```

Связанные: errors.New, fmt.Errorf, Panic.

Panic/Recover

> Runtime-ошибка (как exception).

Defer с recover() ловит panic.

Аналогия: Аварийная остановка.

Пример:

```
defer func() {
    if r := recover(); r != nil {
        fmt.Println("Recovered:", r)
    }
}()
panic("Error!")
```

Связанные: Defer.

8. Concurrency (Параллелизм)

Goroutine

> Лёгкий "поток" — `go func() { ... }()`.

Тысячи могут goroutine работать параллельно.

Аналогия: Фоновая задача, как асинхронный вызов.

Пример:

```
go worker()  // Запуск в фоне
```

Связанные: Channel, sync.WaitGroup.

Channel (Канал)

> Для общения между goroutines. `ch := make(chan int)`.

Буферизованные/небуферизованные.

Аналогия: Труба для передачи данных.

Пример:

```
ch <- 42  // Отправка
val := <-ch  // Получение
```

Связанные: Select, Close.

Select

> Мультиплексор для channels. Ждёт события от нескольких chan.

Пример:

```
select {
    case v := <-ch1: ...
    case <-ch2: ...
    default: ...
}
```

Связанные: Channel.

Mutex (Мьютекс)

> Для синхронизации доступа (из sync). Lock/Unlock.

Аналогия: Замок на двери — только один в комнате.

Пример:

```
var mu sync.Mutex
mu.Lock()
// Критическая секция
mu.Unlock()
```

Связанные: sync.RWMutex (для чтения/записи), Atomic (для простых операций).

9. Другие концепции

Nil

> Специальное значение "ничего" для pointer, slice, map, chan, func, interface.

Аналогия: Пустая ссылка.

Пример:

```
var p *int  // p == nil
```

Связанные: Pointer, Zero value.

Make

> Создаёт и инициализирует slice/map/chan.

Пример:

```
s := make([]int, 5, 10)  // len=5, cap=10
```

Связанные: New (new(T) — аллоцирует и возвращает *T с zero value).

Type assertion (Утверждение типа)

> Извлечение underlying типа из interface. `v, ok := i.(Type)`

Пример: Для any (interface{}).

Связанные: Interface, Type switch.

Build (Сборка)

> Процесс создания бинарника. `go build`, `go install`.

Связанные: Cross-compilation (GOOS=windows go build).

Алфавитный указатель

Array, Build, Channel, Constant, Defer, Embedding, Error, For, Function, Goroutine, If/else, Import, Interface, Main function, Make, Map, Method, Module, Mutex, Nil, Package, Panic/Recover, Pointer, Range, Select, Slice, Struct, Switch, Type, Type assertion, Variable, Variadic function, Zero value.

Продвинутые понятия

Эта "страница" — отдельный справочник по advanced (продвинутым) понятиям и концепциям Go. Он предназначен для тех, кто уже освоил базовые (см. предыдущую страницу справочника). Здесь мы фокусируемся на более сложных темах, которые используются в реальных проектах: от generics и concurrency patterns до low-level инструментов вроде unsafe. Каждое понятие объяснено с примерами, аналогиями, подводными камнями и ссылками на связанные базовые концепции.

В основе спецификация Go (1.23+ на 2025 год) и best practices из "Effective Go".

Справочник организован по категориям.

Для практики: Запускай примеры в `main.go`.

> **Предупреждение**
>
> Advanced темы требуют осторожности — они могут привести к ошибкам (panics, race conditions), так что тестируй код.

1. Типы и generics

Generics (Дженерики)

> Параметризованные типы/функции (с Go 1.18+).

Позволяют писать обобщённый код для разных типов с type parameters. Синтаксис: funcT any или type Set[T comparable] struct {}.

Аналогия: Шаблон, который адаптируется под разные "формы" данных.

Пример:

```
package main

import "fmt"

// Обобщённая функция
func Print[T any](v T) {
    fmt.Println(v)
}

// Обобщённый тип
type Stack[T any] struct {
    items []T
}

func (s *Stack[T]) Push(item T) {
    s.items = append(s.items, item)
}

func main() {
    Print[int](42)     // 42
    Print[string]("Go") // Go

    var intStack Stack[int]
    intStack.Push(1)
    fmt.Println(intStack.items)  // [1]
}
```

Подводные камни: Constraints (any — любой тип, comparable — для ==/!=, ~int — underlying int). Нет specialization (как в C++).

Связанные: Type (базовый), Interface (constraints как interfaces). **Дальше**: Type inference (Go угадывает T).

Type alias (Псевдоним типа)

> Объявление нового имени для существующего типа: type MyInt = int.

Не создаёт новый тип, только синоним.

Аналогия: Никнейм для друга — то же лицо, но другое имя.

Пример:

```
type ID = string
var userID ID = "abc123"
```

Подводный камень: Не совместим с generics напрямую.

Связанные: Type (базовый).

Underlying type (Underlying тип)

> "Базовый" тип под alias или defined type.

Для type MyInt int — underlying int. Полезно в generics (~T для underlying T).

Аналогия: Основа под маской.

Пример: В generics: func[T ~int](v T) {}.

Связанные: Generics.

2. Concurrency patterns (Паттерны параллелизма)

Context (Контекст)

> Пакет context для отмены операций, таймаутов и передачи значений в goroutines.

context.Background() — базовый, WithCancel/WithTimeout/WithValue — производные.

Аналогия: "Билет" с таймером отмены для задач.

Пример:

```
package main

import (
    "context"
    "fmt"
    "time"
)

func worker(ctx context.Context) {
    select {
    case <-ctx.Done():
        fmt.Println("Cancelled:", ctx.Err())
    case <-time.After(2 * time.Second):
        fmt.Println("Done")
```

```
    }}func main() {    ctx, cancel := context.WithTimeout(context.Background(),
time.Second)    defer cancel()    go worker(ctx)    time.Sleep(3 * time.Second)  //
Timeout сработает}
```

Подводные камни: Всегда проверяй ctx.Done(). Не передавай большие данные в Value (это для request-scoped).

Связанные: Goroutine (базовый), Channel.

Дальше: Errgroup (для групп goroutines с ошибками).

Sync.WaitGroup

> Структура для ожидания завершения группы goroutines. Add/Done/Wait.

Аналогия: Счётчик задач — жди, пока все не закончат.

Пример:

```
var wg sync.WaitGroup
wg.Add(1)
go func() {
    defer wg.Done()
    // Работа
}()
wg.Wait()  // Ждём
```

Связанные: Goroutine, Mutex.

Sync.Once

> Выполняет функцию ровно один раз, даже в многопотоке.

Аналогия: Инициализация singleton.

Пример:

```
var once sync.Once
once.Do(func() { fmt.Println("Init") })  // Только раз
```

Связанные: Mutex.

Atomic operations (Атомарные операции)

> Пакет sync/atomic для thread-safe операций без мьютекса (Add, Load, Store).

Аналогия: Неуязвимая операция в многопотоке.

Пример:

```
atomic.AddInt64(&counter, 1)
```

Связанные: Mutex, Race condition (избегай с помощью -race флага: go run -race main.go).

Race condition (Состояние гонки)

> Ошибка, когда goroutines конкурируют за доступ к данным без синхронизации.

Аналогия: Два человека пишут в один блокнот одновременно — хаос.

Связанные: Mutex, Atomic.

Инструмент: `go run -race` для детекции.

3. Ошибки и отладка

Custom error (Пользовательская ошибка)

> Struct, реализующая error interface.

Добавь методы как Unwrap для chaining.

Аналогия: Расширенный флаг ошибки с деталями.

Пример:

```
type MyError struct { Msg string }
func (e *MyError) Error() string { return e.Msg }
errors.Is(err, MyError{})  // Проверка
```

Связанные: Error (базовый), errors.Unwrap (для wrapped errors).

Reflection (Рефлексия)

> Пакет reflect для runtime-инспекции типов/значений. Reflect.TypeOf, ValueOf.

Аналогия: Зеркало для кода — смотри на себя во время выполнения.

Пример:

```
v := reflect.ValueOf(42)
fmt.Println(v.Type())  // int
```

Подводные камни: Медленно, небезопасно — используй редко (для JSON, ORM).

Связанные: Interface.

4. Low-level и интеграция

Unsafe (Unsafe пакет)

> Для обхода type safety: unsafe.Pointer (как void* в C), Sizeof, Offsetof, Alignof.

Аналогия: "Опасная зона" — ручное управление памятью.

Пример:

```
import "unsafe"
i := 42
p := unsafe.Pointer(&i)
```

Подводные камни: Может вызвать UB (undefined behavior) — используй только если знаешь.

Связанные: Pointer (базовый), CGO.

CGO

> Интеграция с C-кодом. #include в комментариях, C. types.

Аналогия: Мост между Go и C.

Пример:

```
// #include <stdio.h>
import "C"
C.puts(C.CString("Hello from C"))
```

Подводные камни: Overhead, не портативно. Флаг: go build -buildmode=c-shared.

Связанные: Unsafe.

5. Тестирование и производительность

Testing (Тестирование)

> Пакет testing для unit-тестов. Функции TestXXX(t *testing.T).

Аналогия: Проверка, работает ли код.

Пример: В _test.go:

```
func TestAdd(t *testing.T) {
    if add(1, 2) != 3 {
        t.Error("Failed")
    }
}
```

Связанные: go test, Table-driven tests.

Benchmark (Бенчмарк)

> Измерение производительности: `BenchmarkXXX(b *testing.B)`.

Пример:

```
func BenchmarkAdd(b *testing.B) {
    for i := 0; i < b.N; i++ {
        add(1, 2)
    }
}
```

Связанные: go test -bench.

Profiling (Профилирование)

> Инструменты для анализа (CPU, memory): pprof.

Пример: import _ "net/http/pprof"; go tool pprof.

Связанные: Testing.

6. Модули и build

Build constraints (Build теги)

> `//go:build` для conditional компиляции (по ОС, arch).

Пример: `//go:build linux` — файл только для Linux.

Связанные: Compilation (базовый).

Vendoring

> Копирование зависимостей в vendor/. go mod vendor.

Аналогия: Локальный склад зависимостей.

Связанные: Module.

Дополнительные ресурсы

Эта страница собирает материалы для самостоятельного углубления. Основной маршрут учебника остаётся достаточным для первого прохода: не пытайтесь изучить всё из списка одновременно. Выбирайте ресурс под текущую задачу — уточнить правило языка, потренироваться или разобрать production-паттерн.

Официальные материалы Go

- Документация Go — стартовая точка для руководств, статей и инструментов.
- A Tour of Go — интерактивное введение в синтаксис и стандартные конструкции.
- Effective Go — принципы идиоматичного Go; читайте после освоения базы, не как замену практике.
- Go by Example — короткие исполняемые примеры по отдельным темам.
- Go Specification — точный источник правил языка. Полезна для спорных деталей, но не предназначена как первый учебник.
- Документация стандартной библиотеки — API пакетов `context`, `net/http`, `database/sql`, `testing` и других.
- Go Blog — статьи команды Go о runtime, toolchain и эволюции языка.

Практика и инструменты

- Go Playground — быстро запустить небольшой автономный пример без локальной установки.
- pkg.go.dev — документация модулей и пакетов, включая версии и лицензии зависимостей.
- Go Wiki — подборка материалов сообщества и практик вокруг языка.
- Awesome Go — каталог библиотек и инструментов. Перед добавлением dependency проверяйте поддержку, security policy и необходимость для проекта.

Книги

- Alan A. A. Donovan, Brian W. Kernighan — The Go Programming Language. Подробная книга по языку и его стандартной библиотеке.
- Jon Bodner — *Learning Go*. Современное практическое введение в язык и его идиомы.
- Katherine Cox-Buday — *Concurrency in Go*. Полезна после базовых глав о goroutine, channels и context; не начинайте с неё первый проход.

Материалы по backend и production

- Документация `net/http` — сервер, клиент, timeouts и transport.
- Документация `database/sql` — connection pool, context, transactions и правила закрытия ресурсов.
- Документация PostgreSQL — SQL, индексы, транзакции и настройка базы.

- OWASP API Security Top 10 — угрозы API: авторизация, resource limits, information disclosure и rate limiting.
- OpenTelemetry Go — tracing и metrics после освоения логов и HTTP context propagation.

Как выбирать внешний материал

1. Предпочитайте официальную документацию для точного поведения языка и API.
2. Проверяйте дату публикации и совместимость с используемой версией Go.
3. Не копируйте production-рецепт без понимания его threat model, инфраструктуры и ограничений.
4. Перед добавлением библиотеки изучите документацию, лицензию, активность поддержки и alternative в standard library.
5. Закрепляйте новое знание небольшим примером, тестом или улучшением Task Manager API.